

Microsoft C#™

Curso de Programación

2ª EDICIÓN

- Programación orientada a objetos
 - Elementos del lenguaje
- Sentencias de control
- Clases de uso común
- Matrices, cadenas y colecciones
- Clases, espacios de nombres y estructuras
- Operadores sobrecargados



- Clases derivadas e interfaces
 - Tipos y métodos genéricos
- Excepciones
- Ficheros
- Estructuras dinámicas
- Algoritmos
- Hilos
- Interfaces gráficas y aplicaciones para Internet

WWW



Fco. Javier Ceballos

Puede descargarse el CD-ROM con las URL para obtener el software de desarrollo y las aplicaciones contenidas en el libro



Ra-Ma®

Microsoft C#[™]
Curso de programación
2ª edición

Microsoft C#[™]

Curso de programación

2ª edición

Fco. Javier Ceballos Sierra

Profesor titular de la
Escuela Politécnica Superior
Universidad de Alcalá





Microsoft C#: Curso de programación. 2ª edición.

© Fco. Javier Ceballos Sierra

© De la edición: RA-MA 2011

MARCAS COMERCIALES: Las marcas de los productos citados en el contenido de este libro (sean o no marcas registradas) pertenecen a sus respectivos propietarios. RA-MA no está asociada a ningún producto o fabricante mencionado en la obra, los datos y los ejemplos utilizados son ficticios salvo que se indique lo contrario.

RA-MA es una marca comercial registrada.

Se ha puesto el máximo empeño en ofrecer al lector una información completa y precisa. Sin embargo, RA-MA Editorial no asume ninguna responsabilidad derivada de su uso, ni tampoco por cualquier violación de patentes ni otros derechos de terceras partes que pudieran ocurrir. Esta publicación tiene por objeto proporcionar unos conocimientos precisos y acreditados sobre el tema tratado. Su venta no supone para el editor ninguna forma de asistencia legal, administrativa ni de ningún otro tipo. En caso de precisarse asesoría legal u otra forma de ayuda experta, deben buscarse los servicios de un profesional competente.

Reservados todos los derechos de publicación en cualquier idioma.

Según lo dispuesto en el Código Penal vigente ninguna parte de este libro puede ser reproducida, grabada en sistema de almacenamiento o transmitida en forma alguna ni por cualquier procedimiento, ya sea electrónico, mecánico, reprográfico, magnético o cualquier otro, sin autorización previa y por escrito de RA-MA; su contenido está protegido por la Ley vigente que establece penas de prisión y/o multas a quienes intencionadamente, reprodujeran o plagiaran, en todo o en parte, una obra literaria, artística o científica.

Editado por:

RA-MA Editorial

C/ Jarama, 3A, Polígono industrial Igarsa

28860 PARACUELLOS DEL JARAMA, Madrid

Teléfono: 91 658 42 80

Telefax: 91 662 81 39

Correo electrónico: editorial@ra-ma.com

Internet: ebooks.ra-ma.com, www.ra-ma.es y www.ra-ma.com

ISBN: 978-84-9964-396-0

Depósito Legal: M-9516-2011

Autoedición: Fco. Javier Ceballos

Filmación e impresión: Closas-Orcoyen, S.L.

Impreso en España

Primera impresión: Marzo 2011

*Nunca consideres el estudio como una obligación
sino como una oportunidad para penetrar
en el bello y maravilloso mundo del saber.*

*Dedico esta obra
a María del Carmen, mi esposa,
y a mis hijos Francisco y Javier.*

CONTENIDO

PARTE 1. PROGRAMACIÓN BÁSICA	1
CAPÍTULO 1. FASES EN EL DESARROLLO DE UN PROGRAMA	3
CAPÍTULO 2. INTRODUCCIÓN A C#	19
CAPÍTULO 3. INTRODUCCIÓN A LA POO.....	33
CAPÍTULO 4. ELEMENTOS DEL LENGUAJE	61
CAPÍTULO 5. ESTRUCTURA DE UN PROGRAMA	83
CAPÍTULO 6. CLASES DE USO COMÚN	109
CAPÍTULO 7. SENTENCIAS DE CONTROL.....	141
CAPÍTULO 8. MATRICES	177
CAPÍTULO 9. MÁS SOBRE MÉTODOS Y COLECCIONES	225
PARTE 2. MECANISMOS DE ABSTRACCIÓN.....	263
CAPÍTULO 10. CLASES, ESPACIOS DE NOMBRES Y ESTRUCTURAS.....	265
CAPÍTULO 11. OPERADORES SOBRECARGADOS	349
CAPÍTULO 12. CLASES DERIVADAS E INTERFACES	381
CAPÍTULO 13. TIPOS Y MÉTODOS GENÉRICOS.....	451

CAPÍTULO 14. EXCEPCIONES	465
CAPÍTULO 15. FLUJOS	489
PARTE 3. DISEÑO Y PROGRAMACIÓN	545
CAPÍTULO 16. ESTRUCTURAS DINÁMICAS	547
CAPÍTULO 17. ALGORITMOS	633
CAPÍTULO 18. HILOS	679
CAPÍTULO 19. INTERFACES GRÁFICAS Y APLICACIONES PARA INTERNET	757
PARTE 4. APÉNDICES.....	793
A. ENTORNO DE DESARROLLO	795
B. .NET PARA LINUX	803
C. CD.....	807
ÍNDICE	809

PRÓLOGO

C#, pronunciado *C Sharp*, es actualmente, junto con Java, uno de los lenguajes de programación más populares en Internet. Pero, además, está disponible para el desarrollo de programas de propósito general. La idea fundamental de esta obra es dar a conocer estas facetas del lenguaje C#, sin olvidar que tiene un alcance completo sobre la *Web*.

En los últimos tiempos C y C++ han sido los lenguajes más utilizados en el desarrollo de aplicaciones en general. Ambos lenguajes proporcionan al programador el nivel de abstracción preciso para abordar el desarrollo de cualquier aplicación por compleja que sea, así como mecanismos de bajo nivel para utilizar las características más avanzadas de las plataformas sobre las que se desarrolla; pero, en general, el tiempo necesario para desarrollar una aplicación resulta largo comparado con otros lenguajes como Visual Basic, que ofrecen, además de facilidad, una elevada productividad en el desarrollo de aplicaciones, aunque eso sí, sacrificando la flexibilidad que los desarrolladores de C y C++ requieren. La solución que Microsoft da a este problema es el lenguaje denominado C#. Se trata de un lenguaje moderno orientado a objetos que permite desarrollar una amplia gama de aplicaciones para la nueva plataforma Microsoft .NET, la cual se caracteriza por proporcionar utilidades y servicios para sacar un provecho total tanto de la informática como de las comunicaciones.

Más que otra cosa, el objetivo de C# es permitir a todos los desarrolladores en general, y a los de C y C++ en particular, abordar el desarrollo de aplicaciones complejas con facilidad y rapidez pero sin sacrificar la potencia y el control que ofrecen C y C++. Es un poco como tomar todas las cosas buenas de Visual Basic y añadirlas a C++, aunque recortando algunas de las tradiciones más ocultas y difíciles de conocer de C y C++. Esto elimina los errores de programación más comunes en C y C++. Por ejemplo:

- El recolector de basura libera al programador del peso que conlleva el manejo manual de la memoria.
- Todos los objetos creados dinámicamente así como las matrices son iniciados a cero, y aunque C# no inicia automáticamente las variables locales, el compilador avisará siempre que se intente utilizar una antes de iniciarla.
- C# unifica el sistema de tipos permitiendo ver a cada uno de ellos en el lenguaje como un objeto.

Resumiendo, C# permite escribir aplicaciones tanto para Internet como aplicaciones de propósito general; esto es, como muchos lenguajes, permite trabajar con todo tipo de datos, crear estructuras dinámicas, trabajar con ficheros, atacar a bases de datos, diseñar interfaces gráficas de usuario, etc. Más aún, C# es un lenguaje simple, potente y orientado a objetos. Su sintaxis incita al programador a generar programas modulares y fácilmente mantenibles.

Actualmente en el mercado hay multitud de herramientas de programación C# como *Microsoft Visual Studio* en sus distintas versiones o *SharpDevelop* por citar algunas. Pero la mejor forma de ver el alcance de C# es utilizando directamente el paquete de desarrollo *Microsoft .NET Framework Software Development Kit* (SDK). Se trata de un software que se puede obtener de la red basado en un conjunto de herramientas de órdenes en línea para editar, compilar, ejecutar y depurar programas C#, entre otros.

Para quién es este libro

Microsoft C#. Curso de programación se centra, casi en su totalidad, en el aprendizaje del lenguaje C#, de la programación orientada a objetos y en el desarrollo de aplicaciones. Un capítulo final le introducirá también en otros conceptos como interfaces gráficas, páginas Web y servicios Web. De forma resumida, esta materia puede agruparse en los siguientes apartados:

- *Programación básica.*
- *Mecanismos de abstracción.*
- *Diseño y programación.*

La primera parte está pensada para que en poco tiempo pueda convertirse en programador de aplicaciones C#. Y para esto, ¿qué necesita? Pues simplemente leer ordenadamente los capítulos del libro, resolviendo cada uno de los ejemplos que en ellos se detallan.

La segunda parte abarca en profundidad la programación orientada a objetos. En la primera parte el autor ha tratado de desarrollar aplicaciones sencillas, para introducirle más bien en el lenguaje y en el manejo de la biblioteca de clases de

.NET Framework que en el diseño de clases de objetos. No obstante, sí ha tenido que quedar claro que un programa orientado a objetos sólo se compone de objetos. Es hora pues de entrar con detalle en la programación orientada a objetos, la cual tiene un elemento básico: la *clase*.

Pero si el autor finalizara el libro con las dos partes anteriores, privaría al lector de saber que aún C# proporciona mucho más. Por eso hay una tercera parte que se dedica a estudiar las estructuras dinámicas de datos, los algoritmos de uso común, los hilos, y a introducirle en el diseño de interfaces gráficas de usuario, en el trabajo con *interfaces gráficas* y en aplicaciones que se ejecutan en un servidor.

Todo ello se ha documentado con abundantes problemas resueltos. Cuando complete todas las partes, todavía no sabrá todo lo que es posible hacer con C#, pero sí habrá dado un paso importante.

Si a continuación quiere ampliar sus conocimientos, puede continuar con mi otro libro titulado *Enciclopedia de Microsoft Visual C#* que cubre ampliamente y con detalle el desarrollo de: aplicaciones que muestran una interfaz gráfica al usuario, aplicaciones para acceso a bases de datos, para *Office*, para Internet (páginas Web) y para móviles. Puede ver más detalles sobre este libro y otros en mi Web: www.fjceballos.es.

El autor considera importante que antes de iniciar el estudio de este libro eche una ojeada a los apéndices. En ellos se expone cómo utilizar el entorno de desarrollo integrado (EDI) para C# denominado *Microsoft Visual Studio*, se explica cómo utilizar el EDI para escribir, ejecutar y depurar un programa o cómo proceder para trabajar sobre la plataforma Linux.

Cómo está organizado el libro

El libro se ha estructurado en los 19 capítulos especificados en el apartado *Resumen del contenido* expuesto anteriormente. Los capítulos del 1 al 9 nos presentan la programación básica: tipos, sentencias, matrices, métodos, etc. Los capítulos del 10 al 14 cubren la programación orientada a objetos (POO) en detalle: clases, espacios de nombres, estructuras, sobrecarga de operadores, clases derivadas, interfaces y excepciones. El 15 nos enseña a trabajar con ficheros con el fin de diseñar aplicaciones en las que los datos manipulados persistan de una ejecución a otra. El 16 presenta el diseño de estructuras dinámicas de datos: listas y árboles binarios. El 17 muestra los algoritmos de búsqueda y ordenación más comunes. El 18 estudia la programación concurrente: hilos. Y, finalmente, el 19 nos introduce en el diseño de aplicaciones que muestran una interfaz de ventanas al usuario y en el desarrollo de aplicaciones para Internet.

Qué se necesita para utilizar este libro

Este libro ha sido escrito utilizando el paquete *Microsoft .NET Framework Software Development Kit* (SDK) versión 4.0, que forma parte del entorno de desarrollo *Microsoft Visual Studio 2010* y que incluye todo lo necesario para escribir, construir, depurar y ejecutar aplicaciones .NET. Por lo tanto, basta con que instale en su máquina *Microsoft Visual Studio 2010* en cualquiera de sus versiones o, como alternativa, descargue e instale desde <http://www.microsoft.com/express/> el paquete *Visual C# 2010 Express*; y, finalmente, sólo para la introducción al desarrollo de aplicaciones para Internet, necesitaría descargar e instalar *Visual Web Developer 2010 Express*. Ambos paquetes incluyen el paquete *SQL Server Express* para trabajar con bases de datos.

Sobre los ejemplos del libro

La imagen del CD de este libro, con las aplicaciones desarrolladas y el software para reproducirlas, puede descargarla desde:

<https://www.tecno-libro.es/ficheros/descargas/9788499643960.zip>

La descarga consiste en un fichero ZIP con una contraseña ddd-dd-dddd-ddd-d que se corresponde con el ISBN de este libro (teclea los dígitos y los guiones).

Agradecimientos

He recibido ayuda de algunas personas durante la preparación de este libro y, por ello, estoy francamente agradecido. También, deseo expresar mi agradecimiento a *Microsoft Ibérica* por poner a mi disposición, en particular, y de todos los lectores en general, el SDK y los entornos de desarrollo integrado que el estudio de esta obra requiere.

Francisco Javier Ceballos Sierra

<http://www.fjceballos.es>

P A R T E

1

Programación básica

- Fases en el desarrollo de un programa
- Introducción a C#
- Introducción a la POO
- Elementos del lenguaje
- Estructura de un programa
- Clases de uso común
- Sentencias de control
- Matrices
- Más sobre métodos y colecciones

FASES EN EL DESARROLLO DE UN PROGRAMA

En este capítulo aprenderá lo que es un programa, cómo escribirlo y qué hacer para que el ordenador lo ejecute y muestre los resultados perseguidos. También adquirirá conocimientos generales acerca de los lenguajes de programación utilizados para escribir programas. Después, nos centraremos en un lenguaje de programación específico y objetivo de este libro, *C#* (*C Sharp*: C bien definido), presentando sus antecedentes y marcando la pauta a seguir para realizar un programa sencillo.

QUÉ ES UN PROGRAMA

Probablemente alguna vez haya utilizado un ordenador para escribir un documento o para divertirse con algún juego. Recuerde que en el caso de escribir un documento, primero tuvo que poner en marcha un procesador de textos, y que si quiso divertirse con un juego, lo primero que tuvo que hacer fue poner en marcha el juego. Tanto el procesador de textos como el juego son *programas* de ordenador.

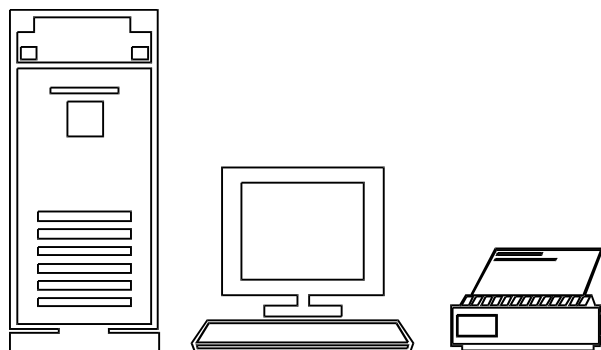
Poner un programa en marcha es sinónimo de ejecutarlo. Cuando ejecutamos un programa, nosotros sólo vemos los resultados que produce (el procesador de textos muestra sobre la pantalla el texto que escribimos; el juego visualiza sobre la pantalla las imágenes que se van sucediendo) pero no vemos el guión seguido por el ordenador para conseguir esos resultados. Ese guión es el programa.

Ahora, si nosotros escribimos un programa, entonces sí que sabemos cómo trabaja y por qué trabaja de esa forma. Esto es una forma muy diferente y curiosa de ver un programa de ordenador, lo cual no tiene nada que ver con la experiencia adquirida en la ejecución de distintos programas.

Ahora, piense en un juego cualquiera. La pregunta es: ¿qué hacemos si queremos enseñar a otra persona a jugar? Lógicamente le explicamos lo que debe hacer; esto es, los pasos que tiene que seguir. Dicho de otra forma, le damos instrucciones de cómo debe actuar. Esto es lo que hace un programa de ordenador. Un *programa* no es nada más que una serie de instrucciones dadas al ordenador en un lenguaje entendido por él, para decirle exactamente lo que queremos que haga. Si el ordenador no entiende alguna instrucción, lo comunicará generalmente mediante mensajes visualizados en la pantalla.

LENGUAJES DE PROGRAMACIÓN

Un programa tiene que escribirse en un lenguaje entendible por el ordenador. Desde el punto de vista físico, un ordenador es una máquina electrónica. Los elementos físicos (memoria, unidad central de proceso, etc.) de que dispone el ordenador para representar los datos son de tipo binario; esto es, cada elemento puede diferenciar dos estados (dos niveles de voltaje). Cada estado se denomina genéricamente *bit* y se simboliza por *0* o *1*. Por lo tanto, para representar y manipular información numérica, alfabética y alfanumérica se emplean cadenas de *bits*. Según esto, se denomina *byte* a la cantidad de información empleada por un ordenador para representar un carácter; generalmente un *byte* es una cadena de ocho *bits*.



Así, por ejemplo, cuando un programa le dice al ordenador que visualice un mensaje sobre el monitor, o que lo imprima sobre la impresora, las instrucciones correspondientes para llevar a cabo esta acción, para que puedan ser entendibles por el ordenador, tienen que estar almacenadas en la memoria como cadenas de *bits*. Esto hace pensar que escribir un programa utilizando ceros y unos (lenguaje máquina) llevaría mucho tiempo y con muchas posibilidades de cometer errores. Por este motivo, se desarrollaron los lenguajes *ensambladores*.

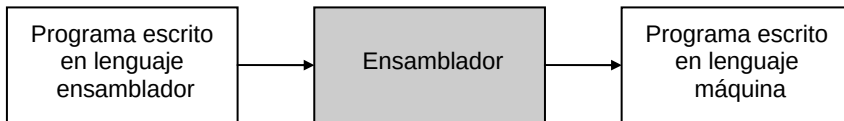
Un lenguaje *ensamblador* utiliza *códigos nemotécnicos* para indicarle al hardware (componentes físicos del ordenador) las operaciones que tiene que reali-

zar. Un código nemotécnico es una palabra o abreviatura fácil de recordar que representa una tarea que debe realizar el procesador del ordenador. Por ejemplo:

```
MOV AH, 4CH
```

El código *MOV* expresa una operación consistente en mover alguna información desde una posición de memoria a otra.

Para traducir un programa escrito en *ensamblador* a lenguaje máquina (código binario), se utiliza un programa llamado *ensamblador* que ejecutamos mediante el propio ordenador. Este programa tomará como datos nuestro programa escrito en lenguaje ensamblador y dará como resultado el mismo programa pero escrito en lenguaje máquina, lenguaje que entiende el ordenador.



Cada modelo de ordenador, dependiendo del procesador que utilice, tiene su propio lenguaje ensamblador. Debido a esto, decimos que estos lenguajes están orientados a la máquina.

Hoy en día son más utilizados los lenguajes orientados al problema o lenguajes de alto nivel. Estos lenguajes utilizan una terminología fácilmente comprensible que se aproxima más al lenguaje humano. En este caso la traducción es llevada a cabo por otro programa denominado *compilador*.

Cada sentencia de un programa escrita en un lenguaje de alto nivel se descompone en general en varias instrucciones en ensamblador. Por ejemplo:

```
printf( "hola" );
```

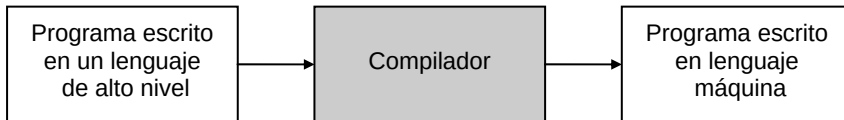
La función **printf** del lenguaje C le dice al ordenador que visualice en el monitor la cadena de caracteres especificada. Este mismo proceso escrito en lenguaje ensamblador necesitará de varias instrucciones. Lo mismo podríamos decir del método **WriteLine** de C#:

```
System.Console.WriteLine("hola");
```

A diferencia de los lenguajes ensambladores, la utilización de lenguajes de alto nivel no requiere en absoluto del conocimiento de la estructura del procesador que utiliza el ordenador, lo que facilita la escritura de un programa.

Compiladores

Para traducir un programa escrito en un lenguaje de alto nivel (programa fuente) a lenguaje máquina se utiliza un programa llamado *compilador*. Este programa tomará como datos nuestro programa escrito en lenguaje de alto nivel y dará como resultado el mismo programa pero escrito en lenguaje máquina, programa que ya puede ejecutar directa o indirectamente el ordenador.



Por ejemplo, un programa escrito en el lenguaje C necesita del compilador C para poder ser traducido. Posteriormente el programa traducido podrá ser ejecutado directamente por el ordenador. En cambio, para traducir un programa escrito en el lenguaje C# necesita del compilador C#; en este caso, el lenguaje máquina no corresponde al del ordenador sino al de una máquina ficticia, denominada máquina virtual, que será puesta en marcha por el ordenador para ejecutar el programa. En .NET esta máquina virtual es el CLR (*Common Language Runtime*).

¿Qué es una máquina virtual? Una máquina que no existe físicamente sino que es simulada en un ordenador por un programa. ¿Por qué utilizar una máquina virtual? Porque, por tratarse de un programa, es muy fácil instalarla en cualquier ordenador; basta con copiar ese programa en su disco duro, por ejemplo. Y, ¿qué ventajas reporta? Pues, en el caso de C#, que un programa escrito en este lenguaje y compilado puede ser ejecutado en cualquier ordenador del mundo que tenga instalada esa máquina virtual. Esta solución hace posible que cualquier ordenador pueda ejecutar un programa escrito en C# independiente de la plataforma que utilice, lo que se conoce como transportabilidad de programas.

Intérpretes

A diferencia de un compilador, un intérprete no genera un programa escrito en lenguaje máquina a partir del programa fuente, sino que efectúa la traducción y ejecución simultáneamente para cada una de las sentencias del programa. Por ejemplo, un programa escrito en el lenguaje *Basic* necesita el intérprete *Basic* para ser ejecutado. Durante la ejecución de cada una de las sentencias del programa, ocurre simultáneamente la traducción.

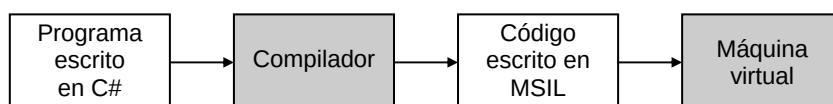
A diferencia de un compilador, un intérprete verifica cada línea del programa cuando se escribe, lo que facilita la puesta a punto del programa. En cambio la ejecución resulta más lenta ya que acarrea una traducción simultánea.

QUÉ ES C#

Para entender lo que es C# es imprescindible decir antes lo que es *Microsoft .NET Framework* o abreviadamente *.NET*. Se trata de un entorno de desarrollo multi-lenguaje diseñado por Microsoft para simplificar la construcción, distribución y ejecución de aplicaciones para Internet. Tiene fundamentalmente tres componentes: una máquina virtual (CLR: *Common Language Runtime*) que procesa código escrito en un lenguaje intermedio (MSIL: *Microsoft Intermediate Language*), una biblioteca de clases (biblioteca .NET Framework) y ASP.NET que proporciona los servicios necesarios para crear aplicaciones *Web*.

Precisamente C# es uno de los lenguajes de programación de alto nivel que pertenecen al paquete .NET (otros lenguajes son Visual Basic, C/C++, etc.). C# es una evolución de C/C++. Con él se pueden escribir tanto programas convencionales como para Internet. Las aplicaciones podrán mostrar una interfaz gráfica al usuario, o bien una interfaz de texto, como hacen las denominadas aplicaciones de consola.

El paquete .NET incluye un compilador (programa traductor) de C# que produce un código escrito en un lenguaje intermedio, común para todos los lenguajes de dicha plataforma, que será el que la máquina virtual ejecutará (esto es, cada lenguaje de la plataforma tiene su compilador que produce código correspondiente a un único lenguaje: MSIL).



Por lo tanto, MSIL es un lenguaje máquina que no es específico de ningún procesador, sino de la máquina virtual de .NET. En realidad se trata de un lenguaje de más alto nivel que otros lenguajes máquina: trata directamente con objetos y tiene instrucciones para cargarlos, guardarlos, iniciarlos, invocar a sus métodos, así como para realizar operaciones aritméticas y lógicas, para controlar el flujo de ejecución, etc. A su vez, la máquina virtual posee un recolector de basura (para eliminar los objetos cuando no estén referenciados) y proporciona traductores del lenguaje intermedio a código nativo para cada arquitectura soportada; se trata de compiladores JIT (*Just in Time*: al instante).

Por otra parte, antes de que el código MSIL pueda ser ejecutado por el procesador de nuestra máquina, debe ser convertido a código nativo. Ésta es la tarea del compilador JIT: producir código nativo para el microprocesador particular de nuestra máquina. Normalmente, el código MSIL es convertido a código nativo

según se va ejecutando (el código que se va obteniendo se va guardando para que esté accesible para subsiguientes llamadas).

¿POR QUÉ APRENDER C#?

Según lo expuesto, es fácil entender que una de las ventajas significativas de C# sobre otros lenguajes de programación es que es independiente de la plataforma (lo mismo podemos decir respecto a los demás lenguajes incluidos en .NET). Esto quiere decir que el código producido por el compilador C# puede transportarse a cualquier plataforma (Intel, Sparc, Motorola, etc.) que tenga instalada una máquina virtual de .NET y ejecutarse. Pensando en Internet esta característica es crucial ya que esta red conecta ordenadores muy distintos.

Otra característica importante de C# es que es un lenguaje de programación orientado a objetos (POO). Los conceptos en los que se apoya esta técnica de programación y sus ventajas serán expuestos en los capítulos siguientes.

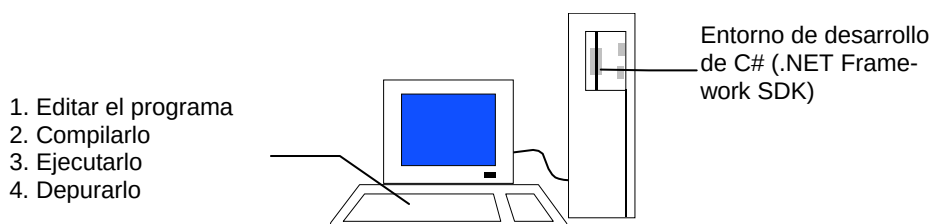
Además de ser transportable y orientado a objetos, C# es un lenguaje fácil de aprender. Tiene un tamaño pequeño que favorece el desarrollo y reduce las posibilidades de cometer errores; a la vez es potente y flexible.

C# está fundamentado en C++. Quiere esto decir que mucha de la sintaxis y diseño orientado a objetos se tomó de este lenguaje. Por lo tanto, a los lectores que estén familiarizados con C++ y la POO les será muy fácil aprender a desarrollar aplicaciones con C#. Quiero advertir a este tipo de potenciales usuarios de C# que, en teoría, en este lenguaje no existen punteros ni aritmética de punteros, las cadenas de caracteres son objetos y la administración de memoria es automática, lo que elimina la problemática que presenta C++ con las lagunas de memoria al olvidar liberar bloques de la misma que fueron asignados dinámicamente.

REALIZACIÓN DE UN PROGRAMA EN C#

En este apartado se van a exponer los pasos a seguir en la realización de un programa, por medio de un ejemplo.

La siguiente figura muestra de forma esquemática lo que un usuario de C# necesita y debe hacer para desarrollar un programa:



Evidentemente, para poder escribir programas se necesita un entorno de desarrollo. *Microsoft*, propietario de C#, proporciona uno de forma gratuita, *.NET Framework SDK*, que se puede obtener en la dirección de Internet:

<http://www.microsoft.com/downloads>

En Internet se encuentran todas las versiones de *.NET Framework* para Windows (para Linux utilice el proyecto *Mono*), con las que podrá realizar todos los ejemplos incluidos en esta obra.

Una vez instalado *.NET Framework* (puede echar una ojeada en la carpeta *C:\Windows\Microsoft.NET* para ver los componentes instalados), sólo falta un editor de código fuente C#. Es suficiente con un editor de texto sin formato; por ejemplo el *Bloc de notas* de Windows. No obstante, todo el trabajo de edición, compilación, ejecución y depuración se hará mucho más fácil si se utiliza un entorno de desarrollo con interfaz gráfica de usuario que integre las herramientas mencionadas, en lugar de tener que utilizar la interfaz de línea de órdenes del SDK, como veremos a continuación.

Entornos de desarrollo integrados (EDI) para C# hay varios, pero por encima de todos destaca *Microsoft Visual Studio*. En el CD correspondiente al libro puede obtener la dirección para obtener versiones gratuitas de *Visual C# Express*, entorno de desarrollo integrado para escribir aplicaciones C# (en el apéndice A puede ver notas sobre su forma de uso), *SQL Express* para trabajar con bases de datos, y *Visual Web Developer Express* para escribir aplicaciones Web. La instalación de uno de estos paquetes, por ejemplo *Visual C# Express*, instala también *.NET Framework*. Una vez instalado puede echar una ojeada a la ruta especificada en el apartado anterior y a esta otra: *C:\Program Files (x86)\Microsoft Visual Studio 10.0*.

Cómo crear un programa

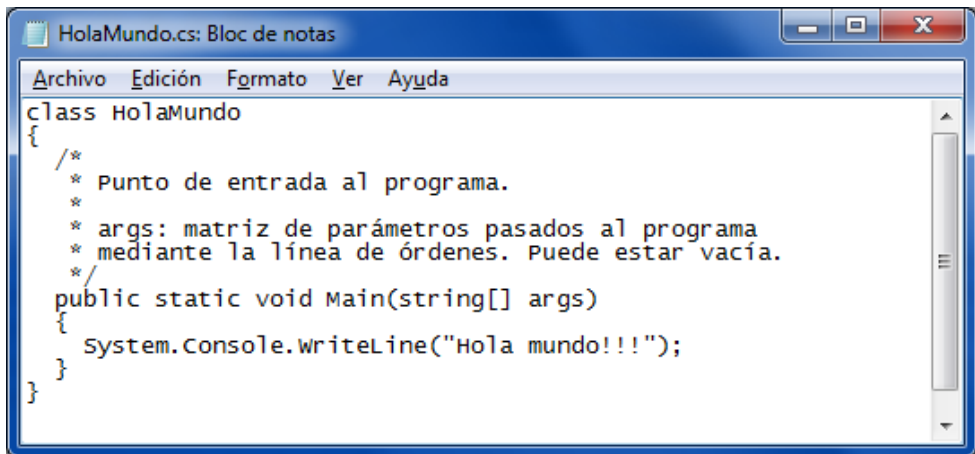
Empecemos con la creación de un programa muy simple: el clásico ejemplo de mostrar un mensaje de saludo.

Este sencillo programa lo realizaremos desde los dos puntos de vista comentados anteriormente: utilizando la interfaz de línea de órdenes del SDK y utilizando un entorno de desarrollo integrado.

Interfaz de línea de órdenes

Empecemos por editar el fichero fuente C# correspondiente al programa. Primeramente visualizaremos el editor de textos que vayamos a utilizar, el cual debe permitir guardar texto sin formato; por ejemplo, el *Bloc de notas* en Windows. El nombre del fichero elegido para guardar el programa en el disco debe tener como extensión cs; por ejemplo *HolaMundo.cs*.

Una vez visualizado el editor, escribiremos el texto correspondiente al programa fuente. Escríbalo tal y como se muestra a continuación. Observe que una *sentencia* del lenguaje C# finaliza con un *punto y coma* y que cada *línea del programa* se finaliza pulsando la tecla *Entrar* (*Enter* o *↵*).



```
class HolaMundo
{
    /*
     * Punto de entrada al programa.
     * args: matriz de parámetros pasados al programa
     * mediante la línea de órdenes. Puede estar vacía.
     */
    public static void Main(string[] args)
    {
        System.Console.WriteLine("Hola mundo!!!");
    }
}
```

¿Qué hace este programa?

Comentamos brevemente cada línea del programa anterior. No se apure si algunos de los términos no quedan muy claros ya que todos ellos se verán con detalle en capítulos posteriores.

La primera línea declara la clase de objetos *HolaMundo*, porque el esqueleto de cualquier programa C# se basa en la definición de una clase. A continuación se escribe el cuerpo de la clase encerrado entre los caracteres { y }. Ambos caracteres definen el bloque de código en el que se escriben las acciones a llevar a cabo por el programa C#. Las clases son la base de los programas C#. Aprenderemos más sobre ellas en los próximos capítulos.

Las siguientes líneas encerradas entre `/*` y `*/` son simplemente un comentario. Los comentarios no son tenidos en cuenta por el compilador, pero ayudan a entender un programa cuando se lee.

A continuación se escribe el método principal **Main**. Observe que un método se distingue por el modificador `()` que aparece después de su nombre y que el bloque de código correspondiente al mismo define las acciones que tiene que ejecutar dicho método. Cuando se ejecuta un programa, C# espera que haya un método **Main**. Este método define el punto de entrada y de salida del programa.

En el ejemplo se observa que el método **Main** llama para su ejecución al método **WriteLine** de la clase **Console** del espacio de nombres **System** de la biblioteca .NET (un espacio de nombres agrupa un conjunto de clases bajo un nombre), que escribe como resultado la expresión que aparece especificada entre comillas. Una secuencia de caracteres entre comillas se denomina *cadena de caracteres*.

Observe también que la sentencia que invoca a **WriteLine** finaliza con punto y coma, no sucediendo lo mismo con la cabecera de la clase *HolaMundo*, ni tampoco con la cabecera del método **Main** porque la sintaxis para ambos indica que a continuación debe escribirse el bloque de código - `{ }` - que definen. Resumiendo: un programa C# se basa en la definición de una clase, una clase contiene métodos, además de otras definiciones, y un método, a su vez, contiene sentencias y otras definiciones, como veremos más adelante.

Guardar el programa escrito en el disco

El programa editado está ahora en la memoria. Para que este trabajo pueda tener continuidad, se debe grabar en el disco utilizando la orden correspondiente del editor. El nombre del programa fuente, programa escrito en C#, puede ser diferente al de la clase que contiene. En nuestro caso, el nombre de la clase es *HolaMundo* y el nombre del fichero *HolaMundo.cs*, pero podría haber sido *saludo.cs*.

Un fichero con código C# no necesita de ningún otro fichero, como por ejemplo, los ficheros de cabecera de otros lenguajes.

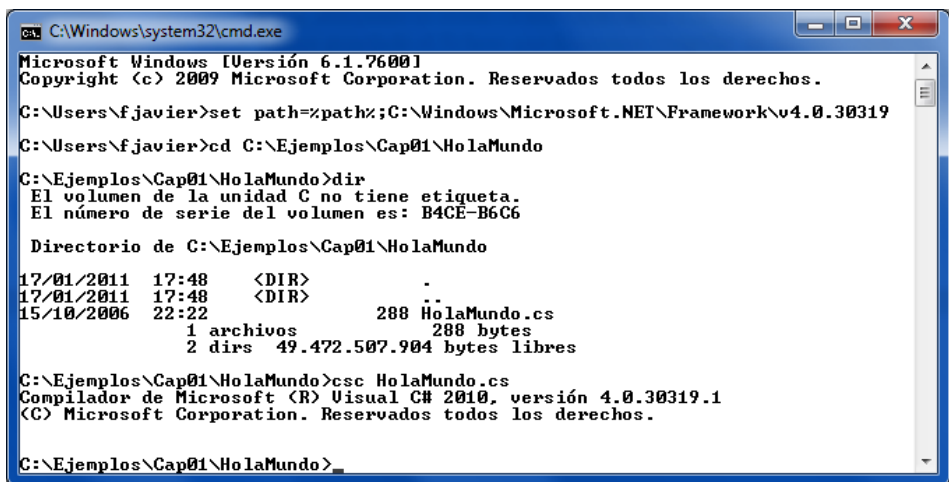
Compilar y ejecutar el programa

El siguiente paso es *compilar* el programa; esto es, traducir el programa fuente a código intermedio (MSIL) para posteriormente poder ejecutarlo. La figura de la página siguiente muestra los pasos a seguir en este proceso que a continuación explicamos.

Según dijimos anteriormente, el *SDK* proporciona un programa ejecutable desde la línea de órdenes, *csc*, para compilar cualquier programa fuente escrito en C#. El resultado será un fichero de nombre igual al del fichero fuente y extensión *.exe* que almacenará el código intermedio obtenido en la traducción, código que puede ser directamente interpretado por la máquina virtual de .NET cuando requiramos ejecutar el programa. Según esto, lo primero es visualizar una ventana que muestre la línea de órdenes (para ello, los usuarios de Windows pueden ejecutar *cmd* o *command* desde la ventana *Inicio - Ejecutar*). Después, para informar al sistema operativo de la ubicación de la utilidad *csc*, desde la línea de órdenes añadiremos a la variable de entorno *path* la ruta de la carpeta donde está almacenada esta utilidad y otras que utilizaremos a continuación. Por ejemplo:

```
set path=%path%;C:\WINDOWS\Microsoft.NET\Framework\vXXX
```

La expresión *%path%* representa el valor actual de la variable de entorno *path*. La ruta *C:\WINDOWS\Microsoft.NET\Framework\vXXX* es donde, en nuestro caso, están ubicadas las utilidades C#.NET. Observe que una ruta va separada de la anterior por un punto y coma. Este trabajo no será necesario hacerlo cuando, al instalar el entorno .NET, esta ruta y otras sean añadidas automáticamente a la variable de entorno *path*. Puede comprobarlo ejecutando *path* desde la línea de órdenes.



```

C:\Windows\system32\cmd.exe
Microsoft Windows [Versión 6.1.7600]
Copyright (c) 2009 Microsoft Corporation. Reservados todos los derechos.

C:\Users\fjavier>set path=%path%;C:\Windows\Microsoft.NET\Framework\v4.0.30319
C:\Users\fjavier>cd C:\Ejemplos\Cap01\HolaMundo
C:\Ejemplos\Cap01\HolaMundo>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: B4CE-B6C6

Directorio de C:\Ejemplos\Cap01\HolaMundo
17/01/2011  17:48    <DIR>          .
17/01/2011  17:48    <DIR>          ..
15/10/2006  22:22                288 HolaMundo.cs
               1 archivos                288 bytes
               2 dirs  49.472.507.904 bytes libres

C:\Ejemplos\Cap01\HolaMundo>csc HolaMundo.cs
Compilador de Microsoft (R) Visual C# 2010, versión 4.0.30319.1
(C) Microsoft Corporation. Reservados todos los derechos.

C:\Ejemplos\Cap01\HolaMundo>_

```

A continuación, utilizando la orden *cd* nos cambiamos a la carpeta de trabajo, carpeta donde hemos guardado el fichero que deseamos compilar; en nuestro caso *C:\Ejemplos\Cap01\HolaMundo*. La orden *dir* nos permitirá ver el contenido de esta carpeta.

Finalmente, compilamos el fichero fuente que almacena el programa. La orden para compilar el programa *HolaMundo.cs* es la siguiente:

```
csc HolaMundo.cs
```

Obsérvese que para compilar un programa hay que especificar la extensión `.cs`. El resultado de la compilación será un fichero `HolaMundo.exe` que contiene el código que ejecutará la máquina virtual de .NET.

Al compilar un programa, se pueden presentar *errores de compilación*, debidos a que el programa escrito no se adapta a la sintaxis y reglas del compilador. Estos errores se irán corrigiendo hasta obtener una compilación sin errores.

Por ejemplo, si al compilar el programa se muestra un mensaje de error como:

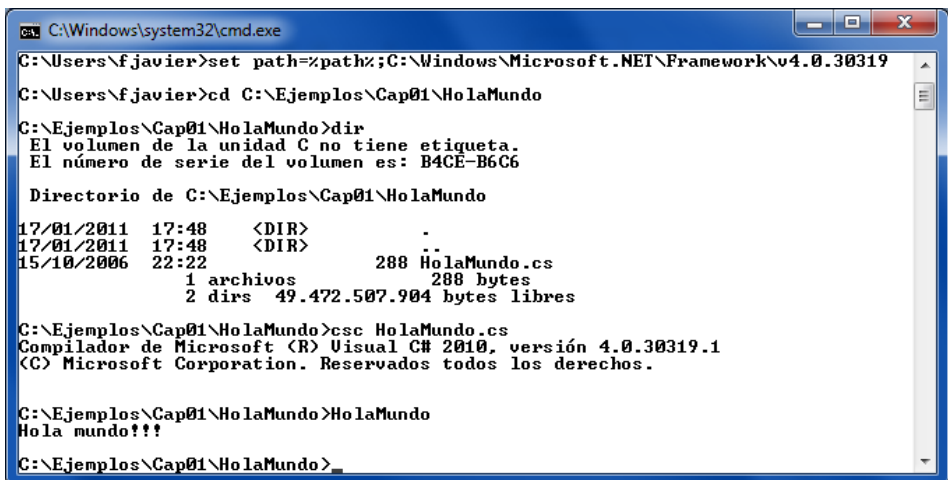
```
error CS5001: El programa 'c:\Ejemplos\Cap01\HolaMundo\HolaMundo.exe' no
contiene ningún método 'Main' estático adecuado para un punto de entrada
```

asegúrese de que el nombre del método **Main** lo ha escrito tal cual, la primera letra mayúscula. C# es un lenguaje sensible a las mayúsculas y minúsculas; por lo tanto, no es lo mismo escribir **main** que **Main**.

Para ejecutar el fichero resultante de la compilación y observar los resultados, basta con escribir en la línea de órdenes el nombre de dicho fichero, en nuestro caso `HolaMundo`, y después pulsar *Entrar*.

```
HolaMundo[Entrar]
```

En la figura siguiente se puede observar el proceso seguido para ejecutar `HolaMundo` desde la línea de órdenes. Observe que al escribir el nombre del programa se han hecho coincidir mayúsculas y minúsculas, pero esto no es necesario. Así mismo, cabe resaltar que la extensión `.exe` no tiene que ser especificada.



```

C:\Windows\system32\cmd.exe
C:\Users\fjavier>set path=%path%;C:\Windows\Microsoft.NET\Framework\v4.0.30319
C:\Users\fjavier>cd C:\Ejemplos\Cap01\HolaMundo
C:\Ejemplos\Cap01\HolaMundo>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: B4CE-B6C6

Directorio de C:\Ejemplos\Cap01\HolaMundo
17/01/2011  17:48    <DIR>          .
17/01/2011  17:48    <DIR>          ..
15/10/2006  22:22                288 HolaMundo.cs
                1 archivos             288 bytes
                2 dirs  49.472.507.904 bytes libres

C:\Ejemplos\Cap01\HolaMundo>csc HolaMundo.cs
Compilador de Microsoft (R) Visual C# 2010, versión 4.0.30319.1
(C) Microsoft Corporation. Reservados todos los derechos.

C:\Ejemplos\Cap01\HolaMundo>HolaMundo
Hola mundo!!!

C:\Ejemplos\Cap01\HolaMundo>
  
```

Una vez ejecutado, se puede observar que el resultado es el mensaje: *Hola mundo!!!*

Biblioteca de clases

C# carece de instrucciones de E/S, de instrucciones para manejo de cadenas de caracteres, etc., con lo que este trabajo queda para la biblioteca de clases provista con el compilador. C# no tiene una biblioteca de su propiedad, sino que utiliza la biblioteca .NET Framework. Todos los lenguajes del paquete .NET utilizan esta misma biblioteca. Una biblioteca está formada por un conjunto de ficheros separados en el disco (con extensión *.dll*, *.lib*, *.tlb*, etc.) que contienen las clases que definen las tareas más comunes, para que nosotros no tengamos que escribirlas. Como ejemplo, hemos visto anteriormente el método **WriteLine** de la clase **Console** del espacio de nombres **System**. Si este método no existiera, sería labor nuestra escribir el código necesario para visualizar los resultados en la ventana.

Cuando sea necesario especificar las bibliotecas en las que están definidas las clases que utiliza nuestro programa, habrá que utilizar la opción */r* del compilador análogamente a como se indica en el ejemplo siguiente:

```
csc /r:System.Windows.Forms.dll,System.Drawing.dll fichero.cs
```

En el código del ejemplo *HolaMundo* se puede observar que para utilizar un método de una clase de la biblioteca simplemente hay que invocarlo y pasarle los argumentos necesarios entre paréntesis. Por ejemplo:

```
System.Console.WriteLine("Hola mundo!!!");
```

Guardar el programa ejecutable en el disco

Como hemos visto, cada vez que se realiza el proceso de *compilación* del programa actual, C# genera automáticamente sobre el disco un fichero *.exe*. Este fichero puede ser ejecutado directamente desde el sistema operativo escribiendo en la línea de órdenes su nombre y pulsando la tecla *Entrar* (esta acción pondrá en marcha la máquina virtual de .NET).

Al ejecutar el programa, pueden producirse *errores durante la ejecución*. Por ejemplo, puede darse una división por cero. Estos errores solamente pueden ser detectados por C# cuando se ejecuta el programa y serán notificados con el correspondiente mensaje de error.

Hay *otro tipo de errores* que no dan lugar a mensaje alguno. Por ejemplo: un programa que no termine nunca de ejecutarse, debido a que presenta un lazo, donde no se llega a dar la condición de terminación. Para detener la ejecución en un

caso como éste se tienen que pulsar las teclas *Ctrl+C* (en un entorno integrado se ejecutará una orden equivalente a *Detener ejecución*).

Depurar un programa

Una vez ejecutado el programa, la solución puede ser incorrecta. Este caso exige un análisis minucioso de cómo se comporta el programa a lo largo de su ejecución; esto es, hay que entrar en la fase de *depuración* del programa.

La forma más sencilla y eficaz para realizar este proceso es utilizar un programa *depurador*. Normalmente los entornos de desarrollo integrados potentes como *Microsoft Visual Studio*, incluidas la versiones *Express* que anteriormente hemos mencionado, incorporan las órdenes necesarias para invocar y depurar un programa con facilidad (véase el apéndice A).

Entorno de desarrollo integrado

Como hemos dicho anteriormente, cuando se utiliza un entorno de desarrollo integrado todo resulta más sencillo, porque las operaciones de crear un proyecto, editarlo, compilarlo, ejecutarlo y depurarlo están automatizadas. En el apéndice A se resume cómo utilizar el entorno de desarrollo *Microsoft Visual Studio*.

EJERCICIOS RESUELTOS

Para practicar con un programa más, escriba el siguiente ejemplo y pruebe los resultados. Hágalo primero desde la línea de órdenes y después con el entorno de desarrollo integrado preferido por usted. El siguiente ejemplo visualiza como resultado la suma, la resta, la multiplicación y la división de dos cantidades enteras.

Abra el procesador de textos o el editor de su entorno integrado y edite el programa ejemplo que se muestra a continuación. Recuerde, el nombre del fichero fuente, programa escrito en C#, puede ser diferente al de la clase que contiene, *CAritmetica*, y debe tener extensión *.cs*.

```
class CAritmetica
{
    /*
     * Operaciones aritméticas
     */
    public static void Main(string[] args)
    {
        int dato1, dato2, resultado;
        dato1 = 20;
        dato2 = 10;
```

```
// Suma
resultado = dato1 + dato2;
System.Console.WriteLine("{0} + {1} = {2}", dato1, dato2, resultado);

// Resta
resultado = dato1 - dato2;
System.Console.WriteLine("{0} - {1} = {2}", dato1, dato2, resultado);

// Producto
resultado = dato1 * dato2;
System.Console.WriteLine("{0} * {1} = {2}", dato1, dato2, resultado);

// Cociente
resultado = dato1 / dato2;
System.Console.WriteLine("{0} / {1} = {2}", dato1, dato2, resultado);
}
}
```

Una vez editado el programa, guárdelo en el disco con el nombre *CAritmetica.cs*.

¿Qué hace este programa? Si nos fijamos en el método principal, **Main**, vemos que se han declarado tres variables enteras (de tipo **int**): *dato1*, *dato2* y *resultado*.

```
int dato1, dato2, resultado;
```

El siguiente paso asigna el valor 20 a la variable *dato1* y el valor 10 a la variable *dato2*.

```
dato1 = 20;
dato2 = 10;
```

A continuación se realiza la suma de esos valores y se escriben los datos y el resultado.

```
resultado = dato1 + dato2;
System.Console.WriteLine("{0} + {1} = {2}", dato1, dato2, resultado);
```

El método **WriteLine** escribe un resultado de la forma:

```
20 + 10 = 30
```

Observe que la expresión resultante está formada por cinco elementos: *dato1*, *" + "*, *dato2*, *" = "* y *resultado*; unos elementos son numéricos y otros son constantes de caracteres. Esto se ha especificado mediante el formato *{0} + {1} = {2}*; una especificación de la forma *{número}* indica que se ha de mostrar el valor del

argumento que está en la posición *número* (en el ejemplo, *dato1* es el argumento que está en la posición 0, *dato2* está en la 1 y *resultado* en la 2); cualquier otro carácter entre las comillas dobles, aparte de las especificaciones, se mostrará tal cual (en el ejemplo, los espacios en blanco, el + y el =).

Un proceso similar se sigue para calcular la diferencia, el producto y el cociente. Para finalizar, compile, ejecute el programa y observe los resultados.

EJERCICIOS PROPUESTOS

1. Practique la edición, la compilación y la ejecución con un programa similar al programa *CAritmetica.cs* realizado en el apartado anterior. Por ejemplo, modifíquelo para que ahora realice las operaciones de sumar, restar y multiplicar con tres datos: *dato1*, *dato2* y *dato3*. En un segundo intento, puede también combinar las operaciones aritméticas.
2. Realice el mismo ejercicio anterior, pero ahora utilizando un entorno de desarrollo integrado (EDI) análogamente a como se indica en el apéndice A.

INTRODUCCIÓN A C#

En este capítulo expondremos de la forma más sencilla posible las operaciones que con mayor frecuencia se realizan en todo programa, tales como definir variables, asignar valores, mostrar un resultado, operaciones aritméticas y de comparación, añadir comentarios y escribir métodos. Todos estos conceptos serán ampliados en capítulos posteriores.

DECLARACIÓN DE UNA VARIABLE

Una variable representa un espacio de memoria para almacenar un valor de un determinado tipo, valor que puede ser modificado a lo largo de la ejecución del bloque donde la variable es accesible, tantas veces como se necesite. La declaración de una variable consiste en enunciar el nombre de la misma y asociarle un tipo. Por ejemplo, el siguiente código declara cuatro variables: *a* de tipo **double**, *b* de tipo **float** y *c* y *r* de tipo **int**:

```
class CAritmetica
{
    public static void Main(string[] args)
    {
        double a;
        float b;
        int c, r;
        // ...
    }
    // ...
}
```

Por definición, una variable declarada dentro de un bloque, entendiendo por bloque al código encerrado entre los caracteres '{' y '}', es accesible sólo dentro

de ese bloque. Más adelante, cuando tratemos con objetos matizaremos el concepto de accesibilidad.

Según lo expuesto, las variables *a*, *b*, *c* y *r* son accesibles sólo desde el método **Main** de la clase *CAritmetica*. En este caso se dice que dichas variables son *locales* al bloque donde han sido declaradas. Una variable local se crea cuando se ejecuta el bloque donde se declara y se destruye cuando finaliza la ejecución de dicho bloque.

Las variables locales no son iniciadas por el compilador C#. Por lo tanto, es nuestra obligación iniciarlas, de lo contrario el compilador visualizará un mensaje de error en todas las sentencias que hagan referencia a esas variables.

```
class CAritmetica
{
    public static void Main(string[] args)
    {
        double a = 0;
        float b = 0;
        int c = 0, r = 0;

        // ...
        System.Console.WriteLine("{0} {1} {2} {3}", a, b, c, r);
    }
}
```

Cuando elija el identificador para declarar una variable, tenga presente que el compilador C# trata las letras mayúsculas y minúsculas como caracteres diferentes. Por ejemplo las variables *dato1* y *Dato1* son diferentes.

Respecto al tipo de una variable, depende del tipo de valor que vaya a almacenar. Distinguimos varios tipos de datos que podemos clasificar en: tipos enteros, **sbyte**, **short**, **int**, **long** y **char**, tipos reales, **float**, **double** y **decimal** y el tipo **bool**.

Cada tipo tiene un rango diferente de valores positivos y negativos, excepto el **bool** que sólo tiene dos valores: **true** y **false**. Por lo tanto, el tipo que se seleccione para declarar cada variable de un determinado programa dependerá del rango y tipo de los valores que vayan a almacenar: enteros, fraccionarios o booleanos.

El tipo **bool** se utiliza para indicar si el resultado de la evaluación de una expresión booleana es verdadero o falso.

El tipo **sbyte** permite declarar datos enteros comprendidos entre -128 y $+127$ (un *byte* se define como un conjunto de 8 bits, independientemente de la plata-

forma). El tipo **short** permite declarar datos enteros comprendidos entre -32768 y $+32767$ (16 bits de longitud), el tipo **int** declara datos enteros comprendidos entre -2147483648 y $+2147483647$ (32 bits de longitud) y el tipo **long** permite declarar datos enteros comprendidos entre los valores -9223372036854775808 y $+9223372036854775807$ (64 bits de longitud). A continuación se muestran algunos ejemplos:

```
sbyte b = 0;
short i = 0, j = 0;
int k = -30;
long l = 125000;
```

A los tipos anteriores, C# añade los correspondientes tipos enteros sin signo: **byte** (0 a 2^8-1), **ushort** (0 a $2^{16}-1$), **uint** (0 a $2^{32}-1$) y **ulong** (0 a $2^{64}-1$).

El tipo **char** es utilizado para declarar datos enteros en el rango 0 a 65535. Los valores 0 a 127 se corresponden con los caracteres ASCII del mismo código. El juego de caracteres ASCII conforma una parte muy pequeña del juego de caracteres UNICODE utilizado por C# para representar los caracteres. UNICODE es un código de 16 bits (valores de 0 a 65535), esto es, cada carácter ocupa 2 bytes, con el único propósito de internacionalizar el lenguaje.

El siguiente ejemplo declara la variable *car* de tipo **char** a la que se le asigna el carácter 'a' como valor inicial (observe que hay una diferencia entre 'a' y *a*; *a* entre comillas simples es interpretada por el compilador C# como un valor, un carácter, y *a* sin comillas sería interpretada como una variable). Las dos declaraciones siguientes son idénticas:

```
char car = 'a';
char car = (char)97; /* la 'a' es el decimal 97 */
```

El tipo **float** (32 bits de longitud) se utiliza para declarar un dato que puede contener una parte decimal. Los datos de tipo **float** almacenan valores con una precisión aproximada de siete dígitos. Por ejemplo:

```
float a = 3.14159F;
float b = 2.2e-5F; /* 2.2e-5 = 2.2 por 10 elevado a -5 */
float c = 2/3F; /* 0,6666667 */
```

Para especificar que una constante es de tipo **float**, hay que añadir al final de su valor la letra 'f' o 'F', de lo contrario será considerada de tipo **double**.

El tipo **double** (64 bits de longitud) se utiliza para declarar un dato que puede contener una parte decimal. Los datos de tipo **double** almacenan valores con una

precisión aproximada de 15-16 dígitos. El siguiente ejemplo declara la variable *a* de tipo real de precisión doble:

```
double a = 3.14159; /* una constante es double por omisión */
```

El tipo **decimal** (128 bits de longitud) se utiliza para declarar un dato que puede contener una parte decimal que proporciona una precisión mayor que un **double**, siendo más apropiado para cálculos monetarios y financieros. Los datos de tipo **decimal** almacenan valores con una precisión aproximada de 28-29 dígitos. El siguiente ejemplo declara la variable *a* de tipo **decimal**:

```
decimal a = 1203.14M;
```

Para especificar que una constante es de tipo **decimal**, hay que añadir al final de su valor la letra 'm' o 'M', de lo contrario será considerada de tipo **double**.

ASIGNAR VALORES

La finalidad de un programa es procesar datos *numéricos* y *cadenas de caracteres* para obtener un resultado. Estos datos, generalmente, estarán almacenados en variables y el resultado obtenido también será almacenado en variables. ¿Cómo son almacenados? Pues a través de los métodos proporcionados por las clases de la biblioteca .NET o bien utilizando una sentencia de asignación de la forma:

variable operador_de_asignación valor

Una sentencia de asignación es asimétrica. Esto quiere decir que se evalúa la expresión que está a la derecha del operador de asignación y el resultado se asigna a la variable especificada a su izquierda. Por ejemplo:

```
resultado = dato1 + dato2;
```

Pero, según lo expuesto, no sería válido escribir:

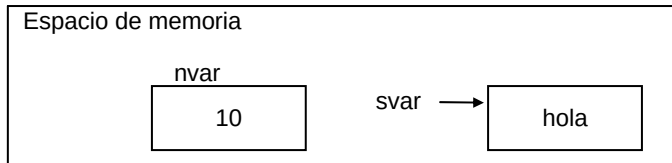
```
dato1 + dato2 = resultado;
```

Mientras que los datos numéricos son almacenados en variables de alguno de los tipos de valores expuestos anteriormente, las cadenas de caracteres son almacenadas en objetos de tipo **string** o en matrices, cuyo estudio se pospone para un capítulo posterior; no obstante veamos un ejemplo. Un objeto de tipo **string** (alias de la clase **System.String**) se define y se le asigna un valor así:

```
string cadena; // cadena permite referenciar un objeto string  
cadena = "hola"; // ambas equivalen a: string cadena = "hola";
```

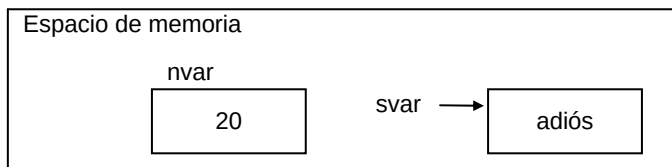
Cuando se asigna un valor a una variable estamos colocando ese valor en una localización de memoria asociada con esa variable. Por ejemplo:

```
int nvar = 10;           // variable de un tipo entero (int)
string svar = "hola";    // referencia a un objeto de tipo string
```



Lógicamente, cuando la variable tiene asignado un valor y se le asigna otro nuevo, el valor anterior es destruido ya que el valor nuevo pasa a ocupar la misma localización de memoria. En el ejemplo siguiente, se puede observar con respecto a la situación anterior que el contenido de *nvar* se modifica con un nuevo valor 20, y que la referencia *svar* también se modifica; ahora contiene la referencia a un nuevo objeto **string** "adiós".

```
nvar = 20;
svar = "adiós";
```



El siguiente ejemplo declara tres variables numéricas *a*, *b* y *c*, y una referencia *s* a una cadena de caracteres; después asigna valores a esas variables.

```
class CAsignarDatos
{
    public static void Main(string[] args)
    {
        double a = 0, b = 0;
        int c = 0;
        string s;

        a = 3.14; b = 2.71; c = 2;
        s = "Datos";
    }
}
```

AÑADIR COMENTARIOS

Un comentario es un mensaje dirigido a cualquiera que lea el código fuente. Añadiendo comentarios se hace más fácil la comprensión de un programa. Tienen como finalidad explicar el código fuente. Se pueden utilizar comentarios acotados o de una sola línea.

Un comentario acotado empieza con los caracteres `/*` y finaliza con los caracteres `*/`. Estos comentarios pueden ocupar más de una línea, pero no pueden anidarse. Por ejemplo:

```
class CAsignarDatos
{
    /*
    * Asignar datos:
    *   a, b, c representan datos numéricos.
    *   s representa una cadena de caracteres.
    */
    public static void Main(string[] args)
    {
        double a = 0, b = 0;
        int c = 0;
        string s;

        a = 3.14; b = 2.71; c = 2;
        s = "Datos";
    }
}
```

Un comentario de una sola línea comienza con una doble barra (`//`) y se extiende hasta el final de la línea. Por ejemplo:

```
class CAsignarDatos
{
    // Asignar datos:
    //   a, b, c representan datos numéricos.
    //   s representa una cadena de caracteres.
    public static void Main(string[] args)
    {
        double a = 0, b = 0;
        int c = 0;
        string s;

        a = 3.14; b = 2.71; c = 2;
        s = "Datos";
    }
}
```

MOSTRAR DATOS POR LA PANTALLA

La biblioteca .NET proporciona un método, **System.Console.WriteLine**, vinculado con la salida estándar, normalmente la pantalla, que permite visualizar datos numéricos de cualquier tipo y cadenas de caracteres.

El siguiente ejemplo declara tres variables numéricas *a*, *b* y *c*, y una referencia *s* a una cadena de caracteres; después asigna valores a las variables y finalmente los muestra.

```
class CMostrarDatos
{
    /*
     * Mostrar datos
     */
    public static void Main(string[] args)
    {
        double a = 0, b = 0;
        int c = 0;
        string s;

        a = 3.14; b = 2.71; c = 2;
        s = "Datos";

        System.Console.WriteLine(s + ": ");
        System.Console.WriteLine(" a = " + a);
        System.Console.WriteLine(" b = " + b);
        System.Console.WriteLine(" c = " + c);
    }
}
```

Ejecución del programa:

```
Datos:
a = 3,14
b = 2,71
c = 2
```

Observe que para mostrar los datos deseados, el método **Main** invoca al método **WriteLine** de la clase **Console** del espacio de nombres **System** de la biblioteca .NET. La expresión a mostrar se incluye entre los paréntesis del método y puede estar formada por elementos numéricos, cadenas de caracteres y constantes de caracteres (conjuntos de caracteres encerrados entre comillas dobles). Para unir esos elementos y formar la expresión a mostrar se utiliza el operador **+**. Cuando el método **WriteLine** muestra la expresión especificada incluye un salto al principio de la línea siguiente (eso es lo que indica **Line**).

EXPRESIONES ARITMÉTICAS

Una expresión es un conjunto de operandos unidos mediante operadores para especificar una operación determinada. Todas las expresiones cuando se evalúan retornan un valor. Por ejemplo, la siguiente expresión retorna la suma de *dato1* y *dato2*:

```
dato1 + dato2
```

C# define cinco operadores aritméticos, que son los siguientes:

- + *Suma*. Los operandos pueden ser enteros o reales.
- *Resta*. Los operandos pueden ser enteros o reales.
- * *Multipliación*. Los operandos pueden ser enteros o reales.
- / *División*. Los operandos pueden ser enteros o reales. Si ambos operandos son enteros, el resultado es entero. En el resto de los casos, el resultado es real.
- % *Módulo* o resto de una división. Los operandos pueden ser enteros o reales. Si ambos operandos son enteros, el resto será entero; en otro caso, el resto será real.

Cuando en una operación aritmética los operandos son de diferentes tipos, ambos son convertidos al tipo del operando de precisión más alta. En una asignación, el resultado obtenido en una operación aritmética es convertido implícita o explícitamente al tipo de la variable que almacena dicho resultado (véase *Conversión entre tipos primitivos* en el capítulo 4).

Así mismo, cuando en una expresión intervienen varios operadores aritméticos se ejecutan de izquierda a derecha y de mayor a menor prioridad. Los operadores *, / y % tienen entre ellos la misma prioridad pero mayor que la de los operadores + y – que también tienen la misma prioridad entre ellos (véase *Prioridad y orden de evaluación* en el capítulo 4). Una expresión entre paréntesis, siempre se evalúa primero; si hay varios niveles de paréntesis son evaluados de más internos a más externos. Por ejemplo:

```
class CAritmetica
{
    /*
     * Operaciones aritméticas
     */

    public static void Main(string[] args)
    {
        double a = 10;
        float b = 20;
```



```

        int c = 2, r = 0;
        r = (int)(7.5 * System.Math.Sqrt(a) - b / c);
        System.Console.WriteLine(r);
    }
}

```

Ejecución del programa:

13

En este ejemplo, primero se realiza la operación *Math.Sqrt(a)* (invoca al método **Sqrt** de la clase **Math** para calcular la raíz cuadrada de *a*) y después, el resultado de tipo **double** que se obtiene se multiplica por 7,5. A continuación se realiza *b / c* convirtiendo previamente *c* al tipo de *b*; el resultado que se obtiene es de tipo **float**. Finalmente se hace la resta de los dos resultados anteriores convirtiendo previamente el resultado de tipo **float** a tipo **double**; se obtiene un resultado de tipo **double** que, como puede observar, es convertido explícitamente a tipo **int**, truncando la parte decimal, para poder almacenarlo en *r*, ya que la conversión **double** a **int** no es posible realizarla implícitamente.

EXPRESIONES CONDICIONALES

En ocasiones interesará dirigir el flujo de ejecución de un programa por un camino u otro en función del valor de una expresión. Para ello, C# proporciona la sentencia **if**. Para ver cómo se utiliza esta sentencia, vamos a realizar un programa que verifique si un número es par. En caso afirmativo imprimirá un mensaje “Número par” y a continuación el valor del número. En caso negativo sólo imprimirá el valor del número:

```

class CRelacion
{
    /*
     * Expresiones condicionales
     */
    public static void Main(string[] args)
    {
        int num = 24;

        if ( num % 2 == 0) // si el resto de la división es igual a 0,
            System.Console.WriteLine("Número par");

        System.Console.WriteLine("Valor: " + num);
    }
}

```

La sentencia **if** del ejemplo anterior se interpreta así: si la condición especificada entre paréntesis, `num % 2 == 0`, es cierta, se invoca a **WriteLine** y se escribe “Número par”; si es falsa, no se hace lo anterior. En cualquiera de los dos casos, se continúa con la siguiente sentencia (`System.Console.WriteLine("Valor: " + num)`). Según esto, el resultado será:

```
Número par
Valor: 24
```

Si el número hubiera sido 23, el resultado hubiese sido sólo `Valor: 23`. La expresión que hay entre paréntesis a continuación de **if** es una *expresión condicional* y el resultado de su evaluación siempre es un valor booleano **true** (verdadero) o **false** (falso); estas dos constantes están predefinidas en C#. Los operadores de relación o de comparación que podemos utilizar en estas expresiones son los siguientes:

<	¿Primer operando <i>menor que</i> el segundo?
>	¿Primer operando <i>mayor que</i> el segundo?
<=	¿Primer operando <i>menor o igual que</i> el segundo?
>=	¿Primer operando <i>mayor o igual que</i> el segundo?
!=	¿Primer operando <i>distinto que</i> el segundo?
==	¿Primer operando <i>igual que</i> el segundo?

Modifiquemos el programa anterior para que ahora indique si el número es par o impar. Para este caso emplearemos una segunda forma de la sentencia **if** que consiste en añadir a la anterior la cláusula **else** (si no):

```
class CRelacion
{
    /*
     * Expresiones condicionales
     */
    public static void Main(string[] args)
    {
        int num = 23;

        if ( num % 2 == 0 ) // si el resto de la división es igual a 0,
            System.Console.WriteLine("Número par");
        else                // si el resto de la división no es igual a 0,
            System.Console.WriteLine("Número impar");

        System.Console.WriteLine("Valor: " + num);
    }
}
```

Ejecución del programa:

*Número impar
Valor: 23*

La sentencia **if** de este otro ejemplo se interpreta así: si la condición especificada entre paréntesis, `num % 2 == 0`, es cierta, se invoca a **WriteLine** y se escribe “Número par” y si no, se invoca a **WriteLine** y se escribe “Número impar”. En cualquiera de los dos casos se continúa con la siguiente sentencia del programa.

A continuación se muestra otra versión del programa anterior que produciría exactamente los mismos resultados. No obstante, representa un estilo peor de programación, ya que repite código, lo que, como hemos visto, se puede evitar.

```
class CRelacion
{
    /*
     * Expresiones condicionales
     */
    public static void Main(string[] args)
    {
        int num = 23;

        if ( num % 2 == 0) // si el resto de la división es igual a 0,
        {
            System.Console.WriteLine("Número par");
            System.Console.WriteLine("Valor: " + num);
        }
        else // si el resto de la división no es igual a 0,
        {
            System.Console.WriteLine("Número impar");
            System.Console.WriteLine("Valor: " + num);
        }
    }
}
```

Se puede observar un nuevo detalle y es que cuando la cantidad de sentencias que se desean ejecutar en función del resultado **true** o **false** de una expresión es superior a una, hay que encerrarlas en un bloque. En el capítulo 7 veremos la sentencia **if** con más detalle.

ESCRIBIR NUESTROS PROPIOS MÉTODOS

De la misma forma que la biblioteca .NET proporciona métodos predefinidos como **WriteLine**, nosotros también podemos añadir a nuestro programa nuestros

propios métodos e invocarlos de la misma forma que lo hacemos con los predefinidos.

Por ejemplo, en el programa siguiente el método **Main** muestra la suma de dos valores cualesquiera; dicha suma la obtiene invocando a un método *sumar* añadido por nosotros que recibe en sus parámetros *x* e *y* los valores a sumar, realiza la suma de ambos *y*, utilizando la sentencia **return**, devuelve el resultado solicitado por **Main**.

```
public static double sumar(double x, double y)
{
    double resultado;
    // Realizar cálculos
    return resultado;
}
```

Han aparecido algunos conceptos nuevos (argumentos pasados a un método y valor retornado por un método). No se preocupe, sólo se trata de un primer contacto. Más adelante estudiaremos todo esto con mayor profundidad. Para una mejor comprensión de lo dicho, piense en el método o función llamado *logaritmo*, que seguro habrá utilizado más de una vez a lo largo de sus estudios. Este método devuelve un valor real correspondiente al logaritmo del valor pasado como argumento: $x = \log(y)$. Bueno, pues compárelo con el método *sumar* y comprobará que estamos hablando de cosas análogas.

Según lo expuesto y aplicando los conocimientos adquiridos hasta ahora, el programa propuesto puede ser como se muestra a continuación:

```
class CAritmetica
{
    /*
     * Método sumar:
     *   parámetros x e y de tipo double
     *   devuelve x + y
     */

    public static double sumar(double x, double y)
    {
        double resultado = 0;
        resultado = x + y;
        return resultado;
    }
}
```

```

public static void Main(string[] args)
{
    double a = 10, b = 20, r = 0;
    r = sumar(a, b);
    System.Console.WriteLine("Suma = " + r);
}
}

```

Observe cómo es la llamada al método *sumar*: $r = \text{sumar}(a, b)$. El método es invocado por su nombre, entre paréntesis se especifican los argumentos con los que debe operar, y el resultado que devuelve se almacena en *r*.

Finalmente, si comparamos el esqueleto del método *sumar* con el del método **Main**, observamos que son muy parecidos: *sumar* devuelve un valor de tipo **double** y **Main** nada (eso es lo que indica **void**) y *sumar* tiene dos parámetros, *x* e *y*, y **Main** sólo uno, *args*.

EJERCICIOS PROPUESTOS

1. Escriba una aplicación que visualice en el monitor los siguientes mensajes:

```

Bienvenido al mundo de C#.
Podrás dar solución a muchos problemas.

```

2. Decida qué tipos de valores necesita para escribir un programa que calcule la suma y la media de cuatro números de tipo **int**. Escriba un programa como ejemplo.
3. Escriba un programa que incluya un método denominado *calcular* que devuelva como resultado el valor de la expresión:

$$\frac{b^2 - 4ac}{2a}$$

El método **Main** invocará a *calcular* pasando los valores de $a = 1$, $b = 5$ y $c = 2$ y mostrará el resultado obtenido.

INTRODUCCIÓN A LA POO

La programación orientada a objetos (POO) es un modelo de programación que utiliza objetos, ligados mediante mensajes, para la solución de problemas. La idea central es simple: organizar los programas a imagen y semejanza de la organización de los objetos en el mundo real.

¿A qué objetos nos referimos? Si nos paramos a pensar en un determinado problema que intentamos resolver podremos identificar entidades de interés, las cuales pueden ser objetos potenciales que poseen un conjunto de *propiedades* o atributos, y un conjunto de *métodos* mediante los cuales muestran su comportamiento. Y no sólo eso, también podremos ver, a poco que nos fijemos, un conjunto de interrelaciones entre ellos conducidas por mensajes a los que responden mediante métodos.

Veamos un ejemplo. Considere una entidad bancaria. En ella identificamos entidades que son cuentas: cuenta del cliente 1, cuenta del cliente 2, etc. Pues bien, una cuenta puede verse como un objeto que tiene unos atributos, *nombre*, *número de cuenta* y *saldo*, y un conjunto de métodos como *IngresarDinero*, *RetirarDinero*, *AbonarIntereses*, *SaldoActual*, *Transferencia*, etc. En el caso de una transferencia:

```
cuenta01.Transferencia(cuenta02);
```

Transferencia sería el mensaje que el objeto *cuenta02* envía al objeto *cuenta01*, solicitando le sea hecha una transferencia, siendo la respuesta a tal mensaje la ejecución del método *Transferencia*. Trabajando a este nivel de abstracción, manipular una entidad bancaria resultará muy sencillo.

PENSAR EN OBJETOS

Según lo estudiado hasta ahora, usted podría pensar en un programa como si fuera una lista de instrucciones que le indican a la máquina qué hacer. En cambio, desde la POO un programa es un conjunto de objetos que dialogan entre sí para realizar las distintas tareas programadas. Para aclararlo, consideremos el ejemplo de la entidad bancaria mencionado anteriormente y pensemos en una concreta: XYZ. Podemos ver a esta entidad como a un objeto que tiene que comunicarse con otros muchos objetos (bolsa, otras entidades bancarias, empresas, etc.) para lograr sus fines: ganar dinero. A su vez, la entidad XYZ tendrá un montón de sucursales distribuidas por toda la geografía. Cada sucursal es otro objeto, de diferentes características que la entidad bancaria, que se comunicará con otras sucursales para satisfacer las peticiones de sus clientes. Pero, ¿y qué es un cliente? Pues otro objeto con sus propias características que se comunicará con otros objetos (sucursales, otros clientes, empresas, etc.) para realizar operaciones desde sus cuentas (transferencias, cargos, ingresos, etc.). Pero y las cuentas, ¿no son también objetos? Evidentemente. Vemos entonces que escribir un programa de gestión para el banco XYZ supondría crear objetos banco, sucursal, cliente, cuenta, etc., que deben comunicarse entre sí para poder responder a las operaciones solicitadas en cada momento.

Clases y objetos

Del ejemplo expuesto anteriormente podemos deducir que la POO se basa en la observación de que, en el mundo real, los objetos se construyen a partir de otros objetos. La combinación de estos objetos es un aspecto de dicha programación, pero también incluye mecanismos y características que hacen que la creación y el uso de objetos sea sencillo y flexible. Un mecanismo importantísimo es la *clase*, y el encapsulamiento y la herencia son dos propiedades o características poderosas.

¿Qué es una *clase* de objetos? Pongamos un ejemplo: piense en un molde para hacer flanes; el molde es la clase y los flanes, los objetos. Esto es, si disponemos de un molde de un litro para hacer flanes de vainilla (ingredientes: leche, vainilla, azúcar, etc.), el molde agrupa las propiedades comunes a todos los flanes de vainilla, pero no todos los flanes tienen por qué tener la misma cantidad de cada ingrediente. Esto es, una *clase* equivale a la generalización de un tipo específico de objetos, pero cada objeto que construyamos de esa clase tendrá sus propios datos.

Un *objeto* de una determinada clase se crea en el momento en que se invoca al operador **new** para dicha *clase*. Por ejemplo, la siguiente línea crea un objeto de la clase o tipo *CCuenta* y asigna a la variable *cuenta01* una referencia al mismo.

```
CCuenta cuenta01 = new CCuenta(); // nueva cuenta
```


Algunos autores emplean el término instancia (traducción directa de *instance*), en el sentido de que una instancia es la representación concreta y específica de una clase; por ejemplo, *cuenta01* es una instancia de la clase *CCuenta*. Desde este punto de vista, los términos instancia y objeto son lo mismo. En este libro se prefiere, no obstante, utilizar el término *objeto*, o bien *ejemplar*.

Cuando se escribe un programa utilizando un lenguaje orientado a objetos, no se definen objetos verdaderos, se definen clases de objetos, donde una clase se ve como una plantilla para múltiples objetos con características similares. Afortunadamente no tendrá que escribir todas las clases que necesite en su programa, porque .NET proporciona una biblioteca de clases estándar para realizar las operaciones más habituales que podamos requerir.

Mensajes y métodos

Un programa orientado a objetos se compone solamente de objetos. Cada uno de ellos es una entidad que tiene unas propiedades particulares, los *atributos*, y unas formas de operar sobre ellos, los *métodos*. Por ejemplo, una ventana de una aplicación Windows es un objeto. El color de fondo, la anchura, la altura, etc., son atributos. Las rutinas, lógicamente transparentes al usuario, que permiten maximizar la ventana, minimizarla, etc., son los métodos.

Cuando se ejecuta un programa orientado a objetos, los objetos están recibiendo, interpretando y respondiendo a *mensajes* de otros objetos. En la POO un *mensaje* está asociado con un *método*, de tal forma que cuando un objeto recibe un mensaje la respuesta a ese mensaje es ejecutar el método asociado. Por ejemplo, cuando un usuario quiere maximizar una ventana de una aplicación Windows, lo que hace simplemente es pulsar el botón de la misma que realiza esa acción. Eso provoca que Windows envíe un mensaje a la ventana para indicar que tiene que maximizarse. Como respuesta a este mensaje se ejecutará el método programado para ese fin.

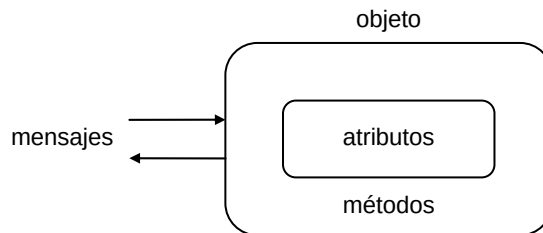
Un *método* se escribe en una *clase* de objetos y determina cómo tiene que actuar el objeto cuando recibe el *mensaje* vinculado con ese método. A su vez, un *método* puede también enviar *mensajes* a otros objetos solicitando una acción o información. En adición, los atributos definidos en la clase permitirán almacenar información para dicho objeto.

Según lo expuesto, podemos decir que la ejecución de un programa orientado a objetos realiza fundamentalmente tres cosas:

1. Crea los objetos necesarios.
2. Los mensajes enviados a unos y a otros objetos dan lugar a que se procese internamente la información.
3. Finalmente, cuando los objetos no son necesarios, son borrados.

DISEÑO DE UNA CLASE DE OBJETOS

Cuando escribimos un programa orientado a objetos, lo que hacemos es diseñar un conjunto de clases, desde las cuales se crearán los objetos necesarios cuando el programa se ejecute. Cada una de estas clases incluye dos partes fácilmente diferenciables: los *atributos* y los *métodos*. Los atributos definen el estado de cada uno de los objetos de esa clase y los métodos, su comportamiento.



Normalmente, los atributos, la estructura más interna del *objeto*, se ocultan a los usuarios del objeto, manteniendo como única conexión con el exterior, los *mensajes*. Esto quiere decir que los atributos de un objeto solamente podrán ser manipulados por los *métodos* del propio objeto. Este conjunto de métodos recibe el nombre de *interfaz*: medio de comunicación con un objeto.

Escribamos una clase de objetos. Siguiendo el ejemplo comentado al principio de este capítulo, podemos crear una clase de objetos *CCuenta* que represente una cuenta bancaria. Abra su entorno de desarrollo integrado favorito y escriba paso a paso el ejemplo que a continuación empezamos a desarrollar:

```
class CCuenta
{
    // Cuerpo de la clase: atributos y método
}
```

Observamos que para declarar una clase hay que utilizar la palabra reservada **class** seguida del nombre de la clase y del cuerpo de la misma. El cuerpo de la clase incluirá entre { y } sus miembros: atributos y métodos.

Atributos

Los atributos son las características individuales que diferencian un objeto de otro. El color de una ventana Windows la diferencia de otras; el DNI de una persona la identifica frente a otras; el número de una cuenta la distingue entre otras; etc. Pensando en la clase de objetos *CCuenta*, elegimos los atributos de interés que van a definir esta clase de objetos:

- ◇ *nombre*: nombre del cliente del banco al que pertenece la cuenta.
- ◇ *cuenta*: número de la cuenta.
- ◇ *saldo*: saldo actual de la cuenta.
- ◇ *tipoDeInterés*: tipo de interés en tanto por cien.

Todos los atributos son definidos en la clase por variables:

```
class CCuenta
{
    private string nombre;
    private string cuenta;
    private double saldo;
    private double tipoDeInterés;

    // ...
}
```

Observe que se han definido cuatro atributos: dos de ellos, *nombre* y *cuenta*, pueden contener una cadena de caracteres (una cadena de caracteres es un objeto de la clase **String** perteneciente a la biblioteca .NET). Los otros dos atributos, *saldo* y *tipoDeInterés*, son de tipo **double**. Recuerde que, en lo sucesivo, cuando utilice estos identificadores, debe respetar las mayúsculas y las minúsculas.

Anteriormente dijimos que, generalmente, los atributos de un objeto de una clase se ocultan a los usuarios del mismo. ¿Qué quiere decir esto? Que un usuario que utilice la clase *CCuenta* en su programa no podrá escribir su código basado directamente en estos atributos, sino que tendrá que acceder a ellos a través de los métodos que implemente la clase, como veremos a continuación; de esta forma, un usuario de la clase *CCuenta* no podrá asignar cualquier valor a los atributos de la misma. Esta protección es la que se consigue justamente con el modificador **private** (generalmente se utilizan los modificadores **private** o **public**). Un miembro declarado privado (**private**) es accesible solamente por los métodos de su propia clase. Esto significa que no se puede acceder a él por los métodos de cualquier otra clase, incluidas las subclases.

Propiedades y métodos

El comportamiento de un objeto queda definido por las acciones que puede emprender. Por ejemplo, pensando acerca de un objeto de la clase *CCuenta*, esto es, de una cuenta de un cliente de un determinado banco, algunas acciones que se pueden realizar sobre ella son:

- ◇ Asignar el nombre de un cliente del banco a una cuenta.
- ◇ Obtener el nombre del cliente de una cuenta.
- ◇ Asignar el número de la cuenta.
- ◇ Obtener el número de la cuenta.
- ◇ Realizar un ingreso.
- ◇ Realizar un reintegro.
- ◇ Asignar el tipo de interés.
- ◇ Obtener el tipo de interés.
- ◇ Obtener el saldo, esto es, el estado de la cuenta.

Para definir este comportamiento se pueden utilizar tanto métodos como propiedades. En general, los métodos representan acciones que un objeto puede realizar, mientras que las propiedades representan información sobre un objeto. Según esto, nombre, número de la cuenta, tipo de interés y saldo podrían considerarse propiedades, e ingreso y reintegro acciones. No obstante, desde el punto de vista de compatibilidad con otros lenguajes (por ejemplo, con C++) todas las acciones enumeradas anteriormente podrían implementarse como métodos.

Los métodos son rutinas de código definidas dentro del cuerpo de la clase, que se ejecutan en respuesta a alguna acción tomada desde dentro de un objeto de esa clase, o bien desde otro objeto de la misma o de otra clase. Recuerde que los objetos se comunican mediante mensajes. El conjunto de mensajes a los que un objeto puede responder se corresponde con el conjunto de métodos que implementa su clase.

Como ejemplo, vamos a agregar a la clase *CCuenta* un método que responda a la acción de asignar el nombre de un cliente del banco a una cuenta:

```
public void asignarNombre(string nom)
{
    if (nom.Length == 0)
    {
        System.Console.WriteLine("Error: cadena vacía");
        return;
    }
    nombre = nom;
}
```

Observe que el método ha sido declarado público (**public**). Un miembro declarado público está accesible para cualquier otra clase o subclase que necesite utilizarlo. La interfaz pública de una clase, o simplemente interfaz, está formada por todos los miembros públicos de la misma.

Como se puede observar, un método consta de su nombre precedido por el tipo del valor que devuelve cuando finalice su ejecución (la palabra reservada **void** indica que el método no devuelve ningún valor) y seguido por una lista de parámetros separados por comas y encerrados entre paréntesis (en el ejemplo, hay un parámetro *nom*). Los paréntesis indican a C# que el identificador, *asignarNombre*, se refiere a un método y no a un atributo. A continuación se escribe el cuerpo del método encerrado entre { y }.

El método *asignarNombre* asegura que el nombre a asignar no sea una cadena vacía (la propiedad **Length** de la clase **String** contiene el número de caracteres que hay almacenados en el objeto **String** que recibe ese mensaje); si el nombre fuera una cadena vacía, simplemente visualizaría un mensaje que lo indicase; en otro caso, asigna la cadena *nom* pasada como argumento al atributo *nombre* del objeto que reciba el mensaje “asignar nombre”.

Cuando decimos que un objeto recibe un mensaje, debemos entender que el mensaje es un concepto que subyace en nuestra mente; la acción real es invocar al método que responde a ese mensaje con el fin de modificar el estado del objeto. Según esto, podemos decir que los nombres de los métodos de una clase forman el conjunto de mensajes a los que un objeto de esa clase puede responder.

Agreguemos dos métodos más para afianzar lo explicado hasta ahora. Uno que permita obtener el nombre de un cliente del banco y otro que permita conocer el saldo de su cuenta:

```
public string obtenerNombre()
{
    return nombre;
}

public double estado()
{
    return saldo;
}
```

Las propiedades tienen un aspecto análogo a un método. Utilizan descriptores de acceso para controlar cómo se establecen y devuelven valores de los atributos a los que se refieren. Estos descriptores de acceso son rutinas de código declaradas dentro de la propiedad para recuperar (**get**) o establecer (**set**) el valor de la misma. Su sintaxis es la siguiente:

```
public tipo nombre_propiedad
{
    get
    {
        // Aquí se devuelve "return" el valor del atributo
    }

    set
    {
        // Aquí se asigna el valor "value" al atributo
    }
}
```

El cuerpo del descriptor de acceso **get** debe devolver el valor del tipo de la propiedad. La ejecución del descriptor de acceso **get** equivale a leer el valor del atributo al que se refiere la propiedad. El descriptor de acceso **set** utiliza un parámetro implícito denominado **value**, que tiene el mismo tipo que la propiedad.

Como ejemplo, vamos a implementar las propiedades que podrían sustituir a los métodos que escribimos anteriormente para acceder a los atributos *nombre* y *saldo*:

```
public string Nombre
{
    get
    {
        return nombre;
    }
    set
    {
        if (value.Length == 0)
        {
            System.Console.WriteLine("Error: cadena vacía");
            return;
        }
        nombre = value;
    }
}
```

Observe que la propiedad ha sido declarada pública (**public**). La explicación es la misma que hemos dado para los métodos. Fíjese también que a través de **set** aseguramos que el nombre a asignar no sea una cadena vacía; si el nombre fuera una cadena vacía, simplemente visualizará un mensaje indicándolo; en otro caso, asignará la cadena **value** al atributo *nombre* del objeto. A través de **get** simplemente devolvemos el valor del atributo.

Normalmente los descriptores de acceso de una propiedad se definen por parejas, utilizando **get** y **set**. Ahora bien, si la propiedad es de sólo lectura (sólo **get**) o de sólo escritura (sólo **set**) puede definirse cada descriptor de acceso de forma individual. Por ejemplo, parece lógico que la propiedad que permita obtener el saldo de una cuenta, sea de sólo lectura:

```
public double Saldo
{
    get { return saldo; }
}
```

Cuando terminemos de escribir todos los métodos y propiedades previstos, tendremos creada la clase *CCuenta* que guardaremos en un fichero denominado *CCuenta.cs*.

```
class CCuenta
{
    // Atributos
    private string nombre;
    private string cuenta;
    private double saldo;
    private double tipoDeInterés;

    // Propiedades y métodos
    public string Nombre
    {
        get
        {
            return nombre;
        }
        set
        {
            if (value.Length == 0)
            {
                System.Console.WriteLine("Error: cadena vacía");
                return;
            }
            nombre = value;
        }
    }

    public string Cuenta
    {
        get
        {
            return cuenta;
        }
        set
        {

```

```
        if (value.Length == 0)
        {
            System.Console.WriteLine("Error: cuenta no válida");
            return;
        }
        cuenta = value;
    }
}

public double Saldo
{
    Get { return saldo; }
}

public void ingreso(double cantidad)
{
    if (cantidad < 0)
    {
        System.Console.WriteLine("Error: cantidad negativa");
        return;
    }
    saldo = saldo + cantidad;
}

public void reintegro(double cantidad)
{
    if (saldo - cantidad < 0)
    {
        System.Console.WriteLine("Error: no dispone de saldo");
        return;
    }
    saldo = saldo - cantidad;
}

public double TipoDeInterés
{
    get
    {
        return tipoDeInterés;
    }
    set
    {
        if (value < 0)
        {
            System.Console.WriteLine("Error: tipo no válido");
            return;
        }
        tipoDeInterés = value;
    }
}
}
```


Para poder crear objetos de esta clase y trabajar con ellos, tendremos que escribir un programa. Según lo estudiado en los capítulos anteriores, en un programa tiene que haber una clase con un método **Main**, puesto que éste es el punto de entrada y de salida del programa. Este requerimiento se puede satisfacer de tres formas. Vamos a comentarlas sobre el ejemplo que estamos desarrollando:

1. Añadir a la clase *CCuenta* un método **Main** declarado **static** que incluya el código del programa (crear objetos *CCuenta* y realizar operaciones con ellos).
2. Añadir en el mismo fichero fuente en el que está almacenada la clase *CCuenta* otra clase, por ejemplo *Test*, que incluya el método **Main**.
3. Tenemos un fichero fuente, *CCuenta.cs*, con la clase *CCuenta*. Añadir en la misma carpeta (directorio) otro fichero fuente, *Test.cs*, con la clase principal (la que contiene el método **Main**), que vamos a denominar *Test*.

Vamos a continuar el ejemplo aplicando el punto tercero, porque es lo más práctico y lo que más se ajusta a lo que hemos denominado programación orientada a objetos, ya que de esta forma cada fichero fuente se corresponde con una clase de objetos. Por lo tanto, abra un nuevo fichero fuente *Test.cs* y añada al mismo la clase *Test* que se muestra a continuación:

```
class Test
{
    public static void Main(string[] args)
    {
        CCuenta cuenta01 = new CCuenta();
        cuenta01.Nombre = "Un nombre";
        cuenta01.Cuenta = "Una cuenta";
        cuenta01.TipoDeInterés = 2.5;

        cuenta01.ingreso(12000);
        cuenta01.reintegro(3000);

        System.Console.WriteLine(cuenta01.Nombre);
        System.Console.WriteLine(cuenta01.Cuenta);
        System.Console.WriteLine(cuenta01.Saldo);
        System.Console.WriteLine(cuenta01.TipoDeInterés);
    }
}
```

El método **Main** tiene que ser estático y generalmente se declara público aunque no es necesario, no devuelve un resultado y tiene un parámetro *args* que es una matriz de una dimensión de cadenas de caracteres (en los capítulos siguientes aprenderá para qué sirve). Analicemos el método **Main** del ejemplo anterior para que tenga una idea clara de lo que hace:

- La primera línea crea un objeto de la clase *CCuenta* y almacena una referencia al mismo en la variable *cuenta01*. Esta variable la utilizaremos para acceder a ese objeto en las líneas siguientes. Ahora quizás empiece a entender por qué anteriormente decíamos que un programa orientado a objetos se compone solamente de objetos.
- Las cinco líneas siguientes establecen un determinado estado para el objeto referenciado por *cuenta01*, enviándole los mensajes *Nombre*, *Cuenta*, etc., para asignar los valores especificados. Se puede observar que para acceder a un miembro del objeto se utiliza el operador punto (*.*).
- En las cuatro últimas líneas el objeto recibe los mensajes *Nombre*, *Cuenta*, *Saldo* y *TipoDeInterés* para obtener los valores correspondientes. La respuesta a estos mensajes es, como ya sabe, la ejecución de los métodos respectivos, que, en este caso, devolverán la información que se mostrará mediante el método **WriteLine**.

En general, para acceder a un miembro de un objeto (atributo, propiedad o método), se utiliza la sintaxis siguiente:

nombre_objeto.nombre_miembro

De esta forma quedan eliminadas las ambigüedades que surgirían si hubiéramos creado más de un objeto. Esto es, supongamos que se hubieran creado dos objetos *CCuenta*: *cuenta01* y *cuenta02* y que para asignar el nombre a uno de ellos se hubiera utilizado la sintaxis *Nombre*=“*Un nombre*”; en este caso surgiría la pregunta: la propiedad *Nombre* a qué objeto corresponde.

Una vez creadas las dos clases que forman nuestro programa, podemos comprobar que tenemos dos ficheros fuente: *CCuenta.cs* y *Test.cs*. ¿Cómo compilamos este programa? Pues, desde la línea de órdenes, utilizando la orden *csc Test.cs CCuenta.cs* (si está utilizando *MS Visual C#*, vea el apéndice A). Esto hará que se compile el fichero *Test.cs* y el fichero *CCuenta.cs*, almacenando el resultado en *Test.exe* (como nombre del fichero ejecutable se utiliza, por omisión de la opción */out:fichero.exe*, el nombre del primer fichero especificado después de *csc*). En nuestro caso el método **Main** de *Test* tiene una línea de código que crea un objeto de la clase *CCuenta*, por lo tanto, durante el proceso de compilación se buscará en la carpeta actual un fichero con la clase requerida; esto es, *CCuenta*. Si no se encuentra, se producirá un error de compilación que lo indicará.

Para ejecutar el programa que acabamos de compilar, escriba la orden *Test*, puesto que *Test* es el fichero que contiene el código ejecutable. Observará los siguientes resultados:

```
Un nombre
Una cuenta
9000
2,5
```

Para finalizar, algunas notas que no debe olvidar:

- Cualquier método de una clase tiene acceso (puede invocar) a todos los otros miembros (atributos, propiedades y métodos) de su clase.
- Un objeto de una clase sólo puede invocar a métodos de su clase; dicho de otra forma, sólo puede responder a los mensajes para los que ha sido programado.

CONSTRUCTORES

Un *constructor* es un método especial de una clase que es llamado automáticamente siempre que se crea un objeto de esa clase. Su función es iniciar el objeto.

Un *constructor* se distingue fácilmente porque tiene el mismo nombre que la clase a la que pertenece y no puede retornar un valor (ni siquiera se puede especificar la palabra reservada **void**). Por ejemplo, si añadiéramos a la clase *CCuenta* un constructor, tendríamos que llamarlo también *CCuenta*. Ahora bien, cuando en una clase no escribimos explícitamente un constructor, C# asume uno por omisión. Por ejemplo, la clase *CCuenta* que hemos escrito anteriormente tiene por omisión un constructor definido así:

```
public CCuenta() {}
```

Un *constructor por omisión* de una clase *C* es un constructor sin parámetros que no hace nada. Sin embargo, es necesario porque, según lo que acabamos de exponer, será invocado cada vez que se construya un objeto sin especificar ningún argumento, en cuyo caso el objeto será iniciado con los valores predeterminados por el sistema (los atributos numéricos a ceros, los alfanuméricos a nulos, y las referencias a objetos a **null**).

Si usted quiere comprobar que un *constructor* es un método especial de una clase que es llamado automáticamente cada vez que se crea un objeto de esa clase, añada el siguiente método a la clase *CCuenta* del programa anterior y podrá verificar que cuando **Main** crea *cuenta01* se visualiza el mensaje “Objeto CCuenta creado”, señal inequívoca de que el constructor ha sido invocado.

```
public CCuenta() { System.Console.WriteLine("Objeto CCuenta creado"); }
```

Como ejemplo, vamos a añadir un constructor a la clase *CCuenta* con el fin de poder iniciar los atributos de cada nuevo objeto con unos valores determinados pasados como argumentos en el instante en el que se solicita crearlo:

```
class CCuenta
{
    // Atributos
    private string nombre;
    private string cuenta;
    private double saldo;
    private double tipoDeInterés;

    // Propiedades y métodos
    public CCuenta() {} // constructor sin parámetros

    public CCuenta(string nom, string cue, double sal, double tipo)
    {
        Nombre = nom;
        Cuenta = cue;
        ingreso(sal);
        TipoDeInterés = tipo;
    }

    // ...
}
```

Siempre que en una clase se define explícitamente un constructor, el constructor implícito (constructor por omisión) es reemplazado por éste. Por eso hemos tenido que definirlo también explícitamente, de lo contrario, intentar crear un objeto sin especificar parámetros daría lugar a un error.

Observe que los constructores, salvo en casos excepcionales, deben declararse siempre públicos para que puedan ser invocados desde cualquier parte.

Una línea como la siguiente invocará al constructor sin parámetros:

```
CCuenta cuenta01 = new CCuenta(); // invoca al constructor CCuenta
```

El operador **new** crea un nuevo objeto, en este caso de la clase *CCuenta*, y a continuación se invoca al constructor de su clase para realizar las operaciones de iniciación que estén programadas. Y una línea como la siguiente invocará al constructor con cuatro parámetros:

```
CCuenta cuenta02 = new CCuenta("Un nombre", "Una cuenta",
                                6000, 3.5);
```

Puede probar lo expuesto hasta ahora modificando la clase *Test* como se muestra a continuación. Puede también realizar una segunda prueba eliminando el constructor sin parámetros de la clase *CCuenta* y podrá comprobar que el compilador le muestra un error en la línea que invoca al constructor sin parámetros.

```
class Test
{
    public static void Main(string[] args)
    {
        CCuenta cuenta01 = new CCuenta();
        CCuenta cuenta02 = new CCuenta("Un nombre", "Una cuenta",
                                         6000, 3.5);

        cuenta01.Nombre = "Un nombre";
        cuenta01.Cuenta = "Una cuenta";
        cuenta01.TipoDeInterés = 2.5;

        cuenta01.ingreso(12000);
        cuenta01.reintegro(3000);

        System.Console.WriteLine(cuenta01.Nombre);
        System.Console.WriteLine(cuenta01.Cuenta);
        System.Console.WriteLine(cuenta01.Saldo);
        System.Console.WriteLine(cuenta01.TipoDeInterés);
        System.Console.WriteLine();
        System.Console.WriteLine(cuenta02.Nombre);
        System.Console.WriteLine(cuenta02.Cuenta);
        System.Console.WriteLine(cuenta02.Saldo);
        System.Console.WriteLine(cuenta02.TipoDeInterés);
    }
}
```

Quizás le haya llamado la atención que ahora en la clase *CCuenta* hay un mismo método definido dos veces, nos referimos al constructor *CCuenta*. Pues bien, cuando en una clase un mismo método se define varias veces con distinto número de parámetros, o bien con el mismo número de parámetros pero diferenciándose una definición de otra en que al menos un parámetro es de un tipo diferente, se dice que el método está *sobrecargado*.

Los métodos sobrecargados pueden diferir también en el tipo del valor retornado. Ahora bien, el compilador C# no admite que se declaren dos métodos que sólo difieran en el tipo del valor retornado; deben diferir también en la lista de parámetros; esto es, lo que importa son el número y el tipo de los parámetros.

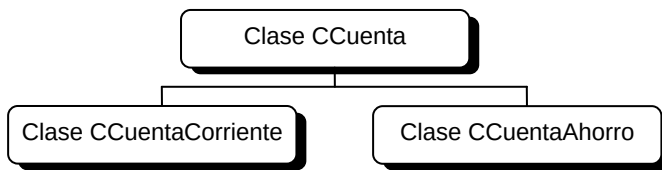
Cuando una clase sobrecarga un método una o más veces, ¿cómo sabe C# cuál tiene que ejecutar cuando se invoque? Pues esto lo hace comparando el número y los tipos de los argumentos especificados en la llamada con los paráme-

tros especificados en las distintas definiciones del método. Por eso, las dos llamadas que el método **Main** de *Test* realiza al método *CCuentas* no presentan ninguna ambigüedad respecto a la definición que se debe ejecutar de este método.

HERENCIA

La herencia es una de las características más importantes en la POO porque permite que una clase herede los atributos y métodos de otra clase (los constructores no se heredan). Esta característica garantiza la reutilización del código.

Con la herencia todas las clases están clasificadas en una jerarquía estricta. Cada clase tiene su superclase (la clase superior en la jerarquía, también llamada clase base), y cada clase puede tener una o más subclases (las clases inferiores en la jerarquía; también llamadas clases derivadas).



Las clases que están en la parte inferior en la jerarquía se dice que *heredan* de las clases que están en la parte superior en la jerarquía.

El término heredar significa que las subclases disponen de todos los atributos, propiedades y métodos de su superclase. Este mecanismo proporciona una forma rápida y cómoda de extender la funcionalidad de una clase. En C# cada clase sólo puede tener una superclase (o clase base), lo que se denomina *herencia simple*.

La clase **Object** es la clase raíz de la jerarquía de clases de la biblioteca .NET; pertenece al espacio de nombres **System**. Por lo tanto, cualquier clase que implementemos en nuestras aplicaciones pasará a ser automáticamente una subclase de esta clase.

Como ejemplo vamos a añadir al programa anterior una nueva clase denominada *CCuentaAhorro* que sea subclase de *CCuenta*. Para ello, edite un nuevo fichero *CCuentaAhorro.cs* y escriba en él el código que se muestra a continuación:

```
class CCuentaAhorro : CCuenta {}
```

La línea anterior define la subclase *CCuentaAhorro* de *CCuenta*. Para indicar tal hecho se utiliza el carácter : (dos puntos) que indica que *CCuentaAhorro* se deriva de *CCuenta*. En el ejemplo propuesto, el cuerpo de *CCuentaAhorro* está

vacío, pero aún así, modificando la clase *Test* como se muestra a continuación, el programa sobre el que venimos trabajando funcionará:

```
class Test
{
    public static void Main(string[] args)
    {
        CCuentaAhorro cuenta01 = new CCuentaAhorro();

        cuenta01.Nombre = "Un nombre";
        cuenta01.Cuenta = "Una cuenta";
        cuenta01.TipoDeInterés = 2.5;

        cuenta01.ingreso(12000);
        cuenta01.reintegro(3000);

        System.Console.WriteLine(cuenta01.Nombre);
        System.Console.WriteLine(cuenta01.Cuenta);
        System.Console.WriteLine(cuenta01.Saldo);
        System.Console.WriteLine(cuenta01.TipoDeInterés);
    }
}
```

Ahora tiene un programa formado por tres clases almacenadas en otros tantos ficheros: *CCuenta.cs*, *CCuentaAhorro.cs* y *Test.cs*. Por lo tanto, para compilar este programa desde la línea de órdenes utilice la orden: *csc Test.cs CCuentaAhorro.cs CCuenta.cs*. A continuación, cuando lo ejecute, el resultado será el mismo que antes:

```
Un nombre
Una cuenta
9000
2,5
```

Echemos una ojeada al método **Main** de la clase *Test*. Observamos que crea un objeto *cuenta01* de la clase *CCuentaAhorro* y después, utilizando los métodos de su superclase, inicia los atributos del objeto y finalmente muestra su estado. ¿Cómo es esto posible si un objeto sólo puede invocar a métodos de su clase? Pues es posible porque *CCuentaAhorro* ha heredado todos los atributos, propiedades y métodos de *CCuenta*, excepto los constructores.

Por lo tanto, si queremos construir objetos *CCuentaAhorro* iniciados con unos determinados valores, tendremos que añadir a esta clase un constructor con los parámetros necesarios. Si además queremos ampliar la funcionalidad de la clase con nuevos atributos y métodos, también podemos hacerlo. Por ejemplo, vamos a añadir a la funcionalidad que ya tiene *CCuentaAhorro* (los atributos, propiedades y métodos heredados de su clase padre), además de un constructor sin argumentos

y otro con ellos, un nuevo atributo *cuotaMantenimiento* y la propiedad *CuotaMantenimiento* para manipularlo:

```
class CCuentaAhorro : CCuenta
{
    // Atributos
    private double cuotaMantenimiento;

    // Propiedades y métodos
    public CCuentaAhorro() {} // constructor sin parámetros

    public CCuentaAhorro(string nom, string cue, double sal,
        double tipo, double mant) :
        base(nom, cue, sal, tipo) // invoca al constructor CCuenta,
    { // esto es, al de la clase base
        CuotaMantenimiento = mant; // inicia cuotaMantenimiento
    }

    public double CuotaMantenimiento
    {
        get
        {
            return cuotaMantenimiento;
        }
        set
        {
            if (value < 0)
            {
                System.Console.WriteLine("Error: cantidad negativa");
                return;
            }
            cuotaMantenimiento = value;
        }
    }
}
```

Así mismo, una subclase puede redefinir cualquier método heredado de su clase padre, siempre que sea necesario que su comportamiento en la subclase sea diferente. Redefinir un método heredado significa volverlo a escribir en la subclase con el mismo nombre, la misma lista de parámetros y el mismo tipo del valor retornado que tenía en la superclase (incluiríamos la palabra **new** para especificar que esta nueva versión reemplaza a la heredada); su cuerpo será adaptado a las necesidades de la subclase. Por ejemplo, supongamos que queremos obligar a disponer de un saldo positivo superior a 1500 para las cuentas de ahorro con un interés igual o mayor de 3,5; esto supone redefinir el método *reintegro* así:

```
class CCuentaAhorro : CCuenta
{
```



```
// ...
public new void reintegro(double cantidad)
{
    double saldo = this.Saldo;
    double tipoDeInterés = this.TipoDeInterés;

    if ( tipoDeInterés >= 3.5)
    {
        if (saldo - cantidad < 1500)
        {
            System.Console.WriteLine("Error: no dispone de esa cantidad");
            return;
        }
    }
    base.reintegro(cantidad); // método reintegro de la clase base,
                             // también llamada superclase
}
```

Una vez escrita la clase *CCuentaAhorro*, pensemos cómo será la estructura de un objeto de esta clase comparada con uno de la clase *CCuenta*. La capacidad de la clase *CCuenta* está soportada por los miembros:

Atributos	Propiedades/Métodos
nombre	constructores <i>CCuenta</i>
cuenta	Nombre
saldo	Cuenta
tipoDeInterés	Saldo
	ingreso
	reintegro
	TipoDeInterés

Y la capacidad de la clase *CCuentaAhorro*, derivada de *CCuenta*, está soportada por los miembros heredados de *CCuenta* (en cursiva y no tachados) más los suyos:

Atributos	Propiedades/Métodos
<i>nombre</i>	constructores <i>CCuenta</i>
<i>cuenta</i>	<i>Nombre</i>
<i>saldo</i>	<i>Cuenta</i>
<i>tipoDeInterés</i>	<i>Saldo</i>
	<i>ingreso</i>
	reintegro
	<i>TipoDeInterés</i>
cuotaMantenimiento	constructores <i>CCuentaAhorro</i>

CuotaMantenimiento reintegro

Observe que los constructores de la clase *CCuenta* no se heredan, puesto que cada clase define el suyo por omisión, y que el método *reintegro* queda oculto por el método del mismo nombre de la clase *CCuentaAhorro*. Observe también que es posible referirse a un miembro oculto utilizando la sintaxis:

base.miembro_oculto

Según el análisis anterior, mientras un posible objeto *CCuenta* contendría los datos *nombre*, *cuenta*, *saldo* y *tipoDeInterés*, un objeto *CCuentaAhorro* contiene los datos *nombre*, *cuenta*, *saldo*, *tipoDeInterés* y *cuotaMantenimiento*. Entonces, ¿cómo se inicia un objeto de la subclase *CCuentaAhorro*? Observando el constructor *CCuentaAhorro* con parámetros vemos que tiene cinco: los cuatro primeros proveen valores para los atributos heredados de *CCuenta* y el quinto, para el nuevo atributo añadido. Para iniciar los atributos heredados de la clase base lo más sencillo es invocar al constructor de la misma; esto se hace mediante la palabra reservada **base** seguida de los argumentos correspondientes encerrados entre paréntesis; esta llamada se escribe a continuación de la cabecera del constructor de la subclase separada por dos puntos, como se observa a continuación:

```
public CCuentaAhorro(string nom, string cue, double sal,
    double tipo, double mant) :
    base(nom, cue, sal, tipo) // invoca al constructor CCuenta
{
    CuotaMantenimiento = mant; // inicia cuotaMantenimiento
}
```

Otra solución podría ser la presentada a continuación:

```
public CCuentaAhorro(string nom, string cue, double sal,
    double tipo, double mant)
{
    Nombre = nom;
    Cuenta = cue;
    ingreso(sal);
    TipoDeInterés = tipo;
    CuotaMantenimineto = mant; // inicia cuotaMantenimiento
}
```

A modo de resumen, los siguientes puntos exponen las reglas a tener en cuenta cuando se define una subclase:

1. Una subclase hereda todos los miembros de su superclase, excepto los constructores, lo que no significa que tenga acceso directo a todos los miembros.

Una consecuencia inmediata de esto es que la estructura interna de datos de un objeto de una subclase estará formada por los atributos que ella define y por los heredados de su superclase.

Una subclase no tiene acceso directo a los miembros privados (**private**) de su superclase.

Una subclase sí puede acceder directamente a los miembros públicos (**public**) de su superclase.

2. Una subclase puede añadir sus propios atributos y métodos. Si el nombre de alguno de estos miembros coincide con el de un miembro heredado, éste último queda oculto para la subclase, que se traduce en que la subclase ya no puede acceder directamente a ese miembro. Lógicamente, lo expuesto tiene sentido siempre que nos refiramos a los miembros de la superclase a los que la subclase podía acceder.
3. Los miembros heredados por una subclase pueden, a su vez, ser heredados por más subclases de ella. A esto se le llama propagación de herencia.

Según lo expuesto, la siguiente versión del constructor *CCuentaAhorro* sería errónea porque una subclase no tiene acceso directo a los miembros privados de su superclase; en nuestro caso, según vimos anteriormente, el acceso tiene que hacerse a través de la interfaz pública de *CCuenta*.

```
public CCuentaAhorro(string nom, string cue, double sal,
                    double tipo, double mant)
{
    nombre = nom;           // error: nombre es privado en CCuenta
    cuenta = cue;           // error: cuenta es privado en CCuenta
    saldo = sal;            // error: saldo es privado en CCuenta
    tipoDeInterés = tipo;   // error: tipoDeInterés es privado en CCuenta
    cuotaMantenimiento = mant;
}
```

Para probar los cambios introducidos hasta ahora, modifique la clase *Test* como se indica a continuación, compile de nuevo el programa y ejecútelo.

```
class Test
{
    public static void Main(string[] args)
    {
        CCuentaAhorro cuenta01 = new CCuentaAhorro(
            "Un nombre", "Una cuenta", 6000, 3.5, 0.60);
        // Cobrar cuota de mantenimiento
        cuenta01.reintegro(cuenta01.obtenerCuotaManten());
    }
}
```

```
// Ingreso
cuenta01.ingreso(6000);
// Reintegro
cuenta01.reintegro(10000);
// ...
System.Console.WriteLine(cuenta01.Nombre);
System.Console.WriteLine(cuenta01.Cuenta);
System.Console.WriteLine(cuenta01.Saldo);
System.Console.WriteLine(cuenta01.TipoDeInterés);
}
}
```

Quizás se pregunte: cómo sabe un método de una clase, por ejemplo *ingreso*, sobre qué objeto está trabajando si en el cuerpo del mismo no se indica nada de forma explícita, como se ve en el código siguiente:

```
public void ingreso(double cantidad)
{
    if (cantidad < 0)
    {
        System.Console.WriteLine("Error: cantidad negativa");
        return;
    }
    saldo = saldo + cantidad;
}
```

La respuesta a la pregunta anterior es que C# utiliza de forma implícita la palabra reservada **this** para almacenar en todo instante una referencia al objeto que invoca al método; según esto, la versión siguiente de *ingreso* es equivalente a la anterior:

```
public void ingreso(double cantidad)
{
    if (cantidad < 0)
    {
        System.Console.WriteLine("Error: cantidad negativa");
        return;
    }
    this.saldo = this.saldo + cantidad; // this referencia al objeto
                                        // que invoca al método
}
```

Para finalizar, habrá comprobado que el mecanismo de herencia proporciona una forma rápida y cómoda de modificar, en la dirección que deseemos, la funcionalidad de una clase. En el ejemplo expuesto, disponíamos de una clase *CCuenta* y reutilizando su definición hemos diseñado una nueva clase *CCuenta-Ahorro* adaptada a unas necesidades particulares. Evidentemente, si hubiéramos tenido que partir de cero, el trabajo y el tiempo de desarrollo hubiera sido mayor.

Esta misma forma de proceder puede emplearse con cualquier biblioteca de clases.

EJERCICIOS RESUELTOS

1. Para practicar un poco más, escriba el siguiente ejemplo y pruebe los resultados. Hágalo primero desde la línea de órdenes y después con el entorno de desarrollo integrado preferido por usted. El siguiente ejemplo muestra una clase *COrdenador* para simular el trabajo con ordenadores.

La clase *COrdenador* puede incluir los siguientes atributos:

- ◇ Marca: Dell, Toshiba, Asus.
- ◇ Procesador: Intel, AMD.
- ◇ Peso: 2, 2.5, 3.

Los atributos también pueden incluir información sobre el estado del objeto; por ejemplo, en el caso de un ordenador, si está encendido o apagado, si la presentación en pantalla está activa o inactiva, etc.

- ◇ ¿encendido?
- ◇ ¿pantalla activa?

```
class COrdenador
{
    private string marca;
    private string procesador;
    private int peso;
    private bool encendido = false;
    private bool pantalla = false;
    // ...
}
```

El comportamiento define las acciones que el objeto puede emprender. Por ejemplo, pensando acerca de un objeto de la clase *COrdenador*, esto es, de un ordenador, algunas acciones que éste puede hacer son:

- ◇ Ponerse en marcha.
- ◇ Apagarse.
- ◇ Desactivar la presentación en la pantalla.
- ◇ Activar la presentación en la pantalla.
- ◇ Cargar una aplicación.

Para definir este comportamiento hay que crear métodos. Como ejemplo, vamos a agregar a la clase *COrdenador* un método que responda a la acción de ponerlo en marcha:

```
public void EncenderOrdenador()
{
    if (encendido == true)
    {
        System.Console.WriteLine("El ordenador ya está encendido");
    }
    else
    {
        encendido = true;
        pantalla = true;
        System.Console.WriteLine("El ordenador ha sido encendido");
    }
}
```

El método *EncenderOrdenador* comprueba si el ordenador está encendido; si lo está, simplemente visualiza un mensaje que lo indica; si no lo está, se enciende y lo comunica mediante un mensaje.

Agreguemos un método más para que el objeto nos muestre su estado:

```
public void Estado()
{
    System.Console.Write("\nEl estado del ordenador es el siguiente:");
    System.Console.Write("\nMarca: " + marca);
    System.Console.Write("\nProcesador: " + procesador);
    System.Console.Write("\nPeso: " + peso + " kg.");

    if (encendido == true)
    {
        System.Console.Write("\nEl ordenador está encendido");
    }
    else
        System.Console.Write("\nEl ordenador está apagado");

    if (pantalla == true)
    {
        System.Console.Write("\nLa pantalla está activada");
    }
    else
        System.Console.Write("\nLa pantalla está desactivada");

    System.Console.WriteLine("\n");
}
```

El método *Estado* visualiza los atributos específicos de un objeto. La secuencia de escape `\n`, así se denomina, introduce un retorno de carro más un avance de línea (caracteres Unicode, CR LF). El método **Write** hace lo mismo que **Write-Line** excepto avanzar a la línea siguiente cuando finaliza de escribir.

¿Cómo accedemos al valor de cada atributo si se han declarado privados por omisión? Podemos proceder igual que lo hicimos en la clase *CCuenta*, implementando un método para establecer el valor del atributo y otro para obtenerlo, o bien implementando una propiedad. Como ejemplo, vamos a implementar la propiedad *Marca* para acceder al atributo privado *marca* de la clase *COrdenador*:

```
public string Marca
{
    get
    {
        return marca;
    }
    set
    {
        if (value == null)
            marca = "marca desconocida";
        else
            marca = value;
    }
}
```

Observe que podemos hacer referencia al valor del atributo mediante la palabra reservada **value**, independientemente de su tipo. Por lo tanto, en nuestro ejemplo, en caso de que el usuario intente asignar a la propiedad un valor nulo, se considerará que la marca es desconocida.

Una vez implementada una propiedad, una sentencia como:

```
miOrdenador.Marca = "Dell";
```

invocaría automáticamente a **set** de *Marca* y asignaría el valor “Toshiba” al atributo *marca*, y una sentencia como:

```
System.Console.WriteLine(miOrdenador.Marca);
```

invocaría automáticamente a **get** de *Marca* y devolvería el valor del atributo *marca*, y lo mostraría.

Observe que operamos con la propiedad *Marca* como si se tratara de un atributo público. Un usuario que utilice la clase *COrdenador* en su programa así lo interpretaría, pero si intenta asignar una cadena nula, se dará cuenta de que dicha asignación no se permite, lo que demuestra que su forma de proceder está siendo supervisada para evitar posibles errores.

Guarde la clase con el nombre *COrdenador.cs*. En este instante, si nuestras pretensiones sólo son las expuestas hasta ahora, ya podemos crear objetos de esta clase y trabajar con ellos. Para ello, vamos a escribir una nueva clase que defina el método **Main**; llamemos a esta clase *MiOrdenador*.

```
class MiOrdenador
{
    public static void Main(string[] args)
    {
        COrdenador miOrdenador = new COrdenador();
        miOrdenador.Marca = "Toshiba";
        miOrdenador.EncenderOrdenador();
        miOrdenador.Estado();
    }
}
```

Analicemos el método **Main** para explicar lo que hace:

- La primera línea crea un objeto de la clase *COrdenador* y almacena una referencia al mismo en la variable *miOrdenador*. Esta variable la utilizaremos para acceder a ese objeto en las siguientes líneas.
- La línea siguiente establece el atributo *marca* del objeto referenciado por *miOrdenador*. Observe que los atributos se han declarado privados.
- En las dos últimas líneas el objeto recibe los mensajes *EncenderOrdenador* y *Estado*. La respuesta a esos mensajes es la ejecución de los métodos respectivos, que fueron explicados anteriormente.

Guarde esta clase con el nombre *MiOrdenador.cs*. Después compílela y ejecútela. Podrá observar que los resultados son los siguientes:

El ordenador ha sido encendido

El estado del ordenador es el siguiente:

Marca: Toshiba

Procesador:

Peso: 0 kg.

El ordenador está encendido

La pantalla está activada

2. Este otro ejemplo muestra una clase que representa números racionales. Esta clase puede ser útil porque muchos números no pueden ser representados exactamente utilizando un número fraccionario. Por ejemplo, el número racional $1/3$ representado como un número fraccionario sería 0,333333, valor más fácil de manipular,

pero a costa de perder precisión. Evidentemente, $1/3 * 3 = 1$, pero $0,333333 * 3 = 0,999999$.

Pensando en un número racional como si de un objeto se tratara, es fácil deducir que sus atributos son dos: el *numerador* y el *denominador*. Y los métodos aplicables sobre los números racionales son numerosos: suma, resta, multiplicación, simplificación, etc. Pero para hacerlo fácil, sólo añadiremos dos métodos sencillos: uno, *AsignarDatos*, para establecer los valores del numerador y del denominador; y otro, *VisualizarRacional*, para visualizar un número racional.

Abra el procesador de textos o el editor de su entorno integrado y edite la aplicación propuesta, como se muestra a continuación:

```
class CRacional
{
    private int Numerador;
    private int Denominador;

    public void AsignarDatos(int num, int den)
    {
        Numerador = num;
        if (den == 0) den = 1; // el denominador no puede ser cero
        Denominador = den;
    }

    public void VisualizarRacional()
    {
        System.Console.WriteLine(Numerador + "/" + Denominador);
    }
}
```

Una vez editada la clase *CRacional*, guárdela en el disco con el nombre *CRacional.cs*. Para poder crear objetos de esta clase y trabajar con ellos, vamos a escribir una nueva clase que defina el método **Main**. Llamemos a esta clase *Racionales*.

```
class Racionales
{
    public static void Main(string[] args)
    {
        // Punto de entrada a la aplicación
        CRacional r1 = new CRacional(); // crear un objeto CRacional
        r1.AsignarDatos(2, 5);
        r1.VisualizarRacional();
    }
}
```

Analicemos el programa. Fijándonos en el método **Main**, vemos que se ha declarado un objeto *r1* de la clase *CRacional*.

```
CRacional r1 = new CRacional();
```

En el siguiente paso se envía el mensaje *AsignarDatos* al objeto *r1*. El objeto responde a este mensaje ejecutando su método *AsignarDatos* que almacena el valor 2 en su numerador y el valor 5 en su denominador; ambos valores han sido pasados como argumentos.

```
r1.AsignarDatos(2, 5);
```

Finalmente, se envía el mensaje *VisualizarRacional* al objeto *r1*. El objeto responde a este mensaje ejecutando su método *VisualizarRacional*, que visualiza sus atributos numerador y denominador en forma de quebrado; en nuestro caso, el número racional 2/5.

```
r1.VisualizarRacional();
```

Para finalizar, compile, ejecute la aplicación y observe que el resultado es el esperado.

EJERCICIOS PROPUESTOS

1. Añada a la aplicación *MiOrdenador.cs* los métodos descritos en el enunciado y que no añadimos, y además, el método *ApagarOrdenador*.
2. Diseñe una clase *CCoche* que represente coches. Incluya los atributos *marca*, *modelo* y *color*; y los métodos que simulen, enviando mensajes, las acciones de arrancar el motor, cambiar de velocidad, acelerar, frenar y parar el motor.
3. Añada a la aplicación *Racionales.cs* los métodos descritos en el enunciado y que no añadimos (sumar, restar, etc.), y además, un constructor con parámetros, así como los métodos que usted desee.

ELEMENTOS DEL LENGUAJE

En este capítulo veremos los elementos que aporta C# para escribir un programa. Algunos ya fueron explicados en el capítulo 2. En cualquier caso, considere este capítulo como soporte para los restantes; esto es, lo que se va a exponer en él, lo irá utilizando en menor o mayor medida en los capítulos sucesivos. Por lo tanto, límitese ahora simplemente a realizar un estudio con el fin de informarse de los elementos con los que contamos.

CARACTERES DE C#

C# utiliza el juego de caracteres UNICODE. De este juego de caracteres, los que frecuentemente utilizamos en español cuando escribimos código C# podríamos agruparlos así:

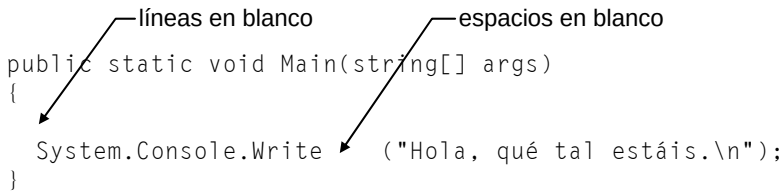
- Letras de la ‘a’ a la ‘z’ mayúsculas y minúsculas, además de las letras acentuadas, la ‘ñ’ y el ‘_’.
- Dígitos: 0 1 2 3 4 5 6 7 8 9.
- Espacios en blanco. Bajo este nombre genérico se agrupan los caracteres espacio en blanco (Unicode SP), tabulador horizontal (Unicode HT), avance de página (Unicode FF), nueva línea (Unicode LF), retorno de carro (Unicode CR) o CR LF (estos dos caracteres son considerados como uno solo: `\n`), porque su finalidad es la misma que la del espacio en blanco: actuar como separadores entre los elementos de un programa para conseguir programas más legibles. Por ejemplo, el siguiente código:

```
public static void Main(string[] args) { System.Console.Write(
"Hola, qué tal estáis.\n"); }
```

puede escribirse de una forma más legible así:

```
public static void Main(string[] args)
{
    System.Console.WriteLine("Hola, qué tal estáis.\n");
}
```

Los espacios en blanco en exceso son ignorados por el compilador. Por ejemplo, el código siguiente se comporta exactamente igual que el anterior:



```
public static void Main(string[] args)
{
    System.Console.WriteLine ("Hola, qué tal estáis.\n");
}
```

- Caracteres especiales: , . ; : ? ' " () [] { } < ! | / \ ~ + % & ^ * _ = >
- Cualquier carácter de los anteriores puede también ser representado por una *secuencia de escape* que está formada por el carácter \ seguido de una *letra* o de una *combinación de dígitos*. Son utilizadas para acciones como nueva línea, tabular y para hacer referencia a caracteres no imprimibles. El lenguaje C# tiene predefinidas las siguientes secuencias de escape:

Secuencia	Carácter	Definición
\0	NUL	Carácter nulo (UNICODE 0)
\n	CR+LF	Ir al principio de la línea siguiente
\t	HT	Tabulador horizontal
\b	BS	Retroceso (<i>backspace</i>)
\r	CR	Retorno de carro sin avance de línea
\f	FF	Alimentación de página (sólo para impresora)
\'	'	Comilla simple
\"	"	Comilla doble
\\	\	Barra invertida (<i>backslash</i>)
\Odd		Carácter ASCII. Representación octal
\xdd		Carácter ASCII. Representación hexadecimal
\u0000		Carácter UNICODE. Representación hexadec.
\a	BEL	Alerta, pitido (\u0007)
\v	VT	Tabulador vertical (\u000B)

Observe en el ejemplo anterior la secuencia de escape \n en la llamada al método **Write**.

TIPOS

Los tipos en C# se clasifican en: tipos *valor* y tipos *referencia*. Una variable de un tipo *valor* almacena directamente un valor (datos en general), mientras que una variable de un tipo *referencia* lo que permite almacenar es una referencia a un objeto (posición de memoria donde está el objeto). Por ejemplo:

```
int suma = 0;           // suma almacena un entero.
CCuenta cta = null;    // cta permitirá almacenar una referencia
                        // a un objeto CCuenta.
```

Tanto las clases como las matrices, interfaces y delegados que estudiaremos más adelante, son tipos *referencia*.

En cuanto a los tipos *valor*, C# los clasifica de la forma siguiente: los tipos primitivos, las estructuras y los tipos enumerados. A su vez, los tipos primitivos se clasifican en tipos enteros, tipos reales y el tipo **bool**.

Tipos enteros:

con signo: **sbyte**, **short**, **int**, **long**.

sin signo: **byte**, **ushort**, **uint**, **ulong** y **char**.

Tipos reales:

coma flotante: **float**, **double**.

coma fija: **decimal**.

Cada tipo primitivo tiene un rango diferente de valores: positivos y negativos en el caso de los tipos con signo, positivos en el caso de tipos sin signo, y **true** y **false** en el caso del tipo **bool**. El tipo que se seleccione para declarar las variables de un determinado programa dependerá del rango y tipo de valores que vayan a almacenar cada una de ellas y de si éstos son enteros o fraccionarios. Para más detalles véase el capítulo 2.

Las estructuras son similares a las clases pero con algunas restricciones. Permiten declarar tipos de datos que pueden encapsular atributos y métodos, pero a diferencia de las clases, son tipos *valor*, no *referencias* a objetos. Por lo tanto, pospondremos su estudio al capítulo dedicado a clases.

Todos los tipos primitivos expuestos tienen una estructura de datos asociada; por ejemplo, el tipo **double** es un alias de **System.Double** (estructura **Double** del espacio de nombres **System**), **char** es un alias de **System.Char**, **bool** es un alias de **System.Boolean**, **int** es un alias de **System.Int32**, etc. Por lo tanto, un dato de un tipo primitivo es un objeto.

Los tipos enumerados son tipos definidos por el usuario. Para crearlos se utiliza la palabra **enum**. Por ejemplo:

```
class Test
{
    enum día {lunes, martes, miércoles, jueves, viernes, sábado, domingo};

    public static void Main(string[] args)
    {
        día díaSem = día.jueves;
        if (díaSem == día.domingo)
            System.Console.WriteLine("fiesta");
        else
            System.Console.WriteLine(díaSem); // escribe "jueves"
    }
}
```

Los valores con los que se forma un tipo enumerado se corresponden con las constantes enteras 0, 1, 2, etc. Según esto, en el ejemplo anterior *lunes* se corresponde con 0, *martes* con 1, *miércoles* con 2, etc. Análogamente al tipo *días*, puede definir cualquier otro tipo enumerado de datos. Esto, en algunas ocasiones, puede ayudar a escribir un código más legible.

LITERALES

Un literal es la expresión de un valor de alguno de los tipos expuestos anteriormente, de un tipo **string** (alias de **System.String**: clase para cadenas de caracteres) o la expresión **null** (valor nulo para cualquier tipo *valor* o *referencia*). Por ejemplo, son literales: 0, 5, 3.14, 'a', '\0' (carácter nulo), "hola" y **null**. En realidad son valores constantes.

Un literal en C# puede ser: un entero, un real, un valor booleano, un carácter, una cadena de caracteres y un valor nulo.

Literales enteros

El lenguaje C# permite especificar un literal entero en base 10 y 16.

En general, el signo + es opcional si el valor es positivo y el signo – estará presente siempre que el valor sea negativo. El tipo de un literal entero depende de su base, de su valor y de su sufijo. La sintaxis para especificar un literal entero es:

$$\{[+]|-\}literal_entero[[_L|_U|_UL]]$$

Si el literal no tiene sufijo, su tipo es el primero de los tipos **int**, **uint**, **long** o **ulong** en el que su valor pueda ser representado.

Si el literal tiene sufijo y éste es *U*, su tipo es el primero de los tipos **uint** o **ulong** en el que su valor pueda ser representado; si es *L*, su tipo es el primero de los tipos **long** o **ulong** y si es *UL*, su tipo es **ulong**.

Un *literal entero decimal* puede tener uno o más dígitos del 0 al 9, de los cuales el primero de ellos es distinto de 0. Por ejemplo:

```
4326           constante entera int
4326L          constante entera long
3426000000     constante entera long
```

Un *literal entero hexadecimal* puede tener uno o más dígitos del 0 al 9 y letras de la A a la F (en mayúsculas o en minúsculas) precedidos por *0x* o *0X* (cero seguido de x). Por ejemplo:

```
256           número decimal 256
0x100         número decimal 256 expresado en hexadecimal
-256          número decimal -256
-0x100        número decimal -256 expresado en hexadecimal
```

Literales reales

Un literal real está formado por una *parte entera*, seguido por un *punto decimal*, y una *parte fraccionaria*. También se permite la notación científica, en cuyo caso se añade al valor una *e* o *E*, seguida por un exponente positivo o negativo.

$\{[+]|-\} \textit{parte-entera} . \textit{parte-fraccionaria} [\{e|E\} \{[+]|-\} \textit{exponente}]$

donde *exponente* representa cero o más dígitos del 0 al 9 y *E* o *e* es el símbolo de exponente de la base 10 que puede ser positivo o negativo ($2E-5 = 2 \times 10^{-5}$). Si la constante real es positiva, no es necesario especificar el signo y si es negativa, lleva el signo menos (-). Por ejemplo:

```
-17.24
17.244283
.008e3
27E-3
```

Una constante real tiene siempre tipo **double**, a no ser que se añada a la misma una *f* o *F*, en cuyo caso será de tipo **float**, o *m* o *M*, en cuyo caso es de tipo **decimal**. Por ejemplo:

17.24F constante real de tipo float

También se pueden utilizar los sufijos *d* o *D* para especificar explícitamente que se trata de una constante de tipo **double**. Por ejemplo:

17.24D constante real de tipo double

Literales de un solo carácter

Los literales de un solo carácter son de tipo **char**. Este tipo de literales está formado por un único carácter encerrado entre *comillas simples*. Una secuencia de escape es considerada como un único carácter. Algunos ejemplos son:

```
' '            espacio en blanco
'x'           letra minúscula x
'\n'          retorno de carro más avance de línea
'\u0007'      pitido
'\0'          carácter nulo (Unicode '\u0000')
```

El valor de una constante de un solo carácter es el código que le corresponde en el juego de caracteres de la máquina.

Literales de cadenas de caracteres

Un literal de cadena de caracteres es una secuencia de caracteres encerrados entre *comillas dobles* (incluidas las secuencias de escape como `\`). Por ejemplo:

```
"Esto es una constante de caracteres"
"3.1415926"
"Paseo de Pereda 10, Santander"
""                                    /* cadena vacía */
"Lenguaje \"C#\" "                   /* produce: Lenguaje "C#" */
```

Este otro ejemplo que se muestra a continuación presenta una cadena de caracteres precedida por el carácter `@`. En este caso se trata de un *literal de cadena plano*, su contenido es interpretado tal cual, sin tener en cuenta las secuencias de escape, según muestra el siguiente ejemplo:

```
@ "\\servidor\carpeta\f.txt"
```

La cadena anterior, utilizando ahora la secuencia de escape `\\`, es equivalente a la siguiente:

```
"\\\\servidor\\\\carpeta\\f.txt"
```


En el ejemplo siguiente el carácter `\n` fuerza a que la cadena “O pulse Entrar” se escriba en una nueva línea:

```
System.Console.Write("Escriba un número entre 1 y 5\n0 pulse En-
trar");
```

Las cadenas de caracteres en C# son objetos de la clase **String** que estudiaremos más adelante. Esto es, cada vez que en un programa se utilice un literal de caracteres, C# crea de forma automática un objeto **String** con el valor del literal.

Las cadenas de caracteres se pueden concatenar (unir) empleando el operador `+`. Por ejemplo, la siguiente sentencia concatena las cadenas “Distancia: ”, *distancia*, y “Km.”.

```
System.Console.WriteLine("Distancia: " + distancia + " Km.");
```

Si alguna de las expresiones no se corresponde con una cadena, como se supone que ocurre con *distancia*, C# la convierte de forma automática en una cadena de caracteres. Más adelante aprenderá el porqué de esto.

Literal null

El literal nulo, **null**, es un valor especial utilizado para especificar que una variable de tipo *valor* o *referencia* no tiene un dato asignado. Por ejemplo:

```
int? n = null;    // valor nulo
string s = null; // referencia nula

n = 2;
if (n != null) // equivale a: if (n.HasValue)
    System.Console.WriteLine(100/n); // equivale a: 100 / n.Value
else
    System.Console.WriteLine("n no tiene un valor asignado");
s = "hola";
if (s != null)
    System.Console.WriteLine(s);
else
    System.Console.WriteLine("s no tiene un valor asignado");
```

El ejemplo anterior enseña que para declarar un variable de un tipo *valor*, en el ejemplo **int**, que acepte el valor **null** hay que utilizar el modificador **?** después del identificador del tipo. Esto resuelve definitivamente el problema de cómo especificar un valor nulo o indefinido para los tipos *valor* (tenga presente que para el tipo entero, cero no es un valor entero nulo, sino que es el valor 0 del dominio de los enteros).

IDENTIFICADORES

Los identificadores son nombres dados a tipos, literales, variables, clases, interfaces, métodos, espacios de nombres y sentencias de un programa. La sintaxis para formar un identificador es la siguiente:

$$\{letra|_|\}[\{letra|dígito|_|\}]\dots$$

lo cual indica que un identificador consta de uno o más caracteres y que el *primer carácter* debe ser una *letra* o el *carácter de subrayado*. No pueden comenzar por un dígito ni pueden contener caracteres especiales.

Las letras pueden ser mayúsculas o minúsculas. Para C# una letra mayúscula es un carácter diferente a esa misma letra en minúscula. Por ejemplo, los identificadores *Suma*, *suma* y *SUMA* son diferentes. Algunos ejemplos son:

```
Suma
Cálculo_Números_Primos
_ordenar
VisualizarDatos
```

PALABRAS CLAVE

Las palabras clave son identificadores predefinidos que tienen un significado especial para el compilador C#. Por lo tanto, un identificador definido por el usuario no puede tener el mismo nombre que una palabra clave. El lenguaje C# tiene las siguientes palabras clave:

abstract	as	base	bool	break
byte	case	catch	char	checked
class	const	continue	decimal	default
delegate	do	double	else	enum
event	explicit	extern	false	finally
fixed	float	for	foreach	goto
if	implicit	in	int	interface
internal	is	lock	long	namespace
new	null	object	operator	out
override	params	private	protected	public
readonly	ref	return	sbyte	sealed
short	sizeof	stackalloc	static	string
struct	switch	this	throw	true
try	typeof	uint	ulong	unchecked
unsafe	ushort	using	virtual	void
volatile	while			

Las palabras clave deben escribirse siempre en minúsculas, tal y como están.

Si alguna vez deseáramos utilizar alguna de estas palabras clave como identificador, tendríamos que utilizar el carácter **@** como prefijo de dicho identificador. Por ejemplo, el siguiente código es válido:

```
class @class
{
    static void @static(bool @bool)
    {
        if (@bool)
            System.Console.WriteLine("verdadero");
        else
            System.Console.WriteLine("falso");
    }
}
```

DECLARACIÓN DE CONSTANTES SIMBÓLICAS

Declarar una constante simbólica significa decirle al compilador C# el nombre de la constante y su valor. Esto se hace utilizando el calificador **const**.

```
class CElementosCSharp
{
    const int cte1 = 1;
    const string cte2 = "Pulse una tecla para continuar";

    void Test()
    {
        const double cte3 = 3.1415926;
        // ...
    }
    // ...
}
```

Como se observa en el ejemplo anterior, declarar una constante simbólica supone anteponer el calificador **const** al tipo y nombre de la constante, que será iniciada con el valor deseado.

```
const tipo identificador = cte, [, identificador = cte.]...
```

Una vez que se haya declarado una constante, por definición, no se le puede asignar otro valor. Por ello, cuando se declara una constante debe ser iniciada con un valor. Por ejemplo, después de haber declarado *cte3* según se muestra en el ejemplo anterior, una sentencia como la siguiente daría lugar a un error:

```
cte3 = 3.14;
```

Si una variable es **const**, también es **static**. Las variables **static** las estudiaremos en el capítulo siguiente.

¿Por qué utilizar constantes?

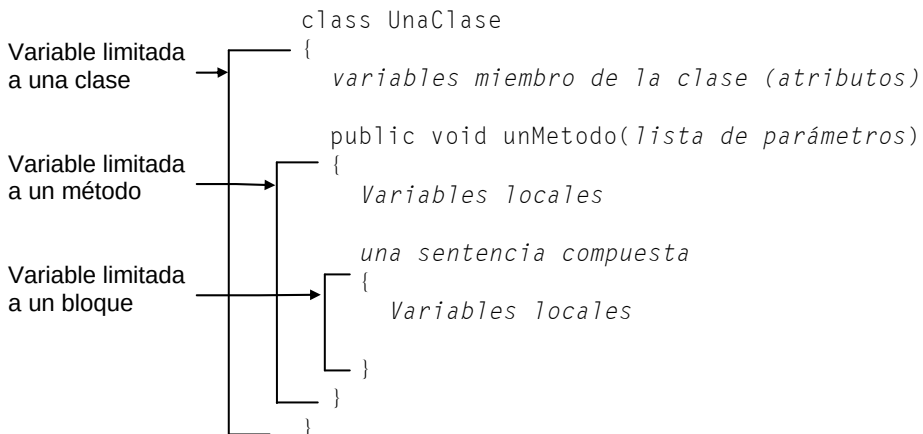
Utilizando constantes es más fácil modificar un programa. Por ejemplo, supongamos que un programa utiliza N veces una constante de valor 3.14. Si hemos definido dicha constante como *const double Pi = 3.14* y posteriormente necesitamos cambiar el valor de la misma a 3.1416, sólo tendremos que modificar una línea, la que define la constante. En cambio, si no hemos declarado *Pi*, sino que hemos utilizado el valor 3.14 directamente N veces, tendríamos que realizar N cambios.

VARIABLES

Una variable representa un espacio de memoria para almacenar un dato de un determinado tipo: *valor* o *referencia*. Dicho dato, a diferencia de una constante, puede cambiar durante la ejecución de un programa. La declaración de una variable se realiza según la sintaxis siguiente:

```
tipo identificador[, identificador]...
```

La declaración de una variable puede realizarse a *nivel de la clase* (atributos de la clase), a *nivel del método* (dentro de la definición de un método) o a *nivel de un bloque de código*. Dependiendo de dónde se declare, su uso estará limitado a la clase, al método o al bloque de código que la define. Este espacio del programa al que queda limitada una variable se denomina *ámbito* de esa variable.



Una *variable miembro de una clase*, también denominada *atributo*, puede ser declarada en cualquier parte dentro de la clase siempre que sea fuera de todo método y estará disponible para todo el código de esa clase.

Una variable declarada dentro de un método es una variable local al método. Los parámetros de un método son también variables locales al método. Y una variable declarada dentro de un bloque correspondiente a una sentencia compuesta también es una variable local a ese bloque.

En general, una *variable local* existe y tiene valor desde su punto de declaración hasta el final del bloque donde está definida. Cada vez que se ejecuta el bloque que la contiene, la variable local es nuevamente definida, y cuando finaliza la ejecución del mismo, la variable local deja de existir. Un elemento con carácter local es accesible solamente dentro del bloque al que pertenece.

En el ejemplo siguiente se declaran tres variables de tipo **short**, una variable de tipo **int** y dos variables de tipo **string**:

```
class CElementosCSharp
{
    short día, mes, año = 2010;

    void Test()
    {
        int contador = 0;
        string Nombre = "", Apellidos = "";
        día = 20;
        Apellidos = "Ceballos";
        // ...
    }
    // ...
}
```

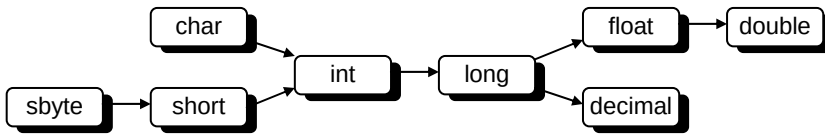
Según lo expuesto, las variables *día*, *mes* y *año* son accesibles desde todos los métodos no **static** de la clase *CElementosCSharp*. En cambio, las variables *contador*, *Nombre* y *Apellidos* han sido declaradas en el bloque de código correspondiente al cuerpo del método *Test*; por lo tanto, sólo serán accesibles en este bloque. En el capítulo siguiente hablaremos de los métodos y atributos **static** de una clase.

Las variables miembro de una clase son iniciadas por omisión por el compilador C# para cada objeto que se declare de la misma: las variables numéricas con 0, los caracteres con '\0' y las referencias a las cadenas de caracteres y el resto de las referencias a otros objetos con **null**. También pueden ser iniciadas explícitamente, como ocurre con *año*. En cambio, las variables locales no son iniciadas por el compilador C#. Por lo tanto, es nuestra obligación iniciarlas, de lo contrario el

compilador visualizará un mensaje de error en todas las sentencias que accedan a esas variables antes de ser iniciadas.

CONVERSIÓN ENTRE TIPOS PRIMITIVOS

Cuando C# tiene que evaluar una expresión en la que intervienen operandos de diferentes tipos, primero convierte, sólo para realizar las operaciones solicitadas, los valores de los operandos al tipo del operando cuya precisión sea más alta. Cuando se trate de una asignación, convierte el valor de la derecha al tipo de la variable de la izquierda siempre que no haya pérdida de información; en otro caso, C# exige que la conversión se realice explícitamente. La figura siguiente resume los tipos con signo, colocados de izquierda a derecha de menos a más precisos; las flechas indican las conversiones implícitas permitidas:



```
// Conversión implícita
sbyte bDato = 1; short sDato = 0; int iDato = 0; long lDato = 0;
float fDato = 0; double dDato = 0; decimal mDato = 0;

sDato = bDato;
iDato = sDato;
lDato = iDato;
fDato = lDato;
mDato = bDato;
dDato = fDato + lDato - iDato * sDato / bDato;
System.Console.WriteLine(dDato); // resultado: 1
```

C# permite una conversión explícita (conversión forzada) del tipo de una expresión mediante una construcción denominada *cast*, que tiene la forma:

(tipo) expresión

Cualquier valor de un tipo entero o real puede ser convertido a o desde cualquier tipo numérico. No se pueden realizar conversiones entre los tipos enteros o reales y el tipo **bool**. Por ejemplo:

```
// Conversión explícita (cast)
byte bDato = 0; short sDato = 0; int iDato = 0; long lDato = 0;
float fDato = 0; double dDato = 2;

fDato = (float)dDato;
lDato = (long)fDato;
iDato = (int)lDato;
```

```
sDato = (short)iDato;
mDato = (decimal)dDato;
bDato = (sbyte)(sDato + iDato - lDato * fDato / dDato);
System.Console.WriteLine(bDato); // resultado: 2
```

La expresión es convertida al tipo especificado si esa conversión está permitida; en otro caso, se obtendrá un error. La utilización apropiada de construcciones *cast* garantiza una evaluación consistente, pero siempre que se pueda es mejor evitarla ya que suprime la verificación de tipo proporcionada por el compilador y por consiguiente puede conducir a resultados inesperados, o cuanto menos, a una pérdida de precisión en el resultado. Por ejemplo:

```
float r;
r = (float)System.Math.Sqrt(10);
```

En el ejemplo anterior, el resultado se redondea perdiendo precisión ya que *Sqrt* devuelve un valor de tipo **double**.

En el capítulo 5 expondremos *Conversión entre tipos referencia* y en el capítulo 6, *Conversión entre tipos valor y tipos referencia*.

OPERADORES

Los operadores son símbolos que indican cómo son manipulados los datos. Se pueden clasificar en los siguientes grupos: aritméticos, relacionales, lógicos, unitarios, a nivel de bits, de asignación y operador condicional. En el capítulo 2 vimos los operadores aritméticos y los de relación. A continuación veremos el resto.

Operadores lógicos

El resultado de una operación lógica (AND, OR, XOR y NOT) es un valor booleano verdadero o falso (**true** o **false**). Las expresiones que dan como resultado valores booleanos (véanse los operadores de relación) pueden combinarse para formar expresiones *booleanas* utilizando los operadores lógicos indicados a continuación. Los operandos deben ser expresiones que den un resultado **bool**.

Operador	Operación
&& o &	<i>AND</i> . Da como resultado true si al evaluar cada uno de los operandos el resultado es true . Si uno de ellos es false , el resultado es false . Si se utiliza && (no &) y el primer operando es false , el segundo operando no es evaluado.
o	<i>OR</i> . El resultado es false si al evaluar cada uno de los operandos el resultado es false . Si uno de ellos es true , el resultado es true . Si se

utiliza `||` (no `|`) y el primer operando es **true**, el segundo operando no es evaluado (el carácter `|` es el Unicode 124).

! *NOT*. El resultado de aplicar este operador es **false** si al evaluar su operando el resultado es **true**, y **true** en caso contrario.

^ *XOR*. Da como resultado **true** si al evaluar cada uno de los operandos el resultado de uno es **true** y el del otro **false**; en otro caso el resultado es **false**.

El resultado de una operación lógica es de tipo **bool**. Por ejemplo:

```
int p = 10, q = 0;
bool r;
r = p != 0 && q != 0;    // r = false
r = p != 0 || q > 0;     // r = true
r = q < p && p <= 10;    // r = true
r = !r;                  // si r = true, entonces r = false
```

Operadores unitarios

Los operadores unitarios se aplican a un solo operando y son los siguientes: `!`, `~`, `++` y `--`. El operador `!` ya lo hemos visto y los operadores `++` y `--` los veremos más adelante.

Operador	Operación
<code>~</code>	Complemento a 1 (cambiar ceros por unos y unos por ceros). El carácter <code>~</code> es el Unicode 126. El operando debe de ser de un tipo int , uint , long o ulong .
<code>-</code>	Cambia de signo al operando (esto es, se calcula el complemento a 2 que es el complemento a 1 más 1). El operando puede ser de un tipo numérico.

El siguiente ejemplo muestra cómo utilizar estos operadores:

```
int a = 2, b = 0, c = 0;

c = -a;    // resultado c = -2
c = ~b;    // resultado c = -1
```

Operadores a nivel de bits

Estos operadores permiten realizar con sus operandos las operaciones AND, OR, XOR y desplazamientos, bit por bit. Los operandos tienen que ser enteros.

Operador	Operación
&	Operación AND a nivel de bits.
	Operación OR a nivel de bits (carácter Unicode 124).
^	Operación XOR a nivel de bits.
<<	Desplazamiento a la izquierda rellenando con ceros por la derecha.
>>	Desplazamiento a la derecha rellenando con el bit de signo por la izquierda.

Los operandos para los operadores &, | y ^ tienen que ser de un tipo entero o **bool**, y para los de desplazamiento el primer operando debe ser de tipo **int**, **uint**, **long** o **ulong** y el segundo de tipo **int**.

```
int a = 255, r = 0, m = 32;

r = a & 0xF; // r=15. Pone a cero todos los bits de a
             // excepto los 4 bits de menor peso.

r = r | m;    // r=47. Pone a 1 todos los bits de r que
             // estén a 1 en m.
r = a & ~0x7; // r=248. Pone a 0 los 3 bits de menor peso de a.
r = a >> 7;   // r=1. Desplazamiento de 7 bits a la derecha.
r = m << 1;   // r=64. Equivale a r = m * 2.
r = m >> 1;   // r=16. Equivale a r = m / 2.
```

Operadores de asignación

El resultado de una operación de asignación es el valor almacenado en el operando izquierdo, lógicamente después de que la asignación se haya realizado. El valor que se asigna es convertido implícita o explícitamente al tipo del operando de la izquierda (véase el apartado *Conversión entre tipos primitivos*). Incluimos aquí los operadores de incremento y decremento porque implícitamente estos operadores realizan una asignación sobre su operando.

Operador	Operación
++	Incremento.
--	Decremento.
=	Asignación simple.
*=	Multipliación más asignación.
/=	División más asignación.
%=	Módulo más asignación.
+=	Suma más asignación.

<code>--=</code>	Resta más asignación.
<code><<=</code>	Desplazamiento a la izquierda más asignación.
<code>>>=</code>	Desplazamiento a la derecha más asignación.
<code>&=</code>	Operación AND sobre bits más asignación.
<code> =</code>	Operación OR sobre bits más asignación.
<code>^=</code>	Operación XOR sobre bits más asignación.

Los operandos tienen que ser del mismo tipo o bien el operando de la derecha tiene que poder ser convertido implícitamente al tipo del operando de la izquierda. A continuación se muestran algunos ejemplos con estos operadores.

```
int x = 0, n = 10, i = 1;
n++;           // Incrementa el valor de n en 1.
++n;          // Incrementa el valor de n en 1.
x = ++n;       // Incrementa n en 1 y asigna el resultado a x.
x = n++;       // Equivale a realizar las dos operaciones
               // siguientes en este orden: x = n; n++.
i += 2;        // Realiza la operación i = i + 2.
x *= n - 3;    // Realiza la operación x = x * (n-3) y no
               // x = x * n - 3.
n >>= 1;       // Realiza la operación n = n >> 1 la cual desplaza
               // el contenido de n 1 bit a la derecha.
```

El operador de incremento incrementa su operando en una unidad independientemente de que se utilice como sufijo o como prefijo; esto es, `n++` y `++n` producen el mismo resultado. Ídem para el operador de decremento.

Ahora bien, cuando se asigna a una variable una expresión en la que intervienen operadores de incremento o de decremento, el resultado difiere según se utilicen estos operadores como sufijo o como prefijo. Si se utilizan como prefijo primero se realizan los incrementos o decrementos y después la asignación (ver más adelante la tabla de prioridad de los operadores). Por ejemplo, `y = ++x` es equivalente a `y = (x += 1)`. En cambio, si se utilizan como sufijo, el valor asignado corresponde a la evaluación de la expresión antes de aplicar los incrementos o los decrementos. Por ejemplo, `y = x++` es equivalente a `(t=x; x+=1; y=t;)`, suponiendo que `t` es una variable del mismo tipo que `x`.

Según lo expuesto, ¿cuál es el valor de `x` después de evaluar la siguiente expresión?

```
x = (a - b++) * (--c - d) / 2
```

Comprobemos el resultado evaluando esta expresión mediante el siguiente código. Observamos que en el cálculo de x intervienen los valores de b sin incrementar y de c decrementado, con lo que el resultado será x igual a -30 .

```
float x = 0, a = 15, b = 5, c = 11, d = 4;
x = (a - b++) * (--c - d) / 2;
```

Resultado:

$x = -30, b = 6, c = 10$

Una expresión de la complejidad de la anterior equivale a calcular la misma expresión sin operadores $++$ y $--$, pero incrementando/decrementando antes las variables afectadas por $++$ y $--$ como prefijo e incrementando/decrementando después las variables afectadas por $++$ y $--$ como sufijo. Esto equivale a escribir el código anterior así:

```
float x = 0, a = 15, b = 5, c = 11, d = 4;
--c; // o bien c--
x = (a - b) * (c - d) / 2;
b++;
```

La aplicación de la regla anterior se complica cuando una misma variable aparece en la expresión, afectada varias veces por los operadores $++$ y $--$ (incluso, reutilizada a la izquierda del signo igual). Por ejemplo:

```
float x = 0, a = 20, b = 10, d = 4;
x = (a - b++) * (--b - d) * b++ / (b - d);
```

Cuando se aplica la regla anterior a un caso como éste, hay que tener en cuenta que los incrementos/decrementos como prefijo afectan a los cálculos que le siguen en la propia expresión; por eso habrá que intercalarlos en el lugar adecuado. El ejemplo siguiente realiza los mismos cálculos que la expresión anterior:

```
float x = 0, a = 20, b = 10, d = 4;
x = (a - b); b++;
--b; x *= (b - d);
x *= b; b++;
x /= (b - d);
```

Resultado:

$x = 85,71429, b = 11$

Este código es mucho más sencillo de entender que la expresión equivalente anterior, y también menos propenso a introducir errores, por lo que se recomienda esta forma de trabajar.

Operador condicional

El operador condicional (`?:`), llamado también operador ternario, se utiliza en expresiones condicionales, que tienen la forma siguiente:

$$\text{operando1} \text{ ? } \text{operando2} \text{ : } \text{operando3}$$

La expresión *operando1* debe ser una expresión booleana. La ejecución se realiza de la siguiente forma:

- Si el resultado de la evaluación de *operando1* es **true**, el resultado de la expresión condicional es *operando2*.
- Si el resultado de la evaluación de *operando1* es **false**, el resultado de la expresión condicional es *operando3*.

El siguiente ejemplo asigna a *mayor* el resultado de $(a > b) ? a : b$, que será *a* si *a* es mayor que *b* y *b* si *a* no es mayor que *b*.

```
double a = 10.2, b = 20.5, mayor = 0;  
mayor = (a > b) ? a : b;
```

PRIORIDAD Y ORDEN DE EVALUACIÓN

Cuando escribimos una expresión como la siguiente, $f = a + b * c / d$, es porque conocemos perfectamente el orden en el que se ejecutan las operaciones. Si este orden no fuera el que esperamos tendríamos que utilizar paréntesis para modificarlo. Esto quiere decir que el compilador C# atribuye a cada operador un nivel de prioridad; de esta forma puede resolver qué operación se ejecuta antes que otra en una expresión. Esta prioridad puede ser modificada utilizando paréntesis. Los paréntesis tienen mayor prioridad y son evaluados de más internos a más externos. Como ejemplo de lo expuesto, la expresión anterior puede escribirse también así: $f = (a + ((b * c) / d))$, lo cual indica que primero se evalúa $b * c$, el resultado se divide por *d*, el resultado se suma con *a* y finalmente el resultado se asigna a *f*.

La tabla siguiente resume las reglas de prioridad y asociatividad de todos los operadores. Las líneas se han colocado de mayor a menor prioridad. Los operadores escritos sobre una misma línea tienen la misma prioridad.

Operador	Asociatividad
() [] . new typeof checked unchecked	izquierda a derecha
- ~ ! ++ -- (tipo)expresión	derecha a izquierda
* / %	izquierda a derecha
+ -	izquierda a derecha
<< >>	izquierda a derecha
< <= > >= is as	izquierda a derecha
== !=	izquierda a derecha
&	izquierda a derecha
^	izquierda a derecha
	izquierda a derecha
&&	izquierda a derecha
	izquierda a derecha
?:	derecha a izquierda
= *= /= %= += -= <<= >>= &= = ^=	derecha a izquierda

En C#, todos los operadores *binarios* excepto los de asignación y el operador ternario son evaluados de izquierda a derecha. En el siguiente ejemplo, primero se asigna *z* a *y* y a continuación *y* a *x*.

```
int x = 0, y = 0, z = 15;
x = y = z;      // resultado x = y = z = 15
```

En C#, los operadores *unitarios* son evaluados de derecha a izquierda. En el siguiente ejemplo, primero se aplica *~* a *y* y a continuación *-* al resultado.

```
int x = 0; int y = 1;
x = -~y;      // resultado x = 2, y = 1
```

EJERCICIOS RESUELTOS

La siguiente aplicación utiliza objetos de una clase *CPolinomio* para evaluar expresiones de la forma:

$$ax^3 + bx^2 + cx + d$$

Un polinomio se puede ver como un objeto que envuelve el exponente, los coeficientes y los métodos que permitan manipularlo. Para hacer sencillo el ejemplo que tratamos de exponer, el exponente del término de mayor grado lo suponemos fijo de valor 3, los coeficientes serán variables, y añadiremos dos métodos:

uno que permita establecer el polinomio con el que deseamos trabajar y otro que permita evaluarlo para un valor de x dado. Resumiendo, los objetos *CPolinomio* tendrán unos atributos que serán los coeficientes y unos métodos *Polinomio* y *ValorPara* para manipularlos.

El método *Polinomio* simplemente asignará los valores pasados como argumentos a los atributos representativos de los coeficientes del polinomio.

El método *ValorPara* evaluará el polinomio para el valor de x pasado como argumento. Este método, utilizando la sentencia **return**, devolverá como resultado el valor calculado. Observe que el tipo devuelto por el método es **double**:

```
public double ValorPara(double x)
{
    double resultado;
    // Realizar cálculos
    return resultado;
}
```

Según lo expuesto y aplicando los conocimientos adquiridos en los capítulos anteriores, escribamos en primer lugar la clase *CPolinomio* como se muestra a continuación.

```
class CPolinomio
{
    // El término de mayor grado tiene exponente 3 fijo

    private double c3, c2, c1, c0; // coeficientes

    public void Polinomio(double a, double b, double c, double d)
    {
        c3 = a; c2 = b; c1 = c; c0 = d;
    }

    public double ValorPara(double x)
    {
        double resultado;
        resultado = c3*x*x*x + c2*x*x + c1*x + c0;
        return resultado; // devolver el valor calculado
    }
}
```

El siguiente paso es escribir otro fichero fuente con una clase *CMiAplicacion*, por ejemplo, que utilice la clase de objetos *CPolinomio*. Esta clase puede ser de la forma siguiente:

```
class CMiAplicacion
{
    public static void Main(string[] args)
    {
        CPolinomio pol1 = new CPolinomio();
        pol1.Polinomio(1, -3.2, 0, 7);

        double r = pol1.ValorPara(1);
        System.Console.WriteLine(r);

        r = pol1.ValorPara(1.5);
        System.Console.WriteLine(r);
    }
}
```

Recuerde que el método **Main** es por donde empieza a ejecutarse la aplicación. Este método crea un objeto *pol1* de la clase *CPolinomio*, envía al objeto *pol1* el mensaje *Polinomio* para establecer los coeficientes del polinomio y a continuación le envía el mensaje *ValorPara* con el objetivo de evaluar el polinomio para el valor de *x* pasado como argumento.

Una vez escrita la aplicación debe guardarla, por ejemplo, con el nombre *CMiAplicacion.cs* y compilarla (*csc CMiAplicacion.cs CPolinomio.cs*). Después puede ejecutarla y observar los resultados. Incluso puede atreverse a evaluar otros polinomios para distintos valores de *x*.

EJERCICIOS PROPUESTOS

1. ¿Qué resultados se obtienen al realizar las operaciones siguientes? Si hay errores en la compilación, corríjalos y dé una explicación de por qué suceden.

```
int a = 10, b = 3, c = 1, d, e;
float x, y;
x = a / b;
c = a < b && c;
d = a + b++;
e = ++a - b;
y = (float)a / b;
```

2. Escriba el valor Unicode de la 'q' y de la 'Q' sin consultar la tabla.

3. Escriba las sentencias necesarias para evaluar el siguiente polinomio para valores de $a = 5$, $b = -1.7$, $c = 2$, $d = 3.5$, $k = 3$ y $x = 10.5$.

$$ax^4 + bx^3 - cx^2 + dx + k$$

4. Escriba de nuevo la clase *CPolinomio* realizada en el apartado *Ejercicios resueltos* para que utilice un constructor para establecer los coeficientes de los polinomios. Después, escriba un programa que utilice objetos *CPolinomio*.

ESTRUCTURA DE UN PROGRAMA

En este capítulo se estudiará cómo es la estructura de un programa C#. Partiendo de un programa ejemplo sencillo analizaremos cada una de las partes que componen su estructura, así tendrá un modelo para realizar sus propios programas. También aprenderá más acerca de las clases y del trabajo con métodos.

ESTRUCTURA DE UN PROGRAMA C#

Puesto que C# es un lenguaje orientado a objetos, un programa C# se compone solamente de objetos. Recuerde que un objeto es la concreción de una clase, y que una clase equivale a la generalización de un tipo específico de objetos. La clase define los atributos del objeto así como los métodos para manipularlos. Muchas de las clases que utilizaremos pertenecen a la biblioteca .NET, por lo tanto ya están escritas y compiladas. Pero otras tendremos que escribirlas nosotros mismos, dependiendo del problema que tratemos de resolver en cada caso.

Todo programa C# está formado por al menos una clase que define un método nombrado **Main**, como se muestra a continuación:

```
public class CMiAplicacion
{
    public static void Main(string[] args)
    {
        // escriba aquí el código que quiere ejecutar
    }
}
```

Una clase que contiene un método **Main** es una plantilla para crear lo que vamos a denominar objeto principal (también llamado objeto aplicación), objeto

que tiene como misión iniciar y finalizar la ejecución del programa. Precisamente, el método **Main** es el punto de entrada y de salida del programa.

Según lo expuesto, la solución de cualquier problema no debe considerarse inmediatamente en términos de sentencias correspondientes a un lenguaje, sino de objetos naturales del problema mismo, abstraídos de alguna manera, que darán lugar a los objetos que intervendrán en la solución del programa. El empleo de este modelo de desarrollo de programas nos conduce al diseño y programación orientada a objetos, modelo que ha sido empleado para desarrollar todos los ejemplos de este libro.

Para explicar cómo es la estructura de un programa C#, vamos a plantear un ejemplo sencillo de un programa que presente una tabla de equivalencia entre grados centígrados y grados *fahrenheit*, como indica la figura siguiente:

-30 C	-22,00 F
-24 C	-11,20 F
.	
.	
.	
90 C	194,00 F
96 C	204,80 F

La relación entre los grados centígrados y los grados *fahrenheit* viene dada por la expresión $\text{grados fahrenheit} = 9/5 * \text{grados centígrados} + 32$. Los cálculos los vamos a realizar para un intervalo de -30 a 100 grados centígrados con incrementos de 6 .

Analicemos el problema. ¿De qué trata el programa? De grados. Entonces podemos pensar en objetos “grados” que encapsulen un valor en grados centígrados y los métodos necesarios para asignar al objeto un valor en grados centígrados, así como para obtener tanto el dato grados centígrados como su equivalente en grados *fahrenheit*. En base a esto, podríamos escribir una clase *CGrados* como se puede observar a continuación:

```
class CGrados
{
    private float gradosC; // grados centígrados

    public void AsignarCentígrados(float gC)
    {
        // Establecer el atributo grados centígrados
        gradosC = gC;
    }
}
```

```

public float ObtenerFahrenheit()
{
    // Retornar los grados fahrenheit equivalentes a gradosC
    return 9F/5F * gradosC + 32;
}

public float ObtenerCentígrados()
{
    return gradosC; // retornar los grados centígrados
}
}

```

El código anterior muestra que un objeto de la clase *CGrados* tendrá una estructura interna formada por el atributo:

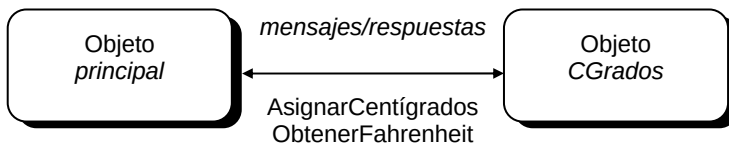
- *gradosC*, grados centígrados,

y una interfaz de acceso formada por los métodos:

- *AsignarCentígrados* que permite asignar a un objeto *CGrados* un valor en grados centígrados.
- *ObtenerFahrenheit* que permite retornar el valor grados *fahrenheit* equivalente a *gradosC* grados centígrados.
- *ObtenerCentígrados* que permite retornar el valor almacenado en el atributo *gradosC*.

Sin casi darnos cuenta estamos abstrayendo (separando por medio de una operación intelectual) los elementos naturales que intervienen en el problema a resolver y construyendo objetos que los representan.

Recordando lo visto anteriormente, un programa C# tiene que tener un objeto principal, que aporte un método **Main**, por donde empezará y terminará la ejecución del programa, además de otros que consideremos necesarios. ¿Cómo podemos imaginar esto de una forma gráfica? La figura siguiente da respuesta a esta pregunta:



Entonces, ¿qué tiene que hacer el objeto principal? Pues, visualizar cuántos grados *fahrenheit* son -30 C, -24 C,..., n grados centígrados,..., 96 C. Y, ¿cómo hace esto? Enviando al objeto *CGrados* los mensajes *AsignarCentígrados* y *ObtenerFahrenheit* una vez para cada valor desde -30 a 100 grados centígrados con

incrementos de 6. El objeto *CGrados* responderá ejecutando los métodos vinculados con los mensajes que recibe. Según esto, el código de la clase que dará lugar al objeto principal puede ser la siguiente:

```
using System;    // utilizar el espacio de nombres System

class CApGrados
{
    // Definición de constantes
    const int limInferior = -30;
    const int limSuperior = 100;
    const int incremento = 6;

    public static void Main(string[] args)
    {
        // Declaración de variables
        CGrados grados = new CGrados(); // objeto grados
        int gradosCent = limInferior;
        float gradosFahr = 0;

        while (gradosCent <= limSuperior) // mientras ... hacer:
        {
            // Asignar al objeto grados el valor en grados centígrados
            grados.AsignarCentígrados(gradosCent);
            // Obtener del objeto grados los grados fahrenheit
            gradosFahr = grados.ObtenerFahrenheit();
            // Escribir la siguiente línea de la tabla
            Console.WriteLine("{0, 8:d} C {1, 8:f2} F", gradosCent, gradosFahr);
            // Siguiente valor
            gradosCent += incremento;
        }
    }
}
```

Seguro que pensará que todo el proceso se podría haber hecho utilizando solamente el objeto principal, escribiendo todo el código en el método **Main**, lo cual es cierto. Pero, lo que se pretende es que pueda ver de una forma clara que, en general, un programa C# es un conjunto de objetos que se comunican entre sí mediante mensajes con el fin de obtener el resultado perseguido, y que la solución del problema puede resultar más sencilla cuando consiga realizar una representación del problema en base a los objetos naturales que se deducen de su enunciado. Piense que en la realidad se enfrentará a problemas mucho más complejos y, por lo tanto, la descomposición en objetos será vital para resolverlos.

Una vez analizado el problema, cree un nuevo proyecto desde su entorno de desarrollo y añada la clase *CGrados*. A continuación, añada la clase aplicación *CApGrados* en el mismo fichero fuente. Después, guarde el proyecto y, finalmente, compile y ejecute la aplicación.

No se preocupe si no entiende todo el código. Ahora lo que importa es que aprenda cómo es la estructura de un programa, no por qué se escriben unas u otras sentencias, cuestión que aprenderá más tarde en éste y en sucesivos capítulos. Este ejemplo le servirá como plantilla para inicialmente escribir sus propios programas. Posiblemente su primera aplicación utilice solamente un objeto aplicación, pero con este ejemplo tendrá un concepto más real de lo que es un programa C#.

En el ejemplo realizado podemos observar que un programa C# consta de:

- Directrices **using** (establece el ámbito donde localizar las clases de la biblioteca .NET utilizadas en nuestro programa).
- Una clase aplicación (la que incluye el método **Main**).
- Otras clases.

Sabemos también que una clase encapsula los atributos de los objetos que describe y los métodos para manipularlos. Pues bien, cada método consta de:

- Definiciones y/o declaraciones.
- Sentencias a ejecutar.

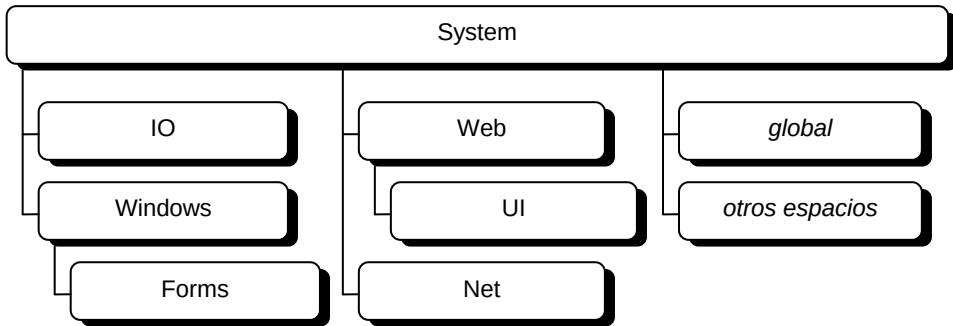
Los apartados que se exponen a continuación explican brevemente cada uno de estos componentes que aparecen en la estructura de un programa C#.

Espacios de nombres

Análogamente a como las carpetas o directorios ayudan a organizar los ficheros en un disco duro, los espacios de nombres ayudan a organizar las clases, evitando conflictos en el caso de clases de igual nombre pero de distintos fabricantes, ya que, supuestamente, se diferenciarán en el espacio de nombres al que pertenecen.

Según lo expuesto, un *espacio de nombres* define un ámbito, lo que proporciona una forma de crear tipos globalmente únicos. Esto permite agrupar las clases en tipos, denominados genéricamente *espacios de nombres*, en función de la labor que desempeñen. Por ejemplo, el espacio de nombres **System.IO** agrupa las clases que permiten a un programa realizar la entrada y salida de información; es como si las clases estuvieran dispuestas en paquetes, de hecho en otros lenguajes estos conjuntos de clases reciben el nombre de *paquetes*.

La biblioteca de clases de C# (biblioteca .NET) está organizada en espacios de nombres dispuestos jerárquicamente. En la figura siguiente se muestra un ejemplo de algunos de ellos:



Algunos de los espacios de nombres que observamos en la figura anterior son: *System*, *System.IO*, *System.Web*, *System.Net*, *System.Windows.Forms*, etc.

Como se puede observar, los espacios de nombres pueden estar anidados; por ejemplo, **IO** y **Web** están anidados en **System**.

Como hemos dicho, cada espacio de nombres hace referencia a un conjunto de clases relacionadas entre sí de alguna forma, o bien simplemente puede incluir otros espacios de nombres. Por ejemplo, el espacio de nombres **System** contiene las clases y estructuras de uso más frecuente, como son las estructuras que definen los tipos primitivos, la clase **Math** proveedora de los métodos matemáticos, la clase **Console** que aporta la funcionalidad para la entrada y salida estándar, etc. y anida otros espacios de nombres, como **IO**, **Web**, etc. A su vez, las clases, independientemente del espacio de nombres al que pertenezcan, forman una jerarquía cuya raíz es la clase **Object**.

Para crear un espacio de nombres se utiliza una declaración **namespace**. Por ejemplo, el código siguiente crearía el espacio de nombres **System** que empaqueta las clases A y B y el espacio de nombres **Windows**, que a su vez empaqueta el espacio **Forms** que incluye las clases C y D:

```
namespace System
{
    class A {}
    class B {}
    namespace Windows
    {
        namespace Forms
        {
            class C {}
            class D {}
        }
    }
}
```

El ejemplo anterior es equivalente a este otro:

```
namespace System
{
    class A {}
    class B {}
    namespace Windows.Forms
    {
        class C {}
        class D {}
    }
}
```

Para referirnos a una *clase* de un espacio de nombres, tenemos que hacerlo utilizando su nombre completo, excepto cuando el espacio de nombres haya sido declarado explícitamente, como veremos a continuación. Por ejemplo, **System.Console** hace referencia a la *clase* **Console** del espacio de nombres **System**; **System.Windows.Forms.Button** hace referencia a la *clase* **Button** del espacio de nombres **System.Windows.Forms**, etc.

Las clases que guardamos en un fichero cuando escribimos un programa pertenecen al espacio de nombres *global* (espacio sin nombre). Por ejemplo, las clases *CGrados* y *CApGrados* del programa anterior pertenecen, por omisión, a este espacio de nombres. De esta forma C# asegura que toda clase pertenece a un espacio de nombres. No obstante, podíamos haber hecho que la clase *CGrados* perteneciera a un espacio de nombres, por ejemplo *MisClases*, así:

```
namespace MisClases
{
    class CGrados
    {
        // ...
    }
}
```

Los espacios de nombres tienen acceso público, característica que no es modificable.

Protección de una clase

La protección de una clase determina la relación que tiene con otras clases de otros proyectos. Distinguimos dos niveles de protección: *interno* (**internal**) y *público* (**public**). Una clase con nivel de protección *interno* sólo puede ser utilizada por las clases de su mismo proyecto (no está disponible para otros proyectos). En cambio, una clase *pública* puede ser utilizada por cualquier otra clase de otro

proyecto. ¿Qué se entiende por utilizar? Que una clase puede crear objetos de otra clase y manipularlos utilizando sus métodos.

Por ejemplo, supongamos que editamos las clases *clase1* y *clase2* como se indica a continuación y las almacenamos, respectivamente, en los ficheros *clase1.cs* y *clase2.cs*:

```
public class Clase1 // clase pública
{
    // ...
}

internal class Clase2 // clase interna
{
    // ...
}
```

A continuación las compilamos y las guardamos en la biblioteca *misClases.dll*:

```
csc /target:library /out:misClases.dll clase1.cs clase2.cs
```

Si ahora realizamos un programa como el siguiente,

```
class Test
{
    public static void Main(string[] args)
    {
        Clase1 objC1 = new Clase1();
        Clase2 objC2 = new Clase2();
    }
}
```

y lo compilamos incluyendo la biblioteca *misClases.dll* que hemos creado para que pueda utilizar las clases *Clase1* y *Clase2*,

```
csc /reference:misClases.dll Test.cs
```

observaremos que se nos muestra un error que indica que la *Clase2* es inaccesible debido a su nivel de protección; se trata de una clase de uso *interno* (**internal**). No sucede lo mismo con *clase1*, puesto que es una clase de uso *público* (**public**).

Por omisión una clase tiene el nivel de protección *interno*; por ejemplo, la clase *Test* de este ejemplo, o la clase *CGrados* del ejemplo anterior, tiene este nivel de protección. En cambio, cuando se desea que una clase tenga protección *pública*, hay que calificarla como tal utilizando la palabra reservada **public**. Echando un vistazo a la documentación de la biblioteca .NET, se puede observar

que la clase **Console** del espacio **System** es pública, razón por la cual se ha podido utilizar en el programa *CAPGrados*.

Directriz using

Una clase de un determinado espacio de nombres puede hacer uso de otra clase de otro espacio de dos formas:

1. Utilizando su nombre completo en todas las partes del código donde haya que referirse a ella. Por ejemplo:

```
System.Console.WriteLine("{0, 8:d} C {1, 8:f2} F", gradosCent, gradosFahr);
```

2. Indicando al compilador el espacio de nombres donde está la clase, lo que posibilita referirse a ella simplemente por su nombre. Para ello utilizaremos la directriz **using**. Por ejemplo:

```
using System; // utilizar el espacio de nombres System

class CAPGrados
{
    // ...
    Console.WriteLine("{0, 8:d} C {1, 8:f2} F", gradosCent, gradosFahr);
    // ...
}
```

En un programa C# puede aparecer cualquier número de directrices **using**, las cuales deben escribirse antes de cualquier otro código; en otras palabras, deben aparecer al principio.

Como se puede comprobar en el ejemplo anterior, declarar un espacio de nombres permite al programa referirse a sus clases más tarde sin utilizar el nombre del espacio. Esto es, la directriz **using** sólo indica al compilador de C# dónde encontrar las clases, no trae nada dentro del programa actual.

En el caso concreto del ejemplo expuesto, si eliminamos la directriz *using System*, el compilador mostrará un error para indicar que no puede encontrar la clase **Console**.

Especificación de alias

Aún en el caso de que utilizemos espacios de nombres distintos para diferenciar clases con igual nombre pero procedentes de distintos fabricantes, pueden darse conflictos si usamos sentencias **using** para importar esos espacios de nombres, ya

que al hacer referencia a una de esas clases comunes sólo por su nombre, el compilador no podrá determinar a cuál de ellas nos referimos. Por ejemplo, si tenemos una clase, *ClaseA*, procedente de distintos fabricantes, *EspacioN.ClaseA* y *EspacioM.ClaseA*, y definimos una clase derivada *ClaseB* de *ClaseA*, ¿cómo sabrá el compilador si lo que queremos es derivar de *EspacioN.ClaseA* o de *EspacioM.ClaseA*? Pues no podrá determinarlo y generará un error informando que existe en el código una referencia ambigua a *ClaseA*.

```
using EspacioN; // define EspacioN.ClaseA
using EspacioM; // define EspacioM.ClaseA

class ClaseB : ClaseA
{
}
```

Para resolver este tipo de ambigüedades, bastaría con hacer referencia a la clase por su nombre completo (incluyendo el espacio de nombres), pero esta forma de proceder puede llegar a ser tediosa, sobre todo si el nombre es muy largo. Para solucionarlo, C# permite definir alias para cualquier tipo de dato utilizando la sintaxis siguiente:

```
using alias = NombreCompletoTipo|EspacioNombres;
```

Estas definiciones de alias deben incluirse al principio de las definiciones. Por ejemplo, el problema presentado anteriormente podría resolverse así:

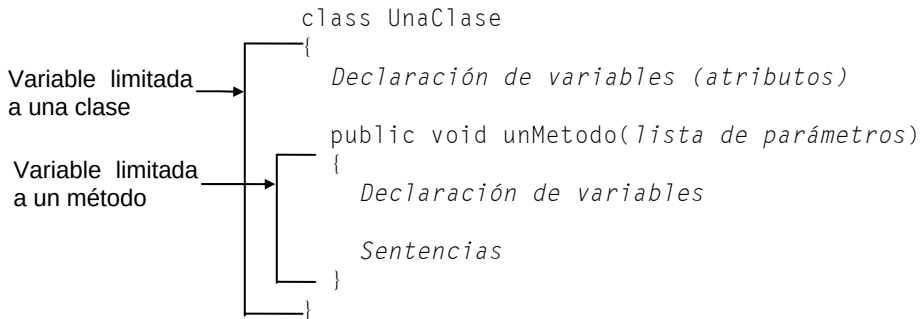
```
using EspacioN;
using EspacioM;
using ClaseA = EspacioN.ClaseA;

class ClaseB : ClaseA
{
}
```

Los alias no tienen porqué ser sólo referentes a tipos, sino que también pueden ser alias de espacios de nombres.

Definiciones y declaraciones

La declaración o la definición de una variable pueden realizarse a *nivel de la clase* (atributos de la clase) o a *nivel del método* (dentro de la definición de un método). Pero, la definición de un método siempre ocurre a nivel de la clase (nota: una declaración es una definición, a menos que no haya asignación de memoria).



En un método, las definiciones o declaraciones se pueden realizar en cualquier lugar; o mejor dicho, en el lugar justo donde se necesiten y no necesariamente al principio del método, antes de todas las sentencias.

Sentencia simple

Una *sentencia simple* es la unidad ejecutable más pequeña de un programa C#. Las sentencias controlan el flujo u orden de ejecución. Una sentencia C# puede formarse a partir de: una palabra clave (**for**, **while**, **if ... else**, etc.), expresiones, declaraciones o llamadas a métodos. Cuando se escriba una sentencia hay que tener en cuenta las siguientes consideraciones:

- Toda sentencia simple termina con un punto y coma (;).
- Dos o más sentencias pueden aparecer sobre una misma línea, separadas una de otra por un punto y coma, aunque esta forma de proceder no es aconsejable porque va en contra de la claridad que se necesita cuando se lee el código de un programa.
- Una sentencia nula consta solamente de un punto y coma. Cuando veamos la sentencia **while**, podrá ver su utilización.

Sentencia compuesta o bloque

Una *sentencia compuesta* o bloque es una colección de sentencias simples incluidas entre llaves – { } –. Un bloque puede contener a otros bloques. Un ejemplo de una sentencia de este tipo es el siguiente:

```

{
    grados.AsignarCentigrados(gradosCent);
    gradosFahr = grados.ObtenerFahrenheit();
    Console.WriteLine("{0, 8:d} C {1, 8:f2} F", gradosCent, gradosFahr);
    gradosCent += incremento;
}

```

Un bloque permite escribir múltiples sentencias donde se permite únicamente una sentencia.

Métodos

Un método es una colección de sentencias que ejecutan una tarea específica. En C#, un método siempre pertenece a una clase y su definición nunca puede contener a la definición de otro método; esto es, C# no permite métodos anidados.

Definición de un método

La definición de un método consta de una *cabecera* y del *cuerpo* del método encerrado entre llaves. La sintaxis para escribir un método es la siguiente (las expresiones entre corchetes son opcionales):

```
[modificador] tipo-resultado nombre-método ([lista de parámetros])
{
    declaraciones de variables locales;
    sentencias;
    [return [(expresión)];]
}
```

Las variables declaradas en el cuerpo del método son locales a dicho método y por definición solamente son accesibles dentro del mismo.

Un *modificador* es una palabra clave que modifica el nivel de protección predeterminado del método. Véase el apartado *Protección de los miembros de una clase* expuesto un poco más adelante.

El *tipo del resultado* especifica qué tipo de valor retorna el método. Éste puede ser cualquier tipo primitivo o referenciado. Para indicar que no se devuelve nada, se utiliza la palabra reservada **void**. El resultado de un método es devuelto a la sentencia que lo invocó, por medio de la siguiente sentencia:

```
return [(expresión)];
```

La sentencia **return** puede ser o no la última y puede aparecer más de una vez en el cuerpo del método. En el caso de que el método no retorne un valor (**void**), se puede omitir o especificar simplemente **return**. Por ejemplo:

```
void mEscribir()
{
    // ...
    return;
}
```

La *lista de parámetros* de un método son las variables que reciben los valores de los argumentos especificados cuando se invoca al mismo. Consiste en una lista de cero, uno o más identificadores con sus tipos, separados por comas. A continuación se muestra un ejemplo:

```
public double Sumar(double x, double y)
{
    // ...
}
```

Método Main

Todo programa C# tiene un método denominado **Main**, y sólo uno. Este método es el punto de entrada al programa y también el punto de salida. Su definición es la que se muestra a continuación:

```
static void Main(string[] args)
{
    // Cuerpo del método
}
```

Como se puede observar, el método **Main** es estático (**static**), no devuelve nada (**void**) y tiene un argumento de tipo **string[]** que almacenará los argumentos pasados en la línea de órdenes cuando se invoca al programa para su ejecución, concepto que estudiaremos posteriormente en otro capítulo.

El método **Main** puede devolver también un **int** y puede escribirse sin parámetros, aunque, por compatibilidad con otros lenguajes, en esta obra utilizaremos el formato expuesto. En cuanto al nivel de accesibilidad, por tratarse del punto de entrada al programa, la máquina virtual .NET lo único que verifica cuando se inicia el programa es que entre los métodos estáticos (**static**), públicos (**public**) o no públicos, haya uno denominado **Main**.

Para más detalles sobre los niveles de accesibilidad, puede ver un poco más adelante los apartados *Protección de los miembros de una clase* y *Miembro de un objeto o de una clase*.

Crear objetos de una clase

Sabemos que las clases son plantillas para crear objetos. Pero, ¿cómo se crea un objeto? Para crear un objeto de una clase hay que utilizar el operador **new**, análogamente a como muestra el ejemplo siguiente:

```
CGrados grados = new CGrados();
```

En este ejemplo se observa que para crear un objeto de la clase *CGrados* hay que especificar a continuación del operador **new** el nombre de la clase del objeto seguido de paréntesis. ¿Por qué paréntesis? ¿Es acaso *CGrados* un método? Así es. Recuerde que toda clase tiene al menos un método predeterminado especial denominado igual que ella, sin parámetros, que se invoca automáticamente cada vez que se crea un objeto; ese método se denomina *constructor* de la clase.

Cuando se crea un nuevo objeto utilizando **new**, C# asigna automáticamente la cantidad de memoria necesaria para ubicar ese objeto. Si no hubiera suficiente espacio de memoria disponible, el operador **new** lanzará una excepción **System.OutOfMemoryException** cuyo estudio posponemos. Después de saber esto quizás se pregunte: ¿quién libera esa memoria y cuándo lo hace? La respuesta es otra vez la misma: C# se encarga de hacerlo en cuanto el objeto no se utilice, cosa que ocurre cuando ya no existe ninguna referencia al mismo. Por ejemplo, en el código que se muestra a continuación, la memoria asignada al objeto *grados* será liberada cuando finalice la ejecución del método **Main**.

```
public static void Main(string[] args)
{
    CGrados grados = new CGrados(); // objeto grados
    // ...
}
```

Ahora basta con que sepa que C# cuenta con una herramienta denominada *recolector de basura* que busca objetos que no se utilizan con el fin de destruirlos liberando la memoria que ocupan.

Cómo acceder a los miembros de un objeto

Para acceder desde un método de la clase principal o de cualquier otra clase a un miembro (atributo, propiedad o método) de un objeto de otra clase diferente se utiliza la sintaxis siguiente: *objeto.miembro*. Por ejemplo:

```
miObjeto.atributo;
miObjeto.propiedad;
miObjeto.metodo([argumentos]);
```

Lógicamente, como pueden existir varios objetos de la misma clase, es necesario especificar de quién es el miembro. Si el miembro es a su vez un objeto, la sintaxis se extiende de igual forma: *objeto.mbroObjeto.miembro*. Recuerde que el operador punto (.) se evalúa de izquierda a derecha.

Cuando el miembro accedido es un método, la interpretación que se hace en programación orientada a objetos es que el objeto ha recibido un mensaje, el es-

pecificado por el nombre del método, y responde ejecutando ese método. Los mensajes que puede recibir un objeto se corresponden con los nombres de los métodos de su clase. Por ejemplo, una sentencia como:

```
grados.AsignarCentígrados(gradosCent);
```

se interpreta como que el objeto *grados* recibe el mensaje *AsignarCentígrados*. Entonces el objeto responde a ese mensaje ejecutando el método de su clase que tenga el mismo nombre. Lógicamente, como el método se ejecuta para un objeto concreto, el cuerpo del mismo no necesita especificar explícitamente de qué objeto es el miembro accedido (véase **this** en el apartado *Herencia* del capítulo 3). Esto es, en el ejemplo siguiente se sabe que *gradosC* pertenece al objeto que está respondiendo al mensaje *AsignarCentígrados*.

```
public void AsignarCentígrados(float gC)
{
    // Establecer el atributo grados centígrados
    gradosC = gC;
}
```

Es importante asimilar que un programa orientado a objetos sólo se compone de objetos que se comunican mediante mensajes. Desde este conocimiento, no tiene sentido pensar que un método se pueda invocar aisladamente, esto es, sin que exista un objeto para el que es invocado. Por ejemplo, si en el método **Main** de nuestro programa ejemplo pudiéramos escribir:

```
AsignarCentígrados(gradosCent);
```

seguro que nos preguntaríamos: ¿a quién se asigna el valor *gradosCent*? Los métodos **static** que estudiaremos más tarde son una excepción a la regla.

Protección de los miembros de una clase

Los miembros de una clase son los atributos, las propiedades y los métodos, y su nivel de protección determina quién puede acceder a los mismos. Los niveles de protección a los que nos referimos son: *público*, *privado* y *protegido*. De éste último y de otros hablaremos en un capítulo posterior.

Por ejemplo, en la clase *CGrados* del programa realizado al principio de este capítulo, hemos definido los atributos privados y los métodos públicos:

```
class CGrados
{
    private float gradosC; // grados centígrados
```

```
public void AsignarCentígrados(float gC)
{
    // Establecer el atributo grados centígrados
    gradosC = gC;
}
// ...
}
```

Un miembro de una clase declarado *privado* puede ser accedido únicamente por los métodos de su clase. En el ejemplo anterior se puede observar que el atributo *gradosC* es privado y es accedido por el método *AsignarCentígrados*.

Si un método de otra clase, por ejemplo el método **Main** de la clase *CApGrados*, incluyera una sentencia como la siguiente:

```
grados.gradosC = 30;
```

el compilador C# mostraría un error que indicaría que el miembro *gradosC* no es accesible desde esta clase, por tratarse de un miembro privado de *CGrados*.

Un miembro de una clase declarado *público* es accesible desde cualquier método definido en su clase o en otra clase. Por ejemplo, en la clase *CApGrados*, se puede observar cómo el objeto *grados* de la clase *CGrados* creado en el método **Main** accede a su método *AsignarCentígrados* con el fin de modificar el valor de su miembro privado *gradosC*.

```
class CApGrados
{
    // ...
    public static void Main(string[] args)
    {
        // Declaración de variables
        CGrados grados = new CGrados();
        // ...
        while (gradosCent <= limSuperior) // mientras ... hacer:
        {
            // Asignar al objeto grados el valor en grados centígrados
            grados.AsignarCentígrados(gradosCent);
            // ...
        }
    }
}
```

Generalmente los atributos de una clase de objetos se declaran privados, estando así ocultos para otras clases, siendo posible el acceso a los mismos únicamente a través de los métodos públicos de dicha clase. El mecanismo de ocultación de miembros se conoce en la programación orientada a objetos como

encapsulación: proceso de ocultar la estructura interna de datos de un objeto y permitir el acceso sólo a través de la interfaz pública definida, entendiendo por interfaz pública el conjunto de miembros públicos de una clase. ¿Qué beneficios reporta la encapsulación? Que un usuario de una determinada clase no pueda escribir código en base a la estructura interna del objeto, sino sólo en base a la interfaz pública; esta forma de proceder obliga a pensar en objetos y a trabajar con ellos. En el capítulo *Clases, espacios de nombres y estructuras* que expondremos más adelante abundaremos más sobre lo dicho y sobre otras muchas cuestiones.

El nivel de protección predeterminado para un miembro de una clase es el *privado*. Recuerde que un miembro de una clase con este nivel de protección no necesita utilizar ningún modificador.

Miembro de un objeto o de una clase

Sabemos que una clase agrupa los atributos y los métodos que definen a los objetos de esa clase. Pero, cada objeto que creamos de esa clase, ¿mantiene una copia tanto de los atributos como de los métodos? Lógicamente, cada objeto mantiene su propia copia de los atributos para almacenar sus datos particulares; pero, de los métodos sólo hay una copia para todos los objetos, lo cual también es lógico, porque cada objeto sólo requiere utilizarlos; por ejemplo, cuando necesite modificar sus atributos. Desde este análisis se dice que los miembros son del objeto; esto es, un mismo atributo tiene un valor específico para cada objeto, y un objeto ejecuta un método en respuesta a un mensaje recibido.

Esta forma de concebir los objetos puede suponer, en ocasiones, un desperdicio de espacio de almacenamiento; por ejemplo, volviendo a la clase *CCuentaAhorro* que expusimos en el capítulo 3, si suponemos que la cuota de mantenimiento es la misma para todas las cuentas de esta clase, sería más eficiente definir el atributo *cuotaMantenimiento* común a todos los objetos de la misma. En este caso, diremos que el atributo *es de la clase* de los objetos, no del objeto, y existe aunque no haya objetos definidos de la clase. Por lo tanto, un *atributo de la clase* almacena información común a todos los objetos de esa clase; esto es, cada objeto no mantiene una copia del mismo, sino que todos comparten una única copia.

¿Cómo se declara que un atributo es de la clase? Para especificar que un atributo va a ser compartido por todos los objetos de su clase, en su declaración hay que anteponer la palabra reservada **static** al nombre del mismo.

Análogamente, un método declarado **static** es un método de la clase, por lo tanto no se ejecuta para un objeto particular, sino que se utiliza para actuar sobre un atributo **static** declarado privado, o bien para realizar alguna operación genérica al margen de los objetos de la clase. Veamos un ejemplo a continuación:

```
class CCuentaAhorro : CCuenta
{
    // Atributos
    private static double cuotaMantenimiento;

    // Métodos
    public CCuentaAhorro() {} // constructor sin parámetros

    public CCuentaAhorro(string nom, string cue, double sal,
        double tipo, double mant) :
        base(nom, cue, sal, tipo) // invoca al constructor CCuenta,
                                // esto es, al de la clase base
    {
        asignarCuotaManten(mant); // inicia cuotaMantenimiento
    }

    public static void asignarCuotaManten(double cantidad)
    {
        if (cantidad < 0)
        {
            System.Console.WriteLine("Error: cantidad negativa");
            return;
        }
        cuotaMantenimiento = cantidad;
    }

    public static double obtenerCuotaManten()
    {
        return cuotaMantenimiento;
    }

    public void reintegro(double cantidad)
    {
        // ...
    }
}
```

Para acceder a un miembro **static** de una clase hay que utilizar el nombre de la clase, como puede verse en el ejemplo siguiente, lo cual es lógico porque nos estamos refiriendo no a un objeto en particular de dicha clase, sino a todos los objetos que el programa haya creado de la misma:

```
class Test
{
    public static void Main(string[] args)
    {
        CCuentaAhorro cuenta01 = new CCuentaAhorro(
            "Un nombre", "Una cuenta", 1000000, 3.5, 300);

        // Cobrar cuota de mantenimiento
    }
}
```

```

    cuenta01.reintegro(CCuentaAhorro.obtenerCuotaManten());
    // ...
}
}

```

En el ejemplo anterior se puede observar que para obtener el valor del atributo privado *cuotaMantenimiento* se ha utilizado un método estático. Si se hubiera utilizado un método no **static**, éste tendría que ser invocado a través de un objeto de su clase, lo que, siendo correcto, resultaría engañoso por tratarse de un atributo de la clase y no del objeto.

El método *obtenerCuotaManten* del ejemplo anterior puede acceder a *cuotaMantenimiento* porque es un miembro estático pero no podría incluir, por ejemplo, una sentencia como *obtenerTipoDeInterés()* porque *obtenerTipoDeInterés* no es un miembro **static**.

Ahora puede comprender por qué el método **Main** es **static**: para que pueda ser invocado, en este caso por el sistema, aunque no exista un objeto de su clase. Y también puede comprender por qué para llamar al método **WriteLine** de la clase **Console** de la biblioteca .NET, se utiliza la sintaxis:

```
System.Console.WriteLine( ... );
```

Porque echando un vistazo a la documentación de C#, observamos que el método **WriteLine** miembro de la clase **Console** del espacio de nombres **System** es público y **static**.

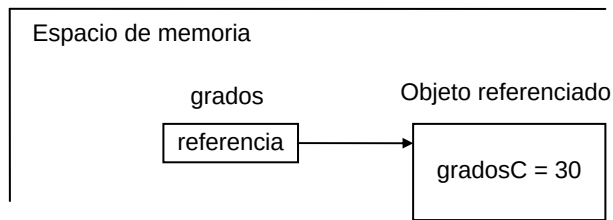
Referencias a objetos

Según lo que hemos aprendido hasta ahora, para crear un objeto de una clase hay que hacerlo explícitamente utilizando el operador **new**. Por ejemplo:

```
CGrados grados = new CGrados();
```

El operador **new** devuelve una referencia al nuevo objeto, que se almacena en una variable del tipo del objeto. En el ejemplo anterior, la referencia devuelta por el operador **new** es almacenada en la variable *grados* del tipo *CGrados*. La clase *CGrados* se encuadra dentro de lo que hemos denominado tipos *referencia*.

Gráficamente puede imaginarse una referencia y el objeto referenciado, ubicados en algún lugar del espacio de memoria correspondiente a su programa, así:



En realidad, una referencia es la posición de memoria donde se localiza un objeto. Observará que anteriormente nos hemos referido a la referencia *grados* como el objeto *grados*. Esto es una forma de abreviar que no crea confusión, ya que *grados* es una variable única y referencia un único objeto *CGrados*. Una expresión como “el objeto referenciado por la variable *grados*” resulta demasiado larga y no aporta más información. Expresándonos en estos términos, cuando se asigne un objeto a otro, o bien se pasen objetos como argumentos a métodos, lo que se está copiando son referencias, no el contenido de los objetos.

El siguiente ejemplo aclarará este concepto. Se trata de otra versión del programa *Racionales* que expusimos al final del capítulo 3, basada en la clase *CRacional* a la que hemos añadido dos constructores (uno sin parámetros y otro con dos parámetros) y un nuevo método estático que permite sumar dos números racionales, devolviendo como resultado el número racional resultante de la suma.

```
public class CRacional
{
    // Atributos
    private int Numerador;
    private int Denominador;

    // Constructores
    public CRacional() {}

    public CRacional(int num, int den)
    {
        AsignarDatos(num, den);
    }

    // Métodos
    public void AsignarDatos(int num, int den)
    {
        Numerador = num;
        if (den == 0) den = 1; // el denominador no puede ser cero
        Denominador = den;
    }

    public void VisualizarRacional()
    {

```

```

        System.Console.WriteLine(Numerador + "/" + Denominador);
    }

    public static CRacional Sumar(CRacional a, CRacional b)
    {
        CRacional r = new CRacional(); // crear un objeto CRacional
        int num = a.Numerador * b.Denominador +
                a.Denominador * b.Numerador;
        int den = a.Denominador * b.Denominador;
        r.AsignarDatos(num, den);
        return r;
    }
}

```

La clase *CRacional* encapsula una estructura de datos formada por dos enteros: *Numerador* y *Denominador*; y para acceder a esta estructura, además de los constructores, proporciona la interfaz pública formada por los métodos:

- *AsignarDatos*, que permite establecer el numerador y el denominador de un número racional.
- *VisualizarRacional*, que permite visualizar un racional en forma de quebrado.
- *Sumar*, que devuelve el número racional resultante de sumar otros dos pasados como argumentos.

Seguidamente, escriba la clase *Racionales* que se muestra a continuación y guárdela en el fichero *Racionales.cs*

```

class Racionales
{
    public static void Main(string[] args)
    {
        // Punto de entrada al programa
        CRacional r1, r2;
        r1 = new CRacional(2, 5); // crear un objeto CRacional
        r2 = r1;

        r1.AsignarDatos(3, 7);
        r1.VisualizarRacional(); // se visualiza 3/7
        r2.VisualizarRacional(); // se visualiza 3/7

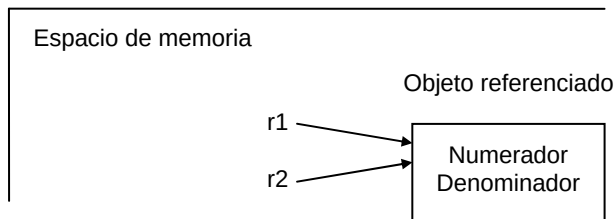
        CRacional r3;
        r2 = new CRacional(2, 5); // crear un objeto CRacional
        r3 = CRacional.Sumar(r1, r2); // r3 = 3/7 + 2/5
        r3.VisualizarRacional(); // se visualiza 29/35
    }
}

```

Compile esta aplicación formada por los ficheros *Racionales.cs* y *CRacional.cs* y observe los resultados.

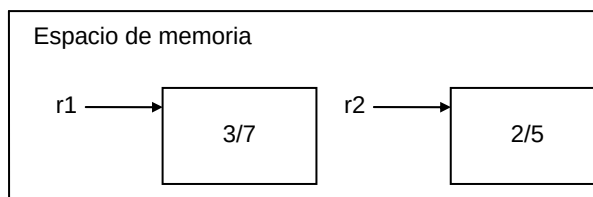
Analizada la clase *CRacional* pasemos a estudiar el método **Main** de la clase *Racionales*. La primera parte de este método declara dos variables *r1* y *r2* de tipo *CRacional*, crea un nuevo objeto *r1* de tipo *CRacional* asignándole el valor 2/5 (se invoca al constructor con dos parámetros) y asigna el valor de *r1* a *r2*.

A continuación, asigna a *r1* un nuevo valor 3/7, ¿cuál es el valor de *r2*? Comprobamos que es el mismo que el de *r1*. ¿Qué ha ocurrido? Que cuando se asignó *r1* a *r2*, simplemente se creó una nueva referencia al mismo objeto referenciado por *r1*. Por lo tanto, modificar el objeto al que se refiere *r1* es modificar el objeto al que se refiere *r2* porque *r1* y *r2* referencian el mismo objeto.



Si realmente lo que deseamos es que *r1* y *r2* señalen a objetos diferentes, hay que utilizar **new** con ambas referencias para crear objetos separados:

```
r1 = new CRacional(3, 7); // crear un objeto CRacional r1
r2 = new CRacional(2, 5); // crear un objeto CRacional r2
```



Como hemos visto, una variable que sea una referencia a un objeto se puede asignar a otra del mismo tipo, pero esta operación no duplica el objeto, sino que dicho objeto queda referenciado por esas dos referencias.

Pasando argumentos a los métodos

La última parte del método **Main** del ejemplo anterior crea un objeto *r2* y le asigna el valor 2/5. A continuación, invoca al método estático *Sumar* pasándole como

argumentos los objetos *r1* y *r2* que queremos sumar; cuando se invoca a un método, el primer argumento es pasado al primer parámetro, el segundo argumento es pasado al segundo parámetro y así sucesivamente. El resultado devuelto por *Sumar* será un objeto *CRacional* que quedará referenciado por *r3*.

```
// ...
CRacional r3;
r2 = new CRacional(2, 5);      // crear un objeto CRacional
r3 = CRacional.Sumar(r1, r2);  // r3 = 3/7 + 2/5
r3.VisualizarRacional();      // se visualiza 29/35
```

Analicemos el método *Sumar*. Este método tiene dos parámetros *a* y *b* de tipo *CRacional*. Después de que el método ha sido invocado desde **Main**, *a* y *b* señalan a los mismos objetos que *r1* y *r2*. Esto significa que los objetos pasados a los parámetros de un método son siempre referencias a dichos objetos, lo cual quiere decir que cualquier modificación que realice el método sobre esos objetos la está haciendo sobre los objetos originales. En cambio, las variables de alguno de los tipos primitivos estudiados en el capítulo 4 (**int**, **float**, **double**, etc.) pasan por valor por tratarse de estructuras, lo cual significa que se pasa una copia, por lo que cualquier modificación que se haga a esas variables dentro del método no afecta a la variable original; no obstante, C# también permite pasar este tipo de variables por referencia, cuestión que estudiaremos en el capítulo 9.

```
public static CRacional Sumar(CRacional a, CRacional b)
{
    CRacional r = new CRacional(); // crear un objeto CRacional
    int num = a.Numerador * b.Denominador +
              a.Denominador * b.Numerador;
    int den = a.Denominador * b.Denominador;
    r.AsignarDatos(num, den);
    return r;
}
```

A continuación, el método *Sumar* utiliza **new** para crear un nuevo objeto *r* (se invoca al constructor sin parámetros) al que asigna el resultado de la suma de los objetos *a* y *b*. Finalmente devuelve *r*. Una vez más lo que se devuelve es una referencia que se copia en *r3*. Finalizado este proceso la variable *r* desaparece por ser local; no sucede lo mismo con el objeto que señalaba, ya que ahora está señalado por *r3*.

El recolector de basura de C# sólo eliminará un objeto cuando no exista ninguna referencia al mismo.

Conversión entre tipos referencia

Con expresiones de algún tipo *referencia* las conversiones explícitas pueden realizarse utilizando una expresión *cast* (véase el capítulo 4), o bien utilizando el operador **as**. La sintaxis de este operador es:

expresión as tipo referencia

A diferencia de la expresión *cast*, el operador **as** nunca lanzará una excepción. En lugar de ello, si la conversión indicada no es posible, el valor resultante será **null**. Por ejemplo:

```
void m(object obj)
{
    string s = obj as string;
    // ...
}
```

EJERCICIOS RESUELTOS

Vamos a realizar otra versión del programa C# que presentaba una tabla de equivalencia entre grados centígrados y grados *fahrenheit*, resuelto al principio de este capítulo. Recuerde que la relación entre los grados centígrados y los grados *fahrenheit* viene dada por la expresión $\text{grados fahrenheit} = 9/5 * \text{grados centígrados} + 32$ y que los cálculos los realizamos para un intervalo de -30 a 100 grados centígrados con incrementos de 6.

Ahora todo el proceso lo vamos a hacer utilizando solamente el objeto principal; esto es, escribiremos una clase *CApGrados* que incluya el método **Main** y otro método con un parámetro que almacene los grados centígrados a convertir, y devuelva como resultado el valor equivalente en grados *fahrenheit*. El programa completo podría ser similar al siguiente:

```
/*
 * Conversión de grados centígrados a fahrenheit:
 * F = 9/5 * C + 32
 */

class CApGrados
{
    // Definición de constantes
    const int limInferior = -30;
    const int limSuperior = 100;
    const int incremento = 6;
```



```

public static float ObtenerFahrenheit(float gradosC)
{
    // Retornar los grados fahrenheit equivalentes a gradosC
    return 9F/5F * gradosC + 32;
}

public static void Main(string[] args)
{
    // Declaración de variables
    int gradosCent = limInferior;
    float gradosFahr = 0;

    while (gradosCent <= limSuperior) // mientras ... hacer:
    {
        // Obtener los grados fahrenheit equivalentes a gradosCent
        gradosFahr = ObtenerFahrenheit(gradosCent);
        // Escribir la siguiente línea de la tabla
        System.Console.WriteLine("{0, 8:d} C {1, 8:f2} F", gradosCent, gradosFahr);
        // Siguiendo valor
        gradosCent += incremento;
    }
}

```

Analicemos el código escrito. La clase *CApGrados* define tres constantes, *limInferior*, *limSuperior* e *incremento*, y dos métodos, *ObtenerFahrenheit* y **Main**. Ambos métodos son **static** lo que permite invocarlos sin necesidad de que exista un objeto de la clase *CApGrados*. ¿Por qué para invocar al método *ObtenerFahrenheit* no se ha empleado la sintaxis *CApGrados.ObtenerFahrenheit(gradosCent)*? Porque el método *ObtenerFahrenheit* pertenece a la misma clase que **Main**, esto es, ambos están en el mismo ámbito, en el de la clase *CApGrados*.

Es evidente que esta forma de implementar un programa puede resultar más sencilla porque no nos obliga a pensar en objetos, pero utilizando un lenguaje orientado a objetos es justamente lo que no debemos hacer. No obstante, para programas sencillos puede ser un modelo útil.

¿Qué cambios habría que realizar si el método *ObtenerFahrenheit* no hubiera sido declarado **static**? En este caso, el método **Main** tendría que definir un objeto de la clase *CApGrados* como se muestra a continuación:

```

public static void Main(string[] args)
{
    CApGrados grados = new CApGrados();
    // ...
    while (gradosCent <= limSuperior) // while ... hacer:
    {
        // Obtener los grados fahrenheit equivalentes a gradosCent
    }
}

```

```
    gradosFahr = grados.ObtenerFahrenheit(gradosCent);  
    // ...  
}
```

EJERCICIOS PROPUESTOS

1. Modifique los límites inferior y superior de los grados centígrados, el incremento, y ejecute de nuevo el programa.
2. Cargue en su entorno de desarrollo integrado el programa *CAppGrados.cs* y modifique la sentencia:

```
return 9F/5F * gradosC + 32;
```

correspondiente al método *ObtenerFahrenheit* de la clase *CGrados*, como se muestra a continuación:

```
return 9/5 * gradosC + 32;
```

Después, compile y ejecute el programa. Explique lo que sucede.

3. Reconstruya el programa *CAppGrados.cs* para que ambas clases, *CGrados* y *CAppGrados* estén almacenadas en un único fichero: *CAppGrados.cs*.

CLASES DE USO COMÚN

Aunque las clases que hemos aprendido a escribir en los capítulos anteriores son la base de nuestras aplicaciones, la potencia, en la práctica, del lenguaje C# viene dada por su biblioteca de clases: la biblioteca .NET Framework. Hay un espacio de nombres que destaca por las clases de propósito general que incluye: **System**.

El espacio de nombres **System** contiene clases que se aplican al lenguaje mismo. Por ejemplo, estructuras que encapsulan los tipos primitivos de datos, la clase **Console** que proporciona los métodos para manipular la entrada/salida (E/S) estándar, la clase **String** para manipular cadenas de caracteres, una clase **Math** que proporciona los métodos correspondientes a las funciones matemáticas de uso más frecuente, la clase **Convert** que proporciona los métodos para convertir un tipo predefinido en otro, etc.

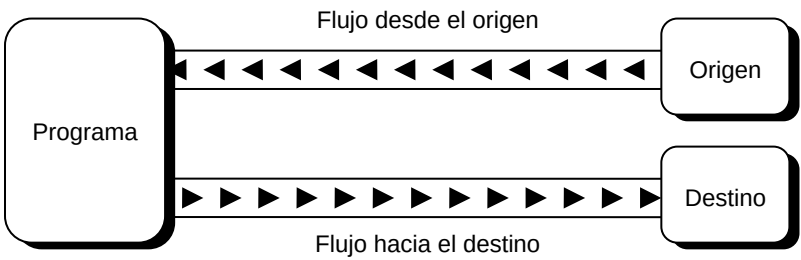
La entrada/salida queda reforzada con las clases aportadas por el espacio de nombres **System.IO** para el manejo de ficheros.

Por otra parte, la finalidad de una aplicación es procesar datos que, generalmente, serán obtenidos de algún medio externo por la propia aplicación (por ejemplo, del teclado o de un fichero en disco) y procesados por la misma con el fin de obtener unos resultados para ser mostrados o almacenados en otro medio externo. En el capítulo 2 vimos una introducción a este tema. Pues bien, en este otro capítulo aprenderá cómo leer y escribir información desde sus aplicaciones utilizando los dispositivos estándar (teclado y pantalla), y a trabajar con las clases utilizadas más frecuentemente.

ENTRADA Y SALIDA

Frecuentemente un programa necesitará obtener información desde un origen o enviar información a un destino. Por ejemplo, obtener información desde el teclado, o bien enviar información a la pantalla. La comunicación entre el origen de cierta información y el destino, se realiza mediante un *flujo* de información (en inglés *stream*).

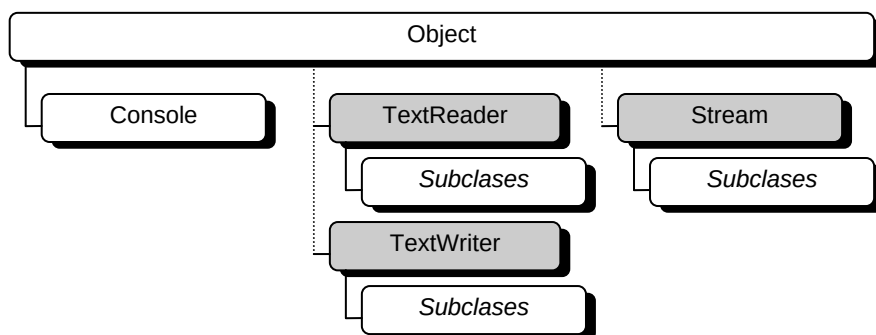
Un *flujo* es un objeto que hace de intermediario entre el programa y el origen o el destino de la información. Esto es, el programa leerá o escribirá en el *flujo* sin importarle desde dónde viene la información o a dónde va y tampoco importa el tipo de los datos que se leen o escriben. Este nivel de abstracción hace que el programa no tenga que saber nada ni del dispositivo ni del tipo de información, lo que se traduce en una facilidad más a la hora de escribir programas.



Entonces, para que un programa pueda obtener información desde un origen tiene que abrir un flujo y leer la información. Análogamente, para que un programa pueda enviar información a un destino tiene que abrir un flujo y escribir la información. Los algoritmos para leer y escribir datos son siempre más o menos los mismos:

Leer	Escribir
<i>Abrir un flujo desde un origen</i>	<i>Abrir un flujo hacia un destino</i>
<i>Mientras haya información</i>	<i>Mientras haya información</i>
<i>Leer información</i>	<i>Escribir información</i>
<i>Cerrar el flujo</i>	<i>Cerrar el flujo</i>

La figura siguiente muestra las clases relacionadas con flujos. **Console** pertenece al espacio **System** y el resto, a **System.IO**. Éstas últimas están divididas en dos grupos distintos, ambos derivados indirectamente (línea de puntos) de la clase **Object** del espacio **System**. El grupo de la izquierda ha sido diseñado para trabajar con datos de tipo **char** y el de la derecha con datos de tipo **byte**



Las clases sombreadas son clases abstractas. Una clase abstracta no permite que se creen objetos de ella. Su misión es proporcionar miembros comunes que serán compartidos por todas sus subclases.

Cuando un programa C# se ejecuta, se abren automáticamente tres flujos identificados por las propiedades de la clase **Console** indicadas a continuación:

- Un flujo desde la entrada estándar (el teclado): **In**.
- Un flujo hacia la salida estándar (la pantalla): **Out**.
- Un flujo hacia la salida estándar de error (la pantalla): **Error**.

Estos flujos pueden redirigirse utilizando los métodos **SetIn** y **SetOut**. El primero permite asignar un nuevo flujo a la propiedad **In** y el segundo a la propiedad **Out**.

```
System.Console.SetIn(nuevoFlujoDeEntrada);
System.Console.SetOut(nuevoFlujoDeSalida);
```

La propiedad **In** hace referencia a un objeto de la clase **System.IO.TextReader** y las propiedades **Out** y **Error** hacen referencia a objetos de la clase **System.IO.TextWriter**. Ambas serán estudiadas en el capítulo dedicado a ficheros de datos.

Flujos de entrada

Cuando un programa define un flujo de entrada, por ejemplo el definido por la propiedad **In** de la clase **Console**, dicho programa es destino de ese flujo de caracteres, y eso es todo lo que se necesita saber.

Dos métodos de la clase **Console** que tienen un especial interés porque permiten a un programa leer datos de la entrada estándar son:

```
public static int Read()  
public static string ReadLine()
```

El método **Read** simplemente lee caracteres individuales del flujo de entrada estándar; concretamente lee el siguiente carácter disponible. Devuelve un entero (**int**) correspondiente al código del carácter leído, o bien un valor negativo cuando en un intento de leer se alcanza el final del flujo (esto es, no hay más datos).

Por ejemplo, el siguiente código lee un carácter del origen vinculado con el flujo **In** (entrada estándar):

```
char car;  
car = (char)System.Console.Read();
```

El método **ReadLine** lee una línea del flujo vinculado con la entrada estándar; concretamente lee la siguiente línea disponible. Devuelve una referencia a un objeto **string** que envuelve la línea leída, o bien un valor **null** cuando no hay datos disponibles.

Una línea está definida como una secuencia de caracteres seguidos por ‘\r’, ‘\n’, o bien por ‘\r\n’. La cadena de caracteres devuelta no contiene el carácter o caracteres de terminación.

Por ejemplo, el siguiente código lee una línea del origen vinculado con el flujo **In** (entrada estándar) y la visualiza en la pantalla:

```
using System;  
  
class LeerUnaCadena  
{  
    public static void Main(string[] args)  
    {  
        string sdato; // variable para almacenar una línea de texto  
  
        Console.Write("Introduzca un texto: ");  
        sdato = Console.ReadLine(); // leer una línea de texto  
        Console.WriteLine(sdato); // escribir la línea leída  
    }  
}
```

Analicemos el método **Main** del programa anterior. Primeramente define una referencia, *sdato*, a un objeto **String**; esto permitirá leer una cadena de caracteres. Después, invocando a **ReadLine**, lee una línea de texto introducida a través del teclado que **ReadLine** guarda en un objeto **String**, devolviendo la referencia al mismo para que sea almacenada en *sdato*. Finalmente visualiza el texto leído.

Flujos de salida

Cuando un programa define un flujo de salida, por ejemplo el definido por la propiedad **Out** de la clase **Console**, el programa es el origen de ese flujo de bytes (es el que envía los bytes), y eso es todo lo que se necesita saber.

Dos métodos de la clase **Console** que tienen un especial interés, porque permiten a un programa escribir datos en la salida estándar, son:

```
public static void Write([arg])
public static void WriteLine([arg])
```

Por ejemplo, el siguiente código (se presentan dos versiones) escribe un valor *n* en el destino vinculado con el flujo **Out** (salida estándar) y sitúa el punto de inserción en la línea siguiente:

```
double n = 10.5;
System.Console.WriteLine(n);           // escribe: 10,5
System.Console.WriteLine("Valor = " + n); // escribe: Valor = 10,5
```

Recuerde que en los capítulos 1 y 2 ya fueron explicadas distintas formas de presentar los resultados en pantalla.

Los métodos **Write** y **WriteLine** son esencialmente los mismos; ambos escriben su argumento en el flujo de salida. La única diferencia entre ellos es que **WriteLine** añade un carácter ‘\n’ (avance al principio de la línea siguiente) al final de su salida, y **Write** no. En otras palabras, la siguiente sentencia:

```
System.Console.Write("El valor no puede ser negativo\n");
```

es equivalente a esta otra:

```
System.Console.WriteLine("El valor no puede ser negativo");
```

En el ejemplo anterior, se puede observar que **Write** añade al final de la cadena de caracteres un carácter ‘\n’ que **WriteLine** no necesita añadir.

Los argumentos para **Write** y **WriteLine** pueden ser de cualquier tipo primitivo o referenciado: **object**, **string**, **char[]**, **char**, **int**, **long**, **float**, **double**, **bool**, **decimal**, **uint**, etc. En adición, hay una versión extra de **WriteLine** que no tiene argumentos y lo que hace es escribir un carácter ‘\n’, lo que se traduce en un avance a la línea siguiente.

Como ejemplo, el siguiente programa utiliza **WriteLine** para escribir datos de varios tipos en la salida estándar.

```
using System;

class TestTiposDatos
{
    // Tipos de datos
    public static void Main(string[] args)
    {
        string sCadena = "Lenguaje C#";
        char[] cMatrizCars = { 'a', 'b', 'c' }; // matriz de caracteres
        int dato_int = 4;
        long dato_long = long.MinValue;        // mínimo valor long
        float dato_float = float.MaxValue;     // máximo valor float
        double dato_double = Math.PI;          // 3.1415926
        bool dato_bool = true;

        Console.WriteLine(sCadena);
        Console.WriteLine(cMatrizCars);
        Console.WriteLine(dato_int);
        Console.WriteLine(dato_long);
        Console.WriteLine(dato_float);
        Console.WriteLine(dato_double);
        Console.WriteLine(dato_bool);
    }
}
```

Los resultados que produce el programa anterior son los siguientes:

```
Lenguaje C#
abc
4
-9223372036854775808
3,4028235E+38
3,14159265358979
True
```

Observe que se puede imprimir un objeto; el primer método **WriteLine** imprime un objeto **string**. Cuando se utiliza **Write** o **WriteLine** para imprimir un objeto, el dato impreso depende del tipo del objeto; en realidad depende de cómo esté programado el método **ToString** de la clase del objeto. En el ejemplo se puede observar que la impresión de un objeto **string** hace que se imprima la cadena de caracteres que almacena.

Salida con formato

Los métodos **Write** y **WriteLine** también permiten mostrar los datos según un formato. Para ello utilizaremos las siguientes formas de estos métodos:


```
public static void Write(formato[, argumento]...)
public static void WriteLine(formato[, argumento]...)
```

formato Especifica cómo va a ser la salida. Es una cadena de caracteres formada por caracteres ordinarios, secuencias de escape y especificaciones de formato. El formato se lee de izquierda a derecha.

```
uint edad = 0;
float peso = 0;

// ...
```

Diagrama de la cadena de formato: `Write("Tiene {0,2:d} años y pesa {1,6:f2} kilos\n", edad, peso);`

especificaciones de formato

caracteres ordinarios

secuencia de escape

argumento Representa el valor o valores a escribir. Cada argumento debe tener su correspondiente especificación de formato y en el mismo orden. Si hay más argumentos que especificaciones de formato, los argumentos en exceso se ignoran.

```
Write("Tiene {0,2:d} años y pesa {1,6:f2} kilos\n", edad, peso);
```

Cuando se ejecute la sentencia anterior, los caracteres ordinarios se escribirán tal cual, las especificaciones de formato serán sustituidas por los valores correspondientes en la lista de argumentos y las secuencias de escape darán lugar al carácter o acción que representan. Así, para *edad* igual a 20 y *peso* igual a 70,5 el resultado será:

```
Tiene 20 años y pesa 70,50 kilos
```

Una especificación de formato está compuesta por:

```
{posición[, ancho][:tipo[decimales]]}
```

Una especificación de formato siempre está incluida entre `{}`. El significado de cada uno de los elementos se indica a continuación:

posición Posición 0, 1, 2, etc., del argumento en la lista de argumentos.

ancho Opcional. Mínimo número de posiciones para la salida. Si el ancho se omite o el valor a escribir ocupa más posiciones de las especifica-

das, el ancho es incrementado en lo necesario. Si este valor es negativo el resultado se justifica a la izquierda dentro del *ancho* especificado; si es positivo la justificación se hace a la derecha.

tipo Opcional. Uno de los caracteres de la tabla mostrada a continuación.

decimales Opcional. Número mínimo de dígitos a mostrar en el caso de enteros o de decimales en el caso de fraccionarios. Cuando se especifica se escribe justo a continuación de *tipo*, sin espacios en blanco.

<i>Carácter</i>	<i>Descripción</i>	<i>Ejemplos</i>	<i>Salida</i>
C o c	Moneda	Console.Write("{0,9:C2}", 4.5); Console.Write("{0,9:C2}", -4.5);	4,50 € -4,50 €
D o d	Enteros	Console.Write("{0:D5}", 45); Console.Write("{0,5:D}", 45);	00045 45
E o e	Científico	Console.Write("{0:E}", 450000);	4,500000E+005
F o f	Coma fija	Console.Write("{0:F2}", 45); Console.Write("{0:F0}", 45); Console.Write("{0,8:F2}", 45); Console.Write("{0,8:F2}", 145.3);	45,00 45 45,00 145,30
G o g	General	Console.Write("{0:G}", 4.5);	4,5
N o n	Numérico	Console.Write("{0:N}", 4500000);	4.500.000,00
P o p	%	Console.Write("{0:P}", 0.12345);	12,35 %
X o x	Hexadecimal	Console.Write("{0:X}", 450); Console.Write("{0:X}", 0xff7a);	1C2 FF7A

Con el formato *general* el número se convierte al punto fijo o la notación científica más compactos, dependiendo del tipo de número y de si hay un especificador de precisión.

Excepciones

Cuando durante la ejecución de un programa ocurre un error que impide su continuación, por ejemplo, una entrada incorrecta de datos o un formato de salida no correcto, C# lanza una excepción, que si no se captura da lugar a un mensaje de error y detiene la ejecución (las excepciones se lanzan, no ocurren). Ahora, si lo que deseamos es que la ejecución del programa no se detenga, habrá que capturarla y manejarla adecuadamente en un intento de reanudar la ejecución.

Las excepciones en C# son objetos de subclases de **Exception**. Por ejemplo, el espacio de nombres **System** define la clase base **SystemException** para las ex-

cepciones predefinidas; por ejemplo: aritméticas, de formato, intentar acceder a un elemento de una matriz con un índice fuera de límites, etc.

Para capturar una excepción hay que hacer dos cosas: una, poner a prueba el código que puede lanzar excepciones dentro de un bloque **try**, y dos, manejar la excepción cuando se lance, en un bloque **catch**. Por ejemplo:

```
try
{
    // Código que puede lanzar una excepción
    Console.WriteLine("{0:f} + {1:f} = {2:d}", a, b, c);
}
catch(Exception e)
{
    // Manejar una excepción de la clase Exception
    System.Console.WriteLine("Error: " + e.Message);
}
```

En el ejemplo anterior, cuando se lance una excepción del tipo **Exception** porque se produjo un error debido a la ejecución de una operación de entrada/salida, el objeto *e* almacenará la información relativa al error ocurrido, información que puede ser manipulada a través de las propiedades y métodos de su clase; por ejemplo, **Message** muestra un mensaje indicando qué error ocurrió.

En el ejemplo anterior, al especificar que el tipo de excepciones que se desea atrapar son de la clase **Exception**, cabe cualquier excepción de cualquier clase derivada de **Exception**. Si quisiéramos filtrar un tipo particular de excepciones deberíamos indicarlo explícitamente. Por ejemplo, cuando ejecute el siguiente programa, observará que lanza una excepción de la clase **FormatException** por utilizar un formato **d** para mostrar un valor **double**. Por lo tanto, dicha excepción será atrapada por el primer bloque **catch**. En cambio, si la excepción fuera de otro tipo, sería atrapada por el segundo bloque **catch**. Cuando una excepción se atrapa, se considera manipulada. Esto es todo lo que necesita saber por ahora.

```
using System;

class Test
{
    public static void Main(string[] args)
    {
        double a = 10, b = 20, c = 0;

        try
        {
            c = a + b;
            Console.WriteLine("{0:f} + {1:f} = {2:d}", a, b, c);
        }
    }
}
```

```
        catch(FormatException)
        {
            Console.WriteLine("Ha ocurrido un error de formato");
        }
        catch(Exception e)
        {
            Console.WriteLine("Error: " + e.Message);
        }
    }
}
```

Obsérvese que el primer bloque **catch** define el tipo de excepción que atraparé, pero no un objeto de ese tipo. Esto quiere decir que el definir un objeto o no es opcional; dependerá de si es o no necesario a la hora de manipular la excepción.

Trabajar con tipos de datos numéricos

Llegados a este punto sabemos cómo leer un carácter, o bien una cadena de caracteres, desde el flujo **In**, y cómo escribir una variable de cualquier tipo (valor o referencia) en el flujo **Out**.

Evidentemente, cualquier operación aritmética requiere de valores numéricos; pero, según lo expuesto, en el mejor de los casos del flujo **In** sólo se puede obtener una cadena de caracteres. Por ejemplo, el código siguiente perteneciente al programa *LeerUnaCadena* realizado anteriormente, permite leer del flujo **In** una cadena de caracteres que será almacenada en el objeto *sdata* de tipo **string**:

```
Console.Write("Introduzca un texto: ");
sdata = Console.ReadLine(); // leer una línea de texto
```

Si cuando se ejecute el método **ReadLine** del ejemplo anterior se teclea, por ejemplo, el dato *456*, estos dígitos serán almacenados en *sdata* como una cadena de caracteres. Ahora bien, para que esa cadena de tres caracteres pueda ser utilizada en una expresión aritmética, tiene que adquirir la categoría de valor numérico, lo que implica convertirla a un valor de alguno de los tipos primitivos. Esto puede hacerse utilizando los métodos proporcionados por las estructuras que encapsulan los tipos de datos numéricos, o bien por la clase **Convert** que proporciona métodos como, por ejemplo, **ToInt32** que permite convertir a **int** cualquier dato en cualquier otro tipo. Por ejemplo:

```
string sCadena;
int dato_int;
sCadena = Console.ReadLine();
dato_int = Convert.ToInt32(sCadena);
```

Lo mismo diríamos respecto a los métodos de la misma clase, **ToByte**, **ToInt16** (short), **ToInt64** (long), **ToSingle** (float), **ToDouble**, **ToString**, etc.

Si la conversión no es posible porque la entrada no se ajusta a un valor del tipo especificado, se lanzará una excepción del tipo **FormatException**.

Estructuras que encapsulan los tipos primitivos

El espacio de nombres **System** proporciona las estructuras **Byte**, **SByte**, **Char**, **Int16**, **UInt16**, **Int32**, **UInt32**, **Int64**, **UInt64**, **Single**, **Double**, **Decimal** y **Boolean**, que encapsulan cada uno de los tipos primitivos estudiados en los capítulos anteriores, proporcionando así una funcionalidad añadida para manipularlos.

Analicemos, por ejemplo, la estructura **Int32**. Un objeto **Int32** encapsula un número entero (dato de tipo **int**) que puede ser manipulado utilizando sus métodos. Por ejemplo, hay métodos para convertir el entero en un **string** o un **string** en un entero. Veamos a continuación los atributos y métodos de esta estructura que tienen un mayor interés para nosotros:

Atributo	Descripción
MinValue	Valor más pequeño de tipo int .
MaxValue	Valor más grande de tipo int .
Método	Descripción
Parse(string)	Convierte una cadena a un valor int .
ToString()	Convierte un valor int en una cadena (objeto string).

El resto de las estructuras tienen métodos análogos y posiblemente incluirán otros atributos y métodos. Por ejemplo, la estructura **Single** proporciona, entre otros, los atributos y los métodos indicados a continuación:

Atributo	Descripción
Epsilon	Valor más pequeño positivo de tipo float .
MinValue	Valor más pequeño de tipo float .
MaxValue	Valor más grande de tipo float .
NaN	No es un Número; constante de tipo float .
NegativeInfinity	Valor infinito negativo de tipo float .
PositiveInfinity	Valor infinito positivo de tipo float .

Método	Descripción
IsInfinity(float)	Devuelve true si el valor del argumento es $\pm\infty$.
IsNaN(float)	Devuelve true si el argumento no es un número.
Parse(string)	Convierte una cadena a un valor float .
ToString()	Convierte un valor float en una cadena (objeto string).

Para la estructura **Double** haríamos una exposición análoga.

De acuerdo con lo expuesto, para obtener, por ejemplo, un entero a partir de una cadena de caracteres proporcionada por **ReadLine** habrá que ejecutar los siguientes pasos:

1. Leer la cadena de caracteres.
2. Convertir el objeto **string** en un entero.

El siguiente código responde a los puntos enunciados:

```
string sdato; // variable para almacenar una cadena
int dato_int; // variable para almacenar un entero
try
{
    sdato = Console.ReadLine(); // leer una cadena de caracteres
    dato_int = Int32.Parse(sdato); // convertir a entero
}
catch (Exception)
{
    dato_int = 0;
    Console.WriteLine("Dato incorrecto. Se asume un 0.");
}
```

En el ejemplo anterior se observa que una vez leída la cadena *sdato*, se invoca al método estático **Parse** de **Int32** para convertir el objeto **string** en un dato de tipo **int**. Si la cadena no es válida para ser convertida en un entero, se lanzará una excepción que será atrapada por el bloque **catch** donde se asumirá un valor cero.

Análogamente, para convertir una cadena de caracteres que representa un número con coma decimal, en un valor de tipo **float**, el código sería el siguiente:

```
string sdato; // variable para almacenar una cadena
float dato_float; // variable para almacenar un float
try
{
    sdato = Console.ReadLine(); // leer una cadena de caracteres
    dato_float = Single.Parse(sdato); // convertir a float
}
```

```

catch (Exception)
{
    dato_float = 0;
    Console.WriteLine("Dato incorrecto. Se asume un 0.");
}

```

Según lo expuesto, partiendo de la cadena devuelta por el método **ReadLine**, podemos escribir un método *datoInt* que la convierta en un número entero y devuelva este valor como resultado. Si el valor leído no fuera válido para convertirlo en un número entero, el método **Parse** lanzaría una excepción de tipo **FormatException** que podríamos manejar. Para no complicar el tema que estamos exponiendo, si se lanza una excepción del tipo descrito vamos a hacer que el método devuelva simplemente un valor significativo (por ejemplo, una constante miembro de la estructura). Según esto podríamos escribir el método *datoInt* así:

```

public static int datoInt()
{
    try
    {
        return Int32.Parse(Console.ReadLine());
    }
    catch (FormatException)
    {
        return Int32.MinValue; // valor más pequeño
    }
}

```

Observe que el argumento del método **Parse** es la cadena de caracteres devuelta por el método **ReadLine**. Si ocurre un error, por ejemplo, porque se introduce una cadena que no es convertible a un número entero, el sistema lanzará una excepción de tipo **FormatException** que será atrapada por el bloque **catch**, lo que dará lugar a que el método *datoInt* devuelva el valor **MinValue** definido como una constante en la estructura **Int32**.

```

dato_int = Leer.datoInt();
if (dato_int == Int32.MinValue)
    Console.WriteLine("El dato introducido no es correcto");

```

Análogamente, podemos escribir otros métodos para convertir una cadena válida, devuelta por el método **ReadLine**, en otros tipos de datos primitivos. Agrupemos todos estos métodos en una clase denominada *Leer*, pero antes veamos cómo ocurren las conversiones entre tipos valor y tipos referencia.

Conversión entre tipos valor y tipos referencia

Este tipo de conversiones permite tratar los datos de tipo valor como objetos.

La conversión de un tipo valor en un tipo referencia empaqueta el dato de tipo valor en un objeto de tipo **Object**. Esta operación se la conoce como *boxing*.

```
int dato = 12345;
object objInt = dato; // boxing
```

La última línea de este ejemplo crea un objeto de la clase **Object** referenciado por *objInt* que empaqueta el valor *dato* de tipo **int**.

La conversión de un tipo **Object** en un tipo valor desempaqueta el objeto de tipo **Object** en un dato del tipo del valor empaquetado. Esta operación se conoce como *unboxing* y se trata de una conversión explícita.

```
// ...
int datoInt = (int)objInt; // unboxing
```

Clase Leer

El objetivo es escribir una clase *Leer* que incluya como miembros métodos análogos al implementado anteriormente, de manera que proporcione una interfaz que cualquier programa puede utilizar para obtener del teclado datos de cualquier tipo primitivo. El código que define esta clase se muestra a continuación. Observe que dicha clase es pública y que ha sido definida dentro del espacio de nombres *MisClases.ES* (mis clases de entrada, salida):

```
using System;

namespace MisClases.ES // espacio de nombres
{
    public class Leer    // clase Leer
    {
        public static short datoShort()
        {
            try
            {
                return Int16.Parse(Console.ReadLine());
            }
            catch (FormatException)
            {
                return Int16.MinValue; // valor más pequeño
            }
        }

        public static int datoInt()
        {
            try
            {
```



```

        return Int32.Parse(Console.ReadLine());
    }
    catch (FormatException)
    {
        return Int32.MinValue; // valor más pequeño
    }
}

public static long datoLong()
{
    try
    {
        return Int64.Parse(Console.ReadLine());
    }
    catch (FormatException)
    {
        return Int64.MinValue; // valor más pequeño
    }
}

public static float datoFloat()
{
    try
    {
        return Single.Parse(Console.ReadLine());
    }
    catch (FormatException)
    {
        return Single.NaN; // No es un Número; valor float.
    }
}

public static double datoDouble()
{
    try
    {
        return Double.Parse(Console.ReadLine());
    }
    catch (FormatException)
    {
        return Double.NaN; // No es un Número; valor double.
    }
}
}
}

```

En la clase *Leer*, se puede observar que todos los métodos, además de públicos, se han declarado **static** con el fin de que puedan ser invocados allí donde se necesiten, sin necesidad de que exista un objeto de dicha clase. Recuerde que la sintaxis para invocar a un método **static** es:

nombreClase.nombreMétodo

Una vez escrita la clase *Leer*, podemos utilizarla como soporte para otros programas. Como ejemplo, vamos a escribir un programa que lea un dato de cada uno de los tipos contemplados en *Leer* y muestre después los valores leídos. Recuerde que para que la clase aplicación que vamos a escribir pueda utilizar la clase *Leer*, si ambas están en la misma carpeta de trabajo, debe compilar la aplicación así: *csc LeerDatos.cs Leer.cs*, suponiendo, evidentemente, que las clases fueron almacenadas en los ficheros especificados en la orden *csc*.

```
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
```

```
using System;
```

```
using MisClases.ES;
```

```
class LeerDatos
```

```
{
```

```
    public static void Main(string[] args)
```

```
    {
```

```
        short dato_short = 0;
```

```
        int dato_int = 0;
```

```
        long dato_long = 0;
```

```
        float dato_float = 0;
```

```
        double dato_double = 0;
```

```
        Console.Write("Dato short: ");
```

```
        dato_short = Leer.datoShort();
```

```
        Console.Write("Dato int: ");
```

```
        dato_int = Leer.datoInt();
```

```
        Console.Write("Dato long: ");
```

```
        dato_long = Leer.datoLong();
```

```
        Console.Write("Dato float: ");
```

```
        dato_float = Leer.datoFloat();
```

```
        Console.Write("Dato double: ");
```

```
        dato_double = Leer.datoDouble();
```

```
        Console.WriteLine(dato_short);
```

```
        Console.WriteLine(dato_int);
```

```
        Console.WriteLine(dato_long);
```

```
        Console.WriteLine(dato_float);
```

```
        Console.WriteLine(dato_double);
```

```
    }
```

```
}
```

Si quisiéramos conocer si un dato, por ejemplo **float**, fue leído correctamente podríamos interrogar el valor devuelto por el método invocado:

```
dato_float = Leer.datoFloat();
```

```
if (float.IsNaN(dato_float))
    Console.WriteLine("El dato introducido no es correcto");
```

Después del trabajo realizado, ya tenemos una forma de leer datos numéricos introducidos a través del teclado. Esto nos permitirá escribir diversos programas que requieren de este proceso. Además, sabemos también cómo convertir números a cadenas de caracteres y viceversa.

¿DÓNDE SE UBICAN LAS CLASES QUE DAN SOPORTE?

Para que C# pueda utilizar una clase (por ejemplo la clase *Leer* implementada anteriormente) debe conocer dónde está almacenada en el sistema de ficheros. De otra forma, cuando se compile el programa se obtendrá un error que indica que esa clase no existe. Una solución sencilla al problema planteado puede ser alguna de las que se plantean a continuación:

1. Copiar las clases que dan soporte a la aplicación en la misma carpeta que la clase aplicación, y obtener el fichero ejecutable a partir de todas ellas. Por ejemplo, si la clase aplicación *LeerDatos.cs* necesita del soporte de la clase *Leer.cs*, copiamos ésta última en la misma carpeta de *LeerDatos.cs* y construimos el fichero ejecutable según se indica a continuación:

```
csc LeerDatos.cs Leer.cs
```

2. Construir una biblioteca con las clases que dan soporte (fichero *.dll*) y copiar esa biblioteca en la misma carpeta que la aplicación que necesite de sus servicios, para después construir el fichero ejecutable.

Para construir una biblioteca de clases, hágalo de forma análoga a como se indica en el ejemplo siguiente:

```
csc /target:library /out:misClases.dll Leer.cs
```

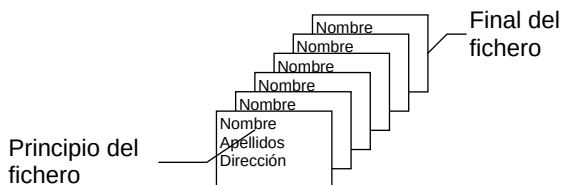
Después, copie la biblioteca *misClases.dll* en el directorio de la aplicación que necesita de sus servicios y construya el fichero ejecutable de forma análoga a como se indica en el ejemplo siguiente:

```
csc /reference:misClases.dll LeerDatos.cs
```

CARÁCTER FIN DE FICHERO

Desde el punto de vista de un usuario de un programa, un dispositivo de entrada o de salida estándar es tratado por el lenguaje C# como si de un fichero de datos en el disco se tratara. Un fichero de datos no es más que una colección de informa-

ción. Los datos que introducimos por el teclado son una colección de información y los datos que visualizamos en el monitor son también una colección de información.



Todo fichero tiene un principio y un final. ¿Cómo sabe un programa que está leyendo datos de un fichero, que se ha llegado al final del mismo y por lo tanto no hay más datos? Por una marca de fin de fichero. En el caso de un fichero grabado en un disco esa marca estará escrita al final del mismo. En el caso del teclado la información procede de lo que nosotros tecleamos, por lo tanto si nuestro programa requiere detectar la marca de fin de fichero, tendremos que teclearla cuando demos por finalizada la introducción de información. Esto se hace pulsando las teclas *Ctrl+D* en UNIX o *Ctrl+Z* en una aplicación de consola en Windows.

Ya que un fichero o un dispositivo siempre es manejado a través de un flujo, hablar del final del flujo es sinónimo de hablar del final del fichero. Por eso, de ahora en adelante nos referiremos al flujo en lugar de al fichero o dispositivo vinculado.

Recuerde que cuando el método **Read** intenta leer y se encuentra con el final del flujo, retorna un valor negativo. Análogamente, cuando el método **ReadLine** intenta leer del flujo y se encuentra con el final del mismo, retorna la constante **null**. Para aclarar lo expuesto, el siguiente ejemplo solicita del teclado un dato *precio*. Entonces, si al mensaje “Precio:” respondemos escribiendo una cantidad, la variable *precio* almacenará ese valor, pero si respondemos pulsando las teclas *Ctrl+Z* (carácter fin de fichero), deberá almacenar el valor *NaN* de tipo **float**.

```
using System;

class Test
{
    public static void Main(string[] args)
    {
        string sdato;
        float precio = 0.0F;

        Console.Write("Precio: ");
        sdato = Console.ReadLine();
        precio = (sdato != null)
            ? float.Parse(sdato)
            : float.NaN;
    }
}
```

```

        Console.WriteLine(precio);
        Console.WriteLine("Continúa el programa");
    }
}

```

Cuando ejecute este programa puede proceder de cualquiera de las dos formas siguientes:

1. Introduciendo un dato válido:

```

Precio: 123,45
123,45
Continúa el programa

```

2. Pulsando las teclas *Ctrl+Z* (marca de final del flujo):

```

Precio: (se pulsan las teclas Ctrl+Z)
NaN
Continúa el programa

```

Una aclaración. El programa anterior utiliza el operador ternario (?) para verificar si se llegó al final del flujo, lo que sucederá cuando se pulsen las teclas *Ctrl+Z*. La expresión booleana *sdato != null* será **true** si se introdujo un dato válido para **ReadLine**, en cuyo caso se asignará a *precio* el resultado de convertir *sdato* a **float**, y será **false** si se pulsaron las teclas *Ctrl+Z*, en cuyo caso se asignará a *precio* el valor *NaN*.

En capítulos posteriores utilizaremos *Ctrl+Z* como condición para finalizar la entrada de un número de datos, en principio indeterminado.

CARACTERES \r\n

Cuando se están introduciendo datos a través del teclado y pulsamos la tecla *Enter* se introducen también los caracteres *\r\n*, correspondientes a los caracteres Unicode *CR LF* (*CR* es el Unicode 13 y *LF* es el Unicode 10). Mientras que en la salida *\n* produce un *CR+LF*, en la entrada se corresponde con un *LF*; esto es, una expresión C# como '*\n*' == 10 daría como resultado **true**.

Por ejemplo, el código siguiente lee un carácter:

```

char opción;

Console.Write("Opción (a, b o c): ");
opción = (char)Console.Read();

```

Cuando se ejecute el método **Read** del ejemplo anterior, si tecleamos la opción *b* y pulsamos la tecla *Entrar*:

b[Entrar]

antes de la lectura, el *buffer* de entrada contendrá la siguiente información:

b	\r	\n														
---	----	----	--	--	--	--	--	--	--	--	--	--	--	--	--	--

y después de la lectura:

\r	\n															
----	----	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

ya que **Read** lee un solo carácter. Estos caracteres sobrantes pueden ocasionarnos problemas si a continuación se ejecuta otra sentencia de entrada que admita datos que sean caracteres. Por ejemplo:

```
char opción;  
string sdato;  
  
Console.Write("Opción (a, b o c): ");  
opción = (char)Console.Read();  
  
Console.Write("Precio: ");  
sdato = Console.ReadLine();  
Console.WriteLine("Continúa el programa");
```

Si ejecutamos este código y tecleamos, por ejemplo, como opción *b* seguida de la pulsación de la tecla *Entrar*, se producirá el siguiente resultado:

```
Opción (a, b o c): b  
Precio: Continúa el programa
```

A la vista del resultado, se observa que cuando se ejecutó **ReadLine** no se detuvo la ejecución del programa para introducir el dato solicitado. ¿Por qué? Porque los caracteres sobrantes *\r* y *\n* son válidos para el método **ReadLine**. Recuerde que este método permite leer una cadena de caracteres hasta encontrar '*\r*' (CR), '*\n*' (LF) o '*\r\n*' (CRLF); estos caracteres son leídos pero no almacenados. Por este motivo es por lo que, en el ejemplo, este método no necesita esperar a que introduzcamos uno o más caracteres para la variable *sdato*.

Del ejemplo anterior se deduce que un flujo lleva asociado un *buffer* (un *buffer* actúa como una memoria intermedia para lecturas/escrituras futuras), de tal forma que cuando un programa ejecute una sentencia de entrada (que solicite datos) los datos obtenidos del origen pueden ser depositados en el *buffer* en bloques más grandes que los que realmente está leyendo el programa (por ejemplo, cuando

se leen datos de un disco la cantidad mínima de información transferida es un bloque equivalente a una unidad de asignación). Esto aumenta la velocidad de ejecución porque la siguiente vez que el programa necesite más datos no tendrá que esperar por ellos porque ya los tendrá en el *buffer*. Por otra parte, cuando se trate de una operación de salida, los datos no serán enviados al destino hasta que no se llene el *buffer* (o hasta que se fuerce el vaciado del mismo implícita o explícitamente), lo que reduce el número de accesos al dispositivo físico vinculado que siempre resulta mucho más lento que los accesos a memoria, aumentando por consiguiente la velocidad de ejecución.

La solución al problema planteado es limpiar los caracteres indeseables del *buffer* de entrada. Una forma sencilla de hacer esto es utilizar el propio método **ReadLine** para hacer una lectura “en falso”, con la única intención de extraer todos los caracteres que haya:

```
char opción;
string sdato;

Console.Write("Opción (a, b o c): ");
opción = (char)Console.Read();
Console.ReadLine(); // limpiar los caracteres \r\n

Console.Write("Precio: ");
sdato = Console.ReadLine();
Console.WriteLine("Continúa el programa");
```

Un *buffer* se limpia automáticamente cuando está lleno (sólo si es de salida), cuando se cierra el flujo, o bien cuando el programa finaliza normalmente.

OTROS MÉTODOS DE LA CLASE Console

La clase **Console** agrega también características avanzadas que permiten controlar aspectos como la posición del cursor, los colores del fondo de la ventana y del texto, las dimensiones de la ventana y el tamaño del *buffer* donde se almacena la información que aparece en ella. Estas posibilidades no resultan imprescindibles para crear una aplicación de consola, pero permiten mejorar tanto el aspecto mostrado en la ventana como la interacción de la misma con el usuario.

Controlar la posición y los colores del texto

Los métodos que hemos utilizado hasta ahora, como **WriteLine** y **ReadLine**, no permiten controlar la posición en la que se muestran o solicitan los datos, ni tampoco establecer los colores del texto. Pues bien, antes de enviar o solicitar cualquier información a la consola con los métodos antes citados, podemos colocar el

cursor en una posición determinada utilizando las propiedades **CursorLeft** y **CursorTop** o bien el método **SetCursorPosition**. Las propiedades son también de lectura, lo que permitirá recuperar la posición actual del cursor en cualquier momento.

También, a través de la propiedad **CursorVisible**, es posible ocultar y mostrar el cursor, así como cambiar su tamaño relativo mediante la propiedad **CursorSize**.

Así mismo, las propiedades **BackgroundColor** y **ForegroundColor** permiten establecer el color de fondo y el del texto, respectivamente. Estas propiedades pueden tomar cualquier valor de la enumeración **ConsoleColor**.

El método **Clear** se encarga de borrar todo el contenido de la consola y establecer el color de fondo asignado a **BackgroundColor**, mientras que la propiedad **Title** asigna el título que mostrará la ventana.

Por ejemplo, las sentencias mostradas a continuación generan el resultado que aparece en la figura siguiente:

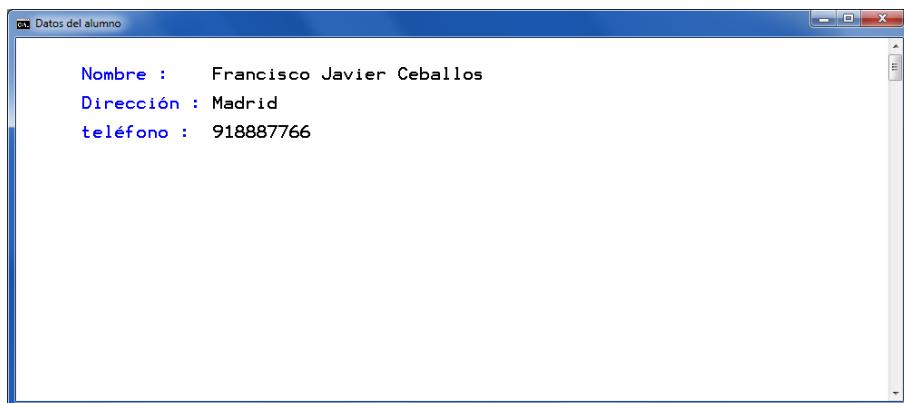
```
string nombre = null, dirección = null, teléfono = null;
```

```
Console.Title = "Datos del alumno";  
Console.BackgroundColor = ConsoleColor.White;  
Console.Clear();  
Console.ForegroundColor = ConsoleColor.Blue;
```

```
Console.SetCursorPosition(6, 2);  
Console.Write("Nombre :");  
Console.SetCursorPosition(6, 4);  
Console.Write("Dirección :");  
Console.SetCursorPosition(6, 6);  
Console.Write("Teléfono :");
```

```
Console.ForegroundColor = ConsoleColor.Black;
```

```
Console.SetCursorPosition(18, 2);  
nombre = Console.ReadLine();  
Console.SetCursorPosition(18, 4);  
dirección = Console.ReadLine();  
Console.SetCursorPosition(18, 6);  
teléfono = Console.ReadLine();
```

Dimensiones de la ventana de la consola

La consola del sistema asigna unas dimensiones predeterminadas a la ventana en la que se ejecuta, 80 columnas por 25 filas, aunque es posible enviar más de 25 líneas de información; las que no se puedan mostrar podremos recuperarlas de un *buffer* interno utilizando la barra de desplazamiento.

Para cambiar el tamaño de la ventana hay que utilizar las propiedades **WindowWidth** (ancho) y **WindowHeight** (alto). Las nuevas dimensiones no deben superar el ancho y alto máximos indicados por **LargestWindowWidth** y **LargestWindowHeight**, respectivamente. Y para controlar el tamaño del *buffer* en el que se almacena la información, disponemos de las propiedades **BufferWidth** y **BufferHeight**. No obstante, éste es un aspecto que normalmente se deja en manos del sistema que actúa según el tamaño de la ventana.

Como ejemplo, podemos añadir las dos sentencias siguientes al inicio del ejemplo anterior:

```
Console.WindowHeight = Console.LargestWindowHeight;  
Console.WindowWidth = Console.LargestWindowWidth;
```

Detección de las pulsaciones del teclado

Los métodos **Read** y **ReadLine** de la clase **Console** no efectúan realmente su trabajo hasta que el usuario pulsa *Entrar*; pero, en ocasiones, necesitamos conocer el carácter tecleado para tomar una determinada acción.

Pues bien, el método **ReadKey** de la clase **Console**, conjuntamente con la propiedad **KeyAvailable**, nos permite saber si hay o no pulsaciones de tecla pendientes de procesar. También, podemos impedir que la combinación *Ctrl+C* inte-

rrumpa la ejecución de la aplicación asignando el valor **true** a la propiedad **TreatControlAsInput**.

El método **ReadKey** devuelve una estructura de tipo **ConsoleKeyInfo** compuesta por tres miembros: **Key**, **KeyChar** y **Modifiers**. El primero indica la tecla pulsada, un valor correspondiente a la enumeración **ConsoleKey**, incluyendo las que no generan caracteres como las teclas de edición y función. Si la tecla genera un carácter podemos obtenerlo a través de **KeyChar**. Finalmente, **Modifiers** permite saber si estaba pulsada alguna de las teclas: *Mayús*, *Control* y *Alt*.

Por ejemplo, suponiendo que quisiéramos leer el teléfono de una determinada persona y asegurarnos de que el usuario sólo introducirá dígitos del 0 al 9, tendríamos que recurrir a un bucle en el que fuésemos leyendo carácter a carácter, comprobando si es un carácter válido, un dígito, y guardándolo. Esto es lo que hace el código siguiente:

```
string s = "";
ConsoleKeyInfo pulsación = default(ConsoleKeyInfo);
do
{
    pulsación = Console.ReadKey(true);
    if ((char.IsDigit(pulsación.KeyChar)))
    {
        s += pulsación.KeyChar;
        Console.Write(pulsación.KeyChar);
    }
}
while ((pulsación.Key != ConsoleKey.Enter));
```

Este código podría introducirse en un método *Leer* que devolviese como resultado el dato leído. De esta forma se evitaría su repetición cada vez que hubiese que facilitar este tipo de información.

MÉTODOS MATEMÁTICOS

La biblioteca de clases .NET incluye una clase llamada **Math** en su espacio de nombres **System**, la cual define un conjunto de operaciones matemáticas de uso común que pueden ser utilizadas por cualquier programa.

La clase **Math** contiene métodos para ejecutar operaciones numéricas elementales tales como raíz cuadrada, exponencial, logaritmo y funciones trigonométricas. Este código muestra un ejemplo con el método raíz cuadrada:

```
double raíz_cuadrada, n = 345.0;
raíz_cuadrada = Math.Sqrt(n);
```

```
Console.WriteLine("La raíz cuadrada de " + n + " es " + raíz_cuadrada);
```

La tabla siguiente resume los miembros de la clase **Math**. Todos los métodos de esta clase son públicos y **static** para que puedan ser invocados sin necesidad de definir un objeto de la clase.

Atributo/Método	Descripción
const double E	Valor del número e (base del logaritmo neperiano o natural).
const double PI	Valor del número π .
tipo Abs (tipo a)	Valor absoluto de a . El <i>tipo</i> , igual en todos los casos, puede ser: decimal , double , float , short , int , long o sbyte .
double Ceiling (double a)	Valor double sin decimales más pequeño que es mayor o igual que a .
double Floor (double a)	Valor double sin decimales más grande que es menor o igual que a .
tipo Max (tipo a , tipo b)	Valor mayor de a y b . El <i>tipo</i> , igual en todos los casos, puede ser entero o real.
tipo Min (tipo a , tipo b)	Valor menor de a y b . El <i>tipo</i> , igual en todos los casos, puede ser entero o real.
tipo Round (tipo a)	Valor entero más cercano a a . El <i>tipo</i> , igual en todos los casos, puede ser decimal o double .
double Sqrt (double a)	Raíz cuadrada de a (a no puede ser negativo).
double Exp (double a)	Valor de e^a .
double Log (double a)	Logaritmo en base e (natural) de a .
double Log10 (double a)	Logaritmo en base 10 de a .
double Pow (double a , double b)	Valor de a^b .
double IEEERemainder (double $f1$, double $f2$)	Resto de una división entre números reales: $c=f1/f2$, siendo c el valor entero más cercano al valor real de $f1/f2$; por lo tanto, el resto puede ser positivo o negativo.
double Acos (double a)	Arco, de 0.0 a π , cuyo coseno es a .
double Asin (double a)	Arco, de $-\pi/2$ a $\pi/2$, cuyo seno es a .
double Atan (double a)	Arco, de $-\pi/2$ a $\pi/2$, cuya tangente es a .
double Atan2 (double a , double b)	Convierte las coordenadas rectangulares (b , a) a polares: (r, θ) .
double Sin (double a)	Seno de a radianes.

Atributo/Método	Descripción
double Cos (double <i>a</i>)	Coseno de <i>a</i> radianes.
double Tan (double <i>a</i>)	Tangente de <i>a</i> radianes.
double Sinh (double <i>a</i>)	Seno hiperbólico de <i>a</i> radianes.
double Cosh (double <i>a</i>)	Coseno hiperbólico de <i>a</i> radianes.
double Tanh (double <i>a</i>)	Tangente hiperbólica de <i>a</i> radianes.
int Sign (<i>tipo a</i>)	Un valor (−1, 0, 1) indicando el signo de <i>a</i> . El <i>tipo</i> puede ser entero o real.

NÚMEROS ALEATORIOS

La biblioteca de clases .NET incluye una clase llamada **Random** en su espacio de nombres **System**, la cual define un conjunto de operaciones relacionadas con la obtención de números al azar. El código siguiente muestra un ejemplo de cómo obtener un número al azar entre 1 y 49:

```
int n = 0;
Random rnd = new Random(); // crear un objeto de la clase Random
n = rnd.Next(1, 50);        // obtener el siguiente número aleatorio
                             // entre 1 y 49, ambos inclusive
Console.WriteLine(n);       // mostrar el número obtenido
```

En esta tabla se resumen los métodos más utilizados de la clase **Random**:

Método	Descripción
int Next ()	Devuelve un número entero positivo.
int Next (int <i>máx</i>)	Devuelve un número entero positivo menor que <i>máx</i> .
int Next (int <i>mín</i> , int <i>máx</i>)	Devuelve un número entero positivo mayor o igual que <i>mín</i> y menor que <i>máx</i> .
double NextDouble ()	Devuelve un número mayor o igual que 0.0 y menor que 1.0.
void NextBytes (byte[] <i>m</i>)	Devuelve una matriz <i>m</i> de bytes con números aleatorios.

EJERCICIOS RESUELTOS

1. Realizar un programa que dé como resultado los intereses producidos y el capital total acumulado de una cantidad *c*, invertida a un interés *r* durante *t* días.

La fórmula utilizada para el cálculo de los intereses es:

$$I = \frac{c * r * t}{360 * 100}$$

siendo:

I = Total de intereses producidos.

c = Capital.

r = Tasa de interés nominal en tanto por ciento.

t = Período de cálculo en días.

La solución de este problema puede ser de la siguiente forma:

- Primero definimos las variables que vamos a utilizar en los cálculos.

```
double c, intereses, capital;
float r;
int t;
```

- A continuación leemos los datos c , r y t .

```
System.Console.Write("Capital invertido: ");
c = Leer.datoDouble();
System.Console.Write("\nA un % anual del: ");
r = Leer.datoFloat();
System.Console.Write("\nDurante cuántos días: ");
t = Leer.datoInt();
```

- Conocidos los datos, realizamos los cálculos. Nos piden los intereses producidos y el capital acumulado. Los intereses producidos los obtenemos aplicando directamente la fórmula. El capital acumulado es el capital inicial más los intereses producidos.

```
intereses = c * r * t / (360 * 100);
capital = c + intereses;
```

- Finalmente, escribimos el resultado.

```
System.Console.WriteLine("Intereses producidos... " + intereses);
System.Console.WriteLine("Capital acumulado..... " + capital);
```

Observe que el desarrollo de un programa, en general, consta de tres bloques colocados en el siguiente orden:



El programa completo se muestra a continuación. Observe que se ha utilizado para la entrada de datos los métodos de la clase *Leer* implementada anteriormente en este mismo capítulo.

```
// Este programa utiliza la clase Leer del espacio de nombres  
// MisClases.ES
```

```
using System;  
using MisClases.ES;
```

```
class CIntereses  
{  
    public static void Main(string[] args)  
    {  
        double c, intereses, capital;  
        float r;  
        int t;  
  
        Console.Write("Capital invertido: ");  
        c = Leer.datoDouble();  
        Console.Write("\nA un % anual del: ");  
        r = Leer.datoFloat();  
        Console.Write("\nDurante cuántos días: ");  
        t = Leer.datoInt();  
  
        intereses = c * r * t / (360 * 100);  
        capital = c + intereses;  
  
        Console.WriteLine("Intereses producidos... " + intereses);  
        Console.WriteLine("Capital acumulado..... " + capital);  
    }  
}
```

- Realice un programa que dé como resultado las soluciones reales x_1 y x_2 de una ecuación de segundo grado, de la forma:

$$ax^2 + bx + c = 0$$

Las soluciones de una ecuación de segundo grado vienen dadas por la fórmula:

$$x_i = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Las soluciones son reales sólo si b^2-4ac es mayor o igual que 0. Con lo aprendido hasta ahora, la solución de este problema puede desarrollarse de la forma siguiente:

- Primero definimos las variables necesarias para los cálculos:

```
double a, b, c, d, x1, x2;
```

- A continuación leemos los coeficientes a , b y c de la ecuación:

```
System.Console.Write("Coeficiente a: "); a = Leer.datoDouble();
System.Console.Write("Coeficiente b: "); b = Leer.datoDouble();
System.Console.Write("Coeficiente c: "); c = Leer.datoDouble();
```

- Nos piden calcular las raíces reales. Para que existan raíces reales tiene que cumplirse que $b^2-4ac \geq 0$; si no, las raíces son complejas conjugadas. Entonces, si hay raíces reales las calculamos; en otro caso, salimos del programa.

Para salir de un programa basta con salir del método **Main**. En general, para salir de un método sin hacer nada más, C# proporciona la sentencia **return**.

```
d = b * b - 4 * a * c;
if (d < 0)
{
    // Si d es menor que 0
    System.Console.WriteLine("Las raíces son complejas.");
    return; // salir
}
// Si d es mayor o igual que 0
System.Console.WriteLine("Las raíces reales son:");
```

- Si hay raíces reales las calculamos aplicando la fórmula.

```
d = Math.Sqrt(d);
x1 = (-b + d) / (2 * a);
x2 = (-b - d) / (2 * a);
```

El método **Sqrt** calcula la raíz cuadrada de su argumento. En el ejemplo, se calcula la raíz cuadrada de d y se almacena el resultado de nuevo en d .

- Por último escribimos los resultados obtenidos.

```
System.Console.WriteLine("x1 = " + x1 + ", x2 = " + x2);
```

El programa completo se muestra a continuación:

```
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES

using System;
using MisClases.ES;

class CEcuacion
{
    public static void Main(string[] args)
    {
        double a, b, c, d, x1, x2;

        Console.Write("Coeficiente a: "); a = Leer.datoDouble();
        Console.Write("Coeficiente b: "); b = Leer.datoDouble();
        Console.Write("Coeficiente c: "); c = Leer.datoDouble();

        d = b * b - 4 * a * c;
        if (d < 0)
        {
            // Si d es menor que 0
            Console.WriteLine("Las raíces son complejas.");
            return; // salir
        }

        // Si d es mayor o igual que 0
        Console.WriteLine("Las raíces reales son:");
        d = Math.Sqrt(d);
        x1 = (-b + d) / (2 * a);
        x2 = (-b - d) / (2 * a);
        Console.WriteLine("x1 = " + x1 + ", x2 = " + x2);
    }
}
```

EJERCICIOS PROPUESTOS

1. Realice un programa que calcule el volumen de una esfera, que viene dado por la fórmula:

$$v = \frac{4}{3} \pi r^3$$

2. Realice un programa que pregunte el nombre y el año de nacimiento y dé como resultado:

Hola nombre, en el año 2030 tendrás n años

3. Realice un programa que evalúe el polinomio

$$p = 3x^5 - 5x^3 + 2x - 7$$

y visualice el resultado con el siguiente formato:

Para $x = \text{valor}$, $3x^5 - 5x^3 + 2x - 7 = \text{resultado}$

4. Realice el mismo programa anterior, pero empleando ahora coeficientes variables:

$$ax^5 - bx^3 + cx - 7.$$

5. Ejecute el siguiente programa, explique lo que ocurre y realice las modificaciones que sean necesarias para su correcto funcionamiento.

```
using System;
public class Test
{
    public static void Main(string[] args)
    {
        char car = '\0';

        Console.Write("Carácter: ");
        car = (char)Console.Read();
        Console.WriteLine(car);

        Console.Write("Carácter: ");
        car = (char)Console.Read();
        Console.WriteLine(car);
    }
}
```

6. Indique qué resultado da el siguiente programa. A continuación ejecute el programa y compare los resultados.

```
using System;
public class Test
{
    public static void Main(string[] args)
    {
        int car1 = 'A', car2 = 65, car3 = 0;

        car3 = car1 + 'a' - 'A';
        Console.WriteLine((char)car3 + " " + car3);

        car3 = car2 + 32;
        Console.WriteLine((char)car3 + " " + car3);
    }
}
```


SENTENCIAS DE CONTROL

Frecuentemente surge la necesidad de ejecutar unas sentencias u otras en función del valor que tomen una o más expresiones en un instante determinado durante la ejecución del programa. Por ejemplo, en el capítulo anterior, cuando calculábamos las raíces de una ecuación de segundo grado, vimos que en función del valor del discriminante las raíces podían ser reales o complejas. En un caso como éste, surge la necesidad de que sea el propio programa el que tome la decisión, en función del valor del discriminante, de si lo que tiene que calcular son dos raíces reales o dos raíces complejas conjugadas.

Así mismo, en más de una ocasión habrá que ejecutar un conjunto de sentencias un número determinado de veces, o bien hasta que se cumpla una determinada condición. Por ejemplo, en el capítulo anterior hemos visto cómo leer un carácter de la entrada estándar. Pero si lo que queremos es leer, no un carácter sino todos los que escribamos por el teclado hasta detectar la marca de fin de fichero, tendremos que utilizar una sentencia repetitiva.

En este capítulo aprenderá a escribir código para que un programa tome decisiones y para que sea capaz de ejecutar bloques de sentencias repetidas veces.

SENTENCIA **if**

La sentencia **if** permite a un programa tomar una decisión para ejecutar una acción u otra, basándose en el resultado verdadero o falso de una expresión. La sintaxis para utilizar esta sentencia es la siguiente:

```
if (condición)
    sentencia 1;
[else
    sentencia 2];
```

donde *condición* es una expresión booleana, y *sentencia 1* y *sentencia 2* representan a una sentencia simple o compuesta (la parte especificada entre [] es opcional). Cada sentencia simple debe finalizar con un punto y coma.

Una sentencia **if** se ejecuta de la forma siguiente:

1. Se evalúa la *condición* y se obtiene un resultado verdadero o falso.
2. Si el resultado es verdadero (**true**) se ejecutará lo indicado por la *sentencia 1*.
3. Si el resultado es falso (**false**) la *sentencia 1* se ignora y se ejecutará lo indicado por la *sentencia 2*, si la cláusula **else** se ha especificado.
4. En cualquier caso, la ejecución continúa en la siguiente sentencia ejecutable que haya a continuación de la sentencia **if**.

A continuación se exponen algunos ejemplos para que vea de una forma sencilla cómo se utiliza la sentencia **if**.

```
if (x != 0)
    b = a / x;
b = b + 1;
```

En este ejemplo, la condición viene impuesta por la expresión $x \neq 0$. Entonces $b = a / x$, que sustituye a la *sentencia 1* del formato general, se ejecutará si la expresión es verdadera (valor de x distinto de 0) y no se ejecutará si la expresión es falsa (valor de x igual a 0). En cualquier caso, se continúa la ejecución en la línea siguiente, $b = b + 1$. Veamos otro ejemplo:

```
if (a < b) c = c + 1;
// siguiente línea del programa
```

En este otro ejemplo, la condición viene impuesta por una expresión $a < b$. Si al evaluar la condición se cumple que a es menor que b , entonces se ejecuta la sentencia $c = c + 1$. En otro caso, esto es, si a es mayor o igual que b , se continúa en la línea siguiente, ignorándose la sentencia $c = c + 1$.

En el ejemplo siguiente, la condición viene impuesta por la expresión $a \neq 0 \ \&\& \ b \neq 0$. Si al evaluar la condición se cumple que a y b son distintas de cero, entonces se ejecuta la sentencia $x = i$. En otro caso, la sentencia $x = i$ se ignora, continuando la ejecución en la línea siguiente.

```
if (a != 0 && b != 0)
    x = i;
// siguiente línea del programa
```

En el ejemplo siguiente, si se cumple que a es igual a $b*5$, se ejecutan las sentencias $x = 4$ y $a = a + x$. En otro caso, se ejecuta la sentencia $b = 0$. En ambos casos, la ejecución continúa en la siguiente línea del programa.

```
if (a == b * 5)
{
    x = 4;
    a = a + x;
}
else
    b = 0;
// siguiente línea del programa
```

Un error típico es escribir un operador `=` en lugar de `==`. Compare la condición de la sentencia **if** del ejemplo anterior con la siguiente:

```
if (a = b * 5)
// ...
```

En este caso, suponiendo por ejemplo que a es de tipo **int**, el compilador mostrará un mensaje de error indicando que no puede convertir un dato de tipo **int** a otro de tipo **bool**, porque la sentencia anterior es equivalente a escribir:

```
a = b * 5;
if (a)
// ...
```

donde se observa que a no puede dar un resultado de tipo **bool**. Sí sería correcto lo siguiente:

```
a = b * 5;
if (a != 0)
// ...
```

que equivale a:

```
if ((a = b * 5) != 0)
// ...
```

En este otro ejemplo que se muestra a continuación, la sentencia **return** se ejecutará solamente cuando car sea igual al carácter `'s'`.

```
if (car == 's')
    return;
```

ANIDAMIENTO DE SENTENCIAS **if**

Observando el formato general de la sentencia **if** cabe una pregunta: ¿se puede escribir otra sentencia **if** como *sentencia 1* o *sentencia 2*? La respuesta es sí. Esto es, las sentencias **if ... else** pueden estar anidadas. Por ejemplo:

```
if (condición 1)
{
    if (condición 2)
        sentencia 1;
}
else
    sentencia 2;
```

Al evaluarse las condiciones anteriores, pueden presentarse los casos que se indican en la tabla siguiente:

condición 1	condición 2	se ejecuta: sentencia 1	sentencia 2
F	F	no	sí
F	V	no	sí
V	F	no	no
V	V	sí	no

(V = verdadero, F = falso, no = no se ejecuta, sí = sí se ejecuta)

En el ejemplo anterior las llaves definen perfectamente que la cláusula **else** está emparejada con el primer **if**. ¿Qué sucede si quitamos las llaves?

```
if (condición 1)
    if (condición 2)
        sentencia 1;
else
    sentencia 2;
```

Ahora podríamos dudar de a qué **if** pertenece la cláusula **else**. Cuando en el código de un programa aparecen sentencias **if ... else** anidadas, la regla para diferenciar cada una de estas sentencias es que “cada **else** se corresponde con el **if** más próximo que no haya sido emparejado”. Según esto la cláusula **else** está emparejada con el segundo **if**. Entonces, al evaluarse ahora las *condiciones 1 y 2*, pueden presentarse los casos que se indican en la tabla siguiente:

condición 1	condición 2	se ejecuta: sentencia 1	sentencia 2
F	F	no	no
F	V	no	no
V	F	no	sí
V	V	sí	no

(V = verdadero, F = falso, no = no se ejecuta, sí = sí se ejecuta)

Es importante observar que una vez que se ejecuta una acción como resultado de haber evaluado las condiciones impuestas, la ejecución del programa continúa en la siguiente línea a la estructura a que dan lugar las sentencias **if ... else** anidadas. Por ejemplo, si en el ejemplo siguiente ocurre que a no es igual a 0 , la ejecución continúa en la siguiente línea del programa.

```
if (a == 0)
    if (b != 0)
        s = s + b;
    else
        s = s + a;
// siguiente línea del programa
```

Si en lugar de la solución anterior, lo que deseamos es que se ejecute $s = s + a$ cuando a no es igual a 0 , entonces tendremos que incluir entre llaves el segundo **if** sin la cláusula **else**; esto es:

```
if (a == 0)
{
    if (b != 0)
        s = s + b;
}
else
    s = s + a;
// siguiente línea del programa
```

Como ejercicio sobre la teoría expuesta, vamos a realizar una aplicación que dé como resultado el menor de tres números a , b y c . La forma de proceder es comparar cada número con los otros dos una sola vez. La simple lectura del código que se muestra a continuación es suficiente para entender el proceso seguido.

```
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
using System;
using MisClases.ES;

public class CMenor
{
    // Menor de tres números a, b y c
```

```
public static void Main(string[] args)
{
    float a, b, c, menor;

    // Leer los valores de a, b y c
    Console.Write("a : "); a = Leer.datoFloat();
    Console.Write("b : "); b = Leer.datoFloat();
    Console.Write("c : "); c = Leer.datoFloat();

    // Obtener el menor
    if (a < b)
        if (a < c)
            menor = a;
        else
            menor = c;
    else
        if (b < c)
            menor = b;
        else
            menor = c;
    Console.WriteLine("Menor = " + menor);
}
```

Ejecución del programa:

```
a : 25,84
b : -3,1
c : 18
Menor = -3,1
```

ESTRUCTURA **else if**

La estructura presentada a continuación aparece con bastante frecuencia y es por lo que se le da un tratamiento por separado. Esta estructura es consecuencia de las sentencias **if** anidadas. Su formato general es:

```
if (condición 1)
    sentencia 1;
else if (condición 2)
    sentencia 2;
else if (condición 3)
    sentencia 3;
.
.
.
else
    sentencia n;
```


La evaluación de esta estructura sucede así: si se cumple la *condición 1*, se ejecuta la *sentencia 1*, y si no se cumple, se examinan secuencialmente las condiciones siguientes hasta el último **else**, ejecutándose la sentencia correspondiente al primer **else if**, cuya *condición* sea cierta. Si todas las condiciones son falsas, se ejecuta la *sentencia n* correspondiente al último **else**. En cualquier caso, se continúa en la primera sentencia ejecutable que haya a continuación de la estructura. Las *sentencias 1, 2,..., n* pueden ser sentencias simples o compuestas.

Por ejemplo, al efectuar una compra en un cierto almacén, si adquirimos más de 100 unidades de un mismo artículo, nos hacen un descuento de un 40%; entre 25 y 100 un 20%; entre 10 y 24 un 10%; y no hay descuento para una adquisición de menos de 10 unidades. Se pide calcular el importe a pagar. La solución se presentará de la siguiente forma:

```
Código artículo..... 111
Cantidad comprada..... 100
Precio unitario..... 100

Descuento..... 20%
Total..... 8000
```

En la solución presentada como ejemplo, se puede observar que como la cantidad comprada está entre 25 y 100, el descuento aplicado es de un 20%.

La solución de este problema puede ser de la forma siguiente:

- Primero definimos las variables que vamos a utilizar en los cálculos.

```
int ar, cc;
float pu, desc;
```

- A continuación leemos los datos *ar*, *cc* y *pu*.

```
System.Console.WriteLine("Código artículo..... ");
ar = Leer.datoInt();
System.Console.WriteLine("Cantidad comprada..... ");
cc = Leer.datoInt();
System.Console.WriteLine("Precio unitario..... ");
pu = Leer.datoFloat();
```

- Conocidos los datos, realizamos los cálculos y escribimos el resultado.

```
if (cc > 100)
    desc = 40F;           // descuento 40%
else if (cc >= 25)
    desc = 20F;           // descuento 20%
```

```

else if (cc >= 10)
    desc = 10F;        // descuento 10%
else
    desc = 0.0F;       // descuento 0%

System.Console.WriteLine("Descuento..... " +
                          desc + "%");
System.Console.WriteLine("Total..... " +
                          cc * pu * (1 - desc / 100));

```

Se puede observar que las condiciones se han establecido según los descuentos de mayor a menor. Como ejercicio, piense o pruebe que ocurriría si establece las condiciones según los descuentos de menor a mayor. La aplicación completa se muestra a continuación.

```
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
using System;
using MisClases.ES;
```

```
public class CDescuento
{
    public static void Main(string[] args)
    {
        int ar, cc;
        float pu, desc;

        Console.Write("Código artículo..... ");
        ar = Leer.datoInt();
        Console.Write("Cantidad comprada..... ");
        cc = Leer.datoInt();
        Console.Write("Precio unitario..... ");
        pu = Leer.datoFloat();
        Console.WriteLine();

        if (cc > 100)
            desc = 40F;           // descuento 40%
        else if (cc >= 25)
            desc = 20F;           // descuento 20%
        else if (cc >= 10)
            desc = 10F;           // descuento 10%
        else
            desc = 0.0F;           // descuento 0%

        Console.WriteLine("Descuento..... " + desc + "%");
        Console.WriteLine("Total..... " +
            cc * pu * (1 - desc / 100));
    }
}
```

SENTENCIA **switch**

La sentencia **switch** permite ejecutar una de varias acciones, en función del valor de una expresión. Es una sentencia especial para decisiones múltiples. La sintaxis para utilizar esta sentencia es:

```
switch (expresión)
{
    [case expresión-constante 1:]
        [sentencia 1;]
    [case expresión-constante 2:]
        [sentencia 2;]
    [case expresión-constante 3:]
        [sentencia 3;]
    .
    .
    .
    [default:]
        [sentencia n;]
}
```

donde *expresión* es una expresión de tipo entero, enumerado o **string** y *expresión-constante* es una constante del mismo tipo que *expresión* o de un tipo que se pueda convertir implícitamente al tipo de *expresión*, y *sentencia* es una sentencia simple o compuesta. En el caso de que se trate de una sentencia compuesta, no hace falta incluir las sentencias simples que la forman entre {}.

La sentencia **switch** evalúa la expresión entre paréntesis y compara su valor con las constantes de cada **case**. La ejecución de las sentencias del bloque de la sentencia **switch** comienza en el **case** cuya constante coincida con el valor de la expresión, y continúa hasta una sentencia que transfiera el control dentro o fuera del bloque de **switch**; esta sentencia debe estar presente por cada **case** así como para **default**. Generalmente se utiliza **break** para transferir el control fuera del bloque de la sentencia **switch**. La sentencia **switch** puede incluir cualquier número de cláusulas **case** y una cláusula **default** como mucho.

Si no existe una constante igual al valor de la expresión, entonces se ejecutan las sentencias que están a continuación de **default**, si esta cláusula ha sido especificada. La cláusula **default** puede colocarse en cualquier parte del bloque y no necesariamente al final.

En una sentencia **switch** es posible hacer declaraciones en el bloque de cada **case**, igual que en cualquier otro bloque, pero no al principio del bloque **switch**, antes del primer **case**. Por ejemplo:

```
switch (m)
{
    int n = 0, k = 2; // declaración no permitida
    case 7:
        int i = 0;      // declaración permitida
        while ( i < m )
        {
            n += (k + i) * 3;
            i++;
        }
        break;
    case 13:
        // ...
        break;
    // ...
}
```

El error que se ha presentado en el ejemplo anterior puede solucionarse así:

```
int n = 0, k = 2;
switch (m)
{
    // ...
}
```

Para ilustrar la sentencia **switch**, vamos a realizar un programa que lea una fecha representada por dos enteros, *mes* y *año*, y dé como resultado los días correspondientes al *mes*. Esto es:

Mes (##): 5

Año (####): 2012

El mes 5 del año 2012 tiene 31 días

Hay que tener en cuenta que febrero puede tener 28 días, o bien 29 si el año es bisiesto. Un año es bisiesto cuando es múltiplo de 4 y no de 100 o cuando es múltiplo de 400. Por ejemplo, el año 2000 por las dos primeras condiciones no sería bisiesto, pero sí lo es porque es múltiplo de 400; el año 2100 no es bisiesto porque aunque sea múltiplo de 4, también lo es de 100 y no es múltiplo de 400.

La solución de este problema puede ser de la siguiente forma:

- Primero definimos las variables que vamos a utilizar en los cálculos.

```
int días = 0, mes = 0, año = 0;
```

- A continuación leemos los datos *mes* y *año*.

```
System.Console.Write("Mes (##): "); mes = Leer.datoInt();
System.Console.Write("Año (####): "); año = Leer.datoInt();
```

- Después comparamos el *mes* con las constantes 1, 2,..., 12. Si *mes* es 1, 3, 5, 7, 8, 10 o 12 asignamos a *días* el valor 31. Si *mes* es 4, 6, 9 u 11 asignamos a *días* el valor 30. Si *mes* es 2, verificaremos si el *año* es bisiesto, en cuyo caso asignamos a *días* el valor 29 y si no es bisiesto, asignamos a *días* el valor 28. Si *mes* no es ningún valor de los anteriores enviaremos un mensaje al usuario indicándole que el mes no es válido. Todo este proceso lo realizaremos con una sentencia **switch**.

```
switch (mes)
{
    case 1: case 3: case 5: case 7: case 8: case 10: case 12:
        días = 31;
        break;
    case 4: case 6: case 9: case 11:
        días = 30;
        break;
    case 2:
        // ¿Es el año bisiesto?
        if ((año % 4 == 0) && (año % 100 != 0) || (año % 400 == 0))
            días = 29;
        else
            días = 28;
        break;
    default:
        System.Console.WriteLine("\nEl mes no es válido");
        break;
}
```

Cuando una constante coincida con el valor de *mes*, se ejecutan las sentencias especificadas a continuación de la misma, siguiendo la ejecución del programa por los bloques de las siguientes cláusulas **case**, a no ser que se tome una acción explícita para abandonar el bloque de la sentencia **switch**. Ésta es precisamente la función de la sentencia **break** al final de cada bloque **case**.

- Por último si el *mes* es válido, escribimos el resultado solicitado.

```
if (mes >= 1 && mes <= 12)
    System.Console.WriteLine("\nEl mes " + mes + " del año " +
        año + " tiene " + días + " días");
```

El programa completo se muestra a continuación:

El que las cláusulas **case** estén una a continuación de otra o una debajo de otra no es más que una cuestión de estilo, ya que C# interpreta cada carácter nueva línea como un espacio en blanco; esto es, el código al que llega el compilador es el mismo en cualquier caso.

La sentencia **break** que se ha puesto a continuación de la última cláusula, en el ejemplo **default**, es necesaria. De esta forma, cuando tengamos que añadir otro caso ya tenemos puesto **break**, con lo que hemos eliminado una posible fuente de errores.

Se deja como ejercicio para el lector declarar la variable *mes* de tipo **string**, lo que permitirá sustituir los meses 1, 2,..., 12 por las cadenas de caracteres correspondientes (enero, febrero,..., diciembre).

SENTENCIA while

La sentencia **while** ejecuta una sentencia, simple o compuesta, cero o más veces, dependiendo del valor de una expresión booleana. Su sintaxis es:

```
while (condición)
    sentencia;
```

donde *condición* es cualquier expresión booleana y *sentencia* es una sentencia simple o compuesta.

La ejecución de la sentencia **while** sucede así:

1. Se evalúa la *condición* y se obtiene un resultado verdadero o falso.
2. Si el resultado es falso (**false**), la *sentencia* no se ejecuta y se pasa el control a la siguiente sentencia en el programa.
3. Si el resultado de la evaluación es verdadero (**true**), se ejecuta la *sentencia* y el proceso descrito se repite desde el punto 1.

Por ejemplo, el siguiente ejemplo, cuyo código podrá ser incluido en cualquier aplicación, solicita obligatoriamente una de las dos respuestas posibles: *s/n* (sí o no).

```
using System;

public class Test
{
    public static void Main(string[] args)
    {
        char car = '\0';
```

```
Console.Write("Desea continuar s/n (sí o no) ");
car = (char)Console.Read();
// Eliminar los caracteres disponibles en el flujo de entrada
Console.ReadLine();
while (car != 's' && car != 'n')
{
    Console.Write("Desea continuar s/n (sí o no) ");
    car = (char)Console.Read();
    Console.ReadLine();
}
}
```

Ejecución del programa:

```
Desea continuar s/n (sí o no) x
Desea continuar s/n (sí o no) c
Desea continuar s/n (sí o no) n
```

Observe que antes de ejecutarse la sentencia **while** se visualiza el mensaje “Desea continuar s/n (sí o no)” y se inicia la condición; esto es, se asigna un carácter a la variable *car* que interviene en la condición de la sentencia **while**.

La sentencia **while** se interpreta de la forma siguiente: mientras el valor de *car* no sea igual al carácter ‘s’ ni al carácter ‘n’, visualizar el mensaje “Desea continuar s/n (sí o no)” y leer otro carácter. Esto obliga al usuario a escribir el carácter ‘s’ o ‘n’ en minúsculas.

El ejemplo expuesto, puede escribirse de forma más simplificada así:

```
char car = '\0';

Console.Write("Desea continuar s/n (sí o no) ");
while ((car = (char)Console.Read()) != 's' && car != 'n')
{
    // Eliminar los caracteres disponibles en el flujo de entrada
    Console.ReadLine();
    Console.Write("Desea continuar s/n (sí o no) ");
}
```

La diferencia de este ejemplo con respecto al anterior es que ahora la *condición* incluye la lectura de la variable *car*, que se ejecuta primero por estar entre paréntesis. A continuación se compara *car* con los caracteres ‘s’ y ‘n’.

El ejemplo anterior requiere leer un solo carácter pero, si tuviéramos que leer una cadena de caracteres, ¿cómo sabrá el programa cuándo finaliza la entrada de información? Esto se puede hacer incluyendo en el **while** una condición de termi-

nación como: leer datos hasta alcanzar la marca de fin de fichero. Recuerde que para el flujo estándar de entrada, esta marca se produce cuando se pulsán las teclas *Ctrl+D* en UNIX, o bien *Ctrl+Z* en aplicaciones Windows de consola, y que cuando **Read** lee una marca de fin de fichero, devuelve un valor negativo. Por ejemplo, el siguiente programa visualiza el código Unicode de cada uno de los caracteres del texto introducido por el teclado.

```
using System;

public class CUnicode
{
    // Código Unicode de cada uno de los caracteres de un texto
    public static void Main(string[] args)
    {
        int car = 0; // car = carácter nulo (\0)

        Console.WriteLine("Introduzca texto.");
        Console.WriteLine("Para terminar pulse Ctrl+z\n");
        while ((car = Console.Read()) > -1)
        {
            if (car != '\r' && car != '\n')
                Console.WriteLine("El código Unicode de " + (char)car +
                                   " es " + car);
        }
    }
}
```

Una solución posible de este programa es la siguiente:

```
hola[Entrar]
El código Unicode de h es 104
El código Unicode de o es 111
El código Unicode de l es 108
El código Unicode de a es 97
adiós[Entrar]
El código Unicode de a es 97
El código Unicode de d es 100
El código Unicode de i es 105
El código Unicode de ó es 243
El código Unicode de s es 115
[Ctrl][z]
```

El resultado mostrado permite observar que el bucle **while** se está ejecutando sin pausa mientras hay caracteres en el flujo de entrada. Cuando dicho flujo queda vacío y se ejecuta el método **Read** de nuevo, la ejecución se detiene a la espera de nuevos datos. Lógicamente, habrá comprendido que aunque se lea carácter a carácter se puede escribir, hasta pulsar *Entrar*, un texto cualquiera.

Los caracteres `\r` y `\n` introducidos al pulsar *Entrar* son ignorados porque así se ha programado.

Bucles anidados

Cuando se incluye una sentencia **while** dentro de otra sentencia **while**, en general una sentencia **while**, **do** o **for** dentro de otra de ellas, estamos en el caso de bucles anidados. Por ejemplo:

```
public static void Main(string[] args)
{
    int i = 1, j = 1;
    while ( i <= 3 )    // mientras i sea menor o igual que 3
    {
        System.Console.Write("Para i = " + i + ": ");
        while ( j <= 4 ) // mientras j sea menor o igual que 4
        {
            System.Console.Write("j = " + j + ", ");
            j++; // aumentar j en una unidad
        }
        System.Console.WriteLine(); // avanzar a una nueva línea
        i++; // aumentar i en una unidad
        j = 1; // iniciar j de nuevo a 1
    }
}
```

Al ejecutar este método se obtiene el siguiente resultado:

```
Para i = 1: j = 1, j = 2, j = 3, j = 4,
Para i = 2: j = 1, j = 2, j = 3, j = 4,
Para i = 3: j = 1, j = 2, j = 3, j = 4,
```

Este resultado demuestra que el bucle exterior se ejecuta tres veces y, por cada una de éstas, el bucle interior se ejecuta, a su vez, cuatro veces. Es así como se ejecutan los bucles anidados: por cada iteración del bucle externo, el interno se ejecuta hasta finalizar todas sus iteraciones.

Observe también que cada vez que finaliza la ejecución de la sentencia **while** interior, avanzamos a una nueva línea, incrementamos el valor de *i* en una unidad e iniciamos de nuevo *j* al valor 1.

Como aplicación de lo expuesto, vamos a realizar un programa que imprima los números *z*, comprendidos entre 1 y 50, que cumplan la expresión:

$$z^2 = x^2 + y^2$$

donde z , x e y son números enteros positivos. El resultado se presentará de la forma siguiente:

Z	X	Y
5	3	4
13	5	12
10	6	8
...	...	...
50	30	40

La solución de este problema puede ser de la siguiente forma:

- Primero definimos las variables que vamos a utilizar en los cálculos.

```
int x = 1, y = 1, z = 0;
```

- A continuación escribimos la cabecera de la solución.

```
System.Console.WriteLine("Z\t" + "X\t" + "Y");
System.Console.WriteLine("_____");
```

- Después, para $x = 1$, e $y = 1, 2, 3, \dots$, para $x = 2$, e $y = 2, 3, 4, \dots$, para $x = 3$, e $y = 3, 4, \dots$, hasta $x = 50$, calculamos la $\sqrt{x^2 + y^2}$; llamamos a este valor z (observe que y es igual o mayor que x para evitar que se repitan pares de valores como $x = 3, y = 4$ y $x = 4, y = 3$). Si z es exacto, escribimos z , x e y . Esto es, para los valores descritos de x e y , hacemos los cálculos:

```
z = (int)Math.Sqrt(x * x + y * y); // z es una variable entera
if (z * z == x * x + y * y)      // ¿la raíz cuadrada fue exacta?
    System.Console.WriteLine(z + "\t" + x + "\t" + y);
```

Además, siempre que obtengamos un valor z mayor que 50 lo desecharemos y continuaremos con un nuevo valor de x y los correspondientes valores de y .

El programa completo se muestra a continuación:

```
using System;

public class CPitagoras
{
    // Teorema de Pitágoras
    public static void Main(string[] args)
    {
        int x = 1, y = 1, z = 0;
        Console.WriteLine("Z\t" + "X\t" + "Y");
```

```
Console.WriteLine("_____");
while (x <= 50)
{
    // Calcular z. Como z es un entero, almacena
    // la parte entera de la raíz cuadrada
    z = (int)Math.Sqrt(x * x + y * y);
    while (y <= 50 && z <= 50)
    {
        // Si la raíz cuadrada anterior fue exacta,
        // escribir z, x e y
        if (z * z == x * x + y * y)
            Console.WriteLine(z + "\t" + x + "\t" + y);
        y = y + 1;
        z = (int)Math.Sqrt(x * x + y * y);
    }
    x = x + 1; y = x;
}
}
```

SENTENCIA **do ... while**

La sentencia **do ... while** ejecuta una sentencia, simple o compuesta, una o más veces dependiendo del valor de una expresión. Su sintaxis es la siguiente:

```
do
    sentencia;
while (condición);
```

donde *condición* es cualquier expresión booleana y *sentencia* es una sentencia simple o compuesta. Observe que la sentencia **do ... while** finaliza con un punto y coma.

La ejecución de una sentencia **do ... while** sucede de la siguiente forma:

1. Se ejecuta el bloque (sentencia simple o compuesta) de **do**.
2. Se evalúa la expresión correspondiente a la *condición* de finalización del bucle y se obtiene un resultado verdadero o falso.
3. Si el resultado es falso (**false**), se pasa el control a la siguiente sentencia en el programa.
4. Si el resultado es verdadero (**true**), el proceso descrito se repite desde el punto 1.

Por ejemplo, el siguiente código obliga al usuario a introducir un valor positivo:

```
double n;
do // ejecutar las sentencias siguientes
{
    System.Console.Write("Número: ");
    n = Leer.datoDouble();
}
while ( n < 0 ); // mientras n sea menor que 0
```

Cuando se utiliza una sentencia **do ... while** el bloque de sentencias se ejecuta al menos una vez, porque la condición de terminación se evalúa al final. En cambio, cuando se ejecuta una sentencia **while** puede suceder que el bloque de sentencias no se ejecute, lo que ocurrirá siempre que la condición de terminación sea inicialmente falsa.

Como ejercicio, vamos a realizar un programa que calcule la raíz cuadrada de un número n por el método de Newton. Este método se enuncia así: sea r_i la raíz cuadrada aproximada de n . La siguiente raíz aproximada r_{i+1} se calcula en función de la anterior así:

$$r_{i+1} = \frac{\frac{n}{r_i} + r_i}{2}$$

El proceso descrito se repite hasta que la diferencia en valor absoluto de las dos últimas aproximaciones calculadas sea tan pequeña como nosotros queramos (teniendo en cuenta los límites establecidos por el tipo de datos utilizado). Según esto, la última aproximación será una raíz válida, cuando se cumpla que:

$$abs(r_i - r_{i+1}) \leq \varepsilon$$

La solución de este problema puede ser de la siguiente forma:

- Primero definimos las variables que vamos a utilizar en los cálculos.

```
double n;           // número
double aprox;       // aproximación a la raíz cuadrada
double antaprox;    // anterior aproximación a la raíz cuadrada
double epsilon;     // coeficiente de error
```

- A continuación leemos los datos n , $aprox$ y $epsilon$.

```
System.Console.Write("Número: ");
n = Leer.datoDouble();
System.Console.Write("Raíz cuadrada aproximada: ");
aprox = Leer.datoDouble();
System.Console.Write("Coeficiente de error: ");
```

```
epsilon = Leer.datoDouble();
```

- Después, se aplica la fórmula de Newton.

```
do
{
    antaprox = aprox;
    aprox = (n / antaprox + antaprox) / 2;
}
while (Math.Abs(aprox - antaprox) >= epsilon);
```

Al aplicar la fórmula por primera vez, la variable *antaprox* contiene el valor aproximado a la raíz cuadrada que hemos introducido a través del teclado. Para sucesivas veces, *antaprox* contendrá la última aproximación calculada.

- Cuando la condición especificada en la sentencia **do ... while** mostrada anteriormente sea falsa, el proceso habrá terminado. Sólo queda imprimir el resultado.

```
Console.WriteLine("La raíz cuadrada de {0:F2} es {1:F2}", n, aprox);
```

El programa completo se muestra a continuación. Para no permitir la entrada de números negativos, se ha utilizado una sentencia **do ... while** que preguntará por el valor solicitado mientras el introducido sea negativo.

```
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
using System;
using MisClases.ES;

public class CRaizCuadrada
{
    // Raíz cuadrada. Método de Newton.
    public static void Main(string[] args)
    {
        double n;           // número
        double aprox;        // aproximación a la raíz cuadrada
        double antaprox;     // anterior aproximación a la raíz cuadrada
        double epsilon;     // coeficiente de error

        do
        {
            Console.Write("Número: ");
            n = Leer.datoDouble();
        }
        while ( n < 0 );
        do
        {
```

```

        Console.Write("Raíz cuadrada aproximada: ");
        aprox = Leer.datoDouble();
    }
    while ( aprox <= 0 );
    do
    {
        Console.Write("Coeficiente de error: ");
        epsilon = Leer.datoDouble();
    }
    while ( epsilon <= 0 );
    do
    {
        antaprox = aprox;
        aprox = (n/antaprox + antaprox) / 2;
    }
    while (Math.Abs(aprox - antaprox) >= epsilon);
    Console.WriteLine("La raíz cuadrada de {0:F2} es {1:F2}", n, aprox);
}
}

```

Si ejecuta este programa para un valor de n igual a 10, obtendrá la siguiente solución:

```

Número: 10
Raíz cuadrada aproximada: 1
Coeficiente de error: 1e-4
La raíz cuadrada de 10,00 es 3,16

```

SENTENCIA for

La sentencia **for** permite ejecutar una sentencia simple o compuesta, repetidamente un número de veces conocido. Su sintaxis es la siguiente:

```

for ([v1=e1 [, v2=e2]...];[condición];[progresión-condición])
    sentencia;

```

- $v1, v2, \dots$, representan variables de control que serán iniciadas con los valores de las expresiones $e1, e2, \dots$
- *condición* es una expresión booleana que si se omite, se supone verdadera.
- *progresión-condición* es una o más expresiones separadas por comas cuyos valores evolucionan en el sentido de que se cumpla la condición para finalizar la ejecución de la sentencia **for**.
- *sentencia* es una sentencia simple o compuesta.

La ejecución de la sentencia **for** sucede de la siguiente forma:

1. Se inician las variables $v1, v2, \dots$

2. Se evalúa la condición de finalización del bucle obteniéndose un resultado verdadero o falso:
 - a) Si el resultado es verdadero (**true**), se ejecuta el bloque de sentencias, se evalúa la expresión que da lugar a la progresión de la condición y se vuelve al punto 2.
 - b) Si el resultado es falso (**false**), la ejecución de la sentencia **for** se da por finalizada y se pasa el control a la siguiente sentencia en el programa.

Por ejemplo, la siguiente sentencia **for** imprime los números del 1 al 100. Literalmente dice: desde *i* igual a 1, mientras *i* sea menor o igual que 100, incrementando la *i* de uno en uno, escribir el valor de *i*.

```
int i;  
for (i = 1; i <= 100; i++)  
    System.Console.Write(i + " ");
```

El siguiente ejemplo imprime los múltiplos de 7 que hay entre 7 y 112. Se puede observar que, en este caso, la variable se ha declarado e iniciado en la propia sentencia **for** por lo que tan sólo puede utilizarse en el mismo (esto no se puede hacer en una sentencia **while**; las variables que intervienen en la condición de una sentencia **while** deben haber sido declaradas e iniciadas antes de que se procese la condición por primera vez).

```
for (int k = 7; k <= 112; k += 7)  
    System.Console.Write(k + " ");
```

En el siguiente ejemplo se puede observar la utilización de la coma como separador de las variables de control y de las expresiones que hacen que evolucionen los valores que intervienen en la condición de finalización.

```
int f, c;  
for (f = 3, c = 6; f + c < 40; f++, c += 2)  
    System.Console.WriteLine("f = " + f + "\tc = " + c);
```

Este otro ejemplo que ve a continuación imprime los valores desde 1 hasta 10 con incrementos de 0.5.

```
for (float i = 1; i <= 10; i += 0.5F)  
    System.Console.Write(i + " ");
```

El ejemplo siguiente imprime las letras del abecedario en orden inverso.

```
char car;  
for (car = 'z'; car >= 'a'; car--)  
    System.Console.Write(car + " ");
```


El ejemplo siguiente indica cómo realizar un bucle infinito. Para salir de un bucle infinito tiene que pulsar las teclas *Ctrl+C*.

```
for (;;)
{
    sentencias;
}
```

Como aplicación de la sentencia **for** vamos a imprimir un tablero de ajedrez en el que las casillas blancas se simbolizarán con una B y las negras con una N. Así mismo, el programa deberá marcar con * las casillas a las que se puede mover un alfil desde una posición dada. La solución será similar a la siguiente:

```
Posición del alfil:
fila      3
columna   4
```

```
B * B N B * B N
N B * B * B N B
B N B * B N B N
N B * B * B N B
B * B N B * B N
* B N B N B * B
B N B N B N B *
N B N B N B N B
```

Desarrollo del programa:

- Primero definimos las variables que vamos a utilizar.

```
int falfil, calfil; // posición inicial del alfil
int fila, columna; // posición actual del alfil
```

- Leer la fila y la columna en la que se coloca el alfil.

```
System.Console.Write(" fila      "); falfil = Leer.datoInt();
System.Console.Write(" columna "); calfil = Leer.datoInt();
```

- Partiendo de la fila 1, columna 1 y recorriendo el tablero por filas,

```
for (fila = 1; fila <= 8; fila++)
{
    for (columna = 1; columna <= 8; columna++)
    {
        // Pintar el tablero de ajedrez
    }
    System.Console.WriteLine(); // cambiar de fila
}
```

imprimir un *, una B o una N dependiendo de las condiciones especificadas a continuación:

- ◇ Imprimir un * si se cumple que la suma o diferencia de la fila y columna actuales coincide con la suma o diferencia de la fila y columna donde se coloca el alfil.
- ◇ Imprimir una B si se cumple que la fila más la columna actuales es par.
- ◇ Imprimir una N si se cumple que la fila más la columna actuales es impar.

```
// Pintar el tablero de ajedrez
if ((fila + columna == falfil + calfil) ||
    (fila - columna == falfil - calfil))
    System.Console.Write("* ");
else if ((fila + columna) % 2 == 0)
    System.Console.Write("B ");
else
    System.Console.Write("N ");
```

El programa completo se muestra a continuación:

```
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
using System;
using MisClases.ES;

public class CAjedrez
{
    // Imprimir un tablero de ajedrez.
    public static void Main(string[] args)
    {
        int falfil, calfil; // posición inicial del alfil
        int fila, columna; // posición actual del alfil

        Console.WriteLine("Posición del alfil:");
        Console.Write("  fila    "); falfil = Leer.datoInt();
        Console.Write("  columna "); calfil = Leer.datoInt();
        Console.WriteLine(); // dejar una línea en blanco

        // Pintar el tablero de ajedrez
        for (fila = 1; fila <= 8; fila++)
        {
            for (columna = 1; columna <= 8; columna++)
            {
                if ((fila + columna == falfil + calfil) ||
                    (fila - columna == falfil - calfil))
                    Console.Write("* ");
                else if ((fila + columna) % 2 == 0)
                    Console.Write("B ");
                else
```

```

        Console.WriteLine("N ");
    }
    Console.WriteLine(); // cambiar de fila
}
}
}

```

SENTENCIA foreach

Repite un grupo de sentencias para cada elemento de una matriz o de una colección. Esta sentencia será estudiada con detalle en un capítulo más adelante.

SENTENCIA break

Anteriormente vimos que la sentencia **break** finaliza la ejecución de una sentencia **switch**. Pues bien, cuando se utiliza **break** en el bloque correspondiente a una sentencia **while**, **do**, **for** o **foreach**, hace lo mismo: finaliza la ejecución del bucle.

Cuando las sentencias **switch**, **while**, **do**, **for**, o **foreach** estén anidadas, la sentencia **break** solamente finaliza la ejecución del bucle donde esté incluida.

Por ejemplo, el bucle interno de la aplicación *CPitagoras* desarrollada anteriormente, podría escribirse también así:

```

while (y <= 50)
{
    // Si la raíz cuadrada anterior fue exacta,
    // escribir z, x e y
    if (z * z == x * x + y * y)
        System.Console.WriteLine(z + "\t" + x + "\t" + y);
    y = y + 1;
    z = (int)Math.Sqrt(x * x + y * y);
    if (z > 50) break; // salir del bucle
}

```

SENTENCIA continue

La sentencia **continue** obliga a ejecutar la siguiente iteración del bucle **while**, **do**, **for** o **foreach** en el que está contenida. Su sintaxis es:

```
continue;
```

Como ejemplo, vea la siguiente aplicación que imprime todos los números entre 1 y 100 que son múltiplos de 5.

```
public class Test
{
    public static void Main(string[] args)
    {
        for (int n = 0; n <= 100; n++)
        {
            // Si n no es múltiplo de 5, siguiente iteración
            if (n % 5 != 0) continue;
            // Imprime el siguiente múltiplo de 5
            System.Console.WriteLine(n + " ");
        }
    }
}
```

Ejecute este programa y observe que cada vez que se ejecuta la sentencia **continue**, se inicia la ejecución del bucle **for** para el siguiente valor de *n*.

SENTENCIA goto

La sentencia **goto** transfiere el control a una línea específica del programa, identificada por una *etiqueta*. Su sintaxis es la siguiente:

```
goto etiqueta;
.
.
.
etiqueta:
sentencia;
```

Si la línea a la que se transfiere el control es una sentencia ejecutable, se ejecuta esa sentencia y las que le siguen. Si no es ejecutable, la ejecución se inicia en la primera sentencia ejecutable que se encuentre a continuación de dicha línea.

No se puede transferir el control fuera del cuerpo del método en el que nos encontremos.

Un uso abusivo de esta sentencia da lugar a programas difíciles de interpretar y de mantener. Por ello, se utiliza solamente en ocasiones excepcionales. La función que desempeña una sentencia **goto** puede suplirse utilizando las sentencias **if...else**, **do**, **for**, **switch**, **while**.

El uso más normal consiste en abandonar la ejecución de alguna estructura profundamente anidada, cosa que no puede hacerse mediante la sentencia **break**, ya que ésta se limita únicamente a un solo nivel de anidamiento.

El siguiente ejemplo muestra cómo se utiliza la sentencia **goto**. Consta de dos bucles **for** anidados. En el bucle interior hay una sentencia **goto** que se ejecutará si se cumple la condición especificada. Si se ejecuta la sentencia **goto** el control es transferido a la primera sentencia ejecutable que haya a continuación de la etiqueta *salir*.

```
const int K = 8;
int f = 0, c = 0, n = 20;

for (f = 0; f < K; f++)
{
    for (c = 0; c < K; c++)
    {
        if (f*c > n) goto salir;
    }
}
salir:
if (f < K && c < K)
    Console.WriteLine("{0}, {1})", f, c);
```

Ejecución del código:

(3, 7)

SENTENCIAS try ... catch

En el capítulo anterior expusimos que cuando durante la ejecución de un programa se produce un error que impide su continuación, el entorno de ejecución lanza una excepción que hace que se visualice un mensaje acerca de lo ocurrido y se detenga la ejecución de dicho programa. Cuando esto ocurra, si no deseamos que la ejecución se detenga, habrá que utilizar la sentencia **try** para poner en alerta al programa acerca del código que puede lanzar una excepción y utilizar **catch** para capturar y manejar cada excepción que se lance. Por ejemplo, si ejecuta la aplicación *Test* que se muestra un poco más adelante, lanzará la excepción del tipo **System.DivideByZeroException** que se indica a continuación:

```
Excepción no controlada: System.DivideByZeroException: Intento de
dividir por cero. en ConsoleApplication1.Test.Main(String[] args)
en C:\...\Test\Test.cs:line 9
```

La información dada por el mensaje anterior, además del tipo de excepción, especifica que ha ocurrido una división por cero en el método **Main**, en la línea 9 del fichero fuente *Test.cs*.

```
public class Test
{
```

```
public static void Main(string[] args)
{
    int dato1 = 0, dato2 = 0, dato3;

    System.Console.WriteLine("Se inicia la aplicación");
    dato1++;
    dato3 = dato1 / dato2;
    dato2++;
    // Otras sentencias
    System.Console.WriteLine(dato1 + " " + dato2 + " " + dato3);
}
}
```

Modifiquemos la aplicación con la intención de capturar la excepción lanzada. El resultado puede ser el siguiente:

```
public class Test
{
    public static void Main(string[] args)
    {
        int dato1 = 0, dato2 = 0, dato3 = 0;

        System.Console.WriteLine("Se inicia la aplicación");
        try
        {
            dato1++;
            dato3 = dato1 / dato2;
            dato2++;
            // Otras sentencias
        }
        catch(System.DivideByZeroException e)
        {
            // Manejar una excepción de tipo DivideByZeroException
            System.Console.WriteLine("Error: " + e.Message);
            dato3 = dato1;
        }
        System.Console.WriteLine(dato1 + " " + dato2 + " " + dato3);
    }
}
```

Ahora, si la sentencia $dato3 = dato1 / dato2$ da lugar a una división por cero, el entorno de ejecución detendrá temporalmente la ejecución de la aplicación y lanzará una excepción de tipo **DivideByZeroException** que será capturada por la sentencia **catch**. La ejecución del programa se reanudará a partir de la primera sentencia perteneciente al bloque **catch** y continuará hasta el final del programa. Se puede observar que la opción que se ha tomado ante la excepción lanzada ha sido suponer una división entre 1; esto es: $dato3 = dato1$. El resultado cuando finalice la aplicación será:

```

Se inicia la aplicación
Error: Attempted to divide by zero.
1 0 1

```

EJERCICIOS RESUELTOS

1. Escribir una clase *CFecha* que permita crear objetos que encapsulen una fecha correcta. El dato año deberá ser igual o mayor que 2000. Se debe tener en cuenta que febrero puede tener 28 o 29 días si el año es bisiesto.

La clase proporcionará una interfaz que permita al usuario construir un objeto con una fecha determinada (por omisión la fecha será 1/1/2000), establecer una nueva fecha y obtener el día, el mes o el año.

```

using System;

public class CFecha
{
    private int día = 1;
    private int mes = 1;
    private int año = 2000;

    public CFecha() {}

    public CFecha(int d, int m, int a)
    {
        EstablecerFecha(d, m, a);
    }

    public void EstablecerFecha(int d, int m, int a)
    {
        día = d; mes = m; año = a;
        if ( !FechaCorrecta() )
        {
            día = 1; mes = 1; año = 2000;
            Console.WriteLine("Fecha no correcta. Se asume 1/1/2000");
        }
    }

    public int ObtenerDía()
    {
        return día;
    }

    public int ObtenerMes()
    {
        return mes;
    }
}

```

```
public int ObtenerAño()
{
    return año;
}

private bool FechaCorrecta()
{
    bool día_correcto = true;
    bool mes_correcto = true;
    bool año_correcto = true;

    año_correcto = año >= 2000;
    switch (mes)
    {
        case 1:        // enero
        case 3:        // marzo
        case 5:        // mayo
        case 7:        // julio
        case 8:        // agosto
        case 10:       // octubre
        case 12:       // diciembre
            día_correcto = día >= 1 && día <= 31;
            break;
        case 4:        // abril
        case 6:        // junio
        case 9:        // septiembre
        case 11:       // noviembre
            día_correcto = día >= 1 && día <= 30;
            break;
        case 2:        // febrero
            // Es el año bisiesto?
            if ((año % 4 == 0) && (año % 100 != 0) || (año % 400 == 0))
                día_correcto = día >= 1 && día <= 29;
            else
                día_correcto = día >= 1 && día <= 28;
            break;
        default:
            mes_correcto = false;
            break;
    }
    return día_correcto && mes_correcto && año_correcto;
}
```

A continuación, escribir un programa que utilizando la clase anterior cree un objeto *CFecha* y después muestre la fecha que encapsula.

```
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
using System;
```



```

using MisClases.ES;

public class Test
{
    public static void Main(string[] args)
    {
        int día, mes, año;
        Console.Write("Día: "); día = Leer.datoInt();
        Console.Write("Mes: "); mes = Leer.datoInt();
        Console.Write("Año: "); año = Leer.datoInt();

        CFecha fecha = new CFecha(día, mes, año);

        Console.WriteLine("Fecha: " + fecha.ObtenerDía() + "/" +
                           fecha.ObtenerMes() + "/" +
                           fecha.ObtenerAño() );
    }
}

```

2. Realizar un programa que a través de un menú permita realizar las operaciones de *sumar, restar, multiplicar, dividir y salir*. Las operaciones constarán solamente de dos operandos. El menú será visualizado por un método sin argumentos, que devolverá como resultado la opción elegida. La ejecución será de la forma siguiente:

```

1. sumar
2. restar
3. multiplicar
4. dividir
5. salir

```

```

Seleccione la operación deseada: 1
Dato 1 ([Entrar] para Dato1 = resultado anterior): 2,5
Dato 2: 3,2
Resultado = 5,7

```

Si a la solicitud del *Dato 1* se responde pulsando la tecla *Entrar*, se asumirá el resultado anterior como *Dato 1*.

La solución de este problema puede ser de la siguiente forma. Primero definimos una clase *CCalculadora* con los atributos *operando1*, *operando2* y *resultado*, y los métodos *EstablecerOperandos*, *Resultado*, *menú*, *Sumar*, *Restar*, *Multiplicar* y *Dividir*.

```

// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
using System;
using MisClases.ES;

```

```
public class CCalculadora
{
    // Simulación de una calculadora
    private double operando1;
    private double operando2;
    private double resultado;

    public void EstablecerOperandos(double op1, double op2)
    {
        operando1 = op1;
        operando2 = op2;
    }

    public double Resultado()
    {
        return resultado;
    }

    public int menú()
    {
        int op;
        do
        {
            Console.WriteLine("\t1. sumar");
            Console.WriteLine("\t2. restar");
            Console.WriteLine("\t3. multiplicar");
            Console.WriteLine("\t4. dividir");
            Console.WriteLine("\t5. salir");
            Console.Write("\nSelecione la operación deseada: ");
            op = Leer.datoInt();
        }
        while (op < 1 || op > 5);
        return op;
    }

    public double Sumar()
    {
        resultado = operando1 + operando2;
        return resultado;
    }

    public double Restar()
    {
        resultado = operando1 - operando2;
        return resultado;
    }

    public double Multiplicar()
    {
        resultado = operando1 * operando2;
        return resultado;
    }
}
```

```

public double Dividir()
{
    resultado = operando1 / operando2;
    return resultado;
}
}

```

Observe que los métodos *Sumar*, *Restar*, *Multiplicar* y *Dividir*, además de almacenar el resultado de la operación que realizan, lo devuelven. Esto es sólo una cuestión de eficacia; de esta forma podremos emplear una llamada al método como argumento en cualquier operación.

A continuación escribimos una clase *Test* que utilizando un objeto *CCalculadora* simule el trabajo con una calculadora. El método **Main** de esta clase, primero solicitará la operación a realizar. Si la operación elegida no fue *salir*, solicitará los operandos *dato1* y *dato2*. Después realizará la operación elegida con los datos introducidos y mostrará el resultado. Las operaciones descritas formarán parte de un bucle infinito formado por una sentencia **while** con el fin de poder encadenar distintas operaciones.

El programa completo se muestra a continuación.

```

// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
using System;
using MisClases.ES;

public class Test
{
    public static void Main(string[] args)
    {
        CCalculadora MiCalculadora = new CCalculadora();
        double dato1 = 0, dato2 = 0;
        int operación = 0;

        while (true)
        {
            operación = MiCalculadora.menú();

            if (operación != 5)
            {
                // Leer datos
                Console.Write("Dato 1 ([Entrar] para Dato1 = resultado anterior): ");
                dato1 = Leer.datoDouble();
                if (Double.IsNaN(dato1)) // se pulsó [Entrar]
                    dato1 = MiCalculadora.Resultado();
                Console.Write("Dato 2: "); dato2 = Leer.datoDouble();
                MiCalculadora.EstablecerOperandos(dato1, dato2);
            }
        }
    }
}

```

```
// Realizar la operación
switch (operación)
{
    case 1:
        MiCalculadora.Sumar();
        break;
    case 2:
        MiCalculadora.Restar();
        break;
    case 3:
        MiCalculadora.Multiplicar();
        break;
    case 4:
        MiCalculadora.Dividir();
        break;
}
// Escribir el resultado
Console.WriteLine("Resultado = " + MiCalculadora.Resultado() + "\n");
}
else
    break;
}
}
```

El método **IsNaN** de la clase **Double** es un método **static** que devuelve **true** si su argumento se corresponde con un dato no numérico y **false** en otro caso.

EJERCICIOS PROPUESTOS

1. Realice un programa que calcule e imprima la suma de los múltiplos de 5 comprendidos entre dos valores a y b . El programa no permitirá introducir valores negativos para a y b , y verificará que a es menor que b . Si a es mayor que b , intercambiará estos valores.
2. Realice un programa que permita evaluar la serie:

$$\sum_{a=0}^b \frac{1}{x + ay}$$

3. Si quiere averiguar su número de Tarot, sume los números de su fecha de nacimiento y a continuación redúzcalos a un único dígito; por ejemplo si su fecha de nacimiento fuera 17 de octubre de 1970, los cálculos a realizar serían:

$$17 + 10 + 1970 = 1997 \Rightarrow 1 + 9 + 9 + 7 = 26 \Rightarrow 2 + 6 = 8$$

lo que quiere decir que su número de Tarot es el 8.

Realice un programa que pida una fecha *día, mes y año* (*día, mes y año* son enteros) y dé como resultado el número de Tarot. El programa verificará si la fecha es correcta, esto es, los valores están dentro de los rangos permitidos.

4. Realice un programa para jugar con el ordenador a acertar números. El ordenador piensa un número y nosotros debemos acertar cuál es, en un número de intentos determinado. Por cada intento sin éxito el ordenador nos irá indicando si el número especificado es mayor o menor que el pensado por él. El número pensado por el ordenador se puede obtener multiplicando por una constante el valor devuelto por el método **Random** de la clase **Math**, y los números pensados por nosotros los introduciremos por el teclado.
5. Realice un programa que calcule las raíces de la ecuación:

$$ax^2 + bx + c = 0$$

teniendo en cuenta los siguientes casos:

- 1) Si a es igual a 0 y b es igual a 0, imprimiremos un mensaje diciendo que la ecuación es degenerada.
- 2) Si a es igual a 0 y b no es igual a 0, existe una raíz única con valor $-c/b$.
- 3) En los demás casos, utilizaremos la fórmula siguiente:

$$x_i = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

La expresión $d = b^2 - 4ac$ se denomina discriminante.

- Si d es mayor o igual que 0, entonces hay dos raíces reales.
- Si d es menor que 0, entonces hay dos raíces complejas de la forma:

$$x + yj, x - yj$$

Indique con literales apropiados los datos a introducir, así como los resultados obtenidos.

CAPÍTULO 8

© F.J.Ceballos/RA-MA

MATRICES

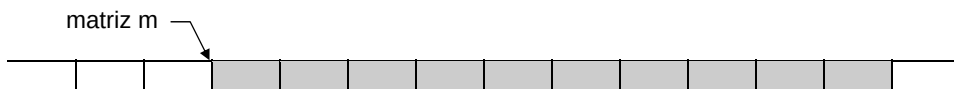
Hasta ahora sólo hemos tenido que trabajar con algunas variables en cada uno de los programas que hemos realizado. Sin embargo, en más de una ocasión tendremos que manipular conjuntos más grandes de valores. Por ejemplo, para calcular la temperatura media del mes de agosto necesitaremos conocer los 31 valores correspondientes a la temperatura media de cada día. En este caso, podríamos utilizar una variable para introducir los 31 valores, uno cada vez, y acumular la suma en otra variable. Pero, ¿qué ocurrirá con los valores que vayamos introduciendo? Que cuando tecleemos el segundo valor, el primero se perderá; cuando tecleemos el tercero, el segundo se perderá, y así sucesivamente. Cuando hayamos introducido todos los valores podremos calcular la media, pero las temperaturas correspondientes a cada día se habrán perdido. ¿Qué podríamos hacer para almacenar todos esos valores? Pues, podríamos utilizar 31 variables diferentes; pero, ¿qué pasaría si fueran 100 o más valores los que tuviéramos que registrar? Además de ser muy laborioso el definir cada una de las variables, el código se vería enormemente incrementado.

En este capítulo, aprenderá a registrar conjuntos de valores, todos del mismo tipo, en unas estructuras de datos llamadas *matrices*. Así mismo, aprenderá a registrar cadenas de caracteres, que no son más que conjuntos de caracteres, o bien, si lo prefiere, matrices de caracteres.

Si las matrices son la forma de registrar conjuntos de valores, todos del mismo tipo (**int**, **float**, **double**, **char**, **String**, etc.), ¿qué haremos para almacenar un conjunto de valores relacionados entre sí, pero de diferentes tipos? Por ejemplo, almacenar los datos relativos a una persona como su *nombre*, *dirección*, *teléfono*, etc. Ya hemos visto que esto se hace definiendo una clase; en este caso, podría ser la clase de objetos *persona*. Posteriormente podremos crear también matrices de objetos, cuestión que aprenderemos más adelante.

INTRODUCCIÓN A LAS MATRICES

Una matriz es una estructura homogénea, compuesta por varios elementos, todos del mismo tipo y almacenados consecutivamente en memoria. Cada elemento puede ser accedido directamente por el nombre de la variable matriz seguido de uno o más subíndices encerrados entre corchetes.



En general, la representación de las matrices se hace mediante variables suscritas o de subíndices y pueden tener una o varias dimensiones (subíndices). A las matrices de una dimensión se les llama también listas y a las de dos dimensiones, tablas.

Desde un punto de vista matemático, en más de una ocasión necesitaremos utilizar variables subindicadas tales como:

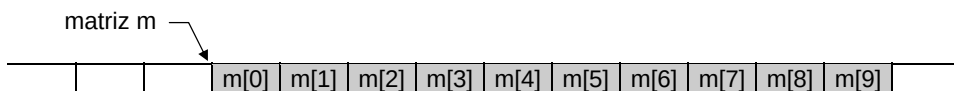
$$v = [a_0, a_1, a_2, \dots, a_i, \dots, a_n]$$

en el caso de un subíndice, o bien:

$$m = \begin{pmatrix} a_{00} & a_{01} & a_{02} & \dots & a_{0j} & \dots & a_{0n} \\ a_{10} & a_{11} & a_{12} & \dots & a_{1j} & \dots & a_{1n} \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ a_{i0} & a_{i1} & a_{i2} & \dots & a_{ij} & \dots & a_{in} \end{pmatrix}$$

si se utilizan dos subíndices. Esta misma representación se puede utilizar desde un lenguaje de programación recurriendo a las matrices que acabamos de definir y que a continuación se estudian.

Por ejemplo, supongamos que tenemos una matriz unidimensional de enteros llamada m , la cual contiene 10 elementos. Estos elementos se identificarán de la siguiente forma:



Observe que los subíndices son enteros consecutivos, y que el primer subíndice vale 0. Un subíndice puede ser cualquier expresión entera positiva.

Así mismo, una matriz de dos dimensiones se representa mediante una variable con dos subíndices (filas, columnas); una matriz de tres dimensiones se representa mediante una variable con tres subíndices, etc. El número máximo de dimensiones o el número máximo de elementos para una matriz, dentro de los límites establecidos por el compilador, depende de la memoria disponible.

Entonces, las matrices según su dimensión se clasifican en unidimensionales y multidimensionales; y según su contenido, en numéricas, de caracteres y de referencias a objetos.

En C#, el tipo de una matriz es un tipo referenciado derivado de la clase abstracta **System.Array**. Cada elemento de una matriz unidimensional puede ser de un tipo primitivo, o bien una referencia a un objeto; y cada elemento de una matriz multidimensional puede ser también de un tipo primitivo, o bien una referencia a otra matriz. A continuación se estudia todo esto detalladamente.

MATRICES NUMÉRICAS UNIDIMENSIONALES

Igual que sucede con otras variables, antes de utilizar una matriz hay que declararla. La declaración de una matriz especifica el nombre de la matriz y el tipo de elementos de la misma.

Para crear y utilizar una matriz hay que realizar tres operaciones: declararla, crearla e iniciarla.

Declarar una matriz

La declaración de una matriz de una dimensión se hace de la forma siguiente:

```
tipo[] nombre;
```

donde *tipo* indica el tipo de los elementos de la matriz, que pueden ser de cualquier tipo primitivo o referenciado; y *nombre* es un identificador que nombra a la matriz. Los corchetes modifican la definición normal del identificador para que sea interpretado por el compilador como una matriz.

Las siguientes líneas de código son ejemplos de declaraciones de matrices:

```
int[] m;  
float[] temperatura;  
COrdenador[] ordenador; // COrdenador es una clase de objetos
```

La primera línea declara una matriz de elementos de tipo **int**; la segunda, una matriz de elementos de tipo **float**; y la tercera, una matriz de objetos *COrdenador*.

Note que las declaraciones no especifican el tamaño de la matriz. El tamaño será especificado cuando se cree la matriz, operación que se hará durante la ejecución del programa.

Crear una matriz

Después de haber declarado una matriz, el siguiente paso es crearla o construirla. Crear una matriz significa reservar la cantidad de memoria necesaria para contener todos sus elementos y asignar al nombre de la matriz una referencia a ese bloque. Esto puede expresarse genéricamente así:

```
nombre = new tipo[tamaño];
```

donde *nombre* es el nombre de la matriz previamente declarada; *tipo* es el tipo de los elementos de la matriz; y *tamaño* es una expresión entera positiva menor o igual que la precisión de un **ulong**, que especifica el número de elementos.

El hecho de utilizar el operador **new** significa que *C# implementa las matrices como objetos*, por lo tanto serán tratadas como cualquier otro objeto.

Las siguientes líneas de código crean las matrices declaradas en el ejemplo anterior:

```
m = new int[10];  
temperatura = new float[31];  
ordenador = new COrdenador[25];
```

La primera línea crea una matriz identificada por *m* con 10 elementos de tipo **int**; es decir, puede almacenar 10 valores enteros; el primer elemento es *m[0]* (se lee: *m sub-cero*), el segundo *m[1]*,..., y el último *m[9]*. La segunda crea una matriz *temperatura* de 31 elementos de tipo **float**. Y la tercera crea una matriz *ordenador* de 25 elementos, cada uno de los cuales puede referenciar a un objeto *COrdenador*. Una matriz de objetos es una matriz de referencias a dichos objetos.

Es bastante común declarar y crear la matriz utilizando una sola sentencia. Esto puede hacerse así:

```
tipo[] nombre = new tipo[tamaño];
```

Las siguientes líneas de código declaran y crean las matrices expuestas en los ejemplos anteriores:

```
int[] m = new int[10];
float[] temperatura = new float[31];
COrdenador[] ordenador = new COrdenador[25];
```

Cuando se crea una matriz, el tamaño de la misma puede ser también especificado durante la ejecución a través de una variable a la que se asignará como valor el número de elementos requeridos. Por ejemplo, la última línea de código del ejemplo siguiente crea una matriz con el número de elementos especificados por la variable *nElementos*:

```
int nElementos;
System.Console.WriteLine("Número de elementos de la matriz: ");
nElementos = Leer.datoInt();
int[] m = new int[nElementos];
```

Iniciar una matriz

Una matriz es un objeto; por lo tanto, cuando se crea, sus elementos son automáticamente iniciados, igual que sucedía con las variables miembro de una clase. Si la matriz es numérica, sus elementos son iniciados a 0 y si no es numérica, a un valor análogo al 0; por ejemplo, los caracteres son iniciados al valor ‘\u0000’, un elemento booleano a **false** y las referencias a objetos, a **null**.

Si deseamos iniciar una matriz con otros valores diferentes a los predeterminados, podemos hacerlo de la siguiente forma:

```
float[] temperatura = {10.2F, 12.3F, 3.4F, 14.5F, 15.6F, 16.7F};
string[] días_semana = {"lunes", "martes", "miércoles",
                        "jueves", "viernes", "sábado", "domingo"};
```

El ejemplo anterior crea una matriz *temperatura* de tipo **float** con tantos elementos como valores se hayan especificado entre llaves y una matriz *días_semana* con siete elementos de tipo **string**.

Acceder a los elementos de una matriz

Para acceder al valor de un elemento de una matriz se utiliza el nombre de la matriz, seguido de un subíndice entre corchetes. Esto es, un elemento de una matriz no es más que una variable subindicada; por lo tanto, se puede utilizar exactamente igual que cualquier otra variable. Por ejemplo, en las operaciones que se muestran a continuación intervienen elementos de una matriz *m*:

```
int[] m = new int[100];
int k = 0, a = 0;
```

```
// ...  
a = m[1] + m[99];  
k = 50;  
m[k]++;  
m[k+1] = m[k];
```

Observe que para referenciar un elemento de una matriz se puede emplear como subíndice una constante, una variable o una expresión de tipo entero. El subíndice especifica la posición del elemento dentro de la matriz. La primera posición es la 0.

Si se intenta acceder a un elemento con un subíndice menor que cero o mayor que el número de elementos de la matriz menos uno, se lanzará una excepción de tipo **IndexOutOfRangeException**, indicando que el subíndice está fuera de los límites establecidos cuando se creó la matriz. Por ejemplo, cuando se ejecute la última línea de código del ejemplo siguiente se lanzará una excepción, puesto que se intenta asignar el valor del elemento de subíndice 99 al elemento de subíndice 100, que está fuera del rango 0 a 99 válido.

```
int[] m = new int[100];  
int k = 0, a = 0;  
// ...  
k = 99;  
m[k+1] = m[k];
```

Tamaño y dimensión de una matriz

¿Cómo podemos asegurarnos de no exceder accidentalmente el final de una matriz? Verificando la longitud de la matriz mediante su propiedad **Length**. La propiedad **Rank** indica la dimensión de la matriz (1, 2, 3, etc.). Por ejemplo:

```
int ne = m.Length; // número de elementos de la matriz m  
int dm = m.Rank;   // dimensión de la matriz m
```

Métodos de una matriz

La clase **Array** del espacio de nombres **System** proporciona un conjunto de métodos que pueden ser utilizados con las matrices (ver el capítulo 9). Entre ellos cabe ahora destacar **Equals** (*bool Equals(object obj)*) y **Clone** (*object Clone()*). El primero permite verificar si dos referencias se refieren a un mismo objeto, y el segundo permite duplicar un objeto, en nuestro caso una matriz (**object** es un alias de **System.Object**).

Por ejemplo, el código expuesto a continuación crea una matriz $m2$ que es una copia de otra matriz existente $m1$. Después pregunta si $m1$ es igual a $m2$; el resultado será **false** puesto que $m1$ y $m2$ se refieren a matrices diferentes.

```
int[] m1 = {10, 20, 30, 40, 50};
int[] m2 = (int[])m1.Clone(); // m2 es una copia de m1
if (m1.Equals(m2)) // equivale a: if (m1 == m2)
    System.Console.WriteLine("m1 y m2 se refieren a la misma matriz");
else
    System.Console.WriteLine("m1 y m2 se refieren a matrices diferentes");
```

Trabajar con matrices unidimensionales

Para practicar con la teoría expuesta hasta ahora, vamos a realizar un programa que asigne datos a una matriz unidimensional m de $nElementos$ elementos y, a continuación, como comprobación del trabajo realizado, escriba el contenido de dicha matriz. La solución será similar a la siguiente:

```
Número de elementos de la matriz: 10
Introducir los valores de la matriz.
m[0]= 1
m[1]= 2
m[2]= 3
...
1 2 3 ...

Fin del proceso.
```

Para ello, en primer lugar definimos la variable $nElementos$ para fijar el número de elementos de la matriz, creamos la matriz m con ese número de elementos y definimos el subíndice i para acceder a los elementos de dicha matriz.

```
int nElementos;
nElementos = Leer.datoInt();
int[] m = new int[nElementos]; // crear la matriz m
int i = 0; // subíndice
```

El paso siguiente es asignar un valor desde el teclado a cada elemento de la matriz.

```
for (i = 0; i < nElementos; i++)
{
    System.Console.Write("m[" + i + "] = ");
    m[i] = Leer.datoInt();
}
```

Una vez leída la matriz la visualizamos para comprobar el trabajo realizado.

```
for (i = 0; i < nElementos; i++)
    System.Console.Write(m[i] + " ");
```

El programa completo se muestra a continuación:

```
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES

using System;
using MisClases.ES;

public class CMatrizUnidimensional
{
    // Creación de una matriz unidimensional
    public static void Main(string[] args)
    {
        int nElementos;

        Console.Write("Número de elementos de la matriz: ");
        nElementos = Leer.datoInt();
        int[] m = new int[nElementos]; // crear la matriz m
        int i = 0; // subíndice

        Console.WriteLine("Introducir los valores de la matriz.");
        for (i = 0; i < nElementos; i++)
        {
            Console.Write("m[" + i + "] = ");
            m[i] = Leer.datoInt();
        }

        // Visualizar los elementos de la matriz
        Console.WriteLine();
        for (i = 0; i < nElementos; i++)
            Console.Write(m[i] + " ");
        Console.WriteLine("\n\nFin del proceso.");
    }
}
```

Matriz de tipo ArrayList

La biblioteca de .NET proporciona una clase **ArrayList** definida en el espacio de nombres **System.Collections** que facilita la creación y manipulación de matrices unidimensionales o listas. Un objeto **ArrayList** pertenece a esa clase de objetos que comúnmente denominamos colecciones de objetos (C#, igual que otros lenguajes, tiene varios tipos de colecciones, según veremos en el capítulo siguiente).

Por ejemplo, para definir una colección **ArrayList**, equivalente a una matriz de una dimensión, la sintaxis a utilizar es la siguiente:

```
ArrayList nombre = new ArrayList();
```

donde *nombre* es un identificador que nombra a la lista. Los elementos de la lista son de tipo **Object**; por lo tanto, la lista de elementos que se pueden agregar a la colección incluye tipos de datos estándar y matrices, así como estructuras y objetos de cualquier clase. El número de elementos de la lista será inicialmente cero.

Por ejemplo, la matriz *m* del ejemplo anterior podría declararse también así:

```
ArrayList m = new ArrayList();
```

En este caso, al tratarse de una colección de elementos de un tipo referenciado, ¿cómo almacenamos un elemento de un tipo primitivo? Recuerde que el espacio de nombres **System** proporciona las estructuras **Byte**, **SByte**, **Char**, **Int16**, **UInt16**, **Int32**, **UInt32**, **Int64**, **UInt64**, **Single**, **Double**, **Decimal** y **Boolean**, que encapsulan cada uno de los tipos primitivos estudiados. Pues bien, para facilitar el trabajo entre estos tipos y el tipo **Object**, C# implementa la característica de empaquetado/desempaquetado (*boxing/unboxing*; véase *Conversión entre tipos valor y tipos referencia* en el capítulo 6) que convierte automáticamente un valor de un tipo primitivo (como **int**) a un objeto de la clase **Object**, por ejemplo, cuando se inserta en una colección, y permite la conversión inversa cuando se obtiene de la colección, pero sólo con colecciones genéricas, y **ArrayList** no es genérica. Un ejemplo:

```
int x = 123, y = 0, i = 0;  
m.Add(x); // añadir el elemento x a la lista m  
y = (int)m[i]; // conversión de Object a int
```

Esta forma de definir una matriz no resulta más complicada y favorece, como veremos a continuación, la manipulación de listas. Esto es así porque la clase **ArrayList** proporciona métodos para acceder a los elementos de la lista, insertar nuevos elementos, eliminar elementos, obtener el número de elementos, buscar elementos, etc. A continuación veremos algunos de los métodos más comunes. Evidentemente esto marca una clara diferencia frente a las matrices primitivas de C#, pero hay que saber que las operaciones de empaquetado/desempaquetado son procesos que consumen muchos recursos, puesto que convertir un tipo de valor en un tipo **Object** implica crear un objeto completamente nuevo.

Añadir un elemento

Para añadir un elemento al final de la colección disponemos del método **Add(*elem*)**. Por ejemplo:

```
ArrayList m = new ArrayList();
int x = 0, y = 0, i = 0;
// ...
m.Add(x);      // añadir el elemento x a la colección
```

Insertar un elemento

Para insertar un elemento en una determinada posición de la colección disponemos del método **Insert(*pos*, *elem*)**. Por ejemplo:

```
m.Insert(i, x); // insertar un elemento x en la posición i
```

Modificar un elemento

Para modificar un elemento determinado de la colección reemplazándolo por otro disponemos del operador **[]**. Por ejemplo:

```
m[i] = y; // modificar el elemento de la posición i
```

Obtener el valor de un elemento

Para obtener el valor de un elemento determinado de la colección disponemos del operador **[]**. Por ejemplo:

```
int y = (int)m[i]; // obtener el valor del elemento de la posición i
```

Tamaño

Para conocer el tamaño de una colección hay que acceder a su propiedad **Count**.

```
int nElementos = m.Count; // tamaño
```

Eliminar elementos

El método **Clear** permite eliminar todos los elementos de la colección, **Remove** permite eliminar la primera aparición de un objeto concreto y **RemoveAt** el elemento que tiene por índice el especificado.

```
m.RemoveAt(i); // elimina el elemento de índice i
m.Remove(x);   // elimina el elemento de valor x
```



```
m.Clear();           // elimina todos los elementos
```

Buscar elementos

El método **Contains** devuelve **true** si la lista contiene el elemento especificado, y los métodos **IndexOf** y **LastIndexOf** devuelven, respectivamente, el índice en la lista del primero y del último elemento especificado, o **-1** si dicho elemento no se localiza. Por ejemplo:

```
if (m.Contains(x)) m.IndexOf(x) = y;
```

Copiar listas

Una lista puede ser copiada en otra invocando a su método **Clone**. Por ejemplo:

```
ArrayList m2 = new ArrayList();
m2 = (ArrayList)m.Clone();
```

Como el método **Clone** devuelve un objeto de la clase **Object**, es necesario realizar una conversión explícita al tipo **ArrayList** de la lista.

El método **Clone** hace una copia superficial de una colección; esto es, sólo copia los elementos de dicha colección, tanto si son tipos *referencia* como si son tipos *valor*, pero no copia los objetos a los que hacen referencia las referencias. En este último caso, las referencias de la nueva colección apuntarán a los mismos objetos que las referencias de la colección original. En contraposición, una copia profunda de una colección copia los elementos y todo aquello a lo que hacen referencia directa o indirectamente los elementos.

Iteradores

Iterar significa hacer la misma cosa repetidas veces. Pues bien, un iterador es una sección de código (un método) que devuelve una secuencia ordenada de valores del mismo tipo. Se puede invocar mediante una sentencia **foreach** de la manera siguiente:

```
foreach (int datoInt in m)
    Console.WriteLine(datoInt);
```

La instrucción **foreach** repite un grupo de instrucciones incluidas en el bucle para cada elemento de una matriz o de un objeto colección. Se utiliza para recorrer los elementos de una colección y obtener la información deseada, pero no se debe utilizar para cambiar el contenido de la colección, ya que se pueden producir efectos secundarios imprevisibles.

Ejemplo

El ejercicio anterior nos enseñó cómo leer una matriz y cómo escribirla. El paso siguiente es aprender a trabajar con los valores almacenados en la matriz. Por ejemplo, pensemos en un programa que lea la nota media obtenida por cada alumno de un determinado curso, las almacene en una matriz y dé como resultado la nota media del curso.

Igual que hicimos en el programa anterior, en primer lugar crearemos una matriz *notas* con cero elementos inicialmente. En este caso interesa que la matriz sea de tipo **float** para que sus elementos puedan almacenar un valor con decimales.

```
ArrayList notas = new ArrayList();
```

La entrada de datos la realizaremos con los métodos que proporciona la clase *Leer* implementada en el capítulo anterior. Los datos leídos serán almacenados en la lista *notas*, proceso que finalizará cuando se introduzca como nota media un valor -1:

```
int i = 0;
Console.WriteLine("Introducir notas medias. Finalizar con -1.");
Console.Write("Alumno número {0,3:D}, nota media: ", ++i);
float nota = Leer.datoFloat();
while (nota != -1)
{
    notas.Add(nota); // añadir un elemento
    Console.Write("Alumno número {0,3:D}, nota media: ", ++i);
    nota = Leer.datoFloat();
}
```

Finalmente se suman todas las notas y se visualiza la nota media. La suma se almacenará en la variable *suma*. Una variable utilizada de esta forma recibe el nombre de acumulador. Es importante que observe que inicialmente su valor es cero.

```
float suma = 0; // suma total de todas las notas medias
foreach (float notaMedia in notas)
    suma += notaMedia;
```

El programa completo se muestra a continuación.

```
using System;
using System.Collections;
using MisClases.ES;
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
```

```

public class CMatrizUnidimensional
{
    // Trabajar con una matriz unidimensional
    public static void Main(String[] args)
    {
        // Matriz notas inicialmente vacía
        ArrayList notas = new ArrayList();

        // Entrada de datos
        int i = 0;
        Console.WriteLine("Introducir notas medias. Finalizar con -1.");
        Console.Write("Alumno número {0,3:D}, nota media: ", ++i);
        float nota = Leer.datoFloat();
        while (nota != -1)
        {
            notas.Add(nota); // añadir un elemento
            Console.Write("Alumno número {0,3:D}, nota media: ", ++i);
            nota = Leer.datoFloat();
        }

        // Sumar las notas
        float suma = 0; // suma total de todas las notas medias
        foreach (float notaMedia in notas)
            suma += notaMedia;

        // Escribir resultados
        int nAlumnos = notas.Count; // número de alumnos
        Console.WriteLine("\n\nNota media del curso: {0,5:F2}\n", suma / nAlumnos);
    }
}

```

Los dos bucles de la aplicación anterior podrían reducirse a uno como se indica a continuación. No se ha hecho por motivos didácticos.

```

float suma = 0; // suma total de todas las notas medias
float nota = Leer.datoFloat();
while (nota != -1)
{
    notas.Add(nota); // añadir un elemento
    suma += nota;
    Console.Write("Alumno número {0,3:D}, nota media: ", ++i);
    nota = Leer.datoFloat();
}

```

Ejecución del programa:

```

Introducir notas medias. Finalizar con -1.
Alumno número   1, nota media: 5,5
Alumno número   2, nota media: 9
Alumno número   3, nota media: 7,5

```

```
Alumno número 4, nota media: 6
Alumno número 5, nota media: 8
Alumno número 6, nota media: -1
```

```
Nota media del curso: 7,20
```

Matrices asociativas

En una matriz asociativa el acceso a los elementos se hace por valor en lugar de por posición (por ejemplo, una matriz *díasMes[13]* que almacene en el elemento de índice 1 los días del mes 1, en el de índice 2 los días del mes 2 y así sucesivamente; ignoramos el elemento de índice 0). En estos casos, la solución del problema resultará más fácil si utilizamos esa coincidencia. Por ejemplo, vamos a realizar un programa que cuente el número de veces que aparece cada una de las letras de un texto introducido por el teclado y a continuación imprima el resultado. Para hacer el ejemplo sencillo, vamos a suponer que el texto sólo contiene letras minúsculas del alfabeto inglés (no hay ni letras acentuadas, ni la *ll*, ni la *ñ*). La solución podría ser de la forma siguiente:

```
Introducir un texto.
Para finalizar pulsar [Ctrl][z]
```

```
las matrices mas utilizadas son las unidimensionales
y las bidimensionales.
```

```
[Ctrl][z]
```

```
a b c d e f g h i j k l m n o p q r s t u v w x y z
-----
9 1 1 3 5 0 0 0 9 0 0 6 4 6 3 0 0 1 1 2 2 0 0 0 1 1
```

Antes de empezar el problema, vamos a analizar algunas de las operaciones que después utilizaremos en el programa. Por ejemplo, la expresión:

```
'z' - 'a' + 1
```

da como resultado 26. Recuerde que cada carácter tiene asociado un valor entero (código Unicode) que es el que utiliza la máquina internamente para manipularlo. Así por ejemplo la *'z'* tiene asociado el entero 122, la *'a'* el 97, etc. Según esto, la evaluación de la expresión anterior es: $122 - 97 + 1 = 26$.

Por la misma razón, si realizamos las declaraciones,

```
int[] c = new int[256]; // primeros 256 caracteres Unicode
int car = 'a';           // car tiene asignado el entero 97
```

la siguiente sentencia asigna a $c[97]$ el valor 10,

```
c['a'] = 10;
```

y esta otra sentencia que se muestra a continuación realiza la misma operación, lógicamente, suponiendo que *car* tiene asignado el carácter 'a'.

```
c[car] = 10;
```

Entonces, si leemos un carácter (de la 'a' a la 'z'),

```
car = System.Console.Read();
```

y a continuación realizamos la operación,

```
c[car]++;
```

¿qué elemento de la matriz *c* se ha incrementado? La respuesta es el de subíndice igual al código correspondiente al carácter leído. Hemos hecho coincidir el carácter leído con el subíndice de la matriz. Así cada vez que leamos una 'a' se incrementará el contador $c[97]$ o lo que es lo mismo $c['a']$; tenemos entonces un contador de 'a'. Análogamente diremos para el resto de los caracteres.

Pero, ¿qué pasa con los elementos $c[0]$ a $c[96]$? Según hemos planteado el problema inicialmente, quedarían sin utilizar (el enunciado decía: con qué frecuencia aparecen los caracteres de la 'a' a la 'z'). Esto, aunque no presenta ningún problema, se puede evitar así:

```
c[car - 'a']++;
```

Para *car* igual a 'a' se trataría del elemento $c[0]$ y para *car* igual a 'z' se trataría del elemento $c[25]$. De esta forma podemos definir una matriz de enteros justamente con un número de elementos igual al número de caracteres de la 'a' a la 'z' (26 caracteres según la tabla Unicode). El primer elemento será el contador de 'a', el segundo el de 'b', y así sucesivamente.

Un contador es una variable que inicialmente vale cero (suponiendo que la cuenta empieza desde uno) y que después se incrementa en una unidad cada vez que ocurre el suceso que se desea contar.

El programa completo se muestra a continuación.

```
using System;
```

```
public class CMatrizAsociativa
{
    // Frecuencia con la que aparecen las letras en un texto.
    public static void Main(string[] args)
    {
        // Crear la matriz c con 'z'-'a'+1 elementos.
        // C# inicia los elementos de la matriz a cero.
        int[] c = new int['z'-'a'+1];
        int car; // subíndice

        // Entrada de datos y cálculo de la tabla de frecuencias
        Console.WriteLine("Introducir un texto.");
        Console.WriteLine("Para finalizar pulsar [Ctrl][z]\n");

        // Leer el siguiente carácter del texto y contabilizarlo
        while ((car = Console.Read()) != -1)
        {
            // Si el carácter leído está entre la 'a' y la 'z'
            // incrementar el contador correspondiente
            if (car >= 'a' && car <= 'z')
                c[car - 'a']++;
        }

        // Mostrar la tabla de frecuencias
        Console.WriteLine();
        // Visualizar una cabecera "a b c ... "
        for (car = 'a'; car <= 'z'; car++)
            Console.Write(" " + (char)car);
        Console.WriteLine("\n -----" +
            "-----");
        // Visualizar la frecuencia con la que han aparecido los caracteres
        for (car = 'a'; car <= 'z'; car++)
            Console.Write("{0,3:D}", c[car - 'a']);
        Console.WriteLine();
    }
}
```

Diccionario

La biblioteca .NET proporciona una interfaz **IDictionary** definida en el espacio de nombres **System.Collections** en la que se basa la implementación de la clase **Hashtable** que facilita la creación y manipulación de matrices asociativas.

Cada elemento de un objeto **Hashtable** es un par *clave-valor* almacenado en una estructura **DictionaryEntry**. El primer elemento del par es la clave que se utiliza como índice para buscar el segundo elemento del par correspondiente al valor asociado. Cada uno de estos valores puede ser establecido o fijado a través de las propiedades **Key** y **Value** de esta estructura. Por ejemplo:

```

Hashtable map = new Hashtable();
map.Add(clave, valor);
// ...
foreach (DictionaryEntry e in map)
{
    clave = e.Key;
    valor = e.Value;
    // ...
}

```

La clave no puede ser **null**, pero el valor sí puede serlo. Este tipo de colecciones son conocidas como diccionarios o mapas y son idóneas para la manipulación de matrices asociativas.

Es frecuente emplear como clave un dato de tipo **String** y como valor asociado cualquier dato de un tipo primitivo o definido por el usuario.

Según lo expuesto, la matriz *c* del ejemplo anterior podría ser sustituida por:

```

Hashtable c = new Hashtable();

```

En este caso, la matriz asociativa *c* está inicialmente vacía. Para añadir un elemento (un par de valores) basta con hacer referencia al mismo y será añadido siempre y cuando no exista. Por ejemplo:

```

c['a'] = 10;           // crea nueva entrada para 'a' y le asigna 10
c['a'] = 7;           // cambia el valor de la entrada de 'a' a 7
int y = (int)c['a'];   // devuelve el valor de la clave 'a' (y=7)

```

Vemos que cuando se introduce un elemento cuya clave existe, el valor actual es reemplazado por el valor nuevo; esto es, las claves no se duplican.

Si lo que necesita es un mapa ordenado por la clave, utilice la clase **Sorted-List**. La funcionalidad que presenta es análoga a la de **Hashtable**.

De acuerdo con esto, podemos volver a escribir el ejemplo “con qué frecuencia aparecen los caracteres de la ‘a’ a la ‘z’” expuesto anteriormente como se indica a continuación:

```

using System;
using System.Collections;

public class CMapa
{
    // Frecuencia con la que aparecen las letras en un texto.
    public static void Main(System.String[] args)
    {

```

```
// Mapa c inicialmente vacío
Hashtable c = new Hashtable();

int car; // clave
int v;    // valor
int eof = -1;

// Iniciar el mapa
for (car = 'a'; car <= 'z'; car++)
    c[car] = 0;

// Entrada de datos y cálculo de la tabla de frecuencias
Console.WriteLine("Introducir un texto.");
Console.WriteLine("Para finalizar pulsar [Ctrl][z]\n");

// Leer el siguiente carácter del texto y contabilizarlo
while ((car = Console.Read()) != eof)
{
    // Si el carácter leído está entre la 'a' y la 'z'
    // incrementar el contador correspondiente
    if (car >= 'a' && car <= 'z')
    {
        v = (int)c[car];
        c[car] = ++v;
    }
}

// Mostrar la tabla de frecuencias
Console.WriteLine();
// Visualizar una cabecera "a b c ... "
for (car = 'a'; car <= 'z'; car++)
    Console.Write(" " + (char)car);
Console.WriteLine("\n -----" +
"-----");
// Visualizar la frecuencia con la que han aparecido los caracteres
for (car = 'a'; car <= 'z'; car++)
    Console.Write("{0,3:D}", c[car]);
Console.WriteLine();
}
```

La clase **Hashtable** proporciona también el método **ContainsKey** que devuelve **true** si hay un elemento con la clave especificada, el método **ContainsValue** que devuelve **true** si hay un elemento con el valor especificado, los métodos **Clear**, **Clone**, y la propiedad **Count** con el mismo significado que en la clase **ArrayList**, el método **Remove** para eliminar el elemento que tiene la clave especificada, etc. A continuación vemos un ejemplo que crea una agenda con pares de elementos *nombre-teléfono*:


```
// Agenda inicialmente vacía
Hashtable agenda = new Hashtable();
String nombre;           // clave
long teléfono = 0;       // valor

// Añadir elementos a la agenda
Console.WriteLine("Introducir pares de valores nombre teléfono.");
Console.WriteLine("Para finalizar, pulse <Entrar>.");

nombre = Console.ReadLine();
while (nombre.Length != 0)
{
    teléfono = Leer.datoLong();
    agenda[nombre] = teléfono;
    nombre = Console.ReadLine();
}

// Mostrar los elementos de la agenda
// Obtener los pares clave-valor de la agenda y mostrarlos
foreach (DictionaryEntry e in agenda)
    Console.WriteLine(e.Key + "    " + e.Value);

// Obtener el número de elementos
Console.WriteLine(agenda.Count + " elementos");
// Borrar el elemento identificado por "xxxx"
if (agenda.ContainsKey("xxxx")) agenda.Remove("xxxx");
// Borrar todos los elementos
if (agenda.Count != 0) agenda.Clear();
```

CADENAS DE CARACTERES

Una cadena de caracteres en C# es un objeto de la clase **String** del espacio de nombres **System** (**string** es un alias de **System.String**). Para entenderlo, piense en un objeto **String** como en una cadena de caracteres almacenada en una matriz unidimensional de elementos de tipo **char**. Por ejemplo:

```
char[] cadena = new char[40];
```

El ejemplo anterior define una matriz *cadena* de 40 caracteres, *cadena[0]* a *cadena[39]*, cada uno de ellos iniciado con el valor '\0'.

En realidad, una cadena de caracteres es una matriz de enteros, porque a cada carácter le corresponde un entero entre 0 y 65535 (código Unicode). Por ejemplo, a la 'a' le corresponde el valor 97, a la 'b' el valor 98, etc. (recuerde que los primeros 128 códigos Unicode coinciden con los primeros 128 códigos ASCII y ANSI; capítulo 2, tipo **char**). Desde este punto de vista, todo lo explicado para matrices de enteros es aplicable a cadenas de caracteres.

Por otra parte, en los capítulos anteriores ya vimos cómo leer cadenas de caracteres utilizando el método **Read**. Entonces, almacenar una cadena de caracteres en una matriz supondrá ejecutar repetidas veces el método **Read** y guardar cada carácter leído en el siguiente elemento libre de esa matriz de caracteres. Por ejemplo, el código siguiente define la variable *cadena* como una matriz de caracteres de longitud 40 y después establece un bucle para leer los caracteres que se tecleen hasta que se pulse la tecla *Entrar* o hasta que se complete la matriz. Cada carácter leído se almacena en el siguiente elemento libre de *cadena*. Finalmente se escribe el contenido de *cadena*, el número de caracteres almacenados y la dimensión de la matriz. Se puede observar que el valor dado por el atributo **Length** no es el número de caracteres almacenado en *cadena*, sino la dimensión de la matriz.

```
char[] cadena = new char[40]; // matriz de 40 caracteres
int i = 0, car;

System.Console.Write("Introducir un texto: ");
while ((car = System.Console.Read()) != '\r' && i < cadena.Length)
{
    cadena[i] = (char)car;
    i++;
}
System.Console.Write("Texto introducido:      ");
System.Console.WriteLine(cadena);
System.Console.WriteLine("Longitud del texto:      " + i);
System.Console.WriteLine("Dimensión de la matriz: " + cadena.Length);
System.Console.ReadLine(); // limpiar el buffer de entrada
```

Observe que el bucle utilizado para leer los caracteres tecleados, podría haberse escrito también así:

```
while ((car = System.Console.Read()) != '\r' && i < cadena.Length)
    cadena[i++] = (char)car;
```

Así mismo, también vimos cómo utilizar el método **ReadLine** de la clase **Console** para leer una cadena de caracteres. El método **ReadLine** lee hasta encontrar el carácter '\r', '\n' o los caracteres '\r\n' introducidos al pulsar la tecla *Entrar*; estos caracteres son leídos pero no almacenados, simplemente son interpretados como delimitadores. Por ejemplo, el código siguiente define una referencia, *cadena*, a un objeto **String** y utilizando el método **ReadLine**, lee una línea de texto desde el teclado. Con la información leída, el método **ReadLine** crea un objeto **String** y devuelve una referencia al mismo que es almacenada en *cadena*. Finalmente, la llamada a **WriteLine** permite visualizar el objeto **String**.

```
string cadena; // referencia a una línea de texto

System.Console.Write("Introduzca un texto: ");
```

```
cadena = System.Console.ReadLine(); // leer una línea de texto
System.Console.WriteLine(cadena); // escribir la línea leída
```

Comparando el método **Read** con el método **ReadLine**, se puede observar que éste último proporciona una forma más cómoda de leer cadenas de caracteres de un flujo y además, devuelve un objeto **String** cuyos métodos, como veremos a continuación, hacen muy fácil la manipulación de cadenas.

Clase String

La clase **String**, que pertenece al espacio de nombres **System**, proporciona métodos para examinar caracteres individuales de una cadena de caracteres, comparar cadenas, buscar y extraer subcadenas, copiar cadenas, convertir cadenas a mayúsculas o a minúsculas, etc. A continuación veremos algunos de los métodos más comunes, pero antes sepa que un objeto **String** representa una cadena de caracteres no modificable. Por lo tanto, una operación como convertir a mayúsculas no modificará el objeto original sino que devolverá un nuevo objeto con la cadena resultante de esa operación.

Así mismo, el lenguaje C# proporciona el operador + para concatenar objetos **String**, así como soporte para convertir otros objetos a objetos **String**. Por ejemplo, en la siguiente línea de código, C# debe convertir las expresiones que aparecen entre paréntesis en objetos **String**, antes de realizar la concatenación.

```
System.Console.WriteLine("Dimensión de la matriz: " + cadena.Length);
```

La línea de código anterior es equivalente a la siguiente:

```
System.Console.WriteLine("Dimensión de la matriz: " +
    (cadena.Length).ToString());
```

String(char[])

En el capítulo 3 dijimos que toda clase tiene al menos un método predeterminado especial denominado igual que ella, que es necesario invocar para crear un objeto; se trata del *constructor* de la clase. La clase **String** proporciona múltiples formas de su constructor **String**; la más útil quizás sea la aquí expuesta, que permite crear un objeto **String** a partir de una matriz de caracteres. Por ejemplo:

```
char[] cadena = new char[40]; // matriz de 40 caracteres
// ...
string str = new String(cadena);
System.Console.WriteLine("Texto introducido: " + str);
```

String ToString()

Este método devuelve el propio objeto **String** que recibe el mensaje **ToString**. Por ejemplo, el siguiente código copia la referencia *str1* en *str2* (no crea un objeto nuevo referenciado por *str2*, a partir de *str1*). El resultado es que las dos variables, *str1* y *str2*, permiten acceder al mismo objeto **String**.

```
string str1 = "abc", str2;  
str2 = str1.ToString(); // equivale a str2 = str1
```

La misma operación puede ser realizada utilizando la expresión *str2 = str1* (vea también el apartado *Referencias a objetos* en el capítulo 5).

Para verificar si dos referencias en general lo son al mismo objeto podemos utilizar el método **ReferenceEquals** de la clase **Object**. Por ejemplo, partiendo del código anterior, la siguiente sentencia mostrará “True”, indicando que *str1* y *str2* hacen referencia al mismo objeto **String**; en otro caso, mostraría “False”:

```
System.Console.WriteLine(System.Object.ReferenceEquals(str1, str2));
```

En general, el método **ToString** suele estar redefinido en cada clase de objetos con el fin de devolver un **String** correspondiente al valor que representan dichos objetos. Por ejemplo:

```
char[] cadena = new char[40]; // matriz de 40 caracteres  
// ...  
string str = (cadena.Length).ToString();
```

void CopyTo(int IndFuente, char[] destino, int IndDes, int nCars)

Este método permite copiar, total o parcialmente, el contenido de un objeto **String** en una matriz de caracteres. Esto es, copia *nCars* caracteres desde la posición *IndFuente* del objeto **String** que recibe el mensaje **CopyTo**, en la matriz *destino* a partir de su posición *IndDes*. Por ejemplo, las siguientes líneas de código copian el **string** *texto* en una matriz de caracteres *cadChar*:

```
string texto = "un texto";  
char[] cadChar = new char[texto.Length];  
texto.CopyTo(0, cadChar, 0, texto.Length);
```

static String Concat(String str1, String str2)

Este método devuelve como resultado un nuevo objeto **String** resultado de concatenar el **String** *str1* con el *str2*. Por ejemplo, el código que se muestra a continuación da como resultado la cadena “Ayer llovió”:

```
string str1 = "Ayer ", str2 = "llovió";
string str3 = System.String.Concat(str1, str2);
```

int CompareTo(String otroString)

Este método compara lexicográficamente el **String** especificado con el objeto **String** que recibe el mensaje **CompareTo** (el método **Equals** realiza la misma operación). El resultado devuelto es un entero:

< 0 si el **String** que recibe el mensaje es menor que el *otroString*,
 = 0 si el **String** que recibe el mensaje es igual que el *otroString* y
 > 0 si el **String** que recibe el mensaje es mayor que el *otroString*.

En otras palabras, el método **CompareTo** permite saber si una cadena está en orden alfabético antes (es menor) o después (es mayor) que otra y el proceso que sigue es el mismo que nosotros ejercitamos cuando lo hacemos mentalmente, comparar las cadenas carácter a carácter distinguiendo las mayúsculas de las minúsculas. La comparación se realiza sobre los valores UNICODE de cada carácter. El siguiente ejemplo compara dos cadenas y escribe “abcde” porque esta cadena está antes por orden alfabético.

```
string str1 = "abcde", str2 = "abcdefg";
if (str1.CompareTo(str2) < 0) // si str1 es menor que str2,
    System.Console.WriteLine(str1);
```

Hay dos métodos más: **Compare** y **CompareOrdinal**, ambos **static**. El método **Compare** permite especificar un tercer parámetro de tipo **bool**; cuando este parámetro toma el valor **false**, este método se comporta igual que **CompareTo** y cuando toma el valor **true** no distingue mayúsculas de minúsculas. Por ejemplo, el resultado de ejecutar el siguiente programa es que *str1* y *str2* son iguales.

```
public class Test
{
    public static void Main(string[] args)
    {
        string str1 = "La provincia de Santander es muy bonita";
        string str2 = "La provincia de SANTANDER es muy bonita";
        string strtemp;
```

```
int resultado;
resultado = System.String.Compare(str1, str2, true);

if( resultado > 0 )
    strtemp = "mayor que ";
else if( resultado < 0 )
    strtemp = "menor que ";
else
    strtemp = "igual a ";
System.Console.WriteLine( str1 + " es " + strtemp + str2 );
}
}
```

El método **CompareOrdinal**, a diferencia de los otros dos, no tiene en cuenta la cultura nacional.

El siguiente ejemplo no escribe nada porque “abc” no está antes por orden alfabético que “Abc”, ya que las mayúsculas están antes por orden alfabético. Esto es así porque en la tabla UNICODE las mayúsculas tienen asociado un valor entero menor que las minúsculas.

```
string str1 = "abc", str2 = "Abc";
if (System.String.CompareOrdinal(str1, str2) < 0)
    System.Console.WriteLine(str1);
```

int Length

Esta propiedad devuelve la longitud o número de caracteres UNICODE (tipo **char**) del objeto **String** que recibe el mensaje **Length**.

El siguiente ejemplo escribe como resultado: Longitud: 39

```
string str1 = "La provincia de Santander es muy bonita";
System.Console.WriteLine("Longitud: " + str1.Length);
```

String ToLower()

Este método convierte a minúsculas las letras mayúsculas del objeto **String** que recibe el mensaje **ToLower**. El resultado es un nuevo objeto **String** en minúsculas.

String ToUpper()

Este método convierte a mayúsculas las letras minúsculas del objeto **String** que recibe el mensaje **ToUpper**. El resultado es un nuevo objeto **String** en mayúsculas. El siguiente ejemplo almacena en *str1* la cadena *str2* en mayúsculas.

```
string str1, str2 = "Santander, tu eres novia del mar...";
str1 = str2.ToUpper();
```

String Trim()

Este método devuelve un objeto **String** resultado de eliminar los espacios en blanco que pueda haber al principio y al final del objeto **String** que recibe el mensaje **Trim**.

bool StartsWith(String prefijo)

Este método devuelve un valor **true** si el *prefijo* especificado coincide con el principio del objeto **String** que recibe el mensaje **StartsWith**.

bool EndsWith(String sufijo)

Este método devuelve un valor **true** si el *sufijo* especificado coincide con el final del objeto **String** que recibe el mensaje **EndsWith**. Un poco más adelante se muestra un ejemplo.

String Substring(int pos, int ncars)

Este método retorna un nuevo **String** que encapsula una subcadena de la cadena almacenada por el objeto **String** que recibe el mensaje **Substring**. La subcadena estará formada por *ncars* a partir de *pos* (el primer carácter está en la posición cero), o por los caracteres que haya hasta el final, si *ncars* no se especifica.

El siguiente ejemplo elimina los espacios en blanco que haya al principio y al final de *str1*, verifica si *str1* finaliza con “gh” y en caso afirmativo obtiene de *str1* una subcadena *str2* formada por los 3 caracteres de *str1* que hay a partir del que está en la posición 2 incluido éste.

```
string str1 = " abcdefgh ", str2 = "";
str1 = str1.Trim();
if (str1.EndsWith("gh"))
    str2 = str1.Substring(2, 3); // str2 = "cde"
```

Operador de indexación

Este operador devuelve el carácter del objeto **String**, que está en la posición especificada. Como el índice del primer carácter es el 0, la posición especificada tiene que estar entre los valores 0 y **Length** – 1, de lo contrario C# lanzará una excepción.

```
string str1 = "abcdefghg";  
char car = str1[2]; // car = 'c'
```

int IndexOf(String str)

Este método devuelve el índice de la primera ocurrencia de la subcadena especificada por *str* en el objeto **String** que recibe el mensaje **IndexOf**. Si *str* no existe **IndexOf** devuelve –1. Puede comenzar la búsqueda por el final en lugar de hacerlo por el principio utilizando el método **LastIndexOf**.

int IndexOfAny(char[] cad)

Este método devuelve el índice de la primera ocurrencia de cualquier carácter en *cad*, en el objeto **String** que recibe el mensaje **IndexOfAny**. Si no hay coincidencias, **IndexOfAny** devuelve –1. Puede comenzar la búsqueda por el final en lugar de hacerlo por el principio utilizando el método **LastIndexOfAny**.

String Replace(String str, String nuevaStr)

Este método devuelve un nuevo **String**, resultado de reemplazar todas las ocurrencias *str* por *nuevaStr* en el objeto **String** que recibe el mensaje **Replace**. Si la subcadena *str* no existiera, entonces se devuelve el objeto **String** original.

```
string str1 = "abcdefghghabcdefgh", str2;  
str2 = str1.Replace("de", "xxx"); // resultado: abcxixfghabcxixfgh
```

String Insert(int pos, String str)

Este método devuelve un nuevo **String**, resultado de insertar la subcadena *str* en el objeto **String** que recibe el mensaje, a partir de la posición *pos*.

String[] Split(char[] seps)

Este método devuelve una matriz de objetos **String** con las subcadenas que hay en el objeto **String** que recibe el mensaje, delimitadas por los separadores especifi-

cados en la matriz de caracteres *seps*. Si dos delimitadores son adyacentes o se encuentra uno al principio o al final, la subcadena correspondiente estará vacía.

```
string str1 = "hola, ¿qué tal?. Bien";
char[] seps = {' ', ',', '.'};
string[] cad = str1.Split(seps);
```

Clase **StringBuilder**

Del estudio de la clase **String** sabemos que un objeto de esta clase no es modificable. Se puede observar y comprobar que los métodos que actúan sobre un objeto **String** con la intención de modificarlo no lo hacen, sino que devuelven un objeto nuevo con las modificaciones solicitadas. En cambio, un objeto **StringBuilder** es un objeto modificable tanto en contenido como en tamaño.

Algunos de los métodos más interesantes que proporciona la clase **StringBuilder**, perteneciente al espacio de nombres **System.Text**, son los siguientes:

StringBuilder([arg])

Este método permite crear un objeto de la clase **StringBuilder**. Se trata del constructor de la clase. El siguiente ejemplo muestra algunas formas de invocar a este método:

```
using System.Text;
// ...
StringBuilder strb1 = new StringBuilder();
StringBuilder strb2 = new StringBuilder(80);
StringBuilder strb3 = new StringBuilder("abcde");

System.Console.WriteLine(strb1 + " " + strb1.Length + " " +
    strb1.Capacity);
System.Console.WriteLine(strb2 + " " + strb2.Length + " " +
    strb2.Capacity);
System.Console.WriteLine(strb3 + " " + strb3.Length + " " +
    strb3.Capacity);
```

La ejecución de las líneas de código del ejemplo anterior da lugar a los siguientes resultados:

```
0 16
0 80
abcde 5 16
```

A la vista de los resultados podemos deducir que cuando **StringBuilder** se invoca sin argumentos construye un objeto vacío con una capacidad inicial para 16 caracteres; cuando se invoca con un argumento entero, construye un objeto vacío con la capacidad especificada; y cuando se invoca con un **String** como argumento, construye un objeto con la secuencia de caracteres proporcionada por el argumento y una capacidad igual $2^n > strb.Length$ para $n \geq 4$.

int Length

Esta propiedad devuelve la longitud o número de caracteres Unicode (tipo **char**) del objeto **StringBuilder** que recibe el mensaje **Length**.

int Capacity

Esta propiedad devuelve la capacidad en caracteres Unicode (tipo **char**) del objeto **StringBuilder** que recibe el mensaje **Capacity**.

StringBuilder Append(tipo x)

Este método permite añadir la cadena de caracteres resultante de convertir el argumento x en un objeto **String**, al final del objeto **StringBuilder** que recibe el mensaje **Append**. El *tipo* del argumento x puede ser **bool**, **char**, **char[]**, **int**, **long**, **float**, **double**, **String**, **Object**, etc. La longitud del objeto **StringBuilder** se incrementa en la longitud correspondiente al **String** añadido.

StringBuilder Insert(int índice, tipo x)

Este método permite insertar la cadena de caracteres resultante de convertir el argumento x en un objeto **String**, en el objeto **StringBuilder** que recibe el mensaje **Insert**. Los caracteres serán añadidos a partir de la posición especificada por el argumento *índice*. El *tipo* del argumento x puede ser **bool**, **char**, **char[]**, **int**, **long**, **float**, **double**, **String**, **Object**, etc. La longitud del objeto **StringBuilder** se incrementa en la longitud correspondiente al **String** insertado.

El siguiente ejemplo crea un objeto **StringBuilder** con la cadena “Mes de del año”, a continuación inserta la cadena “abril” a partir de la posición 7, y por último añade la cadena representativa del entero 2010. El resultado será “Mes de abril del año 2010”.

```
StringBuilder strb = new StringBuilder("Mes de del año ");
strb.Insert("Mes de ".Length, "abril "); // "Mes de ".Length=7
strb.Append(2010);
```

StringBuilder Remove(int p, int n)

Este método elimina n caracteres a partir de la posición p del objeto **StringBuilder** que recibe el mensaje **Remove**. La longitud del objeto se reduce en n y la capacidad se mantiene.

Partiendo del ejemplo anterior, el siguiente ejemplo elimina la subcadena “abril ” del objeto *strb* y añade en su misma posición la cadena “mayo ”. El resultado será “Mes de mayo del año 2010”.

```
StringBuilder strb = new StringBuilder("Mes de del año ");
strb.Insert(7, "abril ");
strb.Append(2010);
strb.Remove(7, 6);
strb.Insert(7, "mayo ");
```

StringBuilder Replace(String str, String nuevaStr, int p, int n)

Este método reemplaza todas las ocurrencias de *str* que haya entre las posiciones p y $p+n-1$ del objeto **StringBuilder** que recibe el mensaje **Replace**, por *nuevaStr*. Si p y n no se especifican, se toman los valores 0 a **Length** – 1. Según lo expuesto, el ejemplo anterior, puede escribirse también así:

```
StringBuilder strb = new StringBuilder("Mes de del año ");
strb.Insert(7, "abril ");
strb.Append(2010);
strb.Replace("abril", "mayo");
```

Operador de indexación

Este operador devuelve el carácter del objeto **StringBuilder**, que está en la posición especificada. Como el índice del primer carácter es el 0, la posición especificada tiene que estar entre los valores 0 y **Length** – 1, de lo contrario C# lanzará una excepción.

```
StringBuilder str1 = new StringBuilder("abcdefgh");
char car = str1[2]; // car = 'c'
```

String ToString()

Este método devuelve como resultado un nuevo **String** copia del objeto **StringBuilder** que recibe el mensaje **ToString**.

El siguiente ejemplo copia en *str* la cadena almacenada en *strb*.

```
StringBuilder strb = new StringBuilder("abcde");  
String str = strb.ToString();
```

MATRICES MULTIDIMENSIONALES Y DE REFERENCIAS

Según lo estudiado a lo largo de este capítulo podemos decir que cada elemento de una matriz unidimensional es de un tipo primitivo, o bien una referencia a un objeto. Entonces, ¿cómo procederíamos si necesitáramos almacenar las temperaturas medias de cada día durante los 12 meses de un año?, o bien ¿cómo procederíamos si necesitáramos almacenar la lista de nombres de los alumnos de una determinada clase? Razonando un poco, llegaremos a la conclusión de que utilizar matrices unidimensionales para resolver los problemas planteados supondrá posteriormente un difícil acceso a los datos almacenados; esto es, responder a las preguntas: ¿cuál es la temperatura media del 10 de mayo?, o bien ¿cuál es el nombre del alumno número 25 de la lista? Será mucho más sencillo si los datos los almacenamos en forma de tabla; en el caso de las temperaturas, una tabla de 12 filas (tantas como meses) por 31 columnas (tantas como los días del mes más largo); y en el caso de los nombres, una tabla de tantas filas como alumnos, y tantas columnas como el número de caracteres del nombre más largo. Por lo tanto, una solución fácil para los problemas planteados exige el uso de matrices de dos dimensiones.

Una matriz multidimensional, como su nombre indica, es una matriz de dos o más dimensiones. C# soporta matrices multidimensionales, y también matrices de matrices con las que se pueden obtener los mismos resultados; esto es, matrices unidimensionales que a su vez referencian a otras matrices unidimensionales, y así sucesivamente, hasta llegar a obtener el número de dimensiones deseadas.

Matrices numéricas

La definición de una matriz numérica de varias dimensiones se hace de la forma siguiente:

```
tipo[...] nombre_matriz = new tipo[expr-1,expr-2...];
```

donde *tipo* es un tipo primitivo entero o real. El número de elementos de una matriz multidimensional es el producto de las dimensiones indicadas por *expr-1*, *expr-2*,... Por ejemplo, la línea de código siguiente crea una matriz de dos dimensiones con $2 \times 3 = 6$ elementos de tipo **int**:

```
int[, ] m = new int[2,3];
```

A partir de la línea de código anterior, C# crea una matriz bidimensional *m* con 2 filas y 3 columnas. Gráficamente podemos imaginarla así:

	col 0	col 1	col 2
fila 0	m_{00}	m_{01}	m_{02}
fila 1	m_{10}	m_{11}	m_{12}

Para acceder a los elementos de la matriz *m*, puesto que se trata de una matriz de dos dimensiones, utilizaremos dos subíndices, el primero indicará la fila y el segundo la columna donde se localiza el elemento, según se puede observar en la figura anterior. Por ejemplo, la primera sentencia del ejemplo siguiente asigna el valor *x* al elemento que está en la fila 1, columna 2; y la segunda, asigna el valor de este elemento al que está en la fila 0, columna 1.

```
m[1,2] = x;
m[0,1] = m[1,2];
```

Para crear una matriz, en general, de *n* dimensiones procederemos de forma análoga. Por ejemplo, el siguiente código crea una matriz de tres dimensiones con *d1*×*d2*×*d3* elementos:

```
int d1 = 2, d2 = 3, d3 = 4;
int[, ,] m = new int[d1,d2,d3];
```

Una matriz numérica de varias dimensiones puede también implementarse utilizando matrices de matrices. Desde esta perspectiva, la definición de una matriz numérica de varias dimensiones se hace de la forma siguiente:

```
tipo[][]... nombre_matriz = new tipo[d1][]...;
nombre_matriz[0] = new tipo[d2][]...;
...
nombre_matriz[d1-1] = new tipo[d2][]...;
...
```

donde *tipo* es un tipo primitivo entero o real. El número de elementos de una matriz multidimensional en el caso de que sea rectangular es el producto de las dimensiones indicadas por *d1*, *d2*... Cuando el número de elementos de las matrices referenciadas por otra matriz sea diferente (submatrices), estaremos en el caso de matrices dentadas.

Por ejemplo, la línea de código siguiente crea una matriz de dos dimensiones con $2 \times 3 = 6$ elementos de tipo **int**:

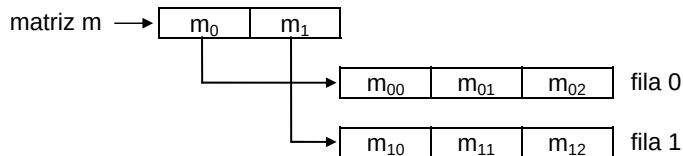
```
// Matriz de 2 filas por 3 columnas:
```

```
// declarar una matriz de matrices de 2 elementos
int[][] m = new int[2][];
// iniciar cada elemento con una matriz de 3 elementos
m[0] = new int[3];
m[1] = new int[3];
```

La definición anterior puede realizarse también así:

```
int[][] m = new int[2][] // 2 filas: 0 y 1
{
    new int[3], // fila 0: 3 columnas
    new int[3] // fila 1: 3 columnas
};
```

A partir del código anterior, C# crea una matriz unidimensional *m* con 2 elementos *m[0]* y *m[1]* que son referencias a otras dos matrices unidimensionales de 3 elementos. Gráficamente podemos imaginarlo así:



Evidentemente, el tipo de los elementos de la matriz referenciada por *m* es **int[]** y el tipo de los elementos de las matrices referenciadas por *m[0]* y *m[1]* es **int**. Además, puede comprobar la existencia y la longitud de las matrices unidimensionales referenciadas por *m*, *m[0]* y *m[1]* utilizando el código siguiente:

```
// ...
System.Console.WriteLine(m.Length); // resultado: 2
System.Console.WriteLine(m[0].Length); // resultado: 3
System.Console.WriteLine(m[1].Length); // resultado: 3
```

Para acceder a los elementos de la matriz *m*, puesto que se trata de una matriz de dos dimensiones, utilizaremos dos subíndices, el primero indicará la fila y el segundo la columna donde se localiza el elemento, según se puede observar en la figura anterior. Por ejemplo, la primera sentencia del ejemplo siguiente asigna el valor *x* al elemento que está en la fila 1, columna 2; y la segunda, asigna el valor de este elemento al elemento *m[0][1]*.

```
m[1][2] = x;
m[0][1] = m[1][2];
```

Si hubiéramos iniciado la fila 0 y la fila 1 con un número de elementos diferente, estaríamos en el caso de una matriz dentada. Por ejemplo:

```
int[][] m = new int[2][] // 2 filas: 0 y 1
{
    new int[7], // fila 0: 7 elementos
    new int[4] // fila 1: 4 elementos
};
```

Para crear una matriz, en general, de n dimensiones utilizando matrices de matrices procederemos de forma análoga. Por ejemplo, el siguiente código crea una matriz de tres dimensiones con $d1 \times d2 \times d3$ elementos:

```
// Matriz de d1*d2*d3 elementos:
int d1 = 2, d2 = 3, d3 = 4;
// declarar una matriz de matrices de d1 elementos
int[][][] m = new int[d1][][];
// iniciar cada elemento de m con una matriz de d2 elementos
for (int i = 0; i < d1; i++)
    m[i] = new int[d2][];
// iniciar cada elemento de m[i] con una matriz de d3 elementos
for (int i = 0; i < d1; i++)
    for (int j = 0; j < d2; j++)
        m[i][j] = new int[d3];
```

Cuando una matriz de matrices tiene forma rectangular, que es cuando las submatrices son todas de la misma longitud, es más eficiente utilizar una matriz multidimensional. En el ejemplo anterior, para la construcción de la matriz m ha sido necesario crear 9 objetos (una matriz externa de $d1$ elementos, dos submatrices de $d2$ elementos y seis submatrices más de $d3$ elementos). En contraposición,

```
int d1 = 2, d2 = 3, d3 = 4;
int[,,] m = new int[d1,d2,d3];
```

crea un solo objeto, una matriz de tres dimensiones, y lleva a cabo la creación en un única sentencia.

Como ejemplo de aplicación de matrices multidimensionales, vamos a realizar un programa que asigne datos a una matriz m de dos dimensiones y a continuación escriba las sumas correspondientes a las filas de la matriz. La ejecución del programa presentará el aspecto siguiente:

```
Número de filas de la matriz: 2
Número de columnas de la matriz: 2
Introducir los valores de la matriz.
m[0,0] = 2
m[0,1] = 5
m[1,0] = 3
m[1,1] = 6
Suma de la fila 0: 7
```

Suma de la fila 1: 9

Fin del proceso.

En primer lugar definimos las variables que almacenarán el número de filas y de columnas de la matriz, y a continuación leemos esos valores desde el teclado, desechando cualquier valor negativo.

```
int nfilas, ncols;    // filas y columnas de la matriz
do
{
    System.Console.Write("Número de filas de la matriz: ");
    nfilas = Leer.datoInt();
}
while (nfilas < 1);    // no permitir un valor negativo
do
{
    System.Console.Write("Número de columnas de la matriz: ");
    ncols = Leer.datoInt();
}
while (ncols < 1);    // no permitir un valor negativo
```

Después, creamos la matriz *m* con el número de filas y columnas especificado, definimos las variables *fila* y *col* para utilizarlas como subíndices fila y columna, respectivamente, y la variable *sumafila* para almacenar la suma de los elementos de una fila:

```
float[,] m = new float[nfilas,ncols]; // crear la matriz m
int fila = 0, col = 0; // subíndices
float sumafila = 0;    // suma de los elementos de una fila
```

El paso siguiente es asignar un valor desde el teclado a cada elemento de la matriz.

```
for (fila = 0; fila < nfilas; fila++)
    for (col = 0; col < ncols; col++)
    {
        System.Console.Write("m[" + fila + "," + col + "] = ");
        m[fila,col] = Leer.datoFloat();
    }
```

Una vez leída la matriz, calculamos la suma de cada fila y visualizamos los resultados para comprobar el trabajo realizado.

```
for (fila = 0; fila < nfilas; fila++)
{
    sumafila = 0;
```



```

    for (col = 0; col < ncols; col++)
        sumafila += m[fila,col];
    System.Console.WriteLine("Suma de la fila " + fila + ": " + sumafila);
}

```

El programa completo se muestra a continuación:

```

// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES
using System;
using MisClases.ES;

public class CMatrizMultidimensional
{
    // Creación de una matriz multidimensional.
    // Suma de las filas de una matriz de dos dimensiones.
    public static void Main(string[] args)
    {
        int nfilas, ncols;      // filas y columnas de la matriz

        do
        {
            Console.Write("Número de filas de la matriz:   ");
            nfilas = Leer.datoInt();
        }
        while (nfilas < 1);      // no permitir un valor negativo

        do
        {
            Console.Write("Número de columnas de la matriz: ");
            ncols = Leer.datoInt();
        }
        while (ncols < 1);      // no permitir un valor negativo

        float[,] m = new float[nfilas,ncols]; // crear la matriz m
        int fila = 0, col = 0; // subíndices
        float sumafila = 0;    // suma de los elementos de una fila

        Console.WriteLine("Introducir los valores de la matriz.");
        for (fila = 0; fila < nfilas; fila++)
        {
            for (col = 0; col < ncols; col++)
            {
                Console.Write("m[" + fila + "," + col + "] = ");
                m[fila,col] = Leer.datoFloat();
            }
        }

        // Visualizar la suma de cada fila de la matriz
        Console.WriteLine();
    }
}

```

```
for (fila = 0; fila < nfilas; fila++)
{
    sumafila = 0;
    for (col = 0; col < ncols; col++)
        sumafila += m[fila,col];
    Console.WriteLine("Suma de la fila " + fila + ": " + sumafila);
}
Console.WriteLine("\nFin del proceso.");
}
```

Seguramente habrá pensado que la suma de cada fila se podía haber hecho simultáneamente a la lectura tal como se indica a continuación.

```
for (fila = 0; fila < nfilas; fila++)
{
    sumafila = 0;
    for (col = 0; col < ncols; col++)
    {
        Console.Write("m[" + fila + "," + col + "] = ");
        m[fila,col] = Leer.datoFloat();
        sumafila += m[fila,col];
    }
    Console.WriteLine("Suma de la fila " + fila + ": " + sumafila);
}
```

No obstante, esta forma de proceder presenta una diferencia a la hora de visualizar los resultados, y es que la suma de cada fila se presenta a continuación de haber leído los datos de la misma.

```
Número de filas de la matriz: 2
Número de columnas de la matriz: 2

Introducir los valores de la matriz.
m[0,0] = 2
m[0,1] = 5
Suma de la fila 0: 7
m[1,0] = 3
m[1,1] = 6
Suma de la fila 1: 9

Fin del proceso.
```

Con este último planteamiento, una solución para escribir los resultados al final sería almacenarlos en una matriz unidimensional y mostrar posteriormente la matriz. Este trabajo se deja como ejercicio para el lector, al igual que la realización del mismo ejemplo pero utilizando matrices de matrices.

Matrices de cadenas de caracteres

Las matrices de cadenas de caracteres son matrices multidimensionales, generalmente de dos dimensiones, en las que cada fila se corresponde con una cadena de caracteres. Entonces, según lo estudiado, una fila puede serlo de una matriz bidimensional, o bien estar representada por un objeto matriz unidimensional, un objeto **String** o un objeto **StringBuilder**.

Haciendo un estudio análogo al realizado para las matrices numéricas multidimensionales, la definición de una matriz de cadenas de caracteres puede hacerse de alguna de las dos formas siguientes:

```
char[,] nombre_matriz = new char[expr_1,expr_2];

char[][] nombre_matriz = new char[expr_1][];
    nombre_matriz[0] = new tipo[expr_2];
    ...
    nombre_matriz[expr_1 - 1] = new tipo[expr_n];
```

Por ejemplo, a partir de la línea de código siguiente, C# crea una matriz bidimensional m con $F \times C$ elementos de tipo **char**:

```
char[,] m = new char[F,C];
```

Gráficamente podemos imaginarla así:

m	$m_{0,0}$	$m_{0,1}$	$m_{0,2}$	...	$m_{0,C-1}$	<i>fila 0</i>
	$m_{1,0}$	$m_{1,1}$	$m_{1,2}$	...	$m_{1,C-1}$	<i>fila 1</i>
	$m_{2,0}$	$m_{2,1}$	$m_{2,2}$	...	$m_{2,C-1}$	<i>fila 2</i>
	...	...	...	...	...	...
	$m_{F-1,0}$	$m_{F-1,1}$	$m_{F-1,2}$	...	$m_{F-1,C-1}$	<i>fila F-1</i>

Pero, manejar cadenas de caracteres almacenadas así resulta laborioso porque todo el trabajo hay que hacerlo accediendo individualmente a cada carácter. Por ejemplo, el siguiente código lee f cadenas de caracteres y las almacena en una matriz m de dos dimensiones:

```
int nFilas = 3, nCarsPorFila = 60, car = 0, f, c;

// Definir la matriz de caracteres
char[,] m = new char[nFilas,nCarsPorFila];

// Leer las cadenas de caracteres
for (f = 0; f < nFilas; f++)
{
    c = 0;
    // Leer una cadena
```

```
while ((car = System.Console.Read()) != '\r' && c < nCarsPorFila)
{
    m[f,c] = (char)car;
    c++; // posición del siguiente carácter
}
System.Console.ReadLine(); // Limpiar el buffer de entrada
}
```

Una vez vista la dificultad que supone trabajar con matrices de caracteres carácter a carácter, el siguiente paso es demostrar que para la mayoría de los problemas en los que intervienen cadenas de caracteres resulta mucho más sencillo utilizar objetos **String** o **StringBuilder**.

Matrices de objetos String o StringBuilder

C# proporciona las clases **String** y **StringBuilder** para hacer de las cadenas de caracteres objetos con sus atributos particulares, los cuales podrán ser accedidos por los métodos de sus clases. Desde este nivel de abstracción el trabajo con matrices de cadenas de caracteres resultará mucho más sencillo.

Para ilustrar la forma de trabajar con matrices de cadenas de caracteres, vamos a realizar un programa que lea una lista de nombres y los almacene en una matriz de objetos **String**. Una vez construida la matriz, visualizaremos su contenido. La solución tendrá el aspecto siguiente:

```
Número de filas de la matriz: 10
Escriba los nombres que desea introducir.
Puede finalizar pulsando las teclas [Ctrl][Z].
Nombre[0]: Mª del Carmen
Nombre[1]: Francisco
Nombre[2]: Javier
Nombre[3]: [Ctrl][Z]

¿Desea visualizar el contenido de la matriz? (s/n): S

Mª del Carmen
Francisco
Javier
```

La solución pasa por realizar los siguientes puntos:

- Definimos la matriz de objetos **String**:

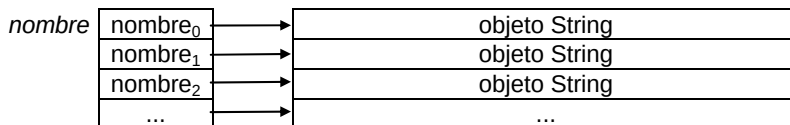
```
String[] nombre = new String[nFilas];
```

Cada elemento de esta matriz será iniciado por C# con el valor **null**, indicando así que la matriz inicialmente no referencia a ningún objeto **String**; esto es, la matriz está vacía.

- Leemos las cadenas de caracteres. Para poder leer una cadena, utilizaremos el método **ReadLine**. Recuerde que este método devuelve una referencia a un objeto **String** que almacena la información leída, referencia que asignaremos al siguiente elemento vacío de la matriz *nombre*. Este proceso lo repetiremos para cada uno de los nombres que leamos. Recuerde también que si el método **ReadLine** intenta leer del flujo y se encuentra con el final del mismo, retornará la constante **null**.

```
for (fila = 0; fila < nFilas; fila++)
{
    System.Console.WriteLine("Nombre[" + fila + "]: ");
    nombre[fila] = System.Console.ReadLine();
    // Si se pulsó [Ctrl][Z], salir del bucle
    if (nombre[fila] == null) break;
}
```

Gráficamente puede imaginarse el proceso descrito de acuerdo con la siguiente estructura de datos, aunque para trabajar resulte más fácil pensar en una matriz unidimensional cuyos elementos *nombre[0]*, *nombre[1]*, etc., son cadenas de caracteres.



- Una vez leídos todos los nombres deseados, los visualizamos si la respuesta a la petición de realizar este proceso es afirmativa.

El programa completo se muestra a continuación.

```
// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES

using System;
using MisClases.ES;

public class CMatrizCadenas
{
    public static void Main(string[] args)
    {
        int nFilas = 0, fila = 0;
```

```
do
{
    Console.Write("Número de filas de la matriz: ");
    nFilas = Leer.datoInt();
}
while (nFilas < 1);           // no permitir un valor negativo

// Matriz de cadenas de caracteres
string[] nombre = new String[nFilas];

Console.WriteLine("Escriba los nombres que desea introducir.");
Console.WriteLine("Puede finalizar pulsando las teclas [Ctrl][Z].");
for (fila = 0; fila < nFilas; fila++)
{
    Console.Write("Nombre[" + fila + "]: ");
    nombre[fila] = Console.ReadLine();
    // Si se pulsó [Ctrl][Z], salir del bucle
    if (nombre[fila] == null) break;
}
Console.WriteLine();
nFilas = fila; // número de filas leídas

char respuesta;
do
{
    Console.Write("¿Desea visualizar el contenido de la matriz? (s/n): ");
    respuesta = ((Console.ReadLine()).ToLower())[0];
}
while (respuesta != 's' && respuesta != 'n');
if ( respuesta == 's' )
{
    // Visualizar la lista de nombres
    Console.WriteLine();
    for (fila = 0; fila < nFilas; fila++)
        Console.WriteLine(nombre[fila]);
}
}
```

Si en lugar de utilizar objetos **String** utilizamos objetos **StringBuilder**, las modificaciones son mínimas. Puede verlas en el código mostrado a continuación:

```
StringBuilder[] nombre = new StringBuilder[nFilas];
String sNombre;

for (fila = 0; fila < nFilas; fila++)
{
    Console.Write("Nombre[" + fila + "]: ");
    // Si se pulsó [Ctrl][Z], salir del bucle
    if ((sNombre = Console.ReadLine()) == null) break;
```

```
nombre[fila] = new StringBuilder(sNombre);
}
```

Otra solución al problema que acabamos de plantear sería utilizar un objeto **ArrayList** como contenedor de los objetos **String** o **StringBuffer**. Esto nos daría libertad para trabajar con un número indeterminado de cadenas de caracteres; esto es, partiendo de cero elementos, iríamos añadiendo cadenas hasta que diéramos por finalizada la entrada.

SENTENCIA **foreach**

La sentencia **foreach** es similar a la sentencia **for**, con la diferencia de que ahora una sentencia simple o compuesta se ejecuta repetidamente por cada elemento de una colección de objetos o de una matriz. Esto es especialmente útil cuando no conocemos cuántos elementos hay en la colección o en la matriz. Su sintaxis es la siguiente:

```
foreach (elemento in colección/matriz)
    sentencia;
```

donde *elemento* se corresponde en cada ejecución de la *sentencia* con el siguiente elemento de la colección o de la matriz indicada. Sin embargo, hay que tener en cuenta que este *elemento* es de sólo lectura, por lo que no se puede utilizar para modificar los valores de la colección.

Por ejemplo, para mostrar el contenido de la matriz *nombre* del ejercicio anterior, podríamos escribir también el siguiente código:

```
foreach (StringBuilder siguienteNombre in nombre)
{
    if (siguienteNombre == null) break;
    Console.WriteLine(siguienteNombre);
}
```

EJERCICIOS RESUELTOS

1. Realice un programa que lea una lista de valores introducida por el teclado. A continuación, y sobre la lista, tiene que buscar los valores máximo y mínimo, y escribirlos.

La solución de este problema puede ser de la siguiente forma:

- Definimos la matriz que va a contener la lista de valores y el resto de las variables necesarias en el programa.

```
int nElementos;    // número de elementos (valor no negativo)

do
{
    System.Console.WriteLine("Número de valores que desea introducir: ");
    nElementos = Leer.datoInt();
}
while (nElementos < 1);

float[] dato = new float[nElementos]; // crear la matriz dato
int i = 0;           // subíndice
float max, min;      // valor máximo y valor mínimo
```

- A continuación leemos los valores que forman la lista. La entrada de datos finalizará cuando se tecleen todos los valores, o bien cuando se teclee un valor no numérico; por ejemplo, pulsar simplemente la tecla *Entrar*. Hagamos un breve repaso de la clase *Leer* que implementamos en el capítulo 6. Los métodos de esta clase devuelven el número entero o decimal introducido a través del teclado. Ahora bien, cuando el valor tecleado no se corresponde con un número, los métodos implementados para leer un entero devuelven el valor **MinValue** (valor mínimo) y los implementados para leer un real devuelven el valor **NaN** (no es un número).

```
for (i = 0; i < dato.Length; i++)
{
    System.Console.WriteLine("dato[" + i + "] = ");
    dato[i] = Leer.datoFloat();
    if (float.IsNaN(dato[i])) break; // salir del bucle
}
nElementos = i; // número de valores leídos
```

El código anterior establece un bucle para leer datos hasta completar la matriz. Si por cualquier circunstancia se decide terminar la entrada de datos antes que se complete la matriz, pulsando, por ejemplo, la tecla *Entrar*, el método *datoFloat* devolverá el valor **NaN**. Para detectar si esto ha ocurrido debemos utilizar el método **IsNaN** de la estructura **float** (**float** es un alias de **System.Single**). Se trata de un método **static** que devuelve **true** si su argumento se corresponde con un dato no numérico, y **false** en otro caso.

- Una vez leída la lista de valores, calculamos el máximo y el mínimo. Para ello suponemos inicialmente que el primer valor es el máximo y el mínimo (como si todos los valores fueran iguales). Después comparamos cada uno de estos dos valores con los restantes de la lista. El valor de la lista comparado pasará a ser el nuevo mayor si es más grande que el mayor actual y pasará a ser el nuevo menor si es más pequeño que el menor actual.


```

max = min = dato[0];
for (i = 0; i < nElementos; i++)
{
    if (dato[i] > max)
        max = dato[i];
    if (dato[i] < min)
        min = dato[i];
}

```

- Finalmente, escribimos el resultado.

```

System.Console.WriteLine("\nValor máximo: " + max);
System.Console.WriteLine("Valor mínimo: " + min);

```

El programa completo se muestra a continuación.

```

// Este programa utiliza la clase Leer del espacio de nombres
// MisClases.ES

using System;
using MisClases.ES;

public class CValoresMaxMin
{
    // Obtener el máximo y el mínimo de un conjunto de valores
    public static void Main(string[] args)
    {
        int nElementos;    // número de elementos (valor no negativo)
        do
        {
            Console.Write("Número de valores que desea introducir: ");
            nElementos = Leer.datoInt();
        }
        while (nElementos < 1);

        float[] dato = new float[nElementos]; // crear la matriz dato
        int i = 0;           // subíndice
        float max, min;      // valor máximo y valor mínimo

        Console.WriteLine("Introducir los valores.\n" +
                           "Para finalizar pulse [Entrar]");
        for (i = 0; i < dato.Length; i++)
        {
            Console.Write("dato[" + i + "]= ");
            dato[i] = Leer.datoFloat();
            if (float.IsNaN(dato[i])) break;
        }
        nElementos = i; // número de valores leídos

        // Obtener los valores máximo y mínimo
    }
}

```

```
if (nElementos > 0)
{
    max = min = dato[0];
    for (i = 0; i < nElementos; i++)
    {
        if (dato[i] > max)
            max = dato[i];
        if (dato[i] < min)
            min = dato[i];
    }
    // Escribir los resultados
    Console.WriteLine("\nValor máximo: " + max);
    Console.WriteLine("Valor mínimo: " + min);
}
else
    Console.WriteLine("\nNo hay datos.");
}
```

2. Escribir un programa que lea una cadena de caracteres y a continuación visualice el símbolo y el valor Unicode de cada uno de los caracteres de la cadena. La solución será de la forma:

```
Introduzca un texto: Hola ¡qué tal?
Carácter = 'H', código Unicode = 72
Carácter = 'o', código Unicode = 111
...
```

El problema consiste en declarar una referencia, *cadena*, a un objeto **String** y asignarle una cadena de caracteres leída desde el teclado utilizando el método **ReadLine**. Una vez creado el objeto **String**, se accede a cada uno de sus elementos para visualizar el carácter que contiene y su valor Unicode.

Observar que el método **WriteLine** visualiza un elemento de tipo **char** como un carácter; por lo tanto, para visualizar su valor Unicode es necesario convertirlo explícitamente a **int**. El programa completo se muestra a continuación.

```
using System;
public class CValorUnicode
{
    // Examinar una cadena de caracteres almacenada en un string
    public static void Main(string[] args)
    {
        string cadena = null; // referencia a una cadena de caracteres

        Console.Write("Introduzca un texto: ");
        cadena = Console.ReadLine(); // leer una línea de texto
```

```

// Examinar la cadena de caracteres
int i = 0;
do
{
    Console.WriteLine("Carácter = " + cadena[i] +
        ", código Unicode = " + (int)cadena[i]);
    i++;
}
while (i < cadena.Length);
}
}

```

3. Escribir un programa que lea una cadena de caracteres de la entrada estándar, la almacene en un objeto **StringBuilder** y utilizando un método *MinusculasMayusculas*, convierta todos los caracteres de la cadena a mayúsculas (la función realizada por este método será la misma que la realizada por **ToUpper**).

El método *MinusculasMayusculas* recibirá como parámetro la cadena de caracteres que contiene el texto a convertir. Dicha cadena estará encapsulada en un objeto **StringBuilder** para poder modificarla. Observe que cuando se llame al método *MinusculasMayusculas*, lo que en realidad se pasa es una referencia al objeto **StringBuilder**, con lo que los cambios realizados por uno u otro son visibles para ambos.

```

static void MinusculasMayusculas(StringBuilder str)
{
    // cuerpo del método
}

```

Para escribir el cuerpo del método compruebe en la tabla Unicode que los caracteres 'A',..., 'Z', 'a',..., 'z' están consecutivos y en orden ascendente de su código (valores 65 a 122). Entonces, pasar un carácter de minúsculas a mayúsculas es tan sencillo como restar al valor entero (código Unicode) asociado con el carácter la diferencia entre los códigos de ese carácter en minúscula y el mismo en mayúscula. Por ejemplo, la diferencia 'a' - 'A' es $97 - 65 = 32$, y es la misma que 'b' - 'B', que 'c' - 'C', etc. Por lo tanto, el método *MinusculasMayusculas* lo que tiene que hacer es acceder a cada carácter de la cadena y comprobar si se trata de una minúscula, en cuyo caso modificará la misma cambiando el valor Unicode correspondiente a ese carácter por el valor Unicode correspondiente al mismo carácter en mayúscula. Esto es:

```

static void MinusculasMayusculas(StringBuilder str)
{
    int i = 0, desp = 'a' - 'A';
    char car = '\0';
}

```

```
for (i = 0; i < str.Length; i++)
{
    car = str[i];
    if (car >= 'a' && car <= 'z')
        str[i] = (char)(car - desp);
}
```

El programa completo se muestra a continuación:

```
using System;
using System.Text;

public class CCadenas
{
    // Convertir una cadena a mayúsculas
    static void MinusculasMayusculas(StringBuilder str)
    {
        int i = 0, desp = 'a' - 'A';
        char car = '\0';

        for (i = 0; i < str.Length; i++)
        {
            car = str[i];
            if (car >= 'a' && car <= 'z')
                str[i] = (char)(car - desp);
        }
    }

    public static void Main(string[] args)
    {
        string scadena = null;
        StringBuilder bcadena = null;

        Console.Write("Introduzca un texto: ");
        scadena = Console.ReadLine(); // leer una línea de texto
        // Construir un objeto StringBuilder a partir de la cadena
        bcadena = new StringBuilder(scadena);

        // Convertir minúsculas a mayúsculas
        MinusculasMayusculas(bcadena); // llamar al método
        System.Console.WriteLine(bcadena); // escribir el resultado
    }
}
```

La solución que se ha dado al problema planteado no contempla los caracteres típicos de nuestra lengua como la ñ o las vocales acentuadas. Este trabajo queda como ejercicio para el lector.

EJERCICIOS PROPUESTOS

1. Se desea realizar un histograma con los pesos de los alumnos de un determinado curso.

Peso	Número de alumnos
21	**
22	****
23	*****
24	*****
..	...

El número de asteriscos se corresponde con el número de alumnos del peso especificado.

Realice un programa que lea los pesos e imprima el histograma correspondiente. Suponer que los pesos están comprendidos entre los valores 10 y 100 kg. En el histograma sólo aparecerán los pesos que se corresponden con uno o más alumnos.

2. Realice un programa que lea una cadena de n caracteres e imprima el resultado que se obtiene cada vez que se realice una rotación de un carácter a la derecha sobre dicha cadena. El proceso finalizará cuando se haya obtenido nuevamente la cadena de caracteres original. Por ejemplo:

HOLA AHOL LAHO OLAH HOLA

3. La mediana de una lista de n números se define como el valor que es menor o igual que los valores correspondientes a la mitad de los números, y mayor o igual que los valores correspondientes a la otra mitad. Por ejemplo, la mediana de:

16 12 99 95 18 87 10

es 18, porque este valor es menor que 99, 95 y 87 (mitad de los números) y mayor que 16, 12 y 10 (otra mitad).

Realice un programa que lea un número impar de valores y dé como resultado la mediana. La entrada de valores finalizará cuando se detecte la marca de fin de fichero.

4. Analice el programa que se muestra a continuación e indique el significado que tiene el resultado que se obtiene.

```
public class Test
{
    public static void Visualizar(byte car)
    {
        int i = 0, bit;
        for (i = 7; i >= 0; i--)
        {
            bit = ((car & (1 << i))!= 0) ? 1 : 0;
            System.Console.Write(bit);
        }
        System.Console.WriteLine();
    }

    public static byte HacerAlgo(byte car)
    {
        return (byte)((((car & 0x01) << 7) | ((car & 0x02) << 5) |
            ((car & 0x04) << 3) | ((car & 0x08) << 1) |
            ((car & 0x10) >> 1) | ((car & 0x20) >> 3) |
            ((car & 0x40) >> 5) | ((car & 0x80) >> 7)));
    }

    public static void Main(string[] args)
    {
        byte car;
        System.Console.Write("Introduce un carácter Unicode: ");
        car = (byte)System.Console.Read();
        Visualizar(car);
        System.Console.WriteLine("\nCarácter resultante:");
        car = HacerAlgo(car);
        Visualizar(car);
    }
}
```

MÁS SOBRE MÉTODOS Y COLECCIONES

En los capítulos anteriores aprendimos lo que es un programa, cómo escribirlo y qué hacer para que el ordenador lo ejecute y muestre los resultados conseguidos; adquirimos conocimientos generales acerca de la programación orientada a objetos; aprendimos acerca de los elementos que aporta C#; analizamos cómo era la estructura de un programa C#; aprendimos a leer datos desde el teclado y a visualizar resultados sobre el monitor; estudiamos las estructuras de control; y aprendimos a trabajar con matrices.

En este capítulo, utilizando los conocimientos adquiridos hasta ahora, vamos a centrarnos en cuestiones más específicas como pasar argumentos a métodos, escribir métodos que devuelvan matrices, pasar argumentos en la línea de órdenes, métodos recursivos, métodos sobrecargados, número de parámetros variable, clasificar los elementos de una matriz, buscar un elemento en una matriz, y colecciones, entre otras cosas.

PASAR UNA MATRIZ COMO ARGUMENTO A UN MÉTODO

En el capítulo 5 se expuso cómo definir un método en una clase y se explicó cómo pasar argumentos a un método. Recuerde que los objetos pasados como argumentos cuando se invoca a un método son siempre referencias a dichos objetos, lo cual significa que cualquier modificación que se haga sobre esos objetos en el cuerpo del método afecta a los objetos originales, y las matrices son objetos. En cambio, las variables de un tipo primitivo son pasadas por valor, lo cual significa que se pasa una copia, por lo que cualquier modificación que se haga sobre esas variables en el cuerpo del método no afecta a la variable original.

Para aclarar lo expuesto, el siguiente ejemplo implementa un método con un parámetro de tipo **double[,]**, que permite multiplicar por 2 los elementos de la matriz numérica de dos dimensiones pasada como argumento.

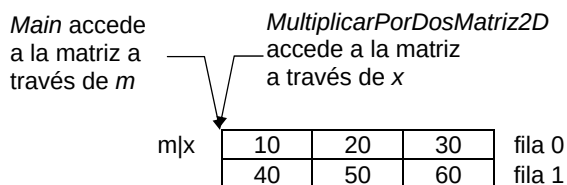
```
public class Test
{
    public static void MultiplicarPorDosMatriz2D(double[,] x)
    {
        for (int f = 0; f < x.GetLength(0); f++)
        {
            for (int c = 0; c < x.GetLength(1); c++)
                x[f,c] *= 2;
        }
    }

    public static void Main(string[] args)
    {
        double[,] m = {{10, 20, 30}, {40, 50, 60}};

        MultiplicarPorDosMatriz2D(m);

        // Visualizar la matriz por filas
        for (int f = 0; f < m.GetLength(0); f++)
        {
            for (int c = 0; c < m.GetLength(1); c++)
                System.Console.Write(m[f,c] + " ");
            System.Console.WriteLine();
        }
    }
}
```

El método **GetLength** de la clase **Array** permite obtener el valor de la dimensión pasada como argumento, del objeto matriz que recibe este mensaje. La primera dimensión es la 0.



La aplicación anterior se ejecuta de la forma siguiente: el método **Main** crea e inicia una matriz *m* de dos dimensiones de tipo **double**. Después invoca al método *MultiplicarPorDosMatriz2D* pasando como argumento la matriz *m*; esto implica que el método tenga un parámetro declarado así: *double[,] x*. Por ser *m* un objeto, el parámetro *x* recibe una referencia a la matriz *m*; esto es, *x* almacenará la posición de memoria donde se encuentra la matriz, no una copia de su contenido. Por

lo tanto, ahora el método *MultiplicarPorDosMatriz2D* tiene acceso a la misma matriz que el método **Main**, como muestra la figura anterior.

¿Cuál es el resultado? Que cuando el método **Main** visualice los elementos de la matriz *m*, éstos aparecerán con los cambios introducidos por el método *MultiplcarPorDosMatriz2D*. Esto es, ambos métodos trabajan sobre la misma matriz.

MÉTODO QUE RETORNA UNA MATRIZ

Según vimos en el capítulo 5, un método puede retornar un valor de cualquier tipo primitivo, o bien una referencia a cualquier clase de objetos. Por lo tanto, en el caso de que un método devuelva una matriz, lo que realmente devuelve es una referencia a la matriz. Aclaremos esto con un ejemplo.

La aplicación siguiente implementa un método que tiene un parámetro de tipo **double[,]** y permite copiar una matriz numérica bidimensional pasada como argumento en otra matriz. El método devuelve como resultado la copia realizada.

```
public class Test
{
    public static double[,] CopiarMatriz2D(double[,] x)
    {
        int filas = x.GetLength(0); // dimensión 0
        int cols = x.GetLength(1); // dimensión 1
        double[,] z = new double[filas,cols];

        for (int f = 0; f < filas; f++)
            for (int c = 0; c < cols; c++)
                z[f,c] = x[f,c];
        return z;
    }

    public static void Main(string[] args)
    {
        double[,] m1 = {{10, 20, 30}, {40, 50, 60}};

        // Copiar una matriz utilizando un método
        double[,] m2 = CopiarMatriz2D(m1);
        m1[0,0] = 77; // modificar un elemento de la matriz original

        // Visualizar la matriz m2
        for (int f = 0; f < m2.GetLength(0); f++)
            for (int c = 0; c < m2.GetLength(1); c++)
                System.Console.Write(m2[f,c] + " ");
        System.Console.WriteLine();
    }
}
```

La aplicación anterior se ejecuta de la forma siguiente: el método **Main** crea e inicia una matriz de dos dimensiones de tipo **double** referenciada por *m1*, y declara una referencia *m2* a una matriz de dos dimensiones del mismo tipo. Después invoca al método *CopiarMatriz2D* pasando como argumento la matriz *m1*. Esto implica que ese método tenga un parámetro declarado así: *double[,] x*, para que pueda recibir una referencia a la matriz *m1*. A continuación, *CopiarMatriz2D* crea una matriz *z* de las mismas características que *x*, copia los elementos de *x* en *z* y devuelve como resultado *z*. Finalmente, la referencia devuelta por *CopiarMatriz2D* es almacenada por el método **Main** en *m2*, que como comprobación visualiza esa matriz.

Otra forma de realizar una copia de una matriz es utilizando el método **Clone** expuesto anteriormente. Por ejemplo, el código mostrado a continuación copia una matriz de dos dimensiones referenciada por *m1* en otra matriz referenciada por *m2*:

```
double[,] m2 = (double[,])m1.Clone();
```

Cuando se trate de una matriz de matrices hay que tener presente que el método **Clone** sólo copia las referencias a las submatrices. Por ejemplo, la matriz *m1* de dimensiones $f \times c$ del ejemplo anterior puede ser reemplazada por una matriz de una dimensión de *f* elementos que son referencias a otras tantas matrices de una dimensión de *c* elementos. Según esto, le será fácil entender el código mostrado a continuación, el cual copia una matriz de matrices referenciada por *m1* en otra matriz referenciada por *m2*:

```
// Matriz de matrices m1
double[][] m1 = new double[2][];
m1[0] = new double[3] {10, 20, 30};
m1[1] = new double[3] {40, 50, 60};

// Copiar una matriz de matrices utilizando el método clone
double[][] m2 = (double[][])m1.Clone();
for (int f = 0; f < m1.Length; f++)
    m2[f] = (double[])m1[f].Clone();
```

Otra forma más de realizar una copia de una matriz es utilizando el método **Copy** de la clase **Array**. Se trata de un método público y estático para copiar matrices multidimensionales, cuya sintaxis es la siguiente:

```
void Copy(Array origen, int posición_origen,
          Array destino, int posición_destino,
          int longitud)
```

donde *origen* es la matriz origen de los datos, *posición_origen* el índice de inicio en la matriz origen, *destino* es la matriz destino de los datos, *posición_destino* el índice de inicio en la matriz destino y *longitud* es el número de elementos que se desean copiar.

Por ejemplo, el código mostrado a continuación, copia una matriz de dos dimensiones referenciada por *m1* en otra matriz referenciada por *m2*:

```
double[,] m1 = {{10, 20, 30}, {40, 50, 60}};

// Copiar una matriz utilizando el método Copy
double[,] m2 = new double[m1.GetLength(0),m1.GetLength(1)];
System.Array.Copy(m1, 0, m2, 0, m1.Length);
```

Si en lugar de tratarse de una matriz de dos dimensiones se trata de una matriz de matrices, habrá que utilizar el método **Copy** para copiar cada submatriz de datos. Por ejemplo, el código mostrado a continuación, copia una matriz de matrices referenciada por *m1* en otra matriz referenciada por *m2*:

```
// Matriz de matrices m1
double[][] m1 = new double[2][];
m1[0] = new double[3] {10, 20, 30};
m1[1] = new double[3] {40, 50, 60};

// Copiar una matriz de matrices utilizando el método Copy
double[][] m2 = new double[2][];
m2[0] = new double[3];
m2[1] = new double[3];
for (int f = 0; f < 2; f++)
    System.Array.Copy(m1[f], 0, m2[f], 0, m1[f].Length);
```

PASAR UN ARGUMENTO DE UN TIPO PRIMITIVO

Cuando un método C# invoca a otro método y le pasa un argumento de un tipo primitivo, pasa una copia de ese argumento. Por ejemplo:

```
public static void Incrementar10(int param)
{
    param += 10;
}

public static void Main(string[] args)
{
    int arg = 1234;
    Incrementar10(arg);
    System.Console.WriteLine(arg);
}
```

La línea sombreada del ejemplo anterior invoca al método *Incrementar10* y copia el valor del argumento *arg* en el parámetro *param* del método. Esto significa que el argumento ha sido pasado por valor. Por lo tanto, cualquier modificación que haga el método sobre *param* no afectará a la variable original. Según lo expuesto el método **Main** mostrará el resultado 1234, valor original de *arg*.

¿Qué hay que hacer para que un método pueda modificar el valor original del argumento que se le pasa? Pasar dicho argumento por referencia.

Según lo estudiado hasta ahora, cuando se pasa un argumento que es un objeto (tipo referenciado), C# no hace una copia del objeto sobre el parámetro correspondiente del método, sino que informa al método acerca del lugar de la memoria donde está ese objeto para que pueda acceder al mismo. Esto es así porque una variable de un tipo referenciado es una referencia al objeto y no el objeto en sí. Desde este punto de vista podemos decir que el objeto pasa por referencia (se pasa la dirección en memoria del objeto) y la variable que referencia al objeto se pasa por valor, puesto que se copia en el parámetro de destino. Por eso decimos que todos los objetos se pasan por referencia; esto es, lo que se copia en el parámetro del método es una referencia al objeto. Para proceder en el mismo sentido con una variable de un tipo primitivo hay que utilizar la palabra reservada **ref** tanto en la definición del método como en la llamada al mismo, detalles que se muestran en el ejemplo siguiente:

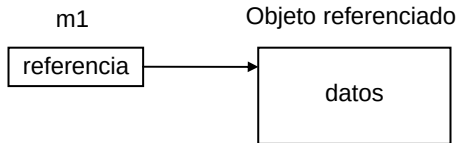
```
public static void Incrementar10(ref int param)
{
    param += 10;
}

public static void Main(string[] args)
{
    int arg = 1234;
    Incrementar10(ref arg);
    System.Console.WriteLine(arg);
}
```

En el ejemplo anterior, el método **Main** define una variable *arg* de tipo **int** iniciada con el valor 1234. Después invoca al método *Incrementar10* pasándole como argumento una referencia (**ref**) a esa variable, lo que supone declarar su parámetro *param* como una referencia (**ref**) a un **int**. Ahora *param* hace referencia al mismo entero que *arg*. Por lo tanto, todos los cambios realizados por el método afectarán a la variable original. Como consecuencia, el resultado mostrado por **Main** será ahora 1244.

PASAR TIPOS REFERENCIA POR REFERENCIA

Según lo estudiado hasta ahora, una variable de un tipo referencia no contiene directamente sus datos, sino una referencia a ellos:



y una referencia (en la figura *m1*), igual que una variable de un tipo primitivo, puede pasarse como argumento a un método, por valor o por referencia. Hasta ahora siempre hemos pasado los tipos referencia por valor, lo que significa pasar los objetos por referencia (se le pasa al método la referencia al objeto).

Veamos un ejemplo. El método *CopiarMatriz2D* expuesto anteriormente podría haberse diseñado según el siguiente prototipo:

```
public static void CopiarMatriz2D(double[,] destino, double[,] origen)
```

El resultado sería el siguiente:

```
public static void CopiarMatriz2D(double[,] destino, double[,] origen)
{
    int filas = origen.GetLength(0); // dimensión 0
    int cols = origen.GetLength(1); // dimensión 1

    for (int f = 0; f < filas; f++)
        for (int c = 0; c < cols; c++)
            destino[f, c] = origen[f, c];
}

public static void Main(string[] args)
{
    double[,] m1 = { { 10, 20, 30 }, { 40, 50, 60 } };

    // Copiar
    double[,] m2 = new double[2, 3]; // sus elementos valen 0
    CopiarMatriz2D(m2, m1);

    // ...
}
```

En este ejemplo, el método **Main** crea e inicia una matriz de dos dimensiones de tipo **double** referenciada por *m1*, y crea otra matriz de dos dimensiones del mismo tipo referenciada por *m2* e iniciada, por omisión, a cero. Después invoca al

método *CopiarMatriz2D* pasando como argumento la matriz *m2* y *m1*; *m2* se copia en *destino* y *m1* en *origen*. Copiar un argumento en su parámetro correspondiente significa pasar ese argumento por valor, pero como lo copiado es una referencia a un objeto decimos que hemos pasado el objeto por referencia; esto es, en este instante hay dos referencias, *m2* y *destino*, apuntando al mismo objeto (matriz de 2×3) y otras dos referencias, *m1* y *origen*, apuntando a otra matriz de 2×3 . Por eso, *CopiarMatriz2D* puede copiar los elementos de una matriz en otra.

Vemos entonces que cuando se pasa un argumento de tipo referencia por valor, es posible cambiar los datos referenciados (en el ejemplo, el objeto matriz *m2* de 2×3). Sin embargo, no se puede cambiar el valor de la propia referencia; es decir, no se puede utilizar la misma referencia para apuntar a un nuevo objeto y hacer que persista fuera del bloque como se intenta en el ejemplo siguiente:

```
public static void CopiarMatriz2D(double[,] destino, double[,] origen)
{
    int filas = origen.GetLength(0); // dimensión 0
    int cols = origen.GetLength(1); // dimensión 1
    destino = new double[filas, cols];

    for (int f = 0; f < filas; f++)
        for (int c = 0; c < cols; c++)
            destino[f, c] = origen[f, c];
}

public static void Main(string[] args)
{
    double[,] m1 = { { 10, 20, 30 }, { 40, 50, 60 } };

    // Copiar
    double[,] m2 = null;
    CopiarMatriz2D(m2, m1);
    // ...
}
```

En esta otra versión del ejemplo anterior, cuando finaliza la ejecución del método *CopiarMatriz2D*, la referencia *m2* sigue valiendo **null**. Dése cuenta que el parámetro *destino* es una variable local al método *CopiarMatriz2D*.

Para que esta última versión sea correcta, se debe pasar el parámetro *m2* por referencia mediante la palabra clave **ref**; de esta forma, se podrá modificar.

```
public static void CopiarMatriz2D(ref double[,] destino, double[,] origen)
{
    int filas = origen.GetLength(0); // dimensión 0
    int cols = origen.GetLength(1); // dimensión 1
    destino = new double[filas, cols];
}
```

```

        for (int f = 0; f < filas; f++)
            for (int c = 0; c < cols; c++)
                destino[f, c] = origen[f, c];
    }

    public static void Main(string[] args)
    {
        double[,] m1 = { { 10, 20, 30 }, { 40, 50, 60 } };

        // Copiar
        double[,] m2 = null;
        CopiarMatriz2D(ref m2, m1);

        // ...
    }

```

En este ejemplo se ve la necesidad de pasar un tipo referencia por referencia.

PARÁMETROS DE SALIDA

Un parámetro de salida es una variable calificada **out**. La palabra clave **out** produce argumentos que se van a pasar por referencia. Esto es, **out** es similar a **ref**, excepto que **ref** requiere que se inicie la variable antes de pasarla. Según esto, el ejemplo anterior podría escribirse también así:

```

    public static void CopiarMatriz2D(out double[,] destino, double[,] origen)
    {
        int filas = origen.GetLength(0); // dimensión 0
        int cols = origen.GetLength(1); // dimensión 1
        destino = new double[filas, cols];

        for (int f = 0; f < filas; f++)
            for (int c = 0; c < cols; c++)
                destino[f, c] = origen[f, c];
    }

    public static void Main(string[] args)
    {
        double[,] m1 = { { 10, 20, 30 }, { 40, 50, 60 } };

        // Copiar
        double[,] m2;
        CopiarMatriz2D(out m2, m1);

        // ...
    }

```

PASAR ARGUMENTOS EN LA LÍNEA DE ÓRDENES

Muchas veces, cuando invocamos a un programa desde el sistema operativo, necesitamos escribir uno o más argumentos a continuación del nombre del programa, separados por un espacio en blanco. Por ejemplo, piense en la orden `ls -l` del sistema operativo UNIX o en la orden de consola `dir /p` del sistema operativo Windows. Tanto `ls` como `dir` son programas, y `-l` y `/p` son opciones o argumentos en la línea de órdenes que pasamos al programa para que tenga un comportamiento diferente al que tiene de forma predeterminada; es decir, cuando no se pasan argumentos.

De la misma forma, nosotros podemos construir aplicaciones C# que admitan argumentos a través de la línea de órdenes. ¿Qué método recibirá esos argumentos? El método **Main**, ya que este método es el punto de entrada a la aplicación y también el punto de salida. Su definición, una vez más, es como se muestra a continuación:

```
public static void Main(string[] args)
{
    // Cuerpo del método
}
```

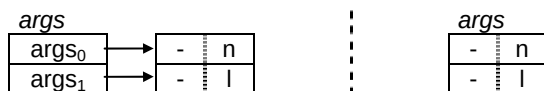
Como se puede observar, el método **Main** tiene un argumento *args* que es una matriz unidimensional de tipo **string**. El nombre *args* puede ser cualquier otro. Esta matriz almacenará los argumentos pasados en la línea de órdenes cuando se invoque a la aplicación para su ejecución de la forma que se observa a continuación. Observe que cada argumento está separado del anterior por un espacio.

MiAplicación argumento1 argumento2...

Cada elemento de la matriz *args* referencia a un argumento, de manera que *args[0]* contiene el primer argumento de la línea de órdenes, *args[1]* el segundo, etc. Por ejemplo, supongamos que tenemos una aplicación C# denominada *Test* que acepta los argumentos `-n` y `-l`. Entonces, podríamos invocar a esta aplicación escribiendo en la línea de órdenes del sistema operativo la siguiente orden:

Test -n -l

Esto hace que automáticamente la matriz *args* de referencias a objetos **string** se cree para contener dos objetos **string**: uno con el primer argumento y otro con el segundo. Puede imaginarla de cualquiera de las dos formas siguientes:



Para clarificar lo expuesto vamos a realizar una aplicación que simplemente visualice los valores de los argumentos que se la han pasado en la línea de órdenes. Esto nos dará una idea de cómo acceder desde un programa a esos argumentos. Supongamos que la aplicación se denomina *Test* y que sólo admite los argumentos *-n*, *-k* y *-l*. Esto quiere decir que podremos especificar de cero a tres argumentos. Los argumentos repetidos y no válidos se desearán. Por ejemplo, la siguiente línea invoca a la aplicación *Test* pasándole los argumentos *-n* y *-l*:

```
Test -n -l
```

El código de la aplicación propuesta se muestra a continuación:

```
public class Test
{
    public static void Main(string[] args)
    {
        // Código común a todos los casos

        System.Console.WriteLine("Argumentos: ");
        if (args.Length == 0)
        {
            // Escriba aquí el código que sólo se debe ejecutar cuando
            // no se pasan argumentos
            System.Console.WriteLine("    ninguno");
        }
        else
        {
            bool argumento_k = false, argumento_l = false,
                argumento_n = false;

            // ¿Qué argumentos se han pasado?
            for (int i = 0; i < args.Length; i++)
            {
                if (args[i].CompareTo("-k") == 0) argumento_k = true;
                if (args[i].CompareTo("-l") == 0) argumento_l = true;
                if (args[i].CompareTo("-n") == 0) argumento_n = true;
            }

            if (argumento_k) // si se pasó el argumento -k:
            {
                // Escriba aquí el código que sólo se debe ejecutar cuando
                // se pasa el argumento -k
                System.Console.WriteLine("    -k");
            }

            if (argumento_l) // si se pasó el argumento -l:
            {
                // Escriba aquí el código que sólo se debe ejecutar cuando
                // se pasa el argumento -l
            }
        }
    }
}
```

```
        System.Console.WriteLine("    -1");
    }

    if (argumento_n) // si se pasó el argumento -n:
    {
        // Escriba aquí el código que sólo se debe ejecutar cuando
        // se pasa el argumento -n
        System.Console.WriteLine("    -n");
    }
}
// Código común a todos los casos
}
```

Al ejecutar este programa, invocándolo como se ha indicado anteriormente, se obtendrá el siguiente resultado:

Argumentos:
-1
-n

MÉTODOS RECURSIVOS

Se dice que un método es recursivo si se llama a sí mismo. El compilador C# permite cualquier número de llamadas recursivas a un método. Cada vez que el método es llamado, sus parámetros y sus variables locales son iniciados.

¿Cuándo es eficaz escribir un método recursivo? La respuesta es sencilla, cuando el proceso a programar sea por definición recursivo. Por ejemplo, el cálculo del factorial de un número, $n! = n(n-1)!$, es por definición un proceso recursivo que se enuncia así: $factorial(n) = n * factorial(n-1)$.

Por lo tanto, la forma idónea de programar este problema es implementando un método recursivo. Como ejemplo, a continuación se muestra un programa que visualiza el factorial de un número. Para ello, se ha escrito un método *factorial* que recibe como parámetro un número entero positivo y devuelve como resultado el factorial de dicho número.

```
using System;
using MisClases.ES; // espacio de nombres de la clase Leer

public class Test
{
    // Cálculo del factorial de un número
    public static long factorial(int n)
    {
```

```

        if (n == 0)
            return 1;
        else
            return checked(n * factorial(n-1));
    }

    public static void Main(string[] args)
    {
        int numero;
        long fac;

        Console.Write("¿Número? ");
        numero = Leer.datoInt();
        try
        {
            fac = factorial(numero);
            Console.WriteLine("\nEl factorial de " + numero + " es: " + fac);
        }
        catch(OverflowException e)
        {
            Console.WriteLine(e.Message);
        }
    }
}

```

Obsérvese que siempre ha de existir al menos una condición de salida que permita devolver un resultado sin necesidad de una nueva llamada recursiva (puede ver más detalles en el capítulo 17).

Por defecto, un programa C# no comprueba la posibilidad de desbordamiento aritmético. Para forzar a su comprobación podemos utilizar la sentencia **checked** de cualquiera de las dos formas siguientes:

```

checked(expresión);

checked
{
    sentencias;
}

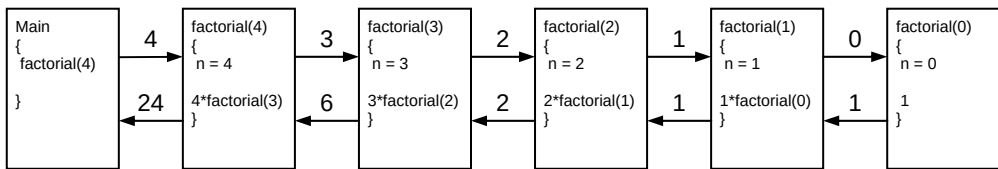
```

Una expresión o un conjunto de sentencias **checked** se revisa para ver si hay desbordamiento aritmético (**unchecked** no comprueba) y en el caso de que lo haya, se lanzará una excepción del tipo **System.OverflowException**. En el ejemplo anterior, si la expresión $n * factorial(n-1)$ produce un desbordamiento, se lanzará una excepción que será atrapada por **Main**.

En la tabla siguiente se ve el proceso seguido por el método *factorial*, durante su ejecución para $n = 4$.

Nivel de recursión	Proceso de ida	Proceso de vuelta
0	factorial(4)	24
1	4 * factorial(3)	4 * 6
2	3 * factorial(2)	3 * 2
3	2 * factorial(1)	2 * 1
4	1 * factorial(0)	1 * 1

Cada llamada al método *factorial* aumenta en una unidad el nivel de recursión. Cuando se llega a $n = 0$, se obtiene como resultado el valor 1 y se inicia la vuelta hacia el punto de partida, reduciendo el nivel de recursión en una unidad cada vez. La columna del centro especifica cómo crece la pila de llamadas hasta obtener un resultado que permita iniciar el retorno por la misma, dando solución a cada una de las llamadas pendientes. Gráficamente podemos representar este proceso así:



Observar que la ejecución de *factorial* se inicia cinco veces; cuando se resuelve *factorial(0)* hay todavía cuatro llamadas pendientes de resolver; cuando se resuelve *factorial(1)* hay todavía tres llamadas pendientes de resolver; etc. Observar también que el parámetro n es una variable local al método, por eso está presente con su valor local en cada una de las ejecuciones.

Conclusión, por cada ejecución recursiva del método, se necesita cierta cantidad de memoria para almacenar las variables locales y el estado en curso del proceso de cálculo con el fin de recuperar dichos datos cuando se acabe una ejecución y haya que reanudar la anterior. Por este motivo, en aplicaciones prácticas es imperativo demostrar que el nivel máximo de recursión es, no sólo finito, sino realmente pequeño.

Según lo expuesto, los algoritmos recursivos son particularmente apropiados cuando el problema a resolver o los datos a tratar se definen en forma recursiva. Sin embargo, el uso de la recursión debe evitarse cuando haya una solución obvia por iteración.

MÉTODOS SOBRECARGADOS

Normalmente, cada método tiene su propio nombre que lo distingue de los demás. No obstante, se pueden presentar casos en los que varios métodos ejecuten la

misma tarea sobre diferentes conjuntos de datos, y puede resultar conveniente que dichos métodos tengan el mismo nombre. En este caso, se dice que el método está sobrecargado.

La sobrecarga de un método es una característica que hace los programas más legibles. Consiste en volver a declarar un método ya declarado, con distinto número y/o tipo de parámetros. Un método sobrecargado no puede diferir solamente en el tipo del valor retornado, sino que debe diferir también en el tipo y/o en el número de sus parámetros formales.

Por ejemplo, supongamos un método *visualizar* para mostrar expresiones al estilo de **WriteLine** pero de una forma más sencilla. Podemos diseñar tantos métodos como casos pensemos que un usuario pueda necesitar. Por ejemplo, escribamos la siguiente clase:

```
public class Test
{
    public static void visualizar(string cad)
    {
        Console.WriteLine(cad);
    }
    public static void visualizar(long n)
    {
        Console.WriteLine(n);
    }
    public static void visualizar(string cad, long n)
    {
        Console.WriteLine(cad + n);
    }
    public static void visualizar(double n)
    {
        Console.WriteLine(n);
    }
    public static void visualizar(string cad, double n)
    {
        Console.WriteLine(cad + n);
    }
    public static void Main(string[] args)
    {
    }
}
```

Cuando el método *visualizar* es llamado, el compilador debe resolver cuál de los métodos con el nombre *visualizar* es invocado. Esto lo hace comparando los tipos de los parámetros actuales con los tipos de los parámetros formales de todos los métodos llamados *visualizar*. Si no encontrara un método exactamente con los

mismos tipos de argumentos, realizaría las conversiones permitidas sobre los parámetros actuales, buscando así un método apropiado.

Para probar lo expuesto, vamos a implementar el método **Main** para que escriba distintos resultados. La idea es que **Main** invoque automáticamente a uno u otro método, dependiendo de los argumentos pasados en la llamada.

```
public static void Main(string[] args)
{
    long ai = 2, bi = 2;
    double ad = 1.5;

    visualizar("Resultados: ");           // invoca al primer método
    visualizar(ai);                       // invoca al segundo método
    visualizar("Dato entero = ", bi);    // invoca al tercer método
    visualizar("Dato real = ", ad);      // invoca al quinto método
}
```

Ejecución del programa:

```
Resultados:
2
Dato entero = 2
Dato real = 1,5
```

ARGUMENTOS CON NOMBRE

Los argumentos con nombre evitan recordar el orden de los parámetros en la lista de parámetros de un método en el momento de ser invocado o llamado (el método define un *parámetro*, y el código de llamada transfiere un *argumento* a dicho parámetro). ¿Cómo? Pues especificando el nombre del parámetro antes del argumento utilizando la sintaxis: *nombre_parámetro: argumento*.

Por ejemplo, se puede llamar a un método *visualizar* de la manera estándar, mediante el envío de los argumentos en el orden de los parámetros definidos en el método:

```
public static void visualizar(int a, float b, double c)
{
    Console.WriteLine("parámetro 1 = {0}, parámetro 2 = {1},
                      parámetro 3 = {2}", a, b, c);
}
```

```
visualizar(2, 3.7F, 8.125);
```

Pero conociendo el nombre de los parámetros, también se puede enviar los argumentos en cualquier orden:

```
visualizar(c: 8.125, a: 2, b: 3.7F);
```

Las especificaciones de argumento con nombre deben aparecer después de haber especificado todos los argumentos fijos:

```
visualizar(2, c: 8.125, b: 3.7F); // correcto
visualizar(b: 3.7F, a: 2, 8.125); // error
```

PARÁMETROS CON VALOR POR OMISIÓN

Todos los parámetros formales de un método, o bien algunos de ellos, esto es, desde un determinado parámetro hasta el final, se pueden declarar con un valor predeterminado que se utilizará cuando en la llamada a dicho método se omita el argumento correspondiente. Por ejemplo, la función *visualizar* que se expone a continuación asume para sus parámetros *a*, *b* y *c* los valores 1, 2.5 y 3.456, respectivamente, cuando éstos se omitan en la llamada.

```
public static void visualizar(int a = 1, float b = 2.5F, double c = 3.456)
{
    Console.WriteLine("parámetro 1 = {0}, parámetro 2 = {1}, " +
        "parámetro 3 = {2}", a, b, c);
}

public static void Main(string[] args)
{
    visualizar();
    visualizar( 2 );
    visualizar( 2, 3.7F );
    visualizar( 2, 3.7F, 8.125 );
    visualizar(2, c: 8.125, b: 3.7F);
}
```

Cuando ejecute este programa, obtendrá los siguientes resultados:

```
parámetro 1 = 1, parámetro 2 = 2.5, parámetro 3 = 3.456
parámetro 1 = 2, parámetro 2 = 2.5, parámetro 3 = 3.456
parámetro 1 = 2, parámetro 2 = 3.7, parámetro 3 = 3.456
parámetro 1 = 2, parámetro 2 = 3.7, parámetro 3 = 8.125
parámetro 1 = 2, parámetro 2 = 3.7, parámetro 3 = 8.125
```

Observe que omitir un argumento en la llamada implica omitir todos los argumentos que le siguen y especificar los que le preceden. Según esto, la siguiente llamada no sería correcta:

```
visualizar(2, , 8.125); // error
```

NÚMERO VARIABLE DE PARÁMETROS

En el ejemplo anterior, si quisiéramos mostrar cualquier otro conjunto de resultados que no pueda ser correspondido por las sobrecargas existentes del método *visualizar*, tendríamos que añadir a la clase *Test* nuevas sobrecargas que satisfagan los resultados perseguidos. Quizás el problema planteado pueda resolverse con un método que admita un número variable de argumentos.

Pues bien, C# soporta métodos con un número variable de parámetros, todos del mismo tipo, y no requiere que el número de argumentos que se pasen sea previamente determinado. ¿Cómo se especifica esta característica? Anteponiendo la palabra clave **params** a la definición del parámetro del método, el cual debe ser una matriz unidimensional. El resultado será que la matriz aceptará un número variable de argumentos. Por ejemplo:

```
public static void visualizar(params int[] parámetros)
{
    for (int i = 0 ; i < parámetros.Length; i++)
        Console.WriteLine(parámetros[i]);
}
```

Esta versión de *visualizar* permite mostrar un número indeterminado de enteros. El calificativo **params** de la matriz *parámetros* indica que puede ser pasado cualquier número de datos de tipo **int**. Por ejemplo, cualquiera de las llamadas siguientes sería correcta, incluso con cero argumentos:

```
visualizar();
visualizar(2);
visualizar(2, 4);
visualizar(2, 4, 6);
```

No se permiten parámetros adicionales después de la palabra clave **params**, ni varias palabras clave **params** en una misma declaración de método.

Esto significa que con una lista de parámetros variable podemos trabajar igual que lo hacemos con una matriz unidimensional. No obstante, hay que tener presentes las siguientes limitaciones: sólo se puede especificar una lista de parámetros variable y, en el caso de que se especifiquen otros parámetros, ésta debe ponerse en último lugar. Según esto, el siguiente método no sería correcto:

```
static void visualizar(string s, params int[] a, params float[] b)
{
    // ...
}
```


El hecho de que la lista de parámetros variable obligue a que todos ellos sean del mismo tipo podría hacer que el uso de esta característica fuera bastante limitado. Pero, si recordamos, C# implementa la característica de empaquetado/desempaquetado (*boxing/unboxing*; véase *Conversión entre tipos valor y tipos referencia* en el capítulo 6) que convierte automáticamente un valor de un tipo primitivo (como **int**) a un objeto de la clase **Object** y también permite la conversión inversa. Estas facilidades permiten escribir métodos como el siguiente:

```
static void visualizar(string s, params object[] p)
{
    // ...
}
```

Este método, cuando sea invocado, requiere un primer argumento de tipo **string** y una lista variable de argumentos de diferentes tipos.

Si ahora compara el siguiente programa con el que hicimos anteriormente, comprobará que presenta muchas más posibilidades:

```
using System;

public class Test
{
    public static void visualizar(params object[] parámetros)
    {
        foreach (object p in parámetros)
            Console.Write(p + " ");
        Console.WriteLine();
    }

    public static void visualizar(string s, params object[] parámetros)
    {
        Console.Write(s);
        foreach (object p in parámetros)
            Console.Write(p + " ");
        Console.WriteLine();
    }

    public static void Main(string[] args)
    {
        long ai = 2, bi = 3;
        double ad = 1.5;

        visualizar("Resultados: ");
        visualizar(ai);
        visualizar("Dato entero = ", bi);
        visualizar("Dato real = ", ad);
        visualizar(ai, bi, ad);
    }
}
```

```
        visualizar("Datos enteros = ", ai, bi);
    }
}
```

Ejecución del programa:

Resultados:

2

Dato entero = 3

Dato real = 1,5

2 3 1,5

Datos enteros = 2 3

LA CLASE Object

Como ya se ha dicho en más de una ocasión, la clase **Object** es la clase raíz de la jerarquía de clases de la biblioteca .NET y pertenece al paquete **System**. Respecto a las clases que implementemos en nuestras aplicaciones, todas pasarán a ser automáticamente subclases de esta clase. Esto se traduce en que todos los métodos de **Object** son heredados por las clases de la biblioteca .NET y por cualquier otra clase que incluyamos en un programa. Las subclases pueden reemplazar, y de hecho reemplazan, algunos de estos métodos, entre los que se incluyen **Equals**, **Finalize**, **GetHashCode** y **ToString**.

bool Equals(Object obj)

El método **Equals** de la clase **Object** retorna **true** si, y sólo si, las dos referencias comparadas señalan al mismo objeto; esto es, proporciona el mismo resultado que el operador **"=="**. Esto es así porque la intención es proporcionar un método que pueda ser sobrescrito en cada una de las subclases de **Object** que requieran una funcionalidad más específica. Por ejemplo, consideremos el siguiente código que define dos referencias a otros dos objetos de la clase **String**:

```
using System;
using System.Text;

public class Test
{
    public static void Main(string[] args)
    {
        String str1 = "abc";
        String str2 = new StringBuilder("abc").ToString();
        // Comparar referencias
        if ((object)str1 == (object)str2)
            Console.WriteLine("Las referencias son al mismo objeto");
    }
}
```

```

    else
        Console.WriteLine("Las referencias son a objetos diferentes.");
    // Comparar contenidos
    if (str1.Equals(str2))
        Console.WriteLine("Mismo contenido.");
    else
        Console.WriteLine("Diferente contenido");
}
}

```

Ejecución del programa:

*Las referencias son a objetos diferentes.
Mismo contenido.*

La expresión `(object)str1 == (object)str2` será **true** si la referencia `str1` es igual a la referencia `str2` (el mismo resultado se obtendría con la expresión `Object.ReferenceEquals(str1, str2)`); esto es, si ambas variables contienen idénticos valores, los cuales se corresponderán con la posición de memoria donde se localice un objeto. En cambio, la expresión `str1.Equals(str2)` compara el contenido de los objetos (igual que haría `str1 == str2`); en este caso verifica si ambas cadenas contienen los mismos caracteres. Esto es así porque el método **Equals** de **Object** (igual que el operador `==`) ha sido sobrescrito en la clase **String** para que haga esta tarea más específica. Todas las subclases de **Object** deberían sobrescribir el método **Equals** para que realicen una comparación que sea útil.

String ToString()

El método **ToString** de la clase **Object** retorna un **String** que almacena el nombre completo de la clase del objeto que recibe el mensaje **ToString**. El ejemplo siguiente permite verificar lo expuesto.

```

public class Test
{
    public static void Main(string[] args)
    {
        Object obj1 = new Object();
        Test obj2 = new Test();
        String obj3 = new StringBuilder("abc").ToString();
        Console.WriteLine(obj1.ToString()); // escribe System.Object
        Console.WriteLine(obj2.ToString()); // escribe Test
        Console.WriteLine(obj3);           // escribe abc
    }
}

```

Con respecto al método **ToString** diremos lo mismo que para **Equals**; esto es, todas las subclases de **Object** deberían sobrescribir el método **ToString** para que proporcione una información que sea útil. Por ejemplo, la clase **StringBuilder** sobrescribe este método para que retorne el **String** contenido en el propio objeto **StringBuilder** que recibe el mensaje **ToString**.

void Finalize()

Este método es invocado por el *recolector de basura* cuando la máquina virtual de .NET determina que no hay más referencias a un objeto.

El método **Finalize** de la clase **Object** es **protected** y no ejecuta ninguna acción en especial; simplemente retorna normalmente. Las subclases de **Object** deberán sobrescribir la definición de este método sólo cuando necesiten ejecutar alguna operación de finalización especial.

int GetHashCode()

Sirve como función *hash* para un tipo concreto. Este método es apropiado para su utilización en algoritmos de *hash* y en estructuras de datos como las tablas *hash*.

Las subclases de **Object** deben reemplazar **GetHashCode** por una implementación que devuelva un código *hash* único.

MÁS SOBRE REFERENCIAS Y OBJETOS String

En el capítulo anterior hicimos una breve exposición acerca de cómo crear un objeto **String** a partir de un literal o a partir de otro **String**. Por ejemplo:

```
string str = "abc";
```

Este ejemplo crea un objeto **String** con el contenido “abc”. Dicho proceso puede realizarse también así:

```
string str = new StringBuilder("abc").ToString();
```

No obstante, es importante saber cuál es el comportamiento de C# (nos referimos al CLR: motor de ejecución) ante las dos formas que acabamos de exponer de crear un objeto **String**.

Cada literal de caracteres es representado internamente por un objeto **String**. Así mismo, el CLR mantiene una tabla, denominada *agrupación interna*, para se-

guir la pista al almacenamiento de objetos **String**. Dicha tabla contiene una sola referencia a cada literal de cadena diferente declarado o creado en el programa. De esta forma, un objeto **String** con un valor determinado sólo existirá una vez. Entonces, cuando el código declara un literal, el CLR añade una referencia al objeto **String** correspondiente a dicha tabla; posteriormente, si aparece el mismo literal en cualquier otra parte del código de la clase, el CLR no añade un nuevo objeto, sino que utiliza el que hay. Esta forma de proceder ahorra memoria y no causa problemas porque los **String** son objetos no modificables; por lo tanto, no hay posibilidad de que una parte del código pueda modificar un objeto **String** compartido por otra parte de código.

Anteriormente, en este capítulo, vimos cómo utilizar el método **Equals** para verificar los contenidos de dos objetos **String**. También vimos que, a diferencia de este método, el método **Object.ReferenceEquals** no compara los contenidos de los objetos referenciados sino las referencias. Esto nos permitirá analizar mediante algunos ejemplos lo expuesto en el párrafo anterior:

```
string str1 = "abc";
string str2 = "abc";

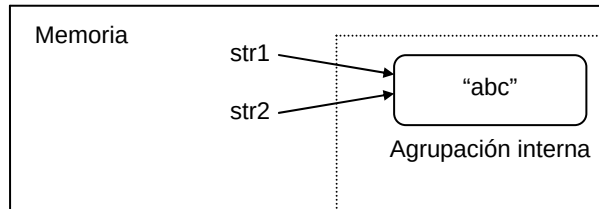
if (str1.Equals(str2))
{
    // el resultado es true siempre
}

if (Object.ReferenceEquals(str1, str2))
{
    // el resultado es true siempre
}
```

En este ejemplo, el resultado de *str1.Equals(str2)* es siempre **true**, lo cual es lógico porque independientemente de que se trate o no de objetos diferentes, los contenidos son los mismos (veremos que se trata de un único objeto). Y el resultado de la expresión *ReferenceEquals(str1, str2)*, en este caso, también es **true** porque ambos identificadores se refieren al mismo objeto. Analicemos por qué.

Cuando se ejecuta la primera línea, el CLR añade el objeto **String** “abc” a la agrupación interna. Cuando se ejecuta la segunda línea no se añade un objeto nuevo porque ya existe uno con el mismo literal.

Según lo expuesto, la primera línea asigna a *str1* una referencia al objeto almacenado en la agrupación interna de objetos **String**; y la segunda línea almacena en *str2* una referencia al mismo objeto. La figura siguiente muestra esto gráficamente:



El método **IsInterned** de la clase **String** permite recuperar una referencia al objeto **String** especificado si está en la agrupación interna; en otro caso devuelve **null**. Por ejemplo:

```
using System;
using System.Text;

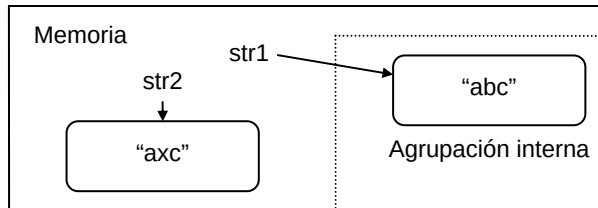
class Test
{
    public static void EsInterno(string str)
    {
        string resu = "El String '" + str;

        if ((string.IsInterned(str) == null))
        {
            resu += "' no es interno.";
        }
        else
        {
            resu += "' es interno.";
        }
        Console.WriteLine(resu);
    }

    public static void Main()
    {
        string str1 = new StringBuilder("abc").ToString();
        string str2 = null;
        Console.Write("String: ");
        str2 = Console.ReadLine();
        EsInterno(str1);
        EsInterno(str2);
    }
}
```

Cuando se ejecuta la primera línea de **Main**, el CLR añade el objeto **String** "abc" a la agrupación interna y almacena una referencia al mismo en `str1`. En cambio, las líneas siguientes sólo almacenarán en `str2` una referencia al objeto que ya existe en la agrupación interna de objetos **String** si el literal introducido a través del teclado coincide con el de `str1`; si no coincide, se crea un nuevo objeto

String referenciado por *str2* fuera de la agrupación interna. La figura siguiente muestra esto gráficamente:



No obstante, es posible colocar los objetos **String** en la agrupación interna utilizando su método estático **Intern** de la clase **String**. Esto puede redundar en un ahorro de memoria en programas que utilicen una gran cantidad de objetos **String**. Por ejemplo:

```
string str1 = new StringBuilder("abc").ToString();
string str2 = null;
Console.Write("String: ");
str2 = Console.ReadLine();
str2 = string.Intern(str2);
```

Finalmente, es importante recordar que cuando un método actúa sobre un objeto **String** el resultado es un nuevo objeto, lo que mantiene intacto el objeto original en todos los casos. Por ejemplo:

```
str2 = str2.Replace('x', 'b');
```

Supongamos que *str2* y *str1* son referencias a un objeto "abc" de la agrupación interna. Después de ejecutarse la línea de código anterior se generará un nuevo objeto referenciado por *str2* sólo en el caso de que no exista otro igual en la agrupación interna. Por lo tanto, el objeto que referenciaba *str2*, que era el mismo que el referenciado por *str1*, permanecerá inalterado en todos los casos. El nuevo objeto generado no se añadirá a la agrupación interna a no ser que se invoque explícitamente al método **Intern**.

LA CLASE Array

La clase **Array** del espacio de nombres **System** contiene varios métodos **static** para manipular matrices. Algunos de estos métodos, como **Equals**, **Clone**, **Copy**, **GetLength** y **GetUpperBound** ya los hemos visto; otros, como **Clear**, **Sort**, **BinarySearch** y **Reverse** los veremos a continuación.

Clear

Este método permite asignar un valor *0* o **null** a todos los elementos de una matriz, o bien a cada elemento de un rango especificado. La sintaxis expresada de forma genérica para utilizar este método es la siguiente:

```
void Clear(tipo[] m, int desdeInd, int longitud)
```

donde *m* es la matriz, *desdeInd* es el índice del primer elemento del rango y *longitud* el número de elementos del mismo, y *tipo* es cualquier tipo de datos de los siguientes: **object**, **string**, **byte**, **char**, **short**, **int**, **long**, **float**, **double**, etc. Como ejemplo, puede probar los resultados que produce el siguiente código:

```
double[] a = {55,50,45,40,35,30,25,20,15,10};  
System.Array.Clear(a, 0, a.Length);
```

Sort

Este método permite ordenar los elementos de una matriz unidimensional en orden ascendente utilizando el algoritmo *quicksort*. Se trata de un algoritmo muy eficiente ya que el tiempo requerido para realizar la ordenación es muy pequeño. La sintaxis expresada de forma genérica para utilizar este método es la siguiente:

```
void Sort(tipo[] m)  
void Sort(tipo[] m, int desdeInd, int longitud)
```

donde *m* es la matriz a ordenar. Cuando sólo queramos ordenar un rango de elementos, utilizaremos el segundo formato de **Sort** donde *desdeInd* es el índice del primer elemento del rango y *longitud* el número de elementos del mismo, y *tipo* es cualquier tipo de datos de los siguientes: **object**, **string**, **byte**, **char**, **short**, **int**, **long**, **float**, **double**, etc. Como ejemplo, puede probar los resultados que produce el siguiente código:

```
double[] a = {55,50,45,40,35,30,25,20,15,10};  
System.Array.Sort(a);
```

Este otro ejemplo ordena la matriz de objetos **string** y después muestra el contenido de dicha matriz:

```
string[] a = {"ccc","ddd","aaa","eee","bbb"};  
System.Array.Sort(a);  
foreach (string str in a)  
    System.Console.WriteLine(str);
```


BinarySearch

Este método permite buscar un valor en una matriz unidimensional que esté ordenada ascendentemente utilizando el algoritmo de *búsqueda binaria*. Se trata de un algoritmo muy eficiente en cuanto a que el tiempo requerido para realizar una búsqueda es muy pequeño. La sintaxis expresada de forma genérica para utilizar este método es la siguiente:

```
int BinarySearch(tipo[] m, tipo clave)
```

donde *m* representa la matriz, *clave* es el valor que se desea buscar del mismo tipo que los elementos de la matriz y *tipo* es cualquier tipo de datos de los siguientes: **object**, **string**, **byte**, **char**, **short**, **int**, **long**, **float**, **double**, etc.

El valor devuelto es un entero correspondiente al índice del elemento que coincide con el valor buscado. Si el valor buscado no se encuentra, entonces el valor devuelto es: $-(\text{punto de inserción}) - 1$ (complemento a 1). El valor de *punto de inserción* es el índice del elemento de la matriz donde debería encontrarse el valor buscado. La expresión $-(\text{punto de inserción}) - 1$ garantiza que el índice devuelto será mayor o igual que cero sólo si el valor buscado es encontrado.

Como ejemplo, analice el siguiente código:

```
double[] a = {10,15,20,25,30,35,40,45,50,55}; // matriz ordenada
int i;
i = System.Array.BinarySearch(a, 25.0); // i = 3
i = System.Array.BinarySearch(a, 27.0); // i = -5
i = System.Array.BinarySearch(a, 5.0); // i = -1
i = System.Array.BinarySearch(a, 60.0); // i = -11
```

Reverse

Este método permite invertir la secuencia de todos los elementos de una matriz unidimensional, o bien de los elementos de un rango especificado. La sintaxis expresada de forma genérica para utilizar este método es la siguiente:

```
void Reverse(tipo[] m)
void Reverse(tipo[] m, int desdeInd, int longitud)
```

donde *m* es la matriz. Cuando sólo queramos invertir la secuencia de un rango de elementos, utilizaremos el segundo formato de **Reverse** donde *desdeInd* es el índice del primer elemento del rango y *longitud* el número de elementos del mismo, y *tipo* es cualquier tipo de datos de los siguientes: **object**, **string**, **byte**, **char**, **short**, **int**, **long**, **float**, **double**, etc.

Un ejemplo de cómo utilizar este método es el siguiente:

```
double[] a = {10,15,20,25,30,35,40,45,50,55};  
System.Array.Reverse(a);
```

COLECCIONES

Hay dos formas de agrupar objetos: utilizando matrices de objetos y mediante colecciones. Cuando utilizamos una matriz, todos sus elementos deben ser del mismo tipo, y para cambiar su tamaño durante la ejecución hay que programarlo; en cambio, cuando utilizamos una colección, sus elementos no tienen por qué compartir el mismo tipo de datos (por ejemplo, los objetos de cualquier tipo se pueden agrupar en una sola colección del tipo **Object**), y su número de elementos puede aumentar y reducirse dinámicamente a medida que cambian las necesidades del programa.

Crear una colección

Las colecciones se crean del mismo modo en que se crean otros objetos. Por ejemplo:

```
System.Collections.clase-colección colección =  
    new System.Collections.clase-colección();
```

Una vez que se ha creado una colección, se pueden agregar o quitar elementos con el método **Add** o **Remove**, respectivamente, quitar todos los elementos con el método **Clear** o averiguar cuántos elementos tiene la colección con la propiedad **Count**. Para devolver elementos específicos de la colección se utiliza la propiedad **Item** (es el operador de indexación: []), y para acceder secuencialmente a toda la colección se utiliza la sentencia **foreach**. Por ejemplo:

```
using System;  
using System.Text;  
using System.Collections;  
  
public class Test  
{  
    public static void Main(string[] args)  
    {  
        ArrayList c = new ArrayList(); // colección c  
  
        // Crear objetos y guardar sus referencias en la colección  
        c.Add(new StringBuilder("abc"));  
        c.Add(25);  
        c.Add(new StringBuilder("def"));
```

```

c.Add(123.45);

// Mostrar los objetos de la colección
for (int i = 0; i < c.Count; i += 2)
{
    System.Console.Write(c[i] + ": ");
    System.Console.WriteLine(c[i + 1]);
}
}
}

```

Ejecución del programa:

```

abc: 25
def: 123,45

```

En este ejemplo se añaden a la colección *c* cuatro objetos de distintos tipos, para finalmente, acceder a la misma y mostrar sus contenidos.

Cada elemento de la colección es de tipo **Object**; por lo tanto, la lista de elementos que se pueden agregar a la colección incluye tipos de datos estándar, objetos y matrices, así como estructuras definidas por el usuario y objetos de cualquier clase. Según esto, el tipo de referencia que devuelve la propiedad **Item** (operador []) es **Object**, lo que se deberá tener en cuenta para acceder a los miembros de la colección posteriormente. Para obtener un elemento de la colección habrá que hacer una conversión explícita. Por ejemplo:

```

StringBuilder s = (StringBuilder)c[0];
int x = (int)c[1];

```

Colecciones predefinidas

El espacio de nombres **System.Collections** contiene interfaces y clases que definen varias colecciones de objetos, como listas, colas, matrices de bits, tablas *hash* y diccionarios. Algunas de estas clases son las siguientes:

- **ArrayList**. Implementa una matriz cuyo tamaño aumenta o disminuye dinámicamente.
- **Queue**. Es una lista del tipo FIFO: *First In First Out* - primero en entrar, primero en salir.
- **Stack**. Es una lista del tipo LIFO: *Last In First Out* - último en entrar, primero en salir.

- **Hashtable.** Colección de elementos de tipo *clave-valor* organizados en función del código *hash* de la *clave*. Sólo se puede tener acceso a los elementos de la colección a través de la *clave* del elemento.
- **SortedList.** Es una versión ordenada de **Hashtable**. En este tipo de colección, se puede acceder a los elementos por la clave o por el índice.

Al tratarse de colecciones, proporcionan una funcionalidad común que puede ser utilizada con múltiples tipos de datos.

Colecciones genéricas

La obtención de un elemento de una colección predefinida, como **ArrayList**, exige una conversión explícita al tipo del dato almacenado en dicho elemento (operación denominada *unboxing*). Éste es el problema de utilizar colecciones de datos diseñadas para almacenar objetos de la clase **Object**. Por ejemplo:

```
ArrayList c = new ArrayList(); // colección c
c.Add(25);
c.Add(123);

int s = 0;
for (int i = 0; i < c.Count; ++i)
    s += (int)c[i];
```

Para solucionar este problema, C# (a partir de su versión 2005) incluye en su biblioteca de clases el espacio de nombres **System.Collections.Generic** que contiene interfaces y clases que definen tipos genéricos que permiten al usuario crear colecciones particularizadas para un tipo específico de datos. El mismo ejemplo realizado anteriormente, pero utilizando ahora la clase genérica **List** del espacio de nombres **Generic**, como alternativa a **ArrayList**, sería así:

```
List<int> c = new List<int>(); // colección c
c.Add(25);
c.Add(123);

int s = 0;
for (int i = 0; i < c.Count; ++i)
    s += c[i];
```

En este caso, cuando se define el objeto colección, se especifica entre ángulos a continuación del nombre de la colección el tipo de los elementos que almacena la misma, en el ejemplo `<int>`, lo que evita tener que realizar la conversión explícita cuando se obtiene un elemento de la colección.

Según lo expuesto, los tipos genéricos son colecciones de datos particularizadas, como tablas *hash*, colas, pilas, diccionarios y listas, definidas en el espacio de nombres **System.Collections.Generic**. Algunas de las clases de este espacio de nombres son las siguientes:

- **Dictionary**. Representa una colección de pares *clave-valor* organizados según la clave.
- **LinkedList**. Representa una lista doblemente enlazada.
- **LinkedListNode**. Representa un nodo en una colección **LinkedList**. Esta clase no se puede heredar.
- **List**. Representa una matriz unidimensional dinámica.
- **Queue**. Representa una cola (primero en entrar, primero en salir).
- **SortedList**. Representa una colección de pares *clave-valor* clasificados ascendentemente por la clave.
- **KeyValuePair**. Representa un elemento *clave-valor* (por ejemplo, un elemento de una colección **SortedList**).
- **SortedDictionary**. Es un diccionario ordenado con operaciones de inserción y eliminación, lo que hace de él una alternativa útil a **SortedList**.
- **Stack**. Representa una pila (último en entrar, primero en salir).

También podemos diseñar nuestros propios tipos genéricos, cuestión que veremos más adelante en otro capítulo.

EJERCICIOS RESUELTOS

1. Utilizando el método **NextDouble** de la clase **Random** del espacio de nombres **System**, realizar un programa que muestre seis números aleatorios diferentes entre 1 y 49 ordenados ascendentemente.

Para producir enteros aleatorios en un intervalo dado puede utilizar la fórmula: $\text{parte_entera_de}((\text{límiteSup} - \text{límiteInf} + 1) * \text{random} + \text{límiteInf})$, siendo *random* un valor mayor o igual que 0 y menor que 1.

La solución al problema planteado puede ser de la siguiente forma:

- Definimos el rango de los números que deseamos obtener, una matriz para almacenarlos y un objeto **Random** que recibirá los mensajes **NextDouble**.

```
int i, k, límiteSup = 49, límiteInf = 1;
int n[] = new int[6];
Random rnd = new Random();
```

- Obtenemos el siguiente número aleatorio y verificamos si ya existe en la matriz, en cuyo caso lo desechamos y volvemos a obtener otro. Este proceso lo repetiremos hasta haber generado todos los números solicitados.

```
for (i = 0; i < n.length; i++)
{
    n[i] = (int)((límiteSup - límiteInf + 1) * rnd.NextDouble() +
                límiteInf);
    for (k = 0; k < i; k++)
        if (n[k] == n[i]) // ya existe
        {
            i--;
            break;
        }
}
```

La sentencia **for** externa define cuántos números se van a generar. Cuando se genera un número se almacena en la siguiente posición de la matriz. Después, la sentencia **for** interna compara el último número generado con todos los anteriormente generados. Si ya existe, se decrementa el índice *i* de la matriz para que cuando sea incrementado de nuevo por el **for** externo apunte al elemento repetido y sea sobrescrito por el siguiente número generado.

- Una vez obtenidos todos los números, ordenamos la matriz y la visualizamos.

```
Array.Sort(n);
for (i = 0; i < n.Length; i++)
    System.Console.Write(n[i] + " ");
```

El programa completo se muestra a continuación.

```
using System;

public class CRandomCSharp
{
    // Obtener números aleatorios dentro de un rango
    public static void Main(string[] args)
    {
        int i, k, límiteSup = 49, límiteInf = 1;
        int[] n = new int[6];
```

```

// Crear un objeto de la clase Random
Random rnd = new Random();

for (i = 0; i < n.Length; i++)
{
    // Obtener un número aleatorio
    n[i] = (int)((límiteSup - límiteInf + 1) * rnd.NextDouble() +
                límiteInf);
    // Verificar si ya existe el último número obtenido
    for (k = 0; k < i; k++)
        if (n[k] == n[i]) // ya existe
        {
            i--; // i será incrementada por el for externo
            break; // salir de este for
        }
}

// Ordenar la matriz
Array.Sort(n);

// Mostrar la matriz
for (i = 0; i < n.Length; i++)
    System.Console.Write(n[i] + " ");
System.Console.WriteLine();
}
}

```

2. Realice un programa que partiendo de dos matrices de cadenas de caracteres ordenadas ascendentemente, construya y visualice una tercera matriz también ordenada ascendentemente. La idea que se persigue es construir la tercera lista ordenada; no construirla y después ordenarla empleando el método **Sort**.

Para ello, el método **Main** proporcionará las dos matrices e invocará a un método cuyo prototipo será el siguiente:

```
int Fusionar(string[] lista1, string[] lista2, string[] lista3);
```

El primer parámetro y el segundo del método *Fusionar* son las dos matrices de partida, y el tercero es la matriz que almacenará los elementos de las dos anteriores.

El proceso de fusión consiste en:

- a) Partiendo de que ya están construidas las dos matrices de partida, tomar un elemento de cada una de las matrices.
- b) Comparar los dos elementos (uno de cada matriz) y almacenar en la matriz resultado el menor.

- El programa completo se muestra a continuación.


```

// Declarar la matriz que va a almacenar el resultado de
// fusionar las dos anteriores
string[] lista3 = new string[lista1.Length + lista2.Length];

// Fusionar lista1 y lista2 y almacenar el resultado en lista3.
// El método "Fusionar" devolverá un 0 cuando no se pueda
// realizar la fusión.
int ind, r;
r = Fusionar(lista1, lista2, lista3);

// Escribir la matriz resultante
if (r != 0)
{
    for (ind = 0; ind < lista3.Length; ind++)
        System.Console.WriteLine(lista3[ind]);
}
else
    System.Console.WriteLine("Error");
}
}

```

Observe que el método *Fusionar* copia referencias. Como se expuso anteriormente en este mismo capítulo, esta forma de proceder ahorra memoria y no causa problemas porque los **string** son objetos no modificables; por lo tanto, no hay posibilidad de que una parte del código pueda modificar un objeto **string** compartido por otra parte de código.

3. Implementar un método que permita construir y mostrar un menú. Dicho método recibirá como parámetro la matriz de cadenas que dan lugar a los elementos del menú y devolverá un entero correspondiente a la opción elegida del mismo.

```

public static int menu(string[] opciones)
{
    int i;
    int opcion = 0;
    Console.WriteLine("\n_____ \n");
    for (i = 1; i <= opciones.Length; ++i)
        Console.WriteLine("    " + i + ". " + opciones[i - 1]);
    Console.WriteLine("_____");

    do
    {
        Console.Write("\nOpción (1 - " + opciones.Length + "): ");
        opcion = Leer.datoInt();
    }
    while (opcion < 1 || opcion > opciones.Length);

    return opcion;
}

```

EJERCICIOS PROPUESTOS

1. Realice un programa que se comporte como un diccionario inglés-español; esto es, solicitará una palabra en inglés y escribirá la correspondiente palabra en español. Para hacer más sencillo el ejercicio, el número de parejas de palabras será variable, pero limitado a un máximo de 100. Por ejemplo, suponer que introducimos las siguientes parejas de palabras:

<i>book</i>	libro
<i>green</i>	verde
<i>mouse</i>	ratón

Una vez finalizada la introducción de las listas de palabras pasamos al modo traducción, de forma que si tecleamos *green*, la respuesta ha de ser *verde*. Si la palabra no se encuentra se emitirá un mensaje que lo indique.

El programa constará al menos de dos métodos:

- a) *crearDiccionario*. Este método creará el diccionario.
 - b) *traducir*. Este método realizará la labor de traducción.
2. Un cuadrado mágico se compone de números enteros comprendidos entre 1 y n^2 , donde n es un número impar que indica el orden de la matriz cuadrada que contiene los números que forman dicho cuadrado mágico. La matriz que forma este cuadrado mágico cumple que la suma de los valores que componen cada fila, cada columna y cada diagonal es la misma. Por ejemplo, un cuadrado mágico de orden 3, implica un valor de $n = 3$ lo que dará lugar a una matriz de 3 por 3. Por lo tanto, los valores de la matriz estarán comprendidos entre 1 y 9 y dispuestos de la forma siguiente:

8	1	6
3	5	7
4	9	2

Realice un programa que visualice un cuadrado mágico de orden impar n . El programa verificará que n es impar y que está comprendido entre 3 y 15.

Una forma de construirlo puede ser: situar el número 1 en el centro de la primera línea, el número siguiente en la casilla situada encima y a la derecha, y así sucesivamente. Es preciso tener en cuenta que el cuadrado se cierra sobre sí mismo, esto es, la línea encima de la primera es la última y la columna a la derecha de la última es la primera. Siguiendo esta regla, cuando el número caiga en una casilla ocupada, se elige la casilla situada debajo del último número situado.

Se deberán realizar al menos los métodos siguientes:

- a) *esImpar*. Este método verificará si n es impar.
- b) *cuadradoMágico*. Este método construirá el cuadrado mágico.

3. Realice un programa que:

- a) Lea dos cadenas de caracteres denominadas *cadena1* y *cadena2* y un número entero n .
- b) Llame a un método:

```
static int compcads(cadena1, cadena2, n);
```

que compare los n primeros caracteres de *cadena1* y de *cadena2*, y devuelva como resultado un valor entero:

```
0      si cadena1 y cadena2 son iguales
1      si cadena1 es mayor que cadena2 (los  $n$  primeros caracteres)
-1     si cadena1 es menor que cadena2 (los  $n$  primeros caracteres)
```

Si n es menor que 1 o mayor que la longitud de la menor de las cadenas, la comparación se hará sin tener en cuenta este parámetro.

- c) Escriba la cadena que sea menor según los n primeros caracteres (esto es, la que esté antes por orden alfabético).

4. Escriba un programa para evaluar la expresión $(ax + by)^n$. Para ello, tenga en cuenta las siguientes expresiones:

$$(ax + by)^n = \sum_{k=0}^n \binom{n}{k} (ax)^{n-k} (by)^k$$

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

$$n! = n * (n-1) * (n-2) * \dots * 2 * 1$$

- a) Escriba un método cuyo prototipo sea:

```
static long factorial(int n);
```

El método *factorial* recibe como parámetro un entero y devuelve el factorial del mismo.

- b) Escriba un método con el prototipo:

```
static long combinaciones(int n, int k);
```

El método *combinaciones* recibe como parámetros dos enteros n y k , y devuelve como resultado el valor de $\binom{n}{k}$.

- c) Escriba un método que tenga el prototipo:

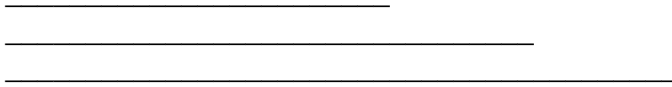
```
static long potencia(int base, int exponente);
```

El método *potencia* recibe como parámetros dos enteros, *base* y *exponente*, y devuelve como resultado el valor de $base^{\text{exponente}}$.

- d) El método **Main** leerá los valores de a , b , n , x e y , y utilizando los métodos anteriores escribirá como resultado el valor de $(ax + by)^n$.

P A R T E

2



Mecanismos de abstracción

- Clases, espacios de nombres y estructuras
- Operadores sobrecargados
- Clases derivadas e interfaces
- Tipos y métodos genéricos
- Excepciones
- Flujos

CLASES, ESPACIOS DE NOMBRES Y ESTRUCTURAS

Seguro que a estas alturas el término *clase* ya le es familiar. En los capítulos expuestos hasta ahora se han desarrollado aplicaciones sencillas, para introducirle más bien en el lenguaje y en el manejo de la biblioteca de clases de .NET que en el diseño de clases. No obstante, sí ha tenido que quedar claro que un programa orientado a objetos sólo se compone de objetos y que un objeto es la concreción de una clase. Sirva como ejemplo las aplicaciones que hemos desarrollado: todas están basadas en una clase aplicación. Es hora pues de entrar con detalle en la programación orientada a objetos, la cual tiene un elemento básico: la *clase*. En este capítulo, aprenderemos también a organizar las clases en espacios de nombres, lo que supone también un nivel más de protección para las mismas.

DEFINICIÓN DE UNA CLASE

Una *clase* es un tipo definido por el usuario que describe los atributos, propiedades y métodos de los objetos que se crearán a partir de la misma. Los *atributos* definen el estado de un determinado objeto, las *propiedades* representan información del mismo, y los *métodos* son las operaciones que definen su comportamiento. Forman parte de estos métodos los *constructores*, que permiten iniciar un objeto, y los *destructores*, que permiten destruirlo. Los atributos, las propiedades y los métodos se denominan en general *miembros* de la clase.

Según hemos aprendido, la definición de una clase consta de dos partes: el *nombre de la clase* precedido por la palabra reservada **class**, y el *cuerpo de la clase* encerrado entre llaves. Esto es:

```
class nombre_clase
{
    cuerpo de la clase
}
```

El *cuerpo de la clase* en general consta de modificadores de acceso (**public**, **protected** y **private**), atributos, propiedades, mensajes y métodos. Un método implícitamente define un mensaje (el nombre del método es el mensaje).

Por ejemplo, un círculo puede ser descrito por la posición *x*, *y* de su centro y por su *radio*. Hay varias cosas que nosotros podemos hacer con un círculo: calcular la longitud de la circunferencia, calcular el área del círculo, etc. Cada círculo es diferente (por ejemplo, tienen el centro o el radio diferente); pero visto como una *clase* de objetos, el círculo tiene propiedades intrínsecas que nosotros podemos agrupar en una definición. El siguiente ejemplo define la clase *Círculo*. Obsérvese cómo los atributos y los métodos forman el cuerpo de la clase.

```
class Círculo
{
    // Miembros privados
    private double x, y; // coordenadas del centro
    private double radio; // radio del círculo

    // Miembros protegidos
    protected void MsgEsNegativo()
    {
        Console.WriteLine("El radio es negativo. Se convierte a positivo.");
    }

    // Miembros públicos
    public Círculo() { } // constructor sin parámetros

    public Círculo(double cx, double cy, double r)
    {
        x = cx; y = cy;
        if (r < 0)
        {
            MsgEsNegativo();
            r = -r;
        }
        radio = r;
    }

    public double LongCircunferencia()
    {
        return 2 * Math.PI * radio;
    }
}
```



```

public double AreaCírculo()
{
    return Math.PI * radio * radio;
}
}

```

Este ejemplo define un nuevo tipo de datos, *Círculo*, que puede ser utilizado dentro de un programa fuente exactamente igual que cualquier otro tipo. Un objeto de la clase *Círculo* tendrá los atributos *x*, *y* y *radio*, los métodos *MsgEsNegativo*, *LongCircunferencia* y *AreaCírculo*, y dos constructores *Círculo*, uno sin parámetros y otro con ellos.

Atributos

Los atributos constituyen la *estructura interna* de los objetos de una clase. Para declarar un atributo, proceda exactamente igual que ha hecho para declarar cualquier otra variable dentro de un método. Por ejemplo:

```

class Círculo
{
    private double x, y;
    private double radio;
    // ...
}

```

En una clase, cada atributo debe tener un nombre único. En cambio, se puede utilizar el mismo nombre con atributos, y con miembros en general, que pertenecen a diferentes clases, porque una clase define su propio ámbito.

Es posible asignar un valor inicial a un atributo de una clase. Por ejemplo, en la clase *Círculo* podemos iniciar el *radio* con el valor 1, aunque generalmente esto no se hace, ya que como expondremos un poco más adelante este tipo de operaciones son típicas del constructor de la clase:

```

class Círculo
{
    private double x, y;
    private double radio = 1;
    // ...
}

```

También podemos declarar como atributos de una clase referencias a otros objetos de clases existentes. El siguiente ejemplo define la clase *Punto* y después declara el atributo *centro* de *Círculo*, de la clase *Punto*.

```
class Punto
{
    private double x, y;

    public Punto() {}
    public Punto(double cx, double cy) { x = cx; y = cy; }
}

class Círculo
{
    private Punto centro; // coordenadas del centro
    private double radio; // radio del círculo
    // ...
}
```

El orden de las clases es indiferente. Esta forma de proceder ya ha sido utilizada en capítulos anteriores. Recuerde, por ejemplo, que en más de una ocasión hemos declarado un atributo de la clase **String**.

Métodos de una clase

Los métodos generalmente forman lo que se denomina *interfaz* o medio de acceso a la estructura interna de los objetos; ellos definen las operaciones que se pueden realizar con sus atributos. Desde el punto de vista de la POO, el conjunto de todos estos métodos se corresponde con el conjunto de mensajes a los que los objetos de una clase pueden responder. Esto significa que los miembros de una clase sólo podrán ser accedidos por medio de objetos de dicha clase.

Para definir un método de una clase, proceda exactamente igual que ha hecho para definir cualquier otro método en las aplicaciones realizadas en los capítulos anteriores. No olvide que una aplicación se basa en una clase. Como ejemplo puede observar los métodos *Círculo* y *LongCircunferencia* de la clase *Círculo*.

```
class Círculo
{
    // ...
    public Círculo(double cx, double cy, double r) // constructor
    {
        x = cx; y = cy;
        if (r < 0)
        {
            MsgEsNegativo();
            r = -r;
        }
        radio = r;
    }
}
```

```
public double LongCircunferencia()
{
    return 2 * Math.PI * radio;
}
// ...
}
```

En C# un método es una definición incluida siempre dentro del cuerpo de una clase. Así mismo, recuerde que los métodos no se pueden anidar.

Propiedades

Una propiedad es un miembro, normalmente público, de una clase que proporciona un mecanismo flexible para asignar u obtener el valor de un atributo privado de la misma. En realidad es un método especial con uno o dos *descriptores de acceso*: **get** y/o **set**. Estos descriptores son rutinas de código declaradas dentro de la propiedad, entre { y }. Su sintaxis es la siguiente:

```
public tipo nombre_propiedad
{
    get
    {
        // Aquí se devuelve, "return", el valor del atributo
    }

    set
    {
        // Aquí se asigna el valor "value" al atributo
    }
}
```

donde *nombre_propiedad* es el nombre dado a la propiedad, *tipo* es el tipo del atributo al que se refiere la propiedad, el descriptor de acceso **get** debe devolver el valor del atributo y el descriptor de acceso **set** debe asignar a dicho atributo el valor almacenado en el parámetro implícito **value**, que tiene el mismo tipo que la propiedad.

Como ejemplo, vamos a implementar la propiedad que permita acceder al atributo *radio* de *Círculo*:

```
public double Radio
{
    get
    {
        return radio;
    }
}
```

```
set
{
    if (value < 0)
    {
        MsgEsNegativo();
        value = -value;
    }
    radio = value;
}
```

Observe que a través de **set** aseguramos que el radio a asignar no sea negativo; si el radio fuera negativo, simplemente lo cambiará a positivo y visualizará un mensaje indicándolo; en otro caso, asignará el valor **value** al atributo *radio* del objeto. A través de **get** simplemente devolvemos el valor del atributo.

El siguiente ejemplo asigna al atributo *radio* de *obj1* el valor 20 por medio de la propiedad *Radio*. El valor a asignar, en este caso 20, es almacenado en el parámetro implícito **value** de la propiedad.

```
Círculo obj1 = new Círculo();
obj1.Radio = 20;
```

Normalmente, los descriptores de acceso de una propiedad se definen por parejas, utilizando **get** y **set**. Ahora bien, si la propiedad es de sólo lectura (sólo **get**) o de sólo escritura (sólo **set**) puede definirse cada descriptor de acceso de forma individual. Por ejemplo, la propiedad *Radio* de sólo lectura sería así:

```
public double Radio
{
    get { return radio; }
}
```

Propiedades auto-implementadas

Una propiedad auto-implementada hace que la declaración de la misma sea más breve cuando no se requiere ninguna lógica adicional en los descriptores de acceso de la propiedad. Al declarar una propiedad como se muestra en el ejemplo siguiente, el compilador crea un atributo de respaldo privado y anónimo que no está accesible directamente. Por ejemplo:

```
public double Radio {get; set;}
```

La declaración de la propiedad *Radio* anterior es equivalente a esta otra:

```
private double _radio;
```

```
public double Radio
{
    get { return _radio; }
    set { _radio = value; }
}
```

El nivel de acceso en los descriptores puede ser diferente:

```
public double Radio {get; private set;}
```

Control de acceso a los miembros de la clase

El concepto de clase incluye la idea de ocultación de datos, que básicamente consiste en que no se puede acceder directamente a los atributos de un objeto, sino que hay que hacerlo a través de las propiedades y métodos de la clase. Esto quiere decir que, de forma general, el usuario de la clase sólo tendrá acceso a una o más propiedades y/o métodos que le permitirán acceder a los miembros privados, ignorando la disposición de éstos. De esta forma se consiguen dos objetivos importantes:

1. Que el usuario no tenga acceso directo a la estructura de datos interna de la clase, para que no pueda generar código basado en esa estructura.
2. Que si en un momento determinado alteramos la definición de la clase, excepto el prototipo de las propiedades y métodos, todo el código escrito por el usuario basado en estas propiedades y métodos no tendrá que ser retocado.

Piense que si el objetivo 1 no se cumpliera, cuando se diera el objetivo 2 el usuario tendría que reescribir el código que hubiera desarrollado basándose en la estructura interna de los datos.

Para controlar el acceso a los miembros de una clase, C# provee las palabras clave **private** (privado), **protected** (protegido) y **public** (público), aunque también es posible omitirlas, en cuyo caso se supone acceso privado. Estas palabras clave, denominadas *modificadores de acceso*, son utilizadas para indicar el tipo de acceso permitido a cada miembro de la clase. Si observamos la clase *Círculo* expuesta anteriormente identificamos miembros privados, protegidos y públicos.

```
class Círculo
{
    // Miembros privados
    private double x, y;        // coordenadas del centro
    private double radio;      // radio del círculo

    // Miembros protegidos
```

```
protected void MsgEsNegativo()
{
    Console.WriteLine("El radio es negativo. Se convierte a positivo.");
}

// Miembros públicos
public Círculo() {} // constructor sin parámetros

// ...
}
```

Otros modificadores de acceso son **internal** y **protected internal**. Los miembros internos sólo son accesibles dentro del mismo ensamblado, y los calificados protegidos más internos son accesibles desde el mismo ensamblado y desde las clases derivadas (en otro capítulo más adelante veremos clases derivadas).

Es importante no olvidar que un miembro de una clase sólo puede ser accedido, implícita o explícitamente, por medio de un objeto de esa clase. En el ejemplo siguiente, el método *AreaCírculo* de la clase *Círculo* es accedido por medio del objeto *c* de la misma clase; en POO se dice que el objeto *c* recibe el mensaje *AreaCírculo* y responde ejecutando el método del mismo nombre.

```
class Test
{
    public static void Main(string[] args)
    {
        Círculo c = new Círculo(100.0, 200.0, 10.0); // invoca al constructor
        double área = c.AreaCírculo(); // c recibe el mensaje AreaCírculo
        double circ = LongCircunferencia(); // error: ¿para qué objeto se invoca?
    }
}
```

Acceso público

Un miembro de una clase declarado **public** (público) puede ser accedido por un objeto de esa clase en cualquier parte donde el objeto en cuestión sea accesible; no hay restricciones de acceso. Los miembros públicos de una clase constituyen la interfaz pública de los objetos de esa clase.

```
class Test
{
    public static void Main(string[] args)
    {
        Círculo c = new Círculo(100.0, 200.0, 10.0);
        double área = c.AreaCírculo(); // correcto, miembro público
    }
}
```

Acceso privado

Un miembro de una clase declarado **private** (privado) puede ser accedido por un objeto de esa clase sólo desde los métodos y propiedades de dicha clase (vea más adelante la referencia implícita **this**). Esto significa que no puede ser accedido desde los métodos y propiedades de cualquier otra clase, incluidas las subclases.

```
public class Test
{
    public static void Main(string[] args)
    {
        Círculo c = new Círculo(100.0, 200.0, 10.0);
        double r = c.radio; // error: miembro privado
    }
}
```

Un miembro de una clase declarado sin modificadores que indiquen el tipo de acceso al mismo, se supone que es privado (**private**).

Acceso protegido

Un miembro de una clase declarado **protected** (protegido) se comporta exactamente igual que uno privado para los métodos y propiedades de cualquier otra clase, excepto para los métodos y propiedades de sus subclases, para las que se comporta como un miembro público.

Acceso interno

Un miembro de una clase declarado **internal** (interno) puede ser accedido por un objeto de esa clase en cualquier parte de la aplicación actual (ensamblado actual) donde el objeto en cuestión sea accesible. Por ejemplo:

```
class CRacional
{
    private int numerador;
    private int denominador;
    internal void AsignarDatos(int num, int den)
    {
        numerador = num;
        if (den == 0) den = 1; // el denominador no puede ser cero
        denominador = den;
    }
    internal void VisualizarRacional()
    {
        Console.WriteLine(numerador + "/" + denominador);
    }
}
```

IMPLEMENTACIÓN DE UNA CLASE

La programación orientada a objetos sugiere separar la implementación de cada clase en un fichero .cs, fundamentalmente para reutilizar y mantener dicha clase posteriormente con facilidad. Como ejemplo, diseñaremos una clase que almacene una fecha, verificando que es correcta; esto es, que el día esté entre los límites 1 y días del mes, que el mes esté entre los límites 1 y 12 y que el año sea mayor o igual que 1582 (año gregoriano).

Parece lógico que la estructura de datos de un objeto fecha esté formada por los atributos *día*, *mes* y *año*, y permanezca oculta al usuario. Por otra parte, las operaciones sobre estos objetos tendrán que permitir asignar una fecha, método *AsignarFecha*, obtener una fecha de un objeto existente, método *ObtenerFecha*, y verificar si la fecha que se quiere asignar es correcta, método *FechaCorrecta*. Estos tres métodos formarán la interfaz pública. Cuando el día corresponda al mes de febrero, el método *FechaCorrecta* necesitará comprobar si el año es bisiesto, para lo que añadiremos el método *Bisiesto*. Ya que un usuario no necesita acceder a este método, lo declararemos protegido con la intención de que, en un futuro, sí pueda ser accedido desde una subclase. Según lo expuesto, podemos escribir una clase denominada *CFecha* así:

```
public class CFecha
{
    // Atributos
    private int día, mes, año;

    // Métodos
    protected bool Bisiesto()
    {
        // cuerpo del método
    }
    public void AsignarFecha(int dd, int mm, int aaaa)
    {
        // cuerpo del método
    }
    public void ObtenerFecha(int[] fecha)
    {
        // cuerpo del método
    }
    public bool FechaCorrecta()
    {
        // cuerpo del método
    }
}
```

El paso siguiente es definir cada uno de los métodos. Al hablar de los modificadores de acceso quedó claro que cada uno de los métodos de una clase tiene ac-

ceso directo al resto de los miembros. Según esto, la definición del método *AsignarFecha* puede escribirse así:

```
public void AsignarFecha(int dd, int mm, int aaaa)
{
    día = dd; mes = mm; año = aaaa;
}
```

Observe que, por ser *AsignarFecha* un método de la clase *CFecha*, puede acceder directamente a los atributos *día*, *mes* y *año* de su misma clase, independientemente de que sean privados. Estos atributos corresponderán en cada caso al objeto que recibe el mensaje *AsignarFecha* (objeto para el que se invoca el método; vea más adelante, en este mismo capítulo, la referencia implícita **this**). Por ejemplo, si declaramos los objetos *fecha1* y *fecha2* de la clase *CFecha*, y enviamos a *fecha1* el mensaje *AsignarFecha*,

```
fecha1.AsignarFecha(dd, mm, aaaa);
```

como respuesta a este mensaje, se ejecuta el método *AsignarFecha* que asigna los datos *dd*, *mm* y *aaaa* al objeto *fecha1*; esto es, a *fecha1.día*, *fecha1.mes* y *fecha1.año*; y si a *fecha2* le enviamos también el mensaje *AsignarFecha*:

```
fecha2.AsignarFecha(dd, mm, aaaa);
```

como respuesta a este mensaje, se ejecuta el método *AsignarFecha* que asigna los datos *dd*, *mm* y *aaaa* al objeto *fecha2*; esto es, a *fecha2.día*, *fecha2.mes* y *fecha2.año*.

Siguiendo las reglas enunciadas, finalizaremos el diseño de la clase escribiendo el resto de los métodos. El resultado que se obtendrá será la clase *CFecha* que se observa a continuación:

```
////////////////////////////////////
// Definición de la clase CFecha
public class CFecha
{
    // Atributos
    private int día, mes, año;

    // Métodos
    protected bool Bisiesto()
    {
        return ((año % 4 == 0) && (año % 100 != 0) || (año % 400 == 0));
    }

    public void AsignarFecha(int dd, int mm, int aaaa)
    {
```

```
        día = dd; mes = mm; año = aaaa;
    }

    public void ObtenerFecha(out int dd, out int mm, out int aaaa)
    {
        dd = día;
        mm = mes;
        aaaa = año;
    }

    public bool FechaCorrecta()
    {
        bool díaCorrecto, mesCorrecto, añoCorrecto;

        // ¿año correcto?
        añoCorrecto = (año >= 1582);

        // ¿mes correcto?
        mesCorrecto = (mes >= 1) && (mes <= 12);

        switch (mes)
        {
            // ¿día correcto?
            {
                case 2:
                    if (Bisiesto())
                        díaCorrecto = (día >= 1 && día <= 29);
                    else
                        díaCorrecto = (día >= 1 && día <= 28);
                    break;
                case 4: case 6: case 9: case 11:
                    díaCorrecto = (día >= 1 && día <= 30);
                    break;
                default:
                    díaCorrecto = (día >= 1 && día <= 31);
                    break;
            }
        }
        return díaCorrecto && mesCorrecto && añoCorrecto;
    }
}
```

Resumiendo: la funcionalidad de esta clase está soportada por los atributos privados *día*, *mes* y *año*, y por los métodos *AsignarFecha*, *ObtenerFecha*, *FechaCorrecta* y *Bisiesto*.

El método público *AsignarFecha* recibe tres enteros y los almacena en los atributos *día*, *mes* y *año* del objeto que recibe el mensaje *AsignarFecha* (objeto para el que se invoca dicho método).

El método público *ObtenerFecha* permite extraer los datos *día*, *mes* y *año* del objeto que recibe el mensaje *ObtenerFecha*.

El método público *FechaCorrecta* verifica si la fecha que se desea asignar al objeto que recibe este mensaje es correcta. Este método devuelve **true** si la fecha es correcta y **false** en caso contrario.

El método protegido *Bisiesto* verifica si el año de la fecha que se desea asignar al objeto que recibe este mensaje es bisiesto. Este método retorna **true** si el año es bisiesto y **false** en caso contrario.

MÉTODOS SOBRECARGADOS

En los capítulos anteriores, al trabajar con las clases de la biblioteca de .NET nos hemos encontrado con algunas que implementan varias veces un mismo método. Por ejemplo, en el capítulo 6 vimos que la clase **Console** implementa varias formas del método **WriteLine**. Por ejemplo:

```
public static void WriteLine()
public static void WriteLine(Double)
public static void WriteLine(String)
public static void WriteLine(String, Object)
```

¿En qué se diferencian estas implementaciones de los métodos **WriteLine**? Pues vemos que se diferencian en el tipo y/o en el número de sus parámetros.

Pues bien, cuando en una clase un mismo método se define varias veces con distinto número de parámetros, o bien con el mismo número de parámetros pero diferenciándose una definición de otra en que al menos un parámetro es de un tipo diferente, se dice que el método está *sobrecargado*.

Los métodos sobrecargados pueden diferir también en el tipo del valor retornado. Ahora bien, el compilador C# no admite que se declaren dos métodos que sólo difieran en el tipo del valor retornado; deben diferir también en la lista de parámetros; esto es, lo que importa son el número y el tipo de los parámetros.

La sobrecarga de métodos elimina la necesidad de nombrar de forma diferente métodos que en esencia hacen lo mismo, como es el caso del método **WriteLine**, o también hace posible que un método se comporte de una u otra forma según el número de argumentos con el que sea invocado.

Como ejemplo, sobrecargaremos el método *AsignarFecha* para que pueda ser invocado con cero argumentos; con un argumento, el día; con dos argumentos, el día y el mes; y con tres argumentos, el día, el mes y el año. Los datos día, mes o

año omitidos en cualquiera de los casos serán obtenidos de la fecha actual proporcionada por el sistema.

La fecha actual del sistema se puede obtener a partir de un objeto de la clase **DateTime** del espacio de nombres **System**. Esta clase proporciona una serie de propiedades tales como **Day**, **Month** o **Year** que podemos utilizar para obtener el dato al que alude.

```
public void AsignarFecha()
{
    // Asignar, por omisión, la fecha actual.
    DateTime fechaActual = DateTime.Now;
    día = fechaActual.Day;
    mes = fechaActual.Month;
    año = fechaActual.Year;
}

public void AsignarFecha(int dd)
{
    AsignarFecha();
    día = dd;
}

public void AsignarFecha(int dd, int mm)
{
    AsignarFecha();
    día = dd; mes = mm;
}

public void AsignarFecha(int dd, int mm, int aaaa)
{
    día = dd; mes = mm; año = aaaa;
}
```

Como se puede observar, el que una definición del método invoque a otra es una técnica de método abreviado que da como resultado métodos más cortos.

Por cada llamada al método *AsignarFecha* que escribamos en un programa, el compilador C# debe resolver cuál de los métodos con el nombre *AsignarFecha* es invocado. Esto lo hace comparando el número y tipos de los argumentos especificados en la llamada, con los parámetros especificados en las distintas definiciones del método. El siguiente ejemplo muestra las posibles formas de invocar al método *AsignarFecha*:

```
fecha.AsignarFecha();
fecha.AsignarFecha(día);
fecha.AsignarFecha(día, mes);
fecha.AsignarFecha(día, mes, año);
```

Si el compilador C# no encontrara un método exactamente con los mismos tipos de argumentos especificados en la llamada, realizaría sobre dichos argumentos las conversiones implícitas permitidas entre tipos, tratando de adaptarlos a alguna de las definiciones existentes del método. Si este intento fracasa, entonces se producirá un error.

NÚMERO VARIABLE DE PARÁMETROS

En el capítulo anterior, vimos que C# soporta métodos con un número variable de parámetros, todos del mismo tipo. Esta característica se especifica anteponiendo la palabra clave **params** a la definición del parámetro del método, el cual debe ser una matriz unidimensional. Por ejemplo, todas las sobrecargas del método *AsignarFecha* podrían ser sustituidas por esta otra versión:

```
public void AsignarFecha(params int[] fecha)
{
    // Obtener la fecha actual.
    int dd, mm, aaaa;
    DateTime fechaActual = DateTime.Now;
    dd = fechaActual.Day;
    mm = fechaActual.Month;
    aaaa = fechaActual.Year;

    // Asignar la fecha pasada (por omisión la fecha actual)
    if (fecha.Length > 0) día = fecha[0]; else día = dd;
    if (fecha.Length > 1) mes = fecha[1]; else mes = mm;
    if (fecha.Length > 2) año = fecha[2]; else año = aaaa;
}
```

Esta versión de *AsignarFecha* permite operar un número indeterminado de enteros. La palabra clave **params** seguida del tipo **int[]** indica que se puede pasar cualquier número de datos de tipo **int**. Por ejemplo, cualquiera de las llamadas siguientes sería correcta:

```
fecha.AsignarFecha();
fecha.AsignarFecha(día);
fecha.AsignarFecha(día, mes);
fecha.AsignarFecha(día, mes, año);
```

También, en el capítulo anterior, vimos que C# soporta métodos con parámetros con valor por omisión y argumentos con nombre. Como ejercicio, sustituya el método anterior por otro con parámetros con valor por omisión.

IMPLEMENTACIÓN DE UNA APLICACIÓN

Una aplicación consiste en una o más clases, de las cuales una de ellas tiene que ser una clase aplicación: clase que incluya el método **Main**. Cuando compile las clases que contiene su aplicación, el compilador C# generará un fichero .exe denominado *emsamblado*.

Para comprobar que la clase *CFecha* que acabamos de diseñar trabaja correctamente, podemos escribir una aplicación *Test* según se muestra a continuación:

```
using System;
using MisClases.ES;

////////////////////////////////////
// Aplicación que utiliza la clase CFecha y Leer. Copiar los
// ficheros CFecha.cs y Leer.cs en la carpeta de esta aplicación.
//
public class Test
{
    // Leer una fecha
    public static void LeerFecha(ref int d, ref int m, ref int a)
    {
        Console.Write("día, ##    : "); d = Leer.datoInt();
        Console.Write("mes, ##    : "); m = Leer.datoInt();
        Console.Write("año, ####  : "); a = Leer.datoInt();
    }

    // Visualizar una fecha
    public static void VisualizarFecha(CFecha fecha)
    {
        int d, m, a;

        fecha.ObtenerFecha(out d, out m, out a);
        Console.WriteLine(d + "/" + m + "/" + a);
    }

    // Establecer una fecha, verificarla y visualizarla
    public static void Main(string[] args)
    {
        CFecha fecha = new CFecha(); // objeto de tipo CFecha
        int día = 1, mes = 1, año = 2001;

        do
        {
            LeerFecha(ref día, ref mes, ref año);
            fecha.AsignarFecha(día, mes, año);
        }
        while (!fecha.FechaCorrecta());
    }
}
```

```

        VisualizarFecha(fecha);
    }
}

```

Notar que la clase *CFecha* declara los atributos *día*, *mes* y *año* privados y el método *Bisiesto* protegido. Esto quiere decir que sólo son accesibles por los métodos de su clase. Si un método de otra clase intenta acceder a uno de estos atributos, el compilador genera un error. Por ejemplo:

```

public static void Main(string[] args)
{
    CFecha fecha = new CFecha();
    // ...

    int dd = fecha.día; // error: día es un miembro privado
    fecha.mes = 1;      // error: mes es un miembro privado
    bool esBisiesto = fecha.Bisiesto(); // error: método protegido
}

```

En cambio, los métodos *AsignarFecha*, *ObtenerFecha* y *FechaCorrecta* son públicos. Por lo tanto, son accesibles, además de por los métodos de su clase, por cualquier otro método de otra clase. Sirva como ejemplo el método *VisualizarFecha* de la clase *Test*. Este método presenta en la salida estándar la fecha almacenada en el objeto que se le pasa como argumento. Observe que tiene que invocar al método *ObtenerFecha* para acceder a los datos de un objeto *CFecha*. Esto es así porque un método que no es miembro de la clase del objeto no tiene acceso a sus datos privados.

El método *LeerFecha* de *Test* obtiene de la entrada estándar la fecha que se desea almacenar en un objeto a través del método *AsignarFecha*.

CONTROL DE ACCESO A UNA CLASE

El control de acceso a una clase determina la relación que tiene esa clase con otras clases de otros espacios de nombres. Distinguimos dos niveles de acceso: interno (**internal**) y público (**public**); no se puede declarar explícitamente como **private**, **protected** o **protected internal**. Una clase con nivel de acceso **internal** significa que sólo se pueden crear objetos de esta clase desde dentro de los archivos del mismo ensamblado. En cambio, una clase con nivel de acceso público significa que cualquiera puede crear objetos de esta clase.

Por omisión, una clase tiene el nivel de acceso interno; por ejemplo, la clase *Círculo* expuesta anteriormente tiene este nivel de acceso (no ha sido declarada **public**, por lo que tiene el nivel de acceso **internal**). En cambio, cuando se desea que una clase tenga nivel de acceso público, hay que calificarla como tal utilizan-

do la palabra reservada **public**; la clase *CFecha* del ejemplo anterior tiene este nivel de acceso. Otro ejemplo: la clase *Leer* utilizada desde la clase *Test* anterior es pública; pero aunque no hubiese sido pública también se podría utilizar desde la clase *Test*, ya que ambas pertenecen al mismo ensamblado.

REFERENCIA **this**

Cada objeto mantiene su propia copia de los atributos pero no de los métodos de su clase, de los cuales sólo existe una copia para todos los objetos de esa clase (piense también que las propiedades son métodos especiales). Esto es, cada objeto almacena sus propios datos, pero para acceder y operar con ellos, todos comparten los mismos métodos definidos en su clase. Por lo tanto, para que un método conozca la identidad del objeto particular para el que ha sido invocado, C# proporciona una referencia al objeto denominada **this**. Así, por ejemplo, si creamos un objeto *fecha1* y a continuación le enviamos el mensaje *AsignarFecha*,

```
fecha1.AsignarFecha(día, mes, año);
```

C# define la referencia **this** de sólo lectura para permitir referirse al objeto *fecha1* en el cuerpo del método que se ejecuta como respuesta al mensaje. Esa definición sería equivalente a la siguiente:

```
readonly CFecha this = fecha1;
```

Y cuando realizamos la misma operación con otro objeto *fecha2*,

```
fecha2.AsignarFecha(día, mes, año);
```

C# define la referencia **this**, para referirse al objeto *fecha2*, de una forma equivalente a la línea siguiente:

```
readonly CFecha this = fecha2;
```

La palabra clave **readonly** es diferente de la palabra clave **const**. Un atributo **const** sólo puede iniciarse en la declaración del mismo, proceso que se realiza durante la compilación. En cambio, un atributo **readonly** puede iniciarse en la declaración o en un constructor; en este último caso, la iniciación ocurrirá durante la ejecución. Esto significa que los atributos **readonly** pueden tener diferentes valores en función del constructor que se utilice.

Según lo expuesto, el método *AsignarFecha* podría ser definido también como se muestra a continuación:


```
public void AsignarFecha(int dd, int mm, int aa)
{
    this.día = dd; this.mes = mm; this.año = aa;
}
```

¿Qué representa **this** en este método? Según lo explicado, **this** es una referencia al objeto que recibió el mensaje *AsignarFecha*; esto es, al objeto sobre el que se está realizando el proceso llevado a cabo por el método *AsignarFecha*.

Observe ahora el método **Main** de la clase *Test* presentada anteriormente. En él hemos declarado un objeto *fecha* de la clase *CFecha* y posteriormente le hemos enviado un mensaje *FechaCorrecta*:

```
do
{
    LeerFecha(ref día, ref mes, ref año);
    fecha.AsignarFecha(día, mes, año);
}
while (!fecha.FechaCorrecta());
```

En este caso, igual que en el ejemplo anterior, el método *FechaCorrecta* conoce con exactitud el objeto sobre el que tiene que actuar, puesto que se ha expresado explícitamente. Pero, ¿qué pasa con el método *Bisiesto* que se encuentra sin referencia directa alguna en el cuerpo del método *FechaCorrecta*?

```
public bool FechaCorrecta()
{
    // ...
    if (Bisiesto())
    // ...
}
```

En este otro caso, la llamada no es explícita como en el caso anterior. Lo que ocurre en la realidad es que todas las referencias a los atributos y métodos del objeto para el que se invocó el método *FechaCorrecta* (objeto que recibió el mensaje *FechaCorrecta*) son implícitamente realizadas a través de **this**. Según esto, la sentencia **if** anterior podría escribirse también así:

```
if (this.Bisiesto())
```

Normalmente en un método no es necesario utilizar esta referencia para acceder a los miembros del objeto implícito, pero es útil cuando haya que devolver una referencia al mismo.

INICIACIÓN DE UN OBJETO

Sabemos que un objeto consta de una estructura interna (los atributos) y de una interfaz que permite acceder y manipular tal estructura (los métodos). Ahora, ¿cómo se construye un objeto de una clase cualquiera? Pues, de forma análoga a como se construye cualquier otra variable de un tipo predefinido. Por ejemplo:

```
int edad;
```

Este ejemplo define la variable *edad* del tipo predefinido **int**. En este caso, el compilador automáticamente reserva memoria para su ubicación, le asigna un valor (cero si se trata de un atributo de una clase, o indeterminado si es local a un método) y procederá a su destrucción, cuando el flujo de ejecución vaya fuera del ámbito donde haya sido definida.

Esto nos hace pensar en la idea de que de alguna manera el compilador llama a un método de iniciación, *constructor*, para iniciar cada una de las variables declaradas, y a un método de eliminación, *destructor*, para liberar el espacio ocupado por dichas variables, justo al salir del ámbito en el que han sido definidas.

Pues bien, con un objeto de una clase ocurre lo mismo. Por ejemplo:

```
CFecha fecha = new CFecha();
```

Con objetos, el compilador proporciona un *constructor* público, por omisión, para cada clase definida (se entiende, por omisión del que escribe la clase; si éste no escribe un constructor, lo aporta el compilador). Este constructor será ejecutado después que el operador **new**, secuencial y recursivamente (un miembro de una clase puede ser iniciado con un objeto de otra clase), reserve memoria para cada uno de los miembros y los inicie. Igualmente, el compilador proporciona para cada clase de objetos un *destructor*, por omisión, que será invocado justo antes de que se destruya un objeto con el fin de permitir realizar tareas de limpieza y liberar recursos.

¿Cómo implementa C# el constructor y el destructor por omisión en una clase? Veámoslo en la clase *CFecha*:

```
public class CFecha
{
    // Atributos
    private int día, mes, año;

    // Métodos
    public CFecha() {} // constructor por omisión
    ~CFecha() {}      // destructor por omisión
}
```

```
// ...
}
```

No obstante, como veremos a continuación, cuando el constructor proporcionado por omisión por C# no satisfaga las necesidades de nuestra clase de objetos, podemos definir uno. Ídem para el destructor; éste es invocado automáticamente por la máquina virtual de .NET siempre que el objeto quede desreferenciado, pero lo que no podemos predecir es el instante en el que esto ocurre.

Constructor

En C#, una forma de asegurar que los objetos siempre contengan valores válidos es escribir un constructor. Un *constructor* es un método especial de una clase que es llamado automáticamente siempre que se crea un objeto de la misma. Su función es iniciar los nuevos objetos de su clase. Cuando se crea un objeto, C# hace lo siguiente:

- Asigna memoria para el objeto por medio del operador **new**.
- Inicia los atributos de ese objeto, ya sea con los valores especificados en su propia declaración, o en su defecto, con los valores predeterminados por el sistema: los atributos numéricos a ceros, los alfanuméricos a nulos y las referencias a objetos a **null**.
- Llama al constructor de la clase que puede ser uno entre varios, según se expone a continuación.

Dado que los constructores son métodos, admiten parámetros igual que éstos. Cuando en una clase no especificamos ningún constructor, el compilador añade uno público por omisión sin parámetros, según se vio en el apartado anterior.

```
public CFecha() { /* Sin código */ }
```

Un *constructor por omisión* de una clase *C* es un constructor sin parámetros que no hace nada. Sin embargo, es necesario porque según lo que acabamos de exponer, será invocado cada vez que se construya un objeto sin especificar ningún argumento, en cuyo caso el objeto será iniciado con los valores especificados en la declaración de los atributos en la clase, o en su defecto, con los valores predeterminados por el sistema.

Un *constructor* se distingue fácilmente porque tiene el mismo nombre que la clase a la que pertenece (por ejemplo, el constructor para la clase *CFecha* se denomina también *CFecha*), no se hereda, no puede retornar un valor (incluyendo **void**) y puede ser declarado como **public**, **private**, **protected**, **internal** o **protected internal**.

Como ejemplo, vamos a añadir un constructor a la clase *CFecha* con el fin de poder iniciar los atributos de cada nuevo objeto con unos valores determinados:

```
public class CFecha
{
    // Atributos
    private int día, mes, año;

    // Métodos
    public CFecha(int dd, int mm, int aaaa) // constructor
    {
        día = dd; mes = mm; año = aaaa;
        if (!FechaCorrecta())
        {
            Console.WriteLine("Fecha incorrecta. Se asigna la actual.");
            AsignarFecha();
        }
    }
    // ...
}
```

Observe que el constructor, salvo en casos excepcionales, debe declararse siempre público para que pueda ser invocado desde cualquier parte de una aplicación donde se cree un objeto de su clase, aunque la clase, que se supone pública, pertenezca a otro espacio de nombres.

Cuando una clase tiene un constructor, éste será invocado automáticamente siempre que se cree un nuevo objeto de esa clase. El objeto se considera construido con los valores predeterminados justo antes de iniciarse la ejecución del constructor. Por lo tanto, a continuación, desde el cuerpo del constructor según se puede observar en el ejemplo anterior, es posible asignar valores a sus atributos, invocar a los métodos de su clase, o bien llamar a métodos de otros objetos.

En el caso de que el constructor tenga parámetros, para crear un nuevo objeto hay que especificar la lista de argumentos correspondiente entre los paréntesis que siguen al nombre de la clase del objeto. El siguiente ejemplo muestra esto con claridad:

```
public class Test
{
    // Visualizar una fecha
    public static void VisualizarFecha(CFecha fecha)
    {
        int d, m, a;
        fecha.ObtenerFecha(out d, out m, out a);
        Console.WriteLine(d + "/" + m + "/" + a);
    }
}
```

```

public static void Main(string[] args)
{
    // La siguiente línea invoca al constructor de la clase CFecha
    CFecha fecha = new CFecha(1, 3, 2015); // objeto de tipo CFecha
    VisualizarFecha( fecha );
}
}

```

Este ejemplo define un objeto *fecha* e inicia sus atributos *día*, *mes* y *año* con los valores 1, 3 y 2015, respectivamente. Para ello, invoca al constructor *CFecha(int dd, int mm, int aaaa)*, le pasa los argumentos 1, 3 y 2015 y ejecuta el código que se especifica en el cuerpo del mismo. Una vez construido el objeto, visualizamos su contenido invocando al método *VisualizarFecha*, que a su vez invoca al método *ObtenerFecha* de dicho objeto para obtener sus atributos. La siguiente línea es la salida del ejemplo anterior:

```
1/3/2015
```

Añadamos ahora al método **Main** de la clase *Test* del ejemplo anterior la línea de código que se indica a continuación. ¿Qué ocurrirá?

```
CFecha otraFecha = new CFecha();
```

Quizás se sorprenda cuando el compilador C# le indique que la clase *CFecha* no tiene ningún constructor sin parámetros, cuando anteriormente habíamos dicho que C# proporciona para toda clase uno. Lo que sucede es que siempre que en una clase se define explícitamente un constructor, el constructor implícito (constructor por omisión) es reemplazado por éste.

Según lo expuesto, la definición explícita del constructor con parámetros *CFecha(int dd, int mm, int aaaa)* ha sustituido al constructor por omisión que C# debería añadir a esa clase. Para solucionar este problema, hay que añadir a la clase un constructor sin parámetros. Por ejemplo, el siguiente:

```
public CFecha() { /* Sin código */ }
```

El constructor anterior realiza la misma función que el constructor por omisión. No obstante, en el caso de la clase *CFecha*, quizás sea más conveniente, añadir un constructor sin parámetros que inicie cada objeto creado con los valores correspondientes a la fecha actual:

```

public CFecha() // constructor
{
    AsignarFecha(); // asignar fecha actual
}

```

Sobrecarga del constructor

Según lo expuesto, es evidente que podemos definir múltiples constructores con el mismo nombre y diferentes parámetros con el fin de poder iniciar un objeto de una clase de diferentes formas. Esto no es nuevo, simplemente es aplicar la técnica de sobrecargar un método, expuesta anteriormente, al constructor de una clase.

Por ejemplo, aplicando lo expuesto, podemos añadir a la clase *CFecha* constructores para iniciar un objeto, por omisión con la fecha actual proporcionada por el método *AsignarFecha* sin parámetros, o bien especificando sólo el día, o el día y el mes, o el día, el mes y el año; los valores no especificados se obtendrán de la fecha actual del sistema proporcionada por *AsignarFecha*. El código siguiente muestra las distintas sobrecargas que satisfacen lo anteriormente expuesto:

```
public CFecha() // constructor sin parámetros
{
    AsignarFecha(); // asignar fecha actual
}

public CFecha(int dd) // constructor con un parámetro
{
    AsignarFecha(); // asignar fecha actual

    día = dd;
    if (!FechaCorrecta())
    {
        Console.WriteLine("Fecha incorrecta. Se asigna la actual.");
        AsignarFecha();
    }
}

public CFecha(int dd, int mm) // constructor con dos parámetros
{
    AsignarFecha(); // asignar fecha actual

    día = dd; mes = mm;
    if (!FechaCorrecta())
    {
        Console.WriteLine("Fecha incorrecta. Se asigna la actual.");
        AsignarFecha();
    }
}

public CFecha(int dd, int mm, int aaaa) // construc. con tres pars.
{
    día = dd; mes = mm; año = aaaa;
    if (!FechaCorrecta())
    {
```

```

        Console.WriteLine("Fecha incorrecta. Se asigna la actual.");
        AsignarFecha();
    }
}

```

Ahora, podemos invocar al constructor *CFecha* con 0, 1, 2 o 3 argumentos, según se puede observar en las líneas de código siguientes:

```

CFecha fecha1 = new CFecha();
CFecha fecha2 = new CFecha(3);
CFecha fecha3 = new CFecha(15, 3);
CFecha fecha4 = new CFecha(1, 3, 2015);

```

Para el caso concreto que acabamos de exponer, podemos sustituir todas las sobrecargas por un único constructor con un número de parámetros variable:

```

public CFecha(params int[] fecha)
{
    AsignarFecha(); // asignar fecha actual

    if (fecha.Length > 0) día = fecha[0];
    if (fecha.Length > 1) mes = fecha[1];
    if (fecha.Length > 2) año = fecha[2];

    if (!FechaCorrecta())
    {
        Console.WriteLine("Fecha incorrecta. Se asigna la actual.");
        AsignarFecha();
    }
}

```

De esta forma, con un solo constructor quedan resueltas todas las posibilidades que hay para invocar al constructor *CFecha*.

Llamar a un constructor

A diferencia de los otros métodos de la clase, un constructor no puede ser invocado directamente, pero sí indirectamente a través de **this**. Para llamar a un constructor en la clase actual desde otro constructor utilice la siguiente sintaxis:

```

public nombre_constructor(argumentos) : this(argumentos) { }

```

El constructor invocado será aquél cuyos parámetros coincidan con los especificados a continuación de **this**. Por ejemplo, el constructor de la clase *CFecha* con un parámetro podría escribirse también así:

```
public CFecha(int dd) // constructor
: this() // invoca al constructor CFecha sin parámetros
{
    día = dd;

    if (!FechaCorrecta())
    {
        Console.WriteLine("Fecha incorrecta. Se asigna la actual.");
        AsignarFecha();
    }
}
```

Asignación de objetos

No olvide que cuando trabaja con objetos lo que realmente manipula desde cualquier método son referencias a los objetos. Por ejemplo:

```
CFecha fecha1 = new CFecha();
CFecha fecha2 = new CFecha(15);
CFecha fecha3 = new CFecha(22, 3);
CFecha fecha4 = fecha1;
fecha3 = fecha2;
```

Este ejemplo crea tres objetos: *fecha1*, *fecha2* y *fecha3*. Después declara una nueva referencia *fecha4* y le asigna *fecha1*, pero tanto *fecha1* como *fecha4* son referencias a objetos *CFecha*, que ahora apuntan al mismo objeto (al referenciado por *fecha1*). Finalmente, asigna *fecha2* a *fecha3*, con lo que ambas referencias apuntarán al objeto referenciado por *fecha2*.

Lo anteriormente expuesto demuestra que el operador de asignación no sirve para copiar un objeto en otro. ¿Cuál es la solución para resolver el problema planteado? Pues, añadir a la clase *CFecha* un método como el siguiente:

```
public void Copiar(CFecha obj)
{
    día = obj.día;
    mes = obj.mes;
    año = obj.año;
}
```

Obsérvese que el método *Copiar* tiene un parámetro que define el objeto a copiar; evidentemente, se trata de un objeto de su misma clase. El método *Copiar* copia miembro a miembro el objeto pasado como argumento en el objeto que recibe el mensaje *Copiar*. Por ejemplo, la siguiente línea de código copia el objeto *fecha2* en el objeto *fecha1*:

```
fecha1.Copiar(fecha2);
```


Si ahora quisiéramos copiar el objeto *fecha3* en el objeto *fecha2* y en el objeto *fecha1*, podríamos proceder, por ejemplo, así:

```
fecha2.Copiar(fecha3);
fecha1.Copiar(fecha2);
```

Pero, ¿qué podemos hacer para poder escribir las dos líneas anteriores en una sola? Esto es, para poder escribir:

```
fecha1.Copiar(fecha2.Copiar(fecha3));
```

Tendríamos que modificar el método *Copiar* como se observa a continuación:

```
public CFecha Copiar(CFecha obj)
{
    día = obj.día;
    mes = obj.mes;
    año = obj.año;

    return this;
}
```

El método *Copiar* devuelve ahora una referencia al objeto resultado de la copia, con lo cual podemos utilizar esta referencia para copiar este objeto en otro; esto es, el hecho de que el método *Copiar* retorne una referencia al objeto resultante permite realizar copias múltiples encadenadas.

Constructor copia

Otra forma de iniciar un objeto es asignándole otro objeto de su misma clase en el momento de su creación. Lógicamente, si se crea un objeto tiene que intervenir un constructor, que recibirá como único argumento el objeto con el que se iniciará. El prototipo para este constructor es de la forma:

modificadores nombre_clase(nombre_clase referencia_objeto)

Se puede observar que un constructor de las características especificadas tiene un solo parámetro, que es una referencia a un objeto de su misma clase. Por tratarse de un constructor no hay un valor retornado. Pues bien, un constructor que se invoca para iniciar un nuevo objeto creado a partir de otro existente es denominado *constructor copia*.

Como ejemplo, añada un constructor copia a la clase *CFecha*. Éste será como se indica a continuación:

```
public CFecha(CFecha obj) // constructor copia
{
    día = obj.día;
    mes = obj.mes;
    año = obj.año;
}
```

Vemos que el constructor copia acepta como argumento una referencia al objeto a copiar y, después, asigna miembro a miembro ese objeto al nuevo objeto construido. Para probar cómo trabaja, puede añadir al método **Main** de la clase **Test** que escribimos anteriormente las siguientes líneas de código:

```
CFecha fecha1 = new CFecha(1, 3, 2015);
CFecha fecha2 = new CFecha(fecha1);
```

Este ejemplo crea e inicia un objeto *fecha1* y a continuación crea otro objeto *fecha2* iniciándole con *fecha1*. A diferencia del método *Copiar* expuesto en el apartado anterior, inicialmente aquí sólo existe un objeto (*fecha1*); después se crea otro objeto (*fecha2*) y se inicia con el primero.

Si analizamos las operaciones que realizan el método *Copiar* y el constructor copia, llegaremos a la conclusión de que son las mismas, excepto en que el constructor copia no retorna nada. Basándonos en este hecho, otra forma de escribir el constructor copia sería invocando al método *Copiar*, forma que resulta especialmente útil cuando escribimos ambos métodos:

```
public CFecha(CFecha obj) // constructor copia
{
    Copiar(obj);
}
```

DESTRUCCIÓN DE OBJETOS

De la misma forma que existe un método que se ejecuta automáticamente cada vez que se construye un objeto, también existe un método que se invoca automáticamente cada vez que se destruye. Este método recibe el nombre de *destructor*.

Cuando un objeto es destruido ocurren varias cosas: se llama al destructor y después, el recolector de basura se encarga de eliminar el objeto, lo que conlleva liberar los recursos que dicho objeto tenga adjudicados, como por ejemplo, la memoria que ocupa.

Un objeto es destruido automáticamente cuando se eliminan todas las referencias al mismo. Una referencia a un objeto puede ser eliminada porque el flujo de

ejecución salga fuera del ámbito donde ella está declarada, o porque explícitamente se le asigne el valor **null**.

Destructor

Un *destructor* es un método especial de una clase que se ejecuta antes de que un objeto de esa clase sea eliminado físicamente de la memoria. Un *destructor* se distingue fácilmente porque tiene el mismo nombre que la clase a la que pertenece precedido por una `~`. Un *destructor* no se hereda ni puede ser sobrecargado, no se puede invocar explícitamente, se invoca automáticamente, y no permite modificadores de acceso, ni tiene parámetros.

Cuando en una clase no especificamos un destructor, el compilador proporciona uno por omisión cuya sintaxis es la siguiente:

```
~nombre_clase() { /* sin código */ }
```

Por ejemplo, el destructor por omisión para la clase *CFecha* es declarado por el compilador C# así:

```
~CFecha() { /* sin código */ }
```

El destructor llama implícitamente al método **Finalize**. Esto es, el código del destructor anterior se traduce implícitamente a:

```
protected override void Finalize()
{
    try
    {
    }
    finally
    {
        base.Finalize();
    }
}
```

En el cuerpo del destructor puede escribir cualquier operación que quiera realizar relacionada con el objeto que se vaya a destruir. Por ejemplo, cuando se crean objetos que encapsulan recursos no administrados, como por ejemplo un flujo para acceso a un fichero o una conexión de red, es recomendable liberar explícitamente esos recursos cuando la aplicación termine de utilizarlos. Ahora bien, la memoria es un recurso administrado por la máquina de ejecución. Esto quiere decir que la memoria asignada a los objetos que ya no son utilizados es liberada por un recolector de basura proporcionado por .NET Framework; la liberación de memoria no es determinista, se libera cuando la máquina de ejecución decide rea-

lizar la recolección de elementos no utilizados sin necesidad de escribir código adicional.

Resumiendo: un destructor es invocado automáticamente justo antes de que el objeto sea recolectado como basura por el *recolector de basura* de .NET Framework. Y, ¿cuándo ocurre esto? Cuando no queden referencias al objeto.

Como ejemplo vamos a añadir a la clase *CFecha* del programa anterior un destructor para que simplemente nos muestre un mensaje cada vez que se destruya un objeto de esa clase. Esto es:

```
public class CFecha
{
    // ...
    ~CFecha() // destructor
    {
        Console.WriteLine("Objeto destruido");
    }

    // ...
}
```

Ejecute ahora la aplicación *Test* cuyo código se muestra a continuación y observe los resultados.

```
public class Test
{
    // ...

    public static void Main(string[] args)
    {
        CFecha fecha1 = new CFecha(1, 3, 2015);
        // Empieza un bloque de código
        {
            CFecha fecha2 = new CFecha(fecha1);
            VisualizarFecha(fecha2);
        } // fin del bloque

        VisualizarFecha(fecha1);
    }
}
```

Analizando este ejemplo, observamos que en el método **Main** se crean dos objetos: uno al nivel del bloque de **Main**, y otro local a un bloque interno a **Main**. Por lo tanto, cada objeto quedará desreferenciado cuando el flujo de ejecución salga fuera del bloque en el que está definido, instante a partir del cual el recolector de basura puede recolectar esos objetos.

Si una clase tiene miembros que son objetos de otras clases, su destructor se ejecuta antes que los destructores de los objetos miembro. En otras palabras, el orden de destrucción es inverso al orden de construcción.

No se puede controlar cuándo se llama al destructor, porque esto lo determina el recolector de basura. Éste comprueba si hay objetos que ya no están siendo utilizados por ninguna aplicación y por cada uno de ellos, llama al destructor (si existe) y reclama la memoria utilizada para almacenar el objeto. También se llama a los destructores cuando se cierra la aplicación. Por ejemplo, cuando se ejecuta el ejemplo anterior se muestra la siguiente información:

```
1/3/2015
1/3/2015
Objeto destruido
Objeto destruido
```

Obsérvese que cuando finaliza la ejecución del bloque donde está definido *fecha2* no se visualiza el mensaje “Objeto destruido”, cosa que no ocurre hasta que finaliza el programa, lo cual corrobora lo expuesto en el párrafo anterior.

Sentencia using

A diferencia de los recursos administrados que se eliminan sin necesidad de código adicional, para eliminar los recursos no administrados, como ficheros abiertos y conexiones de red, tan pronto como el código termine de usarlos, es necesario código adicional; de esta forma, estos recursos quedarán a disposición de otro código que necesite utilizarlos. En este caso nos podemos ayudar de la sentencia **using**, la cual define un ámbito fuera del cual se invoca automáticamente al método **Dispose** del objeto u objetos que intervienen en la misma. Para asegurar que este método estará implementado en las clases de los objetos que intervienen en la sentencia **using**, éstas deben implementar la interfaz **IDisposable** (más adelante explicaremos el concepto de interfaz), ya que es esta interfaz la que proporciona el método **Dispose** que se va a ejecutar para liberar los recursos del objeto, de lo contrario, el compilador mostrará un error. Por ejemplo:

```
using System;
class ClaseX : IDisposable
{
    public ClaseX(string conexión)
    {
        Console.WriteLine("Conexión realizada.");
    }

    void IDisposable.Dispose()
    {

```

```
        // Escribir aquí el código para liberar la conexión
        Console.WriteLine("Conexión liberada.");
    }
}

class Test
{
    static void Main()
    {
        using (ClaseX c = new ClaseX("cadena_de_conexión"))
        {
            Console.WriteLine("Conexión operativa.");
        }
        Console.WriteLine("Finalizó la sentencia using.");
    }
}
```

Obsérvese que la *ClaseX* implementa la interfaz **IDisposable** con el fin de proporcionar el método **Dispose** que debe liberar los recursos de los objetos de dicha clase. La sentencia **using** define un objeto *c* de la *ClaseX*, local al bloque de dicha sentencia, que suponemos hace uso de un recurso no administrado, recurso que deseamos liberar cuando finalice el bloque de **using**, instante en el que se invoca al método **Dispose**. De una sentencia **using** se sale cuando se alcanza el final de la misma o cuando se produce una excepción. Finalmente, en este ejemplo, el objeto *c*, por ser local, será liberado. El resultado después de ejecutar el ejemplo anterior será el siguiente:

```
Conexión realizada.
Conexión operativa.
Conexión liberada.
Finalizó la sentencia using.
```

Muchas clases de la biblioteca .NET (**FileStream**, **StreamReader**, etc.) implementan la interfaz **IDisposable** y, por lo tanto, el método **Dispose** adecuado para liberar los recursos a los que hacen referencia.

El objeto se puede declarar en la sentencia **using**, como se ha hecho en el ejemplo anterior, o delante de ella, del modo siguiente:

```
ClaseX obj = new ClaseX("conexión");
using (obj)
{
    // Operar con obj
}
```

También, se pueden utilizar varios objetos con una sentencia **using**, pero se deben declarar dentro de la sentencia **using**, como sigue:

```
using (ClaseX obj1 = new ClaseX("conexión1"),
        obj2 = new ClaseX("conexión2"))
{
    // Operar con obj1 y obj2.
}
```

Ejecutar el recolector de basura

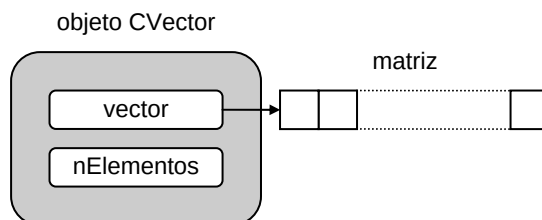
El *recolector de basura* se ejecuta en un subproceso paralelamente a su aplicación limpiando la basura (objetos desreferenciados) en forma silenciosa y en segundo plano y nunca se detiene por más de una pequeña cantidad de tiempo.

Ahora bien, si desea forzar una recolección de basura, puede hacerlo utilizando la funcionalidad aportada por la clase **GC** (*garbage collector*: recolector de basura) del espacio de nombres **System**. Por ejemplo, el método **Collect** de esta clase obliga a que se lleve a cabo la recolección de elementos no utilizados. Sin embargo, esto rara vez será necesario; a lo mejor, si acaba de liberar muchos objetos ya inservibles y quiere que se lleven pronto la basura.

REFERENCIAS COMO MIEMBROS DE UNA CLASE

Un atributo de una clase que sea una referencia requiere, generalmente, de una asignación de memoria, proceso que normalmente realizará el constructor. Sucede entonces que el espacio de memoria asignado es referenciado desde el objeto pero, lógicamente, no pertenece al objeto, lo que puede dar lugar a problemas si no se implementan adecuadamente los métodos que generan un objeto copia de otro de su misma clase. Un ejemplo de este tipo de clases es la clase *Círculo* expuesta al principio de este capítulo; recuerde que tenía un miembro *centro* de la clase *Punto*.

Para ver lo expuesto con detalle, vamos a escribir una clase *CVector* para construir objetos que representen matrices numéricas con un número cualquiera de elementos. Por lo tanto, sería inapropiado definir como miembro privado de la clase *CVector* una matriz con un número fijo de elementos. En su lugar, definiremos una referencia, *vector*, a una matriz de tipo **double**, por ejemplo, para después asignar dinámicamente la cantidad de memoria necesaria para la matriz.



Según lo expuesto, la funcionalidad de la clase *CVector* estará soportada por los atributos:

- *vector*: una referencia a una matriz de valores de tipo **double**.
- *nElementos*: número de elementos de dicha matriz.

```
public class CVector
{
    private double[] vector;
    private int nElementos;

    // ...
}
```

y por los métodos:

- *constructores* para crear un objeto *CVector* con un número de elementos predeterminado, con un número de elementos especificado, a partir de una matriz unidimensional, o bien a partir de otro objeto *CVector*.

El trabajo que tienen que realizar los constructores de la clase *CVector*, dependiendo de los casos, es asignar la memoria necesaria para la matriz de datos e iniciar dicha matriz con ceros (iniciación por omisión), con otra matriz o con otro vector, como podemos ver a continuación:

```
// Crear una matriz con 10 elementos por omisión
public CVector()
{
    nElementos = 10;
    vector = new double[nElementos];
}

// Crear una matriz con ne elementos
public CVector(int ne)
{
    if (ne < 1)
    {
        Console.WriteLine("Nº de elementos no válido: " + ne);
        Console.WriteLine("Se asignan 10 elementos");
        ne = 10;
    }
    nElementos = ne;
    vector = new double[nElementos];
}

// Crear una matriz a partir de otra matriz primitiva
public CVector(double[] m)
{

```



```

nElementos = m.Length;
vector = new double[nElementos];

// Copiar los elementos de la matriz m
for (int i = 0; i < nElementos; ++i)
    vector[i] = m[i];
}

// Constructor copia
public CVector(CVector v)
{
    nElementos = v.nElementos;
    vector = new double[nElementos];

    // Copiar el objeto v
    for (int i = 0; i < nElementos; ++i)
        vector[i] = v.vector[i];
}

```

Obsérvese que el constructor copia no se limita a copiar los atributos del objeto *v* en el objeto referenciado por **this**, porque si hiciera eso (*vector* = *v.vector*) tendríamos una sola matriz referenciada por dos objetos.

- **Copiar**: método que permite asignar un objeto *CVector* a otro. Obsérvese que este método realiza el mismo proceso que el constructor copia; además, retorna una referencia al objeto resultante de la copia.

```

// Copiar un vector en otro
public CVector Copiar(CVector v)
{
    nElementos = v.nElementos;
    vector = new double[nElementos];

    // Copiar el objeto v
    for (int i = 0; i < nElementos; i++)
        vector[i] = v.vector[i];
    return this;
}

```

- **PonerValorEn**: método que permite asignar un dato al elemento especificado de un objeto *CVector*.

```

public void PonerValorEn(int i, double valor)
{
    if (i >= 0 && i < nElementos)
        vector[i] = valor;
    else
        Console.WriteLine("Índice fuera de límites");
}

```

- **ValorEn:** método que devuelve el dato almacenado en el elemento especificado de un objeto *CVector*.

```
public double ValorEn(int i)
{
    if (i >= 0 && i < nElementos)
        return vector[i];
    else
    {
        Console.WriteLine("Índice fuera de límites");
        return Double.NaN;
    }
}
```

- **Longitud:** método que devuelve el número de elementos de un objeto *CVector*.

```
public int Longitud() { return nElementos; }
```

El resultado de encapsular los métodos anteriormente expuestos es la clase *CVector* que se muestra a continuación:

```
using System;

////////////////////////////////////
// Definición de la clase CVector
//
public class CVector
{
    private double[] vector; // matriz vector
    private int nElementos; // número de elementos de la matriz

    // Crear una matriz con 10 elementos por omisión
    public CVector()
    {
        nElementos = 10;
        vector = new double[nElementos];
    }

    // Crear una matriz con ne elementos
    public CVector(int ne)
    {
        if (ne < 1)
        {
            Console.WriteLine("Nº de elementos no válido: " + ne);
            Console.WriteLine("Se asignan 10 elementos");
            ne = 10;
        }
        nElementos = ne;
    }
}
```

```
        vector = new double[nElementos];
    }

    // Crear una matriz a partir de otra matriz primitiva
    public CVector(double[] m)
    {
        nElementos = m.Length;
        vector = new double[nElementos];
        // Copiar los elementos de la matriz m
        for (int i = 0; i < nElementos; ++i)
            vector[i] = m[i];
    }

    // Constructor copia
    public CVector(CVector v)
    {
        nElementos = v.nElementos;
        vector = new double[nElementos];
        // Copiar el objeto v
        for (int i = 0; i < nElementos; ++i)
            vector[i] = v.vector[i];
    }

    // Copiar un vector en otro
    public CVector Copiar(CVector v)
    {
        nElementos = v.nElementos;
        vector = new double[nElementos];
        // Copiar el objeto v
        for (int i = 0; i < nElementos; i++)
            vector[i] = v.vector[i];
        return this;
    }

    public void PonerValorEn(int i, double valor)
    {
        if (i >= 0 && i < nElementos)
            vector[i] = valor;
        else
            Console.WriteLine("Índice fuera de límites");
    }

    public double ValorEn(int i)
    {
        if (i >= 0 && i < nElementos)
            return vector[i];
        else
        {
            Console.WriteLine("Índice fuera de límites");
            return Double.NaN;
        }
    }
}
```

```
    }

    public int Longitud()
    {
        return nElementos;
    }
}
```

El resultado es que cada objeto *CVector* consta de dos bloques de memoria, uno de tamaño fijo que almacena su estructura interna (*vector* y *nElementos*) y otro de longitud variable que almacena los datos (la matriz de tipo **double**).

Para probar la clase expuesta escriba, por ejemplo, la siguiente aplicación:

```
using System;

////////////////////////////////////
// Aplicación que utiliza la clase CVector
//
public class Test
{
    // Visualizar un vector
    public static void VisualizarVector(CVector v)
    {
        int ne = v.Longitud();
        for (int i = 0; i < ne; i++)
            Console.Write(v.ValorEn(i) + " ");
        Console.WriteLine();
    }

    public static void Main(string[] args)
    {
        CVector vector1 = new CVector(5);
        VisualizarVector(vector1);
        CVector vector2 = new CVector();
        for (int i = 0; i < vector2.Longitud(); i++)
            vector2.PonerValorEn(i, (i+1)*10);
        VisualizarVector(vector2);

        CVector vector3 = new CVector(vector2);
        VisualizarVector(vector3);

        double[] x = new double[] { 1, 2, 3, 4, 5, 6, 7 }; // matriz x
        CVector vector4 = new CVector(x);
        VisualizarVector(vector4);

        Console.WriteLine("Fin de la aplicación");
    }
}
```

Analizando a grandes rasgos el código presentado anteriormente, podemos ver que la línea:

```
CVector vector1 = new CVector(5);
```

llama al constructor *CVector(int ne)* y crea un objeto *vector1* con cinco elementos. Las líneas:

```
CVector vector2 = new CVector();  
for (int i = 0; i < vector2.Longitud(); i++)  
    vector2.PonerValorEn(i, (i+1)*10);
```

la primera, llama al constructor *CVector* sin argumentos y crea un objeto *vector2* con 10 elementos por omisión. Después asigna valores a cada uno de los elementos de *vector2*. La línea:

```
CVector vector3 = new CVector(vector2);
```

llama al constructor copia y crea un objeto *vector3* iniciado con los datos del objeto *vector2*. Las líneas:

```
double x[] = { 1, 2, 3, 4, 5, 6, 7 }; // matriz x  
CVector vector4 = new CVector(x);
```

la primera define la matriz *x* y la última llama al constructor *CVector(double[] m)* y crea un objeto *vector4* iniciado con los datos de la matriz *x*.

Como se puede observar, cada vez que se crea un objeto es llamado automáticamente un constructor, lo que garantiza la iniciación del objeto. El que se llame a uno o a otro constructor, depende del número y tipo de argumentos especificados.

Cuando el flujo de ejecución sale fuera del ámbito donde ha sido definido un objeto *CVector*, el recolector de basura eliminará tanto el objeto como la matriz referenciada por el mismo, liberando la memoria ocupada.

Sin embargo, una clase con atributos que son referencias a otros objetos, como es *CVector*, potencialmente tiene problemas. Para comprobarlo, suponga que el constructor copia de la clase *CVector* se hubiera escrito así:

```
public CVector(CVector v) // constructor copia  
{  
    nElementos = v.nElementos;  
    vector = v.vector;  
}
```

Suponga también que en la aplicación anterior el método **Main** fuera como sigue:

```
public static void Main(string[] args)
{
    double[] x = { 1, 2, 3, 4, 5, 6, 7 }; // matriz x
    CVector vector1 = new CVector(x);
    VisualizarVector(vector1); // escribe 1 2 3 4 5 6 7

    // El siguiente bloque define vector2
    {
        CVector vector2 = new CVector(vector1);
        for (int i = 0; i < vector2.Longitud(); i++)
            vector2.PonerValorEn(i, vector2.ValorEn(i)*10);
        VisualizarVector(vector2); // escribe 10 20 30 40 50 60 70
    }
    // vector2 ha sido destruido

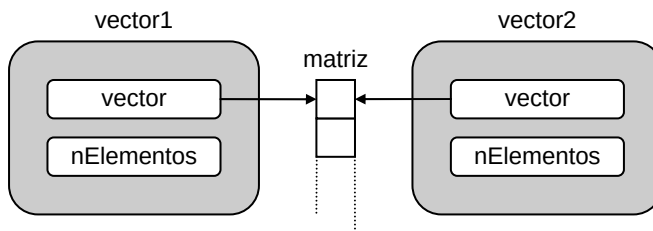
    VisualizarVector(vector1); // escribe 10 20 30 40 50 60 70
    Console.WriteLine("Fin de la aplicación");
}
```

Ahora el método **Main** crea un objeto *vector1* iniciado con los valores de una matriz *x* e incluye un bloque que crea un nuevo objeto *vector2* a partir de *vector1*, para lo cual se invoca al constructor copia.

Observe que ahora este constructor simplemente copia los atributos del objeto *v* en los correspondientes atributos del nuevo objeto creado. Por lo tanto, el resultado de una sentencia como:

```
CVector vector2 = new CVector(vector1);
```

será dos objetos, *vector1* y *vector2*, referenciando la misma matriz. La figura siguiente muestra esto con claridad:



Esto significa que cualquier modificación en uno de los objetos afectará a ambos, justo lo que sucede cuando se ejecuta el código siguiente. Las modificaciones realizadas en el objeto *vector2* afectan de la misma forma a *vector1*:

```
// El siguiente bloque define vector2
{
    CVector vector2 = new CVector(vector1);

    for (int i = 0; i < vector2.Longitud();++i)
        vector2.PonerValorEn(i, vector2.ValorEn(i) * 10);

    VisualizarVector(vector2);
}
```

Piense ahora qué sucederá cuando el flujo de ejecución salga fuera del ámbito de *vector2*. Pues que el objeto *vector2* será enviado a la basura y eliminado por el recolector de basura. ¿Será enviado también a la basura el objeto matriz referenciado por el atributo *vector* de *vector2*? No, porque dicho objeto matriz tiene aún una referencia: *vector* de *vector1*.

Esta misma teoría es aplicable al método *Copiar*. Esto significa que debemos poner un especial interés cuando escribamos métodos que tengan como finalidad duplicar objetos que tienen atributos que son referencias a otros objetos.

REDEFINIR MÉTODOS HEREDADOS DE *Object*

Según vimos en el capítulo anterior, la clase **Object** es la clase raíz de la jerarquía de clases de la biblioteca .NET y de cualquier otra clase que implementemos en nuestras aplicaciones, lo que se traduce en que todas ellas heredan los métodos de **Object**, como **Equals**, **ToString** o **Finalize**, por ejemplo.

¿Cómo han sido implementados estos métodos? Pues de una forma muy genérica, sin pensar en ningún objeto en particular. Por ejemplo, **Equals** proporciona el mismo resultado que el operador “==”; esto es, compara las referencias a los objetos, no sus contenidos, lo cual es lógico: no podemos comparar dos objetos que aún no sabemos cómo son. Ahora bien, una vez diseñada una clase como *CVector*, si necesitamos que el método **Equals** nos diga cómo es un objeto *CVector* con respecto a otro, tenemos que sobrescribir dicho método.

Método **Equals**

Como ejemplo, añada la definición del método **Equals** a la clase *CVector*. Para poder escribir este método, primero debe responder a la siguiente pregunta: ¿cuándo dos objetos *CVector* son iguales? La respuesta es cuando contengan los mismos valores; esto es, cuando las matrices que representan sean idénticas. Basta entonces con que el método **Equals** de la clase *CVector* compare las matrices de los dos objetos a comparar:

```
public bool Equals(CVector v)
{
    if (nElementos != v.nElementos) return false;
    for (int i = 0; i < nElementos; i++)
        if (vector[i] != v.vector[i]) return false;
    return true;
}
```

Para probar los resultados que podemos obtener a partir de este método a diferencia de los obtenidos por el operador “==” escriba la siguiente aplicación:

```
public class Test
{
    // Visualizar un vector
    public static void VisualizarVector(CVector v)
    {
        int ne = v.Longitud();
        for (int i = 0; i < ne; i++)
            Console.Write(v.ValorEn(i) + " ");
        Console.WriteLine();
    }

    public static void Main(string[] args)
    {
        double[] x = new double[] { 1, 2, 3, 4, 5, 6, 7 }; // matriz x
        CVector vector1 = new CVector(x);
        VisualizarVector(vector1); // escribe 1 2 3 4 5 6 7
        CVector vector2 = new CVector(vector1);
        for (int i = 0; i < vector2.Longitud(); i++)
            vector2.PonerValorEn(i, vector2.ValorEn(i) * 10);
        VisualizarVector(vector2); // escribe 10 20 30 40 50 60 70

        if (vector1 == vector2)
            Console.WriteLine("referencias al mismo objeto");
        else
            Console.WriteLine("referencias a objetos diferentes");

        if (vector1.Equals(vector2))
            Console.WriteLine("objetos iguales");
        else
            Console.WriteLine("objetos diferentes");
    }
}
```

Si ejecuta la aplicación *Test* anterior, obtendrá los siguientes resultados:

```
1 2 3 4 5 6 7
10 20 30 40 50 60 70
referencias a objetos diferentes
objetos diferentes
```


MIEMBROS STATIC DE UNA CLASE

Este tema ya fue introducido en el capítulo 5. Por lo tanto, ahora que ya tiene un mayor conocimiento de la POO, el propósito es abundar en detalles con el fin de dejar suficientemente claro cuál es la utilidad de estos miembros.

En ocasiones se hace necesario disponer de una variable global públicamente accesible que no sea parte de un objeto, pero sí de la clase, o bien de un método que no necesite ser invocado para un objeto en particular. Esta funcionalidad es proporcionada en C# por los miembros estáticos (**static**) de una clase.

Atributos static

La última versión de la clase *CFecha* definía un constructor que asignaba una fecha por omisión si la pasada como argumento no era correcta. No obstante, sería más conveniente disponer de un valor por omisión, compartido por todos los objetos de la clase, que además pudiera ser modificado por el usuario, pudiendo incluso llegar a tomar el valor de la fecha actual. Esto se traduce en un atributo del cual sólo es necesario que exista una única copia que pueda ser utilizada por todos los objetos *CFecha*; esto es, una variable con ámbito global accesible directamente, o bien indirectamente a través de un método de la interfaz. La alternativa que ofrece C# para dar solución al problema planteado es declarar el atributo **static**.

Un atributo **static** no es un atributo específico de un objeto (el *día* sí es un atributo específico de una fecha; cada fecha tiene su día), sino más bien es un atributo de la clase; esto es, un atributo del que sólo hay una copia que comparten todos los objetos de la clase. Por esta razón, un atributo **static** existe y puede ser utilizado aunque no exista ningún objeto de su clase.

Como ejemplo, vamos a asociar un atributo *fechaPorOmission* con la clase, no con cada objeto. El código mostrado a continuación muestra cómo hacerlo:

```
public class CFecha
{
    // Atributos
    private int día, mes, año;
    private static CFecha fechaPorOmission;
    // ...
}
```

Un atributo **static** puede ser calificado como **private**, **protected**, **public**, **internal** y **protected internal**. No se puede especificar **static** junto con **const**, porque las constantes ya son **static** de forma implícita, para que sea una constante en lugar de una variable, pero sí con **readonly**. Así mismo, tiene que ser iniciado in-

dependientemente de que se cree o no un objeto de la clase (la iniciación en un constructor sólo sería adecuada cuando ese atributo no fuera requerido antes de que se creara un primer objeto). Una forma de hacerlo es en la propia definición del atributo. Por ejemplo:

```
private static CFecha fechaPorOmision = new CFecha(1, 1, 2001);
```

Ahora bien, en más de una ocasión tendremos que realizar iniciaciones más complejas que éstas que podemos hacer en la propia declaración. En esos casos será necesario añadir a la clase un *constructor estático* cuya sintaxis es la siguiente:

```
static nombre_clase()
{
    // iniciación de los atributos static de la clase
}
```

Un *constructor estático* no permite modificadores de acceso ni tiene parámetros, se le llama automáticamente para iniciar la clase antes de crear el primer objeto o de hacer referencia a cualquier miembro estático y no puede ser llamado directamente. Su función es iniciar los atributos **static** de su clase.

Como ejemplo, vamos a realizar la iniciación de *fechaPorOmision* a través de un constructor estático según se observa a continuación:

```
public class CFecha
{
    // Atributos
    private int día, mes, año;
    private static CFecha fechaPorOmision;

    static CFecha()
    {
        // Iniciación de los atributos de la clase
        fechaPorOmision = new CFecha(1, 1, 2001);
    }

    // Métodos
    // ...
}
```

C# permite cualquier número de constructores estáticos aunque el compilador finalmente los fusionará en uno sólo en el mismo orden en el que aparezcan en la definición de la clase. Este iniciador se ejecutará solamente una vez: cuando el sistema cargue la clase por primera vez.

A continuación y como ejemplo, podríamos utilizar el atributo *fechaPorOmission* en el constructor de la clase para iniciar un objeto cuando la fecha pasada como argumento se omita o sea incorrecta:

```
public CFecha(params int[] fecha)
{
    AsignarFecha(); // asignar fecha actual

    if (fecha.Length > 0) día = fecha[0];
    if (fecha.Length > 1) mes = fecha[1];
    if (fecha.Length > 2) año = fecha[2];
    if (!FechaCorrecta())
    {
        Console.WriteLine("Fecha incorrecta. Se asigna la predeterminada.");
        Copiar(fechaPorOmission);
    }
}
```

Acceder a los atributos static

En el apartado anterior podemos ver cómo los métodos de la clase *CFecha*, por ejemplo el constructor, pueden acceder directamente al atributo *fechaPorOmission* de la misma, igual que acceden al resto de los atributos. Pero, desde otra clase, ¿cómo podríamos acceder a esa información? Si *fechaPorOmission* fuera un atributo declarado **public**, podríamos acceder a él directamente a través del nombre de la clase. Por ejemplo:

```
public static void Main(string[] args)
{
    CFecha fecha1 = new CFecha(1, 3, 2015);

    if (fecha1.FechaCorrecta())
        VisualizarFecha(fecha1);

    VisualizarFecha(CFecha.fechaPorOmission);
}
```

Se puede observar que *CFecha.fechaPorOmission* se comporta como si se tratara de una variable pública, ya que utilizando esta sintaxis podemos acceder a *fechaPorOmission* desde cualquier otra clase.

Puesto que el atributo *fechaPorOmission* se ha declarado privado y no público, tendremos que añadir a la interfaz de la clase un método público que permita el acceso al mismo y, preferiblemente, que se comporte como el atributo.

Métodos static

Un método declarado **static** carece de la referencia **this** por lo que no puede ser invocado para un objeto de su clase, sino que se invoca en general allí donde se necesite utilizar la operación para la que ha sido escrito. Desde este punto de vista es imposible que un método **static** pueda acceder a un miembro no **static** de su clase; por la misma razón, sí puede acceder a un miembro **static**.

Como ejemplo, vamos a añadir a la clase *CFecha* un método *AsignarFechaPorOmission* estático que permita cambiar la fecha de *fechaPorOmission*.

```
public static void AsignarFechaPorOmission(int dd, int mm, int aaaa)
{
    fechaPorOmission.AsignarFecha(dd, mm, aaaa);
}
```

Un método que se declara **static** sólo puede acceder a los miembros (atributos o métodos) **static** de su clase. Por ejemplo, si en el método anterior intenta establecer el atributo *día* a 1, el compilador le mostrará un error indicándole que no se puede hacer referencia a un atributo no estático desde un método estático, ya que éste carece de la referencia **this**.

En cambio, un miembro **static** sí puede ser accedido por un método independientemente de que sea **static** o no. Por ejemplo, antes hemos visto que el constructor *CFecha* accede al atributo **static** *fechaPorOmission*.

Si el acceso al método **static** se hace desde un método de otra clase, dicho método tiene que ser invocado a través del nombre de la clase según se explicó anteriormente para los atributos **static**. Por ejemplo:

```
public static void Main(string[] args)
{
    CFecha.AsignarFechaPorOmission(1, 1, 2010);
    CFecha fecha1 = new CFecha(29, 2, 2015);
    VisualizarFecha(fecha1);
}
```

Se puede observar que el comportamiento de *CFecha.AsignarFechaPorOmission* es igual que el de cualquier otro método de un lenguaje no orientado a objetos. Esto hace posible escribir programas C# utilizando solamente esta clase de métodos, pero entonces se frustraría el propósito más importante de este lenguaje: la POO. No piense por ello que utilizar este tipo de métodos es una trampa. Hay muchas y buenas razones para utilizarlos, y si no observe la utilidad del método siguiente. Se trata de un método **static** de la clase *CFecha* que devuelve la fecha actual del sistema:

```
public static void ObtenerFechaActual(out int d, out int m, out int a)
{
    // Obtener la fecha actual.
    DateTime fechaActual = DateTime.Now;

    d = fechaActual.Day;
    m = fechaActual.Month;
    a = fechaActual.Year;
}
```

Ahora, este método puede utilizarlo indistintamente en los métodos de la clase *CFecha*, o en los de cualquier otra clase. Por ejemplo, vamos a modificar el método *AsignarFecha* de la clase *CFecha* así:

```
public void AsignarFecha(params int[] fecha)
{
    // Obtener la fecha actual.
    int dd, mm, aaaa;
    ObtenerFechaActual(out dd, out mm, out aaaa);

    // Asignar la fecha pasada (por omisión la fecha actual)
    if (fecha.Length > 0) día = fecha[0]; else día = dd;
    if (fecha.Length > 1) mes = fecha[1]; else mes = mm;
    if (fecha.Length > 2) año = fecha[2]; else año = aaaa;
}
```

Un ejemplo de utilización de los métodos expuestos puede ser el siguiente:

```
public static void Main(string[] args)
{
    CFecha.AsignarFechaPorOmision(1, 1, 2010);
    CFecha fecha1 = new CFecha(29, 2, 2015);
    VisualizarFecha(fecha1);

    CFecha fecha2 = new CFecha();
    fecha1.AsignarFecha();
    VisualizarFecha(fecha1);
}
```

Una clase también puede ser declarada **static**. En este caso todos sus miembros deberán ser declarados **static**. No se pueden crear objetos de una clase estática.

DECLARACIONES PARCIALES

Una declaración parcial permite dividir la definición de una clase, estructura o interfaz en varios ficheros. La sintaxis para utilizar este tipo de definición es la siguiente:

[*modificadores*] **partial** *clase|estructura|interfaz*

El modificador **partial** permite dividir, por ejemplo, la definición de una clase en múltiples ficheros. Aplicando esta definición a la clase *CFecha* anterior, podríamos dividir su definición en dos ficheros, denominados *CFecha.cs* y *CFecha-p.cs*, así:

```
// Fichero CFecha.cs
public partial class CFecha
{
    // Atributos
    private int día, mes, año;

    // Métodos
    public CFecha(CFecha obj) // constructor copia
    {
        Copiar(obj);
    }

    public CFecha Copiar(CFecha obj)
    {
        día = obj.día;
        mes = obj.mes;
        año = obj.año;
        return this;
    }

    protected bool Bisiesto()
    {
        // ...
    }

    public void AsignarFecha(params int[] fecha)
    {
        // ...
    }
    public void ObtenerFecha(out int d, out int m, out int a)
    {
        // ...
    }
    public static void ObtenerFechaActual(out int d, out int m, out int a)
    {
        // ...
    }
    public bool FechaCorrecta()
    {
        // ...
    }
}
```

```
// Fichero CFecha-p.cs
public partial class CFecha
{
    // Atributos
    private static CFecha fechaPorOmision;

    // Métodos
    static CFecha()
    {
        // Iniciación de los atributos de la clase
        fechaPorOmision = new CFecha(1, 1, 2001);
    }

    public CFecha(params int[] fecha)
    {
        // ..
    }

    public static void AsignarFechaPorOmision(int dd, int mm, int aaaa)
    {
        fechaPorOmision.AsignarFecha(dd, mm, aaaa);
    }
}
```

Obsérvese que ambas definiciones parciales incluyen el modificador **partial**. Cuando se compile la aplicación todas las partes se combinarán.

CLASES ANIDADAS

Una *clase anidada* es una clase que es un miembro de otra clase. Por ejemplo, en el código mostrado a continuación, *CFecha* es una clase anidada:

```
public class CPersona
{
    // Miembros de CPersona
    private class CFecha
    {
        // Miembros de CFecha
    }
    // Otros miembros de CPersona
}
```

Una clase se debe definir dentro de otra sólo cuando tenga sentido en el contexto de la clase que la incluye o cuando depende de la función que desempeña la clase que la incluye. Por ejemplo una ventana puede definir su propio cursor; en este caso, la ventana puede ser un objeto de una determinada clase y el cursor de una clase anidada de ésta. Otro ejemplo, la clase *Círculo* implementada anterior-

mente podría definir un miembro que fuera la clase *Punto* para definir el centro del círculo o de la circunferencia.

Una clase anidada es un miembro más de la clase que la contiene. En el ejemplo anterior la clase *CFecha* es un miembro más de *CPersona* y como tal se le aplican las mismas reglas que para el resto de los miembros. Según esto, *CFecha* tendrá acceso al resto de los miembros de *CPersona* independientemente de su modificador de acceso (decir *CFecha* implica a los miembros de *CFecha*); *CFecha* puede ser pública, privada o protegida; puede ser estática; etc.

Recuerde: un miembro estático (**static**) es un miembro de la clase y uno no estático es un miembro del objeto y, como ocurría con los métodos estáticos, una clase anidada estática no puede referirse directamente a un miembro del objeto, sólo puede hacerlo a través de un objeto de su clase.

Un objeto de una clase anidada puede existir sólo dentro de un objeto de su clase externa. Así mismo, puesto que se trata de un miembro de su clase externa, tiene acceso directo al resto de los miembros de esa clase. Por ejemplo, una fecha puede ser un miembro de los datos relativos a la descripción de una persona; entonces, la persona puede ser representada por una clase *CPersona* y la fecha por una clase *CFecha*, como puede observar en el código mostrado a continuación:

```
public class CPersona
{
    private string nombre;
    private CFecha fechaNacimiento;

    private class CFecha
    {
        public int Día { get; set; }
        public int Mes { get; set; }
        public int Año { get; set; }

        public CFecha(int dd, int mm, int aa)
        {
            Día = dd; Mes = mm; Año = aa;
        }
        public string ObtenerFecha()
        {
            return Día + "/" + Mes + "/" + Año;
        }
    }

    public CPersona() {}

    public CPersona(string nom, int dd, int mm, int aa)
    {
```



```

        nombre = nom;
        fechaNacimiento = new CFecha(dd, mm, aa);
    }

    public string obtenerNombre() { return nombre; }

    public string ObtenerFechaNa()
    {
        return fechaNacimiento.ObtenerFecha();
    }
}

```

El siguiente ejemplo crea un objeto *CPersona* invocando al constructor de esta clase, el cual, a su vez, invocará al constructor de *CFecha* para crear el objeto *fechaNacimiento* con la fecha pasada como argumento.

```

public class Test
{
    public static void Main(string[] args)
    {
        CPersona unaPersona = new CPersona("Un nombre", 22, 2, 2015);
        Console.WriteLine(unaPersona.obtenerNombre());
        Console.WriteLine(unaPersona.ObtenerFechaNa());
    }
}

```

MÉTODOS EXTENSORES

Algunas veces puede suceder que una clase no proporciona los métodos que necesitamos para especificar la acción que se debería llevar a cabo en el contexto donde se necesita trabajar con un objeto de la misma, bien porque sus definiciones simplemente no existen, o bien porque su definición actual no se ajusta a lo que se necesita en el contexto actual de trabajo. ¿Cómo añadimos estos métodos cuando no tengamos acceso a esa clase?

Como veremos en un capítulo posterior esto se podría hacer escribiendo explícitamente una clase derivada de la existente y añadiendo/reescribiendo esos métodos a los que nos hemos referido. Pero, también podemos añadir (no reescribir) esos métodos durante el desarrollo del proyecto en cuestión utilizando *métodos extensores*.

Los métodos extensores permiten extender o ampliar la funcionalidad de una clase sin recurrir a la herencia (técnica que en ocasiones es imposible, por ejemplo, si la clase ha sido calificada **sealed**). Es importante saber que los métodos extensores, en la resolución de llamadas por parte del compilador, tienen menor resolución que los métodos nativos; esto quiere decir que si la clase implicada en

la extensión tuviera un método nativo con la misma firma que el método extensor, se utilizaría el método nativo, con lo que la reescritura de métodos no tiene efecto, tendríamos que optar por añadir nuevos métodos.

Como ejemplo, volvamos a la aplicación anterior formada por las clases *CFecha*, *CPersona* y *Test*. Cuando ejecutemos este programa observaremos una solución como ésta:

```
Un nombre  
22/2/2015
```

Supongamos que ese formato de fecha proporcionado indirectamente por el método *ObtenerFecha* de *CFecha* no se ajusta a nuestras necesidades y que no tenemos acceso al código fuente de *CFecha* porque, por ejemplo, se trata de una clase definida en una biblioteca (igual que lo son las clases, como *DateTime*, que proporciona el SDK), con lo que no podemos modificar *ObtenerFecha* para que la solución sea:

```
Un nombre  
nació el día 22 del mes 2 de 2015
```

Si tuviéramos alguna posibilidad de reescribir en alguna parte el método *ObtenerFecha* de *CFecha* el problema estaría solucionado. Esa posibilidad existe y será facilitada por un método extensor, pero, según hemos dicho anteriormente, no para reescribir el método, sino para añadir uno nuevo, por ejemplo, *ObtenerFechaEx*. La solución pasa por escribir una clase **static**, en el ejemplo *CFechaEx*, con un método **static**, en este caso *ObtenerFechaEx*, con un primer parámetro precedido por el modificador **this**:

```
public static class CFechaEx  
{  
    public static string ObtenerFechaEx(this CFecha fecha)  
    {  
        return "nació el día " + fecha.Día + " del mes " + fecha.Mes +  
            " de " + fecha.Año;  
    }  
}
```

El modificador **this** es quien indica que se trata de un método extensor (el nombre de la clase puede ser cualquiera) y sólo se permite en el primer parámetro de un método **static** que debe ser del tipo de la clase que se desea ampliar; en nuestro caso, del tipo *CFecha*.

Si ahora modificamos el método *ObtenerFechaNa* para que llame al método extensor *ObtenerFechaEx*, podremos observar que su comportamiento es como si fuera un método nativo de *CFecha*.

```
public sealed class CFecha
{
    // ...
}

public class CPersona
{
    // ...
    public string ObtenerFechaNa()
    {
        return fechaNacimiento.ObtenerFechaEx();
    }
}

class Test
{
    // ...
}
```

Resumiendo, un método extensor es una forma de evitar escribir de forma explícita una clase derivada.

DELEGADOS

Un delegado es un nuevo tipo que hace referencia a un método; en realidad, es un tipo de clase muy especializado (son similares a los punteros a función de C++, pero, a diferencia de éstos, los delegados están orientados a objetos, proporcionan seguridad de tipos y son seguros). Su sintaxis es la siguiente:

```
modificadores delegate tipo_retornado identificador(parámetros);
```

Los *modificadores* pueden ser **private**, **protected**, **public** e **internal**, *identificador* es el nombre del tipo que define al delegado, la lista de *parámetros* es opcional y especifica los parámetros del delegado. Por ejemplo, la siguiente línea de código declara el tipo *TipoOrdenacion* que permitirá definir delegados:

```
public delegate bool TipoOrdenacion(double x, double y);
```

¿Qué método puede asignarse a un delegado? Cualquier método que coincida con el prototipo del delegado, compuesto por los parámetros y el tipo de valor devuelto. Por ejemplo, a los delegados de tipo *TipoOrdenacion* se les podrá asignar

métodos con dos parámetros de tipo **double** y **bool** como tipo del valor retornado, como sucede con el siguiente método:

```
public static bool CompararDosElementos(double x, double y)
{
    return x > y;
}
```

```
TipoOrdenacion delegado = CompararDosElementos;
```

En este ejemplo, la asignación expuesta de un método a un delegado es una simplificación de la siguiente:

```
TipoOrdenacion delegado = new TipoOrdenacion(CompararDosElementos);
```

Esto permite el cambio mediante programación de las llamadas a métodos y la incorporación de nuevo código en las clases existentes. Por ejemplo, volvamos a la clase *CVector* para añadir un método *Ordenar* que permita ordenar los elementos almacenados por su atributo *vector*. La ordenación se hará según indique el método asignado al delegado pasado como argumento. Según esto, podríamos escribir el método *Ordenar* así:

```
public void Ordenar(TipoOrdenacion comparar)
{
    // Método de inserción
    int i, k;
    double x;

    // Desde el segundo elemento
    for (i = 1; i < nElementos; i++)
    {
        x = vector[i];
        k = i - 1;
        // Para k=-1, se ha alcanzado el extremo izquierdo.
        while (k >= 0 && comparar(x, vector[k]))
        {
            vector[k + 1] = vector[k]; // hacer hueco para insertar
            k--;
        }
        vector[k + 1] = x; // insertar x en su lugar
    }
}
```

Cuando se asigna un método a un delegado, éste se comporta exactamente como el método. En el ejemplo anterior, se puede observar que la forma de invocar al delegado *comparar* es la misma que utilizaríamos para invocar al método que tiene asignado. En otras palabras, cuando se ejecuta un delegado, éste le pa-

sará al método que tiene asignado los parámetros especificados y lo ejecutará. El valor devuelto por el delegado será el que devuelva el método al delegado.

La implementación del método *Ordenar* exigirá incluir en la unidad de programación de *CVector* la declaración del tipo *TipoOrdenacion*:

```
// Delegado
public delegate bool TipoOrdenacion(double x, double y);

public class CVector
{
    // Atributos y métodos de la clase CVector
}

public class Test
{
    // Atributos y métodos de la clase Test
}
```

Supongamos ahora que creamos un objeto *vector1* de la clase *CVector* y lo queremos ordenar en orden descendente. Según el código de *Ordenar*, el delegado *comparar* debe hacer referencia a un método que defina el tipo de comparación que deseamos hacer con respecto a dos elementos de *vector*. Según esto, añade a la clase *Test* el método *CompararDosElementos* y cuando se invoque al método *Ordenar* de *vector1* haga que su parámetro *comparar* reciba un delegado de tipo *TipoOrdenacion* que tenga asignado el método añadido a *Test*. Obsérvese que el código de *CompararDosElementos* es parte del algoritmo de ordenación.

```
public class Test
{
    // Visualizar un vector
    public static void VisualizarVector(CVector v)
    {
        int ne = v.Longitud();
        for (int i = 0; i < ne; i++)
            Console.Write(v.ValorEn(i) + " ");
        Console.WriteLine();
    }

    public static bool CompararDosElementos(double x, double y)
    {
        return x > y;
    }

    public static void Main(string[] args)
    {
        double[] x = new double[] { 1, 3, 5, 7, 2, 4, 6 }; // matriz x
        CVector vector1 = new CVector(x);
    }
}
```

```
vector1.Ordenar(CompararDosElementos);  
VisualizarVector(vector1);  
}  
}
```

La llamada a *Ordenar* de *vector1* del ejemplo anterior es la forma simplificada de:

```
vector1.Ordenar(new TipoOrdenacion(CompararDosElementos));
```

Si el método asignado al delegado no fuera **static**, entonces el delegado haría referencia tanto al objeto como al método. Por ejemplo:

```
public bool CompararDosElementos(double x, double y)  
{  
    return x > y;  
}
```

```
public static void Main(string[] args)  
{  
    Test obj = new Test();  
    TipoOrdenacion delegado = obj.CompararDosElementos;  
  
    // ...  
}
```

Los delegados se derivan de la clase **System.Delegate**, pero sólo el sistema y los compiladores pueden derivar explícitamente a partir de esta clase, los usuarios deben emplear la palabra clave **delegate** que proporciona el lenguaje.

Métodos anónimos

Hasta aquí, los delegados han sido creados a partir de un método con nombre. Sin embargo, en una situación en la que crear un método nuevo constituye una sobrecarga no deseada, C# permite crear objetos de un delegado y especificar inmediatamente el bloque de código que el delegado procesará cuando se le invoque. Estos métodos se denominan *métodos anónimos*.

Un método anónimo es básicamente una forma de pasar un bloque de código como argumento a un delegado perteneciente a la lista de parámetros de un método. Por ejemplo, el método **Main** de la clase *Test* anterior también podría escribirse como se muestra a continuación:

```
// Delegado  
public delegate bool TipoOrdenacion(double x, double y);
```

```

public class CVector
{
    // Atributos y métodos de la clase
}

public class Test
{
    // Visualizar un vector
    public static void VisualizarVector(CVector v)
    {
        int ne = v.Longitud();
        for (int i = 0; i < ne; i++)
            Console.Write(v.ValorEn(i) + " ");
        Console.WriteLine();
    }

    public static void Main(string[] args)
    {
        double[] x = new double[] { 1, 3, 5, 7, 2, 4, 6 }; // matriz x
        CVector vector1 = new CVector(x);

        TipoOrdenacion delegado =
            delegate(double a, double b) // método anónimo
            {
                return a > b;
            };
        vector1.Ordenar(delegado);

        VisualizarVector(vector1);
    }
}

```

Mediante los métodos anónimos, se reduce la sobrecarga de codificación al crear delegados sin tener que crear un método independiente.

La llamada a *Ordenar* de *vector1* anterior, también podría escribirse así:

```

vector1.Ordenar(
    delegate(double a, double b)
    {
        return a > b;
    });

```

Expresiones lambda

Las expresiones *lambda*, un recurso tradicional en los lenguajes de programación funcional, pueden transformarse en tipos de delegados para la generación y posterior ejecución de código. Esto es, pueden reemplazar de una manera más concisa a

los métodos anónimos: bloques de código que pueden colocarse *inline* en aquellos sitios donde el compilador espera encontrarse un delegado. La sintaxis de estas expresiones es la siguiente:

(lista de parámetros) => expresión|bloque de sentencias

La lista de parámetros, separados por comas, irá entre paréntesis (éstos pueden omitirse cuando se trate de un único parámetro). A continuación se escribe el símbolo de implicación ($\Rightarrow$) y después, la expresión o bloque de sentencias a ejecutar, tomando como argumentos esos parámetros. El operador $\Rightarrow$ tiene la misma prioridad que la asignación ($=$) y es asociativo por la derecha. A continuación se muestran algunos ejemplos:

```
a => a * 2                // expresión; tipo implícito
(int a) => a * 2          // expresión; tipo explícito
a => { return a * 2; }    // sentencia; tipo implícito
(int a) => { return a * 2; } // sentencia; tipo explícito
(a, b) => a + b           // varios parámetros
() => Console.WriteLine() // sin parámetros
(string s, int x) => s.Length > x // varios parámetros con tipo
```

Cuando el compilador no pueda deducir los tipos de entrada, habrá que especificarlos explícitamente, como ocurre en el último ejemplo.

El cuerpo de una lambda de sentencias puede estar compuesto por cualquier número de sentencias, pero generalmente este número es muy pequeño. Por ejemplo, el método anónimo del ejemplo expuesto en el apartado anterior podría ser sustituido por la siguiente lambda:

```
public delegate bool TipoOrdenacion(double x, double y);
// ...
TipoOrdenacion delegado = (a, b) => { return a > b; };
vector1.Ordenar(delegado);
```

La llamada a *Ordenar* de *vector1* anterior, también podría escribirse así:

```
vector1.Ordenar( (a, b) => { return a > b; } );
```

Otro ejemplo. Vamos a añadir a la clase *CPersona* un método *NacidoAño* que muestre las personas de una lista que hayan nacido en un año determinado. El primer argumento de este método será una referencia a la lista de personas y el segundo un delegado que ejecutará la expresión condicional necesaria.

```
// Delegado
public delegate bool ExprCondicional(CPersona pers);
```



```

public class CPersona
{
    public string Nombre { get; set; }
    public CFecha FechaNac { get; set; }

    public CPersona() {}
    public CPersona(string nom, int dd, int mm, int aa)
    {
        Nombre = nom;
        FechaNac = new CFecha(dd, mm, aa);
    }

    public string MostrarFechaNa()
    {
        return FechaNac.ObtenerFecha();
    }

    public static void NacidosAño(List<CPersona> lista,
                                   ExprCondicional condición)
    {
        foreach (CPersona p in lista)
        {
            if (condición(p))
                Console.WriteLine(p.Nombre);
        }
    }
}

class Test
{
    public static void Main(string[] args)
    {
        List<CPersona> listPersona = new List<CPersona> {
            new CPersona {
                Nombre = "Elena", FechaNac = new DateTime(1990, 6, 25) },
            new CPersona {
                Nombre = "Manuel", FechaNac = new DateTime(1991, 9, 21)}
        };

        CPersona.NacidosAño(listPersona,
                             (CPersona p) => { return p.FechaNac.Año == 1991; } );
    }
}

```

La llamada a *NacidosAño* desde *Test*, incluye en su segundo argumento una expresión lambda. El tipo de *p* no es necesario expresarlo explícitamente, ya que se deduce de la definición del delegado:

```

CPersona.NacidosAño(listPersona,
    (p) => { return p.FechaNac.Año == 1991; } );

```

A modo de resumen, podemos deducir que las expresiones *lambda* comparadas con los métodos anónimos tienen las siguientes ventajas: notación más concisa, expresiva y funcional; permiten definir los parámetros de forma implícita; el cuerpo puede ser una expresión o un bloque de sentencias; y pueden almacenarse en memoria como árboles de expresiones, concepto que se sale fuera de los objetivos de este capítulo.

Multidifusión

Un delegado mantiene una *lista de invocación* para almacenar las referencias a los métodos que llamará cuando se invoque; esto es, un delegado puede llamar a más de un método cuando se invoca, lo que se denomina *multidifusión*. Para agregar un método adicional a la lista de invocación del delegado, se utiliza el operador `+=` (puede utilizar también el `+`). Por ejemplo:

```
TipoDelegado delegado = metodo01;  
delegado += metodo02;
```

En este ejemplo, la lista de invocación de *delegado* hace referencia a dos métodos: *metodo01* y *metodo02*. Cuando se invoque a *delegado*, se llamará a los dos métodos en el orden en el que estén en la lista de invocación del delegado. Si el delegado utiliza parámetros pasados por referencia, todos los cambios efectuados por un método son visibles para el siguiente método. Cuando alguno de los métodos produce una excepción que no se detecta dentro del método, esa excepción se pasa al llamador del delegado y no se llama a ninguno de los métodos siguientes de la lista de invocación. El valor devuelto y el de los parámetros pasados por referencia corresponderán a los del último método invocado. Para quitar un método de la lista de invocación, se utiliza el operador `-=` (puede utilizar también el `-`).

Los delegados con más de un método en su lista de invocación se derivan de **MulticastDelegate**, que es una subclase de **System.Delegate**.

MATRICES DE OBJETOS

Se puede crear una matriz de objetos de cualquier clase, de la misma forma que se crea una matriz de números, de caracteres, de objetos **String**, etc. Por ejemplo, suponiendo que tenemos definida una clase *CPersona* podemos definir la matriz *listaTeléfonos* con 100 elementos de la forma siguiente:

```
CPersona[] listaTeléfonos = new CPersona[100];
```

listaTeléfonos es una matriz de referencias a objetos de la clase *CPersona*. Cada elemento de esta matriz será iniciado por C# con el valor **null**, indicando así que la matriz inicialmente no referencia a ningún objeto *CPersona*; esto es, la matriz está vacía.

Una vez creada la matriz, para asignar un objeto al elemento *i* de la misma se puede utilizar una línea de código como la siguiente:

```
listaTeléfonos[i] = new CPersona([argumentos]);
```

Como ejemplo, supongamos que deseamos mantener una lista de teléfonos. La lista será un objeto que encapsule la matriz de objetos persona, y muestre una interfaz que permita añadir, eliminar y buscar una en la lista.

En un primer análisis sobre el enunciado identificamos dos clases de objetos: personas y lista de teléfonos.

La clase de objetos persona (que denominaremos *CPersona*) encapsulará el nombre, la dirección y el teléfono de cada una de las personas de la lista. Así mismo proporcionará la funcionalidad necesaria para establecer u obtener los datos de cada persona individual.

El listado siguiente muestra un ejemplo de una clase *CPersona* que define los atributos privados *nombre*, *dirección* y *teléfono* relativos a una persona, y las propiedades públicas que forman la interfaz de esta clase de objetos:

- Constructores, con y sin argumentos, para iniciar un objeto persona.
- Propiedades (*Nombre*, *Dirección* y *Teléfono*) para cada uno de los atributos.

```
////////////////////////////////////
// Definición de la clase CPersona
//
public class CPersona
{
    // Atributos
    private string nombre;
    private string dirección;
    private long teléfono;

    // Métodos
    public CPersona() { }

    public CPersona(string nom, string dir, long tel)
    {
        nombre = nom;
        dirección = dir;
```

```
        teléfono = tel;
    }

    public string Nombre
    {
        get { return nombre; }
        set { nombre = value; }
    }

    public string Dirección
    {
        get { return dirección; }
        set { dirección = value; }
    }

    public long Teléfono
    {
        get { return teléfono; }
        set { teléfono = value; }
    }
}
```

Una propiedad como *Nombre* simplemente asigna el nombre pasado como argumento (**value**) al atributo *nombre* del objeto que recibe el mensaje y devuelve el atributo *nombre* del objeto que recibe el mensaje. La explicación para los otros métodos es análoga.

Por ejemplo, el código siguiente crea una matriz primitiva *listaTfnos* de *N* objetos *CPersona*, todos iniciados a **null**, crea un objeto *CPersona*, le asigna un nombre y lo pone en el elemento *i* de la matriz; finalmente lo muestra:

```
public static void Main(string[] args)
{
    const int N = 100;
    CPersona[] listaTfnos = new CPersona[N];
    int i = 0;
    // ...
    listaTfnos[i] = new CPersona();
    listaTfnos[i].Nombre = "Javier";
    Console.WriteLine(listaTfnos[i].Nombre);
}
```

Este otro ejemplo mostrado a continuación, utilizando la clase genérica **List** de la biblioteca .NET (las colecciones se expusieron en el capítulo 9), crea una matriz dinámica *listaTfnos* de objetos *CPersona*, inicialmente vacía, crea un objeto *CPersona*, le asigna un nombre y lo añade a la matriz; finalmente lo muestra:

```
using System.Collections.Generic;
```

```
// ...
public static void Main(string[] args)
{
    List<CPersona> listaTfnos = new List<CPersona>();
    int i = 0;
    CPersona unaPersona = new CPersona();
    // ...
    unaPersona.Nombre = "Javier";
    listaTfnos.Add(unaPersona); // añadir un objeto a la matriz
    Console.WriteLine(listaTfnos[i].Nombre);
}
```

El listado siguiente muestra un ejemplo de lo que puede ser la clase lista de teléfonos, que denominaremos *CListaTfnos*. Define los atributos privados *listaTeléfonos*, matriz de objetos *CPersona*, y los métodos que se describen a continuación:

```
using System.Collections.Generic;

////////////////////////////////////
// Definición de la clase CListaTfnos.
//
public class CListaTfnos
{
    private List<CPersona> listaTeléfonos = new List<CPersona>();

    public CListaTfnos() { ... } // constructor
    public CPersona Registro( int i ) { ... } // acceso al registro i
    public void Añadir(CPersona obj) { ... } // añadir un registro
    public bool Eliminar(long tel) { ... } // eliminar un registro
    public int Buscar(String str, int pos) { ... } // buscar un reg.
    public int Longitud(){ ... } // número de registros
}
```

Para crear un objeto lista de teléfonos escribiremos una línea de código como la siguiente:

```
CListaTfnos listatfnos = new CListaTfnos();
```

Según este ejemplo, la clase *CListaTfnos* tiene que tener un constructor sin argumentos. ¿Qué puede hacer este constructor? El constructor *CListaTfnos* lo que puede hacer es reservar memoria para un número determinado de elementos. La reserva de memoria con antelación tiene la ventaja de no tener que estar adquiriendo lentamente la memoria necesaria para cada elemento, por lo menos hasta que no se agote el espacio reservado, garantizando así un espacio para un número determinado de elementos.

```
public CListaTfnos()
{
    // Reservar espacio para 100 elementos
    listaTeléfonos.Capacity = 100;
}
```

Antes de que se ejecute el cuerpo del constructor anterior, el número de elementos de *listaTeléfonos* vale 0 (resultado de haberse ejecutado el constructor de **List**) y después de que se ejecute, sigue valiendo 0, aunque se ha adquirido memoria para almacenar los 100 objetos *CPersona* primeros. A partir de aquí, cada nuevo elemento que se añada requerirá también de una operación de adquisición de memoria, que hasta 100 objetos está garantizada.

Para añadir un teléfono (objeto *CPersona*) a la lista de teléfonos (objeto *CListaTfnos*) escribiremos un código análogo al siguiente:

```
listatfnos.Añadir(new CPersona(nombre, dirección, teléfono));
```

Cuando el objeto *listatfnos* de la clase *CListaTfnos* recibe el mensaje *Añadir*, responde ejecutando su método *Añadir* que añade al final de *listaTeléfonos* el objeto *CPersona* pasado como argumento. Para realizar esta tarea, *Añadir* invoca al método **Add** de **List**.

```
public void Añadir(CPersona objeto)
{
    listaTeléfonos.Add(objeto);
}
```

Para eliminar un teléfono (objeto *CPersona*) de la lista de teléfonos (objeto *CListaTfnos*) escribiremos un código análogo al siguiente:

```
eliminado = listatfnos.Eliminar(teléfono);
```

Cuando el objeto *listatfnos* de la clase *CListaTfnos* recibe el mensaje *Eliminar*, responde ejecutando su método *Eliminar* que quitará de la lista el elemento correspondiente al teléfono pasado como argumento y decrementará en uno el tamaño de la lista. Para realizar estas dos tareas, primero buscará en la matriz *listaTeléfonos* el objeto *CPersona* que tiene el número de teléfono pasado como argumento y, después, invocará al método **RemoveAt** para quitar ese elemento de la lista. El método *Eliminar* devuelve **true** si se encontró y eliminó el elemento especificado y **false** en caso contrario.

```
public bool Eliminar(long tel)
{
    // Buscar el teléfono y eliminar registro
```

```

    for (int i = 0; i < listaTeléfonos.Count; i++)
        if (listaTeléfonos[i].Teléfono == tel)
        {
            listaTeléfonos.RemoveAt(i);
            return true;
        }
    return false;
}

```

Para buscar un teléfono (objeto *CPersona*) en la lista de teléfonos (objeto *CListaTfnos*) escribiremos un código análogo al siguiente:

```
pos = listatfnos.Buscar(cadenabuscar, posición_inicio_búsqueda);
```

Cuando el objeto *listatfnos* de la clase *CListaTfnos* recibe el mensaje *Buscar*, responde ejecutando su método *Buscar* que recorrerá la lista de teléfonos en busca de un elemento (objeto *CPersona* referenciado) que contenga en su campo *nombre* la subcadena pasada como argumento. La búsqueda se iniciará en la posición pasada como argumento. El método *Buscar* devolverá la posición del elemento buscado, si se encuentra, o -1 en caso contrario.

```

public int Buscar(string str, int pos)
{
    string nom;
    if (str == null) return -1;
    if (pos < 0) pos = 0;
    for (int i = pos; i < listaTeléfonos.Count; i++)
    {
        nom = listaTeléfonos[i].Nombre;
        if (nom == null) continue;
        // ¿str está contenida en nom?
        if (nom.IndexOf(str) > -1)
            return i;
    }
    return -1;
}

```

Otros métodos de interés son *Registro* y *Longitud*. El método *Registro* devuelve el objeto *CPersona* que está en la posición *i* de la matriz *listaTeléfonos*, o un valor **null** si la posición especificada está fuera de límites.

```

public CPersona Registro(int i)
{
    if (i >= 0 && i < listaTeléfonos.Count)
        return listaTeléfonos[i];
    else
    {
        Console.WriteLine("Índice fuera de límites");
    }
}

```

```
        return null;
    }
}
```

El método *Longitud* devuelve el número de elementos que tiene actualmente la matriz *listaTeléfonos*.

```
public int Longitud() { return listaTeléfonos.Count; }
```

Hasta aquí, el diseño de la clase *CPersona* y *CListaTfnos*. El siguiente paso será escribir una aplicación que se ejecute así:

```
1. Buscar
2. Buscar siguiente
3. Añadir
4. Eliminar
5. Salir
```

```
    Opción: 3
nombre:    Javier
dirección: Santander
teléfono:  942232323
```

```
1. Buscar
2. Buscar siguiente
3. Añadir
4. Eliminar
5. Salir
```

```
Opción:
```

A la vista del resultado anterior, esta aplicación mostrará un menú que presentará las operaciones que se pueden realizar sobre la lista de teléfonos. Posteriormente, la operación elegida será identificada por una sentencia **switch** y procesada de acuerdo al esquema presentado a continuación:

```
using System;
using MisClases.ES; // para la clase Leer

////////////////////////////////////
// Aplicación para trabajar con matrices de objetos
//
public class Test
{
    private static int pos = -1;
    private static string cadenabuscar = null;
```



```
public static int menú()
{
    Console.Write("\n\n");
    Console.WriteLine("1. Buscar");
    Console.WriteLine("2. Buscar siguiente");
    Console.WriteLine("3. Añadir");
    Console.WriteLine("4. Eliminar");
    Console.WriteLine("5. Salir");
    Console.WriteLine();
    Console.Write("    Opción: ");
    int op;
    do
        op = Leer.datoInt();
    while (op < 1 || op > 5);
    return op;
}

public static void Buscar(CListaTfnos listatfnos,
                        bool buscar_siguiente)
{
    // ...
}

public static void Añadir(CListaTfnos listatfnos)
{
    // ...
}

public static void Eliminar(CListaTfnos listatfnos)
{
    // ...
}

public static void Main(string[] args)
{
    // Crear un objeto lista de teléfonos vacío (con cero elementos)
    CListaTfnos listatfnos = new CListaTfnos();
    int opción = 0;
    do
    {
        opción = menú();
        switch (opción)
        {
            case 1: // buscar
                Buscar(listatfnos, false);
                break;
            case 2: // buscar siguiente
                Buscar(listatfnos, true);
                break;
            case 3: // añadir
                Añadir(listatfnos);
```

```
        break;
    case 4: // eliminar
        Eliminar(listatfnos);
        break;
    case 5: // salir
        listatfnos = null;
        break;
    }
}
while (opción != 5);
}
```

El siguiente método se ejecuta para las opciones 1 y 2. Permite buscar un elemento que contenga *cadenabuscar*, subcadena que se obtiene del teclado, y si se encuentra, muestra sus datos. Cuando el segundo parámetro de este método es **true**, permite buscar el siguiente elemento que contenga la subcadena utilizada en la última búsqueda.

```
public static void Buscar(CListaTfnos listatfnos,
                        bool buscar_siguiente)
{
    if (!buscar_siguiente)
    {
        Console.Write("conjunto de caracteres a buscar ");
        cadenabuscar = Console.ReadLine();
        pos = listatfnos.Buscar(cadenabuscar, 0);
    }
    else
        pos = listatfnos.Buscar(cadenabuscar, pos + 1);
    if (pos == -1)
        if (listatfnos.Longitud() != 0)
            Console.WriteLine("búsqueda fallida");
        else
            Console.WriteLine("lista vacía");
    else
    {
        Console.WriteLine(listatfnos.Registro(pos).Nombre);
        Console.WriteLine(listatfnos.Registro(pos).Dirección);
        Console.WriteLine(listatfnos.Registro(pos).Teléfono);
    }
}
```

El siguiente método se ejecuta cuando se elige la opción 3. Obtiene los datos nombre, dirección y teléfono del nuevo elemento a añadir desde el teclado, crea un objeto *CPersona* y lo añade.

```
public static void Añadir(CListaTfnos listatfnos)
{
```

```

string nombre, dirección;
long teléfono;
Console.Write("nombre: "); nombre = Console.ReadLine();
Console.Write("dirección: "); dirección = Console.ReadLine();
Console.Write("teléfono: "); teléfono = Leer.datoLong();
listatfnos.Añadir(new CPersona(nombre, dirección, teléfono));
}

```

El siguiente método se ejecuta cuando se elige la opción 4. Obtiene del teclado el número de teléfono correspondiente al objeto *CPersona* que se desea eliminar, busca este objeto y si lo encuentra lo elimina de la lista.

```

public static void Eliminar(CListaTfnos listatfnos)
{
    long teléfono;
    bool eliminado = false;
    Console.Write("teléfono: "); teléfono = Leer.datoLong();
    eliminado = listatfnos.Eliminar(teléfono);
    if (eliminado)
        Console.WriteLine("registro eliminado");
    else if (listatfnos.Longitud() != 0)
        Console.WriteLine("teléfono no encontrado");
    else
        Console.WriteLine("lista vacía");
}

```

ESPACIO DE NOMBRES

Un *espacio de nombres*, como ya expusimos en el capítulo 5, define un ámbito, lo que proporciona una forma de crear tipos globalmente únicos. Esto permite agrupar las clases en tipos, denominados genéricamente *espacios de nombres*, en función de la labor que desempeñen. Incluso, un espacio de nombres puede contener a otros espacios de nombres, estructuras, enumeraciones, delegados e interfaces.

También vimos que la propia biblioteca de clases de .NET estaba organizada en espacios de nombres dispuestos jerárquicamente. La jerarquía a la que nos referimos es análoga a la estructura jerárquica de carpetas o directorios que utilizamos para organizar los ficheros en un disco duro.

Así mismo, sabemos que para referirnos a una *clase* de un espacio de nombres, tenemos que hacerlo anteponiendo al nombre de la misma el nombre de su espacio de nombres, excepto cuando el uso del espacio de nombres se haya puesto de manifiesto explícitamente, como se indica a continuación. Por ejemplo, la clase *CListaTfnos* anterior utiliza, entre otras, la clase **Lista** del espacio de nombres **System.Collections.Generic**, y debido a que la aplicación incluye la línea de código:

```
using System.Collections.Generic;
```

podemos referirnos a esa clase simplemente por su nombre **Lista**. En otro caso, tendríamos que haber utilizado su nombre completo:

```
System.Collections.Generic.Lista;
```

Resumiendo: los espacios de nombres ayudan a organizar las clases en grupos para facilitar el acceso a las mismas cuando las necesitemos en un programa y reducen los conflictos de nombres (lógicamente, la probabilidad de que dos nombres coincidan será menor cuantos más elementos intervengan).

Los espacios de nombres tienen acceso público, característica que no es modificable.

Toda clase pertenece a un espacio de nombres; por omisión, al espacio de nombres global.

Declarar espacios de nombres

Declarar espacios de nombres propios puede ayudar a controlar el ámbito de las clases en proyectos de programación grandes. Para declarar un espacio de nombres hay que utilizar la palabra clave **namespace**, como en el ejemplo siguiente:

```
namespace MisClases.ES // espacio de nombres
{
    public class Leer    // clase Leer
    {
        // Atributos, propiedades y métodos de Leer
    }
}
```

Obsérvese que el nombre del espacio puede ser único, o bien una secuencia de identificadores separados por puntos (“.”). Esta última forma permite a una aplicación definir espacios de nombres anidados sin expresar la anidación explícitamente. En el ejemplo anterior, el nombre *MisClases.ES* indica que el espacio de nombres *ES* está anidado en el espacio *MisClases*. Esto es, la declaración anterior es equivalente a esta otra:

```
namespace MisClases // espacio de nombres
{
    namespace ES // espacio de nombres
    {
        public class Leer // clase Leer
        {
```

```

        // Atributos, propiedades y métodos de Leer
    }
}
}

```

En este último ejemplo, el espacio de nombres *MisClases* es un miembro del espacio de nombres global (espacio de nombres predeterminado). Su nombre completo es *MisClases*, y el espacio de nombres *ES* es un miembro del espacio de nombres *MisClases*. Su nombre completo es *MisClases.ES*.

La directriz **using** nos da acceso a los miembros del espacio de nombres sin necesidad de tener que especificar el espacio de nombres cada vez que accedamos a uno de ellos, y también nos permite definir alias de espacios de nombres o de tipos (para más información, véase el capítulo 5). Por ejemplo:

```

using MisClases.ES; // acceso a los elementos de MisClases.ES
using P = System.IO.Ports; // P es un alias de System.IO.Ports

```

El espacio de nombres global es el espacio de nombres raíz.

Operador ::

El operador **::** siempre aparece entre dos identificadores:

```

identificador1::identificador2

```

Este operador se utiliza para buscar identificadores que estén potencialmente ocultos por otras entidades, proporcionando acceso explícito al espacio de nombres global o a otros que estén potencialmente ocultos por otras entidades. Cuando el identificador izquierdo es **global**, la búsqueda del identificador derecho se inicia en el espacio de nombres global. Por ejemplo, **global::System** siempre hará referencia al espacio de nombres **System** de .NET. Ahora bien, cuando el identificador izquierdo es un alias de un espacio de nombres, la búsqueda del identificador derecho se inicia en ese espacio de nombres.

En general, utilice **global::** para hacer referencia al espacio de nombres global o **alias::** para hacer referencia al espacio de nombres suplantado por *alias* y **.** para calificar tipos o miembros. Por ejemplo:

```

using P = System.IO.Ports; // para utilizar la clase SerialPort

class A
{
    public static int x;
}

```

```
class B
{
    public void M(object P, int A)
    {
        // Usar global::A.x en lugar de A.x
        global::A.x += A;
        // Usar P::SerialPort en lugar de P.SerialPort
        P::SerialPort p = P as P::SerialPort;
        // ...
        p.Close();
    }
}
```

En este ejemplo el método *M* de *B* tiene dos parámetros *P* y *A* que coinciden con los identificadores del alias *P* y de la clase *A*. Por eso la clase *A* es referenciada con *global::A* y la clase **SerialPort** con *P::SerialPort*.

ESTRUCTURAS

Una estructura es una variable de tipo **struct**. Se parece a una clase en que representa una estructura de datos que puede contener atributos y métodos. Sin embargo, a diferencia de una clase, se trata de un tipo valor por lo que se pueden crear objetos de un tipo estructura sin utilizar un operador **new**. Esto es, una variable de tipo **struct** contiene los datos, mientras que una variable de una clase, denominada objeto, contiene una referencia a los datos. Por eso, cuando se pasa una estructura a un método, se pasa por valor y no por referencia.

```
Punto p1 = new Punto(10, 10); // Punto es una estructura
Punto p2 = p1; // se crea p2 como una copia del valor de p1
```

Otras diferencias son que una estructura, al ser de tipo valor, es creada y destruida por el compilador, por lo que sería un error declarar el constructor y el destructor predeterminados (sí se pueden declarar constructores con parámetros), puede implementar interfaces, y sus atributos (no **static**) no pueden iniciarse cuando se definen.

Una estructura no puede heredar de otra estructura o clase, ni puede ser la base de una clase. Sin embargo, las estructuras heredan directamente de **System.ValueType**, que, a su vez, hereda de **System.Object**.

El tipo **struct** puede ser adecuado para representar objetos de poca complejidad, como por ejemplo un punto en un plano. ¿Por qué? Porque aunque es posible representar un punto declarando una clase, en este caso una estructura es más eficiente y si no, piense, por ejemplo, qué sucede si declara una matriz de un número elevado de puntos. En el caso de utilizar una clase *Punto*, se asignará memoria

adicional para almacenar las referencias a cada objeto punto, cosa que no sucede si la matriz es de estructuras *Punto*.

```
public struct Punto
{
    public int x, y;
    public Punto(int cx, int cy)
    {
        x = cx;
        y = cy;
    }
}

public class Test
{
    static void Main(string[] args)
    {
        Punto[] p = new Punto[1000];
        // ...
    }
}
```

EJERCICIOS RESUELTOS

1. Una matriz multidimensional representa un conjunto de elementos sucesivos en memoria que pueden ser accedidos mediante variables suscritas o de subíndices. Por ejemplo:

```
double[, ,] miMatrizDouble = new double[5,10,4];
int i = 0, j = 0, k = 0, conta = 1;
// ...
miMatrizDouble[i,j,k] = conta++;
```

Una construcción similar puede realizarse utilizando una matriz unidimensional y manipularla como si fuera una matriz multidimensional. Para ello, definiremos una clase *CMatriz* con los siguientes atributos:

```
public class CMatriz
{
    private double[] matriz;    // matriz unidimensional
    private int nDims;         // número de dimensiones
    private int[] dimsMatriz;   // valor de cada dimensión
    // ...
};
```

La clase *CMatriz* tiene como función representar una matriz multidimensional. Observe que el miembro *matriz* sirve para referenciar una matriz de una di-

mención de elementos de tipo **double**, que el miembro *nDims* contiene el número de dimensiones y *dimsMatriz* es una referencia a una matriz que contendrá el valor de cada una de ellas.

Un ejemplo de manipulación de un objeto *CMatriz* es el siguiente:

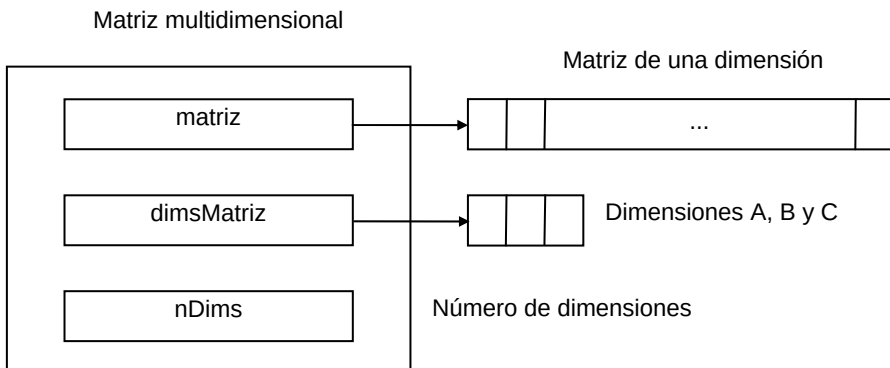
```
const int A = 5;
const int B = 10;
int i, j, conta = 1;
CMatriz m = new CMatriz(A, B); // matriz de 2 dimensiones (A*B)

// Asignar datos a la matriz m
for (i = 0; i < A; i++)
    for (j = 0; j < B; j++)
        m.asignarDato(conta++, i, j);

// Visualizar la matriz m
for (i = 0; i < A; i++)
{
    for (j = 0; j < B; j++)
        Console.Write("{0,8:f2}", m.obtenerDato(i, j));
    Console.WriteLine();
}
```

En este ejemplo, *m* representa una matriz de dos dimensiones. Observe que para acceder a un elemento utilizamos dos subíndices *i* y *j*. Pero como la matriz físicamente es una matriz de una dimensión, la idea fundamental es implementar un mecanismo que convierta una posición dada por 1, 2,..., *n* subíndices en la posición equivalente de la matriz unidimensional. Por ejemplo, si los subíndices del elemento al que deseamos acceder son *i1*, *i2* e *i3* y las dimensiones de la matriz *m* son *d1*, *d2* y *d3*, el desplazamiento se calcula así: $((i1*d2)+i2)*d3+i3$.

Para tres dimensiones, la representación gráfica de la estructura de datos construida es la siguiente:



Según lo expuesto la funcionalidad de la clase *CMatriz* estará formada por los atributos privados descritos, por el método privado:

```
void Construir(int[] dim)
```

y por los métodos públicos,

```
CMatriz(params int[] d)
int TotalElementos()
int Desplazamiento(int[] subind)
void AsignarDato(double dato, params int[] ind)
double ObtenerDato(params int[] ind)
```

Suponiendo que queremos manipular matrices de 1, 2,..., n dimensiones, responda a las siguientes preguntas:

1. Escriba el esqueleto de la definición de la clase *CMatriz*.
2. Escriba el constructor *CMatriz*. Su parámetro se corresponde con los valores de las dimensiones de la matriz. Este método invoca al método *construir* para crear un objeto *CMatriz*.
3. Escriba el método *Construir*. Este método es invocado por los constructores de la clase y comprueba si todas las dimensiones son positivas. Después establece los atributos de *CMatriz*. Tenga presente que *matriz* referencia a una matriz unidimensional que representa a la matriz de 1, 2,..., n dimensiones.

```
void Construir(int[] dim)
```

dim matriz unidimensional de enteros que contiene el valor de cada una de las dimensiones.

Por ejemplo, si *dim.Length* es 2, *dim[0]* y *dim[1]* tienen que ser valores mayores que cero. Entonces el número de elementos de la matriz sería *dim[0] * dim[1]*. Este valor será calculado por el método *TotalElementos* que se expone en el apartado siguiente.

4. Escriba el método *TotalElementos*. Este método calcula el número total de elementos de la matriz de 1, 2,..., n dimensiones.

```
int TotalElementos()
```

El método *TotalElementos* retorna el número total de elementos de la matriz.

5. Escriba el método *Desplazamiento*. Este método calcula la posición que tiene dentro de la matriz unidimensional referenciada por *matriz* el elemento que está en la matriz multidimensional en la posición especificada por los subíndices almacenados en la matriz referenciada por *subind*. Previamente, verifica si los subíndices están dentro de los límites permitidos.

```
int Desplazamiento(int[] subind)
```

El método *Desplazamiento* retorna la posición en la matriz unidimensional del elemento especificado por *subind* o -1 si algún subíndice es inválido.

6. Escriba el método *AsignarDato*. Este método asigna un dato *d* al elemento de la matriz multidimensional, especificado por los subíndices *ind*. *AsignarDato* invoca al método *Desplazamiento* para calcular el desplazamiento.

```
void AsignarDato(double dato, params int[] ind)
```

7. Escriba el método *ObtenerDato*. Este método obtiene un dato del elemento de la matriz multidimensional, especificado por sus subíndices *ind*. *ObtenerDato* invoca al método *Desplazamiento* para calcular el desplazamiento.

```
double ObtenerDato(params int[] ind)
```

El método *ObtenerDato* retorna el valor almacenado en el elemento especificado de la matriz.

8. Utilizando la clase *CMatriz* que acaba de construir, escriba un programa que utilice como cuerpo del método **Main** el expuesto en el enunciado. El resultado que tiene que obtener con este método es:

1,00	2,00	3,00	4,00	5,00	6,00	7,00	8,00	9,00	10,00
11,00	12,00	13,00	14,00	15,00	16,00	17,00	18,00	19,00	20,00
21,00	22,00	23,00	24,00	25,00	26,00	27,00	28,00	29,00	30,00
31,00	32,00	33,00	34,00	35,00	36,00	37,00	38,00	39,00	40,00
41,00	42,00	43,00	44,00	45,00	46,00	47,00	48,00	49,00	50,00

9. ¿Es necesario sobrecargar los métodos *AsignarDato* y *ObtenerDato*?

10. ¿Es necesario un destructor para esta clase? ¿Por qué?

11. ¿Qué métodos se invocan y en qué orden cuando se ejecuta la sentencia?

```
CMatriz m = new CMatriz( A, B );
```

12. ¿Qué métodos se invocan y en qué orden cuando se ejecuta la sentencia?

```
m.ObtenerDato( i, j );
```

La solución a las preguntas 1 a 7 puede obtenerlas del código presentado a continuación. Dicho código corresponde a la definición de la clase *CMatriz*.

```
using System;
////////////////////////////////////
// Matriz multidimensional basada en una unidimensional
//
public class CMatriz
{
    private double[] matriz;    // matriz unidimensional
    private int nDims;          // número de dimensiones
    private int[] dimsMatriz;   // valor de cada dimensión

    private void Construir(int[] dim)
    {
        int i;
        for (i = 0; i < dim.Length; i++)
            if (dim[i] < 1)
            {
                Console.WriteLine("Dimensión nula o negativa");
                System.Environment.Exit(-1);
            }
        // Establecer los atributos
        dimsMatriz = new int[dim.Length];
        for (i = 0; i < dim.Length; i++) dimsMatriz[i] = dim[i];
        nDims = dim.Length;
        matriz = new double[TotalElementos()];
    }

    public CMatriz(params int[] d) // constructor
    {
        int[] dim = null;
        // 10 es la dimensión por omisión
        if (d.Length == 0) dim = new int[] { 10 };
        if (d.Length > 0) dim = new int[d.Length];
        for (int i = 0; i < dim.Length; ++i)
            dim[i] = d[i];
        Construir(dim);
    }

    public int TotalElementos()
    {
        int i, nTElementos = 1;
        // Calcular el número total de elementos de la matriz
        for (i = 0; i < nDims; i++)
            nTElementos *= dimsMatriz[i];
        return nTElementos;
    }
}
```

```

public int Desplazamiento(int[] subind)
{
    int i, desplazamiento = 0;
    for (i = 0; i < nDims; i++)
    {
        // Verificar si los subíndices están dentro del rango
        if (subind[i] < 0 || subind[i] > dimsMatriz[i])
        {
            Console.WriteLine("Subíndice fuera de rango");
            return -1;
        }
        // Desplazamiento equivalente en la matriz unidimensional
        desplazamiento += subind[i];
        if (i + 1 < nDims)
            desplazamiento *= dimsMatriz[i + 1];
    }
    return desplazamiento;
}

public void AsignarDato(double dato, params int[] ind)
{
    // Asignar un valor al elemento especificado de la matriz
    int[] subind = new int[ind.Length];
    int i = 0;
    for (i = 0; i < ind.Length; i++)
    {
        subind[i] = ind[i];
    }
    i = Desplazamiento(subind);
    if (i == -1) // subíndice fuera de rango
        System.Environment.Exit(-1);
    matriz[i] = dato;
}

public double ObtenerDato(params int[] ind)
{
    // Obtener el valor al elemento especificado de la matriz
    int[] subind = new int[ind.Length];
    int i = 0;
    for (i = 0; i < ind.Length; i++)
    {
        subind[i] = ind[i];
    }
    i = Desplazamiento(subind);
    if (i == -1) // subíndice fuera de rango
        System.Environment.Exit(-1);
    return matriz[i];
}
}
////////////////////////////////////

```

A continuación se muestra la respuesta a la pregunta 8.

```

////////////////////////////////////
// Aplicación para trabajar con CMatriz
//
public class Test
{
    public static void Main(string[] args)
    {
        const int A = 5;
        const int B = 10;
        int i, j, conta = 1;
        CMatriz m = new CMatriz(A, B); // matriz de 2 dimensiones (A*B)
        // Asignar datos a la matriz m
        for (i = 0; i < A; i++)
            for (j = 0; j < B; j++)
                m.AsignarDato(conta++, i, j);

        // Visualizar la matriz m
        for (i = 0; i < A; i++)
        {
            for (j = 0; j < B; j++)
                Console.Write("{0,8:f2}", m.ObtenerDato(i, j));
            Console.WriteLine();
        }
    }
}

```

Respuesta a la pregunta 9. Los métodos *AsignarDato* y *ObtenerDato* están indirectamente sobrecargados para poder utilizar sus formas adecuadas según se trate de una matriz de 1, 2, 3,..., n dimensiones.

Respuesta a la pregunta 10. No es necesario escribir el método *~CMatriz* porque es la máquina virtual la que se encarga de gestionar la memoria asignada dinámicamente (operador **new**).

Respuesta a la pregunta 11. Los métodos invocados cuando se ejecuta la sentencia `new CMatriz m(A, B)` son:

```

public CMatriz(params int[] d) // constructor
private void Construir( int[] dim )
public int TotalElementos()

```

Respuesta a la pregunta 12. Los métodos invocados cuando se ejecuta la sentencia `m.ObtenerDato(i, j)` son:

```

public double ObtenerDato(params int[] ind)
public int desplazamiento(int[] subind)

```

EJERCICIOS PROPUESTOS

1. Cuando un alumno accede a la Universidad, se matricula de un conjunto de asignaturas en unos determinados estudios o carrera. Si analizamos este supuesto con la intención de escribir un programa orientado a objetos que permita realizar el seguimiento de las asignaturas de las que un alumno se matricula a lo largo de su estancia en la Universidad para conocer en cada momento su estado actual, podemos llegar a la conclusión de que nuestro programa tiene que manipular alumnos, asignaturas, convocatorias y fechas que darán lugar a las clases *CAlumno*, *CAsignatura*, *CConvocatoria* y *CFecha*. Un alumno se matricula de una o más asignaturas en una fecha determinada y cada vez que se examina de una de ellas consume una convocatoria, teniendo que abandonar los estudios si agota seis convocatorias de una asignatura, lo que nos exige guardar por cada asignatura los datos de cada convocatoria consumida.

La clase *CFecha* representa una fecha (día, mes y año), y define los siguientes atributos y métodos:

- *día, mes, año*: almacenan los atributos día, mes y año.
- *fechaPorOmission*: atributo **static** que almacena una fecha para utilizarla por omisión.
- *constructor*: construye un objeto *CFecha*.
- *constructor copia*: construye un objeto *CFecha* a partir de otro.
- *Copiar*: método que copia un objeto *CFecha* en otro.
- *Bisiesto*: método que comprueba si un año es bisiesto.
- *Asignar/ObtenerFecha*: métodos para asignar y obtener una fecha a/de un objeto *CFecha*. En una operación de asignación los valores de los argumentos no especificados serán sustituidos por los de la fecha actual del sistema.
- *FechaCorrecta*: método que devuelve **true** si una fecha es correcta; en otro caso, devuelve **false**.
- *AsignarFechaPorOmission*: método **static** para asignar una fecha a *fechaPorOmission*.
- *ObtenerFechaActual*: obtiene la fecha actual del sistema.

La clase *CAlumno* representa un alumno matriculado y define los siguientes atributos y métodos:

- *DNI, nombre, dirección*: datos personales del alumno.
- *asignatura*: matriz de asignaturas (**List**) de las que se ha matriculado el alumno desde su inicio. Cada elemento de esta matriz se corresponde con un objeto *CAsignatura*.

- *constructor*: construye un objeto *CAumno* con los valores pasados como argumentos y, además, invoca al método *AsignarDNI* para asegurar que este dato no es negativo.
- *constructor copia*: construye un objeto *CAumno* a partir de otro.
- *Copiar*: método que copia un objeto *CAumno* en otro.
- *Dni, Nombre o Dirección*. Propiedad que permite asignar u obtener el dato especificado (DNI, nombre o dirección).
- *ObtenerAsignatura*: devuelve el objeto *CAsignatura* que ocupa la posición *i* en la matriz de asignaturas (la primera posición es la cero). Si el índice está fuera de los límites, devuelve un **null**.
- *AñadirAsignatura*: añade el objeto *CAsignatura* pasado como argumento al final de la matriz de asignaturas.
- *EstaEnActa*: devuelve **true** si el alumno está en el acta de la asignatura cuyo identificador *id* se pasa como argumento; en otro caso devuelve **false**. Un alumno se considera incluido en el acta de una asignatura *id* si se ha matriculado, no ha aprobado y no ha consumido el total de convocatorias. Cuando se cumplen estos requisitos, el segundo argumento (pasado por referencia) devuelve la posición (0, 1...) de la asignatura en la matriz de asignaturas del objeto *CAumno*.
- *NumeroAsignaturas*: retorna el número de elementos de la matriz de asignaturas.

La clase *CAsignatura* representa una asignatura y define los siguientes atributos y métodos:

- *ID, nombre, fecha*: identificador, nombre y fecha de la primera vez que un alumno se matricula de la asignatura especificada por un objeto de esta clase.
- *convocatorias*: matriz (**List**) de convocatorias consumidas de la asignatura a la que se refiere el objeto *CAsignatura*. Los elementos de esta matriz son de tipo *CConvocatoria*.
- *constructor*: construye un objeto *CAsignatura*.
- *constructor copia*: construye un objeto *CAsignatura* a partir de otro.
- *Copiar*: método que copia un objeto *CAsignatura* en otro.
- *Id, Nombre*. Propiedad que permite asignar u obtener el dato especificado (ID o nombre).
- *AsignarFecha*: este método asigna al atributo *fecha* el objeto *CFecha* pasado como argumento si la fecha es correcta. Este método devuelve un valor **true** si la operación se realiza satisfactoriamente y **false** en caso contrario.
- *ObtenerFecha*: retorna el dato especificado.
- *ObtenerConvocatoria*: devuelve el objeto *CConvocatoria* que ocupa la posición *i* en la matriz de convocatorias (la primera posición es la cero). Si el índice está fuera de límites, devuelve **null**.

- *AñadirConvocatoria*: añade el objeto *CConvocatoria* pasado como argumento al final de la matriz de convocatorias.
- *ConvocatoriasConsumidas*: retorna el número de elementos de la matriz de convocatorias.

La clase *CConvocatoria* representa una convocatoria de una asignatura (número de la convocatoria, fecha y nota) y define los siguientes atributos y métodos:

- *convocatoria*: número, 1, 2, etc., de convocatoria. Este atributo vale 0 cuando el alumno se ha matriculado pero no ha consumido todavía ninguna convocatoria; en este caso el campo *nota* no es significativo.
- *fecha*: **String** con seis dígitos (mes y año: *mmaaaa*) correspondientes a la fecha de la convocatoria.
- *nota*: nota obtenida en esta convocatoria.
- *constructor*: construye un objeto *CConvocatoria*. Con argumentos, asigna los datos pasados, convocatoria y nota, y la fecha actual obtenida del sistema. Sin argumentos, asigna la fecha actual.
- *constructor copia*: construye un objeto *CConvocatoria* a partir de otro.
- *Copiar*: método que copia un objeto *CConvocatoria* en otro.
- *Convocatoria*: propiedad. Verifica que la convocatoria no sea mayor que 6 y la asigna, o bien la retorna.
- *AsignarFecha*: con argumentos (mes y año) verifica que la fecha sea correcta y la asigna. Si no es correcta, asigna la fecha actual obtenida del sistema. Sin argumentos, asigna la fecha actual. La fecha será reducida a un **string** con 6 dígitos correspondientes al mes y al año.
- *ObtenerFecha*: retorna el dato especificado.
- *Nota*: propiedad. Verifica que la nota esté entre los límites 0 y 10, y la asigna, o bien la retorna.

La clase *CEstudios* representa una titulación (identificador y nombre de la titulación) y define los siguientes atributos y métodos:

- *ID, nombre*: atributos, identificador y nombre de la carrera.
- *alumnos*: matriz de alumnos (**List**) que se han matriculado en esta carrera *CEstudios*.
- *asignatura*: mapa de las asignaturas (**Dictionary**<*id_asig*, *nombre_asig*>) de los estudios *CEstudios*.
- *constructor*: construye un objeto *CEstudios* con los valores identificador y nombre de la titulación pasados como argumentos. También, carga las asignaturas con sus identificadores en el mapa *asignaturas*.
- *Id, Nombre*: propiedades. Asignan/obtienen el dato especificado.

- *alumno*: devuelve el objeto *CAumno* que ocupa la posición *i* en la matriz de alumnos.
- *AñadirAlumno*: añade el objeto *CAumno* pasado como argumento al final de la matriz de alumnos.
- *Alumno*: devuelve el objeto *CAumno* que ocupa la posición *i* en la matriz de asignaturas (la primera posición es la cero).
- *NombreAsignatura*: devuelve el nombre de la asignatura del mapa asignaturas que tiene el identificador *id* pasado como argumento.
- *NumeroAlumnos*: retorna el número de elementos de la matriz de alumnos.

Una vez escritas las clases, vamos a escribir un programa que presente un menú con las opciones: matricular alumnos, poner notas, mostrar el expediente de un alumno y salir. La ejecución de este programa será de la forma siguiente:

```
1. Matricular
2. Poner notas
3. Mostrar expediente
4. Salir
```

Opción: 1

```
DNI:      111111
Nombre:    Alfonso Sánchez
Dirección: La Calzada, Barcelona
Asignaturas:
ID:        1234
Nombre:    Fundamentos de program.
```

¿Otra asignatura? s/n: s

```
ID:      1235
```

...

¿Otra asignatura? s/n: n

```
1. Matricular
2. Poner notas
3. Mostrar expediente
4. Salir
```

Opción: 2

```
ID asignatura: 1234
Alfonso Sánchez, nota: 4
Beatriz Galindo, nota: 6
...
```

```
1. Matricular
2. Poner notas
3. Mostrar expediente
4. Salir
```

```
Opción: 3
DNI: 111111
Alumno Alfonso Sánchez:
Asignatura          Convocatoria          Nota
Fundamentos de program.      1              4
                               2              7
Estructura de computad.      1              7
Sistemas operativos          NP
...
```

2. Se quiere escribir un programa para manipular ecuaciones algebraicas o polinómicas dependientes de una variable. Por ejemplo:

$$2x^3 - x + 8.25 \quad \text{más} \quad 5x^5 - 2x^3 + 7x^2 - 3 \quad \text{igual a} \quad 5x^5 + 7x^2 - x + 5.25$$

Cada término del polinomio será representado por una clase *CTérmino* y cada polinomio por una clase *CPolinomio*.

La clase *CTérmino* tendrá dos atributos privados: *coeficiente* y *exponente*, y las propiedades y métodos necesarios para permitir al menos:

- Construir un término, iniciado a cero por omisión.
- Acceder al coeficiente de un término para obtener su valor.
- Acceder al exponente de un término para obtener su valor.
- Obtener la cadena de caracteres equivalente a un término con el formato siguiente: $\{+|- \} 7x^4$.

La clase *CPolinomio* tendrá como miembro privado una matriz que referenciará los términos del polinomio (*polinomio*), así como los métodos necesarios para permitir al menos:

- Construir un polinomio, inicialmente con 0 términos.
- Obtener el número de términos que tiene actualmente el polinomio.
- Asignar un término a un polinomio colocándolo en orden ascendente del exponente. Si el coeficiente es nulo, no se realizará ninguna operación. Cada vez que se inserte un nuevo término, se incrementará automáticamente el tamaño del polinomio en uno. El método encargado de esta operación tendrá un parámetro de la clase *CTérmino*.
- Sumar dos polinomios. El polinomio resultante quedará también ordenado en orden ascendente del exponente.
- Obtener la cadena de caracteres correspondiente a la representación de un polinomio con el formato siguiente: $+ 5x^5 - 1x^1 + 5.25$.

OPERADORES SOBRECARGADOS

El término *operador sobrecargado* se refiere a un operador que es capaz de desarrollar su función en varios contextos diferentes sin necesidad de otras operaciones adicionales. Por ejemplo, la suma $a + b$ en la práctica para nosotros supondrá operaciones comunes diferentes dependiendo de que estemos trabajando en el campo de los números reales o en el campo de los números complejos. Si dotamos al operador $+$ para que, además de sumar reales, permita también sumar complejos, dependiendo esto del tipo de los operandos, entonces diremos que el *operador $+$* está *sobrecargado*.

SOBRECARGAR UN OPERADOR

C# provee la facilidad de asociar un método a un operador estándar, con el fin de que el método sea llamado cuando el compilador detecte este operador en un contexto específico. Se dice entonces que el *operador* está *sobrecargado*, porque los conjuntos de objetos sobre los que puede operar son más.

La sintaxis para declarar un operador sobrecargado es la siguiente:

public static tipo operador operador(parámetros);

donde la palabra clave **operador** más un *operador*, el que se desea sobrecargar, forman el nombre del método (método **static** que no se puede llamar explícitamente, sino que se llamará implícitamente cuando se utilice el operador en su contexto), *tipo* indica el tipo del valor retornado por el método, *operador* es uno de los de la tabla siguiente, y *parámetros* hace referencia a los operandos que intervienen en la operación, según estudiaremos a continuación.

Operadores	Posibilidad de sobrecarga
+, -, !, ~, ++, --, true, false	Operadores unarios que sí se pueden sobrecargar.
+, -, *, /, %, &, , ^, <<, >>	Operadores binarios que sí se pueden sobrecargar.
==, !=, <, >, <=, >=	Los operadores de comparación se pueden sobrecargar, pero por parejas; es decir, si se sobrecarga ==, también se debe sobrecargar !=.
&&,	Los operadores lógicos condicionales no se pueden sobrecargar, pero se evalúan mediante & y , los cuales sí se pueden sobrecargar.
[]	El operador de indexación de matrices no se puede sobrecargar, pero se pueden definir indizadores.
()	El operador de conversión explícita de tipos no se puede sobrecargar, pero se pueden definir nuevos operadores de conversión: explicit e implicit .
+=, -=, *=, /=, %=, &=, =, ^=, <<=, >>=	Los operadores de asignación no se pueden sobrecargar, pero +=, por ejemplo, se puede evaluar con +, que sí se puede sobrecargar.
=, .., ?::, ->, new, is, sizeof, typeof	Estos operadores no se pueden sobrecargar.

Sabemos que un operador unario se aplica sobre un solo operando y que un operador binario se aplica sobre dos operandos. Pues bien, según esto, cuando se sobrecargue un operador unario utilizando un método **static**, éste debe tomar un parámetro, y dos cuando se sobrecargue un operador binario. Por ejemplo:

```
public class C
{
    // Atributos
    // ...

    public static C operator -(C x)
    {
        // ...
    }

    public static C operator -(C x, C y)
    {
        // ...
    }
};
```

```

class Test
{
    public static void Main()
    {
        // a, b y c son objetos de la clase C
        C a = new C(), b = new C(), c = new C();
        b = -c;    // menos unario
        c = a - b; // menos binario
    }
}

```

La sentencia $b = -c$ implícitamente invoca al método *operator -* con un parámetro y $c = a - b$, al método *operator -* con dos parámetros.

En este ejemplo hemos utilizado una clase para definir los objetos que intervienen en las operaciones programadas a través de la sobrecarga de operadores. Pero realmente podríamos usar una estructura en lugar de una clase, si después de un análisis previo de la aplicación decidimos que es más eficiente. Por ejemplo:

```

public struct C
{
    // Atributos
    // ...

    public static C operator -(C x)
    {
        // ...
    }

    public static C operator -(C x, C y)
    {
        // ...
    }
};

class Test
{
    public static void Main()
    {
        C a = new C(), b, c = new C();
        b = -c;    // menos unario
        c = a - b; // menos binario
    }
}

```

Recuerde que una estructura, a diferencia de una clase, es un tipo valor por lo que se pueden crear objetos de un tipo estructura sin utilizar un operador **new** (en el ejemplo *b*).

Cuando se sobrecarga un operador, éste conserva su propiedad de binario o unario, y mantiene invariable su prioridad de evaluación y su asociatividad. Por ello se sugiere que se hagan sobrecargas que no realicen una operación diferente a la esperada por el operador utilizado. Esto es, la sobrecarga de un operador debe ser clara y sin ambigüedades; de lo contrario, nuestra forma natural de pensar respecto a la prioridad de operadores nos puede traicionar. Por ejemplo, si sobrecargamos el `*` para realizar la suma de complejos y el `+` para realizar la multiplicación, tendremos que recordar a la hora de utilizarlos que el operador `+` debe tener mayor prioridad que el operador `*`, lo que va en contra de nuestra forma natural de pensar; otro ejemplo, el operador `^` puede ser el más apropiado para la potenciación, pero el hecho de que su prioridad sea más baja que la del resto de los operadores aritméticos desaconseja su utilización en este contexto.

Los operadores sobrecargados son especialmente útiles cuando se trata de trabajar con tipos abstractos de datos que definen objetos pertenecientes al campo de las matemáticas; por ejemplo, operaciones con números complejos.

Como ejemplo, vamos a escribir un programa que permita realizar la suma y la resta de números complejos. Un número complejo estará definido por un objeto de tipo *Complejo* y para realizar las operaciones solicitadas esta estructura incluirá un método para sobrecargar el operador `+` y otro para el `-`. También incluirá un constructor con dos parámetros, un método *AsignarComplejo* para poder asignar el valor a un complejo y sobrescribirá el método *ToString* heredado de la clase **ValueType** para mostrar el valor de un complejo (el tema de herencia lo trataremos ampliamente en el siguiente capítulo).

```
// complejo.cs - Declaración de la estructura Complejo
using System;

public struct Complejo
{
    private double real; // parte real
    private double imag; // parte imaginaria

    public Complejo(double r, double i) // constructor
    {
        real = r; imag = i;
    }

    // Asignación de complejos
    public void AsignarComplejo(double r, double i)
    {
        real = r; imag = i;
    }

    // Suma de complejos
```

```

public static Complejo operator +(Complejo x, Complejo y)
{
    return new Complejo(x.real + y.real, x.imag + y.imag);
}

// Diferencia de complejos
public static Complejo operator -(Complejo x, Complejo y)
{
    return new Complejo(x.real - y.real, x.imag - y.imag);
}

// Mostrar un número complejo
public override string ToString()
{
    return (String.Format("{0}" +(imag >= 0 ? "+" : "")+ "{1}i", real, imag));
}
}

```

```
// Test.cs - Operaciones con números complejos.
```

```

using System;
using MisClases.ES;

class Test
{
    public static void Main(string[] args)
    {
        Complejo a = new Complejo();
        Complejo b;
        Complejo c = new Complejo(1.5, 2);
        Complejo d;
        double re, im;

        Console.WriteLine("Número complejo:");
        Console.Write("re: "); re = Leer.datoDouble();
        Console.Write("im: "); im = Leer.datoDouble();
        a.AsignarComplejo(re, im);

        b = a;
        d = a + b - c;
        d = d + new Complejo(3, 3);
        Console.WriteLine(d.ToString());
    }
}

```

A continuación realizamos un análisis de cómo trabaja este programa:

```

Complejo a = new Complejo();
Complejo b;
Complejo c = new Complejo(1.5, 2);
Complejo d;

```

Estas sentencias crean las estructuras *a*, *b*, *c* y *d*. La estructura *a* se inicia con el complejo (0,0), las estructuras *b* y *d* son iniciadas por el compilador pero no podrán ser utilizadas si no las asignamos un valor durante la ejecución, y la estructura *c* se inicia con el complejo (1.5, 2) invocando al constructor *Complejo* con dos parámetros.

```
a.AsignarComplejo(re, im);
```

Esta sentencia envía el mensaje *AsignarComplejo* al objeto *a*. La respuesta es que se ejecuta el método *AsignarComplejo* que asigna al complejo *a* los valores *re* e *im* pasados como argumentos.

```
b = a;
```

Esta otra sentencia asigna al complejo *b* el valor del complejo *a*. Esto es así por tratarse de estructuras; si fueran referencias a objetos de una clase, se copiaría la referencia *a* en *b*.

Así mismo, en el tipo *Complejo* observamos que hay dos operadores sobrecargados: $+$ y $-$. Los prototipos de los métodos correspondientes son:

```
public static Complejo operator +(Complejo x, Complejo y)
```

que es invocado por el operador $+$ cuando sus operandos son complejos, y

```
public static Complejo operator -(Complejo x, Complejo y)
```

que es invocado por el operador $-$ cuando sus operandos son complejos.

Estos métodos tienen dos parámetros: el primero almacenará el operando a la izquierda del operador y el segundo, el operando a la derecha. Por lo tanto, la expresión:

```
a + b
```

invoca al método **operator** $+$, el cual, tras realizar la operación requerida, devuelve como resultado un nuevo complejo con el resultado de la suma.

Para el operador $-$ seguiríamos un razonamiento análogo. Siguiendo con el análisis, la sentencia:

```
d = d + new Complejo(3, 3);
```


incrementa el complejo d en el valor del complejo $(3, 3)$. Para realizar esta operación, la expresión `new Complejo(3, 3)` llama al constructor `Complejo` que construye un objeto temporal, el complejo $(3, 3)$.

Finalmente, el método `ToString` devuelve la cadena correspondiente al valor del complejo:

```
d.ToString()
```

Ahora que ya tenemos claro el concepto de sobrecarga de un operador, conviene resaltar algunas restricciones. No se puede sobrecargar un operador binario (operador que se aplica a dos operandos) para crear un operador unario (operador que se aplica a un solo operando). Igualmente, no se puede sobrecargar un operador unario para realizar operaciones binarias. Por supuesto, los operadores que pueden actuar como unarios y binarios se pueden sobrecargar para utilizarlos en uno u otro contexto. Otra restricción importante es que aunque sea posible modificar la definición de un operador (por ejemplo, sobrecargar el operador $\wedge$ para que realice la potenciación), no es posible modificar su precedencia (prioridad).

UNA CLASE PARA NÚMEROS RACIONALES

A modo de ejemplo, vamos a construir una estructura *Racional* que almacena un número como cociente de dos enteros. O sea, un número racional es un número representado por el cociente de dos números enteros (lo que normalmente llamamos quebrado), como $5/7$. El número de la izquierda se denomina numerador y el de la derecha, denominador.

Esta clase es útil porque muchos números no pueden ser representados exactamente utilizando el tipo **float**. Por ejemplo, $1/3 + 1/3 + 1/3$, que es 1 , utilizando el tipo **float** sería $0,3333333 + 0,3333333 + 0,3333333$, que es $0,9999999$. El tipo *Racional* que escribimos a continuación evita este tipo de errores.

En el ejercicio planteado identificamos una entidad que es el número racional. Entonces, *numerador* y *denominador* son atributos de la estructura que hemos decidido llamar *Racional*:

```
public struct Racional
{
    // Atributos
    private long numerador;
    private long denominador;

    // Métodos
}
```

Pensemos ahora en el conjunto de operaciones que podemos realizar con los números racionales (a modo de ejemplo, sólo veremos algunas de las varias posibles):

1. Construir un número racional. Para formar un racional requerimos dos argumentos: numerador y denominador. Por ello definiremos explícitamente un constructor.
2. Operaciones de asignación.
3. Operaciones aritméticas. Suma, resta, multiplicación y división.
4. Comparación de dos números racionales. Igual, distinto, menor y mayor.
5. Operaciones de entrada y salida.
6. Incremento, decremento y cambio de signo.

Empecemos con el constructor. La construcción de un número racional sin argumentos parece lógico que sea la construcción del racional $0/1$. Otras operaciones que debe realizar el constructor es verificar si el denominador es cero, en cuyo caso forzamos a que sea 1, o negativo, en cuyo caso invertimos el signo del numerador y del denominador, y simplificar la fracción si es posible. Según esto la definición del constructor *Racional* puede ser así:

```
public Racional(long num, long den)
{
    numerador = num;
    denominador = den;

    if (den == 0) denominador = 1;
    if (den < 0)
    {
        numerador = -numerador;
        denominador = -denominador;
    }

    Simplificar();
}
```

El método *Simplificar* lo declararemos privado. Por lo tanto, una primera aproximación a la declaración de la clase podría ser así:

```
public struct Racional
{
    // Atributos
    private long numerador;
    private long denominador;
```

```

// Métodos
public Racional(long num, long den)
{
    numerador = num;
    denominador = den;
    if (den == 0) denominador = 1;
    if (den < 0)
    {
        numerador = -numerador;
        denominador = -denominador;
    }
    Simplificar();
}

private Racional Simplificar()
{
    // Máximo común divisor. Algoritmo de Euclides
    long mcd, temp, resto;
    mcd = Math.Abs(numerador);
    temp = denominador;

    while (temp > 0)
    {
        resto = mcd % temp;
        mcd = temp;
        temp = resto;
    }

    // Simplificar
    if (mcd > 1)
    {
        numerador /= mcd;
        denominador /= mcd;
    }
    return this;
}

// Métodos para operaciones de asignación
// Métodos para operaciones aritméticas
// Métodos para operaciones de relación
// Métodos para operaciones de E/S
// Métodos para operaciones sobre un único operando
};

```

El método *Simplificar* utiliza el algoritmo de Euclides para obtener el máximo común divisor (*mcd*) del numerador y del denominador, y simplificar el número racional dividiendo el numerador y el denominador por este *mcd*.

SOBRECARGA DE OPERADORES BINARIOS

A continuación vamos a estudiar cómo sobrecargar los operadores de asignación y aritméticos, los de relación y los de E/S. Dentro de este conjunto de operadores los hay que modifican la estructura interna del objeto, como `+=`, y operadores que producen un nuevo objeto, como `+`.

Sobrecarga de operadores de asignación

En este conjunto de operadores distinguimos, además del operador `=`, otros como `+=`, `/=`, etc.; por ejemplo, partiendo de las siguientes declaraciones:

```
Racional a, b = new Racional(4, 16);
```

para asignar a un racional *a* otro racional *b*, sólo tenemos que escribir:

```
a = b;
```

El operador `=` no se puede sobrecargar. No obstante, trabajando con estructuras (tipos valor) no es necesario. Esto es, siendo *a* y *b* estructuras, cuando se ejecuta la operación anterior el valor de *b* es copiado en *a*.

Analicemos ahora los operadores de asignación compuesta; por ejemplo, el operador `+=`:

```
a += b; // equivale a: a = a + b;
```

Para realizar la operación del ejemplo anterior es necesario definir el operador `+=` para los números racionales. Como los operadores de asignación no se pueden sobrecargar, la solución pasa por sobrecargar el operador `+`, lo que permitirá que se realice la suma y después la asignación ($a = a + b$). Véase a continuación la sobrecarga de los operadores aritméticos.

Las siguientes líneas de código invocan al método **operator +**, y después al operador `=`, para realizar la suma de dos números racionales y almacenar el resultado en el primero:

```
Racional a = new Racional(1, 2), b = new Racional(1, 3);  
a += b; // equivale a: a = a + b
```

Suponiendo implementado el método **operator +**, cuando se ejecute la sentencia `a += b`, el valor de *a* que era $1/2$, será ahora $5/6$ y el valor de *b* no habrá cambiado.

Sobrecarga de operadores aritméticos

Los operadores aritméticos, como `+`, producen un nuevo objeto a partir de sus dos operandos. Para definirlos podemos proceder de dos formas: escribiendo los métodos para que realicen las operaciones a partir de los atributos, o bien para que las realicen invocando a otros métodos ya implementados.

Como ejemplo, a continuación se expone la sobrecarga del operador `+` desde estos dos puntos de vista.

```
public static Racional operator +(Racional r1, Racional r2)
{
    Racional temp = new Racional(r1.numerador * r2.denominador +
                                r1.denominador * r2.numerador,
                                r1.denominador * r2.denominador);

    return temp;
}
```

Esta versión crea un objeto local *temp* invocando al constructor *Racional* con los valores resultantes de la suma, y devuelve *temp* como resultado una vez simplificado. Se puede observar que, en este caso, no es necesario invocar al método *Simplificar* puesto que ya lo hace el constructor. Otra forma de escribir este método es:

```
public static Racional operator +(Racional r1, Racional r2)
{
    return new Racional(r1.numerador * r2.denominador +
                        r1.denominador * r2.numerador,
                        r1.denominador * r2.denominador);
}
```

Esta versión crea un objeto temporal invocando al constructor *Racional* con los valores resultantes de la suma, y devuelve como resultado dicho objeto, una vez simplificado por el propio constructor.

Aritmética mixta

Supongamos ahora que uno de los operandos que interviene en la suma es un entero. Por ejemplo:

```
Racional a = new Racional(1, 2), c;
long b = 3;
c = a + b; // Racional + long
```

Cuando compilamos este código, observaremos que se muestra un error que indica que el operador `+` no se puede aplicar a operandos de tipo *Racional* y **long**. Lógicamente, esto es debido a que no existe una sobrecarga del operador `+` que tenga un primer parámetro de tipo *Racional* y un segundo de tipo **long**. Bien, añadamos esta sobrecarga del operador `+`:

```
public static Racional operator +(Racional r1, long n)
{
    return new Racional(r1.numerador + r1.denominador * n,
                        r1.denominador);
}
```

Después de añadir el método anterior, todo ha funcionado correctamente. Pero supongamos ahora que se ejecuta este otro código:

```
Racional a = new Racional(1, 2), c;
long b = 3;
c = b + a; // long + Racional
```

De nuevo el compilador muestra un error que indica que el operador `+` no se puede aplicar a operandos de tipo **long** y *Racional*. Lógicamente, igual que antes, esto es debido a que no existe una sobrecarga del operador `+` que tenga un primer parámetro de tipo **long** y un segundo de tipo *Racional*. Pues añadamos esta nueva sobrecarga del operador `+`:

```
public static Racional operator +(long n, Racional r1)
{
    return new Racional(r1.numerador + r1.denominador * n,
                        r1.denominador);
}
```

Después de añadir el método anterior, todo ha funcionado correctamente. Pero supongamos ahora que se ejecuta este otro código, donde *b* no es **long** sino **int**:

```
Racional a = new Racional(1, 2), c;
int b = 3;
c = b + a;
```

En este caso todo ha vuelto a funcionar correctamente porque el compilador C# puede promocionar implícitamente un **int** a **long**. Pero, ¿y si *b* fuera de tipo **ulong**? Pues, en este otro caso, como la conversión de **ulong** a **long** lleva aparejada una posible pérdida de información, no es realizada por el compilador, a no ser que se indique explícitamente. Entonces, otra vez más, el compilador mostraría un error que indicaría que el operador `+` no se puede aplicar a operandos de tipo **ulong** y *Racional*. Véase más adelante *Conversiones personalizadas*.

Sobrecarga de operadores de relación

Otras operaciones de uso frecuente con números racionales son las operaciones de relación. Por ejemplo, si quisiéramos saber si dos números racionales *a* y *b* son iguales, lo más sencillo sería escribir una sentencia como la siguiente:

```
Racional a = new Racional(1, 2), b = new Racional(1, 3);
if (a == b) ...
```

Pero sucede que el operador `==` no está definido para los números racionales. La solución es sobrecargar este operador. Pero, si se sobrecargan los operadores de relación, dicha sobrecarga debe realizarse por parejas; es decir, si se sobrecarga `==`, también se debe sobrecargar `!=`, o viceversa. Igual para `<` y `>`, y para `<=` y `>=`. Para el caso del operador `==`, la sobrecarga será un método **operator** `==` con dos parámetros de tipo *Racional*, que devuelva un valor **true** o **false**:

```
public static bool operator ==(Racional r1, Racional r2)
{
    return  r1.numerador * r2.denominador ==
           r1.denominador * r2.numerador;
}

public static bool operator !=(Racional r1, Racional r2)
{
    return  r1.numerador * r2.denominador !=
           r1.denominador * r2.numerador;
}
```

El método **Equals** heredado por *Racional* de **ValueType**, que a su vez hereda de **Object**, produce el mismo resultado que `==`.

Métodos adicionales

Al añadir las sobrecargas de los operadores `==` y `!=`, el compilador nos avisa de que no hemos sobrescrito el método **Object.GetHashCode**, el cual devuelve un valor que sirve como función *hash* para un tipo concreto. Este método puede delegar en la implementación de **Object** única y exclusivamente si una clase define la igualdad de valores como igualdad referencial, que no es el caso de las estructuras (**struct**). Por eso éstas deben reemplazar este método, para proporcionar una función *hash* que garantice una mejor distribución de una tabla *hash*. Por ejemplo, la implementación de **GetHashCode** que la clase **String** proporciona devuelve códigos *hash* únicos para valores de cadena únicos. Entonces, una solución fácil es escribir el método *GetHashCode* de *Racional* basado en la cadena que devuelve su método *ToString*:

```
public override int GetHashCode()
{
    return this.ToString().GetHashCode();
}
```

Lo mismo sucede con el método **Object.Equals**, el compilador avisa de que no lo hemos sobrescrito. En este caso podemos hacerlo así:

```
public override bool Equals(object obj)
{
    return base.Equals(obj); // invoca a ValueType.Equals
}
```

También, podemos estudiar cómo hacer para mostrar un racional en la salida estándar y cómo aceptar un racional desde la entrada estándar.

La primera parte ya fue expuesta al principio de este capítulo. La solucionamos sobrescribiendo el método *ToString* heredado de la clase **ValueType**:

```
public override string ToString()
{
    return String.Format("{0}/{1}", numerador, denominador);
}
```

En el siguiente capítulo dedicado a la herencia trataremos ampliamente la redefinición de métodos heredados.

El método anterior devuelve una cadena de caracteres que puede ser mostrada en la salida estándar utilizando el método **Write** o **WriteLine** así:

```
Console.WriteLine(a.ToString()); // a es un Racional
```

o simplemente así:

```
Console.WriteLine(a); // a es un Racional
```

En cuanto a aceptar un racional desde la entrada estándar, podríamos añadir a la estructura *Racional* un método *LeerRacional* para leer un valor de la forma *entero*, o *entero/entero*. Este método puede ser así:

```
// Leer un número racional
public static bool LeerRacional(out Racional r)
{
    long num = 0, den = 1;
    string sRacional;
    bool correcto = false;
```



```

while (!correcto)
{
    try
    {
        correcto = true;
        Console.Write("(entero[/entero]) ");
        sRacional = Console.ReadLine();
        int ind = sRacional.IndexOf('/');
        if (ind >= 1)
        {
            // entero[/entero]
            num = Int64.Parse(sRacional.Substring(0, ind));
            if (sRacional.Substring(ind + 1).Length > 0)
                den = Int64.Parse(sRacional.Substring(ind + 1));
        }
        if (ind < 0) num = Int64.Parse(sRacional); // entero
    }
    catch (FormatException)
    {
        correcto = false; // no es un entero
    }
}
r = new Racional(num, den); // llamar al constructor
return correcto;
}

```

En el siguiente ejemplo se observa la forma de utilizar el método *LeerRacional* que acabamos de implementar:

```

Racional a;
Racional.LeerRacional(out a);

```

SOBRECARGA DE OPERADORES UNARIOS

La sobrecarga de un operador unario es similar a la de un operador binario. Pensemos, por ejemplo, en las operaciones ++ y --. Estos operadores son los únicos que pueden utilizarse como prefijo o como sufijo sobre un operando. Además, cuando el resultado de una expresión que utiliza estos operadores se asigna a una variable, el valor de ésta será diferente en función de que dichos operadores se hayan utilizado como prefijo o como sufijo, lo que quiere decir que su comportamiento es diferente. No obstante, cuando sobrecarguemos estos operadores, sólo tendremos que definir un método, el resto lo hace el compilador.

Por ejemplo, supongamos la siguiente sentencia:

```
c = ++a;
```

Un método que realice la operación anterior ($++a$) tiene que retornar el valor de a incrementado. Entonces, cuando sobrecarguemos este operador, el método correspondiente devolverá el objeto *Racional* para el que fue invocado incrementado en una unidad. Según lo expuesto, este método puede escribirse de la forma siguiente:

```
public static Racional operator ++(Racional r)
{
    r.numerador += r.denominador;
    return r;
}
```

En cambio, si el operador $++$ se utiliza como *sufijo*, el valor del objeto será también incrementado en una unidad, pero el método **operator** $++$ devolverá el valor del objeto sin incrementar. La distinción entre ambas operaciones es cosa del compilador; esto es, el método para esta acción es el mismo.

El siguiente ejemplo utiliza $++$ como sufijo. Obsérvese que b almacena el valor sin incrementar y que a es incrementado.

```
Racional a = new Racional(1, 2), b;
b = a++;
Console.WriteLine(a); // escribe 3/2
Console.WriteLine(b); // escribe 1/2
```

Este otro ejemplo utiliza $++$ como prefijo. Obsérvese que a es incrementado y que b almacena ese valor incrementado.

```
Racional a = new Racional(1, 2), b;
b = ++a;
Console.WriteLine(a); // escribe 3/2
Console.WriteLine(b); // escribe 3/2
```

Operadores unarios/binarios

Un operador como $-$ puede utilizarse indistintamente como operador unario o como operador binario. Como ya sabemos, el método **operator** $-$ como operador unario tiene un parámetro explícito y devuelve un objeto *Racional* del mismo valor que el que invoca al método, pero de signo contrario. Por ejemplo:

```
Racional a = new Racional(1, 2), b = new Racional(1, 3), c;
c = -a + b;
```

De acuerdo con lo expuesto, este método puede escribirse de la forma siguiente:

```
// Operador - unario
public static Racional operator -(Racional r)
{
    return new Racional(-r.numerador, r.denominador);
}
```

Obsérvese que el objeto para el que se invoca el método no se modifica. Según esto, la siguiente versión de este método también sería correcta porque las estructuras se pasan por valor, pero no sería correcta si *Racional* fuera una clase ya que el objeto sería pasado por referencia y *r* quedaría modificado.

```
public static Racional operator -(Racional r)
{
    r.numerador = -r.numerador;
    return r;
}
```

El método **operator** - como operador binario tiene la misma forma y explicación que el método **operator** + visto anteriormente.

```
// Operador - para dos Racionales
public static Racional operator -(Racional r1, Racional r2)
{
    return new Racional(r1.numerador * r2.denominador -
                        r1.denominador * r2.numerador,
                        r1.denominador * r2.denominador);
}
```

CONVERSIONES PERSONALIZADAS

Hay dos tipos de conversiones: implícitas, las cuales son realizadas automáticamente por el compilador, y explícitas, las cuales fuerzan una determinada conversión utilizando una construcción *cast*. Las situaciones de conversión que se exponen a continuación son de un tipo básico a otro tipo también básico y las realiza el compilador implícitamente:

- Cuando se asigna un valor. Por ejemplo:

```
long a;
int b = 10;
a = b; // el valor de b se convierte a long
```

- Cuando se ejecuta una operación aritmética. Por ejemplo:

```
float a = 10.5F, c;
int b = 5;
```

```
c = a + b; // el valor de b se convierte a float
```

- Cuando se pasa un argumento a un método. Por ejemplo:

```
int a = 2;
double b = Test.logaritmo(a);
// ...

// El valor de a se convierte a float (x = a)
public static float logaritmo(float x)
{
    // ...
}
```

- Cuando se retorna un valor desde un método. Por ejemplo, según se ha definido anteriormente el método *logaritmo*,

```
double b = Test.logaritmo(a); // el valor devuelto por logaritmo se
                             // convierte a double
```

En cambio, cuando trabajamos con clases, por tratarse de tipos definidos por el usuario, tenemos nosotros mismos que construir las conversiones implícitas y/o explícitas que deseamos que realice el compilador cuando utilice un objeto de alguna de ellas. Estas conversiones pueden ser entre clases o estructuras, o entre una clase o estructura y un tipo predefinido. Para ello disponemos de los operadores de conversión.

Por ejemplo, un método que espera un argumento de tipo *C* puede ser invocada no sólo con un argumento de tipo *C*, sino también con un argumento de tipo *X* si existe una conversión de *X* a *C*.

Para declarar un operador de conversión en una declaración de clase o estructura se utiliza la palabra clave **operator**. Una declaración de operador de conversión puede ser de alguna de las formas siguientes:

```
public static {implicit|explicit} operator tipo1(tipo2 objeto2)
{
    tipo1 objeto1 = new tipo1();
    // Escribir aquí el código para convertir de tipo2 a tipo1
    // ...
    return objeto1;
}
```

La conversión se hace del *tipo2* al *tipo1*. Todos los operadores de conversión deben ser implementados mediante métodos **static**.

Las conversiones declaradas **implicit** se producen automáticamente cuando son necesarias y las conversiones declaradas **explicit** requieren especificar la obligación de realizar esa conversión; esto es, los operadores de conversión explícita deben invocarse mediante una conversión de tipo *cast*.

Por ejemplo, para que el código siguiente se pueda ejecutar es necesario que existan conversiones de enteros a racionales y de racionales a reales (**double**).

```
Racional a;
Racional.LeerRacional(out a);
if (a == 0) a = 1; // conversión implícita de entero a racional
double d = (double)a; // conversión explícita de racional a double
Console.WriteLine("Racional: " + a + ", valor decimal: " + d);
```

Conversiones implícitas

Cuando se eliminan las conversiones explícitas innecesarias programándolas como conversiones implícitas, la legibilidad del código mejora. No obstante, como las conversiones implícitas pueden tener lugar sin que el programador las especifique, debe tenerse cuidado para no obtener resultados no deseados.

Como ejemplo vamos a implementar la conversión implícita de números enteros a racionales en la clase *Racional*:

```
public static implicit operator Racional(long i)
{
    // Código para convertir de long a Racional
    return new Racional(i, 1);
}
```

Después de añadir este método a la clase *Racional*, la sentencia siguiente será totalmente válida. En ella se observa que para poder comparar un racional con un entero debe ser posible convertir implícitamente el racional en entero, cosa que no es posible sin perder información, o el entero en racional, lo cual sí es posible (racional formado con ese entero como numerador y denominador 1).

```
if (a == 0) a = 1;
```

Cuando se compile esta sentencia, el compilador realizará implícitamente una conversión de **int** a **long**, conversión predefinida en C#, y de **long** a *Racional*, conversión programada en la estructura *Racional*.

El ejemplo anterior demuestra que el compilador puede ejecutar simultáneamente conversiones estándar y conversiones definidas por el usuario.

En general, los operadores de conversión implícita nunca deberían producir excepciones ni pérdida de información, de modo que puedan utilizarse de forma segura sin intervención del programador. Si un operador de conversión no cumple esos criterios, debería marcarse como **explicit**.

Conversiones explícitas

Cuando una operación de conversión puede producir excepciones o pérdida de información, es aconsejable implementarla como **explicit**. De esta forma, se evita que el compilador realice la conversión automáticamente y se produzcan resultados no deseados.

Como ejemplo vamos a implementar la conversión de números racionales a reales de tipo **double** en la clase *Racional*. Como esta conversión acarrea pérdida de información, vamos a calificarla **explicit** para que cuando se utilice exista la exigencia de expresar explícitamente que se desea realizar dicha conversión.

```
public static explicit operator double(Racional r)
{
    // Código para convertir de Racional a double
    return (double)r.numerador/(double)r.denominador;
}
```

Después de añadir este método a la clase *Racional*, la sentencia mostrada a continuación será totalmente válida. En ella se observa que para poder almacenar un racional en una variable de tipo **double** debe ser posible convertir el racional a **double**, lo cual se puede hacer siempre que se indique de forma explícita; de no ser así, el compilador mostrará un error indicando que no se puede convertir implícitamente el tipo *Racional* en **double**.

```
double d = (double)a;
```

Según hemos dicho anteriormente, también puede definirse un operador de conversión que convierta un objeto de una clase o estructura en otro de otra clase o estructura. Como ejemplo, a continuación se define un operador de conversión de *Racional* a *Complejo*. Añádalo a la estructura *Racional*.

```
public static explicit operator Complejo(Racional r)
{
    // Código para convertir de Racional a Complejo
    return new Complejo((double)r, 0);
}
```

El método *operator Complejo* convierte un objeto *Racional* en otro de la clase *Complejo*. Por ejemplo:

```
Complejo c, b = new Complejo(3, -2);
Racional r = new Racional(1, 2);
c = b + (Complejo)r; // conversión explícita de Racional a Complejo
```

En este ejemplo *r* es un número racional, por lo que al realizarse la operación *b + (Complejo)r*, previamente será llamado el método *operator Complejo* para convertir *r* a complejo, realizándose a continuación la suma.

INDEXACIÓN

El operador de indexación, `[]`, utilizado con matrices, no se puede sobrecargar; no obstante, los tipos pueden definir indizadores que aceptan uno o varios parámetros. Un indizador es una propiedad con parámetros (véase *Propiedades* en el capítulo 10). Los parámetros de un indizador van entre corchetes, como los índices de una matriz, pero se pueden declarar de cualquier tipo, a diferencia de los índices de una matriz, que sólo pueden ser de tipo entero.

Los indizadores permiten indexar una clase, estructura o interfaz de la misma manera que una matriz. Una definición de un indizador tiene la forma siguiente:

```
public int this[int ind] // Declaración de un indizador
{
    get
    {
        // ...
        return nombre_matriz[ind];
    }
    set
    {
        // ...
        nombre_matriz[ind] = value;
    }
}
```

Por ejemplo, volviendo a la clase *CVector* implementada en el capítulo anterior, recordamos que incluía dos métodos, *ValorEn* y *PonerValorEn*, para permitir el acceso al elemento *i* de un objeto *CVector*. Estos métodos estaban definidos así:

```
public void PonerValorEn(int i, double valor)
{
    if (i >= 0 && i < nElementos)
        vector[i] = valor;
    else

        Console.WriteLine("Índice fuera de límites");
}
```

```
public double ValorEn(int i)
{
    if (i >= 0 && i < nElementos)
        return vector[i];
    else
    {
        Console.WriteLine("Índice fuera de límites");
        return Double.NaN;
    }
}
```

Estos métodos permitían a las aplicaciones que utilizaban la clase *CVector* implementar su propia forma de acceso a un objeto *CVector*.

Puesto que un objeto *v* de la clase *CVector* representa una matriz, parece más lógico utilizar la notación *v[i]* para acceder al elemento de índice *i* que *v.ValorEn(i)* o que *vector.PonerValorEn(i, n)*. Según lo expuesto anteriormente, esa notación la podremos utilizar indexando la clase *CVector*, lo que supone sustituir los métodos *ValorEn* y *PonerValorEn* por este indizador:

```
public double this[int ind]    // declaración de un indizador
{
    get
    {
        if (ind < 0 || ind >= nElementos)
        {
            Console.WriteLine("Índice fuera de límites");
            return Double.NaN;
        }
        else
        {
            return vector[ind];
        }
    }

    set
    {
        if (!(ind < 0 || ind >= nElementos))
        {
            vector[ind] = value;
        }
        else
            Console.WriteLine("Índice fuera de límites");
    }
}
```

Ahora, una aplicación que utilice el método anterior mostrará una forma más natural y familiar de acceso a los elementos de un objeto *CVector*. Por ejemplo:


```

public class Test
{
    // Visualizar un vector
    public static void VisualizarVector(CVector v)
    {
        for (int i = 0; i < v.Longitud(); i++)
            Console.Write(v[i] + " ");
        Console.WriteLine();
    }

    public static void Main(string[] args)
    {
        CVector vector = new CVector(20);
        for (int i = 0; i < vector.Longitud(); i++)
            vector[i] = i*2;
        VisualizarVector(vector);
    }
}

```

EJERCICIOS RESUELTOS

1. ¿Qué es un número complejo? Un número complejo está compuesto por dos números reales y se representa de la forma $a+bi$; a recibe el nombre de componente real y b el de componente imaginaria. Si $b = 0$, se obtiene el número real a , lo que quiere decir que los números reales son un caso particular de los números complejos.

Los números complejos cubren un campo que no tiene sentido en el campo de los números reales. Por ejemplo, no existe ningún número real que sea igual a $\sqrt{-9}$. Tampoco tienen sentido las expresiones $(-2)^{3/2}$ o $\log(-2)$. Para resolver este tipo de expresiones se definió la unidad imaginaria $\sqrt{-1}$, que se representa por i . De este modo podemos escribir que:

$$2 + \sqrt{-9} = 2 + 3\sqrt{-1} = 2 + 3i, \text{ que se representa como } (2, 3).$$

Puesto que un número complejo (a, b) es un par ordenado de números reales, puede representarse geoméricamente mediante un punto en el plano; dicho de otra forma, mediante un vector. De aquí se deduce que: $a+bi$, número complejo en forma binómica, es equivalente a $m(\cos \alpha + i \sen \alpha)$, número complejo en forma polar, lo que indica que $a = m \cos \alpha$ y que $b = m \sen \alpha$.

El número positivo $m = \sqrt{a^2 + b^2}$ se denomina *módulo* o *valor absoluto* y el ángulo $\alpha = \arctg(b/a)$ recibe el nombre de *argumento*.

Operaciones aritméticas:

Suma:	$(a,b)+(c,d)=(a+c,b+d)$
Resta:	$(a,b)-(c,d)=(a-c,b-d)$
Multipliación:	$(a,b)*(c,d)=(ac-bd,ad+bc)$
División:	$(a,b)/(c,d)=((ac+bd)/(c^2+d^2),(bc-ad)/(c^2+d^2))$

Estas operaciones y otras quedan perfectamente expuestas en el programa que se muestra a continuación. Las comparaciones entre complejos están referidas a sus módulos.

Según la definición dada, podemos representar un complejo como un objeto que tenga dos componentes, una para representar la parte real y otra para representar la parte imaginaria.

```
public class Complejo
{
    private double real; // parte real
    private double imag; // parte imaginaria
    // ...
};
```

La funcionalidad de esta clase está soportada por dos atributos, *real* e *imag*, que se corresponden con la parte real e imaginaria, respectivamente, del número complejo y varios conjuntos de métodos. Estos métodos se pueden clasificar de la forma siguiente:

- Uno o más constructores. El complejo construido por omisión es el (0, 0).
- Paso de forma polar a binómica.
- Operaciones aritméticas sumar, restar, multiplicar y dividir.
- Comparación de complejos. La igualdad y la desigualdad la realizaremos en módulo y argumento. El resto de las comparaciones tienen sentido cuando sólo se comparan los módulos.
- Operaciones trigonométricas.
- Operaciones logaritmo natural, exponencial, potencia y raíz cuadrada.
- Operaciones de entrada/salida.
- Manipulación de errores.
- Obtención de valores.
- Complejo conjugado, negativo y opuesto.
- Operaciones de asignación.

Implementaremos un constructor con dos argumentos y un operador de conversión de **double** a *Complejo* para hacer posible las operaciones con complejos y con reales.

El método *Error* tendrá un argumento que haga referencia a una cadena de caracteres, la correspondiente al mensaje que deseemos visualizar.

Definiremos una estructura y no una clase, de esta forma no será necesario implementar un método para hacer una copia profunda de un complejo en otro.

A continuación se muestra la implementación de la estructura *Complejo*:

```
using System;
using MisClases.ES;
```

```
public struct Complejo
{
    private double real; // parte real
    private double imag; // parte imaginaria

    static private string[] mensajeError =
    {
        "división por cero",
        "Log(0)",
        "en Pow(z, e), z = 0"
    };
};
```

```
// Manipulación de un error
public static void Error(string mensaje)
{
    Console.WriteLine("\u0007error: " + mensaje);
    System.Environment.Exit(-1);
}
```

```
// Constructores
public Complejo(double r, double i)
{
    real = r; imag = i;
}
```

```
// Conversión de un double en un complejo
public static implicit operator Complejo(double d)
{
    return new Complejo(d, 0);
}
```

```
// Paso de forma polar a binómica:  $m(\cos \alpha + i \sin \alpha) = a + bi$ 
public static Complejo Po_bi(double mod, double alfa)
{
}
```

```
        return new Complejo(mod*Math.Cos(alfa), mod*Math.Sin(alfa));
    }
```

```
// Operaciones aritméticas con complejos
```

```
public static Complejo operator +(Complejo x, Complejo y)
{
    return new Complejo(x.real + y.real, x.imag + y.imag);
}
```

```
public static Complejo operator -(Complejo x, Complejo y)
{
    return new Complejo(x.real - y.real, x.imag - y.imag);
}
```

```
public static Complejo operator *(Complejo x, Complejo y)
{
    return new Complejo(x.real * y.real - x.imag * y.imag,
        x.real * y.imag + x.imag * y.real);
}
```

```
public static Complejo operator /(Complejo x, Complejo y)
{
    double r = 0, i = 0, divisor = y.Norm();

    if (divisor != 0)
    {
        r = (x.real * y.real + x.imag * y.imag) / divisor;
        i = (x.imag * y.real - x.real * y.imag) / divisor;
    }
    else
        Complejo.Error(mensajeError[0]);
    return new Complejo(r, i);
}
```

```
// Comparación de complejos
```

```
public static bool operator ==(Complejo x, Complejo y)
{
    return (x.real == y.real) && (x.imag == y.imag);
}
```

```
public static bool operator !=(Complejo x, Complejo y)
{
    return !(x == y);
}
```

```
// Métodos adicionales
```

```
public override int GetHashCode()
{
    return this.ToString().GetHashCode();
}
```

```
public override bool Equals(object obj)
{
    return base.Equals(obj); // invoca a ValueType.Equals
}

// Para el resto de las comparaciones, comparamos módulos
public static bool operator <(Complejo x, Complejo y)
{
    return x.Mod() < y.Mod();
}

public static bool operator >(Complejo x, Complejo y)
{
    return x.Mod() > y.Mod();
}

public static bool operator <=(Complejo x, Complejo y)
{
    return x.Mod() <= y.Mod();
}

public static bool operator >=(Complejo x, Complejo y)
{
    return x.Mod() >= y.Mod();
}

// Operaciones trigonométricas con complejos
public static Complejo Cos(Complejo c)
{
    return new Complejo(Math.Cos(c.real) * Math.Cosh(c.imag),
        -Math.Sin(c.real) * Math.Sinh(c.imag));
}

public static Complejo Sin(Complejo c)
{
    return new Complejo(Math.Sin(c.real) * Math.Cosh(c.imag),
        Math.Cos(c.real) * Math.Sinh(c.imag));
}

public static Complejo Tan(Complejo c)
{
    return Sin(c) / Cos(c);
}

public static Complejo Cosh(Complejo c)
{
    return new Complejo(Math.Cosh(c.real) * Math.Cos(c.imag),
        Math.Sinh(c.real) * Math.Sin(c.imag));
}
```

```
public static Complejo Sinh(Complejo c)
{
    return new Complejo(Math.Sinh(c.real) * Math.Cos(c.imag),
        Math.Cosh(c.real) * Math.Sin(c.imag));
}

public static Complejo Tanh(Complejo c)
{
    return Sinh(c) / Cosh(c);
}

// Operaciones logarítmicas y exponenciales
public static Complejo Exp(Complejo c)
{
    double m = Math.Exp(c.real);
    return new Complejo(m*Math.Cos(c.imag), m*Math.Sin(c.imag));
}

public static Complejo Log(Complejo c)
{
    double m = c.Mod();
    if (m == 0) Complejo.Error(mensajeError[1]);
    return new Complejo(Math.Log(m), c.Arg());
}

// Potencia
public static Complejo Pow(Complejo c, Complejo e)
{
    if (e.real == 0 && e.imag == 0)
        return new Complejo(1, 0);
    else
        if (c.real == 0 && c.imag == 0)
            Complejo.Error(mensajeError[2]);
        return Exp(Log(c) * e);
}

// Raíz cuadrada
public static Complejo Sqrt(Complejo c)
{
    return Pow(c, new Complejo(0.5, 0.0));
}

// Representación alfanumérica de un complejo
public override string ToString()
{
    return "(" + real + ", " + imag + ")";
}

// Leer un complejo
public static void LeerComplejo(Complejo c)
{
}
```

```

double re = 0, im = 0;
bool error = false;
do
{
    Console.Write("real: "); re = Leer.datoDouble();
    Console.Write("imag: "); im = Leer.datoDouble();
    if (error = Double.IsNaN(re) || Double.IsNaN(im))
        Console.WriteLine("Datos incorrectos");
}
while (error);

c.real = re; c.imag = im;
}

```

```
// Obtención de valores
```

```

public double ParteReal { get { return real; } }
public double ParteImag { get { return imag; } }
public double Mod() { return Math.Sqrt(real*real + imag*imag); }
public double Arg() { return Math.Atan2(imag, real); }
public double Norm() { return real * real + imag * imag; }

```

```
// Operaciones varias
```

```

public Complejo Conjugado() { return new Complejo(real, -imag); }
public Complejo Negativo() { return new Complejo(-real, imag); }

```

```
// Menos unario. Complejos opuestos
```

```

public static Complejo operator -(Complejo r)
{
    return new Complejo(-r.real, -r.imag);
}

```

Finalmente, para probar el funcionamiento de la clase *Complejo*, crearemos una aplicación que agrupe algunos ejemplos de operaciones con números complejos:

```

using System;

////////////////////////////////////
// Aplicación para trabajar con el tipo Complejo
//
public class Test
{
    public static void Main(string[] args)
    {
        Complejo a = new Complejo(3.5, -0.7);
        Complejo b = new Complejo(2.0, 1.5);
        Complejo c = -1;
        Complejo d = new Complejo();
    }
}

```

```
Console.WriteLine("a = " + a);
Console.WriteLine("b = " + b);
Console.WriteLine("c = " + c);
Console.WriteLine("d = " + d);

double mod = a.Mod();
Console.WriteLine("mod(a) = " + mod);
double alfa = a.Arg();
Console.WriteLine("arg(a) = " + alfa);

a = -c;
Console.WriteLine("a = " + a);
a += b;
Console.WriteLine("a + b = " + a);
if (a != new Complejo(0, 0))
{
    c = b / a;
    Console.WriteLine("b/a = " + c);
}
d = Complejo.Po_bi(mod, alfa);
Console.WriteLine("bi(mod, alfa) = " + d);
d = Complejo.Tan(b);
Console.WriteLine("tan(b) = " + d);
d = Complejo.Pow(a, c);
Console.WriteLine("pow(a, c) = " + d);
Complejo.LeerComplejo(d);
Console.WriteLine("d = " + d);
a = Complejo.Log(new Complejo(0, 0));
Console.WriteLine("log(a) = " + a);
}
}
```

EJERCICIOS PROPUESTOS

1. Partiendo de las clases *CTermino* y *CPolinomio* construidas en el capítulo anterior (apartado *Ejercicios propuestos*) y almacenadas en los ficheros *CPolinomio.cs* y *CTermino.cs*:
 1. Sustituya el método *Sumar* por la sobrecarga del operador +.
 2. Suponiendo que queremos construir un polinomio igual a otro existente, ¿es necesario un constructor copia *CPolinomio*? ¿Por qué? En caso afirmativo escríbalo.
 3. Suponiendo que queremos duplicar un polinomio, ¿es necesario un método que permita duplicar un objeto *CPolinomio*? ¿Por qué? En caso afirmativo escríbalo.

4. ¿Es necesario un constructor copia *CTermino*? ¿Por qué? En caso afirmativo escríbalo.
5. ¿Es necesario un método que permita duplicar un objeto *CTermino*? ¿Por qué? En caso afirmativo escríbalo.
6. Sustituya el método *VisualizarTermino* de *CTermino* por la sobrecarga del método *ToString*.
7. ¿Qué métodos de las clases *CTermino* y *CPolinomio* se invocan y en qué orden, cuando se ejecuta la siguiente sentencia?

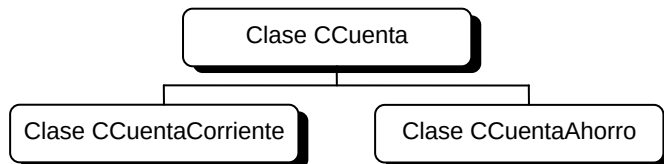
```
PolinomioR = PolinomioA + PolinomioB;
```
8. Añada un método *ParaX* a *CPolinomio* que devuelva el valor del polinomio para un valor de *x* determinado.
9. Escriba el operador de conversión adecuado para que se evalúen expresiones de la forma:

```
double v = PolinomioR; // valor del polinomio para x = 1
```
10. Sobrecargue los operadores `==`, `!=`, `<` y `>` en la clase *CTermino* para saber con respecto a dos términos cuál es el de exponente mayor.
11. ¿Debe sobrescribir los métodos *GetHashCode* y *Equals* de *CTermino*?
12. Reescriba el método que suma dos polinomios para que utilice los operadores de relación de la clase *CTermino*.
13. Si los operadores de relación de la clase *CTermino* los declara como métodos privados, ¿qué ocurre?

CLASES DERIVADAS E INTERFACES

Las características fundamentales de la POO son *abstracción*, *encapsulamiento*, *herencia* y *polimorfismo*. Hasta ahora sólo hemos abordado la *abstracción* y la *encapsulación*. Aunque todas ellas son fundamentales, hay una que destaca: la *herencia*. La *herencia* provee el mecanismo más simple para especificar una forma alternativa de acceso a una clase existente, o bien para definir una nueva clase que añada nuevas características a una clase existente. Esta nueva clase se denomina *clase derivada* o *subclase* y la clase existente, *clase base* o *superclase*.

Con la herencia todas las clases están clasificadas en una jerarquía estricta. Cada clase tiene su superclase (la clase superior en la jerarquía), y cada clase puede tener una o más subclases (las clases inferiores en la jerarquía). Las clases que están en la parte inferior en la jerarquía se dice que *heredan* de las clases que están en la parte superior en la jerarquía. Por ejemplo, la figura siguiente indica que las clases *CCuentaCorriente* y *CCuentaAhorro* heredan de la clase *CCuenta*.



Una jerarquía de clases muestra cómo los objetos se derivan de otros objetos más simples heredando su comportamiento. Los usuarios de C++ y de otros lenguajes de programación orientada a objetos como Java están acostumbrados a ver jerarquías de clases para describir la *herencia*. Los de C# seguirán, en general, los mismos pasos.

CLASES DERIVADAS Y HERENCIA

Vuelva a echar una ojeada a la figura mostrada al principio de este capítulo. Se trata de una jerarquía de clases que puede ser analizada desde dos puntos de vista:

1. Cuando en un principio se abordó el diseño de una aplicación para administrar las cuentas de una entidad bancaria, fue suficiente con las capacidades proporcionadas por la clase *CCuenta*. Posteriormente, la evolución de los mercados bancarios sugirió nuevas modalidades de cuentas. La mejor solución para adaptar la aplicación a esas nuevas exigencias fue definir una clase derivada de *CCuenta* para cada nueva modalidad, puesto que el mecanismo de herencia ponía a disposición de éstas todo el código de su clase base, al que sólo era necesario añadir las nuevas especificaciones. Resulta por lo tanto evidente que la *herencia* es una forma sencilla de *reutilizar el código*.
2. Cuando se abordó el diseño de una aplicación para administrar las cuentas de una entidad bancaria, la solución fue diseñar una clase especializada para cada una de las cuentas y agrupar el código común en una clase base de éstas.

En los dos casos planteados, la herencia es la solución para reutilizar código perteneciente a otras clases. Para ilustrar el mecanismo de herencia vamos a implementar la jerarquía de clases de la figura anterior. La idea es diseñar una aplicación para administrar las cuentas corrientes y de ahorro de los clientes de una entidad bancaria. Como ambas cuentas tienen bastantes cosas en común, hemos decidido agrupar éstas en una clase *CCuenta* de la cual posteriormente derivaremos las cuentas específicas que vayan surgiendo. Según este planteamiento, no parece que tengamos intención de crear objetos de *CCuenta*; más bien la intención es que agrupe el código común que heredarán sus clases derivadas, razón por la que, más adelante, cuando estudiemos clases abstractas, la declararemos abstracta.

Pensemos entonces inicialmente en el diseño de la clase *CCuenta*. Después de un análisis acerca de los factores que intervienen en una cuenta en general, llegamos a la conclusión de que los atributos y métodos comunes a cualquier tipo de cuenta son los siguientes:

Atributo	Significado
<i>nombre</i>	Dato de tipo string que almacena el nombre del propietario de la cuenta.
<i>cuenta</i>	Dato de tipo string que almacena el número de la cuenta.
<i>saldo</i>	Dato de tipo double que almacena el saldo de la cuenta.
<i>tipoDeInterés</i>	Dato de la clase de tipo double que almacena el tipo de interés.

Método	Significado
<i>CCuenta</i>	Es el constructor de la clase. Inicia los datos <i>nombre</i> , <i>cuenta</i> , <i>saldo</i> y <i>tipoDeInterés</i> con los valores pasados como argumentos en la llamada, o con los valores pre-determinados.
<i>AsignarNombre</i>	Permite asignar el dato <i>nombre</i> . Retorna false si el nombre es una cadena vacía, y true en otro caso.
<i>ObtenerNombre</i>	Retorna el dato <i>nombre</i> .
<i>AsignarCuenta</i>	Permite asignar el dato <i>cuenta</i> . Retorna false si la cuenta es una cadena vacía, y true en otro caso.
<i>ObtenerCuenta</i>	Retorna el dato <i>cuenta</i> .
<i>Saldo</i>	Retorna el saldo de la cuenta.
<i>Comisiones</i>	Es un método sin parámetros que será redefinido en las clases derivadas. Se ejecutará los días uno de cada mes para cobrar el importe del mantenimiento de una cuenta.
<i>Ingreso</i>	Es un método que tiene un parámetro <i>cantidad</i> de tipo double que añade la cantidad especificada al saldo actual de la cuenta. Retorna false si la cantidad es negativa, y true en otro caso.
<i>Reintegro</i>	Es un método que tiene un parámetro <i>cantidad</i> de tipo double que resta la cantidad especificada del saldo actual de la cuenta.
<i>AsignarTipoDeInterés</i>	Método que permite asignar el dato <i>tipoDeInterés</i> . Retorna false si el tipo de interés es negativo, y true en otro caso.
<i>ObtenerTipoDeInterés</i>	Método que retorna el dato <i>tipoDeInterés</i> .
<i>Intereses</i>	Método que calcula los intereses producidos.

El código correspondiente a esta clase se expone a continuación:

```

////////////////////////////////////
// Clase CCuenta: clase que agrupa los datos comunes a
// cualquier tipo de cuenta bancaria.
//
public class CCuenta
{
    // Atributos
    private string nombre;
    private string cuenta;
    private double saldo;
    private double tipoDeInterés;

    // Métodos
    public CCuenta() {}

```

```
public CCuenta(string nom, string cue, double sal, double tipo)
{
    // Verificar los datos asignados
    AsignarNombre(nom);
    AsignarCuenta(cue);
    Ingreso(sal);
    AsignarTipoDeInterés(tipo);
}

public bool AsignarNombre(string nom)
{
    if (nom.Length == 0)
    {
        Console.WriteLine("Error: cadena vacía");
        return false;
    }
    nombre = nom;
    return nom.Length != 0;
}

public string ObtenerNombre()
{
    return nombre;
}

public bool AsignarCuenta(string cue)
{
    if (cue.Length == 0)
    {
        Console.WriteLine("Error: cuenta no válida");
        return false;
    }
    cuenta = cue;
    return cue.Length != 0;
}

public string ObtenerCuenta()
{
    return cuenta;
}

public double Saldo()
{
    return saldo;
}

public void Comisiones()
{
    return; // sin comisiones
}
```

```

public double Intereses()
{
    return 0.0; // sin intereses
}

public bool Ingreso(double cantidad)
{
    if (cantidad < 0)
    {
        Console.WriteLine("Error: cantidad negativa");
        return false;
    }
    saldo += cantidad;
    return cantidad >= 0;
}

public void Reintegro(double cantidad)
{
    if (saldo - cantidad < 0)
    {
        Console.WriteLine("Error: no dispone de saldo");
        return;
    }
    saldo -= cantidad;
}

public bool AsignarTipoDeInterés(double tipo)
{
    if (tipo < 0)
    {
        Console.WriteLine("Error: tipo no válido");
        return false;
    }
    tipoDeInterés = tipo;
    return tipo >= 0;
}

public double ObtenerTipoDeInterés()
{
    return tipoDeInterés;
}
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

```

DEFINIR UNA CLASE DERIVADA

Pensemos ahora en un tipo de cuenta específico, como es una cuenta de ahorro. Una cuenta de ahorro tiene las características aportadas por un objeto *CCuenta*, y además algunas otras; por ejemplo, un atributo que especifique el importe que hay

que pagar mensualmente por el mantenimiento de la misma. Esto significa que necesitamos diseñar una nueva clase, *CCuentaAhorro*, que tenga las mismas capacidades de *CCuenta*, pero a las que hay que añadir otras que den solución a las nuevas necesidades.

Una forma de hacer esto sería definir una nueva clase *CCuentaAhorro* con los atributos y métodos de *CCuenta*, a los que añadiríamos los nuevos atributos y métodos, según muestra el diseño siguiente:

```
public class CCuentaAhorro
{
    // Atributos y métodos de CCuenta
    // Nuevos atributos y métodos de CCuentaAhorro
}
```

Esta forma de proceder puede que funcione, pero no deja de ser una mala solución; además de suponer un derroche de tiempo y esfuerzo, todo el trabajo que ya estaba realizado no ha servido para nada. Aquí es donde la herencia juega un papel importante; la utilización de esta característica evitará que recurramos a soluciones como la planteada, o a otras como la siguiente:

```
public class CCuentaAhorro
{
    CCuenta c = new CCuenta(); // inclusión mediante c

    // Nuevos atributos y métodos de CCuentaAhorro
}
```

Según este otro planteamiento, una cuenta de ahorro también es una cuenta; es decir, este diseño especifica que cada operación no definida en la clase *CCuentaAhorro* puede ser servida por un objeto *c* de *CCuenta*. En cambio, aunque esto pueda parecer evidente, no hay nada que indique al compilador que un objeto *CCuentaAhorro* es también un objeto *CCuenta*. Y tampoco una referencia a *CCuentaAhorro* es una referencia a *CCuenta*, con lo cual no será nada fácil construir una lista de objetos de cuentas diferentes. El enfoque correcto es especificar de manera explícita que *CCuentaAhorro* es una extensión *CCuenta*. Y esto, ¿cómo se hace? A través de la herencia, definiendo una clase derivada de la clase existente.

Una *clase derivada* es un nuevo tipo de objetos definido por el usuario que tiene la propiedad de heredar los atributos, propiedades y métodos de otra clase definida previamente, denominada *clase base*. La sintaxis para definir una *clase derivada* es la siguiente:

```
[public|internal] class nombre_clase_derivada : nombre_clase_base
{
```



```
// Cuerpo de la clase derivada
}
```

El símbolo **:** significa que se está definiendo una clase denominada *nombre_clase_derivada* que es una extensión de otra denominada *nombre_clase_base*; también se puede decir que *nombre_clase_derivada* es una clase derivada de *nombre_clase_base*.

El ejemplo mostrado a continuación define la clase *CCuentaAhorro* como una clase derivada de *CCuenta*:

```
public class CCuentaAhorro : CCuenta
{
    // CCuentaAhorro ha heredado los miembros de CCuenta
    // Escriba aquí los nuevos atributos y métodos de CCuentaAhorro
}
```

Si no se especifica el símbolo **:** más el nombre de la clase base, se entiende que la clase base es la clase **Object**. Esto indica que la clase *CCuenta* está derivada de la clase **Object**.

Una clase derivada puede, a su vez, ser una clase base de otra clase, dando lugar así a una *jerarquía de clases*. Por lo tanto, una clase puede ser una clase base directa de una clase derivada, si figura explícitamente en la definición de la clase derivada, o una clase base indirecta si está varios niveles arriba en la jerarquía de clases, y por lo tanto no figura explícitamente en el encabezado de la definición de la clase derivada.

Cuando una clase derivada lo es de una sola clase base, la herencia se denomina *herencia simple* o *derivación simple*. En cambio, cuando lo es de dos o más clases, la herencia se denomina *múltiple* o *derivación múltiple*. C#, a diferencia de otros lenguajes como C++, no permite la herencia múltiple.

Control de acceso a los miembros de las clases

En el capítulo dedicado a clases se expuso que para controlar el acceso a los miembros de una clase, C# provee las palabras clave **private** (privado), **protected** (protegido), **public** (público), **internal** (interno) y **protected internal** (interno y protegido), o bien pueden omitirse (acceso privado). Lo allí estudiado se amplía ahora para las clases derivadas. Para evitar confusiones, la tabla siguiente resume de una forma clara qué clases, o clases derivadas, pueden acceder a los miembros de otra clase, dependiendo del control de acceso especificado. Evidentemente, cuando hablamos de acceso a un miembro nos referimos al acceso directo al mismo a través de su nombre.

Puede ser accedido desde:	Un miembro declarado en una clase como			
	<i>privado</i>	<i>interno</i>	<i>protegido</i>	<i>público</i>
Su misma clase.....	sí	sí	sí	sí
Cualquier clase del mismo ensamblado.....	no	sí	no	sí
Cualquier clase derivada del mismo ensamblado.....	no	sí	sí	sí
Cualquier clase de otro ensamblado.....	no	no	no	sí
Cualquier clase derivada de otro ensamblado.....	no	no	sí	sí

De lo expuesto se deduce que una clase derivada no tiene acceso a ningún miembro privado de su clase base. Aunque esta restricción pueda sorprender, es así para imponer la encapsulación. De otra forma, esto es, si una clase derivada tuviera acceso a los miembros privados de su clase base, bastaría derivar una clase de cualquier otra para acceder a sus miembros privados.

Qué miembros hereda una clase derivada

Los siguientes puntos resumen las reglas a tener en cuenta cuando se define una clase derivada:

1. Una clase derivada hereda todos los miembros de su clase base, excepto los constructores en general y los destructores, lo cual no significa que tenga acceso directo a todos los miembros. Una consecuencia inmediata de esto es que la estructura interna de datos de un objeto de una clase derivada estará formada por los atributos que ella define y por los heredados de su clase base.

Una clase derivada no tiene acceso directo a los miembros privados (**private**) de su clase base, pero sí puede acceder a los miembros públicos (**public**) y protegidos (**protected**) de su clase base; y en el caso de que pertenezca al mismo ensamblado de su clase base, también puede acceder a los miembros internos (**internal**).

2. Una clase derivada puede añadir sus propios atributos y métodos. Si el nombre de alguno de estos miembros coincide con el de un miembro heredado, éste último queda oculto para la clase derivada, que se traduce en que la clase derivada ya no puede acceder directamente a ese miembro. Lógicamente, lo expuesto tiene sentido siempre que nos refiramos a los miembros de la clase

base a los que la clase derivada podía acceder, según el control de acceso aplicado.

3. Los miembros heredados por una clase derivada pueden, a su vez, ser heredados por más clases derivadas de ella. A esto se le llama propagación de herencia.

Continuando con el ejemplo, diseñemos una nueva clase *CCuentaAhorro* que tenga, además de las mismas capacidades de *CCuenta*, las siguientes:

Atributo	Significado
<i>cuotaMantenimiento</i>	Dato de tipo double que almacena la comisión que cobrará la entidad bancaria por el mantenimiento de la cuenta.
Método	Significado
<i>CCuentaAhorro</i>	Es el constructor de la clase. Inicia los atributos de la misma.
<i>AsignarCuotaManten</i>	Establece la cuota de mantenimiento de la cuenta. Retorna false si la cantidad es negativa, y true en otro caso.
<i>ObtenerCuotaManten</i>	Devuelve la cuota de mantenimiento de la cuenta.
<i>Comisiones</i>	Método que se ejecuta los días 1 de cada mes para cobrar el importe correspondiente al mantenimiento de la cuenta.
<i>Intereses</i>	Método que permite calcular el importe correspondiente a los intereses/mes producidos, los cuales serán abonados los días 1 de cada mes.

Los métodos *Comisiones* e *Intereses* obtienen la fecha actual a través de la funcionalidad proporcionada por la clase *CFecha* desarrollada en los capítulos anteriores. Veamos a continuación la definición de *CCuentaAhorro*:

```

////////////////////////////////////
// Clase CCuentaAhorro: clase derivada de CCuenta
//
public class CCuentaAhorro : CCuenta
{
    // Atributos
    private double cuotaMantenimiento;

    // Métodos
    public CCuentaAhorro() { } // constructor sin parámetros

    public bool AsignarCuotaManten(double cantidad)
    {
        if (cantidad < 0)
        {

```

```
        Console.WriteLine("Error: cantidad negativa");
        return false;
    }
    cuotaMantenimiento = cantidad;
    return cantidad >= 0;
}

public double ObtenerCuotaManten()
{
    return cuotaMantenimiento;
}

public void Comisiones()
{
    // Se aplican mensualmente por el mantenimiento de la cuenta
    int día, mes, año;
    CFecha.ObtenerFechaActual(out día, out mes, out año);
    if (día == 1) Reintegro(cuotaMantenimiento);
}

public double Intereses()
{
    int día, mes, año;
    CFecha.ObtenerFechaActual(out día, out mes, out año);

    if (día != 1) return 0.0;
    // Acumular los intereses por mes sólo los días 1 de cada mes
    double interesesProducidos = 0.0;
    interesesProducidos = Saldo() * ObtenerTipoDeInterés() / 1200.0;
    Ingreso(interesesProducidos);

    // Devolver el interés mensual por si fuera necesario
    return interesesProducidos;
}
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

CCuentaAhorro es una clase derivada de la clase base *CCuenta*. Obsérvese que para definir una clase derivada se añade a continuación del nombre de la misma el símbolo **:** más el nombre de la clase base. En la definición de la clase derivada se describen las características adicionales que la distinguen de la clase base.

Veamos a continuación una comparativa de las clases base (clase base) y derivada (clase derivada). La capacidad de la clase *CCuenta* está soportada por:

Atributos	Métodos
nombre	constructores <i>CCuenta</i>

cuenta	AsignarNombre
saldo	ObtenerNombre
tipoDeInterés	AsignarCuenta
	ObtenerCuenta
	Saldo
	Comisiones
	Intereses
	Ingreso
	Reintegro
	AsignarTipoDeInterés
	ObtenerTipoDeInterés

La capacidad de la clase *CCuentaAhorro*, derivada de *CCuenta*, está soportada por los miembros heredados de *CCuenta* (en cursiva y no tachados) más los suyos:

Atributos	Métodos
<i>nombre</i>	constructores <i>CCuenta</i>
<i>cuenta</i>	<i>AsignarNombre</i>
<i>saldo</i>	<i>ObtenerNombre</i>
<i>tipoDeInterés</i>	<i>AsignarCuenta</i>
	<i>ObtenerCuenta</i>
	<i>Saldo</i>
	Comisiones
	Intereses
	<i>Ingreso</i>
	<i>Reintegro</i>
	<i>AsignarTipoDeInterés</i>
	<i>ObtenerTipoDeInterés</i>
cuotaMantenimiento	constructores <i>CCuentaAhorro</i>
	<i>AsignarCuotaManten</i>
	<i>ObtenerCuotaManten</i>
	<i>Comisiones</i>
	<i>Intereses</i>

Obsérvese que la clase *CCuenta* define sus constructores, pero aunque no se definieran, el compilador generaría uno por omisión. Esto implica que los constructores no se heredan. Por la misma razón, tampoco se heredaría el destructor si se escribiera. Por otra parte, los métodos *Comisiones* e *Intereses* quedan ocultos por los métodos del mismo nombre de la clase *CCuentaAhorro*. Un poco más adelante veremos que la clase derivada puede referirse a un miembro oculto utilizando la palabra reservada **base** de C#: **base.miembro_oculto**.

Según el análisis anterior, mientras un posible objeto *CCuenta* contendría los datos *nombre*, *cuenta*, *saldo* y *tipoDeInterés*, un objeto *CCuentaAhorro* contiene los datos *nombre*, *cuenta*, *saldo*, *tipoDeInterés* y *cuotaMantenimiento*.

Escribamos ahora una pequeña aplicación basada en una clase *Test* que cree un objeto *CCuentaAhorro* (la aplicación incluirá los ficheros *Test.cs*, *CCuenta.cs*, *CCuentaAhorro.cs* y *CFecha.cs*):

```
public class Test
{
    public static void Main(string[] args)
    {
        CCuentaAhorro cuenta01 = new CCuentaAhorro();
        cuenta01.AsignarNombre("Un nombre");
        cuenta01.AsignarCuenta("Una cuenta");
        cuenta01.AsignarTipoDeInterés(2.5);
        cuenta01.AsignarCuotaManten(300);
        cuenta01.Ingreso(1000000);
        cuenta01.Reintegro(500000);
        cuenta01.Comisiones();

        // cuenta01 no puede acceder a los miembros privados, como
        // cuenta.
    }
}
```

Cuando compilemos esta aplicación, el compilador de C# nos avisará de que el método *CCuentaAhorro.Comisiones* oculta el método heredado *CCuenta.Comisiones*. Ídem con respecto a los métodos *CCuentaAhorro.Intereses* y *CCuenta.Intereses*. Para evitar esta advertencia se puede utilizar el modificador **new**. Por ejemplo:

```
public new void Comisiones()
{
    // ...
}

public new double Intereses()
{
    // ...
}
```

El modificador **new** oculta explícitamente un miembro heredado de una clase base. Esto significa que la versión de la clase derivada reemplaza a la versión de la clase base. Además, suprime la advertencia a la que nos hemos referido anteriormente y documenta el hecho de que se ha realizado un reemplazo.

Si partimos del hecho de que las clases *CCuenta* y *CCuentaAhorro* pertenecen al mismo ensamblado que la clase *Test*, entonces, un “objeto”, como *cuenta01*, de la clase *CCuentaAhorro*, definido en el ámbito de otra clase (método **Main** de *Test*), puede invocar a cualquiera de los métodos públicos e internos de *CCuentaAhorro* y de *CCuenta*, pero no tiene acceso a sus miembros privados ni protegidos. Si las clases *CCuentaAhorro* y *CCuenta* pertenecieran a otro ensamblado, la clase *Test* sólo tendría acceso a los miembros públicos.

Los “métodos” de una clase derivada no tienen acceso a los miembros privados de su clase base, pero sí lo tienen a sus miembros protegidos y públicos implícitamente a través de **this**, o explícitamente a través de un objeto de su clase; y si la clase derivada pertenece al mismo ensamblado que la clase base, también tiene acceso a sus miembros internos. Por ejemplo, el método *Comisiones* de la clase *CCuentaAhorro* no puede acceder al atributo *saldo* de la clase *CCuenta* porque es privado, pero sí puede acceder a su método público *Reintegro*.

```
public void Comisiones()
{
    // Se aplican mensualmente por el mantenimiento de la cuenta
    int día, mes, año;
    CFecha.ObtenerFechaActual(out día, out mes, out año);
    if (día == 1) Reintegro(cuotaMantenimiento);
}
```

Por lo tanto, si una clase derivada quiere acceder a los miembros privados de su clase base, debe hacerlo a través de la interfaz pública, protegida, o interna en su caso, de dicha clase base.

ATRIBUTOS CON EL MISMO NOMBRE

Como sabemos, una clase derivada puede acceder directamente a un atributo público, protegido, o interno en su caso, de su clase base. ¿Qué sucede si definimos en la clase derivada uno de estos atributos con el mismo nombre que tiene en la clase base? Por ejemplo, supongamos que una clase *ClaseA* define un atributo identificado por *atributo_x*, que después redefinimos en una clase derivada *ClaseB*:

```
public class ClaseA
{
    public int atributo_x = 1;

    public int método_x()
    {
        return atributo_x * 10;
    }
}
```

```
        public int método_y()
        {
            return atributo_x + 100;
        }
    }

    public class ClaseB : ClaseA
    {
        public int atributo_x = 2;

        public int método_x()
        {
            return atributo_x * -10;
        }
    }
```

La definición del atributo *atributo_x* en la clase derivada oculta la definición del atributo con el mismo nombre en la clase base. Por lo tanto, las referencias a *atributo_x* en el código del ejemplo siguiente devolverán el valor de *atributo_x* de la *ClaseB*. Si este atributo no hubiera sido definido en la clase derivada, entonces el valor devuelto sería el valor de *atributo_x* de la clase base.

```
public class Test
{
    public static void Main(string[] args)
    {
        ClaseB objClaseB = new ClaseB();
        Console.WriteLine(objClaseB.atributo_x); // escribe 2
        Console.WriteLine(objClaseB.método_y()); // escribe 101
        Console.WriteLine(objClaseB.método_x()); // escribe -20
    }
}
```

Ahora bien, ¿cómo procederíamos si el *método_x* de la clase *ClaseB* tuviera que acceder obligatoriamente al dato *atributo_x* de la clase base? La solución es sencilla: utilizar para ese atributo nombres diferentes en la clase base y en la clase derivada. No obstante, aun habiendo utilizado el mismo nombre, tenemos una alternativa de acceso: utilizar la palabra reservada **base**. Por ejemplo:

```
        public int método_x()
        {
            return base.atributo_x * -10;
        }
    }
```

Como se puede ver, desde la clase derivada, podemos referirnos al dato *atributo_x* de la clase base con la expresión:

```
base.atributo_x
```


Así mismo, también desde la clase derivada, podríamos referirnos al dato *atributo_x* de la propia clase derivada con la expresión:

```
this.atributo_x
```

En cambio, la expresión siguiente hace referencia al dato *atributo_x* de la *ClaseA*:

```
((ClaseA)this).atributo_x
```

La técnica de realizar una conversión explícita u obligada es la que tendremos que utilizar si necesitamos referirnos a un miembro oculto perteneciente a una clase por encima de la clase base (una clase base indirecta).

REDEFINIR MÉTODOS DE LA CLASE BASE

Cuando se invoca a un método en respuesta a un mensaje recibido por un objeto, C# busca su definición en la clase del objeto. El método que allí se encuentre puede pertenecer a la propia clase o puede haber sido heredado de alguna de sus clases bases (esto último equivale a decir que si no lo encuentra, C# sigue buscando hacia arriba en la jerarquía de clases hasta que lo localice).

Sin embargo, puede haber ocasiones en que deseemos que un objeto de una clase derivada responda al mismo método heredado de su clase base pero con un comportamiento diferente. Esto implica redefinir en la clase derivada el método heredado de su clase base.

Redefinir un método heredado significa volverlo a escribir en la clase derivada con el mismo nombre, la misma lista de parámetros y el mismo tipo del valor retornado que tenía en la clase base; su cuerpo será adaptado a las necesidades de la clase derivada. Esto es lo que se ha hecho con el *método_x* del ejemplo expuesto en el apartado anterior.

Se puede observar que este método ha sido redefinido en la *ClaseB* para que realice unos cálculos diferentes a los que realizaba en la *ClaseA*.

En el método **Main** de la clase *Test* del ejemplo anterior, se creó un objeto *objClaseB* y se invocó a su *método_y*. Como la clase del objeto, *ClaseB*, no define este método, C# ejecuta el heredado. Así mismo, se invocó a su *método_x*; en este caso, existe una definición para este método, que es la que se ejecuta.

Cuando en una clase derivada se redefine un método de una clase base, se oculta el método de la clase base, pero no las sobrecargas que existan del mismo

en dicha clase base. Si el método se redefine en la clase derivada con distinto tipo o número de parámetros, el método de la clase base no se oculta, sino que se comporta como una sobrecarga de ese método. Por ejemplo, el *método_x* tal cual lo hemos redefinido en la clase derivada oculta al método del mismo nombre de la clase base. Pero si lo hubiéramos definido con distinto número de parámetros, por ejemplo con uno, según se muestra a continuación, sería una sobrecarga.

```
public int método_x(int a) // método de ClaseB
{
    return atributo_x * -a;
}
```

El control de acceso de un método que se redefine puede modificarse en cualquier sentido; esto es, se puede hacer que sea más o menos restrictivo que el original. El orden de los tipos de control de acceso de más a menos restrictivo es así: **private**, **internal**, **protected** y **public**.

Para acceder a un método de la clase base que ha sido redefinido en la clase derivada, igual que se expuso para los atributos, tendremos que utilizar la palabra reservada **base**. Por ejemplo, suponga que añadimos el siguiente método a la *ClaseB*:

```
public int método_z()
{
    atributo_x = base.atributo_x + 3;
    return base.método_x() + atributo_x;
}
```

Como se puede observar, podemos referirnos al *método_x* de la clase base con la expresión:

```
base.método_x()
```

Es importante resaltar que **base** puede ser utilizado sólo desde dentro de la clase que proporciona los miembros redefinidos.

Así mismo, como ya vimos cuando se expuso **this**, podríamos referirnos al *método_x* de la clase derivada así:

```
this.método_x()
```

Recuerde que anteriormente expusimos que se puede utilizar el modificador **new** para ocultar explícitamente un miembro heredado de una clase base, aunque se permite ocultar miembros sin utilizar este modificador **new**, lo que generará una advertencia.

CONSTRUCTORES DE LAS CLASES DERIVADAS

Sabemos que cuando se crea un objeto de una clase se invoca a su constructor. También sabemos que los constructores de la clase base no son heredados por sus clases derivadas. En cambio, cuando se crea un objeto de una clase derivada, se invoca a su constructor, que a su vez invoca al constructor sin parámetros de la clase base, que a su vez invoca al constructor de su clase base, y así sucesivamente. Esto sucede así porque una clase derivada contiene todos los atributos de su clase base, y todos tienen que ser iniciados, razón por la que el constructor de la clase derivada tiene que llamar implícita o explícitamente al de la clase base.

Sin embargo, cuando se hayan definido constructores con parámetros tanto en las clases derivadas como en las clases bases, tal vez se desee construir un objeto de la clase derivada iniciándolo con unos valores determinados. En este caso, la definición ya conocida para los constructores de una clase cualquiera se extiende ahora para permitir al constructor de la clase derivada invocar explícitamente al constructor de la clase base para crear objetos de la clase derivada. Esto se hace utilizando la palabra clave **base** según la sintaxis especificada a continuación:

```
public nombre_clase derivada(parámetros-d) : base(parámetros-b)
{
    // cuerpo del constructor de la clase derivada
}
```

En la definición genérica anterior, correspondiente a un constructor con parámetros de una clase derivada, se observa, por una parte, la utilización de la palabra reservada **base** para invocar al constructor de la clase base, y por otra, el cuerpo del constructor de la clase derivada. No especificar una llamada explícita al constructor de la clase base equivale a:

```
public nombre_clase derivada(parámetros-d) : base()
{
    // cuerpo del constructor de la clase derivada
}
```

¿Cómo se ejecuta este código? El orden de ejecución es:

1. Se construyen los atributos del objeto de la clase derivada (los propios y los heredados).
2. Constructor de la clase base.
3. Cuerpo del constructor de la clase derivada.

Lo anterior se traduce en que un objeto de una clase derivada se construye de abajo hacia arriba. Primero se construyen sus atributos (los propios y los hereda-

dos) ejecutando, cuando sea necesario, los constructores de los mismos, y después, se pasa a ejecutar el cuerpo del constructor de la clase base y finalmente se ejecuta el cuerpo del constructor de la clase derivada. Este orden se aplica recursivamente por cada constructor de cada una de las clases.

Por ejemplo, aplicando la teoría expuesta, vamos a añadir a la clase *CCuentaAhorro* un constructor con parámetros. ¿Cuántos parámetros debe tener este constructor para iniciar todos los atributos de un objeto *CCuentaAhorro*? Pues tantos como atributos heredados y propios tenga la clase; en nuestro caso un objeto *CCuentaAhorro* contiene los atributos *nombre*, *cuenta*, *saldo*, *tipoDeInterés* y *cuotaMantenimiento*. Según esto, el constructor podría ser así:

```
public CCuentaAhorro(string nom, string cue, double sal,
double tipo, double mant) : base(nom, cue, sal, tipo)
{
    AsignarCuotaManten(mant);    // inicia cuotaMantenimiento
}
```

La palabra clave **base** del constructor anterior llama al constructor de *CCuenta*, clase base de *CCuentaAhorro*. Lógicamente, la clase *CCuenta* debe tener un constructor con cuatro parámetros del tipo de los argumentos especificados. Después, el cuerpo del constructor invoca al método *AsignarCuotaManten* para iniciar el atributo *cuotaMantenimiento* de *CCuentaAhorro*.

De acuerdo con los constructores definidos en la clase *CCuentaAhorro*, son declaraciones válidas las siguientes:

```
public class Test
{
    public static void Main(string[] args)
    {
        CCuentaAhorro cuenta01 = new CCuentaAhorro();
        CCuentaAhorro cuenta02 = new CCuentaAhorro("Un nombre",
                                                    "Una cuenta", 100000, 2.5, 1);
        // ...
    }
}
```

En este ejemplo, la sentencia primera requiere en *CCuentaAhorro* un constructor sin parámetros y en *CCuenta* otro. En cambio, la segunda sentencia requiere en *CCuentaAhorro* un constructor con parámetros y en *CCuenta* otro que se pueda invocar como se indica a continuación, con el fin de iniciar los atributos definidos en la clase base con los valores pasados como argumentos.

```
base(nombre, cuenta, saldo, tipoInterés);
```

Según lo expuesto, cuando se crea *cuenta01* o *cuenta02*, primero se construye la porción del objeto correspondiente a su clase base y a continuación la porción del objeto correspondiente a su clase derivada. Esto es una forma lógica de operar, ya que permite al constructor de la clase derivada hacer referencia, cuando sea preciso, a los atributos de su clase base que ya han sido iniciados.

Evidentemente, si sólo se desea iniciar algunos de los atributos de un objeto, hay que escribir los constructores adecuados tanto en la clase derivada como en la clase base.

Si la clase base no tiene un constructor de forma explícita o tiene uno que no requiere parámetros, no se necesita invocarlo explícitamente, ya que C# lo invocará automáticamente mediante **base()** sin argumentos. Por el contrario, si tiene un constructor con parámetros, será necesario invocarlo siempre que en la llamada al constructor de la clase derivada se especifiquen argumentos para iniciar atributos heredados de la clase base. En un caso como éste, si el constructor de la clase derivada no incluye una llamada al constructor de su clase base, invocará al constructor sin argumentos (si no lo hubiera, el compilador mostraría un error). Por ejemplo, supongamos que el constructor de *CCuentaAhorro* fuera así:

```
public CCuentaAhorro(string nom, string cue, double sal,
double tipo, double mant)
{
    AsignarCuotaManten(mant);    // inicia cuotaMantenimiento
}
```

En este caso, una declaración como la siguiente invocaría al constructor *CCuenta* sin argumentos, lo que supondría iniciar los atributos heredados de la clase base con los valores por omisión en vez de con los valores especificados.

```
CCuentaAhorro cuenta02 = new CCuentaAhorro("cliente02",
                                           "1111111111", 100000, 2.5, 1);
```

¿Sería correcto invocar al constructor de la clase base desde la primera línea del cuerpo del constructor de la clase derivada en lugar de hacerlo por medio de **base()**?

```
public CCuentaAhorro(String nom, String cue, double sal,
double tipo, double mant)
{
    new CCuenta(nom, cue, sal, tipo);
    AsignarCuotaManten(mant);    // inicia cuotaMantenimiento
}
```

La respuesta es no. Como ya se ha explicado, si el constructor fuera el de este ejemplo, primero invocaría al constructor *CCuenta* sin argumentos y después eje-

cutaría el código escrito en el cuerpo del mismo; la primera línea invocaría al constructor *CCuenta* creando un objeto temporal sin ningún efecto, y la segunda asignaría al objeto *CCuentaAhorro* la cuota de mantenimiento especificada.

COPIA DE OBJETOS

La copia de objetos, según vimos en el capítulo anterior, puede resolverse escribiendo los métodos: constructor copia y/o un método *Copiar*. Como una clase derivada contiene todos los atributos de su clase base, y todos tienen que ser copiados, la mejor solución es definir estos métodos tanto en las clases base como en las clases derivadas para que éstas puedan invocar a las versiones de sus clases base. A continuación se muestra un ejemplo de la utilización de estos métodos:

```
public static void Main(string[] args)
{
    CCuentaAhorro cuenta01 = new CCuentaAhorro();
    CCuentaAhorro cuenta02 = new CCuentaAhorro("cliente02",
                                                "1111111111", 200000, 1.75, 1);

    // ...
    cuenta01.Copiar(cuenta02);
    CCuentaAhorro cuenta03 = new CCuentaAhorro(cuenta02);

    // ...
}
```

¿Cómo sería el constructor copia y el método *Copiar* de la clase *CCuentaAhorro*? Si necesitamos definir estos métodos, podemos hacerlo así:

```
public CCuentaAhorro(CCuentaAhorro ca) : base(ca)
{
    cuotaMantenimiento = ca.cuotaMantenimiento;
}

public CCuentaAhorro Copiar(CCuentaAhorro ca)
{
    base.Copiar(ca);
    cuotaMantenimiento = ca.cuotaMantenimiento;
    return this;
}
```

Obsérvese cómo los métodos expuestos invocan a la versión del mismo en la clase base.

También, según lo estudiado en el capítulo de clases, el constructor copia podría escribirse así:

```
public CCuentaAhorro(CCuentaAhorro ca)
{
    Copiar(ca); // explícitamente: this.Copiar(ca);
}
```

Esta versión primero hace una llamada al constructor sin argumentos de *CCuenta* (**base()**) y después, llama a *CCuentaAhorro.Copiar* que, a su vez, llama a *CCuenta.Copiar*.

¿Cómo son el constructor copia y el método *Copiar* de la clase *CCuenta*? Si necesitamos definir estos métodos, podemos hacerlo así:

```
public CCuenta(CCuenta c)
{
    Copiar(c);
}

public CCuenta Copiar(CCuenta c)
{
    nombre = c.nombre;
    cuenta = c.cuenta;
    saldo = c.saldo;
    tipoDeInterés = c.tipoDeInterés;
    return this;
}
```

Entonces, ¿qué ocurre cuando se ejecuta una línea como la que se muestra sombreada a continuación?

```
CCuentaAhorro cuenta02 = new CCuentaAhorro("cliente02",
                                             "1111111111", 200000, 1.75, 1);
CCuentaAhorro cuenta03 = new CCuentaAhorro(cuenta02);
```

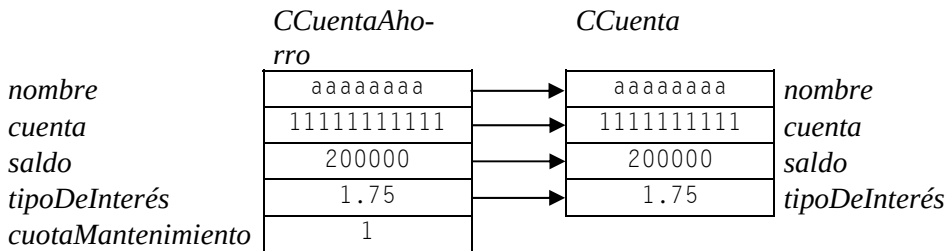
1. Se invoca al constructor copia *CCuentaAhorro* pasando como argumento el objeto *cuenta02*. Su parámetro *ca* referencia a este objeto.
2. Se invoca al constructor copia *CCuenta* pasando como argumento el objeto *CCuentaAhorro* referenciado por *ca*. Su parámetro *c* referencia a este objeto. Pero *c* es una referencia a un objeto *CCuenta* y todo ha funcionado correctamente. Esto es debido a que existe una conversión implícita de referencias a objetos de la clase derivada a sus correspondientes de la clase base. En el ejemplo, la referencia *ca* a *CCuentaAhorro* es convertida en una referencia a un objeto *CCuenta*.

El ejemplo siguiente utiliza el método *Copiar* para copiar el objeto *cuenta02* en *cuenta01*. La explicación de cómo sucede es análoga a la anterior.

```
CCuentaAhorro cuenta01 = new CCuentaAhorro();
CCuentaAhorro cuenta02 = new CCuentaAhorro("cliente02",
                                             "1111111111", 200000, 1.75, 1);
// ...
cuenta01.Copiar(cuenta02);
```

También, un objeto de una clase base pública puede ser iniciado con un objeto de una clase derivada de ella invocando a su constructor copia, o bien invocando a su método *Copiar* si se trata de una copia sobre un objeto existente. Por ejemplo, el código siguiente sería correcto:

```
CCuenta cuenta01 = new CCuenta();
CCuentaAhorro cuenta02 = new CCuentaAhorro("aaaaaaaa",
                                             "1111111111", 200000, 1.75, 1);
// ...
cuenta01.Copiar(cuenta02);
CCuenta cuenta03 = new CCuenta(cuenta02);
// ...
```



No necesitamos duplicar los objetos **string** porque cada cadena nueva que asignemos a cualquiera de ellos generará un nuevo objeto **string**.

DESTRUCTORES DE LAS CLASES DERIVADAS

El destructor de una clase base no es heredado por sus clases derivadas. En cuanto a cómo se destruyen los objetos de las clases derivadas diremos que son destruidos en el orden inverso a como son contruidos. Esto es, primeramente son llamados los destructores para sus miembros (para los heredados y para los propios), después se ejecuta el cuerpo del destructor de la clase derivada y por último se ejecuta el destructor de la clase base. Como los destructores no pasan argumentos, no requieren una sintaxis especial.

Por ejemplo, en la aplicación expuesta anteriormente, el destructor de la clase *CCuenta* está definido de la forma siguiente:

```
~CCuenta() { /* no hace nada */ }
```


Análogamente, el destructor de la clase *CCuentaAhorro* está definido así:

```
~CCuentaAhorro() { /* no hace nada */ }
```

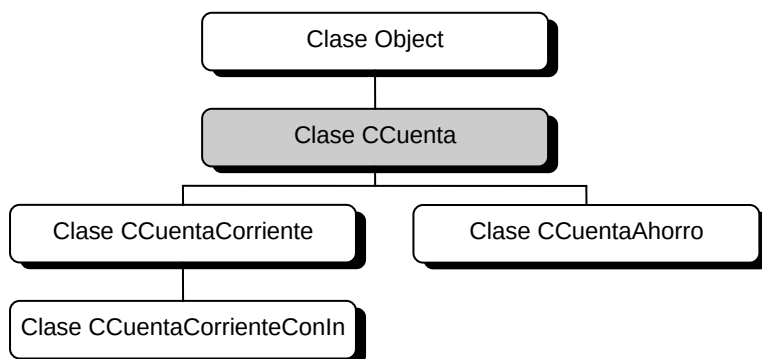
Siempre que se destruya un objeto de la clase *CCuentaAhorro*, primero se ejecutarán los destructores de sus atributos, después se ejecutará el cuerpo de *~CCuentaAhorro* y por último se ejecutará el destructor *~CCuenta*.

JERARQUÍA DE CLASES

Una clase derivada puede así mismo ser una clase base de otra clase, y así sucesivamente. En la siguiente figura se puede ver esto con claridad.

El conjunto de clases así definido da lugar a una *jerarquía de clases*. Cuando cada clase derivada lo es de una sola clase base, como ocurre en C#, la estructura jerárquica recibe el nombre de árbol de clases.

La raíz del árbol es la clase que representa el tipo más general, y las clases terminales en el árbol (nodos hoja) representan los tipos más especializados.



Las reglas que podemos aplicar para diseñar la clase *CCuentaCorriente* derivada de la clase base *CCuenta* o la clase *CCuentaCorrienteConIn* derivada de la clase base *CCuentaCorriente* son las mismas que hemos aplicado anteriormente para diseñar la clase *CCuentaAhorro* derivada de la clase base *CCuenta*, y lo mismo diremos para cualquier otra clase derivada que deseemos añadir. Esto quiere decir que para implementar una clase derivada como *CCuentaCorrienteConIn*, nos es suficiente con conocer a fondo su clase base *CCuentaCorriente* sin importarnos *CCuenta*.

Observe que la clase *CCuenta* actúa como clase base de más de una clase, concretamente de las clases *CCuentaAhorro* y *CCuentaCorriente*.

Como ejemplo, vamos a completar la jerarquía de clases expuesta con las clases que faltan: *CCuentaCorriente* y *CCuentaCorrienteConIn*.

La clase *CCuentaCorriente* es una nueva clase que hereda de la clase *CCuenta*. Por lo tanto, tendrá todos los miembros de su clase base, a los que añadiremos los siguientes:

Atributo	Significado
<i>transacciones</i>	Dato de tipo int que almacena el número de transacciones efectuadas sobre esa cuenta.
<i>importePorTrans</i>	Dato de tipo double que almacena el importe que la entidad bancaria cobrará por cada transacción.
<i>transExentas</i>	Dato de tipo int que almacena el número de transacciones gratuitas.

Método	Significado
<i>CCuentaCorriente</i>	Es el constructor de la clase. Inicia los atributos de la misma.
<i>CCuentaCorriente</i>	Constructor copia. Crea un objeto a partir del objeto pasado como argumento.
<i>Copiar</i>	Copia el objeto pasado como argumento en el objeto que recibe el mensaje <i>Copiar</i> .
<i>DecrementarTransacciones</i>	Decrementa en 1 el número de transacciones.
<i>AsignarImportePorTrans</i>	Establece el importe por transacción. Retorna false si el importe es negativo, y true en otro caso.
<i>ObtenerImportePorTrans</i>	Devuelve el importe por transacción.
<i>AsignarTransExentas</i>	Establece el número de transacciones exentas. Retorna false si el número de transacciones es negativo, y true en otro caso.
<i>ObtenerTransExentas</i>	Devuelve el número de transacciones exentas.
<i>Ingreso</i>	Añade la cantidad especificada al saldo actual de la cuenta e incrementa el número de transacciones.
<i>Reintegro</i>	Resta la cantidad especificada del saldo actual de la cuenta e incrementa el número de transacciones.
<i>Comisiones</i>	Se ejecuta el día 1 de cada mes para cobrar el importe de las transacciones efectuadas que no estén exentas y pone el número de transacciones a cero.
<i>Intereses</i>	Se ejecuta el día 1 de cada mes para calcular el importe correspondiente a los intereses/mes producidos y añadirlo al saldo. Hasta 3.000 euros al 0,5%. El resto al interés establecido.

Aplicando la teoría expuesta hasta ahora y procediendo de forma similar a como lo hicimos para construir la clase derivada *CCuentaAhorro*, la definición de la clase *CCuentaCorriente* es la siguiente:

```

////////////////////////////////////
// Clase CCuentaCorriente: clase derivada de CCuenta
//
public class CCuentaCorriente : CCuenta
{
    // Atributos
    private int transacciones;
    private double importePorTrans;
    private int transExentas;

    // Métodos
    public CCuentaCorriente() {} // constructor sin parámetros

    public CCuentaCorriente(String nom, String cue, double sal,
        double tipo, double imptrans, int transex)
        : base(nom, cue, sal, tipo) // invoca al constructor CCuenta
    {
        transacciones = 0; // inicia transacciones
        AsignarImportePorTrans(imptrans); // inicia importePorTrans
        AsignarTransExentas(transex); // inicia transExentas
    }

    public CCuentaCorriente(CCuentaCorriente cc) : base(cc)
    {
        transacciones = cc.transacciones;
        importePorTrans = cc.importePorTrans;
        transExentas = cc.transExentas;
    }

    public CCuentaCorriente Copiar(CCuentaCorriente cc)
    {
        base.Copiar(cc);
        transacciones = cc.transacciones;
        importePorTrans = cc.importePorTrans;
        transExentas = cc.transExentas;

        return this;
    }

    public void DecrementarTransacciones()
    {
        transacciones--;
    }

    public bool AsignarImportePorTrans(double imptrans)
    {

```

```
        if (imptrans < 0)
        {
            Console.WriteLine("Error: cantidad negativa");
            return false;
        }

        importePorTrans = imptrans;
        return imptrans >= 0;
    }

    public double ObtenerImportePorTrans()
    {
        return importePorTrans;
    }

    public bool AsignarTransExentas(int transex)
    {
        if (transex < 0)
        {
            Console.WriteLine("Error: cantidad negativa");
            return false;
        }
        transExentas = transex;
        return transex >= 0;
    }

    public int ObtenerTransExentas()
    {
        return transExentas;
    }

    public bool Ingreso(double cantidad)
    {
        if (base.Ingreso(cantidad))
        {
            transacciones++;
            return true;
        }
        else
            return false;
    }

    public void Reintegro(double cantidad)
    {
        base.Reintegro(cantidad);
        transacciones++;
    }

    public void Comisiones()
    {
        // Se aplican mensualmente por el mantenimiento de la cuenta
    }
}
```

```

    int día, mes, año;
    CFecha.ObtenerFechaActual(out día, out mes, out año);
    if (día == 1)
    {
        int n = transacciones - transExentas;
        if (n > 0) Reintegro(n * importePorTrans);
        transacciones = 0;
    }
}

public double Intereses()
{
    int día, mes, año;
    CFecha.ObtenerFechaActual(out día, out mes, out año);

    if (día != 1) return 0.0;

    // Acumular los intereses por mes sólo los días 1 de cada mes
    double interesesProducidos = 0.0;
    // Hasta 3000 euros al 0.5%. El resto al interés establecido.
    if (Saldo() <= 3000)
        interesesProducidos = Saldo() * 0.5 / 1200.0;
    else
    {
        interesesProducidos = 3000 * 0.5 / 1200.0 +
            (Saldo() - 3000) * ObtenerTipoDeInterés() / 1200.0;
    }
    Ingreso(interesesProducidos);
    // Este ingreso no debe incrementar las transacciones
    DecrementarTransacciones();

    // Devolver el interés mensual por si fuera necesario
    return interesesProducidos;
}
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

```

Se puede observar que el constructor de la clase *CCuentaCorriente* tiene los parámetros necesarios para iniciar sus datos miembro, excepto *transacciones* que inicialmente vale 0, y los heredados de su clase base. Dicho constructor llama primero al constructor de su clase base y después a los métodos de la propia clase que permiten iniciar de forma segura los atributos de la misma. También se ha implementado un constructor sin parámetros, un constructor copia y el método *copiar*.

Procediendo de forma similar a como lo hemos hecho para las clases *CCuentaAhorro* y *CCuentaCorriente*, construimos a continuación la clase *CCuentaCorrienteConIn* (cuenta corriente con intereses) derivada de *CCuentaCorriente*.

Supongamos que este tipo de cuenta se ha pensado para incentivar con una rentabilidad mayor respecto a *CCuentaCorriente* a los clientes que conserven un saldo mínimo en su cuenta.

Digamos que se trata de una cuenta de tipo *CCuentaCorriente* que precisa un saldo mínimo de 3.000 euros para que pueda acumular intereses. Según esto, *CCuentaCorrienteConIn*, además de los miembros heredados, sólo precisa implementar sus constructores y variar el método *Intereses*:

Método	Significado
<i>CCuentaCorrienteConIn</i>	Es el constructor de la clase. Inicia los atributos de un objeto <i>CCuentaCorrienteConIn</i> .
<i>CCuentaCorrienteConIn</i>	Constructor copia. Crea un objeto a partir del objeto pasado como argumento.
<i>Copiar</i>	Copia el objeto pasado como argumento en el objeto que recibe el mensaje <i>Copiar</i> .
<i>Intereses</i>	Permite calcular el importe/mes correspondiente a los intereses producidos. Precisa un saldo mínimo de 3.000 euros.

La definición correspondiente a esta clase se expone a continuación:

```

////////////////////////////////////
// Clase CCuentaCorrienteConIn: clase derivada de CCuentaCorriente
//
public class CCuentaCorrienteConIn : CCuentaCorriente
{
    // Métodos
    public CCuentaCorrienteConIn() {} // constructor sin parámetros

    public CCuentaCorrienteConIn(String nom, String cue, double sal,
    double tipo, double imptrans, int transex)
    : base(nom, cue, sal, tipo, imptrans, transex)
    {
    }

    public CCuentaCorrienteConIn(CCuentaCorrienteConIn cci) : base(cci)
    {
    }

    public CCuentaCorrienteConIn Copiar(CCuentaCorrienteConIn cci)
    {
        base.Copiar(cci);

        return this;
    }
}

```

```

public double Intereses()
{
    int día, mes, año;
    CFecha.ObtenerFechaActual(out día, out mes, out año);

    if (día != 1 || Saldo() < 3000) return 0.0;
    // Acumular interés mensual sólo los días 1 de cada mes
    double interesesProducidos = 0.0;
    interesesProducidos = Saldo() * ObtenerTipoDeInterés() / 1200.0;
    Ingreso(interesesProducidos);
    // Este ingreso no debe incrementar las transacciones
    DecrementarTransacciones();

    // Devolver el interés mensual por si fuera necesario
    return interesesProducidos;
}
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

```

La clase *CCuenta* es la “clase base directa” (o simplemente clase base) de *CCuentaAhorro* y de *CCuentaCorriente*, y es una “clase base indirecta” para *CCuentaCorrienteConIn*. Mientras que una clase derivada tiene una única clase base directa, puede tener varias clases bases indirectas: todas las que haya en el camino para llegar desde su clase base hasta la clase raíz; esto es importante porque lo que una clase derivada hereda de su clase base, será heredado a su vez por una clase derivada de ella, y así sucesivamente.

Una clase derivada que redefina un método heredado sólo tiene acceso a su propia versión y a la publicada por su clase base directa. Por ejemplo, las clases *CCuenta* y *CCuentaCorriente* incluyen cada una su versión del método *Ingreso*; y la clase derivada *CCuentaCorrienteConIn* hereda el método *Ingreso* de *CCuentaCorriente*. Entonces, *CCuentaCorrienteConIn*, además de a su propia versión, sólo puede acceder a la versión de su clase base directa por medio de la palabra clave **base** (en este caso ambas versiones son la misma), pero no puede acceder a la versión de su clase base indirecta *CCuenta* (**base.base** no es una expresión admitida por el compilador C#).

Según lo expuesto las líneas de código:

```

Ingreso(interesesProducidos);
DecrementarTransacciones();

```

del método *Intereses* de la clase *CCuentaCorriente* podrían ser sustituidas por la indicada a continuación, puesto que el método *Ingreso* de *CCuenta* no actúa sobre las transacciones:

```

base.Ingreso(interesesProducidos);

```

En cambio, en el método *Intereses* de la clase *CCuentaCorrienteConIn* no podemos proceder de la misma forma porque desde esta clase no se puede acceder a la versión de *Ingreso* de *CCuenta*.

A continuación se presenta una aplicación con algunos ejemplos de operaciones con objetos de las clases pertenecientes a la jerarquía construida:

```
public class Test
{
    public static void Main(string[] args)
    {
        CCuentaAhorro cuenta01 = new CCuentaAhorro(
            "cliente 01", "1111111111", 10000, 3.5, 1);

        Console.WriteLine(cuenta01.ObtenerNombre());
        Console.WriteLine(cuenta01.ObtenerCuenta());
        Console.WriteLine(cuenta01.Saldo());
        Console.WriteLine(cuenta01.ObtenerTipoDeInterés());
        Console.WriteLine(cuenta01.Intereses());

        CCuentaCorrienteConIn cuenta02 = new CCuentaCorrienteConIn();
        cuenta02.AsignarNombre("cliente 02");
        cuenta02.AsignarCuenta("1234567890");
        cuenta02.AsignarTipoDeInterés(3.0);
        cuenta02.AsignarTransExentas(0);
        cuenta02.AsignarImportePorTrans(1.0);

        cuenta02.Ingreso(20000);
        cuenta02.Reintegro(10000);
        cuenta02.Intereses();
        cuenta02.Comisiones();
        Console.WriteLine(cuenta02.ObtenerNombre());
        Console.WriteLine(cuenta02.ObtenerCuenta());
        Console.WriteLine(cuenta02.Saldo());

        CCuentaCorrienteConIn cuenta03 =
            new CCuentaCorrienteConIn("cliente01", "1234567891",
                10000, 3.5, 1.0, 6);
        CCuentaCorrienteConIn cuenta04 =
            new CCuentaCorrienteConIn(cuenta03);
        CCuentaCorrienteConIn cuenta05 = new CCuentaCorrienteConIn();
        cuenta05.Copiar(cuenta04);
    }
}
```

En la aplicación anterior se puede observar cómo el método **Main** construye dos objetos: *cuenta01* de la clase *CCuentaAhorro* y *cuenta02* de la clase *CCuentaCorrienteConIn*. Para construir *cuenta01* se ha utilizado el constructor *CCuentaAhorro* con argumentos. Una vez construido, observe que responde a una serie

de mensajes ejecutando los métodos del mismo nombre, unos heredados de su clase base, como *ObtenerNombre*, y otros propios, como *Intereses*. En cambio, para construir *cuenta02* se ha utilizado el constructor *CCuentaCorrienteConIn* sin argumentos. Una vez construido, se puede también observar que responde a una serie de mensajes ejecutando los métodos del mismo nombre, unos heredados de su clase base directa, como *Reintegro*, otros heredados de su clase base indirecta, como *AsignarNombre*, y otros propios, como *Intereses*. Para finalizar, se han construido otros objetos *CCuentaCorrienteConIn* para probar cómo trabajan los constructores copia y el método *Copiar*.

Finalmente, indicar que aunque en ninguna clase de nuestra jerarquía han intervenido miembros **static**, su comportamiento en cuanto a la herencia se refiere es el mismo que el de los otros miembros, pero teniendo presente que son miembros de la clase; y si es necesario, cuando se trate de métodos, también pueden ser redefinidos, aunque, en este caso, el nombre de la clase indicará la versión del método que se invocará. Una advertencia: si definiera, por ejemplo, en *CCuenta* el atributo *tipoDeInterés static*, lógicamente se mantendría una única copia que utilizarían tanto los objetos de *CCuenta* como los de sus clases derivadas.

REFERENCIAS A OBJETOS DE UNA CLASE DERIVADA

Las referencias a objetos de una clase derivada pueden ser declaradas y manipuladas de la misma forma que las referencias a objetos de una clase cualquiera, tal y como ya expusimos en capítulos anteriores. Veamos algunos ejemplos basados en la jerarquía de clases que acabamos de construir:

```
public static void fa(CCuentaCorriente c)
{
    string cuenta = c.ObtenerCuenta();
    double saldo = c.Saldo();
    // ...
}

public static void Main(string[] args)
{
    CCuentaCorriente cuenta01 = new CCuentaCorriente("cliente01",
        "1234567890", 10000, 3.5, 1, 6);
    fa(cuenta01); //--> CCuentaCorriente c = cuenta01;
}
```

El método **Main** de este ejemplo declara un objeto *cuenta01* de la clase *CCuentaCorriente* derivada de *CCuenta*. Después invoca al método *fa* que define una referencia *c* al objeto *cuenta01* pasado como argumento. Una vez que disponemos de la referencia a un objeto podemos trabajar con él como lo hemos venido haciendo hasta ahora, según muestra el método *fa*.

Conversiones implícitas

El ejemplo anterior no aporta nada que nos sorprenda; operaciones como éstas ya han sido expuestas anteriormente. Pero, ¿qué pasaría si *cuenta01* fuera un objeto de la clase derivada *CCuentaCorrienteConIn* de *CCuentaCorriente*? Por ejemplo:

```
public static void Main(string[] args)
{
    CCuentaCorrienteConIn cuenta01;
    cuenta01 = new CCuentaCorrienteConIn("cliente01","1234567890",
                                           10000, 3.5, 1.0, 6);
    fa(cuenta01);
    // ...
}
```

Si ejecutamos este ejemplo, comprobaremos que los resultados obtenidos son los mismos que obtuvimos con el ejemplo anterior. Esto es así porque C# permite convertir implícitamente una referencia a un objeto de una clase derivada en una referencia a su clase base directa o indirecta. Veamos otro ejemplo:

```
public static void fa(CCuenta c)
{
    string cuenta = c.ObtenerCuenta();
    string nombre = c.ObtenerNombre();
    // ...
}

public static void Main(string[] args)
{
    CCuentaCorriente cuenta01 =
        new CCuentaCorriente("cliente01", "1234567891",
                              10000, 3.5, 1.0, 6);

    CCuentaCorrienteConIn cuenta02 =
        new CCuentaCorrienteConIn("cliente02", "1234567892",
                                   20000, 2.0, 1.0, 6);

    fa(cuenta01);
    fa(cuenta02);
    // ...
}
```

En el ejemplo anterior el método *fa* declara una referencia a un objeto *CCuenta*, la cual utilizamos después para referenciar indistintamente a un objeto *cuenta01* de la clase *CCuentaCorriente*, o a un objeto *cuenta02* de la clase *CCuentaCorrienteConIn*.

Cuando accedemos a un objeto por medio de una variable no del tipo del objeto, sino del tipo de alguna de sus clases base (directas o indirectas) según muestra el ejemplo anterior, es el tipo de la variable el que determina qué mensajes puede recibir el objeto referenciado; dicho de otra forma, es este tipo el que determina qué métodos pueden ser invocados por el objeto referenciado. ¿Cuáles son esos métodos? Pues los correspondientes al tipo de la variable que utilizamos para hacer referencia al objeto, no los de la clase del objeto.

Resumiendo: cuando accedemos a un objeto de una clase derivada por medio de una referencia a su clase base, ese objeto sólo puede ser manipulado por los métodos de su clase base. Por ejemplo, modifiquemos el método *fa* como se muestra a continuación (**Main** no se modifica):

```
public static void fa(CCuenta c)
{
    c.AsignarImportePorTrans(0.90); // error
    c.AsignarTransExentas(10);      // error
    // ...
}
```

Este último ejemplo sigue la misma pauta que el anterior. Pero ahora observamos que un intento de acceder al método *AsignarImportePorTrans* ocasiona un error de compilación. Esto es porque el tipo de la variable *c*, que es *CCuenta*, determina que el objeto referenciado por ella sólo puede recibir mensajes de la clase de dicha variable; dicho de otra forma, sólo puede ser manipulado por métodos de la clase *CCuenta* (propios y heredados). Lo mismo diremos respecto al mensaje *AsignarTransExentas*.

Así mismo, cuando se invoca a un método que está definido en la clase base y redefinido en sus clases derivadas, la versión que se ejecuta depende también del tipo de la variable que lo referencia, no de la clase del objeto referenciado. Por ejemplo, modifiquemos el método *fa* como se muestra a continuación (**Main** no se modifica):

```
public static void fa(CCuenta c)
{
    double intereses = c.Intereses();
    // ...
}
```

En el ejemplo anterior el método *fa* declara una referencia a un objeto *CCuenta*, la cual utilizamos después (véase el método **Main** anterior) para referenciar indistintamente a un objeto *cuenta01* de la clase *CCuentaCorriente*, o a un objeto *cuenta02* de la clase *CCuentaCorrienteConIn*. Por otra parte, el método *Intereses* está definido en la clase base *CCuenta* y redefinido en sus clases derivadas

CCuentaCorriente y *CCuentaCorrienteConIn*, pero observamos que independientemente del objeto referenciado, la expresión *c.Intereses()* invocará a *CCuenta.Intereses()*.

Conversiones explícitas

La conversión contraria, esto es, de una referencia a un objeto de la clase base a una referencia a su clase derivada, no se puede hacer, aunque se fuerce a ello utilizando una construcción *cast* (el error, en este caso, se detecta durante la ejecución), excepto cuando el objeto al que se tiene acceso a través de la referencia a la clase base es un objeto de esa clase derivada. Por ejemplo:

```
CCuentaCorriente cuenta01 =
    new CCuentaCorriente("cliente01", "1234567891",
                          10000, 3.5, 1.0, 6);
CCuentaCorrienteConIn cuenta;

cuenta = cuenta01; // error de compilación: conversión implícita
                  // no permitida

// La siguiente línea durante la ejecución lanza una excepción de
// tipo System.InvalidCastException: No se puede convertir un objeto
// de tipo 'CCuentaCorriente' al tipo 'CCuentaCorrienteConIn'
cuenta = (CCuentaCorrienteConIn)cuenta01;
// La línea anterior sería válida si cuenta01 referenciara a un
// objeto de la clase de cuenta, esto es, CCuentaCorrienteConIn:
// CCuentaCorriente cuenta01 =
//     new CCuentaCorrienteConIn("cliente01", "1234567891",
//                               10000, 3.5, 1.0, 6);
```

INFORMACIÓN DE TIPOS DURANTE LA EJECUCIÓN

Según hemos visto anteriormente, cualquier operación con referencias a objetos requiere que se tenga una referencia de un tipo apropiado para el objeto.

El operador **is** de C#, cuya sintaxis es: *objeto is tipo* devuelve **true** si el *objeto* proporcionado se puede convertir al *tipo* proporcionado sin producir una excepción y **false** en caso contrario. Según esto, la excepción que lanza el ejemplo anterior, por no ser el objeto referenciado por *cuenta01* compatible con la clase de *cuenta*, podría evitarse así:

```
if (cuenta01 is CCuentaCorrienteConIn)
    cuenta = (CCuentaCorrienteConIn)cuenta01;
```

MÉTODOS VIRTUALES

En los apartados anteriores hemos visto que cuando se invoca a un método que está definido en la clase base y redefinido en sus clases derivadas, la versión que se ejecuta depende del tipo de la referencia que se utilice para invocar al mismo.

Por ejemplo, si echamos una ojeada a la clase *CCuenta* y a sus clases derivadas, definidas anteriormente, observamos que el método *Intereses* de la clase *CCuenta* ha sido redefinido en todas sus clases derivadas, directas o indirectas (el método *Comisiones* también, excepto en *CCuentaCorrienteConIn*). Si ahora ejecutamos el código siguiente:

```
public class Test
{
    public static void fa(CCuenta c)
    {
        double intereses = c.Intereses();
        // ...
    }

    public static void Main(string[] args)
    {
        CCuentaCorriente cuenta01 = new CCuentaCorriente("cliente01",
            "1234567891", 10000, 3.5, 1.0, 6);
        fa(cuenta01);
    }
}
```

el comportamiento del compilador es el esperado, aunque quizás no el deseado, ya que el método *Intereses* que se ejecuta pertenece a la clase *CCuenta*, que como podemos observar es el tipo de la referencia *c*, cuando quizás deseábamos que se ejecutase el método *Intereses* de la clase del objeto referenciado.

La solución al problema planteado pasa porque sea el mismo sistema el que se encargue de la identificación durante la ejecución de la clase de los objetos apuntados, mecanismo que C# proporciona por medio de los métodos *virtuales*.

Un método *virtual* es un miembro de una clase base que puede ser redefinido (**override**) en cada una de las clases derivadas de ésta, y una vez redefinido puede ser accedido mediante una referencia a la clase base, resolviéndose la llamada en función del tipo del objeto referenciado. Una clase con métodos virtuales se denomina *tipo polimórfico*.

Un método se declara *virtual* escribiendo el modificador **virtual** en la declaración del método en la clase base:

```
public virtual tipo nombre_método(parámetros)
{
    // Cuerpo del método
}
```

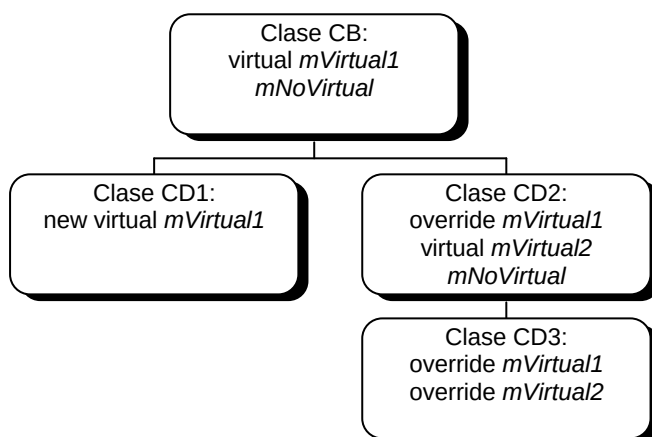
Cuando se declara un método virtual, toda clase que hereda el método puede implementar su propia versión. La clase derivada puede entonces reemplazar (**override**) el método base virtual o lo puede ocultar (**new**). Si se especifica la palabra clave **new** se indica al compilador que esta definición oculta la definición contenida en la clase base; éste es el comportamiento predeterminado, esto es, cuando no se especifica la palabra clave **override** ni la palabra clave **new**, el compilador emite una advertencia y el método de la clase derivada oculta el método de la clase base, aspecto que ya estudiamos anteriormente. Si se especifica la palabra clave **override** se indica al compilador que esta definición es la versión aportada por esta clase derivada del método virtual de su clase base.

Cuando durante la ejecución se invoca a un método virtual a través de una referencia a una clase base, se comprueba en la clase del objeto referenciado si hay una versión del mismo calificada **override**. Si la hay se ejecuta y si no, se asciende por la jerarquía de clases en busca de una, la cual podría llegar a ser el método original de esa clase base calificado virtual. Esto quiere decir que los métodos virtuales deben ser redefinidos en las clases que necesiten aportar su propia versión utilizando el modificador **override**, de lo contrario, no se aplicará el mecanismo virtual:

```
public override tipo nombre_método(parámetros)
{
    // Cuerpo del método
}
```

La redefinición de un método virtual en una clase derivada debe tener el mismo nombre, número y tipos de parámetros, y tipo del valor retornado que en la clase base; en otro caso, se producirá un error. Además, una clase derivada puede contener sus propios métodos virtuales; esto es, métodos virtuales no heredados de su clase base.

El siguiente ejemplo muestra de una forma práctica cómo se implementan métodos virtuales y cómo se ejecutan las llamadas a métodos virtuales y no virtuales. La jerarquía de clases utilizada en este ejemplo es la de la figura siguiente:



El código para poner en práctica la jerarquía de clases expuesta con los métodos descritos puede ser el siguiente:

```
public class CB
{
    public virtual void mVirtual1() // método virtual
    {
        Console.WriteLine("método virtual 1 en CB");
    }
    public void mNoVirtual() // método no virtual
    {
        Console.WriteLine("método no virtual en CB");
    }
}
```

```
class CD1 : CB
{
    public new virtual void mVirtual1() // método virtual
    {
        Console.WriteLine("método virtual 1 en CD1");
    }
}
```

```
public class CD2 : CB
{
    public override void mVirtual1() // método virtual redefinido
    {
        Console.WriteLine("método virtual 1 en CD2");
    }
    public virtual void mVirtual2() // método virtual
    {
        Console.WriteLine("método virtual 2 en CD2");
    }
}
```

```
public new void mNoVirtual()      // método no virtual
{
    Console.WriteLine("método no virtual en CD2");
}
}

class CD3 : CD2
{
    public override void mVirtual1() // método virtual redefinido
    {
        Console.WriteLine("método virtual 1 en CD3");
    }
    public override void mVirtual2() // método virtual redefinido
    {
        Console.WriteLine("método virtual 2 en CD3");
    }
}

public class Test
{
    public static void Main(string[] args)
    {
        CB[] mrCB = new CB[4] {new CB(),new CD1(),new CD2(),new CD3()};

        foreach (CB r in mrCB)
        {
            r.mVirtual1();
            r.mNoVirtual();
        }
        Console.WriteLine("-----");
        CD1 rCD1 = new CD1();
        rCD1.mVirtual1();
        Console.WriteLine("-----");
        CD2[] mrCD2 = new CD2[2] { new CD2(), new CD3()};
        foreach (CD2 r in mrCD2)
            r.mVirtual2();
    }
}
```

Los resultados que produce el programa anterior son los siguientes:

```
método virtual 1 en CB
método no virtual en CB
método virtual 1 en CB
método no virtual en CB
método virtual 1 en CD2
método no virtual en CB
método virtual 1 en CD3
método no virtual en CB
-----
```


método virtual 1 en CD1

método virtual 2 en CD2

método virtual 2 en CD3

Observe cómo una llamada a un método virtual se resuelve en función del tipo del objeto referenciado y cómo una llamada a un método no virtual se resuelve en función del tipo de la referencia. Fíjese también que la clase *CD1* define su propio método virtual (**new virtual**; **new** oculta el método base virtual) y en la clase *CD2* que redefine el método virtual heredado (**override** reemplaza el método base virtual) y aporta otro método virtual nuevo.

Resumiendo lo expuesto hasta aquí y después de un análisis de los resultados del ejercicio anterior, se deduce que:

- Una llamada a un método virtual se resuelve siempre en función del tipo del objeto referenciado.
- Una llamada a un método normal (no virtual) se resuelve en función del tipo de la referencia.

Según lo expuesto, para declarar virtual los métodos *Comisiones* e *Intereses* de la clase *CCuenta*, clase raíz de la jerarquía de clases que hemos construido en este capítulo, edite esta clase y proceda como se indica a continuación:

```
public class CCuenta
{
    // Atributos
    // ..

    // Métodos
    // ..
    public virtual void Comisiones()
    {
        return; // sin comisiones
    }

    public virtual double Intereses()
    {
        return 0.0; // sin intereses
    }
    // ..
}
```

Obsérvese que tanto el método *Comisiones* como *Intereses* se han declarado virtuales en la clase base. ¿Con qué intención? Pues para poder redefinirlos en las clases derivadas de *CCuenta* y que sea el sistema el que resuelva la llamada a una

u otra forma de esos métodos en función del objeto referenciado en dicha llamada. Estas múltiples formas de un mismo método, esto es, las distintas redefiniciones en la clase derivada, tienen que calificarse como **override**:

```
public class CCuentaAhorro : CCuenta
{
    // ..
    public override void Comisiones()
    {
        // ...
    }

    public override double Intereses()
    {
        // ...
    }
    // ..
}
```

```
public class CCuentaCorriente : CCuenta
{
    // ..
    public override void Comisiones()
    {
        // ...
    }

    public override double Intereses()
    {
        // ...
    }
    // ..
}
```

```
public class CCuentaCorrienteConIn : CCuentaCorriente
{
    // ..
    public override double Intereses()
    {
        // ...
    }
    // ..
}
```

Una vez realizadas las modificaciones anteriores, volvemos a ejecutar el ejemplo con el que empezábamos este apartado:

```
public class Test
{
```

```

public static void fa(CCuenta c)
{
    double intereses = c.Intereses();
    // ...
}

public static void Main(string[] args)
{
    CCuentaCorriente cuenta01 = new CCuentaCorriente("cliente01",
        "1234567891", 10000, 3.5, 1.0, 6);
    fa(cuenta01);
}
}

```

y comprobaremos que:

c.Intereses() llama al método *CCuentaCorriente.Intereses()*

Por lo tanto, el método invocado pertenece, como deseábamos, a la misma clase que el objeto referenciado por *c* (en el ejemplo, el objeto referenciado es *cuenta01* de la clase *CCuentaCorriente*). Quiere esto decir que el mecanismo *virtual* garantiza que el objeto será manipulado por los métodos de su clase.

Si una clase derivada no provee una redefinición de un método declarado *virtual* en su clase base, una llamada al mismo hace que se ejecute el definido en su clase base. Por ejemplo, si *CCuentaCorriente* no redefiniera el método *Intereses*, la llamada *c.Intereses()* del ejemplo anterior invocaría al método *Intereses* de la clase *CCuenta*. Éste es el motivo de por qué un método *virtual* se declara en la clase base. Dicho de otra forma, cuando la clase derivada no redefine el método *virtual* de su clase base, hereda la implementación de la clase base, de manera que una llamada al mismo a través de un objeto de la clase derivada hace que se ejecute el método *virtual* de su clase base.

Una declaración **override** no puede cambiar la accesibilidad del método *virtual*, esto es, el método **override** y el método *virtual* deben tener el mismo modificador de nivel de acceso.

Un método **static** no puede ser declarado **virtual**, ya que un método *virtual* sólo es llamado para objetos de su clase.

Constructores virtuales

¿Pueden definirse constructores *virtuales*? Pensemos sobre ello. Un método *virtual* tiene que invocarse para un objeto existente, pero un constructor no, ya que su función es construir el objeto; desde este punto de vista, no tiene sentido hablar

de un constructor virtual. Un método virtual es invocado a través de una referencia, pero la forma del método que se invoca será una u otra en función del tipo del objeto referenciado. En cambio, un constructor es exclusivo de un tipo exacto de objetos, otra razón más por la que no puede ser virtual. Entonces, la respuesta a la pregunta inicial es que C# no admite constructores virtuales, pero resulta fácil simularlos. Supongamos una clase *CB*:

```
CB obj = new CB(); // invoca al constructor CB
```

Esta operación podría también formar parte del cuerpo de un método que devuelva el objeto construido:

```
public CB Nuevo()  
{  
    return new CB(); // invoca al constructor CB  
}
```

Si este método lo declaramos virtual en una clase base *CB* y lo redefinimos en una clase derivada *CD* para que devuelva un objeto *CD*, tendremos la posibilidad de crear un nuevo objeto sin conocer su tipo exacto. Por lo tanto, *Nuevo* es un método que simula a un constructor virtual.

Análogamente, podemos escribir otro método virtual *Clonar* que devuelva un duplicado del objeto para el cual es invocado, objeto que sabemos está referenciado por **this**. Tenemos así otro método que simula a un constructor copia virtual.

```
public CB Clonar()  
{  
    return new CB(this); // invoca al constructor copia de CB  
}  
  
public CB(CB obj) // constructor copia  
{  
    // Cuerpo del método  
}
```

El programa que se muestra a continuación, pone en práctica lo expuesto:

```
using System;  
  
public class CB  
{  
    private int i;  
  
    // Constructor por omisión  
    public CB() { Console.WriteLine("constructor CB"); }
```

```

// Constructor copia
public CB(CB obj)
{
    Console.WriteLine("constructor copia CB");
    i = obj.i;
}
// Destructor
~CB() { Console.WriteLine("destructor CB"); }
// ...
public virtual CB Nuevo() { return new CB(); }
public virtual CB Clonar() { return new CB(this); }
}

```

```

public class CD : CB
{
    private double d;
    // Constructor por omisión
    public CD() { Console.WriteLine("constructor CD"); }
    // Constructor copia
    public CD(CD obj) : base(obj)
    {
        Console.WriteLine("constructor copia CD");
        d = obj.d;
    }
    // Destructor
    ~CD() { Console.WriteLine("destructor CD"); }
    // ...
    public override CB Nuevo() { return new CD(); }
    public override CB Clonar() { return new CD(this); }
}

```

```

public class Test
{
    public static CB CrearObjeto(CB obj)
    {
        return obj.Nuevo();
    }

    public static void Main(string[] args)
    {
        CB cb = new CB(); // semilla para CrearObjeto
        CD cd = new CD(); // semilla para CrearObjeto

        CB[] obj = new CB[4];
        obj[0] = CrearObjeto(cb); // crea un objeto de tipo CB
        obj[1] = CrearObjeto(cd); // crea un objeto de tipo CD
        obj[2] = obj[0].Clonar();
        obj[3] = obj[1].Clonar();
    }
}

```

Obsérvese cómo la función *CrearObjeto* crea un nuevo objeto de la misma clase que el referenciado por su parámetro, y cómo el método *Clonar* crea una copia del objeto para el que se invoca dicho método.

Se puede observar también que el tipo del objeto devuelto por los métodos *Nuevo* y *Clonar* de la clase derivada *CD* es *CB*, para cumplir el requisito de que la redefinición de un método virtual en una clase derivada debe tener el mismo nombre, número y tipos de parámetros, y tipo del valor retornado que en la clase base.

Destructores virtuales

No cabe hablar de destructores virtuales porque es el recolector de basura el que tiene la tarea de realizar el seguimiento de cada objeto referenciado en una aplicación. Por lo tanto, cuando una variable que referencia a un objeto de una clase derivada, durante la ejecución, sale fuera del ámbito en el que ha sido definido, el objeto es recolectado como basura por el recolector de basura, si no quedan más referencias al mismo, lo que implica que se ejecute primero el destructor de esa clase derivada y después el de su clase base.

POLIMORFISMO

Conseguir que los métodos de una clase base y sus redefiniciones en sus clases derivadas se comporten adecuadamente, independientemente del tipo del medio realmente empleado para acceder a los mismos (referencia a una clase), se denomina *polimorfismo* (facultad de asumir muchas formas), mientras que una clase con métodos virtuales se denomina *tipo polimórfico*.

Según hemos estudiado anteriormente, para conseguir en C# un comportamiento polimórfico, los métodos deben ser definidos virtuales (**virtual**) en la clase base y redefinidos (**override**) en las clases derivadas, y los objetos deben ser manipulados mediante referencias a la clase base. Si se dan estas condiciones, cuando se invoque a uno de esos métodos, la versión que se ejecutará corresponderá a la clase del objeto referenciado y no a la clase de la variable que lo referencia.

Así mismo, sabemos que una referencia a una clase derivada puede ser convertida implícitamente por C# en una referencia a su clase base directa o indirecta. Esto significa que es posible referirse a un objeto de una clase derivada utilizando una variable del tipo de su clase base.

Según lo expuesto, y en un intento de buscar una codificación más genérica, pensemos en una matriz de referencias en la que cada elemento señale a un objeto de alguna de las clases derivadas de la jerarquía construida anteriormente. ¿De qué tipo deben ser los elementos de la matriz? Según el párrafo anterior deben ser de la clase *CCuenta*; de esta forma ellos podrán almacenar indistintamente referencias a objetos de cualquiera de las clases derivadas. Por ejemplo:

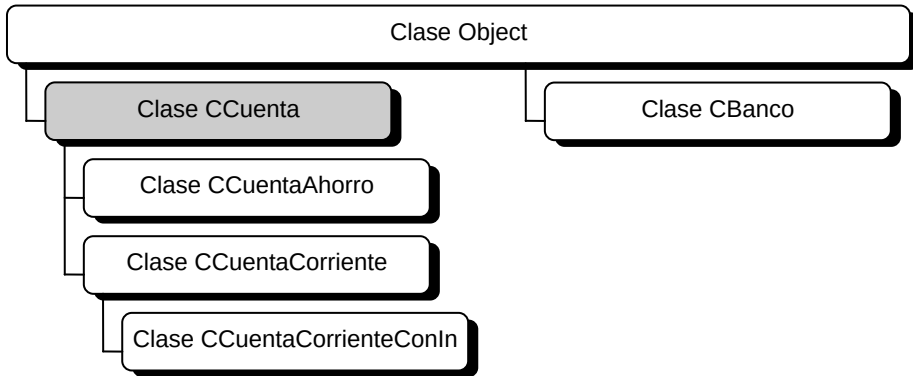
```
public class Test
{
    public static void Main(string[] args)
    {
        List<CCuenta> cuenta = new List<CCuenta>();

        // Crear objetos y guardar sus referencias en la matriz
        cuenta.Add(new CCuentaAhorro("cliente00", "3000123450",
                                     10000, 2.5, 0.90));
        cuenta.Add(new CCuentaCorriente("cliente01", "6000123450",
                                     10000, 1.5, 1.0, 6));
        cuenta.Add(new CCuentaCorrienteConIn("cliente02",
                                     "4000123450", 10000, 3.5, 1.0, 6));

        for (int i = 0; i < cuenta.Count; i++)
        {
            Console.Write(cuenta[i].ObtenerNombre() + ": ");
            Console.WriteLine(cuenta[i].Intereses());
        }
    }
}
```

Este ejemplo define una lista *cuenta* de tipo *CCuenta* con cero elementos inicialmente. Después crea un objeto de una de las clases derivadas y almacena su referencia en el primer elemento de la matriz; aquí C# realizará una conversión implícita del tipo de la referencia devuelta por **new** al tipo *CCuenta*. Este proceso se repetirá para cada objeto nuevo que deseemos crear (en nuestro caso un total de tres veces). Finalmente, utilizando un bucle mostramos el nombre del cliente y los intereses que le corresponderán por mes. Pregunta: ¿en cuál de las dos líneas de este bucle se aplica la definición de polimorfismo? Lógicamente en la última porque, según lo estudiado hasta ahora, invoca a las distintas definiciones del método *Intereses* utilizando el mismo medio de acceso: una referencia a *CCuenta*.

Como ejemplo, vamos a escribir un programa que cree un objeto que represente a una entidad bancaria con un cierto número de cuentas. Este objeto estará definido por una clase que denominaremos *CBanco* y las cuentas serán objetos de alguna de las clases de la jerarquía construida en los apartados anteriores.



La estructura de datos que represente el banco tiene que ser capaz de almacenar objetos *CCuentaAhorro*, *CCuentaCorriente* y *CCuentaCorrienteConIn*. Sabiendo que cualquier referencia a un objeto de una clase derivada puede convertirse implícitamente en una referencia a un objeto de su clase base, la estructura idónea es una matriz de referencias a la clase base *CCuenta*. Esta matriz será dinámica; esto es, aumentará en un elemento cuando se añada un objeto de alguna de las clases derivadas y disminuirá en uno cuando se elimine; inicialmente tendrá cero elementos. Según esto, la clase *CBanco*, que no pertenece a nuestra jerarquía, tendrá los atributos y métodos que se exponen a continuación:

Atributo	Significado
<i>cuentas</i>	Matriz de referencias de tipo <i>CCuenta</i> .

Método	Significado
<i>CBanco</i>	Es el constructor de la clase. Inicia la matriz <i>cuentas</i> con cero elementos.
<i>Añadir</i>	Añade un objeto de alguna de las clases derivadas de <i>CCuenta</i> al final de la matriz <i>cuentas</i> .
<i>Asignar</i>	Asigna un objeto de alguna de las clases derivadas de <i>CCuenta</i> a la posición <i>i</i> de la matriz <i>cuentas</i> . Devuelve true si la operación se realiza con éxito y false en caso contrario.
<i>Obtener</i>	Devuelve una referencia al objeto que está en la posición <i>i</i> de la matriz <i>cuentas</i> o null si <i>i</i> está fuera de límites.
<i>Eliminar</i>	Elimina el elemento de la matriz correspondiente al número de cuenta pasado como argumento. Devuelve true si la operación se realiza con éxito y false en caso contrario.
<i>Buscar</i>	Devuelve la posición en la matriz <i>cuentas</i> del objeto cuyo titular (nombre total o parcial) o cuenta coincida con el valor pasado como argumento, o -1 si no hay coincidencia.
<i>Longitud</i>	Devuelve el número de elementos de la matriz.

La definición correspondiente a esta clase se expone a continuación:

```
using System;
using System.Collections.Generic;

////////////////////////////////////
// Clase CBanco: clase que mantiene una matriz de referencias a
// objetos de cualquier tipo de cuenta bancaria.
//
public class CBanco
{
    // Matriz de objetos
    private List<CCuenta> cuentas = new List<CCuenta>();

    public CBanco()
    {
        // Reservar espacio para 100 elementos (elementos iniciales: cero)
        cuentas.Capacity = 100;
    }

    public void Añadir(CCuenta obj)
    {
        // Añadir un objeto a la matriz
        cuentas.Add(obj);
    }

    public bool Asignar( int i, CCuenta objeto )
    {
        // Asignar al elemento i de la matriz, un nuevo objeto
        if (i >= 0 && i < cuentas.Count)
        {
            cuentas[i] = objeto;
            return true;
        }
        else
        {
            Console.WriteLine("Índice fuera de límites");
            return false;
        }
    }

    public CCuenta Obtener(int i)
    {
        // Devolver la referencia al objeto i de la matriz
        if (i >= 0 && i < cuentas.Count)
            return cuentas[i];
        else
        {

```

```
        Console.WriteLine("Índice fuera de límites");
        return null;
    }
}

public bool Eliminar(string s)
{
    // Buscar la cuenta y eliminar el objeto
    for ( int i = 0; i < cuentas.Count; ++i )
        if (s.CompareTo(cuentas[i].ObtenerCuenta()) == 0)
        {
            cuentas.RemoveAt(i);
            return true;
        }
    return false;
}

public int Buscar(string str, int pos)
{
    // Buscar un objeto y devolver su posición
    string nom, cuen;
    if (str == null) return -1;
    if (pos < 0) pos = 0;
    for ( int i = pos; i < cuentas.Count; i++ )
    {
        // Buscar por el nombre
        nom = cuentas[i].ObtenerNombre();
        if (nom == null) continue;
        // ¿str está contenida en nom?
        if (nom.IndexOf(str) > -1)
            return i;
        // Buscar por la cuenta
        cuen = cuentas[i].ObtenerCuenta();
        if (cuen == null) continue;
        // ¿str está contenida en cuen?
        if (cuen.IndexOf(str) > -1)
            return i;
    }
    return -1;
}

public int Longitud() { return cuentas.Count; }
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

Analizando la clase *CBanco*, observamos que su constructor inicia la matriz *cuentas* con un espacio para 100 elementos, aunque el tamaño o número de elementos inicial es cero, y que las operaciones de añadir, asignar, obtener, eliminar, etc., un elemento en *cuentas* se basan en invocar al método adecuado del objeto **List**.

Quizás piense, ¿qué sucede con la cuenta referenciada cuando se elimina un elemento de la matriz? Pues que ese objeto queda sin referenciar, condición suficiente para que sea enviado a la basura y recogido por el recolector de basura.

Para finalizar, queda escribir una aplicación que utilizando la clase *CBanco*, construya la entidad bancaria objetivo del ejemplo propuesto. Esta aplicación presentará un menú como el indicado a continuación:

1. *Saldo*
2. *Buscar siguiente*
3. *Ingreso*
4. *Reintegro*
5. *Añadir*
6. *Eliminar*
7. *Mantenimiento*
8. *Salir*

Opción:

La operación elegida será identificada por una sentencia **switch** y procesada de acuerdo al esquema presentado a continuación:

```
public class Test
{
    public static CCuenta LeerDatos(int op) { ... }

    public static int Menú() { ... }

    public static void Main(string[] args)
    {
        // Crear un objeto banco vacío (con cero elementos)
        CBanco banco = new CBanco();

        do
        {
            opción = Menú();

            switch (opción)
            {
                case 1: // saldo
                    // Buscar un elemento por el nombre o por la cuenta.
                    // La subcadena de búsqueda será obtenida del teclado.
                    pos = banco.Buscar(cadenabuscar, 0);
                    // Si se encuentra, mostrar nombre, cuenta y saldo
                    break;
                case 2: // buscar siguiente
                    // Buscar el siguiente elemento que contenga la subcadena
                    // utilizada en la última búsqueda (case 1).
                    pos = banco.Buscar(cadenabuscar, pos + 1);
```

```
        // Si se encuentra, mostrar nombre, cuenta y saldo
        break;
    case 3: // ingreso
    case 4: // reintegro
        // Ingresar una cantidad en la cuenta o reintegrarla.
        // Ambos datos se solicitarán del teclado.
        pos = banco.Buscar(cuenta, 0);
        if (opción == 3)
            banco.Obtener(pos).Ingreso(cantidad);
        else
            banco.Obtener(pos).Reintegro(cantidad);
        break;
    case 5: // añadir
        // Añadir una nueva cuenta. El objeto correspondiente
        // será devuelto por el método static LeerDatos de esta
        // aplicación, que obtendrá los datos desde el teclado.
        banco.Añadir(LeerDatos(tipo_objeto));
        break;
    case 6: // eliminar
        // Eliminar la cuenta especificada.
        banco.Eliminar(cuenta);
        break;
    case 7: // mantenimiento
        // Cobrar comisiones e ingresar intereses
        for (pos = 0; pos < banco.Longitud(); pos++)
        {
            banco.Obtener(pos).Comisiones();
            banco.Obtener(pos).Intereses();
        }
        break;
    case 8: // salir
        banco = null;
        break;
    }
    while(opción != 8);
}
```

El listado completo de la aplicación *Test* se muestra a continuación. Se puede observar que la clase aplicación utiliza tres métodos estáticos: *LeerDatos*, *Menú* y el método **Main**.

El método *LeerDatos* recibe como parámetro un valor 1, 2 o 3 dependiendo del tipo de objeto que se desee crear: *CCuentaAhorro*, *CCuentaCorriente* o *CCuentaCorrienteConIn*. Lee los atributos correspondientes al tipo de cuenta elegido e invoca al constructor adecuado. El método devuelve una referencia al nuevo objeto construido. Este método será invocado cada vez que se elija la opción *Añadir* una nueva cuenta.

El método *Menú* visualiza el menú anteriormente mostrado, y devuelve el entero correspondiente a la opción elegida.

El método **Main** crea el objeto *banco* e invoca repetidamente al método *Menú* para permitir elegir la operación programada que se desee realizar en ese instante.

```
using System;
using MisClases.ES; // clase Leer

////////////////////////////////////
// Aplicación para trabajar con la clase CBanco y la jerarquía
// de clases derivadas de CCuenta
//
public class Test
{
    // Para la entrada de datos se utiliza Leer
    public static CCuenta LeerDatos(int op)
    {
        CCuenta obj = null;
        string nombre, cuenta;
        double saldo, tipoi, mant;
        Console.Write("Nombre.....: ");
        nombre = Console.ReadLine();
        Console.Write("Cuenta.....: ");
        cuenta = Console.ReadLine();
        Console.Write("Saldo.....: ");
        saldo = Leer.datoDouble();
        Console.Write("Tipo de interés.....: ");
        tipoi = Leer.datoDouble();
        if (op == 1)
        {
            Console.Write("Mantenimiento.....: ");
            mant = Leer.datoDouble();
            obj = new CCuentaAhorro(nombre, cuenta, saldo, tipoi, mant);
        }
        else
        {
            int transex;
            double imptrans;
            Console.Write("Importe por transacción: ");
            imptrans = Leer.datoDouble();
            Console.Write("Transacciones exentas...: ");
            transex = Leer.datoInt();
            if (op == 2)
                obj = new CCuentaCorriente(nombre, cuenta, saldo, tipoi,
                                            imptrans, transex);
            else
                obj = new CCuentaCorrienteConIn(nombre, cuenta, saldo,
                                                tipoi, imptrans, transex);
        }
    }
}
```

```
        return obj;
    }

    public static int Menú()
    {
        Console.Write("\n\n");
        Console.WriteLine("1. Saldo");
        Console.WriteLine("2. Buscar siguiente");
        Console.WriteLine("3. Ingreso");
        Console.WriteLine("4. Reintegro");
        Console.WriteLine("5. Añadir");
        Console.WriteLine("6. Eliminar");
        Console.WriteLine("7. Mantenimiento");
        Console.WriteLine("8. Salir");
        Console.WriteLine();
        Console.Write("    Opción: ");
        int op;
        do
        {
            op = Leer.datoInt();
            while (op < 1 || op > 8);
        }
        return op;
    }

    public static void Main(string[] args)
    {
        // Crear un objeto banco vacío (con cero elementos)
        CBanco banco = new CBanco();

        int opción = 0, pos = -1;
        string cadenabuscar = null;
        string cuenta;
        double cantidad;
        bool eliminado = false;

        do
        {
            opción = Menú();
            switch (opción)
            {
                case 1: // saldo
                    Console.Write("Nombre total o parcial, o cuenta ");
                    cadenabuscar = Console.ReadLine();
                    pos = banco.Buscar(cadenabuscar, 0);
                    if (pos == -1)
                    {
                        if (banco.Longitud() != 0)
                            Console.WriteLine("búsqueda fallida");
                        else
                            Console.WriteLine("no hay cuentas");
                    }
                    else
                    {
                        Console.WriteLine(banco.Obtener(pos).ObtenerNombre());
                    }
                }
            }
        }
    }
}
```

```

        Console.WriteLine(banco.Obtener(pos).ObtenerCuenta());
        Console.WriteLine(banco.Obtener(pos).Saldo());
    }
    break;
case 2: // buscar siguiente
    pos = banco.Buscar(cadenabuscar, pos + 1);
    if (pos == -1)
        if (banco.Longitud() != 0)
            Console.WriteLine("búsqueda fallida");
        else
            Console.WriteLine("no hay cuentas");
    else
    {
        Console.WriteLine(banco.Obtener(pos).ObtenerNombre());
        Console.WriteLine(banco.Obtener(pos).ObtenerCuenta());
        Console.WriteLine(banco.Obtener(pos).Saldo());
    }
    break;
case 3: // ingreso
case 4: // reintegro
    Console.Write("Cuenta: ");
    cuenta = Console.ReadLine();
    pos = banco.Buscar(cuenta, 0);
    if (pos == -1)
        if (banco.Longitud() != 0)
            Console.WriteLine("búsqueda fallida");
        else
            Console.WriteLine("no hay cuentas");
    else
    {
        Console.Write("Cantidad: ");
        cantidad = Leer.datoDouble();
        if (opción == 3)
            banco.Obtener(pos).Ingreso(cantidad);
        else
            banco.Obtener(pos).Reintegro(cantidad);
    }
    break;
case 5: // añadir
    Console.Write("Tipo de cuenta: 1-(CA), 2-(CC), 3-(CCI) ");
    do
        opción = Leer.datoInt();
    while (opción < 1 || opción > 3);
    banco.Añadir(LeerDatos(opción));
    break;
case 6: // eliminar
    Console.Write("Cuenta: "); cuenta = Console.ReadLine();
    eliminado = banco.Eliminar(cuenta);
    if (eliminado)
        Console.WriteLine("registro eliminado");
    else

```

```
        if (banco.Longitud() != 0)
            Console.WriteLine("cuenta no encontrada");
        else
            Console.WriteLine("no hay cuentas");
        break;
    case 7: // mantenimiento
        for (pos = 0; pos < banco.Longitud(); pos++)
        {
            banco.Obtener(pos).Comisiones();
            banco.Obtener(pos).Intereses();
        }
        break;
    case 8: // salir
        banco = null;
        break;
    }
    while(opción != 8);
}
}
////////////////////////////////////
```

También aquí se puede observar que el mantenimiento de las cuentas (case 7) resulta sencillo gracias a la definición de polimorfismo. Esto es, el método *Comisiones* o *Intereses* que se invoca para cada cliente depende del tipo del objeto referenciado por el elemento accedido de la matriz *cuentas* de *banco*.

MÉTODOS EN LÍNEA

Cuando el compilador C# conoce con exactitud qué método tiene que llamar para responder al mensaje que se ha programado, puede tomar la iniciativa de reemplazar la llamada al método por el cuerpo del mismo. Se dice entonces que el método está en línea. El que se produzca esta circunstancia, por ejemplo, porque el método es corto, redundará en tiempos de ejecución más bajos ya que se evita que el intérprete C# tenga que llamar al método. En principio, en C#, todos los métodos de una clase pueden ser métodos en línea.

¿Cuándo un método no podrá pasar a ser un método en línea? Cuando el compilador no sepa con exactitud a qué versión del método tiene que invocar. Veamos; si como en el ejemplo anterior, tenemos una matriz de referencias a objetos de las clases derivadas *CCuentaAhorro*, *CCuentaCorriente* o *CCuentaCorrienteConIn*, ¿cómo sabe el compilador a qué método *Interés*, por ejemplo, tiene que llamar? El compilador no puede saber esto. Cuando esto sucede, el compilador produce código que permitirá a la máquina virtual .NET consultar durante la ejecución qué método tiene que invocar. Como la máquina virtual sí sabe a qué objeto, *CCuentaAhorro*, *CCuentaCorriente* o *CCuentaCorrienteConIn*, se refiere

cada uno de los elementos de la matriz, el código añadido por el compilador será suficiente para determinar qué método tiene que invocar para cada uno de los objetos.

La consulta dinámica acerca del método que hay que invocar es rápida, pero no tan rápida como invocar a un método directamente. Afortunadamente no hay muchos casos en los que C# necesite usar la consulta dinámica. Por ejemplo, los métodos estáticos (**static**) y los privados (**private**) pueden ser invocados directamente; y si son cortos son candidatos a ser métodos en línea. Si un método es estático es invocado anteponiendo el nombre de su clase; y si es privado no puede ser invocado por un método que no sea de su clase. Por lo tanto, en ninguno de los dos casos habrá que tomar una decisión acerca de a qué método hay que llamar.

CLASES Y MÉTODOS ABSTRACTOS

En una jerarquía de clases, una clase es tanto más especializada cuanto más alejada esté de la raíz, entendiendo por clase raíz aquella de la cual heredan directa o indirectamente el resto de las clases de la jerarquía; y al contrario, es tanto más genérica cuanto más cerca esté de la raíz. Sirva de ejemplo la clase **Object**, clase raíz de la jerarquía de clases de .NET, cuyo objetivo es definir los atributos y el comportamiento comunes a todas las clases. Desde este punto de vista, podemos decir que la clase *CCuenta* de la aplicación anterior es una clase genérica, ya que la razón de su existencia es proporcionar los atributos y comportamientos que serán compartidos por todas sus clases derivadas, *CCuentaAhorro*, *CCuentaCorriente* y *CCuentaCorrienteConIn*, destinadas a la creación de cuentas más especializadas.

Cuando una clase se diseña para ser genérica, es casi seguro que no necesitaremos crear objetos de ella. Una clase que se comporte de la forma descrita se denomina clase *abstracta* y se define como tal calificándola explícitamente *abstracta* (**abstract**). Por ejemplo:

```
public abstract class MiClaseAbs
{
    // Cuerpo de la clase
}
```

Una clase abstracta puede contener el mismo tipo de miembros que una clase que no lo sea, y además pueden contener métodos abstractos, que una clase no abstracta no puede contener, pero no se permite crear objetos de ella.

¿Qué es un método abstracto? Es un método calificado **abstract** con la particularidad de que no tiene cuerpo. ¿Por qué no tiene cuerpo? Porque la idea es

proporcionar métodos que deban ser redefinidos en las clases derivadas de la clase abstracta, con la intención de adaptarlos a las necesidades particulares de éstas.

También es importante saber que un método abstracto es, implícitamente, un método virtual.

Por ejemplo, considerando a *CCuenta* una clase genérica, la podemos declarar abstracta y también podemos declarar abstractos los métodos *Comisiones* e *Intereses*, ya que, además de ser virtuales, son específicos para cada cuenta, esto es, no es necesario que aporten una definición genérica:

```
public abstract class CCuenta
{
    // ...
    public abstract void Comisiones();
    public abstract double Intereses();
    // ...
}
```

En el caso de que el método heredado por la clase derivada sea abstracto, como sucede en nuestro ejemplo, es obligatorio redefinirlo, de lo contrario la clase derivada debería ser declarada también abstracta. Por ejemplo:

```
public class CCuentaAhorro : CCuenta
{
    // ..
    public override void Comisiones()
    {
        // ...
    }

    public override double Intereses()
    {
        // ...
    }
    // ..
}
```

Ahora, puede intentar declarar la clase *CCuenta* no abstracta y comprobará que el compilador C# le muestra un mensaje indicándole que una clase sólo puede contener métodos abstractos si es abstracta.

CLASES Y MÉTODOS FINALES

El modificador **sealed** (final, sellado o no heredable) se puede aplicar a clases y a métodos y propiedades de éstas. Una clase **sealed** no se puede heredar. Un méto-

do **sealed** reemplaza a un método virtual de una clase base, pero él no puede ser reemplazado en una clase derivada. El modificador **sealed** en un método o propiedad siempre se debe utilizar con **override**; esto es, se trata de un método o propiedad de una clase derivada. Las estructuras son tipos **sealed** implícitamente.

```
public sealed class Clase
{
    // Esta clase no se puede heredar
}
```

Es un error utilizar una clase **sealed** como una clase base o utilizar el modificador **abstract** con una clase sellada.

INTERFACES

De forma genérica una interfaz se define así: un dispositivo o un sistema utilizado por entidades inconexas para interactuar. Según esta definición un control remoto es una interfaz, el idioma inglés es una interfaz, etc. Análogamente, una interfaz C# es un dispositivo que permite interactuar a objetos no relacionados entre sí. Las interfaces C# en realidad definen un conjunto de mensajes que se puede aplicar a muchas clases de objetos, a los que cada una de ellas debe responder de forma adecuada. Por eso, una interfaz recibe también el nombre de protocolo.

Definir una interfaz

Una interfaz consta de dos partes: el *nombre de la interfaz* precedido por la palabra reservada **interface**, y el *cuerpo de la interfaz* encerrado entre llaves. Esto es:

```
[modificadores] interface nombre_interfaz [: interfaces-base]
{
    Cuerpo de la interfaz
}
```

Los modificadores son los mismos que especificamos para las clases: **internal** (es el modificador por omisión) y **public**, y su significado es el mismo. Una interfaz puede incluirse en un espacio de nombres exactamente igual que una clase.

El cuerpo de la interfaz sólo puede incluir declaraciones de métodos (no sus definiciones).

El símbolo **:** significa que se está definiendo una interfaz que es una extensión de otras; también se puede decir que es una interfaz derivada de otras; estas otras se especifican a continuación de **:** separadas por comas. Como habrá observado, a

diferencia de las clases, una interfaz puede derivarse de más de una interfaz base. Una interfaz así definida hereda todos los métodos de sus interfaces base, excepto los métodos que queden ocultos porque se redefinan.

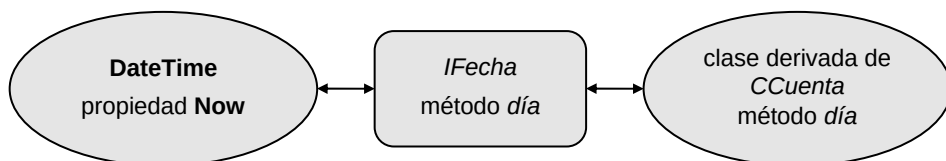
El nombre de una interfaz se puede utilizar en cualquier lugar donde se pueda utilizar el nombre de una clase.

Un ejemplo: la interfaz *IFecha*

En la jerarquía de clases implementada anteriormente en este mismo capítulo, nosotros declaramos las clases *CCuentaAhorro*, *CCuentaCorriente* y *CCuentaCorrienteConIn* como parte de un conjunto de clases para administrar distintos tipos de cuentas bancarias. Todas estas clases tienen varios métodos en común; así que para facilitar, no sólo el diseño, sino el trabajo con matrices de objetos de dichas clases, nosotros implementamos una clase base genérica, *CCuenta*, que encapsula los atributos y los métodos comunes a todas esas clases; incluso, alguno de esos métodos, como *Comisiones* e *Intereses*, no tenía sentido definirlos en la clase base porque debían ser después particularizados para cada una de las clases derivadas. Esto nos condujo a definir esos métodos como abstractos, lo que implicó definir la clase base también abstracta.

Las interfaces, al igual que las clases y métodos abstractos, proporcionan plantillas de comportamiento que se espera sean implementadas por otras clases. Esto es, una interfaz C# declara un conjunto de métodos, pero no los define (sólo aporta los prototipos de los métodos) porque todos son declarados implícitamente abstractos (**abstract**) y públicos (**public**). Una interfaz no puede contener atributos (campos).

Para comprender con claridad las interfaces vamos a realizar un ejemplo de una interfaz *IFecha* que va a ser utilizada para que dos clases (en este caso la estructura **DateTime** y una de las clases derivadas de *CCuenta*) interactúen entre sí.



La estructura **DateTime** es un proveedor de servicios; en nuestro ejemplo, notificará el día a los objetos derivados de *CCuenta* cuando intenten ejecutar sus métodos *Comisiones* o *Intereses*. Para ello, como se muestra a continuación, proporciona la propiedad **Now** que devuelve el dato (día, mes, etc.) solicitado:

```
DateTime.Now.Day
DateTime.Now.Month
DateTime.Now.Year
```

Según hemos planteado el problema, cualquier objeto derivado de *CCuenta* que quiera utilizar un objeto **DateTime** debe implementar el método *día* proporcionado por la interfaz *IFecha*. Este método es el medio utilizado por el objeto **DateTime** para notificar al objeto derivado de *CCuenta* el día actual. Según lo expuesto la interfaz *IFecha* puede tener el aspecto siguiente:

```
// Interfaz IFecha: métodos para obtener el día, mes y año
public interface IFecha
{
    int día();
    int mes();
    int año();
}
```

Se puede observar que una interfaz sólo declara los métodos, no los define, que son implícitamente públicos y abstractos.

Utilizar una interfaz

Para utilizar una interfaz hay que añadir el nombre de la misma de forma análoga a como se muestra en los ejemplos siguientes:

```
public class Clase : Interfaz
public class ClaseDerivada : ClaseBase, Interfaz
public class Clase : Interfaz1, Interfaz2
...
```

Siguiendo con el ejemplo iniciado en el apartado anterior, una clase derivada de *CCuenta* como *CCuentaAhorro* que utilice la interfaz *IFecha* debe definir los métodos aportados por esa interfaz:

```
public class CCuentaAhorro : CCuenta, IFecha
{
    // ...
    public override void Comisiones()
    {
        // Se aplican mensualmente por el mantenimiento de la cuenta
        if (día() == 1) Reintegro(cuotaMantenimiento);
    }

    public override double Intereses()
    {
        if (día() != 1) return 0.0;
    }
}
```

```
// Acumular los intereses por mes sólo los días 1 de cada mes
double interesesProducidos = 0.0;
interesesProducidos = Saldo() * ObtenerTipoDeInterés() / 1200.0;
Ingreso(interesesProducidos);

// Devolver el interés mensual por si fuera necesario
return interesesProducidos;
}

// Implementación de los métodos de la interfaz IFecha
public int día()
{
    return DateTime.Now.Day;
}
public int mes() { return 0; } // no se necesita
public int año() { return 0; } // no se necesita
}
```

Como una interfaz sólo aporta declaraciones de métodos, es nuestra obligación definir todos los métodos en cada una de las clases que utilice la interfaz. No podemos elegir y definir sólo aquellos métodos que necesitemos. De no hacerlo, el compilador C# mostraría un error.

Si una clase implementa una interfaz, todas sus clases derivadas heredarán los nuevos métodos que se hayan implementado en la clase base. Por ejemplo, modifiquemos la clase *CCuentaCorriente* para que utilice también la interfaz *IFecha*:

```
public class CCuentaCorriente : CCuenta, IFecha
{
    // ...

    public override void Comisiones()
    {
        // Se aplican mensualmente por el mantenimiento de la cuenta
        if (día() == 1)
        {
            // ...
        }
    }

    public override double Intereses()
    {
        if (día() != 1) return 0.0;
        // ...
    }

    // Implementación de los métodos de la interfaz IFecha
    public int día()
    {
```

```

    return DateTime.Now.Day;
}

public int mes() { return 0; } // no se necesita
public int año() { return 0; } // no se necesita
}

```

Como la clase *CCuentaCorriente* implementa la interfaz *IFecha*, su clase derivada *CCuentaCorrienteConIn* heredará los nuevos métodos y constantes. Por lo tanto, no es necesario agregar a la definición de esta clase el nombre de la interfaz *IFecha*.

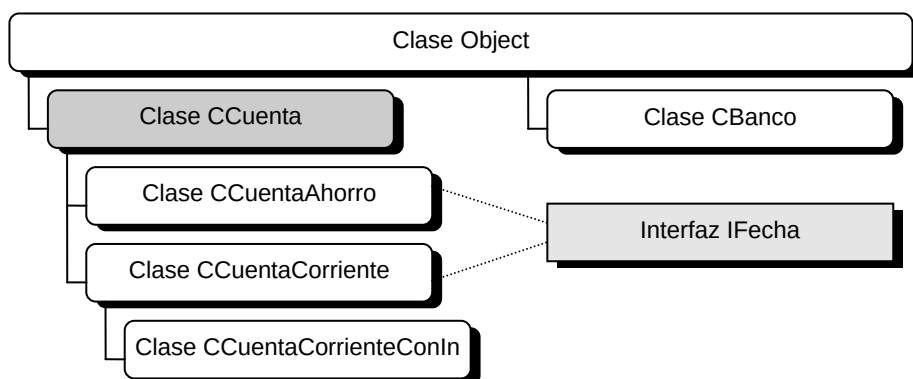
```

public class CCuentaCorrienteConIn : CCuentaCorriente
{
    // ...

    public override double Intereses()
    {
        if (día() != 1 || Saldo() < 3000) return 0.0;
        // ...
    }
}

```

Una vez realizadas en nuestra jerarquía de clases las modificaciones propuestas como consecuencia de haber añadido la interfaz *IFecha*, el resultado obtenido desde un punto de vista gráfico es el siguiente:



Probablemente habrá pensado que hubiéramos obtenido el mismo resultado implementando la interfaz *IFecha* en la clase base *CCuenta*. Pues, así es, tiene razón. El hecho de haber implementado la interfaz en las clases derivadas ha sido puramente didáctico.

Clase abstracta frente a interfaz

Llegado a este punto, se preguntará ¿en qué difiere una interfaz de una clase abstracta? Puesto que una interfaz es simplemente una lista de declaraciones de métodos, ¿sería equivalente la clase *IFecha* siguiente a la interfaz *IFecha*?

```
public abstract class IFecha
{
    public abstract int día();
    public abstract int mes();
    public abstract int año();
}
```

La respuesta a la pregunta anterior es no. Si *IFecha* es una clase abstracta, entonces todas las clases derivadas de *CCuenta*, como *CCuentaAhorro*, que quisieran utilizar su funcionalidad para interactuar con **DateTime** tendrían que derivarse de ella. Pero sucede que las clases derivadas a las que nos referimos ya tienen una clase base y no pueden tener otra, ya que C# no permite la herencia múltiple de clases; sí permite que una interfaz se derive de múltiples interfaces. Por lo tanto, en casos como el presentado hay que utilizar una interfaz.

Lo anterior es una explicación práctica. Una explicación conceptual puede ser que **DateTime** no debe forzar a sus usuarios a establecer una relación entre clases. Esto es, no importa la clase; lo único que importa es implementar uno o más métodos específicos. Al fin y al cabo, una interfaz no es más que un protocolo que una clase implementa cuando necesita utilizarlo.

Evidentemente nuestro problema en concreto tiene una solución, que es derivar la clase *CCuenta* de la clase abstracta *IFecha* e implementar en *CCuenta* los métodos proporcionados por *IFecha*. Pero nuestro objetivo no es dar solución a este problema, sino presentar ejemplos adecuados acerca de lo que se quiere explicar.

Utilizar una interfaz como un tipo

Una interfaz es un nuevo tipo de datos; un tipo referenciado. Por lo tanto, el nombre de una interfaz se puede utilizar en cualquier lugar donde pueda aparecer el nombre de cualquier otro tipo de datos.

Una variable del tipo de una interfaz espera referenciar un objeto que tenga implementada dicha interfaz, de lo contrario el compilador C# mostrará un error.

Por ejemplo, se puede declarar una matriz *cuentas* que sea de tipo *IFecha* y asignar a cada elemento un objeto de algunas de las clases derivadas de *CCuenta*:


```

IFecha[] cuentas = new IFecha[3];
CCuentaAhorro cuenta0 = new CCuentaAhorro();
cuentas[0] = cuenta0;
// ...

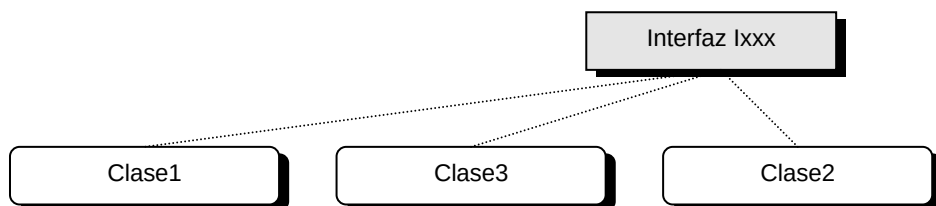
((CCuentaAhorro)cuentas[0]).AsignarNombre("cliente0");
Console.WriteLine(cuenta0.ObtenerNombre());
// ...

```

En el ejemplo anterior la variable *cuentas[0]* de tipo *IFecha* hace referencia a un objeto *CCuentaAhorro* que implementa esa interfaz.

Así mismo, se puede observar que es posible convertir implícitamente referencias a objetos que implementan una interfaz en referencias a esa interfaz y viceversa, pero en este caso, explícitamente. Lógicamente, con lo que sabemos, podemos deducir que con una referencia a la interfaz sólo se tiene acceso a los métodos declarados en dicha interfaz.

El siguiente ejemplo muestra cómo tres clases no relacionadas, *Clase1*, *Clase2* y *Clase3*, por el hecho de implementar la misma interfaz *Ixxx*, permiten definir una matriz de tipo *Ixxx* y aplicar la definición de polimorfismo.



```

public interface Ixxx
{
    void m(); // método m
    void p(); // método p
}

public class Clase1 : Ixxx
{
    public void m()
    {
        Console.WriteLine("método m de Clase1");
    }
    public void p() {}
}

public class Clase2 : Ixxx
{
    public void m()
    {

```

```
        Console.WriteLine("método m de Clase2");
    }
    public void p() {}
}

public class Clase3 : Ixxx
{
    public void m()
    {
        Console.WriteLine("método m de Clase3");
    }
    public void p() {}
}

public class Test
{
    public static void Main (string[] args)
    {
        Ixxx[] objs = new Ixxx[3]; // matriz de referencias a objetos
        objs[0] = new Clase1();
        objs[1] = new Clase2();
        objs[2] = new Clase3();
        for( int i = 0; i < objs.Length; i++)
            objs[i].m(); // invoca al método m del objeto Clase1, Clase2
                        // o Clase3 referenciado por objs[i]
    }
}
```

Interfaces frente a herencia múltiple

A menudo se piensa en las interfaces como en una alternativa a la herencia múltiple. Pero la realidad es que ambos conceptos, interfaz y herencia múltiple, son bastantes diferentes, a pesar de que las interfaces pueden resolver problemas similares. En particular:

- Desde una interfaz, una clase no puede heredar campos.
- Desde una interfaz, una clase no puede heredar definiciones de métodos.
- La jerarquía de interfaces es independiente de la jerarquía de clases. De hecho, varias clases pueden implementar la misma interfaz y no pertenecer a la misma jerarquía de clases. En cambio, cuando se habla de herencia múltiple, todas las clases pertenecen a la misma jerarquía.

Para qué sirve una interfaz

Después de todo lo expuesto es posible que aún no esté claro cuál es el sentido de utilizar interfaces. Si analizamos el ejercicio realizado anteriormente basado en la

jerarquía de clases *CCuenta* y en la interfaz *IFecha*, seguro que llegaremos a alguna conclusión similar a la siguiente: puesto que los métodos *día*, *mes* y *año* pertenecen a la clase derivada que los implementa, ¿para qué queremos la interfaz? Pensando así, para nada.

Una interfaz se utiliza para definir un protocolo de conducta que puede ser implementado por cualquier clase en una jerarquía de clases. La utilidad que esto pueda tener puede resumirse en los puntos siguientes:

- Captar similitudes entre clases no relacionadas sin forzar entre ellas una relación artificial. Una acción de este tipo permitiría incluso definir una matriz de objetos de esas clases y aplicar, si fuera necesario, la definición de polimorfismo, según hemos comprobado en el ejercicio anterior.
- Declarar métodos que una o más clases deben implementar en determinadas situaciones. Por ejemplo, la interfaz **IComparable** de la biblioteca .NET define un método de comparación generalizado **CompareTo**. Entonces, una estructura o una clase puede implementar esta interfaz y definir este método para crear una forma de comparación específica para propósitos tales como la ordenación. Otro ejemplo es interfaz **IDisposable** que proporciona el método **Dispose** implementado por los objetos utilizados en la sentencia **using**.
- Publicar la interfaz de programación de una clase sin descubrir cómo está implementada. En este caso, otros desarrolladores recibirían la clase compilada y la interfaz correspondiente.

Implementar múltiples interfaces

Una clase puede implementar una o más interfaces. Por ejemplo:

```
public class MiClase : interfaz1, interfaz2, interfaz3
{
    // ...
}
```

Cuando una clase implemente múltiples interfaces puede suceder que dos o más interfaces diferentes implementen el mismo método. Si esto ocurre, proceda de alguna de las formas indicadas a continuación:

- Si los métodos tienen el mismo prototipo, basta con definir uno en la clase.
- Si los métodos difieren en el número o tipo de sus parámetros, estamos en el caso de una sobrecarga del método; implemente todas las sobrecargas.

- Si los métodos sólo difieren en el tipo del valor retornado, no existe sobrecarga y el compilador produce un error, ya que dos métodos pertenecientes a la misma clase no pueden diferir sólo en el tipo del resultado.

EJERCICIOS RESUELTOS

1. Volviendo al ejercicio del banco realizado en el apartado *Polimorfismo*, pensemos ahora en qué ocurriría si, por cualquier causa, tuviéramos que hacer una copia del objeto *CBanco* de la aplicación anterior; por ejemplo, porque la necesitamos para hacer pruebas. ¿Serviría una copia de los atributos del objeto *CBanco* actual en otro objeto *CBanco*? La respuesta es no, porque el objeto copia tendría una matriz *cuentas* (atributo *cuentas* de *CBanco*) que haría referencia a los mismos objetos que el objeto *CBanco* original, lo que daría lugar a que las pruebas realizadas sobre la copia repercutirían sobre el original. ¿Cuál sería la solución al problema planteado?

La solución sería hacer una copia profunda del objeto *CBanco*. Para ponerlo en práctica, vamos a añadir al menú anterior dos opciones nuevas:

1. *Saldo*
2. *Buscar siguiente*
3. *Ingreso*
4. *Reintegro*
5. *Añadir*
6. *Eliminar*
7. *Mantenimiento*
8. *Copia de seguridad*
9. *Restaurar copia de seguridad*
10. *Salir*

Después de modificar el método *Menú* para que acepte las dos opciones añadidas, modificaremos el método **Main** con el fin de añadir el proceso adecuado para cada una de ellas. La opción 8 simplemente realizará un duplicado del banco; esto nos permitirá realizar pruebas con los datos actualmente en línea y cuando hayamos terminado, podemos volver al estado del banco original restaurando la copia (opción 9).

```
public static void Main(string[] args)
{
    // Crear un objeto banco vacío (con cero elementos)
    CBanco banco = new CBanco();
    CBanco copiabanco = null; // para la copia de seguridad
    // ...

    case 8: // copia de seguridad
        if (banco.Longitud() == 0) break;
```

```

        if (copiabanco == null)
        {
            copiabanco = new CBanco(banco);
            Console.WriteLine("copia realizada con éxito");
        }
        else
        {
            Console.WriteLine("existe una copia, restaurarla");
            break;
        }
        case 9: // restaurar copia de seguridad
            if (copiabanco == null) break;
            banco = copiabanco;
            Console.WriteLine("copia de seguridad restaurada");
            copiabanco = null;
            break;
        case 10: // salir
            if (copiabanco != null) copiabanco = null;
            banco = null;
            break;
    }
}
while(opción != 10);
}

```

Observe las dos líneas sombreadas del ejemplo anterior: la primera invoca al constructor copia y la segunda asigna la referencia de la copia de seguridad a *banco*; esto es, restaura la copia de seguridad.

¿Cómo implementaríamos el constructor copia y el método *Copiar* en la clase *CBanco*? Veámoslo a continuación:

```

public class CBanco
{
    // ...
    public CBanco(CBanco banco) // constructor copia
    {
        Copiar(banco);
    }

    public CBanco Copiar(CBanco banco)
    {
        // Eliminar las cuentas del objeto CBanco destino (this)
        if (cuentas.Count != 0) cuentas.Clear();
        // Copiar el banco origen en el banco destino
        for (int i = 0; i < banco.cuentas.Count; i++)
            cuentas.Add(banco.cuentas[i]);
        return this;
    }
    // ...
}

```

Puesto que tanto el constructor copia como el método *Copiar* tienen como objetivo copiar un objeto en otro, el código involucrado en la copia es el mismo, por eso el constructor copia invoca al método *Copiar*. Algo que necesita hacer el método *Copiar*, que no hace el constructor copia, es borrar la matriz de objetos destino reduciéndola a una matriz con cero elementos, antes de realizar la copia. Observemos ahora cómo se copia el objeto origen en el destino:

```
for (int i = 0; i < banco.cuentas.Count; i++)
    cuentas.Add(banco.cuentas[i]);
```

La copia que realiza este código no es correcta. ¿Por qué? Porque estamos copiando las referencias a los objetos (*banco.cuentas[i]* es una referencia de tipo *CCuenta*) y no estamos duplicando los objetos; al final, ambas matrices, origen y destino, estarán referenciando a los mismos objetos. Lo que tenemos que hacer entonces es duplicar el objeto referenciado por *banco.cuentas[i]*. Veamos una primera aproximación:

```
for (int i = 0; i < banco.cuentas.Count; i++)
    cuentas.Add(new CCuenta(banco.cuentas[i]));
```

Analizando el código anterior observamos que lo que se crea es un nuevo objeto de la clase *CCuenta*, cuando lo más seguro es que el objeto pasado como argumento sea de alguna de sus clases derivadas. Por lo tanto, lo que hay que hacer es invocar al constructor copia de su clase, dato que podemos obtener a través del operador **is**:

```
for (int i = 0; i < banco.cuentas.Count; i++)
{
    if (banco.cuentas[i] is CCuentaAhorro)
        c = new CCuentaAhorro((CCuentaAhorro)banco.cuentas[i]);
    else if (banco.cuentas[i] is CCuentaCorriente)
        c = new CCuentaCorriente((CCuentaCorriente)banco.cuentas[i]);
    else if (banco.cuentas[i] is CCuentaCorrienteConIn)
        c = new CCuentaCorrienteConIn(
            (CCuentaCorrienteConIn)banco.cuentas[i]);
    cuentas.Add(c);
}
```

Aunque, si utilizamos un método virtual que simule al constructor copia de la clase base y de sus derivadas, según explicamos anteriormente, el problema estará resuelto de una forma más óptima, ya que siempre es aconsejable utilizar los métodos virtuales en vez de información de tipo cuando sea necesario discriminar entre tipos durante la ejecución, como es el caso. Este método, que denominaremos *Clonar*, lo declararemos virtual en la clase base *CCuenta* y lo redefiniremos en todas sus derivadas:

```
public abstract class CCuenta
{
    // ...
    public abstract CCuenta Clonar();
    // ...
}
```

¿Cómo sería el método *Clonar* de la clase *CCuentaAhorro*? Teniendo definido el constructor copia, podemos hacerlo así:

```
public override CCuenta Clonar()
{
    return new CCuentaAhorro(this);
}
```

Obsérvese que el método *Clonar* devuelve un objeto nuevo de su clase, copia del objeto que recibió el mensaje.

Procediendo de forma análoga, podemos añadir a las clases *CCuentaCorriente* y *CCuentaCorrienteConIn* sus respectivos métodos *Clonar*:

```
public override CCuenta Clonar()
{
    return new CCuentaCorriente(this);
}

public override CCuenta Clonar()
{
    return new CCuentaCorrienteConIn(this);
}
```

Según lo expuesto, el constructor copia de la clase *CBanco* quedaría definitivamente implementado así:

```
public CBanco Copiar(CBanco banco)
{
    // Eliminar las cuentas del objeto CBanco destino (this)
    if (cuentas.Count != 0) cuentas.Clear();
    // Copiar el banco origen en el banco destino
    for (int i = 0; i < banco.cuentas.Count; i++)
        cuentas.Add(banco.cuentas[i].Clonar());
    return this;
}
```

Si ahora ejecuta de nuevo la aplicación, observará que todo funciona como esperábamos.

EJERCICIOS PROPUESTOS

1. Partiendo de las clases *CEstudios*, *CAlumno*, *CAsignatura*, *CConvocatoria* y *CFecha* construidas en el capítulo 10 (apartado *Ejercicios propuestos*), vamos a añadir dos nuevas clases *CAsignaturaOb* y *CAsignaturaOp* derivadas de *CAsignatura* con el fin de poder disponer de una lista de asignaturas obligatorias y de otra de optativas. Con este nuevo diseño tenemos que pensar en que ahora la matriz *CAlumno.asignatura* de tipo *CAsignatura* hará referencia a objetos *CAsignaturaOb* y *CAsignaturaOp*. Entonces, cuando necesitemos duplicar un objeto *CAlumno*, tendremos que duplicar los objetos *CAsignaturaOb* y *CAsignaturaOp* referenciados por la matriz *asignatura*. Esto requerirá conocer el tipo de los objetos, para lo que necesitaremos trabajar con clases polimórficas. Esto es, las clases *CAsignaturaOb* y *CAsignaturaOp* redefinirán un método *Clonar* declarado en la clase base *CAsignatura*; su misión será duplicar un objeto *CAsignaturaOb* o *CAsignaturaOp*.

Modifique la clase aplicación *Test.cs* suponiendo que las asignaturas con *ID* 1 a 70 son obligatorias y las asignaturas con *ID* 71 a 99 optativas.

Modifique el método *Copiar* de la clase *CAlumno* para que permita realizar una copia profunda de la matriz *asignatura* utilizando el método *Clonar*. Asegúrese de que el constructor copia realiza también este proceso.

Añada al menú dos opciones nuevas, con la intención de poder hacer una copia de seguridad de un objeto de la clase *CEstudios*. Añada a las clases *Test* y *CEstudios* el código necesario para permitir realizar estas operaciones.

1. Matricular
2. Poner notas
3. Mostrar expediente
4. Copia de seguridad
5. Restaurar copia de seguridad
6. Salir

2. Se quiere escribir un programa para manipular ecuaciones algebraicas o polinómicas dependientes de las variables *x*, *y*. Por ejemplo:

$$2x^3y - xy^3 + 8.25 \text{ más } 5x^5y - 2x^3y + 7x^2 - 3 \text{ igual a } 5x^5y + 7x^2 - xy^3 + 5.25$$

TIPOS Y MÉTODOS GENÉRICOS

Hasta ahora, un método o una clase han sido diseñados para trabajar con un tipo específico de datos. Los *tipos genéricos* o *tipos parametrizados* permiten definir una clase, estructura o una interfaz una sola vez y después crear objetos de ella de diferentes tipos; esto es, son un mecanismo C# que permite que un tipo pueda ser utilizado como parámetro en la definición de un método o de una clase. En realidad, este mecanismo tiene poco interés para el diseñador del método o de la clase, pero tiene verdadera importancia para el usuario de ese método o clase, ya que le permitirá elegir el tipo de datos que necesite en cada momento.

Según lo expuesto, resulta evidente que las *clases genéricas* simplifican la implementación de clases que definen contenedores (colecciones de objetos), puesto que el tipo de los objetos contenidos es un argumento en la definición de la clase; por ejemplo, las listas dinámicas y las matrices son ejemplos de contenedores. También es posible definir métodos genéricos para trabajar con un amplio número de tipos. El código que se muestra como ejemplo a continuación define matrices de diferentes tipos utilizando la clase genérica **List**:

```
List<int> li = new List<int>();  
List<string> ls = new List<string>();  
List<CCuenta> lc = new List<CCuenta>();
```

Obsérvese el tipo de declaración de las variables *li*, *ls* y *lc*. La primera define una lista de enteros, que es lo que especifica el parámetro **int**, la segunda define una lista de cadenas de caracteres, que es lo que especifica el parámetro **string**, y la tercera define una lista de cuentas de ahorro, que es lo que especifica el parámetro *CCuenta*.

Una programación que utiliza tipos como parámetros recibe el nombre de *programación genérica*. La idea de este capítulo es introducirle en el diseño, im-

plementación y utilización de clases genéricas. La biblioteca estándar de .NET, según hemos podido comprobar en los capítulos anteriores, presenta muchas de sus abstracciones como clases genéricas en el espacio de nombres **System.Collections.Generic**; por ejemplo, **List**, **Dictionary** y **SortedList**.

DEFINICIÓN DE CLASES GENÉRICAS

La definición de una clase genérica se hace según la sintaxis siguiente:

```
class nombre_clase<lista de parámetros> { ... }
```

Puede observarse que a continuación del nombre se especifican, entre ángulos, la lista de *parámetros de tipo* separados por comas, incluso pueden escribirse distintas sobrecargas de un tipo genérico que difieran en el número de *parámetros de tipo*. Por ejemplo:

```
public class MiClase<T1> { ... }  
public class MiClase<T1, T2> { ... }  
public class MiClase<T3, T4> { ... } // error
```

La última línea generaría un error durante la compilación porque ya hay un tipo *MiClase* con dos parámetros de tipo.

Los parámetros de la lista serán utilizados para implementar los métodos de la clase, bien como parámetros del método o como tipo del valor retornado. En general, diríamos que se pueden utilizar donde utilizamos cualquier otro tipo.

Como ejemplo, vamos a escribir una nueva versión de la clase *CVector* que implementamos en el capítulo 10. Esta nueva versión definirá la clase genérica *CVector<T>*. El código podría ser así:

```
using System;  
////////////////////////////////////  
// Definición de la clase genérica CVector  
//  
public class CVector<T>  
{  
    private T[] vector; // matriz vector  
    private int nElementos; // número de elementos de la matriz  
  
    // Crear una matriz con 10 elementos por omisión  
    public CVector()  
    {  
        nElementos = 10;  
        vector = new T[nElementos];  
    }  
}
```

```

// Crear una matriz con ne elementos
public CVector(int ne)
{
    if (ne < 1)
    {
        Console.WriteLine("Nº de elementos no válido: " + ne);
        Console.WriteLine("Se asignan 10 elementos");
        ne = 10;
    }
    nElementos = ne;
    vector = new T[nElementos];
}

// Crear una matriz a partir de otra matriz primitiva
public CVector(T[] m)
{
    nElementos = m.Length;
    vector = new T[nElementos];
    // Copiar los elementos de la matriz m
    for (int i = 0; i < nElementos; ++i)
        vector[i] = m[i];
}

// Constructor copia
public CVector(CVector<T> v)
{
    Copiar(v);
}

// Copiar un vector en otro
public CVector<T> Copiar(CVector<T> v)
{
    nElementos = v.nElementos;
    vector = new T[nElementos];
    // Copiar el objeto v
    for (int i = 0; i < nElementos; i++)
        vector[i] = v.vector[i];
    return this;
}

public int Longitud()
{
    return nElementos;
}

public T this[int ind] // declaración de un indizador
{
    get
    {
        if (ind < 0 || ind >= nElementos)
        {

```

```
        Console.WriteLine("Índice fuera de límites");
        return default(T);
    }
    else
    {
        return vector[ind];
    }
}
set
{
    if (!(ind < 0 || ind >= nElementos))
    {
        vector[ind] = value;
    }
    else
        Console.WriteLine("Índice fuera de límites");
}
}
```

En el código anterior se puede observar que los métodos están desarrollados en torno al tipo *T*.

Normalmente, para crear clases genéricas se empieza a partir de una clase concreta existente, cambiando los tipos que consideremos por parámetros de tipo.

Valor predeterminado para una variable de tipo *T*

Un problema asociado a las clases y métodos genéricos es cómo asignar un valor predeterminado a una variable de un tipo parametrizado *T* cuando no se sabe si *T* será un tipo referencia o un tipo valor, o bien si *T* es un tipo valor, si será un valor numérico o una estructura.

Dada una variable *v* de un tipo parametrizado *T*, la sentencia *v* = *null* sólo es válida si *T* es un tipo referencia y *v* = 0 sólo funcionará con los tipos valor numéricos, pero no con las estructuras. La solución es utilizar la palabra clave **default**, que devolverá **null** para los tipos referencia y 0 para los tipos valor numéricos. En el caso de las estructuras, devolverá cada miembro de la estructura iniciado con el valor *cero* o **null**, dependiendo de si son tipos valor o referencia. En el ejemplo anterior de la clase *CVector<T>*, el indizador muestra cómo utilizar la palabra clave **default** (vea más adelante, el apartado *Alternativas al método Obtener* del capítulo 16, la discusión sobre **default**):

```
public T this[int ind]    // declaración de un indizador
{
```

```

get
{
    if (ind < 0 || ind >= nElementos)
        return default(T);
    else
        return vector[ind];
}
set
{
    if (!(ind < 0 || ind >= nElementos))
        vector[ind] = value;
}
}

```

Parámetros de tipos genéricos

En una definición de tipo o método genérico, un parámetro de tipo es un marcador para un tipo especificado por una aplicación al crear un ejemplar del tipo genérico. Una clase genérica, como *CVector<T>*, no se puede utilizar tal cual, porque no es realmente un tipo, sino una plantilla de un tipo. Para utilizar *CVector<T>*, el código de la aplicación debe declarar y crear un objeto de un tipo construido especificando un argumento de tipo dentro de los corchetes angulares. Este argumento puede ser cualquier tipo reconocido por el compilador. Por ejemplo, a continuación, escribimos una clase aplicación que utiliza la clase genérica *CVector* para crear una colección de valores de tipo **double**:

```

////////////////////////////////////
// Aplicación que utiliza la clase genérica CVector
//
public class Test
{
    // Visualizar un vector
    public static void VisualizarVector(CVector<double> v)
    {
        for (int i = 0; i < v.Longitud(); i++)
            Console.Write(v[i] + " ");
        Console.WriteLine();
    }

    public static void Main(string[] args)
    {
        double[] x = { 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0 }; // matriz x
        CVector<double> vector1 = new CVector<double>(x);
        VisualizarVector(vector1); // escribe 1 2 3 ... 7
        CVector<double> vector2 = new CVector<double>(vector1);
        for (int i = 0; i < vector2.Longitud(); i++)
        {

```

```
        vector2[i] = vector2[i] * 10.0;
    }
    VisualizarVector(vector2); // escribe 10 20 30 ... 70
}
}
```

¿Cómo compila C# una clase genérica? Quizás, lo primero que se le venga a la mente sea que el compilador C# sustituirá el parámetro *T* por el tipo seleccionado; en el ejemplo, *T* por **double**. Pues no, el compilador cuando compila *CVector<T>*, o cualquier otro tipo genérico, lo hace igual que con cualquier tipo normal. Evidentemente, el código compilado contiene información adicional para saber que hay parámetros de tipo.

Durante la ejecución, cuando una aplicación, por ejemplo la aplicación *Test* anterior, haga referencia a *CVector<double>*, el sistema de ejecución comprobará si alguien ya pidió *CVector<double>*. Si no, construirá el tipo *CVector<double>* a partir del código compilado sustituyendo *T* por **double**:

```
public class CVector<double>
{
    private double[] vector; // matriz vector
    // ...
}
```

¿Y si definimos otro objeto *CVector* de tipo **int**?

```
int[] n = { 11, 12, 13, 14, 15, 16, 17 }; // matriz n
CVector<int> vector3 = new CVector<int>(n);
VisualizarVector(vector3); // escribe 11 12 13 ... 17
```

¿Crearé el sistema de ejecución otra clase sustituyendo *T* por **int**? Evidentemente, sólo si aún no existe el tipo *CVector<int>* porque el código ejecutado hasta el instante no lo solicitó, el sistema de ejecución lo construirá a partir del código compilado sustituyendo *T* por **int**:

```
public class CVector<int>
{
    private int[] vector; // matriz vector

    // ...
}
```

Si usted está familiarizado con las plantillas de C++, observará que éstas y las clases genéricas C# son similares, con la diferencia de que C++ construye un tipo específico a partir de uno genérico durante la compilación y C# lo hace durante la ejecución.

En realidad, las colecciones genéricas tienen un mejor comportamiento porque permiten al compilador verificar cualquier operación que añada un objeto a la colección, y porque el tipo del objeto recuperado de la colección es conocido, por lo tanto no hay necesidad de hacer una conversión explícita.

Herencia

Según hemos visto en el apartado anterior, una aplicación puede hacer referencia a la clase genérica especificando un argumento de tipo para crear un tipo construido cerrado (*ClaseB<int>*), pero también podría omitir la especificación del parámetro de tipo, por ejemplo al especificar una clase base genérica, para crear un tipo construido abierto (*ClaseB<T>*). Por ejemplo:

```
class ClaseA
{
    // ...
}

class ClaseB<T>
{
    // ...
}

class ClaseX<T> : ClaseA // Tipo concreto
{
    // ...
}

class ClaseY<T> : ClaseB<int> // Tipo construido cerrado
{
    // ...
}

class ClaseZ<T> : ClaseB<T> // Tipo construido abierto
{
    // ...
}
```

Las clases genéricas pueden heredar de clases base construidas cerradas o construidas abiertas.

Las clases no genéricas (concretas) pueden heredar de las clases base construidas cerradas, pero no de las clases construidas abiertas porque no hay ninguna forma de poder proporcionar el argumento de tipo necesario.

```
class ClaseC : ClaseB<int> // Tipo construido cerrado
{
    // ...
}
```

```
// ...  
}  
  
class ClaseD : ClaseB<T> // Error: tipo construido abierto  
{  
    // ...  
}
```

MÉTODOS GENÉRICOS

No solamente las clases se pueden parametrizar; los métodos también, ya sean constructores, métodos **static** o métodos no **static**. La sintaxis para declarar los *parámetros de tipo* de un método es la misma que hemos empleado para las clases. Por ejemplo, continuando con la aplicación anterior, cuando intentemos compilar el siguiente código se mostrará un error porque no existe una sobrecarga del método *VisualizarVector* con un parámetro de tipo *CVector<int>*:

```
public static void Main(string[] args)  
{  
    double[] x = { 1.0, 2.0, 3.0, 4.0, 5.0, 6.0, 7.0 }; // matriz x  
    CVector<double> vector1 = new CVector<double>(x);  
    VisualizarVector(vector1);  
  
    int[] n = { 11, 12, 13, 14, 15, 16, 17 }; // matriz n  
    CVector<int> vector3 = new CVector<int>(n);  
    VisualizarVector(vector3);  
}
```

La solución al problema planteado sería escribir un método genérico *VisualizarVector* con un parámetro de tipo *T* que represente el tipo de los datos contenidos en *CVector<T>*. Este método sería el siguiente:

```
public static void VisualizarVector<T>(CVector<T> v)  
{  
    for (int i = 0; i < v.Longitud(); i++)  
        Console.Write(v[i] + " ");  
    Console.WriteLine();  
}
```

Para llamar a este método podríamos hacerlo de cualquiera de las dos formas siguientes:

```
VisualizarVector<double>(vector1);  
VisualizarVector(vector1);
```

La primera forma especifica el tipo explícitamente en la llamada, en nuestro caso **double**, y en la segunda no.

Cuando se omite el argumento o argumentos de tipo, el compilador los obtiene de los argumentos especificados en la llamada (en el ejemplo de *vector1*); lógicamente, con métodos que no tienen parámetros es necesario especificar el tipo explícitamente en la llamada. Estas reglas se aplican tanto a los métodos estáticos, como a los métodos no estáticos de una clase.

Si define un método genérico que acepte los mismos parámetros de tipo que la clase contenedora, el compilador generará una advertencia porque en el método, el argumento suministrado para el tipo *T* ocultará el argumento suministrado para el tipo *T* de la clase. Para un caso como el especificado, considere proporcionar otro identificador para el parámetro de tipo del método.

```
class ClaseG<T>
{
    void MetodoX<T>() { } // Advertencia: T
}
```

Igual que expusimos para las clases genéricas, los métodos genéricos se pueden sobrecargar con diferente número de parámetros de tipo.

DELEGADOS GENÉRICOS

Un delegado puede definir también sus propios parámetros de tipo (los delegados fueron expuestos en el capítulo 10). Por ejemplo:

```
public delegate bool TipoOrdenacion<T>(T x, T y);

public static bool CompararDosElementos(double x, double y)
{
    return x > y;
}
```

```
TipoOrdenacion<double> delegado = CompararDosElementos;
```

Como ejemplo, en el capítulo 10, al exponer los delegados, añadimos a la clase *CVector* un método *Ordenar* que tenía un parámetro: un delegado que hacía referencia al método que definía el proceso a seguir para comparar dos datos del tipo de los elementos del vector a ordenar. Puesto que ahora *CVector* es una clase genérica, podemos generalizar también este delegado según se muestra en el código expuesto a continuación:

```
public delegate bool TipoOrdenacion<T>(T x, T y);

public class CVector<T>
{
```

```
private T[] vector; // matriz vector

// ...
public void Ordenar(TipoOrdenacion<T> comparar)
{
    // ...
}

public class Test
{
    // ...

    public static bool CompararDosElementos(double x, double y)
    {
        return x > y;
    }

    public static void Main(string[] args)
    {
        double[] a = new double[] { 1, 3, 5, 7, 2, 4, 6 }; // matriz a
        CVector<double> vector1 = new CVector<double>(a);

        TipoOrdenacion<double> delegado = CompararDosElementos;

        vector1.Ordenar(delegado);
        VisualizarVector(vector1);
    }
}
```

Evidentemente estamos suponiendo que los objetos *x* e *y* comparados son de un tipo capaz de ser ordenado. Un tipo cualquiera, por ejemplo *CFecha*, será un tipo ordenado si incluye uno o más métodos que permitan conocer si un objeto de ese tipo es menor que otro del mismo tipo (en el caso de *CFecha*, si una fecha es anterior a otra), lo que se podría conseguir sobrecargando los operadores de relación. En este sentido, quizás, en el ejemplo anterior, haya pensado definir el método *CompararDosElementos* como un método genérico. No lo hacemos porque el operador < no se puede aplicar a operandos de tipo *T*.

Un delegado puede definirse fuera o dentro de una clase. Cuando se defina dentro de una clase genérica podrá utilizar los parámetros de tipo de la clase genérica igual que lo hacen los métodos de clase. Puesto que el delegado genérico del ejemplo anterior participa del mismo parámetro de tipo que la clase genérica, podemos modificar la aplicación anterior así:

```
public class CVector<T>
{
    public delegate bool TipoOrdenacion(T x, T y);
```

```

private T[] vector; // matriz vector

// ...
public void Ordenar(TipoOrdenacion comparar)
{
    // ...
}

public class Test
{
    // ...

    public static void Main(string[] args)
    {
        double[] x = new double[] { 1, 3, 5, 7, 2, 4, 6 }; // matriz x
        CVector<double> vector1 = new CVector<double>(x);

        CVector<double>.TipoOrdenacion delegado = CompararDosElementos;

        vector1.Ordenar(delegado);
        VisualizarVector(vector1);
    }
}

```

Obsérvese que ahora, el código que declara el delegado *TipoOrdenacion* no define un parámetro *T* sino que utiliza el de la clase contenedora. También, en este caso, el código que hace referencia en la clase *Test* al delegado debe especificar el argumento de tipo de la clase contenedora, según muestra en el ejemplo.

Delegado genérico Predicate

Este delegado representa a un método que define un conjunto de criterios y determina si el objeto especificado cumple esos criterios. Su sintaxis es la siguiente:

public delegate bool Predicate<T>(T obj)

donde *T* es el tipo del objeto que se va a comparar y *obj* es el objeto que se va a comparar según los criterios definidos en el método representado por este delegado. Por ejemplo, la clase genérica **List<T>** define un método **Exists**:

```
public bool Exists(Predicate<T> delegado)
```

para determinar si **List** contiene elementos que cumplen las condiciones definidas por el predicado especificado. Veamos un ejemplo a continuación.

El siguiente ejemplo crea una lista de objetos **string** y antes de añadir un elemento nuevo a la lista invoca el método **Exists** de la misma para comprobar si ya existe, en cuyo caso no se añadirá.

```
using System;
using System.Collections.Generic;

public class Test
{
    private static List<string> lista;

    public static void Main()
    {
        // Lista de objetos string
        lista = new List<string>();
        lista.AddRange(new string[] { "aaa", "bbb", "ccc" });
        string nuevoElemento = "bbbb";
        // Delegado para Exists
        bool existe = lista.Exists(
            delegate(string elementoLista)
            {
                return elementoLista == nuevoElemento;
            }
        );
        // Añadir el elemento nuevo a la lista si no existe
        if (existe)
            Console.WriteLine("Ya existe el elemento " + nuevoElemento);
        else
            lista.Add(nuevoElemento);
    }
}
```

EJERCICIOS RESUELTOS

1. Añada a la clase genérica *CVector* el método **CompareTo** de la interfaz **IComparable** para que permita comparar dos objetos *CVector* así:

vector1.CompareTo(vector2)

Este método devolverá:

- 0 si *vector1* y *vector2* tienen el mismo número de elementos y sus pares de elementos correspondientes (los que ocupan la misma posición) son iguales.
- 1 si *vector1* tiene menos elementos que *vector2*.

- 1 si *vector1* tiene más elementos que *vector2* o tiene el mismo número de elementos pero sus pares de elementos correspondientes no son iguales.

```
using System;
```

```
public class CVector<T> : IComparable<CVector<T>>
{
    // Delegado
    public delegate bool TipoOrdenacion(T x, T y);

    private T[] vector; // matriz vector
    // ...
```

```
public int CompareTo(CVector<T> obj)
{
    if (vector.Length == obj.vector.Length)
        return 0;
    else if (vector.Length < obj.vector.Length)
        return -1;
    else // vector.Length > obj.vector.Length
        return 1;
}
```

```
public int CompareTo(CVector<T> obj, TipoOrdenacion comparar)
{
    bool iguales = true;
    if (vector.Length == obj.vector.Length)
    {
        for (int i = 0; iguales && i < vector.Length; ++i)
            if (comparar(vector[i], obj.vector[i])) iguales = false;
        if (iguales) return 0; else return 1;
    }
    return this.CompareTo(obj);
}
}
```

El método **CompareTo** de la interfaz **IComparable** es un método con el siguiente formato:

```
int CompareTo(T obj); // valor retornado: -1 (>), 0 (==), -1 (<)
```

Para escribirlo en la clase *CVector*, ésta debe implementar la interfaz a la que pertenece. La redefinición del método tiene que hacerse respetando el prototipo proporcionado; en la primera sobrecarga que hemos escrito de este método y que no cubre nuestros objetivos, de ahí que hayamos introducido una segunda sobrecarga que permita comparar dos objetos *CVector*.

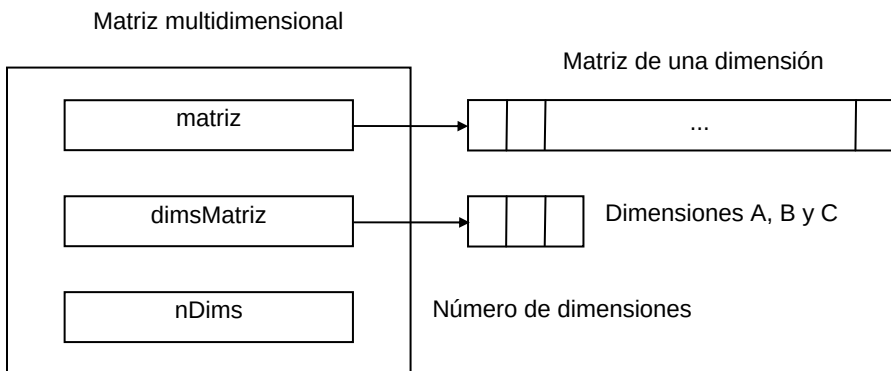
EJERCICIOS PROPUESTOS

1. En el apartado de ejercicios resueltos del capítulo 10 escribimos una clase para manipular una matriz multidimensional construida sobre la base de una matriz unidimensional. Para ello, definiremos una clase *CMatriz* con los siguientes atributos:

```
public class CMatriz
{
    private double[] matriz;    // matriz unidimensional
    private int nDims;          // número de dimensiones
    private int[] dimsMatriz;   // valor de cada dimensión
    // ...
};
```

Observe que el miembro *matriz* sirve para referenciar una matriz de una dimensión de elementos de tipo **double**, que el miembro *nDims* contiene el número de dimensiones y *dimsMatriz* es una referencia a una matriz que contendrá el valor de cada una de ellas.

Para tres dimensiones, la representación gráfica de la estructura de datos construida es la siguiente:



Partiendo de esta definición, escriba una clase genérica *CMatriz<T>* que permita manipular matrices multidimensionales de tipo *T*.

EXCEPCIONES

El lenguaje C# incorpora soporte para manejar situaciones anómalas, conocidas como “excepciones”, que pueden ocurrir durante la ejecución de un programa. Con el sistema de manipulación de excepciones de C#, un programa puede comunicar eventos inesperados a un contexto de ejecución más capacitado para responder a tales eventos anormales. Estas excepciones son manejadas por código fuera del flujo normal de control del programa.

Las excepciones proporcionan una manera limpia de verificar errores; esto es, sin abarrotar el código básico de una aplicación utilizando sistemáticamente los códigos de retorno de los métodos en sentencias **if** y **switch** para controlar los posibles errores que se puedan dar. Veamos con un ejemplo, a qué nos estamos refiriendo:

```
int códigoDeError = 0;
códigoDeError = leerFichero(nombre);
if ( códigoDeError != 0 )
{
    // Ocurrió un error al leer el fichero
    switch( códigoDeError )
    {
        case 1:
            // No se encontró el fichero
            // ...
            break;
        case 2:
            // El fichero está corrupto
            // ...
            break;
        case 3:
            // El dispositivo no está listo
            // ...
    }
}
```

```
        break;
    default:
        // Otro error
        // ...
        break;
    }
}
else
{
    // Procesar los datos leídos del fichero
}
```

El código del ejemplo anterior trata de leer un fichero almacenado en el disco invocando al método *leerFichero*. Este método devuelve un valor 0 si se ejecuta satisfactoriamente y un valor distinto de 0 en otro caso. Para analizar este hecho se ha utilizado una sentencia **if**. En el caso de que se produzca un error, una sentencia **switch** se encargará de verificar qué es lo que ha ocurrido y de tratar de resolverlo de la mejor forma posible. Lo que se persigue es que el programa no sea abortado inesperadamente por el sistema, sino diseñar una continuación o terminación normal dentro de lo ocurrido.

Observemos el código que ha sido necesario escribir para tratar un posible error debido a no poder leer un fichero del disco. Pensemos, ¿cuántos errores más podrían abortar nuestra aplicación? Para que esto no suceda, ¿se imagina la complejidad del código escrito una vez añadido todo el necesario para tratar cada uno de ellos? El manejo de excepciones ofrece una forma de separar explícitamente el código que maneja los errores del código básico de una aplicación, haciéndola más legible, lo que desemboca en un buen estilo de programación. Por ejemplo:

```
try
{
    // Código de la aplicación
}
catch(clase_de_excepción e)
{
    // Código de tratamiento de esta excepción
}
catch(otra_clase_de_excepción e)
{
    // Código de tratamiento para otra clase de excepción
}
```

Básicamente, el esquema anterior dice que si el código de la aplicación no puede realizar alguna operación, se espera lance una excepción que será tratada por el código de tratamiento especificado para esa clase de excepción, o en su defecto por el sistema de ejecución de .NET.

A lo largo de este capítulo comprobará que: el manejo de excepciones reduce la complejidad de la programación; los métodos que invocan a otros no necesitan comprobar valores de retorno; si el método invocado finaliza de forma normal, el que llamó está seguro de que no ocurrió ninguna situación anómala; etc.

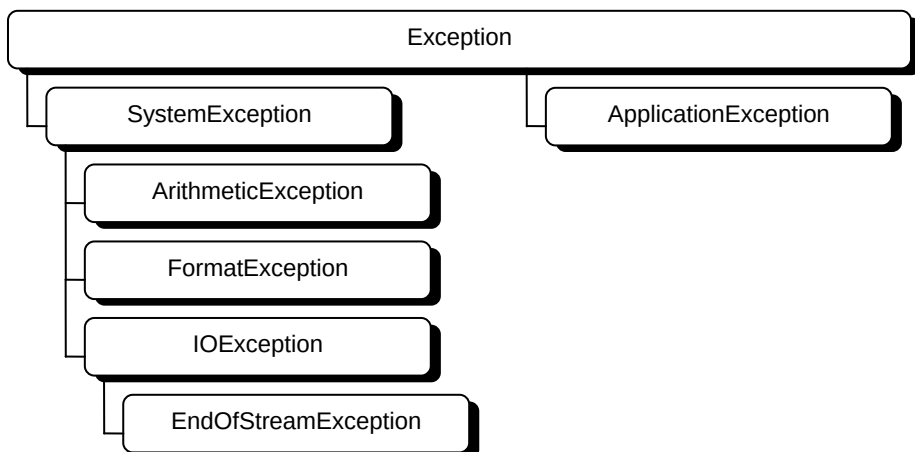
EXCEPCIONES DE C#

Durante el estudio de los capítulos anteriores, seguro que se habrá encontrado con excepciones como las siguientes:

Clase de excepción	Significado
DivideByZeroException	Se produce cuando se intenta dividir un valor entero o decimal entre cero.
IndexOutOfRangeException	Una matriz fue accedida con un índice ilegal (fuera de los límites permitidos).
NullReferenceException	Se intentó utilizar null donde se requería un objeto.
FormatException	Se produce cuando el formato de un argumento no es el adecuado.

¿Qué es lo que ocurrió entonces cuando durante la ejecución de su programa se lanzó una excepción? Seguramente el programa dejó de funcionar y el sistema de ejecución visualizó algún mensaje acerca de lo ocurrido. Si no es esto lo que deseamos, tendremos que aprender a manipular las excepciones.

Las excepciones en C# son objetos de clases derivadas de la clase **Exception** definida en el espacio de nombres **System**. Por ejemplo, cuando se lanza una excepción **ArithmeticException**, automáticamente C# crea un objeto de esta clase. La figura siguiente muestra algunas de las clases de la jerarquía de excepciones:



La clase **Excepción** cubre las excepciones que una aplicación normal puede manipular. Tiene varias clases derivadas entre las que destacan: **SystemException** y **ApplicationException**.

La clase **SystemException** define la clase base para las excepciones predefinidas en el espacio de nombres **System**. El motor de ejecución de .NET, tras realizar comprobaciones sobre los datos que intervienen en una determinada operación, lanzará la excepción adecuada derivada de **SystemException** si tal comprobación no resulta satisfactoria; por ejemplo, errores de índices fuera de límites en una matriz. Esto puede suceder durante la ejecución de cualquier método.

La clase **ApplicationException** cubre las excepciones lanzadas por las acciones llevadas a cabo por un programa del usuario y no por el motor de ejecución de .NET (CLR).

La clase **IOException** cubre las excepciones ocurridas al ejecutar una operación de entrada o salida. Este grupo de excepciones pertenece al espacio de nombres **System.IO**.

Cuando diseñe una aplicación que cree sus propias excepciones, derive esas excepciones de la clase **Exception**.

Resumiendo, una excepción se genera cuando en una aplicación en ejecución ocurre cualquier situación de error o comportamiento inesperado. Las excepciones se pueden lanzar a causa de un error en el código de la aplicación o en el código al que se llama (como una biblioteca compartida), porque no estén disponibles recursos del sistema operativo, por condiciones inesperadas que encuentra el CLR (por ejemplo, código que no se puede comprobar), etc. De todas estas excepciones, la aplicación se puede recuperar de algunas de ellas, pero de otras no. Si bien es posible recuperarse de la mayoría de las excepciones derivadas de **ApplicationException**, no lo es de la mayoría de excepciones derivadas de **SystemException**.

A continuación aprenderá con detalle cómo capturar, crear y lanzar excepciones, además de otras cosas.

MANEJAR EXCEPCIONES

Cuando un método se encuentra con una anomalía que no puede resolver, lo lógico es que *lance* (**throw**) una excepción, esperando que quien lo llamó directa o indirectamente la *capture* (**catch**) y maneje la anomalía. Incluso él mismo podría

capturar y manipular dicha excepción. Si la excepción no se captura, la aplicación finalizará automáticamente.

Por ejemplo, ¿recuerda la clase *Leer* desarrollada en el capítulo 6? Según puede observar a continuación, el método *datoDouble* de esta clase invoca al método **Parse** con el propósito de devolver el valor **double** correspondiente a la cadena leída. Pero sucede que **Parse** puede lanzar una excepción de la clase **FormatException** si la cadena de caracteres a analizar no se corresponde con un valor **double**. Para manejar esta situación hay que capturar la excepción cuando se lance, para lo cual se utiliza un bloque **catch**, y para poder capturarla hay que encerrar el código que puede lanzarla en un bloque **try**.

```
using System;

namespace MisClases.ES // espacio de nombres
{
    public class Leer    // clase Leer
    {
        // ...
        public static double datoDouble()
        {
            try
            {
                return Double.Parse(Console.ReadLine());
            }
            catch(FormatException)
            {
                return Double.NaN; // No es un Número; valor double.
            }
        }
    }
}
```

Las palabras **try** y **catch** trabajan conjuntamente y pueden traducirse así: “poner a prueba un fragmento de código por si lanzara una excepción; si se ejecuta satisfactoriamente, continuar con la ejecución del programa; si no, capturar la excepción lanzada y manejarla”.

Lanzar una excepción

Lanzar una excepción equivale a crear un objeto de la clase de la excepción para manipularlo fuera del flujo normal de ejecución del programa. Para lanzar una excepción se utiliza la palabra reservada **throw** y para crear un objeto, **new**. Por ejemplo, volviendo al método *datoDouble* de la clase *Leer* expuesta anteriormente, si ocurre un error cuando se ejecute el método **Parse** se supone que éste ejecutará una sentencia similar a la siguiente:

```
if (error) throw new FormatException();
```

Esta sentencia lanza una excepción de la clase **FormatException** lo que implica crear un objeto de esta clase. Un objeto de éstos contiene información acerca de la excepción, incluyendo su tipo y el estado del sistema cuando el error ocurrió.

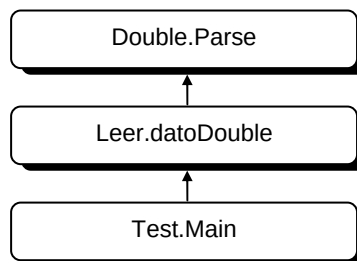
Capturar una excepción

Una vez lanzada la excepción, el sistema es responsable de encontrar a alguien que la capture con el objetivo de manipularla. El conjunto de esos “alguien” es el conjunto de métodos especificados en la pila de llamadas hasta que ocurrió el error. Por ejemplo, consideremos la siguiente aplicación, que invoca al método *datoDouble* de la clase *Leer* con la intención de leer un dato:

```
public class Test
{
    public static void Main(string[] args)
    {
        double d;
        Console.Write("Dato: ");
        d = Leer.datoDouble();

        // ...
    }
}
```

Cuando se ejecute esta aplicación y se invoque al método *datoDouble*, la pila de llamadas crecerá como se observa en la figura siguiente:



Si al ejecutarse el método **Parse** ocurriera un error, según hemos visto anteriormente, éste lanzaría una excepción de la clase **FormatException** que interrumpirá el flujo normal de ejecución. Después, el sistema buscaría en la pila de llamadas hacia abajo y comenzando por el propio método que produjo el error, uno que implemente un manejador que pueda capturar esta excepción. Si el siste-

ma, descendiendo por la pila de llamadas, no encontrara este manejador, el programa terminaría.

Para implementar un manejador para una clase de excepción hay que hacer las dos cosas que se indican a continuación:

1. Encerrar el código que puede lanzar la excepción en un bloque **try**. En la clase *Leer* presentada anteriormente, el método *datoDouble* tiene un bloque **try** que encierra la llamada al método **Parse**, además de a otras sentencias:

```
try
{
    return Double.Parse(Console.ReadLine());
}
```

2. Escribir un bloque **catch** capaz de capturar la excepción lanzada. En la clase *Leer* presentada anteriormente, el método *datoDouble* tiene un bloque **catch** capaz de capturar excepciones de la clase **FormatException** y de sus clases derivadas:

```
catch(FormatException e)
{
    Console.WriteLine("Error: " + e.Message);
    return Double.NaN; // No es un Número; valor double.
}
```

En este manejador se observa un parámetro *e* que referencia al objeto que se creó cuando se lanzó la excepción capturada. Para manipularla, además de escribir el código que consideremos adecuado, disponemos de la funcionalidad proporcionada por la clase **FormatException**, y a la que podremos acceder mediante el objeto *e*. Por ejemplo, la propiedad **Message** devuelve una cadena con información acerca de la excepción ocurrida.

Si el parámetro del manejador no se va a utilizar, se puede omitir como se muestra en el ejemplo siguiente:

```
catch(FormatException)
{
    return Double.NaN; // No es un Número; valor double.
}
```

Un manejador de excepción, **catch**, sólo se puede utilizar a continuación de un bloque **try** o de otro manejador de excepción (bloque **catch**). Las palabras clave **try** y **catch**, por definición, van seguidas de un bloque que encierra el código relativo a cada una de ellas, razón por la cual es obligatorio utilizar llaves: **{}**.

El siguiente ejemplo muestra un manejador para una excepción de tipo *ValorNoValidoException* y otro para una excepción de tipo **Exception**:

```
catch (ValorNoValidoException e)
{
    // Manejador de la excepción ValorNoValidoException
}
catch (Exception e)
{
    // Manejador para cualquier otra excepción
}
```

Cuando una excepción es capturada se considera manejada, lo que significa que cualquier otro manejador existente no será tenido en cuenta.

Excepciones derivadas

Cuando se trata de manejar excepciones, un bloque **try** puede estar seguido de uno o más bloques **catch**, tantos como excepciones diferentes tengamos que manejar. Cada **catch** tendrá un parámetro de la clase **Exception**, de alguna clase derivada de ésta, o bien de una clase de excepción definida por el usuario. Cuando se lance una excepción, el bloque **catch** que la capture será aquél cuyo parámetro sea de la clase de la excepción o de una clase base directa o indirecta. Debido a esto, el orden en el que se coloquen los bloques **catch** tiene que ser tal, que cualquiera de ellos debe permitir alcanzar el siguiente, de lo contrario el compilador producirá un error.

Por ejemplo, si el primer bloque **catch** especifica un parámetro de la clase **Exception**, ningún otro bloque que le siga podría alcanzarse; esto es, cualquier excepción lanzada sería capturada por ese primer bloque, ya que cualquier referencia a una clase derivada puede ser convertida implícitamente por C# en una referencia a su clase base directa o indirecta.

En cambio, en el ejemplo siguiente, una excepción de la clase **EndOfStreamException** será capturada por el primer bloque **catch**; una excepción de la clase **IOException** será capturada por el bloque segundo; una excepción de la clase **FileNotFoundException**, clase derivada de **IOException**, será capturada también por el bloque segundo; y una excepción de la clase **ApplicationException**, clase derivada de **Exception**, será capturada por el bloque tercero.

```
try
{
    // ...
}
```

```

catch(EndOfStreamException e)
{
    // Manejar esta clase de excepción
}
catch(IOException e)
{
    // Manejar esta clase de excepción o de alguna de sus clases
    // derivadas, excepto EndOfStreamException
}
catch(Exception e)
{
    // Manejar esta clase de excepción o de alguna de sus clases
    // derivadas, excepto EndOfStreamException e IOException
}

```

Capturar cualquier excepción

Especificar **catch** sin parámetros significa “capturar cualquier excepción”. Por ejemplo, en el código del ejemplo anterior podríamos sustituir el último bloque **catch** por otro que no especifique el tipo de excepción a capturar; en este caso se supone el tipo **Exception**, lo que permite capturar todas las excepciones. Incluso podríamos a continuación relanzar la excepción para que fuera tratada por otro manejador.

```

// ...
catch
{
    // Finalización adecuada para el proceso en curso.
    throw; // lanzar otra vez la excepción
}

```

Relanzar una excepción

Después de haber capturado una excepción, el manejador puede volverla a lanzar; esto se hace ejecutando **throw** sin ningún operando, según se puede observar en el ejemplo anterior. Esto puede ser útil en los casos en los que el manejador que capturó una excepción decida que no puede tratar completamente el error, por ejemplo, porque no está disponible en este lugar toda la información que necesita, en cuyo caso hace lo que puede y después vuelve a lanzar la excepción otra vez, con la intención de que el tratamiento pueda ser continuado por otro manejador.

BLOQUE DE FINALIZACIÓN

Si no se trata de manejar excepciones, sino de realizar alguna acción necesaria después de salir del bloque **try** (por ejemplo, liberar algún recurso externo que se

haya adquirido, cerrar un fichero, etc.) ponga el código adecuado dentro de un bloque **finally** colocado después del bloque **try** o de un bloque **catch**. El bloque **finally** deber ser siempre el último.

La ejecución del bloque **finally** queda garantizada independientemente de que finalice o no la ejecución del bloque **try**. Quiere esto decir que aunque se abandone la ejecución del bloque **try** porque, por ejemplo, se ejecute una sentencia **return**, el bloque **finally** se ejecuta.

Para aclarar lo expuesto analicemos el siguiente ejemplo. Se trata de un método para escribir en un fichero datos procedentes de una matriz. ¿Recuerda la aplicación *Test* que escribimos en el capítulo 12 que operaba sobre un objeto *CBanco*? Pues, suponga ahora que queremos añadir al menú que presentaba esta aplicación una opción más que permita escribir en un fichero en disco una lista con los nombres de los clientes del banco (en el capítulo siguiente aprenderemos a trabajar con ficheros). Para ello, añadiremos a la clase *Test* el método que se expone a continuación. Dicho método se ejecutará cuando se seleccione esa opción:

```
public static void EscribirDatos(CBanco banco, string fich)
{
    StreamWriter fcli = null;
    CCuenta cliente;

    CListaClientes lista = new CListaClientes(banco.Longitud());
    try
    {
        for (int i = 0; i < banco.Longitud(); i++)
        {
            cliente = banco.Obtener(i);
            lista.Añadir(cliente.ObtenerNombre(), i);
        }

        // Abrir el fichero para escribir. Se crea el flujo fcli;
        fcli = new StreamWriter(fich);
        lista.Escribir(fcli);
    }
    catch(IOException e)
    {
        Console.WriteLine(e.Message);
    }
    finally
    {
        // Cerrar el fichero
        if (fcli != null) fcli.Close();
    }
}
```


El objeto *lista* de la clase *CListaClientes* (el ejemplo completo puede encontrarlo en la carpeta *Cap14\finally* del CD) encapsula una matriz que almacenará la lista de los clientes del banco. El método *Añadir* puede lanzar una excepción ***IndexOutOfRangeException*** si el índice utilizado para acceder a los elementos de la matriz encapsulada en el objeto *CListaClientes* tiene un valor fuera de los límites establecidos. Los métodos *StreamWriter* y *Escribir* pueden lanzar una excepción ***IOException*** si el fichero no puede abrirse, o bien ocurre un error de escritura. Las excepciones no estamos obligados a manejarlas, pero es conveniente hacerlo siempre que exista la posibilidad de que se puedan lanzar. Por eso se ha añadido un manejador para la excepción ***IOException***. Finalmente se ha añadido un bloque ***finally*** para garantizar que, ocurra lo que ocurra, el fichero será cerrado cuando se abandone el método *escribirDatos*.

¿Es realmente necesario el bloque ***finally***? En el ejemplo anterior, el método *EscribirDatos* no proporciona un manejador de excepciones por si ocurre un error durante la ejecución del método *Añadir* (añade un nombre en la posición *i* de la matriz). Entonces, si ocurriera un error porque el índice pasado estuviera fuera de límites, ¿cómo cerraríamos el fichero? La respuesta es: el bloque ***finally*** es lo último que se ejecuta antes de abandonar el método *EscribirDatos*, lo que ocurrirá cuando:

1. El bloque ***try*** finalice de ejecutarse satisfactoriamente.
2. Se lance una excepción ***IOException***.
3. Se lance una excepción ***IndexOutOfRangeException***.

En el primer caso, después del bloque ***try*** se ejecutará el bloque ***finally*** y se saldrá del método *EscribirDatos*. En el segundo caso, se ejecutará el bloque ***catch*** proporcionado por *EscribirDatos*, después el bloque ***finally*** y se saldrá del método. Y en el tercer caso, se ejecutará el bloque ***catch*** proporcionado por el sistema, después el bloque ***finally*** y se saldrá del método.

Si hubiéramos incluido un manejador para el caso 3, podríamos cerrar el fichero en los bloques ***try*** y ***catch*** y prescindir del bloque ***finally***, pero a costa de duplicar código y hacer menos legible el programa.

Los bloques ***try*** y ***finally*** pueden también utilizarse conjuntamente, sin que sea necesario incluir un bloque ***catch***.

CREAR EXCEPCIONES

En alguna ocasión puede que necesitemos crear nuestras propias excepciones, a pesar de que en la biblioteca de clases de .NET hay una gran cantidad de ellas que podemos utilizar sin más. En cualquier caso, todos los tipos de excepción se co-

rresponden con una clase derivada de **Exception**, clase raíz de la jerarquía de clases de excepciones de C#.

La clase **Exception** incluye propiedades que ayudan a identificar la ubicación del código, el tipo, el fichero de ayuda y el motivo de la excepción:

Data	Permite obtener una colección de pares clave-valor que proporcionan información adicional definida por el usuario acerca de la excepción.
HelpLink	Permite obtener o establecer un vínculo al fichero de ayuda asociado a esta excepción.
InnerException	Permite obtener el objeto Exception que causó la excepción actual.
Message	Permite obtener un mensaje de texto para informar de la naturaleza del error y recomendar alguna acción para solucionar el problema.
Source	Devuelve o establece el nombre de la aplicación o del objeto que generó el error.
StackTrace	Permite obtener el estado de la pila de llamadas en el momento en que se produjo la excepción. Puede utilizarse para determinar la parte del código en la que se ha producido un error, ya que muestra todos los métodos a los que se ha llamado y los números de línea del fichero de código fuente donde se realizan las llamadas.
TargetSite	Permite obtener el método que produce la excepción actual.

Durante la creación del objeto de excepción, puede pasarse una cadena de texto al constructor para que se describan los detalles de esa excepción en concreto. Si no se le proporciona al constructor ningún argumento de mensaje de error, se utiliza el mensaje de error predeterminado. Por ejemplo:

```
public void Añadir(string nombre, int i)
{
    if (i < 0 || i >= nElementos)
        throw new Exception("El índice es " + i +
                             " y tiene que ser >= 0 y < " + nElementos);
    listaClientes[i] = nombre + "\n";
}
```

Los errores que ocurren en la mayoría de los programas se corresponden con excepciones de alguna de las clases derivadas de **Exception**, razón por la que esta clase será la clase base directa o indirecta de las nuevas clases de excepción que creemos.

En general, crearemos un nuevo tipo de excepción cuando queramos manejar un determinado tipo de error no contemplado por las excepciones proporcionadas por la biblioteca de .NET. Por ejemplo, para crear un tipo de excepción *ValorNoValidoException*, con la intención de manejar un error “valor no válido”, podemos diseñar una clase como la siguiente:

```
public class ValorNoValidoException : Exception
{
    public ValorNoValidoException() {}
    public ValorNoValidoException(string mensaje) : base(mensaje) {}

    // ...
}
```

Según se observa en este ejemplo, la clase base de la nueva clase de excepción *ValorNoValidoException* es **Exception**, e implementa dos constructores: uno sin parámetros y otro con un parámetro de tipo **string**; esto es lo más habitual. El parámetro de tipo **string** es el mensaje que devolverá la propiedad **Message** heredada de la clase **Exception**. Para ello, el constructor *ValorNoValidoException* debe invocar al constructor de la clase base y pasar como argumento dicha cadena, la cual será almacenada en el miembro de datos correspondiente de la clase **Exception**. Es una buena práctica de programación finalizar el nombre de sus clases de excepción con la palabra “Exception”.

La clase de excepción *ValorNoValidoException* relacionada con el error “valor no válido” ya está creada. Lógicamente, siempre que se implemente una clase de excepción es porque durante el desarrollo de una clase, por ejemplo *CMiClase*, se ha observado que su código para determinados valores durante la ejecución puede presentar una anomalía de la que los usuarios de esa clase deben ser informados para que la puedan tratar.

¿Qué aspecto tiene *CMiClase*? Según lo expuesto, el código que implementa esta clase ante determinados valores produce un error. Habrá entonces que añadir el código que chequee si se producen esos valores y en caso afirmativo lanzar la excepción programada para este caso. Por ejemplo:

```
public class CMiClase
{
    // ...
}
```

```
public void m(int a)
{
    // ...
    if (a == 0)
        throw new ValorNoValidoException("Error: valor cero");
    // ...
}
// ...
}
```

Según hemos estudiado anteriormente, lanzar una excepción equivale a crear un objeto de ese tipo de excepción. En el ejemplo se observa que la circunstancia que provocará el error es que el parámetro *a* del método *m* de *CMiClase* sea 0; en este caso, el método *m* lanza (**throw**) una excepción de la clase *ValorNoValidoException* creando un objeto de esta clase. Para crear (**new**) ese objeto se invoca al constructor *ValorNoValidoException* pasando como argumento, en este caso, la cadena “Error: valor cero”.

FLUJO DE EJECUCIÓN

Supongamos que necesitamos trabajar con la clase *CMiClase* implementada en el apartado anterior, cuyo método *m* declara una excepción de tipo *ValorNoValidoException*. Un método de cualquier otra clase que utilice el método *m* de esta clase debe detectar esa posible anomalía, de lo contrario cuando se produzca esa anomalía el programa mostrará un error y finalizará. Dicho método expresará esa necesidad encerrando el código que puede *intentar* (**try**) producir tal anomalía en un bloque **try** con un manejador para esa excepción. Por ejemplo:

```
public class Test
{
    public static void Main(string[] args)
    {
        int x = 0;
        CMiClase obj = new CMiClase();
        try
        {
            obj.m(x);
        }
        catch (ValorNoValidoException e)
        {
            Console.WriteLine(e.Message);
        }

        Console.WriteLine("Continúa la ejecución");
    }
}
```

Tenga en cuenta que un manejador tiene alcance a las variables locales del método donde se ha definido pero no a las variables locales definidas en el bloque **try** o en otros bloques **catch**.

Cuando un método utilizando **throw** lanza una excepción, crea un objeto de la clase de excepción especificada, que interrumpe el flujo de ejecución del programa y vuelve por la pila de llamadas hasta encontrar uno que sepa capturar la excepción (que contenga un bloque **catch** con un argumento de la clase de la excepción o de alguna de sus clases base). La ejecución del programa se transfiere entonces directamente al método que capturó la excepción para que ejecute el manejador. Si el manejador, una vez ejecutado, permite que la aplicación continúe, la ejecución se transfiere a la primera línea ejecutable que haya a continuación del último manejador del bloque **try**. Según esto, cuando se ejecute el método **Main** del ejemplo anterior se obtendrá el siguiente resultado:

```
Error: valor cero
Continúa la ejecución
```

Si un método lanza una excepción y en la vuelta por la pila de llamadas no se encuentra otro que la capture, el programa finalizará. En cambio, si se encuentra un manejador para esa excepción, se ejecuta. En el supuesto de que en la pila de llamadas quedaran otros métodos que pudieran capturarla, no serán tenidos en cuenta; esto es, sólo se tiene en cuenta el manejador del método por el que haya pasado el flujo de control más recientemente.

A su vez, si el método contiene una lista de manejadores sólo se ejecutará el correspondiente a la excepción lanzada; esto es, el comportamiento es el mismo que el de una sentencia **switch**, pero con la diferencia de que los **case** necesitan sentencias **break** y los **catch** no.

Según hemos visto, una excepción se captura en un bloque **catch** que declare un argumento de su clase o clase base; pero como lo que se lanza es un objeto (**throw new...** especifica una llamada al constructor de la clase de excepción), si necesitáramos transmitir información adicional desde el punto de lanzamiento al manejador, lo podremos hacer dotando de parámetros al constructor.

Una excepción se considera manejada desde el momento en que se entra en su manejador, así que cualquier otra excepción lanzada desde el cuerpo de éste deberá ser capturada por algún otro método cuya llamada se encuentre en la pila; si la excepción no es capturada, el programa finaliza. Esto explica por qué el siguiente código no provoca un bucle infinito:

```
public static void OtroMétodo()
{
```

```
// ...
try
{
    // ...
}
catch(ValorNoValidoException e)
{
    // ...
    throw new ValorNoValidoException();
}
// ...
}
```

Según lo expuesto anteriormente, una vez capturada una excepción, el manejador puede decidir volver a lanzarla para que sea procesada por otro manejador. En el ejemplo anterior se puede observar que *OtroMétodo* declara que puede lanzar una excepción de la clase *ValorNoValidoException*.

También podría suceder que el propio método que lanza la excepción la capturara, como puede observar en el ejemplo siguiente:

```
public class CMiClase
{
    // ...
    public void m(int a)
    {
        try
        {
            if (a == 0)
                throw new ValorNoValidoException("Error: valor cero");
            // ...
        }
        catch (ValorNoValidoException e)
        {
            Console.WriteLine(e.Message);
        }
        // ...
    }
    // ...
}
```

Lo que sucede es que escribir un método que lance una o más excepciones y que él mismo las capture es anticiparnos a las necesidades que pueda tener el usuario de la clase que proporciona ese método, en cuanto al tratamiento de la excepción se refiere.

Combinar ambas formas (declarar la excepción y además capturarla) no sirve de nada cara al usuario, porque si un método lanza una excepción y la captura, en

el supuesto de que en la pila de llamadas quedaran otras que pudieran capturarla, no serán tenidas en cuenta; esto es, sólo se ejecuta el manejador del método por el que haya pasado el flujo de control más recientemente.

CUÁNDO UTILIZAR EXCEPCIONES Y CUÁNDO NO

No todos los programas necesitan responder lanzando una excepción a cualquier situación anómala que se produzca. Por ejemplo, si partiendo de unos datos de entrada estamos haciendo una serie de cálculos más o menos complejos con la única finalidad de observar unos resultados, quizás la respuesta más adecuada a un error sea interrumpir sin más el programa, no antes de haber lanzado un mensaje apropiado y haber liberado los recursos adquiridos que aún no hayan sido liberados. Otro ejemplo, podemos utilizar la clase de excepción **IndexOutOfRangeException** para manejar el error que se produce cuando se rebasan los límites de una matriz o de una colección, pero quizás sea más conveniente anticiparnos utilizando la propiedad **Length** de la matriz o la propiedad **Count** de la colección en una sentencia **if** para prevenir que esto no suceda. Por ejemplo:

```
public CPersona Registro(int i)
{
    if (i >= 0 && i < listaTeléfonos.Count)
        return listaTeléfonos[i];
    else
    {
        Console.WriteLine("Índice fuera de límites");
        return null;
    }
}
```

En cambio, si estamos construyendo una biblioteca estamos obligados a evitar todos los errores que se puedan producir cuando su código sea ejecutado por cualquier programa que la utilice.

Por último, no todas las excepciones tienen que servir para manipular errores. Puede también manejar excepciones que no sean errores.

EJERCICIOS RESUELTOS

1. Modificar la aplicación realizada en el capítulo 10 sobre el mantenimiento de una lista de teléfonos, para que el método *Registro* lance la excepción “índice fuera de límites” cuando sea preciso. Recuerde que el método *Registro* devolvía el objeto *CPersona* que estaba en la posición *i* de la matriz *listaTeléfonos*, o un valor **null** si la posición especificada estaba fuera de límites.

Entre las clases de excepciones que proporciona C# está **IndexOutOfRangeException**. **Exception**: excepción lanzada cuando al acceder a un elemento de una matriz o de una colección el índice está fuera de límites; ésta es la excepción que lanzará el operador de indexación invocado por *Registro* cuando al acceder a un elemento de la colección, $i < 0 \parallel i \geq \text{listaTeléfonos.Count}$. Entonces, podemos escribir el método *Registro* así:

```
public CPersona Registro(int i)
{
    return listaTeléfonos[i];
}
```

Ahora habrá que capturar la excepción de tipo **IndexOutOfRangeException** que puede lanzar el operador de indexación cuando invoquemos a *Registro*. Esto se puede hacer en diferentes puntos. Puesto que *Registro* es invocado desde el método *Buscar* de *Test* y éste es invocado desde **Main**, podemos capturar la excepción en cualquiera de esos métodos. No lo haremos en *Registro* porque en el tratamiento de la excepción tendría que devolver **null** y esto no es solución. Lo haremos en *Buscar* o en **Main** de *Test* como se indica a continuación:

- En el método *Buscar* simplemente añadiríamos un bloque **catch** como el siguiente:

```
public static void Buscar(CListaTfnos listatfnos,
                        bool buscar_siguiente)
{
    // ...
    try
    {
        // código que invoca al método Registro
    }
    catch(IndexOutOfRangeException e)
    {
        Console.WriteLine("Error: índice fuera de límites");
    }
}
```

- En el método **Main** podemos añadir la sentencia **try ... catch** a todo o a parte de su código. Por ejemplo:

```
do
{
    opción = menú();
    try
    {
        switch (opción)
        {
```



```

        case 1: // buscar
            Buscar(listatfnos, false);
            break;
        case 2: // buscar siguiente
            Buscar(listatfnos, true);
            break;
        case 3: // añadir
            Añadir(listatfnos);
            break;
        case 4: // eliminar
            Eliminar(listatfnos);
            break;
        case 5: // salir
            listatfnos = null;
    }
}
catch(IndexOutOfRangeException)
{
    Console.WriteLine("Error: índice fuera de límites");
}
}
while(opción != 5);

```

2. En el capítulo 6 implementamos una clase *Leer* con los miembros siguientes: *datoShort*, *datoInt*, *datoLong*, *datoFloat* y *datoDouble*. Cada uno de estos métodos tenía el siguiente aspecto:

```

public static double datoDouble()
{
    try
    {
        return Double.Parse(Console.ReadLine());
    }
    catch(FormatException)
    {
        return Double.NaN; // No es un Número; valor double.
    }
}

```

Este método devuelve el valor de tipo **double** retornado a su vez por el método **Parse** de la clase **Double**, resultado de convertir la cadena de caracteres devuelta por **ReadLine**. Pero, ¿qué ocurre si la cadena de caracteres devuelta por **ReadLine** no se corresponde con un **double**? Pues que al ejecutarse el método **Parse**, el sistema de ejecución lanza una excepción **FormatException** que es capturada por el manejador, que devuelve el valor **Double.NaN**.

Una alternativa al manejador anterior podría ser otro que ante un dato no válido (por ejemplo: xxx, 3z5, etc.) solicitara teclear un dato correcto. También con-

viene que nuestro método informe de cómo transcurrió su ejecución devolviendo, por ejemplo, un valor **true** (ejecución satisfactoria) o **false**. Esto obliga a pasar al método la variable a leer por referencia. Así mismo, el método devolverá **false** cuando se teclee el carácter fin de fichero (*Ctrl+Z*); en este caso, **ReadLine** devuelve **null**. De esta forma, podremos utilizar *Ctrl+Z* como marca para finalizar una entrada masiva de datos. Según esto, podemos reescribir el método *datoDouble* así:

```
public static bool datoDouble(ref double n)
{
    try
    {
        n = Double.Parse(Console.ReadLine());
        return true;
    }
    catch (ArgumentNullException)
    {
        // Si ReadLine devolvió null, se pulsó Ctrl+z
        return false;
    }
    catch (Exception)
    {
        Console.WriteLine("Dato no válido. Vuelva a introducirlo.");
        return datoDouble(ref n);
    }
}
```

En el código anterior, se puede observar que ante una entrada no válida, el manejador llama recursivamente al método, excepto si se pulsó *Ctrl+z*.

Como ejemplo, el siguiente código, utilizando los métodos de la clase anterior, permite introducir datos de tipo **double** hasta pulsar las teclas *Ctrl+z*:

```
public class LeerDatos
{
    public static void Main(string[] args)
    {
        double dato = 0; int i = 0;
        double[] a = new double[100];

        Console.WriteLine("Introducir datos. Finalizar con Ctrl+z");
        Console.Write("Dato double: ");
        while (i < 100 && Leer.datoDouble(ref dato))
        {
            a[i++] = dato;
            Console.Write("Dato double: ");
        }
    }
}
```

```

        Console.WriteLine("Datos leídos " + i);
        for (int k = 0; k < i; ++k)
            Console.Write(a[k] + " ");
    }
}

```

Siguiendo un desarrollo análogo podemos escribir el resto de los métodos.

3. Escribir una clase genérica *Leer<T>* con un método *Dato* que permita suplantar a los métodos *datoShort*, *datoInt*, *datoLong*, *datoFloat* y *datoDouble* escritos en el ejercicio anterior.

Tener presente que C# no permite asignar a una variable de tipo *T* un valor de un tipo predeterminado, como **int**, ni tampoco admite conversiones explícitas de un tipo predeterminado a *T*.

La solución al problema planteado podría ser así:

```

using System;

namespace MisClases.ES // espacio de nombres
{
    public class Leer<T> // clase genérica Leer
    {
        private static T dato;

        public static bool Dato(ref T n)
        {
            try
            {
                if (n is short)
                    Leer<short>.dato = short.Parse(Console.ReadLine());
                else if (n is int)
                    Leer<int>.dato = int.Parse(Console.ReadLine());
                else if (n is long)
                    Leer<long>.dato = long.Parse(Console.ReadLine());
                else if (n is float)
                    Leer<float>.dato = float.Parse(Console.ReadLine());
                else if (n is double)
                    Leer<double>.dato = double.Parse(Console.ReadLine());
                else
                    return false;

                n = Leer<T>.dato;
                return true;
            }
            catch (ArgumentNullException)
            {
                // Si ReadLine devolvió null, se pulsó Ctrl+z
            }
        }
    }
}

```

```
        return false;
    }
    catch (Exception)
    {
        Console.WriteLine("Dato no válido. Vuelva a introducirlo.");
        return Dato(ref n);
    }
}
}
```

Obsérvese que la clase define un atributo *dato* de tipo genérico, *T*, tipo que se particulariza para realizar la asignación a *dato* de un valor de un tipo predeterminado (por ejemplo **int**) y se vuelve a generalizar para asignar el valor almacenado en *dato* al parámetro *n* de tipo *T* pasado por referencia. Esto es así porque C# no permite asignar a una variable de tipo *T* un valor de un tipo predeterminado.

Como ejemplo, el siguiente código, utilizando el método *Dato* de la clase genérica *Leer<T>*, permite introducir datos de tipo **int** hasta pulsar las teclas *Ctrl+z*:

```
public class LeerDatos
{
    public static void Main(string[] args)
    {
        int dato = 0; int i = 0;
        int[] a = new int[100];

        Console.WriteLine("Introducir datos. Finalizar con Ctrl+z");

        Console.Write("Dato int: ");
        while (i < 100 && Leer<int>.Dato(ref dato))
        {
            a[i++] = dato;
            Console.Write("Dato int: ");
        }

        Console.WriteLine("Datos leídos " + i);
        for (int k = 0; k < i; ++k)
            Console.Write(a[k] + " ");
    }
}
```

También, cuando se define una clase genérica, se pueden aplicar restricciones a las clases de tipos que el código de cliente puede usar para argumentos de tipo cuando utiliza dicha clase. Las restricciones se especifican mediante la palabra clave **where**. Por ejemplo:

```
public class Leer<T> where T : struct
{
    // ...
}
```

A continuación se muestran los tipos de restricciones posibles:

- where T : struct** *T* debe ser un tipo valor. Se puede especificar cualquier tipo de valor excepto **Nullable**.
- where T : class** *T* debe ser un tipo referencia, entre los que se incluye cualquier clase, interfaz, delegado o matriz.
- where T : new()** *T* debe tener un constructor público sin parámetros. Cuando se utiliza junto con otras restricciones, se debe especificar la restricción **new()** en último lugar.
- where T : clase** *T* debe ser la clase base especificada, o bien debe derivarse de la misma.
- where T: interfaz** *T* debe ser o implementar la interfaz especificada. Se pueden especificar varias restricciones de interfaz. La interfaz con restricciones también puede ser genérica.
- where T : U** El argumento de tipo proporcionado para *T* debe ser o derivarse del argumento proporcionado para *U*. Esto se denomina una restricción de tipo *naked*.

Las restricciones se suelen utilizar para dar garantía al compilador, cuando se examina un elemento en una lista genérica para determinar si es válido o compararlo con otro elemento, de que el operador o método que se llama es compatible con cualquier argumento de tipo que el código de cliente pudiera especificar. Una vez que el compilador tiene esta garantía, puede permitir que se llame a los métodos de ese tipo dentro de la clase genérica.

EJERCICIOS PROPUESTOS

1. La clase *CCuenta* que implementamos en el capítulo 12 tiene un método *Reintegro* que muestra un mensaje “Error: no dispone de saldo” cuando se intenta retirar una cantidad y no hay suficiente saldo. Modifique esta clase para que el método *Reintegro* lance una excepción *SaldoInsuficienteException*.

La clase *SaldoInsuficienteException* tendrá dos atributos, uno de la clase *CCuenta* para hacer referencia a la cuenta que causó el problema y otro de tipo **double** para

almacenar la cantidad solicitada. Así mismo, tendrá un constructor sin parámetros, otro con ellos, y sobrescribirá la propiedad **Message**. El constructor con parámetros *SaldoInsuficienteException* tendrá dos que harán referencia a la cuenta causante del problema y a la cantidad solicitada. La propiedad **Message** generará un mensaje de error basado en la información almacenada en los atributos y devolverá un objeto **string** con ese mensaje.

Atrape la excepción *SaldoInsuficienteException* en la parte de la aplicación que considere más adecuada.

Cuando haya finalizado pruebe la jerarquía de la clase *CCuenta* junto con la clase *CBanco* que también implementamos en ese capítulo.

CAPÍTULO 15

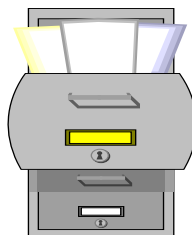
© F.J.Ceballos/RA-MA

FLUJOS

Todos los programas realizados hasta ahora obtenían los datos necesarios para su ejecución de la entrada estándar y visualizaban los resultados en la salida estándar. Por otra parte, una aplicación podrá retener los datos que manipula en su espacio de memoria, sólo mientras esté en ejecución; es decir, cualquier dato introducido se perderá cuando la aplicación finalice.

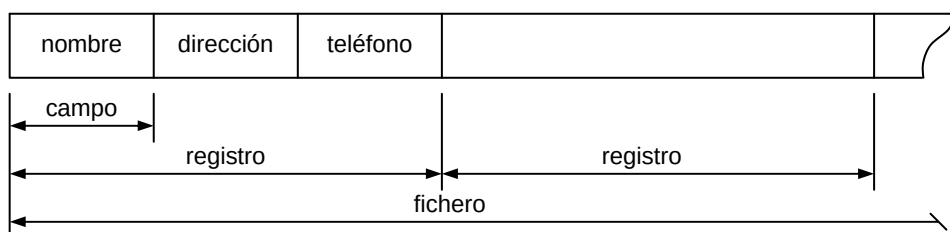
Por ejemplo, si hemos realizado un programa con la intención de construir una agenda, lo ejecutamos y almacenamos los datos *nombre*, *apellidos* y *teléfono* de cada uno de los componentes de la agenda en una matriz, los datos estarán disponibles mientras el programa esté en ejecución. Si finalizamos la ejecución del programa y lo ejecutamos de nuevo, tendremos que volver a introducir otra vez todos los datos.

La solución para hacer que los datos persistan de una ejecución para otra es almacenarlos en un fichero en el disco en vez de en una matriz en memoria. Entonces, cada vez que se ejecute la aplicación que trabaja con esos datos, podrá leer del fichero los que necesite y manipularlos. Nosotros procedemos de forma análoga en muchos aspectos de la vida ordinaria; almacenamos los datos en fichas y guardamos el conjunto de fichas en lo que generalmente denominamos fichero o archivo.



Desde el punto de vista informático, un fichero o archivo es una colección de información que almacenamos en un soporte magnético para poderla manipular en cualquier momento. Esta información se almacena como un conjunto de registros, los cuales contienen, generalmente, los mismos campos. Cada campo almacena un dato de un tipo predefinido o de un tipo definido por el usuario. El registro más simple estaría formado por un carácter.

Por ejemplo, si quisiéramos almacenar en un fichero los datos relativos a la agenda de teléfonos a la que nos hemos referido anteriormente, podríamos diseñar cada registro con los campos *nombre*, *dirección* y *teléfono*. Según esto y desde un punto de vista gráfico, puede imaginarse la estructura del fichero así:



Cada campo almacenará el dato correspondiente. El conjunto de campos descritos forma lo que hemos denominado registro, y el conjunto de todos los registros forman un fichero que almacenaremos, por ejemplo, en el disco bajo un nombre.

Por lo tanto, para manipular un fichero que identificamos por un nombre, son tres las operaciones que tenemos que realizar: abrir el fichero, escribir o leer registros del fichero y cerrar el fichero. En la vida ordinaria hacemos lo mismo, abrimos el cajón que contiene las fichas (fichero), tomamos una ficha (registro) para leer datos o escribir datos y, finalizado el trabajo con la ficha, la dejamos en su sitio y cerramos el cajón de fichas (fichero).

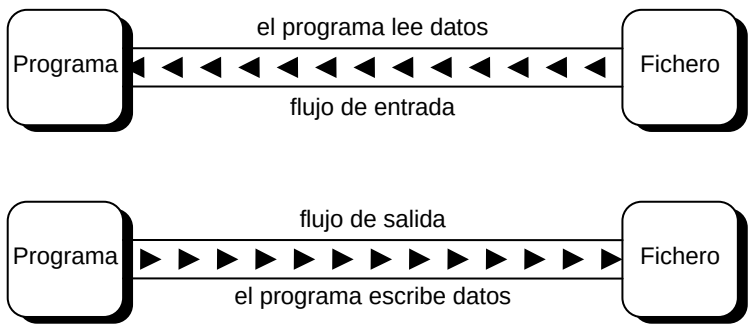
En programación orientada a objetos, hablaremos de objetos más que de registros, y de sus atributos más que de campos.

Podemos agrupar los ficheros en dos tipos: ficheros de la aplicación (son los ficheros .cs, .exe, etc. que forman la aplicación) y ficheros de datos (son los que proveen de datos a la aplicación). A su vez, C# ofrece dos tipos diferentes de acceso a los ficheros de datos: secuencial y aleatorio.

Para dar soporte al trabajo con ficheros, la biblioteca .NET proporciona varias clases de entrada/salida (E/S) que permiten leer y escribir datos a, y desde, ficheros y dispositivos (en el capítulo 6 trabajamos con algunas de ellas).

VISIÓN GENERAL DE LOS FLUJOS DE E/S

Sabemos que la comunicación entre un programa y el origen o el destino de cierta información se realiza mediante un *flujo* de información (en inglés *stream*) que no es más que un objeto que hace de intermediario entre el programa, y el origen o el destino de la información. De esta forma, el programa leerá o escribirá en el *flujo* sin importarle desde dónde viene la información o a dónde va.



Este nivel de abstracción hace que un programa no tenga que saber nada del dispositivo, lo que se traduce en una facilidad más a la hora de escribir programas, ya que los algoritmos para leer y escribir datos serán siempre más o menos los mismos:

Leer	Escribir
<i>Abrir un flujo desde un fichero</i>	<i>Abrir un flujo hacia un fichero</i>
<i>Mientras haya información</i>	<i>Mientras haya información</i>
<i>Leer información</i>	<i>Escribir información</i>
<i>Cerrar el flujo</i>	<i>Cerrar el flujo</i>

El espacio de nombres **System.IO** de la biblioteca .NET contiene una colección de clases que soportan estos algoritmos para leer y escribir. Por ejemplo, la clase **FileStream** subclase de **Stream** permite escribir o leer datos de un fichero byte a byte; análogamente, las clases **StreamWriter** y **StreamReader** subclases de **TextWriter** y **TextReader**, respectivamente, permiten escribir o leer caracteres (**char**); y las clases **BinaryReader** y **BinaryWriter** permiten leer y escribir datos de cualquier tipo primitivo en binario y cadenas de caracteres.

ABRIENDO FICHEROS PARA ACCESO SECUENCIAL

El tipo de acceso más simple a un fichero de datos es el secuencial. Un fichero abierto para *acceso secuencial* es un fichero que puede almacenar registros de

cualquier longitud, incluso de un solo byte. Cuando la información se escribe registro a registro, éstos son colocados uno a continuación de otro, y cuando se lee, se empieza por el primer registro y se continúa al siguiente hasta alcanzar el final.

Este tipo de acceso generalmente se utiliza con ficheros de texto en los que se escribe toda la información desde el principio hasta el final y se lee de la misma forma. En cambio, los ficheros de texto no son los más apropiados para almacenar grandes series de números, porque cada número es almacenado como una secuencia de bytes; esto significa que un número entero de nueve dígitos ocupa nueve bytes en lugar de los cuatro requeridos para un entero. De ahí que a continuación se expongan distintos tipos de flujos: de bytes y de caracteres para el tratamiento de texto, y binarios para el tratamiento de números.

Flujos de bytes

Los datos pueden ser escritos o leídos de un fichero byte a byte utilizando un flujo de la clase **FileStream**.

FileStream

Un flujo de la clase **FileStream** permite leer y escribir bytes en un fichero. Esta clase, además de los métodos que hereda de la clase **Stream**, proporciona los constructores siguientes:

```
FileStream(string nombre, FileMode modo)
FileStream(string nombre, FileMode modo, FileAccess acceso)
```

El primer constructor abre un flujo de entrada y salida (para leer y escribir) vinculado con el fichero especificado por *nombre*, mientras que el segundo hace lo mismo, pero con la posibilidad de especificar el tipo de acceso (leer, escribir, o leer y escribir).

El parámetro *nombre* es una cadena de caracteres que especifica la ruta donde está guardado o se guardará el fichero. Dicha cadena puede escribirse de alguna de las dos formas siguientes (la ventaja de utilizar la combinación del símbolo @ y comillas es que las secuencias de escape no se procesan):

```
"c:\\temp\\doc.txt"
@"c:\temp\doc.txt"
```

El parámetro *modo*, del tipo enumerado **FileMode**, puede tomar uno de los valores siguientes:

CreateNew	Crear un nuevo fichero. Si el fichero existe se lanzará una excepción del tipo IOException .
Truncate	Abrir un fichero existente. Una vez abierto, el fichero será truncado a cero bytes de longitud.
Create	Crear un nuevo fichero. Si el fichero existe será sobrescrito. Es equivalente a utilizar CreateNew si no existe el fichero y a utilizar Truncate en caso contrario.
Open	Abrir un fichero existente. Si el fichero no existe se lanzará una excepción del tipo FileNotFoundException .
OpenOrCreate	Abrir un fichero si existe; si no, se crea un nuevo fichero.
Append	Abrir un fichero para añadir datos al final del mismo si existe, o crear un nuevo fichero si no existe.

El parámetro *acceso*, del tipo enumerado **FileAccess**, puede tomar uno de los valores siguientes:

Read	Permite acceder al fichero para realizar operaciones de lectura.
ReadWrite	Permite acceder al fichero para realizar operaciones de lectura y escritura.
Write	Permite acceder al fichero para realizar operaciones de escritura.

Un ejemplo aclarará los conceptos expuestos. La aplicación C# siguiente, definida por la clase *CEscribirBytes* mostrada a continuación, lee una línea de texto desde el teclado y la guarda en un fichero denominado *texto.txt*. Para ello:

1. Define una matriz *buffer* de 81 bytes.
2. Define un flujo *fs* hacia un fichero denominado *texto.txt*. Tenga presente que si el fichero existe, se borrará en el momento de definir el flujo que, en este caso, permite su acceso sólo para escribir.

```
FileStream fs = new FileStream("texto.txt", FileMode.Create,
                             FileAccess.Write);
```

3. Lee una línea de texto desde el teclado de longitud *nbytes* (*nbytes* < 81) y la almacena en *buffer*.

```
while ((car = Console.Read()) != '\r' && nbytes < buffer.Length)
{
    buffer[nbytes] = (byte)car;
    nbytes++;
}
```

4. Escribe explícitamente la línea de texto en el flujo (implícitamente la escribe en el fichero). Esto se hace cuando el flujo recibe el mensaje **Write**, lo que origina que se ejecute el método **Write**, en este caso con tres parámetros: el primero es una referencia a la matriz que contiene los bytes que deseamos escribir, el segundo es la posición en la matriz del primer byte que se desea escribir y el tercero, el número de bytes a escribir.

```
fs.Write(buffer, 0, nbytes);
```

El programa completo se muestra a continuación:

```
using System;
using System.IO;

public class CEscribirBytes
{
    public static void Main(string[] args)
    {
        FileStream fs = null;
        byte[] buffer = new byte[81];
        int nbytes = 0, car;

        try
        {
            // Crear un flujo hacia el fichero texto.txt
            fs = new FileStream("texto.txt",
                               FileMode.Create, FileAccess.Write);

            Console.WriteLine(
                "Escriba el texto que desea almacenar en el fichero:");
            while ((car = Console.Read()) != '\r' && nbytes < buffer.Length)
            {
                buffer[nbytes] = (byte)car;
                nbytes++;
            }

            // Escribir la línea de texto en el fichero
            fs.Write(buffer, 0, nbytes);
        }
        catch(IOException e)
        {
            Console.WriteLine("Error: " + e.Message);
        }
    }
}
```

Cuando ejecute la aplicación escriba una línea de texto y pulse la tecla *Entrar*. A continuación, en la línea de órdenes del sistema, teclee *type texto.txt* en Windows, o bien *cat texto.txt* en UNIX, para mostrar el texto del fichero y comprobar que todo ha funcionado como esperaba.

Si lo que desea es añadir información al fichero, cree el flujo hacia el mismo como se indica a continuación:

```
FileStream fs = new FileStream("texto.txt", FileMode.Append,
                             FileAccess.Write);
```

En este caso, si el fichero no existe se crea y si existe, los datos que se escriban en él se añadirán al final.

Es una buena costumbre cerrar un flujo cuando ya no se vaya a utilizar más, para lo cual se invocará a su método **Close**. En la biblioteca .NET puede observar en la documentación que el método **Close** de **FileStream** declara que puede lanzar también una excepción de la clase **IOException**.

En los casos en los que sea necesario realizar alguna acción antes de salir de un método (por ejemplo, cerrar un fichero) ponga el código adecuado dentro de un bloque **finally** colocado a continuación de un bloque **try** o de un bloque **catch**. El bloque **finally** deber ser siempre el último.

Aplicando lo expuesto a la aplicación anterior, el código quedaría así:

```
try
{
    // ...
    // Escribir la línea de texto en el fichero
    fs.Write(buffer, 0, nbytes);
}
catch(IOException e)
{
    Console.WriteLine("Error: " + e.Message);
}
finally
{
    if (fs != null) fs.Close();
}
```

La ejecución del bloque **finally** queda garantizada independientemente de que finalice o no la ejecución del bloque **try**. Quiere esto decir que aunque se abandone la ejecución del bloque **try** porque, por ejemplo, se ejecute una sentencia **return**, el bloque **finally** se ejecuta.

Quizás haya pensado invocar al método **Close** después de haber ejecutado el método **Write** dentro del bloque **try**:

```
try
{
```

```
// ...  
// Escribir la línea de texto en el fichero  
fs.Write(buffer, 0, nbytes);  
if (fs != null) fs.Close();  
}  
// ...
```

Aunque esta forma de proceder también es válida no es tan eficiente como la anterior porque, ¿qué sucedería si el método **Write** lanzara una excepción? No se ejecutaría **Close**, aunque finalmente el sistema se encargaría de cerrar el flujo.

El siguiente ejemplo es una aplicación C# que lee el texto guardado en el fichero *texto.txt* creado por la aplicación anterior y lo almacena en una matriz denominada *buffer*.

La aplicación definida por la clase *CLeerBytes* mostrada a continuación realiza lo siguiente:

1. Define las matrices *bBuffer* y *cBuffer* de 81 bytes.

```
char[] cBuffer = new char[81];  
byte[] bBuffer = new byte[81];
```

2. Define un flujo *fe* desde un fichero denominado *texto.txt*. Tenga presente que si el fichero no existe, se lanzará una excepción que lo indicará.

```
FileStream fe = new FileStream("texto.txt", FileMode.Open,  
                             FileAccess.Read);
```

3. Lee el texto desde el flujo y lo almacena en *bBuffer*. Esto se hace cuando el flujo recibe el mensaje **Read**, lo que origina que se ejecute el método **Read**, en este caso con tres parámetros: el primero es una referencia a la matriz que almacenará los bytes leídos, el segundo es la posición en la matriz del primer byte que se desea almacenar y el tercero, el número máximo de bytes que se leerán. El método devuelve el número de bytes leídos o -1 si no hay más datos porque se ha alcanzado el final del fichero.

```
nbytes = fe.Read(bBuffer, 0, 81);
```

4. Crea un objeto **String** con los datos leídos. Como el constructor **String** utilizado requiere como primer argumento una matriz de tipo **char**, previamente se copia la matriz *bBuffer* de tipo **byte** en otra *cBuffer* de tipo **char**.

```
Array.Copy(bBuffer, cBuffer, bBuffer.Length);  
String str = new String(cBuffer, 0, nbytes);
```

El programa completo se muestra a continuación:

```
using System;
```

```

using System.IO;

public class CLeerBytes
{
    public static void Main(string[] args)
    {
        FileStream fe = null;
        char[] cBuffer = new char[81];
        byte[] bBuffer = new byte[81];
        int nbytes;
        try
        {
            // Crear un flujo desde el fichero texto.txt
            fe = new FileStream("texto.txt",
                               FileMode.Open, FileAccess.Read);
            // Leer del fichero una línea de texto
            nbytes = fe.Read(bBuffer, 0, 81);
            // Crear un objeto string con el texto leído
            Array.Copy(bBuffer, cBuffer, bBuffer.Length);
            String str = new String(cBuffer, 0, nbytes);
            // Mostrar el texto leído
            Console.WriteLine(str);
        }
        catch(IOException e)
        {
            Console.WriteLine("Error: " + e.Message);
        }
        finally
        {
            // Cerrar el fichero
            if (fe != null) fe.Close();
        }
    }
}

```

Flujos de caracteres

Los datos pueden ser escritos o leídos de un fichero carácter a carácter en un formato portable (UTF-8: *UCS Transformation Format*, formato de 8 bits en el que cada carácter Unicode emplea uno o más bytes) utilizando flujos de las clases **StreamWriter** y **StreamReader**, subclases de **TextWriter** y **TextReader**, respectivamente.

StreamWriter

Un flujo de la clase **StreamWriter** permite escribir caracteres en un fichero. Esta clase, además de los métodos que hereda de la clase **TextWriter**, proporciona los constructores siguientes:

```
StreamWriter(string nombre)
StreamWriter(Stream flujo)
```

El primer constructor abre un flujo de salida (para escribir) hacia el fichero especificado por *nombre* (ver la clase **FileStream**), mientras que el segundo hace lo mismo, pero desde otro flujo existente de la clase **Stream** o de sus derivadas.

Como ejemplo, la siguiente aplicación definida por la clase *CEscribirCars*, lee líneas de texto desde el teclado y las guarda en un fichero denominado *doc.txt*.

```
using System;
using System.IO;

public class CEscribirCars
{
    public static void Main(string[] args)
    {
        StreamWriter sw = null;
        String str;

        try
        {
            // Crear un flujo hacia el fichero doc.txt
            sw = new StreamWriter("doc.txt");

            Console.WriteLine(
                "Escriba las líneas de texto a almacenar en el fichero.\n" +
                "Finalice cada línea pulsando la tecla <Entrar>.\n" +
                "Para finalizar pulse sólo la tecla <Entrar>.\n");
            // Leer una línea de la entrada estándar
            str = Console.ReadLine();
            while (str.Length != 0)
            {
                // Escribir la línea leída en el fichero
                sw.WriteLine(str);
                // Leer la línea siguiente
                str = Console.ReadLine();
            }
        }
        catch(IOException e)
        {
            Console.WriteLine("Error: " + e.Message);
        }
        finally
        {
            if (sw != null) sw.Close();
        }
    }
}
```


El programa anterior crea un fichero *doc.txt* que almacena las líneas de texto introducidas a través del teclado. Si el fichero existe, se trunca a longitud 0. Si no queremos que esto suceda, podemos crear un flujo utilizando un constructor según el segundo formato expuesto; por ejemplo:

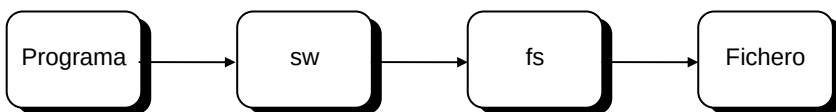
```
public class CEscribirCars
{
    public static void Main(string[] args)
    {
        FileStream fs = null;
        StreamWriter sw = null;
        String str;

        try
        {
            // Crear un flujo hacia el fichero doc.txt
            fs = new FileStream("doc.txt", FileMode.Append,
                               FileAccess.Write);
            sw = new StreamWriter(fs);

            // ...
        }
    }
}
```

Observe que un flujo de la clase **StreamWriter** actúa como un filtro; esto es, los datos enviados al destino son transformados mediante alguna operación; en este caso, sufren una conversión a un formato portable (UTF-8) cuando son almacenados y viceversa cuando son recuperados a través de un flujo de la clase **StreamReader**.

Según lo expuesto, una aplicación basada en el código anterior escribirá en el filtro *sw* los datos a almacenar en el fichero *doc.txt*, flujo que a su vez está conectado al flujo *fs* abierto hacia ese fichero. La figura siguiente muestra de forma gráfica lo expuesto:



El procedimiento genérico para utilizar un filtro, como al que nos estamos refiriendo, es básicamente así:

- Se crea un flujo asociado con un origen o destino de los datos.
- Se asocia un filtro con el flujo anterior.
- Finalmente, el programa leerá o escribirá datos a través de ese filtro.

La funcionalidad de la clase **StreamWriter** está soportada por varios métodos entre los que cabe destacar **Write**, **WriteLine**, **Flush** y la propiedad **BaseStream**.

Write permite escribir cualquier dato de un tipo primitivo pero como una cadena caracteres (esto ocupa bastante espacio de disco, por eso la biblioteca .NET nos ofrece como alternativa las clases **BinaryWriter** y **BinaryReader** que veremos un poco más adelante), matrices de caracteres, objetos **String** y, en general, cualquier objeto de la clase **Object** o de sus subclases. **WriteLine** hace lo mismo que **Write** y además añade los caracteres ‘\r\n’. **Flush** limpia el *buffer* de salida volcando la información en el fichero vinculado con el flujo. La propiedad **BaseStream** permite acceder al flujo subyacente (en la figura anterior a *fs*) a través del cual podremos invocar a sus métodos particulares. Para más información recurra a la ayuda proporcionada con el SDK.

StreamReader

Un flujo de la clase **StreamReader** permite leer caracteres desde un fichero. Esta clase, además de los métodos que hereda de la clase **TextReader**, proporciona los constructores siguientes:

```
StreamReader(string nombre)
StreamReader(Stream flujo)
```

El primer constructor abre un flujo de entrada (para leer) desde el fichero especificado por *nombre* (ver la clase **FileStream**), mientras que el segundo hace lo mismo, pero desde otro flujo existente de la clase **Stream** o de sus derivadas.

Como ejemplo, la siguiente aplicación definida por la clase **CLeerCars**, lee líneas de texto desde el fichero denominado *doc.txt* y las muestra en la salida estándar.

```
using System;
using System.IO;

public class CLeerCars
{
    public static void Main(string[] args)
    {
        StreamReader sr = null;
        String str;

        try
        {
            // Crear un flujo desde el fichero doc.txt
            sr = new StreamReader("doc.txt");
            // Leer del fichero una línea de texto
            str = sr.ReadLine();
            while (str != null)
            {
```

```

        // Mostrar la línea leída
        Console.WriteLine(str);
        // Leer la línea siguiente
        str = sr.ReadLine();
    }
}
catch(IOException e)
{
    Console.WriteLine("Error: " + e.Message);
}
finally
{
    // Cerrar el fichero
    if (sr != null) sr.Close();
}
}
}

```

La funcionalidad de esta clase está soportada por varios métodos entre los que cabe destacar **Read**, **ReadLine**, **Peek** y **DiscardBufferedData**. **Read** permite leer un carácter o un conjunto caracteres. **ReadLine** permite leer una línea. Una línea está definida como una secuencia de caracteres seguidos por '\r', '\n', o bien por '\r\n'. La cadena de caracteres devuelta no contiene el carácter o caracteres de terminación. **Peek** permite obtener el siguiente carácter a leer pero sin extraerlo del *buffer* asociado con el flujo. **DiscardBufferedData** permite eliminar los caracteres que actualmente hay en el *buffer* asociado con el flujo. Para más información recurrir a la ayuda proporcionada con el SDK.

Clases File, Directory y Path

Estas clases soportan la manipulación del nombre de un fichero o de un directorio que puede existir en el sistema de ficheros de la máquina; por lo tanto, sus métodos permitirán interrogar al sistema sobre todas las características de ese fichero o directorio. Todos los métodos de estas clases son **static** para que puedan ser invocados sin necesidad de que exista un objeto de ellas.

Para referirnos a un fichero o a un directorio, lo más sencillo es formar un **string** a partir de su nombre al que podremos añadir opcionalmente su ruta de acceso (relativa o absoluta). Por ejemplo, las siguientes rutas son válidas:

```

"c:\MiDir\MiFichero.txt"
@"c:\MiDir\MiFichero.txt"
@"c:\MiDir"
"MiDir\MiSubDir\MiFichero.txt"
@"MiDir\MiSubDir\MiFichero.txt"
@"MiDir\MiSubDir"

```

```
"MiFichero.txt"  
"\\\\\\MiServidor\\MiDirCompartido\\MiFichero.txt"
```

La tabla siguiente resume algunos de los métodos de la clase **File**:

Método	Significado
Copy	Copia un fichero existente en un nuevo fichero.
Delete	Borra un fichero.
Exists	Determina si un fichero existe.
GetAttributes	Obtiene los atributos de un fichero.
Move	Mueve un fichero a una nueva localización.
SetAttributes	Establece los atributos de un fichero.

Esta otra tabla resume algunos de los métodos de la clase **Directory**:

Método	Significado
CreateDirectory	Crea todos los directorios y subdirectorios especificados por la ruta pasada como argumento.
Delete	Borra un directorio y su contenido.
Exists	Determina si un directorio existe.
GetFiles	Obtiene los ficheros de un directorio.
GetParent	Conduce al directorio padre.
Move	Mueve un directorio y su contenido a una nueva ruta.

La tabla siguiente resume algunos de los métodos de la clase **Path**:

Método	Significado
ChangeExtension	Cambia la extensión de un fichero.
Combine	Combina dos rutas.
GetDirectoryName	Retorna la ruta de un fichero.
GetExtension	Retorna la extensión de la ruta especificada.
GetFileName	Retorna el nombre y la extensión de la ruta especificada.
GetFileNameWithoutExtension	Retorna el nombre sin la extensión de la ruta especificada.
GetPathRoot	Retorna la raíz de la ruta especificada.
HasExtension	Determina si una ruta tiene extensión.
IsPathRoot	Determina si la ruta incluye la raíz.

Para más detalles sobre los métodos anteriores recurra a la ayuda proporcionada con el SDK.

Como ejemplo, utilizando las capacidades de la clase **File** podemos modificar la aplicación *CLeerCars* para que solicite un nombre de un fichero existente:

```
// ...
StreamReader sr = null;
String str;

try
{
    // Obtener el nombre del fichero de la entrada estándar
    do
    {
        Console.Write("Nombre del fichero: ");
        str = Console.ReadLine();
    }
    while (!File.Exists(str));

    // Crear un flujo desde el fichero str
    sr = new StreamReader(str);

    // Leer del fichero una línea de texto
    str = sr.ReadLine();
    // ...
}
// ...
```

Análogamente, utilizando las capacidades de la clase **File** podemos modificar la aplicación *CEscribirCars* para que verifique si existe el fichero en el que se van a escribir los datos leídos desde el teclado:

```
// ...
StreamWriter sw = null;
String str;

try
{
    // Obtener el nombre del fichero de la entrada estándar
    Console.Write("Nombre del fichero: ");
    str = Console.ReadLine();

    char resp = 's';
    if (File.Exists(str))
    {
        Console.Write("El fichero existe ¿desea sobrescribirlo? (s/n) ");
        resp = (char)Console.Read();
        // Saltar los bytes no leídos del flujo de entrada estándar
        Console.ReadLine();
    }
    if (resp != 's') return;
```

```
// Crear un flujo hacia el fichero doc.txt
sw = new StreamWriter(str);
// ...
}
// ...
```

Flujos de datos de tipos primitivos

Muchas aplicaciones requieren escribir en un fichero datos de tipos primitivos (**bool**, **byte**, **double**, **float**, **long**, **int**, **short**, etc.) no como cadenas de caracteres, sino en formato binario para posteriormente recuperarlos como tal. Para estos casos, el espacio de nombres **System.IO** proporciona las clases **BinaryReader** y **BinaryWriter**, las cuales permiten leer y escribir, respectivamente, datos de cualquier tipo primitivo en formato binario y cadenas de caracteres en formato UTF-8. Entonces, ¿por qué no se han analizado previamente? Pues, simplemente porque no pueden utilizarse con los dispositivos ASCII de E/S estándar. Un flujo **BinaryReader** sólo puede leer datos almacenados en un fichero a través de un flujo **BinaryWriter**.

A continuación podrá observar que los flujos de estas clases, análogamente a los flujos de caracteres vistos anteriormente, actúan como filtros conectados a un flujo de la clase **Stream**, otra razón por la que **BinaryReader** y **BinaryWriter** no pueden utilizarse con los dispositivos de E/S estándar, ya que los flujos **In** y **Out** no son de esta clase. En este caso, los datos de tipos primitivos son tratados en binario y las cadenas de caracteres en formato UTF-8. El procedimiento para utilizar un filtro ya fue descrito anteriormente.

BinaryWriter

Un flujo de la clase **BinaryWriter** permite a una aplicación escribir datos de cualquier tipo primitivo. Se puede crear un flujo de esta clase utilizando el siguiente constructor:

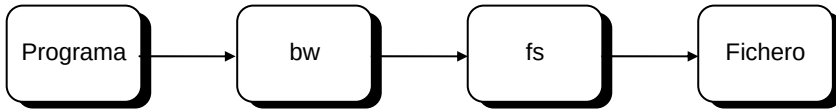
```
BinaryWriter(Stream flujo)
```

Este constructor abre un flujo de salida (para escribir) hacia el fichero vinculado con otro flujo existente de la clase **Stream** o de sus derivadas.

Veamos un ejemplo. Las siguientes líneas de código definen un flujo *bw* que permitirá escribir datos de tipos primitivos en un fichero *datos.dat*:

```
FileStream fs = new FileStream("datos.dat", FileMode.Create,
                             FileAccess.Write);
BinaryWriter bw = new BinaryWriter(fs);
```

Un programa que quiera almacenar datos en el fichero *datos.dat* escribirá tales datos en el filtro *bw*, que a su vez está conectado al flujo *fs* abierto hacia ese fichero. La figura siguiente muestra de forma gráfica lo expuesto:



El siguiente fragmento de código muestra cómo utilizar el flujo anterior para almacenar los datos *nombre*, *dirección* y *teléfono* en un fichero especificado por *nombreFichero*:

```

FileStream fs = new FileStream(nombreFichero, FileMode.Create,
                             FileAccess.Write);
BinaryWriter bw = new BinaryWriter(fs);

// Almacenar el nombre, la dirección y el teléfono en el fichero
bw.Write("un nombre");
bw.Write("una dirección");
bw.Write(942334455L);

bw.Close(); fs.Close();
  
```

Los métodos más utilizados de esta clase se resumen en la tabla siguiente:

Método/propiedad	Descripción
Write(byte)	Escribe un valor de tipo byte .
Write(bytes[])	Escribe una cadena como una secuencia de bytes.
Write(char)	Escribe un valor de tipo char .
Write(char[])	Escribe una cadena como una secuencia de caracteres.
Write(short)	Escribe un valor de tipo short .
Write(int)	Escribe un valor de tipo int .
Write(long)	Escribe un valor de tipo long .
Write(Decimal)	Escribe un valor de tipo Decimal .
Write(float)	Escribe un valor de tipo float .
Write(double)	Escribe un valor de tipo double .
Write(string)	Escribe una cadena de caracteres en formato UTF-8; el primero o los dos primeros bytes especifican el número de bytes de datos escritos a continuación.
BaseStream	Obtiene el flujo subyacente (<i>fs</i> en la figura anterior).
Close	Cierra el flujo y libera los recursos adquiridos.
Flush	Limpia el <i>buffer</i> asociado con el flujo enviando su contenido al fichero subyacente.
Seek	Establece el puntero de L/E en el flujo.

BinaryReader

Un flujo de la clase **BinaryReader** permite a una aplicación leer datos de cualquier tipo primitivo escritos por un flujo de la clase **BinaryWriter**. Se puede crear un flujo de esta clase utilizando el siguiente constructor:

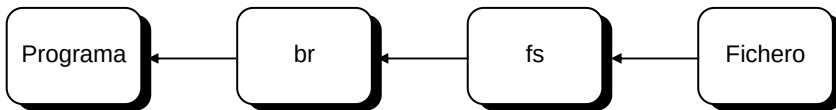
```
BinaryReader(Stream flujo)
```

Este constructor abre un flujo de entrada (para leer) desde el fichero vinculado con otro flujo existente de la clase **Stream** o de sus derivadas.

Veamos un ejemplo. Las siguientes líneas de código definen un flujo *br* que permitirá leer datos de tipos primitivos desde un fichero *datos.dat*:

```
FileStream fs = new FileStream("datos.dat", FileMode.Open,  
                             FileAccess.Read);  
BinaryReader br = new BinaryReader(fs);
```

Un programa que quiera leer datos del fichero *datos.dat* leerá tales datos del filtro *br*, que a su vez está conectado al flujo *fs* abierto desde ese fichero. La figura siguiente muestra de forma gráfica lo expuesto:



El siguiente fragmento de código muestra cómo utilizar el flujo anterior para leer los datos *nombre*, *dirección* y *teléfono* desde un fichero especificado por *nombreFichero*:

```
FileStream fs = new FileStream(nombreFichero, FileMode.Open,  
                             FileAccess.Read);  
BinaryReader br = new BinaryReader(fs);  
// Leer el nombre, la dirección y el teléfono desde el fichero  
nombre = br.ReadString();  
dirección = br.ReadString();  
teléfono = br.ReadInt64();  
  
br.Close(); fs.Close();
```

Los métodos más utilizados de esta clase se resumen en la tabla siguiente:

Método/propiedad	Descripción
ReadByte	Devuelve un valor de tipo byte .
ReadBytes	Devuelve un valor de tipo byte[] (matriz de bytes).

ReadChar	Devuelve un valor de tipo char .
ReadChars	Devuelve un valor de tipo char[] (matriz de caracteres).
ReadInt16	Devuelve un valor de tipo short .
ReadInt32	Devuelve un valor de tipo int .
ReadInt64	Devuelve un valor de tipo long .
ReadDecimal	Devuelve un valor de tipo Decimal .
ReadSingle	Devuelve un valor de tipo float .
ReadDouble	Devuelve un valor de tipo double .
ReadString	Devuelve una cadena de caracteres en formato UTF-8; el primero o los dos primeros bytes especifican el número de bytes de datos que serán leídos a continuación.
BaseStream	Obtiene el flujo subyacente (<i>fs</i> en la figura anterior).
Close	Cierra el flujo y libera los recursos adquiridos.
PeekChar	Obtiene el siguiente carácter sin extraerlo.

Un ejemplo de acceso secuencial

Después de la teoría expuesta hasta ahora acerca del trabajo con ficheros, habrá observado que la metodología de trabajo se repite. Es decir, para escribir datos en un fichero:

- Definimos un flujo hacia el fichero en el que deseamos escribir datos.
- Leemos los datos del dispositivo de entrada o de otro fichero y los escribimos en nuestro fichero. Este proceso se hace normalmente registro a registro. Para ello, utilizaremos los métodos proporcionados por la interfaz del flujo.
- Cerramos el flujo.

Para leer datos de un fichero existente:

- Abrimos un flujo desde el fichero del cual queremos leer los datos.
- Leemos los datos del fichero y los almacenamos en variables de nuestro programa con el fin de trabajar con ellos. Este proceso se hace normalmente registro a registro. Para ello, utilizaremos los métodos proporcionados por la interfaz del flujo.
- Cerramos el flujo.

Esto pone de manifiesto que un fichero no es más que un medio permanente de almacenamiento de datos, dejando esos datos disponibles para cualquier programa que necesite manipularlos. Lógicamente, los datos serán recuperados del fichero en el mismo orden y con el mismo formato con el que fueron escritos, de lo contrario los resultados serán inesperados. Es decir, si en el ejercicio siguiente los datos son guardados en el orden: una cadena, otra cadena y un **long**, tendrán

que ser recuperados en este orden y con este mismo formato. Sería un error recuperar primero un **long**, después una cadena y finalmente la otra cadena, o recuperar primero una cadena, después un **float** y finalmente la otra cadena; etc.

Escribir en un fichero

Como ejemplo, vamos a realizar una aplicación que lea de la entrada estándar grupos de datos (registros), definidos de la forma que se indica a continuación, y los almacene en un fichero.

```
String nombre, dirección;  
long teléfono;
```

Para ello, escribiremos una clase aplicación *CrearListaTfnos* con dos métodos: *crearFichero* y **Main**:

```
using System;  
using System.IO;  
using MisClases.ES; // espacio de nombres de la clase Leer<T>  
  
public class CrearListaTfnos  
{  
    public static void CrearFichero(String fichero)  
    {  
        // Cuerpo del método  
    }  
  
    public static void Main(string[] args)  
    {  
        // Cuerpo del método  
    }  
}
```

El método *CrearFichero* recibe como parámetro un objeto **String** que almacena el nombre del fichero que se desea crear, y realiza las tareas siguientes:

- Crea un flujo hacia el fichero especificado por el objeto **String**, que permite escribir datos de tipos primitivos y cadenas de caracteres.
- Lee grupos de datos *nombre*, *dirección* y *teléfono* de la entrada estándar y los escribe en el fichero.
- Si durante su ejecución, él o alguno de los métodos invocados por él lanza una excepción del tipo **IOException** no atrapada por ellos, será atrapada por el método que le invoque, en nuestro caso por **Main** (véase el capítulo *Excepciones*).

Según lo expuesto, el método *CrearFichero* puede escribirse así:

```
public static void CrearFichero(String fichero)
{
    BinaryWriter bw = null; // salida de datos hacia el fichero
    char resp;
    try
    {
        // Crear un flujo hacia el fichero que permita escribir
        // datos de tipos primitivos y cadenas de caracteres.
        bw = new BinaryWriter(new FileStream(
            fichero, FileMode.Create, FileAccess.Write));

        // Declarar los datos a escribir en el fichero
        String nombre, dirección;
        long teléfono = 0;

        // Leer datos de la entrada estándar y escribirlos
        // en el fichero
        do
        {
            Console.Write("nombre:   "); nombre = Console.ReadLine();
            Console.Write("dirección: "); dirección = Console.ReadLine();
            Console.Write("teléfono: "); Leer<long>.Dato(ref teléfono);

            // Almacenar un nombre, una dirección y un teléfono en
            // el fichero
            bw.Write(nombre);
            bw.Write(dirección);
            bw.Write(teléfono);

            Console.Write("¿desea escribir otro registro? (s/n) ");
            resp = (char)Console.Read();
            // Eliminar los caracteres sobrantes en el flujo de entrada
            Console.ReadLine();
        }
        while (resp == 's');
    }
    finally
    {
        // Cerrar el flujo
        if (bw != null) bw.Close();
    }
}
```

El método **Main** realiza las tareas siguientes:

- Crea un objeto **String** a partir del nombre del fichero leído desde la entrada estándar.

- Verifica si el fichero existe.
- Si no existe, o bien si existe y se desea sobrescribir, invoca al método *crearFichero* pasando como argumento el objeto **String** que almacena el nombre del fichero.

Según lo expuesto, el método **Main** puede escribirse así:

```
public static void Main(string[] args)
{
    String nombreFichero = null;    // nombre del fichero

    try
    {
        // Obtener el nombre del fichero
        Console.Write("Nombre del fichero: ");
        nombreFichero = Console.ReadLine();

        // Verificar si el fichero existe
        char resp = 's';
        if (File.Exists(nombreFichero))
        {
            Console.Write("El fichero existe. ¿Desea sobrescribirlo? (s/n) ");
            resp = (char)Console.Read();
            // Eliminar los caracteres sobrantes en el flujo de entrada
            Console.ReadLine();
        }
        if (resp == 's')
        {
            CrearFichero(nombreFichero);
        }
    }
    catch (IOException e)
    {
        Console.WriteLine("Error: " + e.Message);
    }
}
```

Leer de un fichero

Para leer el fichero creado por la aplicación anterior, vamos a escribir otra aplicación basada en una clase *MostrarListaTfnos*. Esta clase definirá dos métodos: *MostrarFichero* y **Main**.

El método *MostrarFichero* recibe como parámetro un objeto **string** que almacena el nombre del fichero que se desea leer y realiza las tareas siguientes:

- Si el fichero especificado existe, crea un flujo desde el mismo para leer datos de tipos primitivos y cadenas de caracteres.
- Lee un grupo de datos *nombre*, *dirección* y *teléfono* desde el fichero y los muestra. Cuando se alcance el final del fichero, el método utilizado para leer lanzará una excepción del tipo **EndOfStreamException**, instante en el que finalizará la ejecución de este método.
- Si durante su ejecución alguno de los métodos invocados lanza una excepción **IOException**, este método no la atrapará, dejando esta labor a algún método en la pila de llamadas (en nuestro caso, al método **Main**).

El método **Main** recibe como parámetro el nombre del fichero que se desea leer y realiza las tareas siguientes:

- Verifica si al ejecutar la aplicación se pasó un argumento con el nombre del fichero cuyo contenido se desea visualizar.
- Si no se pasó un argumento, la aplicación solicitará el nombre del fichero.
- Invoca al método *MostrarFichero* pasando como argumento el nombre del fichero cuyo contenido se desea visualizar.

Según lo expuesto, la clase *MostrarListaTfnos* puede escribirse así:

```
using System;
using System.IO;

public class MostrarListaTfnos
{
    public static void MostrarFichero(String fichero)
    {
        BinaryReader br = null; // flujo entrada de datos
                                // desde el fichero

        try
        {
            // Verificar si el fichero existe
            if (File.Exists(fichero))
            {
                // Si existe, abrir un flujo desde el mismo para leer
                br = new BinaryReader(new FileStream(
                    fichero, FileMode.Open, FileAccess.Read));

                // Declarar los datos a leer desde el fichero
                String nombre, dirección;
                long teléfono;
```

```
do
{
    // Leer un nombre, una dirección y un teléfono desde el
    // fichero. Cuando se alcance el final del fichero el
    // método utilizado para leer lanzará una excepción del
    // tipo EndOfStreamException.
    nombre = br.ReadString();
    dirección = br.ReadString();
    teléfono = br.ReadInt64();

    // Mostrar los datos nombre, dirección y teléfono
    Console.WriteLine(nombre);
    Console.WriteLine(dirección);
    Console.WriteLine(teléfono);
    Console.WriteLine();
}
while (true);
}
else
    Console.WriteLine("El fichero no existe");
}
catch (EndOfStreamException)
{
    Console.WriteLine("Fin del listado");
}
finally
{
    // Cerrar el flujo
    if (br != null) br.Close();
}
}

public static void Main(string[] args)
{
    try
    {
        if (args.Length == 0)
        {
            // Obtener el nombre del fichero
            Console.Write("Nombre del fichero: ");
            string nombreFichero = Console.ReadLine();
            MostrarFichero(nombreFichero);
        }
        else
        {
            MostrarFichero(args[0]);
        }
    }
    catch (IOException e)
    {
    }
```

```
        Console.WriteLine("Error: " + e.Message);  
    }  
}  
}
```

ABRIENDO FICHEROS PARA ACCESO ALEATORIO

Hasta este punto, hemos trabajado con ficheros de acuerdo con el siguiente esquema: abrir el fichero, leer o escribir hasta el final del mismo, y cerrar el fichero. Pero no hemos leído o escrito a partir de una determinada posición dentro del fichero. Esto es particularmente importante cuando necesitamos modificar algunos de los valores contenidos en el fichero o cuando necesitamos extraer una parte concreta dentro del fichero.

La clase **Stream** del espacio de nombres **System.IO**, así como su derivada **FileStream**, proporcionan métodos que permiten este tipo de acceso directo. Además, un flujo de esta clase permite realizar tanto operaciones de lectura como de escritura sobre el fichero vinculado con el mismo.

Un fichero accedido aleatoriamente es comparable a una matriz. En una matriz para acceder a uno de sus elementos utilizamos un índice. En un fichero accedido aleatoriamente el índice es sustituido por un puntero de lectura o escritura (L/E). Dicho puntero es situado automáticamente al principio del fichero cuando éste se abre para leer y/o escribir, excepto cuando se abre para añadir que se coloca al final. Por lo tanto, una operación de lectura o de escritura comienza en la posición donde esté el puntero dentro del fichero; finalmente, su posición coincidirá justo a continuación del último byte leído o escrito.

Propiedades y métodos para acceso aleatorio

La clase **FileStream**, que se deriva directamente de **Stream**, implementa las propiedades **Position** y **Length** y el método **Seek** que combinados entre sí facilitan el acceso aleatorio a un fichero.

Position

Esta propiedad devuelve la posición actual en bytes del puntero de L/E en el fichero. Piense en el puntero de L/E análogamente a como lo hace cuando piensa en el índice de una matriz. Este puntero marca siempre la posición donde se iniciará la siguiente operación de lectura o de escritura en el fichero. También permite establecer dicha posición, incluso más allá del final del fichero; esta acción no cambia la longitud del fichero; la longitud del fichero sólo cambiará si, a continuación, realizamos una operación de escritura. Su sintaxis es:

```
public long Position
```

Length

Esta otra propiedad devuelve la longitud del fichero en bytes. Su sintaxis es:

```
public long Length
```

El siguiente ejemplo utiliza las propiedades **Position** y **Length** para verificar si se ha llegado al final del fichero vinculado con el flujo *fs*:

```
if( fs.Length == fs.Position )
{
    Console.WriteLine("Fin del fichero.");
}
```

Seek

El método **Seek** mueve el puntero de L/E a una nueva localización desplazada *desp* bytes de la posición *pos* del fichero. Los desplazamientos pueden ser positivos o negativos. El desplazamiento requerido puede ir más allá del final del fichero; esta acción no cambia la longitud del fichero; la longitud del fichero sólo cambiará si, a continuación, realizamos una operación de escritura. Su sintaxis es:

```
public long seek(long desp, SeekOrigin pos)
```

La posición *pos* es un valor del tipo enumerado **SeekOrigin** que define las siguientes constantes:

pos	Descripción
Begin	Hace referencia a la primera posición en el fichero.
Current	Hace referencia a la posición actual del puntero de L/E.
End	Hace referencia a la última posición en el fichero.

Según lo expuesto y suponiendo un flujo, por ejemplo *fs*, definido así:

```
FileStream fs = new FileStream("datos", FileMode.Open,
                               FileAccess.Read);
```

las dos líneas de código siguientes sitúan el puntero de L/E, la primera *desp* bytes antes del final del fichero y la segunda *desp* bytes después de la posición actual.

```
fs.Seek(-desp, SeekOrigin.End);
fs.Seek(desp, SeekOrigin.Current);
```


En cambio, si el flujo (por ejemplo *br*) está definido así:

```
FileStream fs = new FileStream("datos", FileMode.Open,
                               FileAccess.Read);
BinaryReader br = new BinaryReader(fs);
```

como el flujo *br* no soporta el método **Seek** tendremos que recurrir a través de su propiedad **BaseStream** al método **Seek** del flujo *fs* que subyace bajo *br*. Por ejemplo, la línea de código siguiente sitúa el puntero de L/E *desp* bytes después de la posición actual en el fichero vinculado con *br*:

```
br.BaseStream.Seek(desp, SeekOrigin.Current);
```

Igual que ocurría al trabajar con ficheros accedidos secuencialmente, los datos que deseemos escribir en un fichero accedido aleatoriamente, deben guardarse uno a uno utilizando el método adecuado de la clase del flujo vinculado. Por ejemplo, las siguientes líneas de código escriben en el fichero “datos” a partir de la posición *desp* del mismo, los atributos *nombre*, *dirección* y *teléfono* relativos a un objeto *CPersona*:

```
CPersona objeto;
// ...
FileStream fs = new FileStream("datos", FileMode.OpenOrCreate,
                               FileAccess.ReadWrite);
bw = new BinaryWriter(fs);
bw.BaseStream.Seek(desp, SeekOrigin.Begin);
bw.Write(objeto.Nombre);
bw.Write(objeto.Dirección);
bw.Write(objeto.Teléfono);
```

Si para nuestros propósitos pensamos en los atributos *nombre*, *dirección* y *teléfono* como si de un registro se tratara, ¿cuál es el tamaño en bytes de ese registro? Si escribimos más registros, ¿todos tienen el mismo tamaño? Evidentemente no; el tamaño de cada registro dependerá del número de caracteres almacenados en los **string** *nombre* y *dirección* (*teléfono* es un dato de tamaño fijo, 8 bytes, puesto que se trata de un **long**). ¿A cuento de qué viene esta exposición?

Al principio de este apartado dijimos que el acceso aleatorio a ficheros es particularmente importante cuando necesitamos modificar algunos de los valores contenidos en el fichero, o bien cuando necesitamos extraer una parte concreta dentro del fichero. Esto puede resultar bastante complicado si las unidades de grabación que hemos denominado registros no son todas iguales, ya que intervienen los factores de: posición donde comienza un registro y longitud del registro. Tenga presente que cuando necesite reemplazar el registro *n* de un fichero por otro, no debe sobrepasarse el número de bytes que actualmente tiene. Todo esto es viable llevando la cuenta en una matriz de la posición de inicio de cada uno de los regis-

tros y de cada uno de los campos si fuera preciso (esta información se almacenaría en un fichero índice para su utilización posterior), pero resulta mucho más fácil si todos los registros tienen la misma longitud.

Como ejemplo, supongamos que deseamos mantener una lista de teléfonos. Cada elemento de la lista será un objeto persona con los atributos *nombre*, *dirección* y *teléfono*. A su vez, la lista de teléfonos será otro objeto que encapsule un flujo vinculado con un fichero y muestre una interfaz que permita añadir, eliminar y buscar una persona en la lista. Cada registro del fichero contendrá los atributos *nombre*, *dirección* y *teléfono* de cada uno de los objetos persona que forman la lista.

La clase CPersona

La clase de objetos persona (que denominaremos *CPersona*) encapsulará el nombre, la dirección y el teléfono de cada una de las personas de la lista; así mismo, proporcionará la funcionalidad necesaria para establecer u obtener los datos de cada persona individual.

El listado siguiente muestra un ejemplo de una clase *CPersona* que define los atributos privados *nombre*, *dirección* y *teléfono* relativos a una persona, y las propiedades públicas que forman la interfaz de esta clase de objetos. Esto es:

- Constructores, con y sin argumentos, para iniciar un objeto persona.
- Propiedades *Nombre*, *Dirección* y *Teléfono* para cada uno de los atributos.
- Una propiedad denominada *Tamaño* que devuelve la longitud en bytes correspondiente a los atributos de un objeto *CPersona*.

```
////////////////////////////////////  
// Definición de la clase CPersona  
//  
public class CPersona  
{  
    // Atributos  
    private string nombre;  
    private string dirección;  
    private long teléfono;  
  
    // Métodos  
    public CPersona() {}  
  
    public CPersona(string nom, string dir, long tel)  
    {  
        nombre = nom;  
        dirección = dir;
```

```

        teléfono = tel;
    }

    public string Nombre
    {
        get { return nombre; }
        set { nombre = value; }
    }

    public string Dirección
    {
        get { return dirección; }
        set { dirección = value; }
    }

    public long Teléfono
    {
        get { return teléfono; }
        set { teléfono = value; }
    }

    public int Tamaño
    {
        // Longitud en bytes de los atributos (un long = 8 bytes)
        get { return nombre.Length*2 + dirección.Length*2 + 8; }
    }
}

```

Una propiedad como *Nombre* simplemente asigna el nombre pasado como argumento al atributo *nombre* del objeto que recibe el mensaje, o bien devuelve el atributo *nombre* del objeto que recibe el mensaje. La explicación para las otras propiedades es análoga. Por ejemplo:

```

CPersona obj = new CPersona();
obj.Nombre = "Javier";
System.Console.WriteLine(obj.Nombre); // escribe: Javier

```

La clase CListaTfnos

El listado siguiente muestra un ejemplo de lo que puede ser la clase lista de teléfonos, que denominaremos *CListaTfnos*. Define los atributos privados *fs*, flujo de entrada y salida vinculado con el fichero que almacenará la lista de teléfonos, *bw*, flujo hacia *fs*, *br*, flujo desde *fs*, *nregs*, número de registros, *tamañoReg*, tamaño del registro en bytes (hemos supuesto que la suma de las longitudes de *nombre*, *dirección* y *teléfono* no superarán los 140 bytes), y *regsEliminados*, indica si se eliminó algún registro (**true** si se eliminó algún registro y **false** en caso contrario), así como los métodos que se describen a continuación.

Constructor *CListaTfnos*

Cuando desde algún método se crea un objeto *CListaTfnos*, ¿qué esperamos que ocurra? Lógicamente que se cargue la lista de teléfonos especificada, o bien que se cree una nueva cuando el fichero especificado no exista. Por ejemplo:

```
string fichero = "listatfnos.dat";
CListaTfnos listatfnos = new CListaTfnos(fichero);
```

Según lo expuesto, la lista de teléfonos especificada se cargará desde un fichero almacenado en el disco denominado *listatfnos.dat* y si este fichero no existe, se creará uno nuevo con ese nombre. Para ello, el constructor de la clase *CListaTfnos* abrirá un flujo para acceso aleatorio tanto para escribir como para leer del fichero especificado, almacenará una referencia al mismo en el atributo *fs* de la clase, abrirá un flujo *bw* hacia *fs* que permita escribir datos de cualquier tipo primitivo y cadenas de caracteres, abrirá un flujo *br* desde *fs* que permita leer datos de cualquier tipo primitivo y cadenas de caracteres, y en *nregs*, almacenará el número de registros existentes en el fichero. Un atributo más, *tamañoRegs*, especificará el tamaño que hayamos previsto para cada registro y otro, *regsEliminados*, tomará el valor **true** cuando se marque algún registro para eliminar. En nuestro caso, la información almacenada en un registro se corresponde con el *nombre*, *dirección* y *teléfono* de un objeto *CPersona*.

Ateniéndonos a lo explicado, veamos a continuación el esqueleto de la clase *CListaTfnos* y el constructor de la misma:

```
////////////////////////////////////
// Definición de la clase CListaTfnos.
//
using System;
using System.IO;
public class CListaTfnos
{
    private FileStream fs;           // flujo subyacente
    private BinaryWriter bw;        // flujo hacia el fichero
    private BinaryReader br;        // flujo desde el fichero
    private int nregs;              // número de registros
    private int tamañoReg = 140;    // tamaño del registro en bytes
    private bool regsEliminados = false; // true si se
                                    // eliminaron registros
}
```

```
public CListaTfnos(string fichero)
{
    if (Directory.Exists(fichero))
        throw new IOException(Path.GetFileName(fichero) + " no es un fichero");
    fs = new FileStream(fichero, FileMode.OpenOrCreate, FileAccess.ReadWrite);
    bw = new BinaryWriter(fs);
}
```

```

        br = new BinaryReader(fs);

        // Como es casi seguro que el último registro no ocupe el
        // tamaño fijado, utilizamos Ceiling para redondear por encima.
        nregs = (int)Math.Ceiling((double)fs.Length / (double)tamañoReg);
    }

    public void CerrarFichero() {bw.Close(); br.Close(); fs.Close();}

    public int NúmeroDeRegs() { return nregs; } // nº de registros

    public bool EscribirReg(int i, CPersona obj)
    {
        // ...
    }

    public void AñadirReg(CPersona obj)
    {
        // ...
    }

    public CPersona LeerReg(int i)
    {
        // ...
    }

    public bool EliminarReg(long tel)
    {
        // ...
    }

    public bool TieneRegsEliminados()
    {
        // ...
    }

    public int BuscarReg(string str, int pos)
    {
        // ...
    }
}

```

Observe que el constructor de la clase verifica si el argumento pasado corresponde a un nombre de directorio existente; si es así, lanza una excepción del tipo **IOException** (*throw new IOException(mensaje)*) para que sea atrapada por el método que lo invoque (en este caso, la ejecución del constructor se abandona); en otro caso, abre un flujo que permitirá acceder aleatoriamente al fichero especificado para leer y escribir, y calcula el número de registros de dicho fichero. Si el fichero no existe, se crea con cero registros.

En general, hemos previsto que cualquier excepción del tipo **IOException** que se lance debido a un error de E/S sea atrapada en la función **Main**.

Para entender el cálculo de la variable *nregs*, recuerde que el desplazamiento realizado con **Seek** puede ir más allá del final del fichero; esta acción no cambiará la longitud del fichero, a no ser que a continuación, realicemos una operación de escritura; por eso, el tamaño del último registro será menor o igual que *tamañoReg*. Esto es, para que todos los registros del fichero tengan la misma longitud, como veremos en el método *EscribirReg*, antes de escribir el siguiente registro realizaremos un desplazamiento múltiplo de *tamañoReg*:

```
// Situar el puntero de L/E en el registro i
bw.BaseStream.Seek(i * tamañoReg, SeekOrigin.Begin);
// Escribir el registro i
```

Escribir un registro en el fichero

El método *EscribirReg* se ha diseñado para que permita escribir los atributos de un objeto *CPersona* dentro del fichero a partir de una posición determinada. Tiene dos parámetros: el primero indica el número de registro que se desea escribir, que puede coincidir con un registro existente, en cuyo caso se sobrescribirá éste último, o bien con el número del siguiente registro que se puede añadir al fichero; y el segundo, hace referencia al objeto *CPersona* cuyos atributos deseamos escribir. El método devolverá un valor **true** si se ejecuta satisfactoriamente y **false** en otro caso.

```
public bool EscribirReg( int i, CPersona obj )
{
    if (i >= 0 && i <= nregs)
    {
        if (obj.Tamaño + 4 > tamañoReg)
            Console.WriteLine("tamaño del registro excedido");
        else
        {
            // Situar el puntero de L/E
            bw.BaseStream.Seek(i * tamañoReg, SeekOrigin.Begin);
            bw.Write(obj.Nombre);
            bw.Write(obj.Dirección);
            bw.Write(obj.Teléfono);
            return true;
        }
    }
    else
        Console.WriteLine("número de registro fuera de límites");
    return false;
}
```

Se observa que lo primero que hace el método es verificar si el número de registro es válido (cuando *i* sea igual a *nregs* es porque se quiere añadir un registro al final del fichero). El primer registro es el cero. Después comprueba que el tamaño de los atributos del objeto *CPersona* más 4 no supere el tamaño establecido para el registro (más 4 porque cada vez que **Write** escribe un **string**, añade 1 o 2 bytes iniciales para dejar constancia del número de bytes que se escriben; esto permitirá posteriormente al método **ReadString** saber cuántos bytes tiene que leer). Si el tamaño está dentro de los límites permitidos, sitúa el puntero de L/E en la posición de inicio correspondiente a ese registro dentro del fichero y escribe los atributos del objeto uno a continuación de otro (vea la definición de **Seek**).

Añadir un registro al final del fichero

El método *AñadirReg* tiene como misión añadir un nuevo registro al final del fichero. Para ello, invoca al método *EscribirReg* pasando como argumentos la posición que ocupará el nuevo registro, que coincide con el valor de *nregs*, y el objeto cuyos atributos se desean escribir.

```
public void AñadirReg(CPersona obj)
{
    if (EscribirReg( nregs, obj )) nregs++;
}
```

Leer un registro del fichero

Para leer un registro del fichero que almacena la lista de teléfonos, la clase *CListaTfnos* proporciona el método *LeerReg*. Este método tiene un parámetro para identificar el número de registro que se desea leer y devuelve el objeto *CPersona* creado a partir de los datos *nombre*, *dirección* y *teléfono* leídos desde el fichero.

```
public CPersona LeerReg(int i)
{
    if (i >= 0 && i < nregs)
    {
        // Situar el puntero de L/E
        br.BaseStream.Seek(i * tamañoReg, SeekOrigin.Begin);

        string nombre, dirección;
        long teléfono;

        nombre = br.ReadString();
        dirección = br.ReadString();
        teléfono = br.ReadInt64();

        return new CPersona(nombre, dirección, teléfono);
    }
}
```

```
else
{
    Console.WriteLine("número de registro fuera de límites");
    return null;
}
```

Lo primero que hace el método *LeerReg* es verificar si el número de registro es válido (el primer registro es el cero). Si el número de registro está dentro de los límites permitidos, sitúa el puntero de L/E en la posición de inicio correspondiente a ese registro dentro del fichero y lee los datos *nombre*, *dirección* y *teléfono* (esto se hace enviando al flujo *br* bajo el cual subyace el flujo *fs* vinculado con el fichero, el mensaje **ReadString** para recuperar los datos de tipo **string** y **ReadInt64** para recuperar el dato de tipo **long**). Finalmente, devuelve un objeto *CPersona* construido a partir de los datos leídos (el valor devuelto será **null** si el número de registro está fuera de límites).

Eliminar un registro del fichero

Puesto que el fichero manipulado se corresponde con una lista de teléfonos, parece lógico buscar el registro que se desee eliminar por el número de teléfono, ya que éste es único. Para este propósito escribiremos un método *EliminarReg* con un parámetro que almacene el número de teléfono a eliminar y que devuelva un valor **true** si la operación se realiza con éxito, o **false** en caso contrario.

```
public bool EliminarReg(long tel)
{
    CPersona obj;
    // Buscar el teléfono y marcar el registro para
    // posteriormente eliminarlo
    for ( int reg_i = 0; reg_i < nregs; reg_i++ )
    {
        obj = LeerReg(reg_i);
        if (obj.Teléfono == tel)
        {
            obj.Teléfono = 0;
            EscribirReg( reg_i, obj );
            regsEliminados = true;
            return true;
        }
    }
    return false;
}
```

El proceso seguido por el método *EliminarReg* es leer registros del fichero, empezando por el registro cero, y comprobar por cada uno de ellos si el teléfono coincide con el valor pasado como argumento (este proceso recibe el nombre de

búsqueda secuencial). Si existe un registro con el número de teléfono buscado, no se borra físicamente del fichero, sino que se marca el registro poniendo un cero como número de teléfono. Esta forma de proceder deja libertad al usuario de la clase *CListaTfños* para eliminar de una sola vez todos los registros marcados al finalizar su aplicación (lo que redundará en velocidad de ejecución), para restaurar un registro marcado para eliminar, para crear un histórico, etc.

¿Hay registros marcados para eliminar?

Para saber si se marcaron registros para eliminar después de haber trabajado con un fichero, añadimos a la clase *CListaTfños* el método *TieneRegsEliminados*. Este método devuelve el valor del atributo privado *regsEliminados*, que inicialmente vale **false** y que es cambiado a **true** por el método *EliminarReg* cuando se marca un registro para borrar.

```
public bool TieneRegsEliminados()
{
    return regsEliminados;
}
```

Buscar un registro en el fichero

Una operación muy común en el trabajo con registros es localizar uno determinado. ¿Cómo buscar un teléfono en una lista de teléfonos? Lo más común es buscar por el nombre del propietario de ese teléfono, aunque también podría realizarse la búsqueda por la dirección. El método *BuscarReg* que se expone a continuación permite realizar la búsqueda por cualquier subcadena perteneciente al nombre. Para ello, utiliza dos parámetros: la subcadena a buscar y a partir de qué registro del fichero se desea buscar. Si la búsqueda termina con éxito, el método devuelve el número del registro correspondiente; en otro caso devuelve el valor -1.

```
public int BuscarReg(string str, int pos)
{
    // Buscar un registro por una subcadena del nombre
    // a partir de un registro determinado
    CPersona obj;
    string nom;

    if (str == null) return -1;
    if (pos < 0) pos = 0;

    for ( int reg_i = pos; reg_i < nregs; reg_i++ )
    {
        obj = LeerReg(reg_i);
        nom = obj.Nombre;
        // ¿str está contenida en nom?
```

```
        if (nom.IndexOf(str) > -1)
            return reg_i;
    }
    return -1;
}
```

Se observa que el método *BuscarReg*, al igual que el método *EliminarReg*, realiza una búsqueda secuencial desde el registro *pos*, comprobando si el nombre de alguno de ellos contiene la subcadena *str*. Lógicamente, al realizar una búsqueda secuencial, el resultado será el número del primer registro que contenga en su nombre la subcadena pasada como argumento; pero también es evidente que es posible continuar la búsqueda a partir del siguiente registro, invocando de nuevo al método *BuscarReg*, pasando como argumentos la misma subcadena y el número de registro siguiente al devuelto en el proceso de búsqueda anterior.

Un ejemplo de acceso aleatorio a un fichero

Hasta aquí, el diseño de la clase *CPersona* y *CListaTfnos*. El siguiente paso será escribir una aplicación que se ejecute así:

```
1. Buscar
2. Buscar siguiente
3. Modificar
4. Añadir
5. Eliminar
6. Salir
   Opción: 4

nombre:   Javier
dirección: Santander
teléfono: 942232323

1. Buscar
2. Buscar siguiente
3. Modificar
4. Añadir
5. Eliminar
6. Salir

   Opción:
```

A la vista del resultado anterior, esta aplicación mostrará un menú que presentará las operaciones que se pueden realizar sobre la lista de teléfonos. Posteriormente, la operación elegida será identificada por una sentencia **switch** y procesada de acuerdo al esquema presentado a continuación:

```

public class Test
{
    static CListaTfnos listatfnos;

    public static bool Modificar(int nreg)
    {
        // ...
    }

    public static void Actualizar(string fActual)
    {
        // ...
    }

    public static int Menú()
    {
        // ...
    }

    public static void Main(string[] args)
    {
        // Crear un objeto lista de teléfonos vacío (con 0 elementos)
        // o con el contenido del fichero listatfnos.dat si existe.
        string fichero = "listatfnos.dat";
        listatfnos = new CListaTfnos(fichero);

        do
        {
            opción = Menú();
            switch (opción)
            {
                case 1: // buscar
                    // Buscar un registro que contenga "cadenabuscar".
                    // Esta subcadena será obtenida del teclado.
                    pos = listatfnos.BuscarReg(cadenabuscar, 0);
                    // Si se encuentra, mostrar sus datos
                    break;
                case 2: // buscar siguiente
                    // Buscar el siguiente registro que contenga la subcadena
                    // utilizada en la última búsqueda.
                    pos = listatfnos.BuscarReg(cadenabuscar, pos + 1);
                    // Si se encuentra, mostrar sus datos.
                    break;
                case 3: // modificar
                    // Modificar el registro pos. La posición pos será
                    // obtenida del teclado.
                    modificado = Modificar(pos);
                    break;
                case 4: // añadir
                    // Obtener del teclado los datos nombre, dirección y
                    // teléfono del nuevo elemento a añadir, y añadirlo.

```

```
        listatfnos.AñadirReg(new CPersona(nombre, dirección, teléfono));
        break;
    case 5: // eliminar
        // Obtener del teclado el número de teléfono a eliminar y
        // eliminarlo de la lista.
        eliminado = listatfnos.EliminarReg(teléfono);
        break;
    case 6: // salir
        // Actualizar la lista de teléfonos si hay registros
        // marcados para eliminar
    }
}
while(opción != 6);
}
```

La ejecución de la aplicación se iniciará por el método **Main** que, en primer lugar, crea el objeto *CListaTfnos* cuya interfaz nos dará acceso aleatorio al fichero especificado. Después, ejecutará un bucle que invocará al método *Menú* encargado de solicitar una de las opciones presentadas por él:

```
public static int Menú()
{
    Console.Write("\n\n");
    Console.WriteLine("1. Buscar");
    Console.WriteLine("2. Buscar siguiente");
    Console.WriteLine("3. Modificar");
    Console.WriteLine("4. Añadir");
    Console.WriteLine("5. Eliminar");
    Console.WriteLine("6. Salir");
    Console.WriteLine();
    Console.Write("    Opción: ");
    int op = 0;
    do
        Leer<int>.Dato(ref op);
    while (op < 1 || op > 6);
    Console.WriteLine();
    return op;
}
```

Elegida una opción del menú presentado, una sentencia **switch** permitirá ejecutar el código que dará solución a la operación seleccionada. Las opciones *Buscar*, *Buscar siguiente*, *Añadir* y *Eliminar* simplemente envían al objeto *CListaTfnos* el mensaje que hace que se ejecute el método de su clase que realiza la operación especificada. Las opciones *Modificar* y *Salir* se explican a continuación. El código completo de la aplicación puede obtenerlo en la carpeta *Cap15\AccesoAleatorio* del CD que acompaña al libro.

Modificar un registro

Una operación importante en el trabajo con ficheros, que se puede realizar de forma rápida y fácil cuando se permite el acceso aleatorio al mismo es modificar alguna parte concreta de la información almacenada en él. En nuestro caso, el objetivo es modificar un registro. Para ello, vamos a añadir a la clase *Test* un método estático denominado *Modificar* con un parámetro que identifique el número de registro del fichero que se desea modificar. Si durante la ejecución no sabemos con exactitud el número del registro que se desea modificar, podemos utilizar las opciones *Buscar* y *Buscar siguiente* para obtenerlo.

Para realizar tal modificación, el proceso seguido por el método es:

- Leer el registro correspondiente al número pasado como argumento y crear un objeto *CPersona* a partir de los datos leídos. Esto permitirá manipular el registro utilizando la interfaz del objeto.
- Presentar un menú que permita modificar el *nombre*, la *dirección* o el *teléfono*, así como salir del proceso guardando los cambios efectuados, o bien salir sin guardar los cambios. Los nuevos datos serán solicitados desde el teclado.
- Una vez realizadas las modificaciones, si se eligió salir guardando los cambios efectuados, el método enviará al objeto *CListaTfnos* el mensaje *Escribir-Reg* pasando como argumento el número de registro que se está modificando y el objeto *CPersona* que aporta los nuevos atributos; el resultado es que se sobrescribe en el fichero el registro especificado.

```
public static bool Modificar(int nreg)
{
    string nombre, dirección;
    long teléfono = 0;
    int op = 0;

    // Leer el registro
    CPersona obj = listatfnos.LeerReg(nreg);
    if (obj == null) return false;

    // Modificar el registro
    do
    {
        Console.WriteLine("\n\n");
        Console.WriteLine("Modificar el dato:");
        Console.WriteLine("1. Nombre");
        Console.WriteLine("2. Dirección");
        Console.WriteLine("3. Teléfono");
        Console.WriteLine("4. Salir y salvar los cambios");
```

```
Console.WriteLine("5. Salir sin salvar los cambios");
Console.WriteLine();
Console.Write("    Opción: ");
Leer<int>.Dato(ref op);

switch( op )
{
    case 1: // modificar nombre
        Console.Write("nombre:   ");
        nombre = Console.ReadLine();
        obj.Nombre = nombre;
        break;
    case 2: // modificar dirección
        Console.Write("dirección: ");
        dirección = Console.ReadLine();
        obj.Dirección = dirección;
        break;
    case 3: // modificar teléfono
        Console.Write("teléfono:  ");
        Leer<long>.Dato(ref teléfono);
        obj.Teléfono = teléfono;
        break;
    case 4: // guardar los cambios
        // Operación pospuesta a la salida del switch
        listatfnos.EscribirReg(nreg, obj);
        return true;
    case 5: // salir sin guardar los cambios
        break;
}
}
while(op != 5);

return false;
}
```

Actualizar el fichero

Los datos del fichero con el que estamos trabajando pueden verse alterados por tres procesos diferentes: *Modificar*, *Añadir* o *Eliminar* un registro. En el caso de *Modificar* o *Añadir* un registro los cambios son realizados directamente sobre el fichero. Pero en el caso de *Eliminar* un registro, éste simplemente es marcado con un número de teléfono 0 para su posterior eliminación, si se cree conveniente. En nuestro caso, vamos a escribir en la clase aplicación un método *Actualizar* que se invoque cuando el usuario de la aplicación seleccione la opción *Salir*, con el objeto de actualizar el fichero, eliminando físicamente los registros marcados.

```
case 6: // salir
    // guardar lista
    if (listatfnos.TieneRegsEliminados())
```

```

    Actualizar(fichero);
    listatfnos = null;
    break;

```

El proceso seguido para realizar lo expuesto es sencillo. Básicamente se creará un fichero temporal (fichero que existe durante un corto espacio de tiempo; mientras lo necesitemos) para guardar todos los registros del fichero actual cuyo número de teléfono sea distinto de cero. Después de realizar esta operación, cerraremos ambos ficheros y utilizaremos la interfaz de la clase **File** para borrar el fichero actual y renombrar (mover) el fichero temporal para que sea el actual.

```

public static void Actualizar(string fActual)
{
    // Crear un fichero temporal
    string ficheroTemp = "listatfnos.tmp";
    CListaTfnos ftemp = new CListaTfnos(ficheroTemp);
    int nregs = listatfnos.NúmeroDeRegs();
    // Copiar en el fichero temporal todos los registros del
    // fichero actual que en su campo teléfono no tengan un 0
    CPersona obj;
    for ( int reg_i = 0; reg_i < nregs; reg_i++ )
    {
        obj = listatfnos.LeerReg(reg_i);
        if (obj.Teléfono != 0)
            ftemp.AñadirReg(obj);
    }
    listatfnos.CerrarFichero();
    ftemp.CerrarFichero();
    File.Delete(fActual);
    File.Move(ficheroTemp, fActual); // renombrar
}

```

UTILIZACIÓN DE DISPOSITIVOS ESTÁNDAR

La salida de un programa puede también ser enviada a un dispositivo de salida que no sea el disco o la pantalla; por ejemplo, a una impresora conectada al puerto USB o al puerto paralelo. Como C# no tiene definido un flujo estándar para el puerto de impresión, la solución pasa por utilizar la clase **PrintDocument**.

La clase **PrintDocument** del espacio de nombres System.Drawing.Printing define un objeto reutilizable que envía el resultado a una impresora. Normalmente, para imprimir un documento, se crea un objeto de esta clase, se establecen las propiedades que describen cómo imprimir y se llama al método **Print** para iniciar el proceso de impresión.

Una solicitud de impresión genera el evento **PrintPage**. Por lo tanto, controlando este evento, podemos dibujar en la página el texto correspondiente al documento. Entre la información que **Print** envía al controlador, hay un objeto **Graphics** que está vinculado con la impresora establecida por defecto. Las siguientes líneas de código muestran cómo realizar esto:

```
using System.Drawing;
using System.Drawing.Printing;

public class Test
{
    private string cadena;
    private Font fuente;

    public void ImprimirCadena(string data)
    {
        PrintDocument pd = new PrintDocument();
        fuente = new Font("Times New Roman", 10);
        cadena = data;
        // Indicar que el evento PrintPage se controlará con el
        // método Imprimir
        pd.PrintPage += new PrintPageEventHandler(Imprimir);
        pd.Print(); // invoca al método Imprimir
    }

    private void Imprimir(object obj, PrintPageEventArgs ev)
    {
        float pos_X = 10;
        float pos_Y = 20;
        ev.Graphics.DrawString(cadena, fuente, Brushes.Black,
                               pos_X, pos_Y, new StringFormat());
    }

    public static void Main(string[] args)
    {
        Test ap = new Test();
        ap.ImprimirCadena("Hola mundo.\n");
    }
}
```

Basándonos en este ejemplo, vamos a escribir una clase *ImprimirFichero*, perteneciente al espacio de nombres *MisClases.ES*, que presente un método público *ImprimirDocumento* que permita escribir por la impresora el fichero de texto pasado como argumento, según muestra el ejemplo siguiente:

```
using MisClases.ES;

public class Test
{
```



```

public static void Main(string[] args)
{
    ImprimirFichero impre = new ImprimirFichero();
    impre.ImprimirDocumento("texto.txt");
}
}

```

La clase *ImprimirFichero* incluirá los atributos *fuelle*, para definir el tipo de letra que utilizaremos en el documento, y *sr*, para definir el flujo utilizado para acceder al fichero que se desea imprimir, así como los métodos *ImprimirDocumento* e *ImprimirPagina*.

El método *ImprimirDocumento* define el flujo *sr* vinculado con el fichero pasado como argumento, define el tipo de letra a utilizar, crea el objeto **PrintDocument** utilizado para imprimir y especifica el método que se tiene que ejecutar como respuesta al evento **PrintPage** generado cuando se ejecuta **Print**.

El método *ImprimirPagina* obtiene los márgenes izquierdo y superior, calcula el número de líneas por página, lee cada una de las líneas del fichero y las imprime. Siempre que se complete una página, se añadirá una página adicional, lo que implica volver a llamar a *ImprimirPagina*, para lo cual, este método simplemente asignará el valor **true** a la propiedad **HasMorePages** del objeto **PrintPageEventArgs** (argumentos del evento **PrintPage**) pasado como argumento.

```

using System;
using System.IO;
using System.Drawing;
using System.Drawing.Printing;

namespace MisClases.ES // espacio de nombres
{
    public class ImprimirFichero
    {
        private Font fuente;
        private StreamReader sr;

        // Imprimir el contenido de un fichero
        public void ImprimirDocumento(string fichero)
        {
            try
            {
                sr = new StreamReader(fichero);

                try
                {
                    fuente = new Font("Arial", 10);
                    PrintDocument pd = new PrintDocument();
                    pd.PrintPage += new PrintPageEventHandler(this.ImprimirPagina);
                }
            }
        }
    }
}

```

```
        pd.Print(); // invoca a ImprimirPagina
    }
    finally
    {
        sr.Close();
    }
}
catch(Exception e)
{
    Console.WriteLine(e.Message);
}
}

// Respuesta al evento PrintPage producido por Print()
private void ImprimirPagina(object obj, PrintPageEventArgs ev)
{
    float lineasPorPag = 0;
    float pos_Y = 0;
    int cuenta = 0;
    float margenIzq = ev.MarginBounds.Left;
    float margenSup = ev.MarginBounds.Top;
    string linea = null;

    // Calcular el número de líneas por página
    float altoFuente = fuente.GetHeight(ev.Graphics);
    lineasPorPag = ev.MarginBounds.Height/altoFuente;

    // Imprimir cada una de las líneas del fichero
    while (cuenta < lineasPorPag && ((linea=sr.ReadLine()) != null))
    {
        pos_Y = margenSup + (cuenta * altoFuente);
        ev.Graphics.DrawString(linea, fuente, Brushes.Black,
                               margenIzq, pos_Y,
                               new StringFormat());

        cuenta++;
    }

    // Si hay más líneas, imprimir otra página
    if(linea != null)
        ev.HasMorePages = true;
    else
        ev.HasMorePages = false;
}
}
```

Como ejemplo, vamos a añadir al menú de la aplicación realizada en el apartado anterior una opción *imprimir* que permita obtener la lista de teléfonos por la impresora.

```

case 6: // imprimir
    ImprimirListaTfnos();
    break;

case 7: // salir
    // ...

```

El código anterior indica que cuando el usuario seleccione la opción 6 del menú de la aplicación, se invocará al método estático *ImprimirListaTfnos* de la clase aplicación *Test*. Este método creará un fichero temporal para almacenar los datos a imprimir, obtendrá el número total de registros del fichero “lista de teléfonos” y establecerá un bucle para guardar cada uno de ellos en el fichero temporal. Finalmente, imprimirá este fichero, utilizando un objeto *ImprimirFichero* y una vez imprimido, lo eliminará. El código completo se muestra a continuación:

```

public static void ImprimirListaTfnos()
{
    // Almacenar lo que se va a imprimir en un fichero temporal
    StreamWriter flujoS = new StreamWriter("temp");

    int nregs = listatfnos.NúmeroDeRegs(); // número de registros
    CPersona obj;

    for (int n = 0; n < nregs; n++)
    {
        // Imprimir el registro n de la lista de teléfonos
        flujoS.WriteLine("Registro: " + n);
        obj = listatfnos.LeerReg(n);
        flujoS.WriteLine(obj.Nombre);
        flujoS.WriteLine(obj.Dirección);
        flujoS.WriteLine(obj.Teléfono);
        flujoS.WriteLine(); // saltar una línea
    }
    flujoS.Close(); // cerrar el flujo hacia la impresora

    ImprimirFichero impre = new ImprimirFichero();
    impre.ImprimirDocumento("temp");
    File.Delete("temp");
}

```

SERIACIÓN DE OBJETOS

La seriación se puede definir como el proceso de almacenamiento del estado de un objeto en un medio de almacenamiento. Durante este proceso, los atributos del objeto y el nombre de la clase, incluido el ensamblado que contiene la clase, se convierten en una secuencia de bytes que, a continuación, se escribe en un medio de almacenamiento. Cuando se realiza el proceso inverso, conocido como deseriación, se crea una copia exacta del objeto original.

¿Por qué guardar un objeto? El motivo más importante es mantener el estado de ese objeto en un medio de almacenamiento de forma que se pueda volver a crear una copia exacta en una fase posterior.

En el siguiente ejemplo, se creará un objeto de una clase de objetos denominada *MiClase* y se guardarán sus datos en un fichero; a continuación, se recuperarán los datos del fichero para volver a crear otro objeto.

Para guardar los valores de un objeto *MiClase*, hay que declarar esa clase con el atributo **Serializable**, y si una clase necesita controlar su proceso de seriación, puede implementar la interfaz **ISerializable**. Por ejemplo:

```
using System;
using System.Collections.Generic;

[Serializable]
public class MiClase
{
    private string nombre;
    private List<double> notas;

    public MiClase(string s)
    {
        nombre = s;
        notas = new List<double>();
    }

    public void AñadirNota(double nota)
    {
        notas.Add(nota);
    }

    public void Mostrar()
    {
        Console.WriteLine(nombre);
        foreach (double nota in notas) Console.WriteLine(nota);
    }
}
```

Obsérvese que hay un atributo, *notas*, que es una colección de objetos, lo cual indica que **List<T>** es una clase cuyos objetos se pueden seriar. Entonces, cuando queramos seriar un conjunto de objetos de una determinada clase serializable, basta con almacenarlos en una colección y seriar la colección.

Si alguno de los atributos no se desea seriar hay que declararlo así:

```
[NonSerialized] private tipo atributo;
```

Ya tenemos una clase cuyos objetos se pueden seriar. Este proceso requiere utilizar los espacios de nombres **System.IO** y **System.Runtime.Serialization**.

```
using System.IO;
using System.Runtime.Serialization.Formatters.Binary;
```

La línea anterior indica que se va a utilizar un formateador para guardar el objeto en un formato binario. Otro formato que podemos utilizar es *Soap*.

El paso siguiente es escribir el código para seriar el objeto:

```
if (objeto != null)
{
    Stream fs = File.Create("misObjetos.bin");
    BinaryFormatter seriador = new BinaryFormatter();
    seriador.Serialize(fs, objeto);
    fs.Close();
}
```

En el código anterior, primero se comprueba que el objeto existe. Si es así, se crea un flujo de la clase **Stream** para guardar el objeto en un fichero binario y un objeto de la clase **BinaryFormatter** para traducirlo.

Para recuperar los datos del fichero y almacenarlos en un objeto de la clase de procedencia, hay que escribir el código siguiente:

```
if (File.Exists("misObjetos.bin"))
{
    Stream fs = File.OpenRead("misObjetos.bin");
    BinaryFormatter deseriador = new BinaryFormatter();
    MiClase nuevoObjeto;
    nuevoObjeto = (MiClase)deseriador.Deserialize(fs);
    fs.Close();
}
```

En el código anterior, primero se comprueba que el fichero existe. Si es así, se crea un flujo de la clase **Stream** para leer del fichero binario los datos correspondientes al objeto y un objeto de la clase **BinaryFormatter** para traducirlos.

Para probar el código anterior puede escribir la clase *Test* mostrada a continuación:

```
using System;
using System.IO;
using System.Runtime.Serialization.Formatters.Binary;
```

```
public class Test
{
    public static void Main()
    {
        MiClase objeto = new MiClase("Un nombre");
        objeto.AñadirNota(7.5);
        objeto.AñadirNota(8);
        objeto.AñadirNota(8.5);

        if (objeto != null)
        {
            objeto.Mostrar();
            Console.WriteLine("Seriación del objeto...");
            Stream fs = File.Create("misObjetos.bin");
            BinaryFormatter seriador = new BinaryFormatter();
            seriador.Serialize(fs, objeto);
            fs.Close();
        }

        if (File.Exists("misObjetos.bin"))
        {
            Console.WriteLine("Deseriación del objeto...");
            Stream fs = File.OpenRead("misObjetos.bin");
            BinaryFormatter deseriador = new BinaryFormatter();
            MiClase nuevoObjeto;
            nuevoObjeto = (MiClase)deseriador.Deserialize(fs);
            fs.Close();
            nuevoObjeto.Mostrar();
        }
    }
}
```

EJERCICIOS RESUELTOS

1. Escribir una clase aplicación denominada *CopiarFichero* que permita copiar el contenido de un fichero en otro. La aplicación será invocada de la forma siguiente:

`CopiarFichero <fichero fuente> <fichero destino>`

Este ejemplo utilizará la clase **File** para asegurarse de que el *fichero fuente* existe. También utilizará esta clase para saber si el *fichero destino* existe, en cuyo caso no puede estar protegido contra escritura. Si el fichero destino no existe, se utilizará la clase **Directory** para verificar que el directorio padre, dado por el directorio actual o por la ruta relativa o absoluta especificada, se trata de un directorio existente y no protegido contra escritura.

Para leer los bytes del fichero fuente y escribirlos en el destino, este ejemplo utilizará la clase **FileStream**.

La funcionalidad de la clase *CopiarFichero* estará soportada fundamentalmente por el método *Copiar* que tiene el prototipo siguiente:

```
public static void Copiar(string fichFuente, string fichDestino)
```

El método *Copiar* básicamente chequea la existencia y permisos de los ficheros fuente y destino y copia el fichero fuente en el destino; en el caso de que el fichero destino ya exista, pregunta si se desea sobrescribir. Así mismo, si ocurre algún error, este método lanzará una excepción del tipo **IOException**, indicando lo ocurrido, que será atrapada por **Main**. Finalmente, si al abrir alguno de los ficheros el nombre especificado no es válido, la aplicación lanzará una excepción del tipo **ArgumentException** que será atrapada por el propio método *Copiar*. También utiliza un bloque **finally** para cerrar los flujos abiertos justo antes de abandonar la ejecución del mismo.

El hecho de que el método *Copiar* lance una excepción del tipo **IOException** (*throw new IOException(mensaje)*) implica la finalización de su ejecución.

A continuación se muestra la aplicación completa, suficientemente comentada como para no tener que abundar en más explicaciones:

```
using System;
using System.IO;

public class CopiarFichero
{
    public static void Copiar(string fichFuente, string fichDestino)
    {
        // Si el fichero fuente y el destino son el mismo fichero...
        if (fichFuente.CompareTo(fichDestino) == 0)
            throw new IOException("No se puede copiar un fichero " +
                                   "sobre sí mismo");

        // Definiciones de variables, referencias y objetos
        FileStream fFuente = null;
        FileStream fDestino = null;

        try
        {
            // Asegurarse de que el fichero "fichFuente" existe
            if (!File.Exists(fichFuente))
                throw new IOException("No existe el fichero " + fichFuente);

            // Si "fichDestino" existe, asegurarse de que no está
```

```
// protegido contra escritura y preguntar si se quiere
// sobrescribir.
if (File.Exists(fichDestino))
{
    FileAttributes atributos = File.GetAttributes(fichDestino);
    if ((atributos & FileAttributes.ReadOnly) != 0)
        throw new IOException("No se puede escribir en " +
                                "el fichero " + fichDestino);
    // Indicar que el fichero ya existe y preguntar si se
    // desea sobrescribir.
    Console.Write("El fichero " + fichDestino + " existe. " +
                  "¿Desea sobrescribirlo? (s/n): ");
    // Leer la respuesta
    char resp = (char)Console.Read();
    Console.ReadLine(); // limpiar el buffer de entrada
    if (resp == 'n' || resp == 'N')
        throw new IOException("Copia cancelada");
}
else
{
    // Si se ha introducido una ruta absoluta, por ejemplo:
    // c:\ejemplos\c#\mifichero
    // y "mifichero" no existe verificar que el directorio
    // padre "c:\ejemplos\c#" existe y no está protegido
    // contra escritura. Ídem para una ruta relativa.
    DirectoryInfo infoDir; // información sobre el directorio
    infoDir = Directory.GetParent(fichDestino);
    // Ruta del directorio padre de fichDestino
    string dirPadre = infoDir.FullName;
    if (!Directory.Exists(dirPadre))
        throw new IOException("El directorio " + dirPadre +
                                " no existe");
    FileAttributes atributos = File.GetAttributes(dirPadre);
    if ((atributos & FileAttributes.ReadOnly) != 0)
        throw new IOException("No se puede escribir en el " +
                                "directorio " + dirPadre);
}

// Para realizar la copia, abrir un flujo de entrada desde
// el fichero fuente y otro de salida hacia el destino.
fFuente = new FileStream(fichFuente, FileMode.Open,
                        FileAccess.Read);
fDestino = new FileStream(fichDestino, FileMode.Create,
                        FileAccess.Write);

// Copiar el fichero fuente en el destino
byte[] buffer = new byte[1024];
int nbytes;
while (true)
{
    nbytes = fFuente.Read(buffer, 0, 1024);
    if (nbytes == 0) break; // se llegó al final del fichero
```



```

        fDestino.Write(buffer, 0, nbytes);
    }
}
catch(ArgumentException)
{
    Console.WriteLine("El nombre del directorio o del fichero " +
        "no es válido");
}
finally
{
    // Cerrar los flujos que estén abiertos
    if (fFuente != null) fFuente.Close();
    if (fDestino != null) fDestino.Close();
}
}

public static void Main(string[] args)
{
    // Main debe recibir dos parámetros: el fichero fuente y
    // el destino.
    if (args.Length != 2)
        Console.WriteLine("Sintaxis: CopiarFichero " +
            "<fichero fuente> <fichero destino>");
    else
    {
        try
        {
            Copiar(args[0], args[1]); // realizar la copia
        }
        catch(IOException e)
        {
            Console.WriteLine("Error: " + e.Message);
        }
    }
}
}

```

2. Queremos escribir una aplicación denominada *Grep* que permita buscar palabras en uno o más ficheros de texto. Como resultado se visualizará, por cada uno de los ficheros, su nombre, el número de línea y el contenido de la misma para cada una de las líneas del fichero que contenga la palabra buscada.

La clase aplicación, *Grep*, deberá proporcionar al menos los siguientes métodos:

- a) *BuscarCadena* para buscar una cadena de caracteres dentro de otra. El prototipo de este método será:

```
static bool BuscarCadena(string cadena1, string cadena2)
```

Este método devolverá **true** si *cadena2* se encuentra dentro de *cadena1*; en otro caso, devolverá **false**.

- b) *BuscarEnFich* para buscar una cadena de caracteres en un fichero de texto e imprimir el número y el contenido de la línea que contiene la cadena. El prototipo de este método será:

```
static void BuscarEnFich(string nombrefich, string cadena)
```

- c) **Main** para que utilizando los métodos anteriores permita buscar una palabra en uno o más ficheros.

La forma de invocar a la aplicación será así:

```
Grep palabra fichero_1 fichero_2... fichero_n
```

A continuación se muestra la aplicación completa, suficientemente comentada.

```
using System;
using System.IO;

public class Grep
{
    public static bool BuscarCadena(string cadena1, string cadena2)
    {
        // ¿cadena2 está contenida en cadena1?
        if (cadena1.IndexOf(cadena2) > -1)
            return true; // sí
        else
            return false; // no
    }

    public static void BuscarEnFich(string nombrefich, string cadena)
    {
        // Definiciones de variables
        StreamReader sr = null;
        try
        {
            // Asegurarse de que el fichero existe
            if (!File.Exists(nombrefich))
            {
                Console.WriteLine("No existe el fichero " + nombrefich);
                return;
            }
            // Abrir un flujo de entrada desde el fichero fuente
            sr = new StreamReader(nombrefich);
```

```
// Buscar cadena en el fichero fuente
string linea;
int nroLinea = 0;
while ((linea = sr.ReadLine()) != null)
{
    // Si se alcanzó el final del fichero,
    // ReadLine devuelve null
    nroLinea++; // contador de líneas
    if (BuscarCadena(linea, cadena))
        Console.WriteLine(nombrefich + " " + nroLinea + " " +
                           linea);
}
}
catch(IOException e)
{
    Console.WriteLine("Error: " + e.Message);
}
finally
{
    // Cerrar el flujo
    if (sr != null) sr.Close();
}
}

public static void Main(string[] args)
{
    // Main debe recibir dos o más parámetros: la cadena a buscar
    // y los ficheros fuente. Por ejemplo:
    // Grep catch Grep.cs Leer.cs

    if (args.Length < 2)
        Console.WriteLine("Sintaxis: Grep " + "<cadena> " +
                           "<fichero 1> <fichero 2>...");
    else
    {
        for (int i = 1; i < args.Length; i++)
            // Buscar args[0] en args[i]
            BuscarEnFich(args[i], args[0]);
    }
}
}
```

EJERCICIOS PROPUESTOS

1. Realizar un programa que permita crear un fichero nuevo, abrir uno existente, añadir, modificar o eliminar registros, y visualizar el contenido del fichero. El nombre del fichero será introducido a través del teclado. Cada registro del fichero estará formado por los datos *referencia* y *precio*. Así mismo, para que el usuario pueda elegir cualquiera de las operaciones enunciadas, el programa visualizará en pantalla un menú similar al siguiente:

1. Fichero nuevo
2. Abrir fichero
3. Añadir registro
4. Modificar registro
5. Eliminar registro
6. Visualizar registros
7. Salir

Opción:

No se permitirá crear un fichero nuevo ni abrir un fichero cuando ya haya uno abierto. Cuando se intente abrir un fichero que no exista, se ofrecerá la posibilidad de mostrar un listado del directorio actual. Finalmente, la opción *Visualizar registros* permitirá mostrar aquellos registros cuya referencia sea una especificada, o bien contenga una subcadena especificada.

Se deberá realizar al menos un método para cada una de las opciones, excepto para *Salir*.

2. Suponga que tenemos en el disco dos ficheros denominados *alumnos* y *modificaciones*. La estructura de cada uno de los registros para ambos ficheros se corresponde con los atributos de una clase como la siguiente:

```
class CRegistro
{
    // Atributos
    private string nombre;
    private float nota;

    // Métodos
    // ...
}
```

Suponga también que ambos ficheros están clasificados ascendentemente por el campo *nombre*.

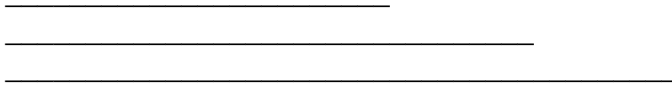
En el fichero *modificaciones* se han grabado las modificaciones que posteriormente realizaremos sobre el fichero *alumnos*. En *modificaciones* hay como máximo un registro por alumno y el total de registros se corresponde con:

- Registros que también están en el fichero *alumnos* pero que han variado en su campo *nota*.
- Registros nuevos; esto es, registros que no están en el fichero *alumnos*.
- Registros que también están en el fichero *alumnos* y que deseamos *eliminar*. Estos registros se distinguen porque su campo *nota* vale -1 .

Se pide realizar un programa que permita obtener a partir de los ficheros *alumnos* y *modificaciones* un tercer fichero siguiendo los criterios de actualización anteriormente descritos. El fichero resultante terminará llamándose *alumnos*.

P A R T E

3



Diseño y programación

- Estructuras dinámicas
- Algoritmos
- Hilos
- Interfaces gráficas y aplicaciones para Internet

ESTRUCTURAS DINÁMICAS

La principal característica de las estructuras dinámicas es la facultad que tienen para variar su tamaño y hay muchos problemas que requieren de este tipo de estructuras. Esta propiedad las distingue claramente de las estructuras estáticas fundamentales como las matrices. Cuando se crea una matriz, su número de elementos se fija en ese instante y después no puede agrandarse o disminuirse elemento a elemento, conservando el espacio actualmente asignado; en cambio, cuando se crea una estructura dinámica eso sí es posible.

Por lo tanto, no es posible asignar una cantidad fija de memoria para una estructura dinámica, y como consecuencia un compilador no puede asociar direcciones explícitas con las componentes de tales estructuras. La técnica que se utiliza más frecuentemente para resolver este problema consiste en realizar una asignación dinámica para las componentes individuales, al tiempo que son creadas durante la ejecución del programa, en vez de hacer la asignación de una sola vez para un número de componentes determinado.

Cuando se trabaja con estructuras dinámicas, el compilador asigna una cantidad fija de memoria para mantener la dirección del componente asignado dinámicamente, en vez de hacer una asignación para el componente en sí. Esto implica que debe haber una clara distinción entre datos y referencias a datos, y que consecuentemente se deben emplear tipos de datos cuyos valores sean referencias a otros datos.

Cuando se asigna memoria dinámicamente para un objeto de un tipo cualquiera, se devuelve una referencia a la zona de memoria asignada. Para realizar esta operación disponemos en C# del operador **new** (vea en el capítulo 5, el apartado *Crear objetos de una clase*).

Este capítulo introduce técnicas en programación orientada a objetos para construir estructuras abstractas de datos. Una vez que haya trabajado los ejemplos de este capítulo, será capaz de explotar en sus aplicaciones la potencia de las listas enlazadas, pilas, colas y árboles binarios.

LISTAS LINEALES

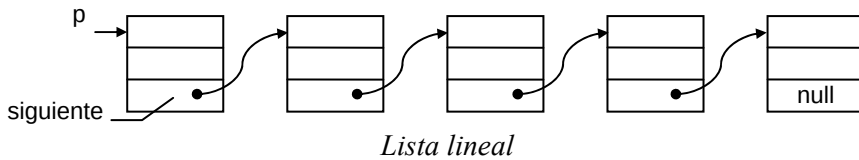
Las matrices primitivas de C#, como sabemos, son colecciones de elementos todos del mismo tipo, ubicados en memoria uno a continuación de otro; el número de elementos es fijado en el instante de crear la matriz. Si más adelante, durante la ejecución del programa, necesitáramos modificar su tamaño para que contenga más o menos elementos, la única alternativa posible sería asignar un nuevo espacio de memoria del tamaño requerido y, además, copiar en él los datos que necesitamos conservar de la matriz original. La nueva matriz pasará a ser la matriz actual y la matriz origen se destruirá, si ya no fuera necesaria.

Es evidente que cada vez que necesitemos añadir o eliminar un elemento a una colección de elementos, la solución planteada en el párrafo anterior no es la más idónea; seguro que estamos pensando en algún mecanismo que nos permita añadir un único elemento a la colección, o bien eliminarlo. Este mecanismo es viable si en lugar de trabajar con matrices lo hacemos con listas lineales. Una lista lineal es una colección, originalmente vacía, de elementos u objetos de cualquier tipo no necesariamente consecutivos en memoria, que durante la ejecución del programa puede crecer o decrecer elemento a elemento según las necesidades previstas en el mismo.

Según la definición dada surge una pregunta: si los elementos no están consecutivos en memoria, ¿cómo pasamos desde un elemento al siguiente cuando recorramos la lista? La respuesta es que cada elemento debe almacenar información de dónde está el siguiente elemento o el anterior, o bien ambos. En función de la información que cada elemento de la lista almacene respecto a la localización de sus antecesores y/o predecesores, las listas pueden clasificarse en: listas simplemente enlazadas, listas circulares, listas doblemente enlazadas y listas circulares doblemente enlazadas.

Listas lineales simplemente enlazadas

Una *lista lineal simplemente enlazada* es una colección de objetos (elementos de la lista), cada uno de los cuales contiene datos o una referencia a los datos, y una referencia al siguiente objeto en la colección (elemento de la lista). Gráficamente puede representarse de la forma siguiente:



Para construir una lista lineal, primero tendremos que definir el tipo de los elementos que van a formar parte de la misma. Por ejemplo, cada elemento de la lista puede definirse como una estructura de datos con dos o más miembros: una referencia al elemento siguiente y una variable que defina el área de datos. El área de datos puede ser de un tipo predefinido o de un tipo definido por el usuario. Según esto, el tipo de cada elemento de una lista puede venir definido de la forma siguiente:

```
public class CElementoLse
{
    // Atributos
    // Defina aquí los datos o las referencias a los datos
    // ...
    public CElementoLse siguiente; // referencia al siguiente elemento

    // Métodos
    public CElementoLse() {} // constructor sin parámetros
    // ...
}
```

Se puede observar que la clase *CElementoLse* definirá una serie de atributos correspondientes a los datos que deseemos manipular, además de un atributo especial, denominado *siguiente*, para permitir que cada elemento pueda hacer referencia a su sucesor formando así una lista enlazada.

Una vez creada la clase de objetos *CElementoLse* la asignación de memoria para un elemento se haría así:

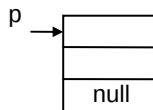
```
public class Test
{
    public static void Main(string[] args)
    {
        CElementoLse p; // referencia a un elemento
        // Asignar memoria para un elemento
        p = new CElementoLse();
        // Este elemento no tiene un sucesor
        p.siguiente = null;
        // Operaciones cualesquiera
        // Permitir que se libere la memoria ocupada por el elemento p
        p = null;
    }
}
```

El código *CElementoLse* *p* define una referencia *p* a un objeto de la clase *CElementoLse*. La sentencia *p = new CElementoLse()* crea (asigna memoria para) un objeto de tipo *CElementoLse*, genera una referencia (dirección de memoria) que direcciona este nuevo objeto y asigna esta referencia a la variable *p*. La sentencia *p.siguiente = null* asigna al miembro *siguiente* del objeto referenciado por *p* el valor **null**, indicando así que después de este elemento no hay otro; esto es, que este elemento es el último de la lista.

El valor **null**, referencia nula, permite crear estructuras de datos finitas. Así mismo, suponiendo que *p* hace referencia al principio de la lista, diremos que dicha lista está vacía si *p* vale **null**. Por ejemplo, después de ejecutar las sentencias:

```
p = null;           // lista vacía
p = new CElementoLse(); // elemento p
p.siguiente = null;  // no hay siguiente elemento
```

tenemos una lista de un elemento:

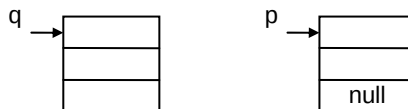


Para añadir un nuevo elemento a la lista, procederemos así:

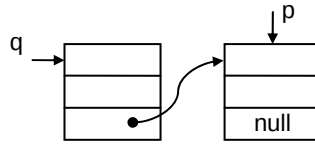
```
q = new CElementoLse(); // crear un nuevo elemento
q.siguiente = p; // almacenar la localización del elemento siguiente
p = q; // p hace referencia al principio de la lista
```

donde *q* es una referencia a un objeto de tipo *CElementoLse*. Ahora tenemos una lista de dos elementos. Observe que los elementos nuevos se añaden al principio de la lista.

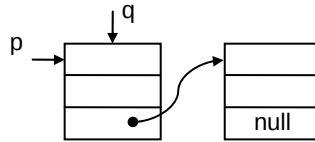
Para verlo con claridad analicemos las tres sentencias anteriores. Partimos de que tenemos una lista referenciada por *p* con un solo elemento. La sentencia *q = new CElementoLse()* crea un nuevo elemento:



La sentencia *q.siguiente = p* hace que el sucesor del elemento creado sea el anteriormente creado. Observe que ahora *q.siguiente* y *p* tienen el mismo valor; esto es, la misma dirección, por lo tanto, referencian el mismo elemento:



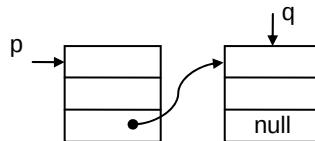
Por último, la sentencia $p = q$ hace que la lista quede de nuevo referenciada por p ; es decir, para nosotros p es siempre el primer elemento de la lista.



Ahora p y q referencian al mismo elemento, al primero. Si ahora se ejecutara una sentencia como la siguiente, ¿qué sucedería?

```
q = q.siguiente;
```

¿Quién es $q.siguiente$? Es el atributo *siguiente* del objeto referenciado por q que contiene la dirección de memoria donde se localiza el siguiente elemento al referenciado por p . Si este valor se lo asignamos a q , entonces q referenciará al mismo elemento que referencia $q.siguiente$. El resultado es que q referencia ahora al siguiente elemento como se puede ver en la figura mostrada a continuación:



Esto nos da una idea de cómo avanzar elemento a elemento sobre una lista. Si ejecutamos de nuevo la misma sentencia:

```
q = q.siguiente;
```

¿Qué sucede? Sucede que como $q.siguiente$ vale **null**, a q se le ha asignado el valor **null**. Conclusión, cuando en una lista utilizamos una referencia para ir de un elemento al siguiente, en el ejemplo anterior q , diremos que hemos llegado al final de la lista cuando q toma el valor **null**.

Operaciones básicas

Las operaciones que podemos realizar con listas incluyen fundamentalmente las siguientes:

1. Insertar un elemento en una lista.
2. Buscar un elemento en una lista.
3. Borrar un elemento de una lista.
4. Recorrer los elementos de una lista.
5. Borrar todos los elementos de una lista.

Partiendo de las definiciones:

```
public class CElementoLse
{
    // Atributos
    public int dato;
    public CElementoLse siguiente; // referencia al siguiente elemento

    // Métodos
    public CElementoLse() {} // constructor sin parámetros
    public CElementoLse( int d ) // constructor con parámetros
    {
        dato = d;
    }
}

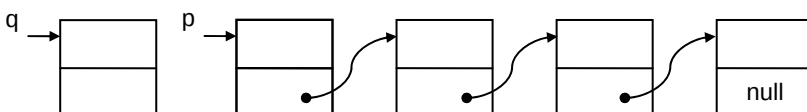
public class Test
{
    public static void Main( string[] args )
    {
        CElementoLse p, q, r; // referencias
        // ...
    }
}
```

vamos a exponer en los siguientes apartados cómo realizar cada una de las operaciones básicas. Observe que por sencillez vamos a trabajar con una lista de enteros.

Insertión de un elemento al comienzo de la lista

Supongamos una lista lineal referenciada por *p*. Para insertar un elemento al principio de la lista, primero se crea el elemento y después se reasignan las referencias, tal como se indica a continuación:

```
q = new CElementoLse();
```

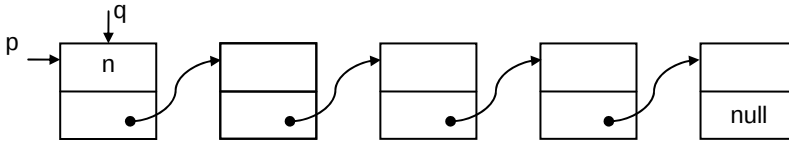


```

q.dato = n;          // asignación de valores
q.siguiente = p;    // reasignación de referencias
p = q;

```

El orden en el que se realizan estas operaciones es esencial. El resultado es:



Esta operación básica nos sugiere cómo crear una lista. Para ello, y partiendo de una lista vacía, no tenemos más que repetir la operación de insertar un elemento al comienzo de una lista, tantas veces como elementos deseemos que tenga dicha lista. Veámoslo a continuación:

```

////////////////////////////////////
// Crear una lista lineal simplemente enlazada
//
public class Test
{
    public static void Main(string[] args)
    {
        CElementoLse p, q; // referencias
        int n = 0;

        // Crear una lista de enteros
        Console.WriteLine("Introducir datos. Finalizar con Ctrl+Z.");

        p = null; // lista vacía

        Console.Write("dato: ");
        while (Leer<int>.Dato(ref n))
        {
            q = new CElementoLse();
            q.dato = n;
            q.siguiente = p;
            p = q;
            Console.Write("dato: ");
        }
    }
}

```

Notar que el orden de los elementos en la lista es inverso al orden en el que han llegado. Así mismo, como ya es habitual, utilizamos la clase genérica *Leer* diseñada en el capítulo 14, para leer datos desde el teclado.

Buscar en una lista un elemento con un valor x

Supongamos que queremos buscar un determinado elemento en una lista cuyo primer elemento está referenciado por p . La búsqueda es secuencial y termina cuando se encuentra el elemento, o bien cuando se llega al final de la lista.

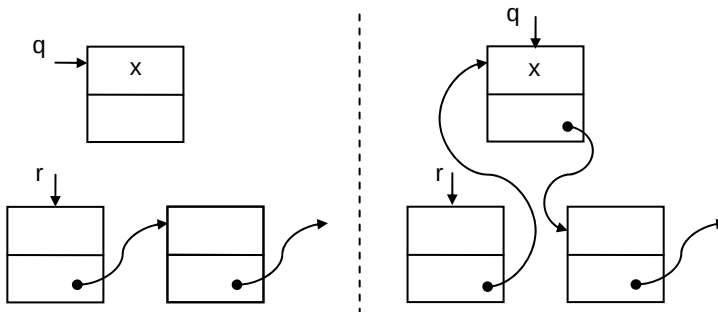
```
q = p; // q referencia el primer elemento de la lista
Console.WriteLine("dato a buscar: "); Leer<int>.Dato(ref x);
while (q != null && q.dato != x)
    q = q.siguiente; // q referencia al siguiente elemento
```

Observe el orden de las expresiones que forman la condición del bucle **while**. Sabemos que en una operación **&&** (AND), cuando una de las expresiones es falsa la condición ya es falsa, por lo que el resto de las expresiones no necesitan ser evaluadas. De ahí que cuando q valga **null**, la expresión $q.dato$ no será evaluada, de lo contrario el sistema de ejecución lanzaría una excepción **NullPointerException**. Finalmente, la variable q quedará apuntando al elemento buscado, o valdrá **null** si ese elemento no se encuentra.

Inserción de un elemento en general

La inserción de un elemento en la lista, a continuación de otro elemento cualquiera referenciado por r , es de la forma siguiente:

```
q = new CElementoLse();
q.dato = x; // valor insertado
q.siguiente = r.siguiente;
r.siguiente = q;
```

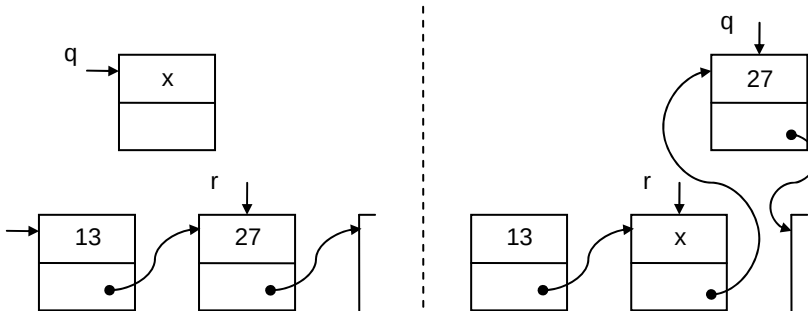


Inserción en la lista detrás del elemento referenciado por r

La inserción de un elemento en la lista antes de otro elemento referenciado por r , se hace insertando un nuevo elemento detrás del elemento referenciado por

r intercambiando previamente los valores del nuevo elemento y del elemento referenciado por r .

```
q = new CElementoLse();
q.dato = r.dato;      // copiar miembro a miembro un objeto en otro
q.siguiete = r.siguiete;
r.dato = x;           // valor insertado
r.siguiete = q;
```

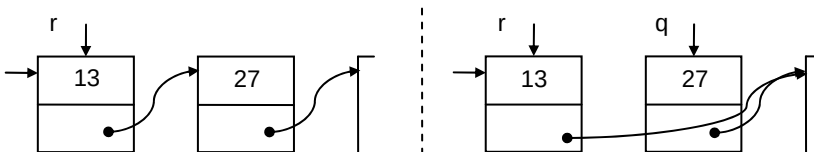


Insertión en la lista antes del elemento referenciado por r

Borrar un elemento de la lista

Para borrar el sucesor de un elemento referenciado por r , las operaciones a realizar son las siguientes:

```
q = r.siguiete;      // q referencia el elemento a borrar
r.siguiete = q.siguiete; // enlazar los elementos anterior
                      // y posterior al borrado
q = null; // objeto referenciado por q a la basura (Borrar)
```

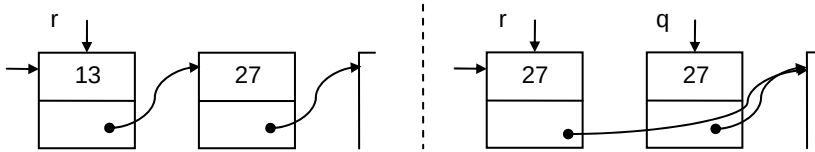


Borrar el sucesor del elemento referenciado por r

Un objeto es enviado a la basura sólo cuando se eliminan todas las referencias que permiten acceder al mismo (véase *Destrucción de objetos* en el capítulo 10).

Observe que para acceder a los miembros de un elemento, éste tiene que estar referenciado por una variable. Por esta razón, lo primero que hemos hecho ha sido referenciar el elemento a borrar por q .

Para borrar un elemento referenciado por r , las operaciones a realizar son las siguientes:



Borrar el elemento referenciado por r

```
q = r.siguiente;
r.dato = q.dato; // copiar miembro a miembro un objeto en otro
r.siguiente = q.siguiente;
q = null;        // objeto referenciado por q a la basura (Borrar)
```

Como ejercicio, escribir la secuencia de operaciones que permitan borrar el último elemento de una lista.

Recorrer una lista

Supongamos que hay que realizar una operación con todos los elementos de una lista, cuyo primer elemento está referenciado por p . Por ejemplo, escribir el valor de cada elemento de la lista. La secuencia de operaciones sería la siguiente:

```
q = p; // salvar la referencia al primer elemento de la lista
while (q != null)
{
    Console.Write(q.dato + " ");
    q = q.siguiente;
}
```

Borrar todos los elementos de una lista

Borrar todos los elementos de una lista equivale a enviar a la basura a cada uno de los elementos de la misma. Supongamos que queremos borrar una lista, cuyo primer elemento está referenciado por p . La secuencia de operaciones es la siguiente:

```
q = p; // q referencia el primer elemento de la lista
while (q != null)
{
    p = p.siguiente; // p referencia al siguiente elemento
    q = null;        // objeto referenciado por q a la basura
    q = p;           // q hace referencia al mismo elemento que p
}
```

Observe que antes de borrar el elemento referenciado por *q*, hacemos que *p* referencie al siguiente elemento, porque si no perderíamos el resto de la lista; la referenciada por *q.siguiente*. Y ¿por qué perderíamos la lista? Porque se pierde la única referencia que nos da acceso a la misma. Entonces, para borrar una lista cuyo primer elemento está referenciado por *p* bastaría con hacer:

```
p = null; // borrar la lista referenciada por p
```

Evidentemente, el proceso anterior no es necesario. Para eliminar una lista basta con poner a **null** la variable que hace referencia al primer elemento de la misma, porque esto implica que todos los elementos de ella queden desreferenciados y sean enviados a la basura para ser recogidos por el recolector de basura.

UNA CLASE PARA LISTAS LINEALES

Basándonos en las operaciones básicas sobre listas lineales descritas anteriormente, vamos a escribir a continuación una clase que permita crear una lista lineal simplemente enlazada en la que cada elemento conste de dos miembros: un valor real de tipo **double** y una referencia a un elemento del mismo tipo.

La clase la denominaremos *CListaLinealSE* (Clase *Lista Lineal* Simplemente Enlazada). Dicha clase incluirá un atributo *p* para almacenar de forma permanente una referencia al primer elemento de la lista, y una clase anidada *CElemento* que definirá la estructura de un elemento de la lista, que según hemos indicado anteriormente será así:

```
private class CElemento
{
    // Atributos
    public double dato;
    public CElemento siguiente; // siguiente elemento

    // Métodos
    public CElemento() {} // constructor
}
```

Finalmente, para simplificar, la interfaz pública de la clase *CListaLinealSE* proporcionará solamente los métodos siguientes: un constructor sin parámetros, *AñadirAlPrincipio*, *Tamaño* y *Obtener*.

El constructor dará lugar a una lista vacía. El método *AñadirAlPrincipio* permitirá añadir un nuevo elemento al principio de la lista, en nuestro caso un valor de tipo **double** recibido como parámetro por el método, *Tamaño* devolverá el número de elementos de la lista y *Obtener* tendrá un primer parámetro donde de-

volverá el dato almacenado por el elemento de la posición especificada por su segundo parámetro (la primera posición es la 0) y devolverá un valor **true** si existe un elemento en la posición especificada, o bien el valor **false** si la lista está vacía o la posición especificada está fuera de límites.

Según lo expuesto, *CListaLinealSE* será así:

```
using System;
using MisClases.ES;
// Crear una lista lineal simplemente enlazada
//
public class CListaLinealSE
{
    // Elemento de una lista lineal simplemente enlazada
    private class CElemento
    {
        // Atributos
        public double dato;
        public CElemento siguiente; // siguiente elemento
        // Métodos
        public CElemento() {} // constructor
    }

    // p: referencia al primer elemento de la lista
    private CElemento p = null;
    private int nElementos = 0; // número de elementos de la lista

    public CListaLinealSE() {} // constructor

    public int Tamaño()
    {
        return nElementos;
    }

    // Añadir un elemento al principio de la lista
    public void AñadirAlPrincipio(double n)
    {
        CElemento q = new CElemento();
        q.dato = n; // asignación de valores
        q.siguiente = p; // reasignación de referencias
        p = q;
        nElementos++;
    }

    public bool Obtener(out double e, int i)
    {
        e = 0;
        if (p == null)
        {
```

```

        Console.WriteLine("lista vacía");
        return false;
    }

    CElemento q = p; // referencia al primer elemento
    if (i >= 0)
    {
        // Posicionarse en el elemento i
        for (int n = 0; q != null && n < i; n++)
            q = q.siguiente;

        // Retornar los datos
        if (q != null)
        {
            e = q.dato;
            return true;
        }
    }
    // Índice fuera de límites
    return false;
}
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

```

Apoyándonos en esta clase, vamos a escribir una aplicación basada en una clase *Test* que cree una lista lineal simplemente enlazada que almacene una serie de valores de tipo **double** introducidos desde el teclado. Finalmente, para verificar que todo ha sucedido como esperábamos, mostraremos la lista de valores.

Para llevar a cabo lo expuesto, el método **Main** de esta aplicación realizará tres cosas:

1. Definirá un objeto *lse* de la clase *CListaLinealSE*.
2. Solicitará datos de tipo **double** del teclado y los añadirá a la lista, para lo cual enviará al objeto *lse* el mensaje *AñadirAlPrincipio* por cada dato que añada.
3. Mostrará los datos, para lo cual enviará al objeto *lse* el mensaje *Obtener(out d, i)* para $i = 0, 1, 2, \dots, nElementos - 1$, siendo *nElementos* el tamaño de la lista.

```

using System;
using MisClases.ES;
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
// Crear una lista lineal simplemente enlazada
//
public class Test
{
    public static void Main(string[] args)
    {
        // Crear una lista lineal vacía
        CListaLinealSE lse = new CListaLinealSE();
    }
}

```

```
// Leer datos reales y añadirlos a la lista
double n = 0;

Console.WriteLine("Introducir datos. Finalizar con Ctrl+Z.");
Console.Write("dato: ");
while (Leer<double>.Dato(ref n))
{
    lse.AñadirAlPrincipio(n);
    Console.Write("dato: ");
}

// Mostrar la lista de datos
Console.WriteLine();
double d;
int tam = lse.Tamaño();
for (int i = 0; i < tam; i++)
{
    lse.Obtener(out d, i);
    Console.Write(d + " ");
}
}
```

Lo que hace el segmento *mostrar la lista de datos* es obtener y visualizar los valores de los elementos $0, 1, 2, \dots, tam - 1$, de la lista *lse*. No obstante, es preciso señalar que ésta es una forma poco eficiente de visualizar una lista, puesto que el método *Obtener* para acceder a cada elemento tiene que iniciar el avance hacia él siempre desde el principio de la lista, no desde el último elemento obtenido. La solución sería sencilla si mantenemos una referencia al último elemento accedido, cuestión que abordaremos al hablar de listas circulares doblemente enlazadas.

Clase genérica para listas lineales

La clase *CListaLinealSE* implementada anteriormente ha sido diseñada para manipular listas de un tipo específico de elementos: datos de tipo **double**. No cabe duda que esta clase tendría un mayor interés para el usuario si estuviera diseñada para permitir listas de objetos de cualquier tipo. Ésta es la dirección en la que vamos a trabajar a continuación. En otros lenguajes como C++, esto se hace utilizando plantillas. En C# se hace utilizando clases genéricas.

Según esto, para que la clase *CListaLinealSE* permita listas de objetos de cualquier tipo, basta con que su clase anidada *CElemento* (clase de cada uno de los elementos de la lista) tenga un atributo que sea una referencia de tipo *T*. Un atributo así definido tiene que pertenecer a una clase genérica.

Esta modificación implica dos cambios más: el parámetro del método *AñadirAlPrincipio* tiene que ser ahora de tipo *T*, y el primer parámetro del método *Obtener* tiene que ser también de tipo *T*.

Resumiendo, estamos hablando de una clase genérica *CListaLinealSE<T>*. Ahora, para definir una lista *lse* de elementos de tipo **double** procederíamos así:

```
CListaLinealSE<double> lse = new CListaLinealSE<double>();
```

Para completar la clase *CListaLinealSE* vamos a añadir otros métodos de interés. Todos se describen en la tabla siguiente:

Método	Significado
<i>Tamaño</i>	Devuelve el número de elementos de la lista. No tiene parámetros.
<i>Añadir</i>	Añade un elemento en la posición <i>i</i> . Tiene dos parámetros: posición <i>i</i> y una referencia al objeto a añadir. Devuelve true si la operación se ejecuta satisfactoriamente y false en caso contrario.
<i>AñadirAlPrincipio</i>	Añade un elemento al principio. Tiene un parámetro que es una referencia al objeto a añadir. Devuelve true o false , igual que <i>Añadir</i> .
<i>AñadirAlFinal</i>	Añade un elemento al final. Tiene un parámetro que es una referencia al objeto a añadir. Devuelve true o false , igual que <i>Añadir</i> .
<i>Borrar</i>	Borra el elemento de la posición <i>i</i> . Tiene un parámetro que indica la posición <i>i</i> del objeto a borrar. Devuelve true cuando el objeto es borrado o false si la lista está vacía o el índice está fuera de límites.
<i>BorrarPrimero</i>	Borra el primer elemento. No tiene parámetros. Devuelve true si el objeto es borrado o false si la lista está vacía.
<i>BorrarÚltimo</i>	Borra el último elemento. No tiene parámetros. Devuelve true si el objeto es borrado o false si la lista está vacía.
<i>Obtener</i>	Almacena en su primer parámetro de tipo <i>T</i> el elemento de la posición <i>i</i> especificada en su segundo parámetro. Devuelve true si existe el elemento, o bien false si la lista está vacía o el índice está fuera de límites.
<i>ObtenerPrimero</i>	Almacena en su parámetro de tipo <i>T</i> el primer elemento. Devuelve false si la lista está vacía y true en otro caso.
<i>ObtenerÚltimo</i>	Almacena en su parámetro de tipo <i>T</i> el último elemento. Devuelve false si la lista está vacía y true en otro caso.

A continuación se muestra el código completo de la clase *CListaLinealSE*. Observar que además de los métodos especificados en la tabla anterior, se ha añadido a la clase *CElemento* un constructor con parámetros.

```
using System;
// Lista lineal simplemente enlazada
//
public class CListaLinealSE<T>
{
    // Elemento de una lista lineal simplemente enlazada
    private class CElemento
    {
        // Atributos
        public T datos;
        public CElemento siguiente; // siguiente elemento
        // Métodos
        public CElemento() { } // constructor
        public CElemento(T d, CElemento s) // constructor
        {
            datos = d;
            siguiente = s;
        }
    }

    // p: referencia al primer elemento de la lista.
    // Es el elemento de cabecera.
    private CElemento p = null;
    private int númeroDeElementos = 0;

    public CListaLinealSE() {} // constructor

    public int Tamaño
    {
        // Devuelve el número de elementos de la lista
        get { return númeroDeElementos; }
    }

    public bool Añadir(int i, T obj)
    {
        // Añadir un elemento en la posición i
        if (i > númeroDeElementos || i < 0)
        {
            Console.WriteLine("índice fuera de límites");
            return false;
        }

        // Crear el elemento a añadir
        CElemento q = new CElemento(obj, null);
```



```

// Si la lista referenciada por p está vacía, añadirlo sin más
if (númeroDeElementos == 0)
{
    // Añadir el primer elemento
    p = q;
    númeroDeElementos++;
    return true;
}

// Si la lista no está vacía, encontrar el punto de inserción
CElemento elemAnterior = p;
CElemento elemActual = p;

// Posicionarse en el elemento i
for (int n = 0; n < i; n++)
{
    elemAnterior = elemActual;
    elemActual = elemActual.siguiente;
}
// Dos casos:
// 1) Insertar al principio de la lista
// 2) Insertar después del anterior (incluye insertar al final)
if ( elemAnterior == elemActual ) // insertar al principio
{
    q.siguiente = p;
    p = q; // cabecera
}
else // insertar después del anterior
{
    q.siguiente = elemActual;
    elemAnterior.siguiente = q;
}
númeroDeElementos++;
return true;
}

public bool AñadirAlPrincipio(T obj)
{
    // Añadir un elemento al principio
    return Añadir(0, obj);
}

public bool AñadirAlFinal(T obj)
{
    // Añadir un elemento al final
    return Añadir(Tamaño, obj);
}

public bool Borrar(int i)
{
    // Borrar el elemento de la posición i

```

```
if (i >= númeroDeElementos || i < 0)
    return false;

// Entrar en la lista y encontrar el índice del elemento
CElemento elemAnterior = p;
CElemento elemActual = p;

// Posicionarse en el elemento i
for (int n = 0; n < i; n++)
{
    elemAnterior = elemActual;
    elemActual = elemActual.siguiente;
}
// Dos casos:
// 1) Borrar el primer elemento de la lista
// 2) Borrar el siguiente a elemAnterior (elemActual)
if ( elemActual == p ) // 1)
    p = p.siguiente; // cabecera
else // 2)
    elemAnterior.siguiente = elemActual.siguiente;
númeroDeElementos--;
return true;
// El elemento referenciado por elemActual será enviado a la
// basura (borrado) al quedar desreferenciado, por ser
// elemActual una variable local.
}

public bool BorrarPrimero()
{
    // Borrar el primer elemento
    return Borrar(0);
}

public bool BorrarÚltimo()
{
    // Borrar el último elemento
    return Borrar(Tamaño - 1);
}

public bool Obtener(out T e, int i)
{
    // Obtener el elemento de la posición i
    e = default(T);
    if (i >= númeroDeElementos || i < 0)
        return false;

    CElemento q = p; // referencia al primer elemento
    // Posicionarse en el elemento i
    for (int n = 0; n < i; n++)
        q = q.siguiente;
```

```

        // Retornar los datos
        e = q.datos;
        return true;
    }

    public bool ObtenerPrimero(out T e)
    {
        // Retornar el primer elemento
        e = default(T);
        return Obtener(out e, 0);
    }

    public bool ObtenerÚltimo(out T e)
    {
        // Retornar el último elemento
        e = default(T);
        return Obtener(out e, Tamaño - 1);
    }
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////

```

Como ejercicio, supongamos que deseamos crear una lista lineal simplemente enlazada con la intención de almacenar los nombres de los alumnos de un determinado curso y sus notas de la asignatura de Programación. Según este enunciado, ¿a qué tipo de objeto hará referencia cada elemento de la lista? Pues, a objetos cuya estructura interna sea capaz de almacenar un nombre (dato de tipo **string**) y una nota (dato de tipo **double**). Además, estos objetos podrán recibir una serie de mensajes con la intención de extraer o modificar su contenido. La clase representativa de los objetos descritos la vamos a denominar *CDatos* y puede escribirse de la forma siguiente:

```

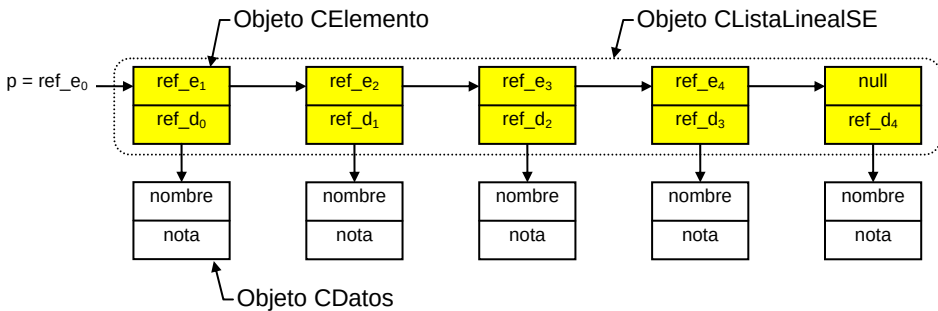
public class CDatos
{
    // Propiedades
    public string Nombre { get; set; }
    public double Nota { get; set; }

    // Métodos
    public CDatos() { }           // constructor sin parámetros

    public CDatos(string nom, double n) // constructor con parámetros
    {
        Nombre = nom;
        Nota = n;
    }
}

```

Sólo nos queda realizar una aplicación que utilizando las clases *CListaLinealSE* y *CDatos* cree una lista lineal y ponga en práctica las distintas operaciones que sobre ella pueden realizarse. La figura siguiente muestra de forma gráfica la estructura de datos que queremos construir. Observe que, en realidad, la lista lo que mantiene son referencias a los datos (objetos *CDatos*) y no los datos en sí, aunque, por sencillez, también resulta aceptable pensar que éstos forman parte de la lista lineal. La variable *p* es una referencia (*ref_e₀*) al elemento de índice 0; este elemento mantiene una referencia (*ref_e₁*) al elemento de la lista de índice 1 y una referencia (*ref_d₀*) al objeto de datos correspondiente, y así sucesivamente.



El código de la aplicación *Test* que se muestra a continuación enseña cómo crear y manipular una estructura de datos como la de la figura anterior:

```

using System;
using MisClases.ES;

////////////////////////////////////
// Crear una lista lineal simplemente enlazada
//
public class Test
{
    public static void MostrarLista(CListaLinealSE<CDatos> lse)
    {
        // Mostrar todos los elementos de la lista
        int i = 0, tam = lse.Tamaño;
        CDatos obj;
        while (i < tam)
        {
            lse.Obtener(out obj, i);
            Console.WriteLine(i + ".- " + obj.Nombre + " " + obj.Nota);
            i++;
        }
    }

    public static void Main(string[] args)
    {
        // Crear una lista lineal vacía
    }
}

```

```

CListaLinealSE<CDatos> lse = new CListaLinealSE<CDatos>();

// Leer datos y añadirlos a la lista
string nombre;
double nota = 0;
Console.WriteLine("Introducir datos. Finalizar con Ctrl+Z.");
Console.Write("nombre: ");
while ((nombre = Console.ReadLine()) != null)
{
    Console.Write("nota:  ");
    Leer<double>.Dato(ref nota);
    lse.AñadirAlFinal(new CDatos(nombre, nota));
    Console.Write("nombre: ");
}

// Añadir un objeto al principio
lse.AñadirAlPrincipio(new CDatos("abcd", 10));
// Añadir un objeto en la posición 1
lse.Añadir(1, new CDatos("defg", 9.5));

Console.WriteLine("\n");
// Mostrar el primero
CDatos obj;
lse.ObtenerPrimero(out obj);
Console.WriteLine("Primero: " + obj.Nombre + " " +
                  obj.Nota);

// Mostrar el último
lse.ObtenerÚltimo(out obj);
Console.WriteLine("Último: " + obj.Nombre + " " +
                  obj.Nota);

// Mostrar todos
Console.WriteLine("Lista:");
MostrarLista(lse);

// Borrar el elemento de índice 2
if (!lse.Borrar(2))
    Console.WriteLine("Error: elemento no borrado");

// Modificar el elemento de índice 1
lse.Obtener(out obj, 1);
obj.Nota = 9;
// Mostrar todos
Console.WriteLine("Lista:");
MostrarLista(lse);
}
}

```

Para finalizar, y como ejercicio, apoyándonos en la aplicación anterior, ¿qué hace la línea de código siguiente?

```
while (lse.Borrar(0));
```

La clase **LinkedList<T>** del espacio de nombres **System.Collections.Generic** de la biblioteca .NET representa una lista doblemente enlazada (estas listas serán expuestas un poco más adelante. Por lo tanto, podría utilizarse también como una lista simplemente enlazada. Como ejercicio, puede reconstruir la aplicación anterior utilizando esta clase en lugar de la clase *CListaLinealSE* (puede ver la solución en la carpeta *Cap16\LinkedList* del CD). En este caso, la clase *CDatos* debe implementar la interfaz **IEquatable** y redefinir el método **Equals** para que el método **LinkedList.Find** funcione adecuadamente.

La interfaz **IEquatable<T>** define el método genérico **Equals**, que determina la igualdad de objetos del tipo que se está implementando. La interfaz **IComparable<T>** define el método genérico **CompareTo**, que determina el criterio de ordenación de objetos del tipo que se está implementando.

```
public class CDatos : IEquatable<CDatos>
{
    // Atributos
    private string nombre;
    private double nota;

    // Métodos
    // ...

    public bool Equals(CDatos obj)
    {
        return (obj.nombre == this.nombre);
    }
}
```

Alternativas al método Obtener

Volvamos al código de la clase *CListaLinealSE<T>* y observemos el prototipo del método *Obtener*:

```
public bool Obtener(out T e, int i)
```

Quizás haya pensado que se podía haber devuelto el objeto solicitado a través de **return** en lugar de utilizar un parámetro por referencia. Esto es:

```
public T Obtener(int i)
{
```

```

    // Obtener el elemento de la posición i
    if (i >= númeroDeElementos || i < 0)
        return null;
    CElemento q = p; // referencia al primer elemento
    // Posicionarse en el elemento i
    for (int n = 0; n < i; n++)
        q = q.siguiente;

    // Retornar los datos
    return q.datos;
}

public T ObtenerPrimero()
{
    return Obtener(0);
}

public T ObtenerÚltimo()
{
    return Obtener(Tamaño - 1);
}

```

El problema con esta versión es que el compilador presenta un error: “no se puede convertir **null** en el parámetro de tipo *T* porque podría ser un tipo de valor”, y da como solución reemplazar **null** por **default(*T*)**. ¿Qué hace **default**? Especifica el valor predeterminado para *T*, que será **null** para los tipos referencia y 0 para los tipos valor. Pero si suponemos que *T* es **int**, por ejemplo, el valor predeterminado para un **int** es cero y cero también es un entero; como vemos, ahora el problema está en distinguir si se devolvió el entero 0 o es que ocurrió un error. Ante esto, otra solución sería lanzar una excepción. Por ejemplo:

```

public T Obtener(int i)
{
    // Obtener el elemento de la posición i
    if (i >= númeroDeElementos || i < 0)
        throw new IndexOutOfRangeException();
    // ...
}

```

Ahora, el método que realice una llamada a *Obtener* tendría que atrapar la posible excepción que se puede lanzar.

Una última solución puede ser reemplazar *T* por **object**:

```

public object Obtener(int i)
{
    // ...
}

```

Esta solución funciona bien, pero exige realizar conversiones explícitas cada vez que invoquemos a *Obtener* para convertir **object** al tipo correspondiente.

Iteradores

Un iterador es un método diseñado para realizar una iteración personalizada sobre una matriz o una colección, que devuelve una secuencia ordenada de valores del mismo tipo. Se invoca desde el código por medio de la instrucción **foreach**.

Como ejemplo, vamos a añadir a la clase *CListaLinealSE<T>* un iterador que permita realizar una operación como la siguiente:

```
foreach (CDatos e in lse)
    Console.WriteLine(e.Nombre + " " + e.Nota);
```

Para que una clase de objetos admita la semántica **foreach** debe implementar el método **GetEnumerator** de la interfaz **IEnumerable**. Este método devuelve una colección que admite iteración porque implementa la interfaz **IEnumerator**, de aquí que el objeto devuelto reciba también el nombre de enumeración. La interfaz **IEnumerator** proporciona los métodos **Current**, **MoveNext** y **Reset** necesarios para iterar. Según lo expuesto, podemos añadir a la clase *CListaLinealSE<T>* el método siguiente:

```
public class CListaLinealSE<T> : IEnumerable
{
    // ...
    public IEnumerator GetEnumerator()
    {
        List<T> lista = new List<T>();
        T e;
        for (int i = 0; i < númeroDeElementos; i++)
        {
            Obtener(out e, i);
            lista.Add(e);
        }
        return lista.GetEnumerator();
    }
}
```

Este método es invocado automáticamente cuando se ejecuta la sentencia **foreach** para iterar sobre un objeto de la clase a la que pertenece dicho método. En el ejemplo, para iterar sobre *lse*, **foreach** invoca al método **GetEnumerator** de *lse* que devuelve la colección de objetos sobre los que se va a iterar.

Otra alternativa que proporciona C# para realizar una iteración personalizada sobre una colección es utilizar la instrucción **yield**. En este caso, al crear el itera-

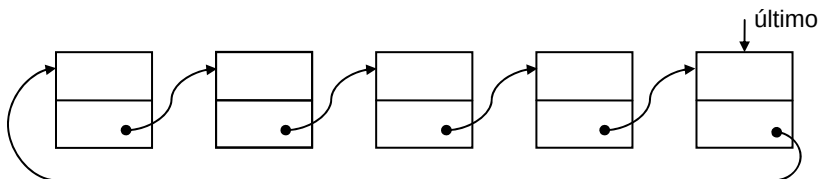
dor no hay que implementar toda la interfaz **IEnumerator** del espacio de nombres **System.Collections**, porque cuando el compilador detecte el iterador utilizando **yield**, generará automáticamente una clase anidada con los métodos **Current**, **MoveNext** y **Reset** de la interfaz **IEnumerator** necesarios para iterar.

```
public IEnumerator GetEnumerator()
{
    if (p == null) yield break;
    CElemento q = p; // referencia al primer elemento
    while (q != null)
    {
        yield return q.datos;
        q = q.siguiente;
    }
}
```

La palabra clave **yield** se utiliza para especificar el valor o los valores devueltos. El código del iterador utiliza la instrucción **yield return** para devolver cada elemento (cuando esto sucede, se guarda la ubicación actual para poder continuar desde ella la próxima vez que se llame al iterador) y **yield break** para finalizar la iteración, cuando sea necesario.

LISTAS CIRCULARES

Una *lista circular* es una lista lineal en la que el último elemento apunta al primero. Entonces es posible acceder a cualquier elemento de la lista desde cualquier punto dado. Las operaciones sobre una lista circular resultan más sencillas, ya que se evitan casos especiales. Por ejemplo, el método *Añadir* de la clase *CListaLinealSE<T>* expuesta anteriormente contempla dos casos: insertar al principio de la lista e insertar a continuación de un elemento. Con una lista circular, estos dos casos se reducen a uno. La siguiente figura muestra cómo se ve una lista circular simplemente enlazada.



Cuando recorremos una lista circular, diremos que hemos llegado al final de la misma cuando nos encontremos de nuevo en el punto de partida, suponiendo, desde luego, que el punto de partida se guarda de alguna manera en la lista; por ejemplo, con una referencia fija al mismo. Esta referencia puede ser al primer elemento de la lista; también puede ser al último elemento, en cuyo caso también

es conocida la dirección del primer elemento. Otra posible solución sería poner un elemento especial identificable en cada lista circular como lugar de partida. Este elemento especial recibe el nombre de elemento de *cabecera* de la lista. Esto presenta, además, la ventaja de que la lista circular no estará nunca vacía.

Como ejemplo, vamos a construir una lista circular con una referencia fija al último elemento. Una lista circular con una referencia al último elemento es equivalente a una lista lineal recta con dos referencias, una al principio y otra al final.

Para construir una lista circular, primero tendremos que definir la clase de objetos que van a formar parte de la misma. Por ejemplo, cada elemento de la lista puede definirse como una estructura de datos con dos miembros: una referencia al elemento siguiente y otra al área de datos. El área de datos puede ser de un tipo predefinido o de un tipo definido por el usuario. Según esto, el tipo de cada elemento de la lista puede venir definido de la forma siguiente:

```
private class CElemento
{
    // Atributos
    public T datos;           // referencia a los datos
    public CElemento siguiente; // siguiente elemento

    // Métodos
    public CElemento() { } // constructor

    public CElemento(T d, CElemento s) // constructor
    {
        datos = d;
        siguiente = s;
    }
}
```

Vemos que por tratarse de una lista lineal simplemente enlazada, aunque sea circular, la estructura de sus elementos no varía con respecto a lo estudiado anteriormente.

Podemos automatizar el proceso de implementar una lista circular diseñando una clase *CListaCircularSE* (Clase *Lista Circular Simplemente Enlazada*) que proporcione los atributos y métodos necesarios para crear cada elemento de la lista, así como para permitir el acceso a los mismos. Esta clase nos permitirá posteriormente derivar otras clases que sean más específicas; por ejemplo, una clase para manipular *pilas* o una clase para manipular *colas*. Estas estructuras de datos las estudiaremos un poco más adelante.

Clase **CListaCircularSE<T>**

La clase *CListaCircularSE* que vamos a implementar incluirá un atributo *último* que valdrá **null** cuando la lista esté vacía y cuando no, referenciará siempre a su último elemento, otro atributo *númeroDeElementos* que almacenará el número de elementos de la lista, una clase anidada, *CElemento*, que definirá la estructura de los elementos, y los métodos indicados en la tabla siguiente:

Propiedad/Método	Significado
<i>Tamaño</i>	Propiedad que devuelve el número de elementos de la lista.
<i>AñadirAlPrincipio</i>	Método que añade un elemento al principio (el primer elemento es el referenciado por <i>último.siguiente</i>). Tiene un parámetro que es una referencia de tipo <i>T</i> al objeto a añadir. No devuelve ningún valor.
<i>AñadirAlFinal</i>	Método que añade un elemento al final (el último elemento siempre estará referenciado por <i>último</i>). Tiene un parámetro que es una referencia de tipo <i>T</i> al objeto a añadir. No devuelve ningún valor.
<i>Borrar</i>	Método que borra el elemento primero (el primer elemento es el referenciado por <i>último.siguiente</i>). No tiene parámetros. Devuelve una referencia al objeto borrado o null si la lista está vacía.
<i>Obtener</i>	Método que devuelve el elemento de la posición <i>i</i> , o bien null si la lista está vacía o el índice está fuera de límites. Tiene un parámetro correspondiente a la posición <i>i</i> del objeto que se desea obtener.

A continuación se presenta el código correspondiente a la definición de la clase *CListaCircularSE<T>*:

```
using System;
////////////////////////////////////
// Lista lineal circular simplemente enlazada
//
public class CListaCircularSE<T> : IEnumerable
{
    // Elemento de una lista lineal circular simplemente enlazada
    private class CElemento
    {
        // Atributos
        public T datos;           // referencia a los datos
        public CElemento siguiente; // siguiente elemento

        // Métodos
        public CElemento() { } // constructor
    }
}
```

```
public CElemento(T d, CElemento s) // constructor
{
    datos = d;
    siguiente = s;
}

// último: referencia el último elemento.
// último.siguiente referencia al primer elemento de la lista.
private CElemento último = null;
private int númeroDeElementos = 0;

public CListaCircularSE() {} // constructor

public int Tamaño
{
    // Devuelve el número de elementos de la lista
    get { return númeroDeElementos; }
}

public void AñadirAlPrincipio(T obj)
{
    // Añade un elemento al principio de la lista.
    // Crear el nuevo elemento.
    CElemento q = new CElemento(obj, null);

    if( último != null ) // existe una lista
    {
        q.siguiente = último.siguiente;
        último.siguiente = q;
    }
    else // inserción del primer elemento
    {
        último = q;
        último.siguiente = q;
    }
    númeroDeElementos++;
}

public void AñadirAlFinal(T obj)
{
    // Añade un elemento al final de la lista.
    // Por lo tanto, último referenciará este nuevo elemento.
    // Crear el nuevo elemento.
    CElemento q = new CElemento(obj, null);

    if( último != null ) // existe una lista
    {
        q.siguiente = último.siguiente;
        último = último.siguiente = q;
    }
}
```

```

        else // inserción del primer elemento
        {
            último = q;
            último.siguiente = q;
        }
        númeroDeElementos++;
    }

    public object Borrar()
    {
        // Devuelve una referencia a los datos del primer elemento de
        // la lista y borra este elemento.
        if( último == null )
        {
            Console.WriteLine( "Lista vacía\n" );
            return null;
        }

        CElemento q = último.siguiente;
        T obj = q.datos;

        if( q == último )
            último = null;
        else
            último.siguiente = q.siguiente;

        númeroDeElementos--;
        return obj;
        // El elemento referenciado por q es enviado a la basura, al
        // quedar desreferenciado cuando finaliza este método por ser
        // q una variable local.
    }

    public object Obtener(int i)
    {
        // Obtener el elemento de la posición i
        if (i >= númeroDeElementos || i < 0)
            return null;

        CElemento q = último.siguiente; // primer elemento
        // Posicionarse en el elemento i
        for (int n = 0; n < i; n++)
            q = q.siguiente;

        // Retornar los datos
        return q.datos;
    }

    public IEnumerator GetEnumerator()
    {
        List<T> lista = new List<T>();

```

```
        for (int i = 0; i < númeroDeElementos; i++)
            lista.Add((T)Obtener(i));
        return lista.GetEnumerator();
    }
}
////////////////////////////////////////////////////////////////
```

Una vez que hemos escrito la clase *CListaCircularSE* vamos a realizar una aplicación que utilizándola cree una lista circular y ponga a prueba las distintas operaciones que sobre ella pueden realizarse. Los elementos de esta lista serán objetos de la clase *CDatos* utilizada en ejemplos anteriores. El código de esta aplicación puede ser el siguiente:

```
using System;
using MisClases.ES;
////////////////////////////////////////////////////////////////
// Crear una lista lineal circular simplemente enlazada
//
public class Test
{
    public static void MostrarLista(CListaCircularSE<CDatos> lcse)
    {
        // Mostrar todos los elementos de la lista
        int i = 0;
        foreach (CDatos e in lcse)
            Console.WriteLine(i++ + ".-  " + e.Nombre + " " + e.Nota);
    }

    public static void Main(string[] args)
    {
        // Crear una lista circular vacía
        CListaCircularSE<CDatos> lcse = new CListaCircularSE<CDatos>();

        // Leer datos y añadirlos a la lista
        string nombre;
        double nota = 0;

        Console.WriteLine("Introducir datos. Finalizar con Ctrl+Z.");
        Console.Write("nombre: ");
        while ((nombre = Console.ReadLine()) != null)
        {
            Console.Write("nota:  ");
            Leer<double>.Dato(ref nota);
            lcse.AñadirAlFinal(new CDatos(nombre, nota));
            Console.Write("nombre: ");
        }

        // Añadir un objeto al principio
        lcse.AñadirAlPrincipio(new CDatos("abcd", 10));
    }
}
```

```

        Console.WriteLine("\n");
        // Mostrar la lista
        Console.WriteLine("Lista:");
        MostrarLista(lcse);

        // Borrar el elemento primero
        CDatos obj = (CDatos)lcse.Borrar();

        // Mostrar la lista
        Console.WriteLine("Lista:");
        MostrarLista(lcse);
    }
}

```

PILAS

Una *pila* es una lista lineal en la que todas las inserciones y supresiones se hacen en un extremo de la lista. Un ejemplo de esta estructura es una pila de platos. En ella, el añadir o quitar platos se hace siempre por la parte superior de la pila. Este tipo de listas recibe también el nombre de listas *LIFO* (*last in first out* - último en entrar, primero en salir).

Las operaciones de meter y sacar en una pila son conocidas en los lenguajes ensambladores como *push* y *pop*, respectivamente. La operación de sacar un elemento de la pila suprime dicho elemento de la misma.

Para trabajar con pilas podemos diseñar una clase *CPila<T>* (Clase *Pila*) derivada de la clase base *CListaCircularSE<T>*, que soporte los métodos:

Método	Significado
<i>MeterEnPila</i>	Mete un elemento en la cima de la pila (todas las inserciones se hacen por el principio de la lista). Tiene un parámetro que es una referencia de tipo <i>T</i> al objeto a añadir. No devuelve ningún valor.
<i>SacarDePila</i>	Saca el primer elemento de la cima de la pila, eliminándolo de la misma (todas las supresiones se hacen por el principio de la lista). No tiene parámetros. Devuelve una referencia al objeto sacado de la pila o null si la pila está vacía.

Según lo expuesto, la definición de esta clase puede ser así:

```

////////////////////////////////////
// Pila: lista en la que todas las inserciones y supresiones se
// hacen en un extremo de la misma.
//

```

```
public class CPila<T> : CListaCircularSE<T>
{
    public CPila() {}

    public void MeterEnPila(T obj)
    {
        AñadirAlPrincipio(obj);
    }

    public T SacarDePila()
    {
        return (T)Borrar();
    }
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

El constructor de la clase *CPila<T>* llama primero al constructor de la clase base que crea una lista con cero elementos. El que la lista sea circular es transparente al usuario de la clase.

Para meter el elemento referenciado por el parámetro *obj* en la pila, el método *MeterEnPila* invoca al método *AñadirAlPrincipio* de la clase base *CListaCircularSE<T>*; y para sacar el elemento de la cima de la pila y eliminarlo de la misma, el método *SacarDePila* invoca al método *Borrar* de la clase base.

Observe que la derivación de la clase *CPila<T>* de *CListaCircularSE<T>* no oculta al usuario la interfaz pública de ésta, lo que permitiría utilizarla. Para ocultarla podríamos declarar protegidos todos sus miembros públicos.

COLAS

Una *cola* es una lista lineal en la que todas las inserciones se hacen por un extremo de la lista (por el final) y todas las supresiones se hacen por el otro extremo (por el principio). Por ejemplo, una fila en un banco. Este tipo de listas recibe también el nombre de listas *FIFO* (*first in first out* - primero en entrar, primero en salir). Este orden es la única forma de insertar y recuperar un elemento de la cola. Una cola no permite acceso aleatorio a un elemento específico. Tenga en cuenta que la operación de sacar elimina el elemento de la cola.

Para trabajar con colas podemos diseñar una clase *CCola<T>* (Clase *Cola*) derivada de la clase base *CListaCircularSE<T>*, que soporte los siguientes métodos:

Método	Significado
<i>MeterEnCola</i>	Mete un elemento al final de la cola (todas las inserciones se hacen por el final de la lista). Tiene un parámetro que es una referencia de tipo <i>T</i> al objeto a añadir. No devuelve ningún valor.
<i>SacarDeCola</i>	Saca el primer elemento de la cola, eliminándolo de la misma (todas las supresiones se hacen por el principio de la lista). No tiene parámetros. Devuelve una referencia al objeto sacado de la cola o null si la cola está vacía.

Según lo expuesto, la definición de esta clase puede ser así:

```

////////////////////////////////////
// Cola: lista en la que todas las inserciones se hacen por un
// extremo de la lista (por el final) y todas las supresiones se
// hacen por el otro extremo (por el principio).
//
public class CCola<T> : CListaCircularSE<T>
{
    public CCola() {}

    public void MeterEnCola(T obj)
    {
        AñadirAlFinal(obj);
    }

    public T SacarDeCola()
    {
        return (T)Borrar();
    }
}
////////////////////////////////////

```

El constructor de la clase *CCola<T>* llama primero al constructor de la clase base que crea una lista con cero elementos. El que la lista sea circular es transparente al usuario de la clase.

Para meter el elemento referenciado por el parámetro *obj* en la cola, el método *MeterEnCola* invoca al método *AñadirAlFinal* de la clase base *CListaCircularSE<T>*; y para sacar el elemento de la cola y eliminarlo de la misma, el método *SacarDeCola* invoca al método *Borrar* de la clase base.

Para que un usuario de las clases *CPila* y *CCola* no tenga acceso a la interfaz de *CListaCircularSE<T>* basta con ocultar dicha interfaz, lo que se podría hacer declarando protegidos todos sus miembros públicos.

EJEMPLO

El siguiente ejemplo muestra cómo utilizar las clases *CPila* y *CCola*. En el método **Main** creamos una pila y una cola de objetos de la clase *CDatos*. Para comprobar que las listas se han creado correctamente, mostramos a continuación los contenidos de las mismas. Obsérvese también el orden de los elementos en ambas listas. Además, para certificar que cuando se saca un elemento de una pila o de una cola éste es eliminado, intentamos mostrar por segunda vez el contenido de las mismas; el resultado es un mensaje de que están vacías.

```
using System;
using MisClases.ES;

////////////////////////////////////
// Pilas y colas
//
public class Test
{
    public delegate T SacarDelegado<T>();

    public static void MostrarLista(CListaCircularSE<CDatos> lista)
    {
        SacarDelegado<CDatos> dSacar = null;
        if (lista is CPila<CDatos>)
            dSacar = ((CPila<CDatos>)lista).SacarDePila;
        else if (lista is CCola<CDatos>)
            dSacar = ((CCola<CDatos>)lista).SacarDeCola;

        // Mostrar todos los elementos de la lista
        int i = 0;
        CDatos obj = dSacar();
        if (obj == null) Console.WriteLine("Lista vacía");
        while (obj != null)
        {
            Console.WriteLine(i++ + ".- " + obj.Nombre + " " + obj.Nota);
            obj = dSacar();
        }
    }

    public static void Main(string[] args)
    {
        // Crear una pila y una cola vacías
        CPila<CDatos> pila = new CPila<CDatos>();
        CCola<CDatos> cola = new CCola<CDatos>();

        // Leer datos y añadirlos a ambas
        string nombre;
        double nota = 0;
        Console.WriteLine("Introducir datos. Finalizar con Ctrl+Z.");
```

```

Console.Write("nombre: ");
while ((nombre = Console.ReadLine()) != null)
{
    Console.Write("nota:  ");
    Leer<double>.Dato(ref nota);
    pila.MeterEnPila(new CDatos(nombre, nota));
    cola.MeterEnCola(new CDatos(nombre, nota));
    Console.Write("nombre: ");
}
Console.WriteLine("\n");

// Mostrar la pila
Console.WriteLine("\nPila:");
MostrarLista(pila);

// Mostrar la pila por segunda vez
Console.WriteLine("\nPila:");
MostrarLista(pila);

// Mostrar la cola
Console.WriteLine("\nCola:");
MostrarLista(cola);

// Mostrar la cola por segunda vez
Console.WriteLine("\nCola:");
MostrarLista(cola);
}
}

```

Si ejecutamos esta aplicación e introducimos los siguientes datos,

Introducir datos. Finalizar con Ctrl+Z.

```

nombre: Alumno 1
nota: 7.5
nombre: Alumno 2
nota: 8.5
nombre: Alumno 3
nota: 9.5
nombre: [Ctrl+Z]

```

se mostrarán los siguientes resultados, los cuales indican que el operador **is** ha funcionando correctamente discriminando la clase de objeto que se deseaba mostrar en cada instante, y que las operaciones de sacar en las pilas y colas eliminan el objeto sacado de las mismas.

```

Pila:
0.- Alumno 3 9.5
1.- Alumno 2 8.5
2.- Alumno 1 7.5

```

```
Pila:  
lista vacía
```

```
Cola:  
0.- Alumno 1 7.5  
1.- Alumno 2 8.5  
2.- Alumno 3 9.5
```

```
Cola:  
lista vacía
```

Obsérvese en estos resultados, que en las pilas el último objeto en entrar es el primero en salir y en las colas, el primero en entrar es el primero en salir.

Puede observarse también que el método *MostrarLista* ha sido escrito para mostrar indistintamente los elementos de una pila o de una cola. Para ello, define una variable de un tipo de delegado coincidente con el prototipo de los métodos *SacarDePila* y *SacarDeCola*. Entonces, dependiendo de que sea una pila o una cola lo que se desea mostrar, esta variable almacenará la referencia al método *SacarDePila* o *SacarDeCola*.

LISTA DOBLEMENTE ENLAZADA

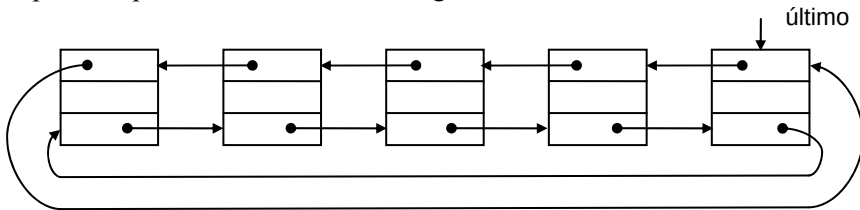
En una lista doblemente enlazada, a diferencia de una lista simplemente enlazada, cada elemento tiene información de dónde se encuentra el elemento posterior y el elemento anterior. Esto permite recorrer la lista en ambas direcciones. Este tipo de listas es útil cuando la inserción, borrado y movimiento de los elementos son operaciones frecuentes. Una aplicación típica es un procesador de textos, donde el acceso a cada línea individual se hace a través de una lista doblemente enlazada.

Las operaciones sobre una lista doblemente enlazada normalmente se realizan sin ninguna dificultad. Sin embargo, casi siempre es mucho más fácil la manipulación de las mismas cuando existe un doble enlace entre el último elemento y el primero, estructura que recibe el nombre de *lista circular doblemente enlazada*. Para moverse sobre una lista circular, es necesario almacenar de alguna manera un punto de referencia; por ejemplo, mediante una referencia al último elemento de la lista.

En el apartado siguiente se expone la forma de construir y manipular una lista circular doblemente enlazada.

Lista circular doblemente enlazada

Una *lista circular doblemente enlazada* (*lcde*) es una colección de objetos, cada uno de los cuales contiene datos o una referencia a los datos, una referencia al elemento siguiente en la colección y una referencia al elemento anterior. Gráficamente puede representarse de la forma siguiente:



Para construir una lista de este tipo, primero tendremos que definir la clase de objetos que van a formar parte de la misma. Por ejemplo, cada elemento de la lista puede definirse como una estructura de datos con tres miembros: una referencia al elemento siguiente, otra al elemento anterior y una variable que defina el área de datos. El área de datos puede ser de un tipo predefinido o de un tipo definido por el usuario. Según esto, el tipo de cada elemento de la lista puede venir definido de la forma siguiente:

```
private class CElemento
{
    // Atributos
    public T datos;           // referencia a los datos
    public CElemento anterior; // anterior elemento
    public CElemento siguiente; // siguiente elemento
    // Métodos
    public CElemento() {} // constructor
}
```

Podemos automatizar el proceso de implementar una lista circular doblemente enlazada, diseñando una clase *CListaCircularDE<T>* (Clase *Lista Circular Doblemente Enlazada*) que proporcione los atributos y métodos necesarios para crear cada elemento de la lista, así como para permitir el acceso a los mismos. La clase que diseñamos a continuación cubre estos objetivos.

Clase *CListaCircularDE<T>*

La clase *CListaCircularDE<T>* que vamos a implementar incluirá los atributos *último*, *actual*, *númeroDeElementos* y *posición*. El atributo *último* valdrá **null** cuando la lista esté vacía y cuando no, referenciará siempre a su último elemento; *actual* hace referencia al último elemento accedido; *númeroDeElementos* es el número de elementos que tiene la lista y *posición* indica el índice del elemento re-

ferenciado por *actual*. Así mismo, incluye una clase anidada, *CElemento*, que definirá la estructura de los elementos, y los métodos indicados en la tabla siguiente:

Propiedad/Método	Significado
<i>CListaCircularDE</i>	Es el constructor. Inicia <i>último</i> y <i>actual</i> a null , <i>númeroDeElementos</i> a 0 y <i>posición</i> a -1 (la posición del primer elemento de la lista es la 0).
<i>Tamaño</i>	Propiedad que devuelve el número de elementos de la lista.
<i>Insertar</i>	Añade un elemento a continuación del referenciado por <i>actual</i> . El elemento añadido pasa a ser el elemento <i>actual</i> . Tiene un parámetro que es una referencia de tipo <i>T</i> al objeto a añadir. No devuelve ningún valor.
<i>Borrar</i>	Borra el elemento referenciado por <i>actual</i> . No tiene parámetros. Devuelve una referencia al objeto borrado o null si la lista está vacía.
<i>IrAlSiguiente</i>	Avanza la posición <i>actual</i> al siguiente elemento. Si esta posición coincide con <i>númeroDeElementos</i> - 1, permanece en ella. No tiene parámetros y no devuelve ningún valor.
<i>IrAlAnterior</i>	Retrasa la posición <i>actual</i> al elemento anterior. Si esta posición coincide con la 0, permanece en ella. No tiene parámetros y no devuelve ningún valor.
<i>IrAlPrincipio</i>	Hace que la posición <i>actual</i> sea la 0. No tiene parámetros y no devuelve ningún valor.
<i>IrAlFinal</i>	Hace que la posición <i>actual</i> sea la <i>númeroDeElementos</i> - 1. No tiene parámetros y no devuelve ningún valor.
<i>IrAl</i>	Avanza la posición <i>actual</i> al elemento de índice <i>i</i> (el primer elemento tiene índice 0). Tiene un parámetro y devuelve true si la operación de mover se realiza con éxito o false si la lista está vacía o el índice está fuera de límites.
<i>Obtener</i>	Devuelve el elemento referenciado por <i>actual</i> , o bien null si la lista está vacía. No tiene parámetros.
<i>Obtener(i)</i>	Devuelve el elemento de la posición <i>i</i> , o bien null si la lista está vacía o el índice está fuera de límites. Tiene un parámetro correspondiente a la posición <i>i</i> del objeto que se desea obtener.
<i>Modificar</i>	Establece nuevos datos para el elemento <i>actual</i> . Tiene un parámetro que es una referencia de tipo <i>T</i> al nuevo objeto. No devuelve ningún valor.

A continuación se presenta el código correspondiente a la definición de la clase *CListaCircularDE<T>*:

```

////////////////////////////////////
// La clase lista circular doblemente enlazada permite manipular
// los elementos que componen una lista de este tipo.
//
public class CListaCircularDE<T>
{
    // Elemento de una lista lineal circular simplemente enlazada
    private class CElemento
    {
        // Atributos
        public T datos;           // referencia a los datos
        public CElemento anterior; // anterior elemento
        public CElemento siguiente; // siguiente elemento

        // Métodos
        public CElemento() { } // constructor
    }

    private CElemento último;
        // referencia al último elemento de la lista
    private CElemento actual;
        // referencia al último elemento accedido
    private long númeroDeElementos;
        // número de elementos que tiene la lista
    private long posición;
        // posición del elemento actual

    public CListaCircularDE() // constructor
    {
        actual = último = null;
        númeroDeElementos = 0L;
        posición = -1L; // la posición del primer elemento será la 0
    }

    public long Tamaño
    {
        // Permite saber el tamaño de la lista
        get { return númeroDeElementos; }
    }

    public void Insertar(T obj)
    {
        // Añade un nuevo elemento a la lista a continuación
        // del elemento actual; el nuevo elemento pasa a ser el
        // actual.
        CElemento q;

        if (último == null) // lista vacía
        {
            último = new CElemento();

```

```
// Las dos líneas siguientes inician una lista circular.
último.anterior = último;
último.siguiente = último;
último.datos = obj;      // asignar datos.
actual = último;
posición = 0L;           // ya hay un elemento en la lista.
}
else // existe una lista
{
    q = new CElemento();
    // Insertar el nuevo elemento después del actual.
    actual.siguiente.anterior = q;
    q.siguiente = actual.siguiente;
    actual.siguiente = q;
    q.anterior = actual;
    q.datos = obj;

    // Actualizar parámetros.
    posición++;
    // Si el elemento actual es el último, el nuevo elemento
    // pasa a ser el actual y el último.
    if( actual == último )
        último = q;
    actual = q; // el nuevo elemento pasa a ser el actual.
} // fin else

númeroDeElementos++; // incrementar en uno el número de elementos.
}

public object Borrar()
{
    // El método borrar devuelve los datos del elemento
    // referenciado por actual y lo elimina de la lista
    // (al quedar desreferenciado es enviado a la basura)
    CElemento q;
    T obj;

    if( último == null ) return ( null ); // lista vacía.

    if( actual == último ) // se trata del último elemento.
    {
        if( númeroDeElementos == 1L ) // hay un solo elemento
        {
            obj = último.datos;
            último = actual = null;
            númeroDeElementos = 0L;
            posición = -1L;
        }
        else // hay más de un elemento
        {
            actual = último.anterior;

```



```

        último.siguiete.anterior = actual;
        actual.siguiete = último.siguiete;
        obj = último.datos;
        último = actual;
        posición--;
        númeroDeElementos--;
    } // fin del bloque else
} // fin del bloque if( actual == último )
else // el elemento a borrar no es el último
{
    q = actual.siguiete;
    actual.anterior.siguiete = q;
    q.anterior = actual.anterior;
    obj = actual.datos;
    actual = q;
    númeroDeElementos--;
}
return obj;
}

public void IrAlSiguiete()
{
    // Avanza la posición actual al siguiente elemento.
    if (posición < númeroDeElementos - 1)
    {
        actual = actual.siguiete;
        posición++;
    }
}

public void IrAlAnterior()
{
    // Retrasa la posición actual al elemento anterior.
    if ( posición > 0L )
    {
        actual = actual.anterior;
        posición--;
    }
}

public void IrAlPrincipio()
{
    // Hace que la posición actual sea el principio de la lista.
    actual = último.siguiete;
    posición = 0L;
}

public void IrAlFinal()
{
    // El final de la lista es ahora la posición actual.
    actual = último;
}

```

```
        posición = númeroDeElementos - 1;
    }

    public bool IrAl(long i)
    {
        // Posicionarse en el elemento i
        if (i >= númeroDeElementos || i < 0) return false;
        IrAlPrincipio();
        // Posicionarse en el elemento i
        for (long n = 0; n < i; n++)
            IrAlSiguiente();
        return true;
    }

    public object Obtener()
    {
        // El método obtener devuelve la referencia a los datos
        // asociados con el elemento actual.
        if ( último == null ) return null; // lista vacía
        return actual.datos;
    }

    public object Obtener(long i)
    {
        // El método obtener devuelve la referencia a los datos
        // asociados con el elemento de índice i.
        if (!IrAl(i)) return null;
        return Obtener();
    }

    public void Modificar(T pNuevosDatos)
    {
        // El método modificar establece nuevos datos para el
        // elemento actual.
        if(último == null) return; // lista vacía
        actual.datos = pNuevosDatos;
    }
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

Cuando se declara un objeto de la clase *CListaCircularDE<T>*, se ejecuta el constructor de la misma que realiza las siguientes operaciones:

- Crea una lista vacía (*último* = *actual* = **null**). En todo momento, el último elemento de la lista está apuntado por *último*, y *actual* apunta al elemento sobre el que se realizará la siguiente operación.
- Asigna un valor 0 al atributo *númeroDeElementos* y un valor -1 a *posición*; el valor de este atributo pasará a ser 0 cuando se añada el primer elemento.

El método *Insertar* de la clase *CListaCircularDE<T>* añade un elemento a la lista a continuación del elemento *actual*. Contempla dos casos: que la lista esté vacía, o que la lista ya exista. El elemento insertado pasa a ser el elemento *actual*, y si se añade al final, éste pasa a ser el *último* y el *actual*. Añadir un elemento implica realizar los enlaces con el anterior y siguiente elementos y actualizar los parámetros *actual*, *númeroDeElementos* y *último*, si procede.

El método *Borrar* devuelve una referencia al objeto de datos asociado con el elemento *actual*, elemento que será enviado a la basura cuando finalice la ejecución del método por quedar desreferenciado. Contempla dos casos: que el elemento a borrar sea el *último* o que no lo sea. Si el elemento a borrar es el *último*, y sólo quedaba éste, los atributos de la lista deben iniciarse igual que lo hizo el constructor; si quedaban más de uno, el que es ahora el nuevo *último* pasa a ser también el elemento *actual*. Si el elemento a borrar no era el *último*, el elemento siguiente al eliminado pasa a ser el elemento *actual*. El método devuelve **null** si la lista está vacía.

Para el resto de los métodos es suficiente con la explicación dada al principio de este apartado, además de en el código.

Ejemplo

El siguiente ejemplo muestra cómo utilizar la clase *CListaCircularDE<T>*. Primeramente creamos un objeto *lcde*, correspondiente a una lista circular doblemente enlazada, en la que cada elemento almacenará una referencia a un objeto *CDatos*; y a continuación realizamos varias operaciones de inserción, movimiento y borrado, para finalmente visualizar los elementos de la lista y comprobar si los resultados son los esperados.

```
using System;
////////////////////////////////////
// Lista circular doblemente enlazada
//
public class Test
{
    public static void mostrarLista(CListaCircularDE<CDatos> lista)
    {
        // Mostrar todos los elementos de la lista
        long i = 0, tam = lista.Tamaño;
        CDatos obj;
        while (i < tam)
        {
            obj = (CDatos)lista.Obtener(i);
            Console.WriteLine(i + ".- " + obj.Nombre + " " + obj.Nota);
            i++;
        }
    }
}
```

```
        if (tam == 0) Console.WriteLine("lista vacía");
    }

    public static void Main(string[] args)
    {
        // Crear una lista vacía
        CListaCircularDE<CDatos> lcde = new CListaCircularDE<CDatos>();

        // Insertar elementos
        lcde.Insertar(new CDatos("alumno1", 7.8));
        lcde.Insertar(new CDatos("alumno2", 6.5));
        lcde.Insertar(new CDatos("alumno3", 10));
        lcde.Insertar(new CDatos("alumno4", 8.6));

        // Ir al elemento de la posición 2
        lcde.IrAl(2);

        // Borrar el elemento actual (posición 2)
        lcde.Borrar();

        // Ir al anterior
        lcde.IrAl(1);
        lcde.Insertar(new CDatos("nuevo alumno3", 9.5));

        // Ir al final
        lcde.IrAlFinal();
        lcde.Insertar(new CDatos("alumno5", 8.5));

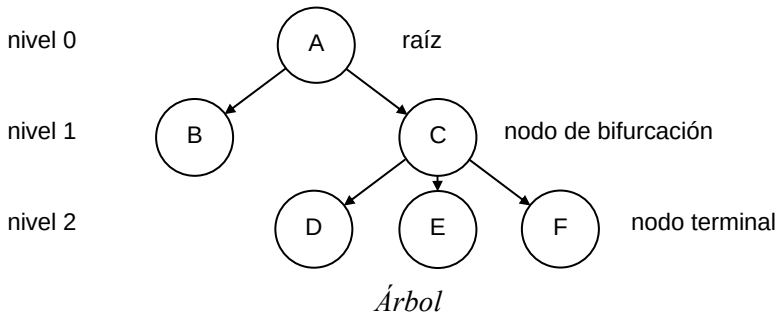
        // Ir al anterior
        lcde.IrAlAnterior();
        lcde.Modificar(new CDatos("alumno4", 5.5));

        // Mostrar la lista
        Console.WriteLine("\nLista:");
        mostrarLista(lcde);
    }
}
```

ÁRBOLES

Un árbol es una estructura no lineal formada por un conjunto de nodos y un conjunto de ramas.

En un árbol existe un nodo especial denominado *raíz*. Así mismo, un nodo del que sale alguna rama recibe el nombre de *nodo de bifurcación* o *nodo rama* y un nodo que no tiene ramas recibe el nombre de *nodo terminal* o *nodo hoja*.



De un modo más formal, diremos que un árbol es un conjunto finito de uno o más nodos tales que:

- Existe un nodo especial llamado *raíz* del árbol, y
- los nodos restantes están agrupados en $n > 0$ conjuntos disjuntos $A_1, \dots, A_n$, cada uno de los cuales es a su vez un árbol que recibe el nombre de *subárbol de la raíz*.

Evidentemente, la definición dada es recursiva; es decir, hemos definido un árbol como un conjunto de árboles, que es la forma más apropiada de definirlo.

De la definición se desprende, que cada nodo de un árbol es la raíz de algún subárbol contenido en la totalidad del mismo.

El número de ramas de un nodo recibe el nombre de *grado* del nodo.

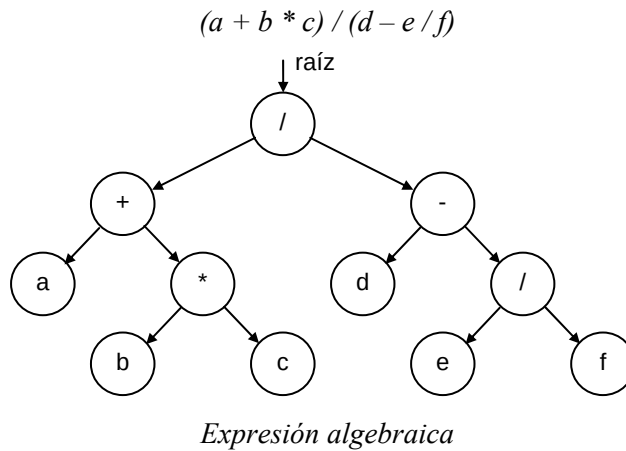
El nivel de un nodo respecto al nodo raíz se define diciendo que la raíz tiene nivel 0 y cualquier otro nodo tiene un nivel igual a la distancia de ese nodo al nodo raíz. El máximo de los niveles se denomina *profundidad* o *altura* del árbol.

Es útil limitar los árboles en el sentido de que cada nodo sea a lo sumo de grado 2. De esta forma cabe distinguir entre subárbol izquierdo y subárbol derecho de un nodo. Los árboles así formados se denominan *árboles binarios*.

Árboles binarios

Un árbol binario es un conjunto finito de nodos que consta de un *nodo raíz* que tiene dos subárboles binarios denominados *subárbol izquierdo* y *subárbol derecho*.

Las expresiones algebraicas, debido a que los operadores que intervienen son operadores binarios, nos dan un ejemplo de estructura en árbol binario. La figura siguiente nos muestra un árbol que corresponde a la expresión aritmética:



El árbol binario es una estructura de datos muy útil cuando el tamaño de la estructura no se conoce, se necesita acceder a sus elementos ordenadamente, la velocidad de búsqueda es importante o el orden en el que se insertan los elementos es casi aleatorio.

En definitiva, un árbol binario es una colección de objetos (nodos del árbol) cada uno de los cuales contiene datos o una referencia a los datos, una referencia a su subárbol izquierdo y una referencia a su subárbol derecho. Según lo expuesto, la estructura de datos representativa de un nodo puede ser de la forma siguiente:

```
// Nodo de un árbol binario
private class CNode
{
    // Atributos
    public T datos;           // referencia a los datos
    public CNode izquierdo;  // raíz del subárbol izquierdo
    public CNode derecho;    // raíz del subárbol derecho
    // Métodos
    public CNode() {}        // constructor
}
```

La definición dada de árbol binario sugiere una forma natural de representar árboles binarios en un ordenador. Una variable *raíz* referenciará el árbol y cada nodo del árbol será un objeto de la clase *CNode*. Esto es, la declaración genérica de un árbol binario puede ser así:

```
public class CArbolBinario<T>
{
    // Atributos del árbol binario
    private CNode raíz;        // raíz del árbol

    // Nodo de un árbol binario
```

```

private class CNodo
{
    // Atributos
    public T datos;           // referencia a los datos
    public CNodo izquierdo;   // raíz del subárbol izquierdo
    public CNodo derecho;     // raíz del subárbol derecho
    // Métodos
    public CNodo() {}         // constructor
}

// Métodos del árbol binario
}

```

Si el árbol está vacío, *raíz* es igual a **null**; en otro caso, *raíz* es una referencia al nodo raíz del árbol y según se puede observar en el código anterior, este nodo tiene tres atributos: una referencia a los datos y dos referencias más, una a su subárbol izquierdo y otra a su subárbol derecho.

Formas de recorrer un árbol binario

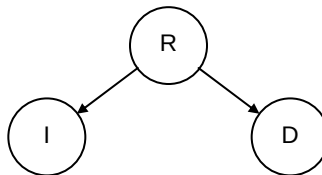
Observe la figura “expresión algebraica” mostrada anteriormente. Partiendo del nodo raíz, ¿qué orden seguimos para poder evaluar la expresión que representa el árbol? Hay varios algoritmos para el manejo de estructuras en árbol y un proceso que generalmente se repite en estos algoritmos es el de recorrido de un árbol. Este proceso consiste en examinar sistemáticamente los nodos de un árbol, de forma que cada nodo sea visitado solamente una vez.

Básicamente se pueden utilizar tres formas para recorrer un árbol binario: *preorden*, *inorden* y *postorden*. Cuando se utiliza la forma *preorden*, primero se visita la raíz, después el subárbol izquierdo y por último el subárbol derecho; en cambio, si se utiliza la forma *inorden*, primero se visita el subárbol izquierdo, después la raíz y por último el subárbol derecho; y si se utiliza la forma *postorden*, primero se visita el subárbol izquierdo, después el subárbol derecho y por último la raíz.

R: raíz

I: subárbol izquierdo

D: subárbol derecho



preorden: R, I, D

inorden: I, R, D

postorden: I, D, R

Formas de recorrer un árbol

Evidentemente, las definiciones dadas son definiciones recursivas, ya que, recorrer un árbol utilizando cualquiera de ellas, implica recorrer sus subárboles empleando la misma definición.

Si se aplican estas definiciones al árbol binario de la figura “expresión algebraica” mostrada anteriormente, se obtiene la siguiente solución:

```
Preorden:      / + a * b c - d / e f
Inorden:       a + b * c / d - e / f
Postorden:     a b c * + d e f / - /
```

El recorrido en preorden produce la notación *prefija*; el recorrido en inorden produce la notación *convencional*; y el recorrido en postorden produce la notación *postfija* o *inversa*.

Los nombres de preorden, inorden y postorden derivan del lugar en el que se visita la raíz con respecto a sus subárboles. Estas tres formas se exponen a continuación como tres métodos recursivos de la clase *CarbolBinario*, con un único parámetro *r* que representa la raíz del árbol cuyos nodos se quieren visitar.

```
////////////////////////////////////
// Clase árbol binario.
//
public class CarbolBinario<T>
{
    // Atributos del árbol binario
    private CNode raíz;          // raíz del árbol

    // Nodo de un árbol binario
    public class CNode
    {
        // Atributos
        public T datos;          // referencia a los datos
        public CNode izquierdo; // raíz del subárbol izquierdo
        public CNode derecho;   // raíz del subárbol derecho
        // Métodos
        public CNode() {}        // constructor
    }

    // Métodos del árbol binario
    public CarbolBinario() {}    // constructor

    public void Preorden(CNode r)
    {
        if ( r != null )
        {
            // Escribir aquí las operaciones a realizar
            // con el nodo referenciado por r
        }
    }
}
```



```

        Preorden( r.izquierdo); // se visita el subárbol izquierdo
        Preorden( r.derecho);   // se visita el subárbol derecho
    }
}

```

```

public void Inorden(CNodo r)
{
    if ( r != null )
    {
        Inorden( r.izquierdo); // se visita el subárbol izquierdo
        // Escribir aquí las operaciones a realizar
        // con el nodo referenciado por r
        Inorden( r.derecho);   // se visita el subárbol derecho
    }
}

```

```

public void Postorden(CNodo r)
{
    if ( r != null )
    {
        Postorden( r.izquierdo); // se visita el subárbol izquierdo
        Postorden( r.derecho);   // se visita el subárbol derecho
        // Escribir aquí las operaciones a realizar
        // con el nodo referenciado por r
    }
}
}

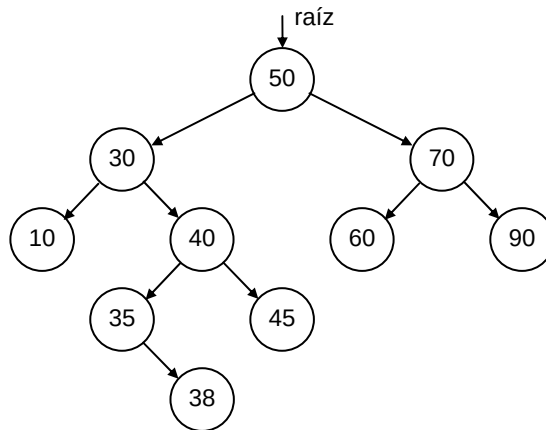
```

////////////////////////////////////

ÁRBOLES BINARIOS DE BÚSQUEDA

Un *árbol binario de búsqueda* es un árbol ordenado; esto es, las ramas de cada nodo están ordenadas de acuerdo con las siguientes reglas: para todo nodo a_i , todas las claves del subárbol izquierdo de a_i son menores que la clave de a_i , y todas las claves del subárbol derecho de a_i son mayores que la clave de a_i .

Con un árbol de estas características, encontrar si un nodo de una clave determinada existe o no es una operación muy sencilla. Por ejemplo, observando la figura siguiente, localizar la clave 35 es aplicar la definición de árbol de búsqueda; esto es, si la clave buscada es menor que la clave del nodo en el que estamos, pasamos al subárbol izquierdo de este nodo para continuar la búsqueda, y si es mayor, pasamos al subárbol derecho. Este proceso continúa hasta encontrar la clave o hasta llegar a un subárbol vacío, árbol cuya raíz tiene un valor **null**.



Árbol binario de búsqueda

En C# podemos automatizar el proceso de implementar un árbol binario de búsqueda diseñando una clase *CArbolBinB* (Clase *Arbol Binario de Búsqueda*) que proporcione los atributos y métodos necesarios para crear cada nodo del árbol, así como para permitir el acceso a los mismos. La clase que diseñamos a continuación cubre estos objetivos.

Clase *CArbolBinB*<T>

La clase *CArbolBinB*<T> que vamos a implementar incluirá un atributo protegido *raíz* para referenciar la raíz del árbol y cuatro constantes públicas relacionadas con los posibles errores que se pueden dar relativos a un nodo: *CORRECTO*, *NO_DATOS*, *YA_EXISTE* y *NO_EXISTE*. El atributo *raíz* valdrá **null** cuando el árbol esté vacío. Así mismo, incluye una clase anidada, *CNodo*, que definirá la estructura de los nodos, y los métodos indicados en la tabla siguiente:

Método	Significado
<i>CArbolBinB</i>	Es el constructor. Crea un árbol vacío (<i>raíz</i> = null).
<i>Buscar</i>	Busca un nodo determinado en el árbol. Tiene un parámetro para almacenar una referencia de tipo <i>T</i> a los datos que permitirán localizar el nodo en el árbol. Devuelve una referencia al área de datos del nodo, o bien null si el árbol está vacío o no existe un nodo con esos datos.
<i>Insertar</i>	Inserta un nodo en el árbol binario basándose en la definición de árbol binario de búsqueda. Tiene un parámetro que es una referencia de tipo <i>T</i> al objeto a añadir. Devuelve un 0, definido por la constante <i>CORRECTO</i> , si la operación de inserción se realiza satisfactoriamente, o un valor distinto

	de 0, definido por alguna de las constantes siguientes: <i>NO_DATOS</i> si la raíz es igual a null , o <i>YA_EXISTE</i> si el nodo con esos datos ya existe.
<i>Borrar</i>	Borra un nodo de un árbol binario de búsqueda. Tiene un parámetro para almacenar una referencia de tipo <i>T</i> a los datos que permitirán localizar en el árbol el nodo que se desea borrar. Devuelve una referencia al área de datos del nodo borrado, o bien null si el árbol está vacío o no existe un nodo con esos datos.
<i>Inorden</i>	Recorre un árbol binario utilizando la forma <i>inorden</i> . Tiene dos parámetros: el primero especifica la referencia al nodo a partir del cual se realizará la visita; el valor del primer parámetro sólo será tenido en cuenta si el segundo es false , porque si es true (valor por omisión) se asume que el primer parámetro es la raíz del árbol. No devuelve ningún valor.
<i>Comparar</i>	Método que debe ser redefinido por el usuario en una clase derivada para especificar el tipo de comparación que se desea realizar con dos nodos del árbol. Debe devolver un entero indicando el resultado de la comparación (-1, 0 o 1 si <i>nodo1</i> < <i>nodo2</i> , <i>nodo1</i> == <i>nodo2</i> , o <i>nodo1</i> > <i>nodo2</i> , respectivamente). Este método es invocado por los métodos <i>Insertar</i> , <i>Borrar</i> y <i>Buscar</i> .
<i>Procesar</i>	Método que debe ser redefinido en una clase derivada para especificar las operaciones que se desean realizar con el nodo visitado. Es invocado por el método <i>Inorden</i> .

A continuación se presenta el código correspondiente a la definición de la clase *CArbolBinB*:

```

////////////////////////////////////
// Clase abstracta: árbol binario de búsqueda. Para utilizar los
// métodos proporcionados por esta clase, tendremos que crear
// una clase derivada de ella y redefinir los métodos:
// Comparar y Procesar.
//
public abstract class CArbolBinB<T>
{
    // Atributos del árbol binario
    private CNode raíz = null; // raíz del árbol

    // Nodo de un árbol binario
    private class CNode
    {
        // Atributos

```

```
public T datos;           // referencia a los datos
public CNode izquierdo;   // raíz del subárbol izquierdo
public CNode derecho;     // raíz del subárbol derecho
// Métodos
public CNode() {}         // constructor
}

// Posibles errores que se pueden dar relativos a un nodo
public const int CORRECTO = 000;
public const int NO_DATOS = 100;
public const int YA_EXISTE = 101;
public const int NO_EXISTE = 102;

// Métodos del árbol binario
public CArbolBinB() {}    // constructor

// El método siguiente debe ser redefinido en una clase derivada
// para que permita comparar dos nodos del árbol por el atributo
// que necesitemos en cada momento.
public abstract int Comparar(T obj1, T obj2);

// El método siguiente debe ser redefinido en una clase derivada
// para que permita especificar las operaciones que se deseen
// realizar con el nodo visitado.
public abstract void Procesar(T obj);

public object Buscar(T obj)
{
}

public int Insertar(T obj)
{
}

public object Borrar(T obj)
{
}

private void Inorden(CNode r, bool nodoRaíz)
{
    // El método recursivo Inorden visita los nodos del árbol
    // utilizando la forma inorden; esto es, primero se visita
    // el subárbol izquierdo, después se visita la raíz, y por
    // último, el subárbol derecho.
    // Si el segundo parámetro es true, la visita comienza
    // en la raíz independientemente del primer parámetro.

    CNode actual = null;

    if ( nodoRaíz )
        actual = raíz; // partir de la raíz
```

```

    else
        actual = r;    // partir de un nodo cualquiera
    if ( actual != null )
    {
        Inorden( actual.izquierdo, false ); // visitar subárbol izq.
        // Procesar los datos del nodo visitado
        Procesar( actual.datos );
        Inorden( actual.derecho, false ); // visitar subárbol dcho.
    }
}

public void Inorden()
{
    // Si el segundo argumento es true, la visita comienza
    // en la raíz independientemente del primer argumento.
    Inorden(null, true);
}
}
////////////////////////////////////

```

Buscar un nodo en el árbol

El método *Buscar*, cuyo código se muestra a continuación, permite acceder a los datos de un nodo del árbol.

```

public object Buscar(T obj)
{
    // El método Buscar permite acceder a un determinado nodo.
    CNode actual = raíz;
    int nComp = 0;

    // Buscar un nodo que tenga asociados los datos dados por obj
    while ( actual != null )
    {
        if ( ( nComp = Comparar( obj, actual.datos ) ) == 0 )
            return( actual.datos ); // CORRECTO (nodo encontrado)
        else if ( nComp < 0 ) // buscar en el subárbol izquierdo
            actual = actual.izquierdo;
        else // buscar en el subárbol derecho
            actual = actual.derecho;
    }
    return null; // NO_EXISTE
}

```

El parámetro *obj* se refiere al objeto de datos, que suponemos apuntado por un nodo del árbol, al que deseamos acceder. Este método devuelve un valor **null** si el objeto referenciado por *obj* no se localiza en el árbol, o bien una referencia al objeto de datos del nodo localizado.

Por definición de árbol de búsqueda, sabemos que sus nodos tienen que estar ordenados utilizando como clave alguno de los atributos de *obj*. Según esto, el método *Buscar* se escribe aplicando estrictamente esa definición; esto es, si la clave buscada es menor que la clave del nodo en el que estamos, continuamos la búsqueda en su subárbol izquierdo y si es mayor, entonces continuamos la búsqueda en su subárbol derecho. Este proceso continúa hasta encontrar la clave, o bien hasta llegar a un subárbol vacío (subárbol cuya raíz tiene un valor **null**).

Para saber si una clave es igual, menor o mayor que otra invocaremos al método *Comparar* pasando como argumentos los objetos de datos que contienen los atributos que se desean comparar. Como tales atributos, dependiendo de la aplicación, pueden ser bien de algún tipo numérico o bien de tipo alfanumérico o alfabético, la implementación de este método hay que posponerla al diseño de la aplicación que utilice esta clase, razón por la que *Comparar* ha sido definido como un método abstracto. Para ello, como veremos un poco más adelante, derivaremos una nueva clase de ésta, y redefiniremos este método.

Insertar un nodo en el árbol

El método *Insertar* cuyo código se muestra a continuación permite añadir un nodo que aún no existe en el árbol.

```
public int Insertar(T obj)
{
    // El método Insertar permite Añadir un nodo que aún no está
    // en el árbol.
    CNodeo último = null, actual = raíz;
    int nComp = 0;

    if ( obj == null ) return NO_DATOS;

    // Comienza la búsqueda para verificar si ya hay un nodo con
    // estos datos en el árbol
    while (actual != null)
    {
        if ((nComp = Comparar( obj, actual.datos )) == 0)
            break; // se encontró el nodo
        else
        {
            último = actual;
            if ( nComp < 0 ) // buscar en el subárbol izquierdo
                actual = actual.izquierdo;
            else // buscar en el subárbol derecho
                actual = actual.derecho;
        }
    }
}
```

```

if ( actual == null ) // no se encontró el nodo, Añadirlo
{
    CNodo nuevoNodo = new CNodo();
    nuevoNodo.datos = obj;
    nuevoNodo.izquierdo = nuevoNodo.derecho = null;

    // El nodo a añadir pasará a ser la raíz del árbol total si
    // éste está vacío, del subárbol izquierdo de "último" si la
    // comparación fue menor, o del subárbol derecho de "último" si
    // la comparación fue mayor.
    if ( último == null ) // árbol vacío
        raíz = nuevoNodo;
    else if ( nComp < 0 )
        último.izquierdo = nuevoNodo;
    else
        último.derecho = nuevoNodo;
    return CORRECTO;
} // fin del bloque if ( actual == null )
else // el nodo ya existe en el árbol
    return YA_EXISTE;
}

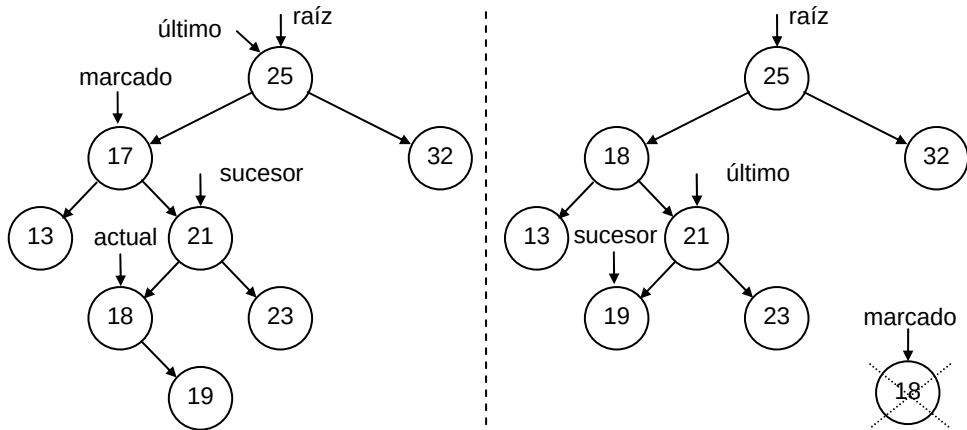
```

El parámetro *obj* se refiere al objeto de datos que será referenciado por el nodo que se añadirá al árbol. Devuelve un entero *NO_DATOS* si *obj* es **null**, *CORRECTO* si la operación de insertar se ejecuta con éxito, y *YA_EXISTE* si ya hay un nodo con los datos referenciados por *obj*.

Este método lo primero que hace es verificar si ya hay un nodo con estos datos en el árbol (para realizar esta operación se sigue el mismo proceso descrito en el método *Buscar*), en cuyo caso, como ya se indicó en el párrafo anterior, lo notificará. Si ese nodo no se encuentra, el proceso de búsqueda conducirá hasta un nodo terminal, posición donde lógicamente debe añadirse el nuevo nodo que almacenará la referencia *obj* a los datos.

Borrar un nodo del árbol

A continuación se estudia el problema de borrar un determinado nodo de un árbol que tiene las claves ordenadas. Este proceso es una tarea fácil si el nodo a borrar es un nodo terminal o si tiene un único descendiente. La dificultad se presenta cuando deseamos borrar un nodo que tiene dos descendientes (en la figura, 17), ya que con una sola referencia no se puede apuntar en dos direcciones. En este caso, el nodo a borrar será reemplazado, bien por el nodo más a la derecha (13) de su subárbol izquierdo (nodo raíz, 13), o bien por el nodo más a la izquierda (18) de su subárbol derecho (nodo raíz, 21). Obsérvese que la forma en la que se ha elegido el nodo empleado en la sustitución, que después se eliminará, conserva la definición de árbol de búsqueda en el árbol resultante.



Borrar el nodo con clave 17

En la figura anterior, la variable *actual* apunta a la raíz del subárbol en el que continúa la búsqueda; inicialmente su valor es *raíz*. La variable *marcado* apunta al nodo a borrar una vez localizado. La variable *último* apunta finalmente al último nodo visitado antes del nodo a borrar.

Para encontrar el nodo a borrar (17), se desciende por el árbol aplicando los criterios que lo definen. Una vez localizado, comprobamos si se corresponde con:

1. Un nodo terminal (no tiene descendientes).
2. Un nodo que no tiene subárbol izquierdo.
3. Un nodo que no tiene subárbol derecho.
4. Un nodo que tiene subárbol izquierdo y derecho.

En los casos 1, 2 y 3 se elimina el nodo apuntado por *marcado* y se actualizan los enlaces del nodo apuntado por *último* para establecer sus descendientes.

En el caso 4, decidimos aplicar el algoritmo de sustitución a partir del subárbol derecho del nodo a borrar. Esto significa descender por este subárbol buscando el nodo más a la izquierda (18), que quedará apuntado por *actual*. Después se copian los datos de *actual* en *marcado*, con la intención de que *actual* pase a ser el nodo a borrar y convertir, de esta forma, el caso 4 en un caso 2 que es más sencillo de tratar; por lo tanto, si *actual* no tiene subárbol derecho, la referencia *marcado.derecho* debe pasar a valer **null**, *marcado* debe apuntar al nuevo nodo a borrar, *último* al nodo anterior y *sucesor* al descendiente de *marcado*. A partir de aquí, la ejecución del método continúa como si se tratara de un caso 2. Finalmente, el nodo apuntado por *marcado* es eliminado.


```

public object Borrar(T obj)
{
    // El método Borrar permite eliminar un nodo del árbol.
    CNode último = null, actual = raíz;
    CNode marcado = null, sucesor = null;
    int nAnteriorComp = 0, nComp = 0;
    if (obj == null) return null; // NO_DATOS

    // Comienza la búsqueda para verificar si hay un nodo con
    // estos datos en el árbol.
    while ( actual != null )
    {
        nAnteriorComp = nComp; // resultado de la comparación anterior
        if (( nComp = Comparar( obj, actual.datos )) == 0)
            break; // se encontró el nodo
        else
        {
            último = actual;
            if ( nComp < 0 ) // buscar en el subárbol izquierdo
                actual = actual.izquierdo;
            else // buscar en el subárbol derecho
                actual = actual.derecho;
        }
    }
    // fin del bloque while ( actual != null )

    if ( actual != null ) // se encontró el nodo
    {
        marcado = actual;
        if (( actual.izquierdo == null && actual.derecho == null ))
            // se trata de un nodo terminal (no tiene descendientes)
            sucesor = null;
        else if ( actual.izquierdo == null ) // nodo sin subárbol izq.
            sucesor = actual.derecho;
        else if ( actual.derecho == null ) // nodo sin subárbol derecho
            sucesor = actual.izquierdo;
        else // nodo con subárbol izquierdo y derecho
        {
            // Referencia del subárbol derecho del nodo a Borrar
            sucesor = actual = actual.derecho;

            // Descender al nodo más a la izquierda en el subárbol
            // derecho de este nodo (el de valor más pequeño)
            while ( actual.izquierdo != null )
                actual = actual.izquierdo;
            // Sustituir el nodo a Borrar por el nodo más a la izquierda
            // en el subárbol derecho que pasará a ser el nodo a Borrar
            marcado.datos = actual.datos;
            if (actual.derecho == null) marcado.derecho = null;
            // Identificar el nuevo nodo a Borrar
            marcado = actual; // éste es ahora el nodo a Borrar
            último = sucesor;
        }
    }
}

```

```
        sucesor = actual.derecho;
    }
    // Actualizar los enlaces prescindiendo de marcado
    if ( último != null )
    {
        if ( nAnteriorComp < 0 )
            último.izquierdo = sucesor;
        else
            último.derecho = sucesor;
    }
    else
        raíz = sucesor;

    return marcado.datos;; // CORRECTO
    // "marcado" será enviado a la basura
}
else // el nodo buscado no está en el árbol
    return null; // NO_EXISTE
}
```

Utilización de la clase **CArbolBinB<T>**

La clase *CArbolBinB<T>* es abstracta; por lo tanto, para hacer uso del soporte que proporciona para la construcción y manipulación de árboles binarios de búsqueda, tendremos que derivar una clase de ella y redefinir los métodos abstractos heredados: *Comparar* y *Procesar*. La redefinición de estos métodos está condicionada a la clase de objetos que formarán parte del árbol.

Como ejemplo, vamos a construir un árbol binario de búsqueda en el que cada nodo se corresponda con un objeto de la clase *CDatos* ya utilizada anteriormente en este mismo capítulo. Esto implica pensar en la clave de ordenación que se utilizará para construir el árbol. En nuestro ejemplo vamos a ordenar los nodos del árbol por el atributo *nombre* de *CDatos*. Se trata entonces de una ordenación alfabética; por lo tanto, el método *Comparar* debe ser redefinido para que devuelva *-1*, *0* o *1* según sea el *nombre* de un objeto *CDatos*, menor, igual o mayor, respectivamente, que el *nombre* del otro objeto con el que se compara.

Pensemos ahora en el proceso que deseamos realizar con cada nodo accedido. En el ejemplo, simplemente nos limitaremos a mostrar los datos *nombre* y *nota*. Según esto, el método *Procesar* obtendrá los datos *nombre* y *nota* del objeto *CDatos* pasado como argumento y los mostrará.

```
////////////////////////////////////
// Clase derivada de la clase abstracta CArbolBinB<T>. Redefine
// los métodos: Comparar y Procesar.
//
```

```

public class CarbolBinarioDeBusqueda : CarbolBinB<CDatos>
{
    // Permite comparar dos nodos del árbol por el atributo nombre.
    public override int Comparar(CDatos obj1, CDatos obj2)
    {
        string str1 = obj1.Nombre;
        string str2 = obj2.Nombre;
        return str1.CompareTo(str2);
    }

    // Permite mostrar los datos del nodo visitado.
    public override void Procesar(CDatos obj)
    {
        string nombre = obj.Nombre;
        double nota = obj.Nota;
        Console.WriteLine(nombre + " " + nota);
    }
}
////////////////////////////////////

```

Ahora puede comprobar de una forma clara que los métodos *Comparar* y *Procesar* dependen del tipo de objetos que almacenemos en el árbol que construyamos. Por esta razón no pudieron ser implementados en la clase *CarbolBinB*, sino que hay que implementarlos para cada caso particular.

Cuando se declare un objeto de la clase *CarbolBinarioDeBusqueda*, el constructor de esta clase invoca al de su clase base *CarbolBinB<T>*, que creará un árbol vacío (*raíz = null*). El atributo *raíz* apunta siempre a la raíz del árbol.

Finalmente, escribiremos una aplicación *Test* que, utilizando la clase *CarbolBinarioDeBusqueda*, cree un objeto *arbolbb* correspondiente a un árbol binario de búsqueda en el que cada nodo haga referencia a un objeto *CDatos* que encapsule el nombre de un alumno y la nota de una determinada asignatura que está cursando. Con el fin de probar que todos los métodos proporcionados por la clase funcionan adecuadamente (piense en los métodos heredados y en los redefinidos), la aplicación realizará las operaciones siguientes:

1. Creará un objeto *arbolbb* de la clase *CarbolBinarioDeBusqueda*.
2. Solicitará parejas de datos *nombre* y *nota*, a partir de las cuales construirá los objetos *CDatos* que añadiremos como nodos en el *arbolbb*.
3. Durante la construcción del árbol, permitirá modificar la nota de un alumno ya existente, o bien eliminarlo. Para discriminar una operación de otra tomaremos como referencia la nueva nota: si es positiva, entenderemos que deseamos modificar la nota del alumno especificado y si es negativa, que hay que eliminarlo.

4. Finalmente, mostrará los datos almacenados en el árbol para comprobar que todo ha sucedido como esperábamos.

```
using System;
using MisClases.ES;
////////////////////////////////////
// Crear un árbol binario de búsqueda
//
public class Test
{
    public static void Main(string[] args)
    {
        CarbolBinarioDeBusqueda arbolbb = new CarbolBinarioDeBusqueda();

        // Leer datos y añadirlos al árbol
        string nombre;
        double nota = 0;
        int cod;

        Console.WriteLine("Introducir datos. Finalizar con Ctrl+Z.");
        Console.Write("nombre: ");
        while ((nombre = Console.ReadLine()) != null)
        {
            Console.Write("nota: ");
            Leer<double>.Dato(ref nota);
            cod = arbolbb.Insertar(new CDatos(nombre, nota));

            if (cod == CarbolBinarioDeBusqueda.YA_EXISTE)
            {
                // Si ya existe, distinguimos dos casos:
                // 1. nota nueva >= 0; cambiamos la nota
                // 2. nota nueva < 0; borramos el nodo
                if (nota >= 0)
                {
                    CDatos datos = (CDatos)arbolbb.Buscar(
                                                                new CDatos(nombre, nota));
                    datos.Nota = nota;
                }
                else
                {
                    arbolbb.Borrar(new CDatos(nombre, nota));
                    Console.WriteLine("nodo borrado");
                }
            }
            Console.Write("nombre: ");
        }
        Console.WriteLine("\n");

        // Mostrar los nodos del árbol
        Console.WriteLine("\nArbol:");
```

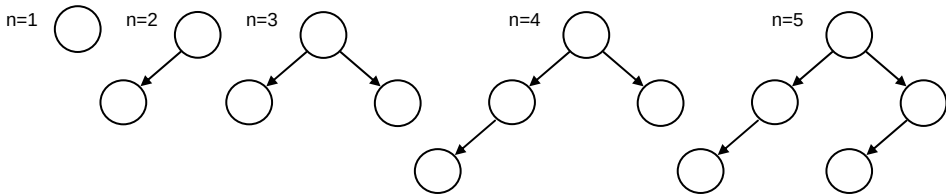
```

        arbolbb.Inorden();
    }
}

```

ÁRBOLES BINARIOS PERFECTAMENTE EQUILIBRADOS

Un árbol binario está perfectamente equilibrado si, para todo nodo, el número de nodos en el subárbol izquierdo y el número de nodos en el subárbol derecho difieren como mucho en una unidad.



Árboles perfectamente equilibrados

Como ejemplo, considere el problema de construir un árbol perfectamente equilibrado siendo los valores almacenados en los nodos n objetos de la clase *CDatos* implementada anteriormente en este mismo capítulo. Recuerde que cada objeto de esta clase encapsula el nombre de un alumno y la nota de una determinada asignatura que está cursando.

Esto puede realizarse fácilmente distribuyendo los nodos, según se leen, equitativamente a la izquierda y a la derecha de cada nodo. El proceso recursivo que se indica a continuación es la mejor forma de realizar esta distribución. Para un número dado n de nodos y siendo ni (nodos a la izquierda) y nd (nodos a la derecha) dos enteros, el proceso es el siguiente:

1. Utilizar un nodo para la raíz.
2. Generar el subárbol izquierdo con $ni = n/2$ nodos utilizando la misma regla.
3. Generar el subárbol derecho con $nd = n - ni - 1$ nodos utilizando la misma regla.

Cada nodo del árbol consta de los siguientes miembros: *datos*, referencia al subárbol *izquierdo* y referencia al subárbol *derecho*.

```

private class CNodo
{
    // Atributos
    public T datos;           // referencia a los datos
    public CNodo izquierdo;   // raíz del subárbol izquierdo
    public CNodo derecho;     // raíz del subárbol derecho
}

```

```
// Métodos
public CNodo() {}           // constructor
}
```

En C# podemos automatizar el proceso de implementar un árbol binario perfectamente equilibrado diseñando una clase *CArbolBinE<T>* (Clase *Arbol Binario Equilibrado*) que proporcione los atributos y métodos necesarios para crear cada nodo del árbol, así como para permitir el acceso a los mismos.

Clase *CArbolBinE<T>*

La clase *CArbolBinE<T>* que vamos a implementar incluirá un atributo privado *raíz* para referenciar la raíz del árbol. El atributo *raíz* valdrá **null** cuando el árbol esté vacío. Así mismo, incluye la clase anidada *CNodo* que define la estructura de los nodos, y los métodos indicados en la tabla siguiente:

Método	Significado
<i>CArbolBinE</i>	Es el constructor; como es igual que el constructor por omisión, podría omitirse. Crea un árbol vacío (<i>raíz</i> a null).
<i>ConstruirArbol</i>	Es un método privado que permite construir un árbol binario perfectamente equilibrado. Tiene un parámetro de tipo int que se corresponde con el número de nodos que va a tener el árbol. Devuelve una referencia a la raíz del árbol.
<i>ConstruirArbolEquilibrado</i>	Invoca al método <i>ConstruirArbol</i> pasando como argumento el número de nodos y almacena el valor devuelto por él, en el atributo <i>raíz</i> de la clase (si al usuario se le permitiera invocar a <i>ConstruirArbol</i> tendría que tener acceso al atributo <i>raíz</i> devuelto por este método, de aquí esta solución). No devuelve nada.
<i>Buscar</i>	Busca un nodo determinado en el árbol. Hay dos formas públicas de este método que permitirán invocarlo con uno o con dos argumentos. El primer parámetro hace referencia a los datos del nodo a localizar en el árbol y el segundo, opcional, es una referencia a un entero que especifica, inicialmente, la posición del nodo donde se iniciará la búsqueda, y finalmente, la posición del nodo encontrado (consideramos que la primera posición es la 0 referida al orden de acceso según la forma <i>inorden</i>). Devuelve una referencia al área de datos del nodo, o bien null si el árbol está vacío o no existe un nodo con esos datos.
<i>Inorden</i>	Recorre un árbol binario utilizando la forma <i>inorden</i> . Tiene dos parámetros: el primero especifica la referencia al nodo

	a partir del cual se realizará la visita; el valor del primer parámetro sólo será tenido en cuenta si el segundo es false , porque si es true (valor por omisión) se asume que el primer parámetro es la raíz del árbol. No devuelve ningún valor.
<i>LeerDatos</i>	Método abstracto que debe ser redefinido por el usuario en una clase derivada para que permita leer los datos a los que hace referencia un nodo del árbol. Devuelve el objeto de datos. Es invocado por el método <i>ConstruirArbol</i> .
<i>Comparar</i>	Método abstracto que debe ser redefinido por el usuario en una clase derivada para especificar el tipo de comparación que se desea realizar con dos nodos del árbol. Debe devolver un entero indicando el resultado de la comparación (−1, 0 o 1 si <i>nodo1</i> < <i>nodo2</i> , <i>nodo1</i> == <i>nodo2</i> , o <i>nodo1</i> > <i>nodo2</i> , respectivamente). Este método es invocado por los métodos <i>Insertar</i> , <i>Borrar</i> y <i>Buscar</i> .
<i>Proceso</i>	Método abstracto que debe ser redefinido por el usuario en una clase derivada para especificar las operaciones que se desean realizar con el nodo visitado. Es invocado por el método <i>Inorden</i> .

A continuación se presenta el código correspondiente a la definición de la clase *CArbolBinE*:

```

////////////////////////////////////
// Clase abstracta: árbol binario perfectamente equilibrado.
// Para utilizar los métodos proporcionados por esta clase,
// tendremos que crear una clase derivada de ella y redefinir
// los métodos: LeerDatos, Comparar y Procesar.
//
public abstract class CArbolBinE<T>
{
    // Atributos del árbol binario
    private CNode raíz = null; // raíz del árbol

    // Nodo de un árbol binario
    private class CNode
    {
        // Atributos
        public T datos;           // referencia a los datos
        public CNode izquierdo;  // raíz del subárbol izquierdo
        public CNode derecho;    // raíz del subárbol derecho

        // Métodos
        public CNode() {}        // constructor
    }
}

```

```
// Métodos del árbol binario
public CArbolBinE() {}           // constructor

// El método siguiente debe ser redefinido en la clase derivada
// para que permita leer los datos que serán referenciados por
// un nodo del árbol. Devuelve el objeto de datos.
public abstract T LeerDatos();

// El método siguiente debe ser redefinido en una clase derivada
// para que permita comparar dos nodos del árbol por el atributo
// que necesitemos en cada momento.
public abstract int Comparar(T obj1, T obj2);

// El método siguiente debe ser redefinido en la clase derivada
// para que permita especificar las operaciones que se deseen
// realizar con el nodo visitado.
public abstract void Procesar(T obj);

private CNode ConstruirArbol(int n)
{
    // Construye un árbol de n nodos perfectamente equilibrado
    CNode nodo = null;
    int ni = 0, nd = 0;

    if ( n == 0 )
        return null;
    else
    {
        ni = n / 2;          // nodos del subárbol izquierdo
        nd = n - ni - 1;    // nodos del subárbol derecho
        nodo = new CNode();
        // Construcción inorden: da lugar a un árbol de búsqueda
        // cuando los datos se introduzcan ordenados ascendentemente
        nodo.izquierdo = ConstruirArbol(ni);
        nodo.datos = LeerDatos();
        nodo.derecho = ConstruirArbol(nd);
        return nodo;
    }
}

public void ConstruirArbolEquilibrado(int n)
{
    raíz = ConstruirArbol(n);
}

public object Buscar(T obj)
{
    int pos = 0;
    return Buscar(obj, ref pos);
}
```



```

public object Buscar(T obj, ref int posición)
{
    object datos = null;
    int pos_ini = 0;
    Buscar(obj, raíz, ref datos, ref posición, ref pos_ini);
    return (T)datos;
}

private void Buscar(T obj, CNode r, ref object datos,
                    ref int pos, ref int i)
{
    // El método Buscar permite acceder a un determinado nodo.
    // Si los datos especificados por "obj" se localizan en el
    // árbol referenciado por "r" a partir de la posición "pos",
    // "Buscar" devuelve en "datos" la referencia a esos datos;
    // en otro caso, devuelve null (valor inicial).
    // Los nodos se consideran numerados (0, 1, 2,...) según
    // el orden en el que son accedidos por el método "Inorden".
    CNode actual = r;

    if ( actual != null && datos == null )
    {
        Buscar(obj, actual.izquierdo, ref datos, ref pos, ref i);

        if (datos == null && pos-- <= 0)
            // La primera condición que aparece en el if anterior es
            // necesaria para que una vez encontrado el nodo no se
            // decremente "pos" en el camino de retorno por la pila de
            // llamadas.
            if ( Comparar( obj, actual.datos ) == 0 )
            {
                datos = actual.datos; // nodo encontrado
                pos = i;
            }
            i++; // posición del siguiente nodo que será accedido
            Buscar(obj, actual.derecho, ref datos, ref pos, ref i);
    }
}

private void Inorden(CNode r, bool nodoRaíz)
{
    // El método recursivo Inorden visita los nodos del árbol
    // utilizando la forma inorden; esto es, primero se visita
    // el subárbol izquierdo, después se visita la raíz, y por
    // último, el subárbol derecho.
    // Si el segundo argumento es true, la visita comienza
    // en la raíz independientemente del primer argumento.
    // "i" es la posición del nodo en proceso.
    // "pos" devuelve la posición del nodo encontrado.
    CNode actual = null;

```

```
        if ( nodoRaíz )
            actual = raíz; // partir de la raíz
        else
            actual = r;    // partir de un nodo cualquiera
        if ( actual != null )
        {
            Inorden( actual.izquierdo, false ); // visitar subárbol izq.
            // Procesar los datos del nodo visitado
            Procesar( actual.datos );
            Inorden( actual.derecho, false ); // visitar subárbol dcho.
        }
    }

    public void Inorden()
    {
        // Si el segundo argumento es true, la visita comienza
        // en la raíz independientemente del primer argumento.
        Inorden(null, true);
    }
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

El proceso de construcción lo lleva a cabo el método privado recursivo denominado *ConstruirArbol*, el cual construye un árbol de n nodos (este método es invocado por el método público *ConstruirArbolEquilibrado*).

El método *ConstruirArbol* tiene un parámetro entero que se corresponde con el número de nodos del árbol y devuelve una referencia al nodo raíz del árbol construido. En realidad diremos que devuelve una referencia a cada subárbol construido, lo que permite realizar los enlaces entre nodos. Observe que para cada nodo se ejecutan las dos sentencias siguientes:

```
nodo.izquierdo = ConstruirArbol(ni);
nodo.derecho = ConstruirArbol(nd);
```

que asignan a los atributos *izquierdo* y *derecho* de cada nodo las referencias a sus subárboles izquierdo y derecho, respectivamente.

El método *Buscar* permite acceder a unos datos determinados, recorriendo el árbol desde el nodo raíz y comenzando la búsqueda desde cualquier nodo. El árbol se recorre en la forma *inorden*. Para facilitar la labor del usuario de la clase, se han implementado tres sobrecargas de este método: dos públicas con uno y dos parámetros respectivamente, y otra privada con cinco. Los métodos públicos invocan al privado asegurando de esta manera que la referencia a los datos buscados, tercer parámetro de éste último, valga inicialmente **null**, valor que será devuelto si no se encuentra el nodo. El cuarto parámetro indica la posición del nodo a partir del cual se quiere realizar la búsqueda (independientemente de que

el árbol se empieza a recorrer desde el nodo raíz); de esta forma se puede buscar un nodo aunque su clave de búsqueda esté repetida. Y el quinto parámetro permite almacenar la posición del nodo actual; de esta forma se puede devolver la posición del nodo encontrado.

```
void buscar(T obj, CNode r, ref object datos, ref int pos, ref int i)
```

Este método es recursivo. Analicemos sus tres últimos parámetros para ver por qué son pasados por referencia: el tercero para poder devolver al método llamante una referencia al área de datos, el cuarto para poder devolver la posición del nodo donde están esos datos, y el quinto, para que todas las ejecuciones de este método en curso puedan actualizar la posición del nodo actual en proceso. Esto es, este último parámetro se ha definido así para disponer de una región de almacenamiento que pueda ser compartida por las distintas ejecuciones en curso de *Buscar* (piense que es un método recursivo y que cada llamada a sí mismo genera una nueva ejecución de dicho método). Con una variable local al método no podríamos evidentemente realizar la función que tiene *i* (por ser local al método, su valor no podría ser alterado por cualquiera de las ejecuciones en curso); una alternativa sería utilizar un miembro **static**, con la precaución de ponerlo a 0 al finalizar una búsqueda y antes de iniciar la siguiente.

Utilización de la clase **CArbolBinE<T>**

La clase *CArbolBinE<T>* es una clase abstracta; por lo tanto, para hacer uso del soporte que proporciona para la construcción y manipulación de árboles binarios perfectamente equilibrados, tendremos que derivar una clase de ella y redefinir los métodos abstractos heredados: *LeerDatos*, *Comparar* y *Procesar*. La redefinición de estos métodos está condicionada a la clase de objetos que formarán parte del árbol. Como ejemplo, vamos a construir un árbol binario perfectamente equilibrado en el que cada nodo haga referencia a un objeto de la clase *CDatos* ya utilizada anteriormente en este mismo capítulo.

El método *LeerDatos* obtendrá los datos *nombre* y *nota*, a partir de ellos construirá un objeto *CDatos* y devolverá el objeto construido para su inserción en el árbol. Los métodos *Comparar* y *Procesar* se definen igual que en la clase *CArbolBinarioDeBusqueda*.

Según lo expuesto, la clase *CArbolBinarioEquilibrado* derivada de *CArbolBinE<T>* puede ser de la forma siguiente:

```
using System;
using MisClases.ES;
```

```
////////////////////////////////////
```

```
// Clase derivada de la clase abstracta CarbolBinE<T>. Redefine
// los métodos: LeerDatos, Comparar y Procesar.
//
public class CarbolBinarioEquilibrado : CarbolBinE<CDatos>
{
    // Leer los datos que serán referenciados por un nodo del árbol.
    public override CDatos LeerDatos()
    {
        string nombre;
        double nota = 0.0;
        Console.Write("nombre: "); nombre = Console.ReadLine();
        Console.Write("nota:  "); Leer<double>.Dato(ref nota);
        return new CDatos(nombre, nota);
    }

    // Permite comparar dos nodos del árbol por el atributo nombre.
    public override int Comparar(CDatos obj1, CDatos obj2)
    {
        string str1 = obj1.Nombre;
        string str2 = obj2.Nombre;
        return str1.CompareTo(str2);
    }

    // Permite mostrar los datos del nodo visitado.
    public override void Procesar(CDatos obj)
    {
        string nombre = obj.Nombre;
        double nota = obj.Nota;
        Console.WriteLine(nombre + " " + nota);
    }
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

Cuando se declare un objeto de la clase *CarbolBinarioEquilibrado*, el constructor de esta clase invoca al constructor *CarbolBinE<T>* de su clase base, que creará un árbol vacío (*raíz* = **null**). El atributo *raíz* apunta siempre a la raíz del árbol.

Finalmente, escribiremos una aplicación *Test* que, utilizando la clase *CarbolBinarioEquilibrado*, cree un objeto *arbolbe* correspondiente a un árbol binario perfectamente equilibrado en el que cada nodo haga referencia a un objeto *CDatos*. De forma resumida la aplicación *Test*:

1. Creará un objeto *arbolbe* de la clase *CarbolBinarioEquilibrado*.
2. Construirá el árbol equilibrado de *n* nodos, enviando al objeto *arbolbe* el mensaje *ConstruirArbolEquilibrado*.
3. Buscará un determinado nodo enviando al objeto *arbolbe* el mensaje *Buscar*.

4. Finalmente, mostrará los datos almacenados en el árbol para comprobar que todo ha sucedido como esperábamos.

```
using System;
using MisClases.ES;

////////////////////////////////////
// Crear un árbol binario perfectamente equilibrado de n nodos
//
public class Test
{
    public static void Main(string[] args)
    {
        CArbolBinarioEquilibrado arbolbe = new CArbolBinarioEquilibrado();
        int númeroDeNodos = 0, pos = 0;
        Console.Write("Número de nodos: ");
        Leer<int>.Dato(ref númeroDeNodos);
        arbolbe.ConstruirArbolEquilibrado(númeroDeNodos);
        Console.WriteLine();

        // Buscar todas las ocurrencias de nombre
        string nombre;
        Console.Write("nombre a buscar: ");
        nombre = Console.ReadLine();
        CDatos objBuscar = new CDatos(nombre, 0);
        // Buscar la primera ocurrencia
        CDatos obj = (CDatos)arbolbe.Buscar(objBuscar, ref pos);
        if ( obj == null ) Console.WriteLine("La búsqueda falló");
        while (obj != null)
        {
            Console.WriteLine(pos + ".- " + obj.Nombre + " " + obj.Nota);
            // Buscar más ocurrencias con el mismo nombre
            pos++;
            obj = (CDatos)arbolbe.Buscar(objBuscar, ref pos);
        }

        // Mostrar los nodos del árbol
        Console.WriteLine("\nArbol:");
        arbolbe.Inorden();
    }
}
```

EJERCICIOS RESUELTOS

1. Realizar una aplicación que permita crear una lista lineal de elementos de cualquier tipo clasificados ascendentemente. La lista vendrá definida por un objeto de una clase abstracta que denominaremos *CListaLinealSEO* (Clase *Lista Lineal Simplemente Enlazada Ordenada*) y cada elemento de la lista será un objeto de la clase siguiente:

```
private class CElemento
{
    // Atributos
    public T datos;
    public CElemento siguiente; // siguiente elemento

    // Métodos
    // ...
}
```

La clase *CListaLinealSEO* debe incluir los atributos:

```
private CElemento p = null;           // elemento de cabecera
private CElemento elemAnterior = null; // elemento anterior
private CElemento elemActual = null;  // elemento actual
```

El atributo *elemActual* de la lista hará referencia al último elemento accedido y *elemAnterior* al anterior al actual, excepto cuando el elemento actual sea el primero, en cuyo caso ambas referencias señalarán a ese elemento. También incluirá los métodos:

```
public bool ListaVacía()
public abstract int Comparar(T obj1, T obj2)
public bool Buscar(T obj)
public void Añadir(T obj)
public object Borrar(T obj)
public object BorrarPrimero()
public object ObtenerPrimero()
public object ObtenerActual()
public object ObtenerSiguiente()
```

Todos los métodos expuestos, excepto *ListaVacía*, deben actualizar las referencias *elemActual* y *elemAnterior*.

El método *Comparar* debe ser redefinido por el usuario en una clase derivada para especificar el tipo de comparación que se desea realizar con dos elementos de la lista. Según esto, debe devolver un entero indicando el resultado de la comparación (-1 , 0 o 1 si $obj1 < obj2$, $obj1 == obj2$, o $obj1 > obj2$, respectivamente). Este método es invocado directamente por el método *Buscar* e indirectamente por los métodos *Añadir* y *Borrar*, que invocan a *Buscar*.

El método *ListaVacía* devuelve **true** si la lista está vacía y **false** en caso contrario.

El método *Buscar* localiza el punto de inserción de un elemento en una lista ordenada y almacena en *elemActual* una referencia al elemento buscado, si existe, o al siguiente, si no existe, y en *elemAnterior* una referencia al elemento anterior.

Tiene un parámetro para almacenar una referencia a los datos de tipo *T* que permitirán localizar el elemento en la lista, y devuelve un valor **true** o **false** dependiendo del resultado de la búsqueda.

El método *Añadir* inserta un elemento en la lista en orden ascendente según una clave seleccionada del área de datos. Para localizar el punto de inserción invocará a *Buscar*. La inserción se hará entre los elementos *elemAnterior* y *elemActual*. Tiene un parámetro que es una referencia de tipo *T* a los datos a añadir. No devuelve nada.

El método *Borrar* borra un elemento de la lista. Para buscarlo invoca al método *Buscar*. Tiene un parámetro para almacenar una referencia a los datos de tipo *T* que permitirán localizar en la lista el elemento que se desea borrar. Si la operación se realiza satisfactoriamente, el método retorna una referencia al área de datos del elemento borrado, o bien **null** si la lista está vacía.

El método *BorrarPrimero* borra el elemento primero de la lista. Devuelve una referencia al área de datos del elemento borrado, o bien **null** si la lista está vacía.

El método *ObtenerPrimero* devuelve una referencia al área de datos del elemento primero y *ObtenerActual* una referencia al área de datos del elemento actual. Ambos devuelven **null** si la lista está vacía.

El método *ObtenerSiguiente* devuelve una referencia al área de datos del elemento siguiente al actual, o bien **null** si la lista está vacía o se intenta ir más allá del último elemento.

Según el enunciado, la clase *CListaLinealSEO* puede ser como se muestra a continuación:

```

////////////////////////////////////
// Clase abstracta CListaLinealSEO<T>:
//   Lista lineal simplemente enlazada ordenada ascendentemente.
//
public abstract class CListaLinealSEO<T>
{
    // p: referencia al primer elemento de la lista.

    private CElemento p = null;           // elemento de cabecera
    private CElemento elemAnterior = null; // elemento anterior
    private CElemento elemActual = null;  // elemento actual

    // Elemento de una lista lineal simplemente enlazada
    private class CElemento
    {
        // Atributos
    }
}

```

```
public T datos;
public CElemento siguiente; // siguiente elemento

// Métodos
public CElemento() {} // constructor

public CElemento(T d, CElemento s) // constructor
{
    datos = d;
    siguiente = s;
}

public CListaLinealSE0() {} // constructor

// El método siguiente debe ser redefinido en una subclase para
// que permita comparar dos elementos de la lista por el atributo
// que necesitamos en cada momento.
public abstract int Comparar(T obj1, T obj2);

public bool ListaVacía()
{
    return p == null;
}

public bool Buscar(T obj)
{
    // Buscar el punto de inserción de un elemento en una lista
    // ordenada. El método almacena en elemActual la referencia
    // del elemento buscado si existe, o del siguiente si no existe
    // y en elemAnterior la referencia del elemento anterior.
    int r = 0;
    elemAnterior = elemActual = null;

    // Si la lista referenciada por p está vacía, retornar.
    if ( ListaVacía() ) return false;

    // Si la lista no está vacía, encontrar el elemento.
    elemAnterior = p;
    elemActual = p;
    // Posicionarse en el elemento buscado.
    while (elemActual != null &&
           (r = Comparar(obj, elemActual.datos)) > 0)
    {
        elemAnterior = elemActual;
        elemActual = elemActual.siguiente;
    }
    return r == 0; // devuelve: true = encontrado, o
                  // false = no encontrado
}
```



```

public void Añadir(T obj)
{
    // Añadir un elemento en orden ascendente según una clave
    // proporcionada por obj.
    CElemento q = new CElemento(obj, null); // crear el elemento

    // Si la lista referenciada por p está vacía, añadirlo sin más
    if ( ListaVacía() )
    {
        // Añadir el primer elemento
        p = q;
        elemAnterior = elemActual = p; // actualizar referencias
        return;
    }

    // Si la lista no está vacía, encontrar el punto de inserción
    Buscar(obj); // establece los valores de elemAnterior y elemActual

    // Dos casos:
    // 1) Insertar al principio de la lista
    // 2) Insertar después del anterior (incluye insertar al final)
    if ( elemAnterior == elemActual ) // insertar al principio
    {
        q.siguiente = p;
        p = q; // cabecera
        elemAnterior = elemActual = p; // actualizar referencias
    }
    else // insertar después del anterior
    {
        q.siguiente = elemActual;
        elemAnterior.siguiente = q;
        elemActual = q; // actualizar referencia
    }
}

public object Borrar(T obj)
{
    // Borrar un determinado elemento.
    // Si la lista está vacía, retornar.
    if ( ListaVacía() ) return null;

    // Si la lista no está vacía, buscar el elemento. El método
    // Buscar establece los valores de elemAnterior y elemActual
    if (Buscar(obj) == false) return null; // no está

    // Dos casos:
    // 1) Borrar el primer elemento de la lista
    // 2) Borrar el siguiente a elemAnterior (elemActual)
    if ( elemActual == p ) // 1)
        p = p.siguiente; // cabecera
    else // 2)

```

```
        elemAnterior.siguiente = elemActual.siguiente;

        T borrado = elemActual.datos;
        elemActual = elemActual.siguiente; // actualizar referencia
        return borrado; // retornar el elemento borrado.
        // El elemento referenciado por borrado será enviado a la
        // basura al quedar desreferenciado, por tratarse de una
        // variable local.
    }

    public object BorrarPrimero()
    {
        // Si la lista está vacía, retornar.
        if ( ListaVacía() ) return null;
        // Borrar el primer elemento de la lista
        elemActual = p;
        p = p.siguiente; // cabecera
        T borrado = elemActual.datos;
        // Actualizar punteros
        elemAnterior = elemActual = p;
        return borrado;
    }

    public object ObtenerPrimero()
    {
        // Devolver una referencia a los datos del primer elemento.
        // Si la lista está vacía, devolver null.
        if ( ListaVacía() ) return null;
        elemActual = elemAnterior = p;
        return p.datos;
    }

    public object ObtenerActual()
    {
        // Devolver una referencia a los datos del elemento actual.
        // Si la lista está vacía, devolver null.
        if ( ListaVacía() || elemActual == null ) return null;
        return elemActual.datos;
    }

    public object ObtenerSiguiente()
    {
        // Devolver una referencia a los datos del elemento siguiente
        // al actual y hacer que éste sea el actual. Si la lista
        // está vacía o se intenta ir más allá del último,
        // devolver null.
        if ( ListaVacía() || elemActual == null ) return null;
        // Avanzar un elemento
        elemAnterior = elemActual;
        elemActual = elemActual.siguiente;
        if ( elemActual != null )
```



```
        obj = (CDatos)lse.ObtenerSiguiente();
    }
}

public static void Main(string[] args)
{
    // Crear una lista lineal vacía
    CListaLinealSEOrdenada lse = new CListaLinealSEOrdenada();

    // Leer datos y añadirlos a la lista
    CDatos obj;
    string nombre;
    double nota = 0;
    Console.WriteLine("Introducir datos. Finalizar con Ctrl+Z.");
    Console.Write("nombre: ");
    while ((nombre = Console.ReadLine()) != null)
    {
        Console.Write("nota: ");
        Leer<double>.Dato(ref nota);
        lse.Añadir(new CDatos(nombre, nota));
        Console.Write("nombre: ");
    }
    Console.WriteLine("\n");

    // Borrar un elemento determinado
    Console.Write("nombre del alumno a borrar: ");
    nombre = Console.ReadLine();
    obj = (CDatos)lse.Borrar(new CDatos(nombre, 0));
    if (obj == null)
        Console.WriteLine("Error: elemento no borrado");

    // Obtener el siguiente
    obj = (CDatos)lse.ObtenerSiguiente();
    if (obj != null)
        Console.WriteLine("Nombre: " + obj.Nombre +
                           ", nota: " + obj.Nota);

    // Mostrar todos
    Console.WriteLine("Lista:");
    MostrarLista(lse);
}
}
```

2. Escribir una aplicación para que, utilizando una pila, simule una calculadora capaz de realizar las operaciones de +, −, * y /. La mayoría de las calculadoras aceptan la notación *infija* y unas pocas la notación *postfija*. En éstas últimas, para sumar 10 y 20 introduciríamos primero 10, después 20 y por último el +. Cuando se introducen los operandos, se colocan en una pila y cuando se introduce el operador, se sacan dos operandos de la pila, se calcula el resultado y se introduce en

la pila. La ventaja de la notación postfija es que expresiones complejas pueden evaluarse fácilmente sin mucho código. La calculadora del ejemplo propuesto utilizará la notación *postfija*.

De forma resumida, el programa realizará las siguientes operaciones:

- a) Leerá un dato, operando u operador, y lo almacenará en la variable *oper*.
- b) Analizará *oper*; si se trata de un operando lo mete en la pila y si se trata de un operador saca los dos últimos operandos de la pila, realiza la operación indicada por dicho operador y mete el resultado en la pila para poder utilizarlo como operando en una posible siguiente operación.

Para realizar esta aplicación utilizaremos las clases *CPila<T>* derivada de *CListaCircularSE<T>*, *CDatos* y *Leer<T>*. Como estas clases ya han sido implementadas, en este ejercicio nos limitaremos a utilizar los recursos que proporcionan.

El programa completo se muestra a continuación:

```
using System;
using MisClases.ES;
////////////////////////////////////
// Calculadora utilizando una pila. Esta aplicación, además de las
// clases necesarias de la biblioteca de .NET, utiliza las clases:
// CPila<T> derivada de CListaCircularSE<T> y Leer<T>.
//
public class Test
{
    private static CPila<Double> pila = new CPila<Double>();
    private static double[] operando = { 0, 0 }; // operando 0 y 1

    public static void ObtenerOperandos()
    {
        operando[1] = pila.SacarDePila();
        operando[0] = pila.SacarDePila();
    }

    public static void Main(string[] args)
    {
        // oper almacena la entrada realizada desde el teclado
        string oper = null;

        Console.WriteLine("Operaciones: + - * /\n");
        Console.WriteLine("Forma de introducir los datos:");
        Console.WriteLine(">primer operando [Entrar]");
        Console.WriteLine(">segundo operando [Entrar]");
        Console.WriteLine(">operador [Entrar]\n");
        Console.WriteLine("Para salir pulse q\n");
```

```
do
{
    try
    {
        Console.Write("> ");
        oper = Console.ReadLine(); // leer un operando o un operador
        switch (oper[0]) // verificar el primer carácter
        {
            case '+':
                ObtenerOperandos();
                Console.WriteLine(operando[0] + operando[1]);
                pila.MeterEnPila(operando[0] + operando[1]);
                break;
            case '-':
                ObtenerOperandos();
                Console.WriteLine(operando[0] - operando[1]);
                pila.MeterEnPila(operando[0] - operando[1]);
                break;
            case '*':
                ObtenerOperandos();
                Console.WriteLine(operando[0] * operando[1]);
                pila.MeterEnPila(operando[0] * operando[1]);
                break;
            case '/':
                ObtenerOperandos();
                if (operando[1] == 0)
                {
                    Console.WriteLine("\nError: división por cero");
                    break;
                }
                Console.WriteLine(operando[0] / operando[1]);
                pila.MeterEnPila(operando[0] / operando[1]);
                break;
            case 'q':
                // salir
                break;
            default: // es un operando
                pila.MeterEnPila(Double.Parse(oper));
                break;
        }
    }
}
catch (FormatException)
{
    Console.Write("Error: dato no es válido. Teclee otro: ");
}
catch (Exception)
{
    Console.Write("Error. La operación no es válida.");
    if (oper.Length == 0) oper = " ";
}
}
```

```

        while (oper[0] != 'q');
    }
}

```

3. Escribir una aplicación que permita calcular la frecuencia con la que aparecen las palabras en un fichero de texto. La forma de invocar al programa será:

```
palabras fichero_de_texto
```

donde *fichero_de_texto* es el nombre del fichero de texto del cual deseamos obtener la estadística.

El proceso de contabilizar las palabras que aparezcan en el texto de un determinado fichero lo podemos realizar de la forma siguiente:

- a) Se lee la información del fichero y se descompone en palabras, entendiendo por palabra una secuencia de caracteres delimitada por espacios en blanco, tabuladores, signos de puntuación, etc.
- b) Cada palabra deberá insertarse por orden alfabético ascendente junto con un contador que indique su número de apariciones, en el nodo de una estructura en árbol. Esto facilitará la búsqueda.
- c) Una vez construido el árbol de búsqueda, se presentará por pantalla una estadística con el siguiente formato:

...

```

Obtener = 1
palabras = 1
permita = 1
programa = 1
que = 2
queremos = 1
será = 1
estadística = 1
texto = 2
un = 1
una = 1

```

```

Total palabras: 44
Total palabras diferentes: 35

```

Según lo expuesto, cada nodo del árbol tendrá que hacer referencia a un área de datos que incluya tanto la palabra como el número de veces que apareció en el texto. Estos datos serán los atributos de una clase *CDatos* definida así:

```
using System;

public class CDatos
{
    // Atributos
    private string palabra;
    private int contador;

    // Métodos
    public CDatos() {}           // constructor sin parámetros

    public CDatos(string pal) // constructor con un parámetro
    {
        palabra = pal;
        contador = 0;
    }

    public CDatos(string pal, int cont) // constructor con dos pars.
    {
        palabra = pal;
        contador = cont;
    }

    public string Palabra
    {
        get { return palabra; }
        set { palabra = value; }
    }

    public int Contador
    {
        get { return contador; }
        set { contador = value; }
    }
}
```

El árbol de búsqueda que tenemos que construir será un objeto de la clase *CarbolBinarioDeBusqueda* derivada de *CarbolBinB<T>*. Recuerde que la clase *CarbolBinB<T>* fue implementada anteriormente en este mismo capítulo, al hablar de árboles binarios de búsqueda. La razón de por qué derivamos una clase de *CarbolBinB<T>* es porque esta clase es abstracta, y se diseñó así para obligar al usuario a redefinir sus métodos abstractos *Comparar*, que deberá devolver un valor -1 , 0 o 1 especificando la relación de orden que existe entre dos nodos del árbol, y *Procesar*, para especificar las operaciones que se deseen realizar con el nodo visitado.

```
using System;
```

```
////////////////////////////////////
```



```
// Clase derivada de la clase abstracta CARbolBinB<T>. Redefine
// los métodos: Comparar y Procesar.
//
public class CARbolBinarioDeBusqueda : CARbolBinB<CDatos>
{
    // Permite comparar dos nodos del árbol por el atributo
    // nombre.
    public int totalPalabras = 0;
    public int totalPalabrasDiferentes = 0;

    // Permite comparar dos nodos del árbol por el atributo
    // palabra.
    public override int Comparar(CDatos obj1, CDatos obj2)
    {
        String str1 = obj1.Palabra;
        String str2 = obj2.Palabra;
        return str1.CompareTo(str2);
    }

    // Permite mostrar los datos del nodo visitado.
    public override void Procesar(CDatos obj)
    {
        string palabra = obj.Palabra;
        int contador = obj.Contador;
        Console.WriteLine(palabra + " = " + contador);
        totalPalabras += contador;
        totalPalabrasDiferentes++;
    }
}
////////////////////////////////////
```

Se puede observar que el método *Procesar* de la clase *CARbolBinarioDeBusqueda*, además de visualizar la información almacenada en el objeto *CDatos* referenciado por el nodo visitado, contabiliza el número total de palabras del texto procesado y el número total de palabras diferentes (esto es, como si todas hubieran aparecido sólo una vez en el texto). El resto de los métodos ya fueron explicados al hablar de árboles binarios de búsqueda.

Sólo queda construir una aplicación que cree un objeto de la clase *CARbolBinarioDeBusqueda* a partir de las palabras almacenadas en un fichero y presente los resultados pedidos. El código de esta aplicación se va a apoyar en tres métodos: **Main**, *LeerFichero* y *SepararPalabras*.

El método **Main** verificará, cuando se ejecute la aplicación, que se haya pasado como parámetro el nombre del fichero de texto, invocará al método *LeerFichero* y una vez construido el árbol, lo recorrerá para visualizar los resultados pedidos.

El método *LeerFichero* abre el fichero y lo lee línea a línea. Cada línea leída será pasada como argumento al método *SepararPalabras* para su descomposición en palabras con el fin de añadirlas al árbol binario de búsqueda que ha sido declarado como un atributo de la clase aplicación. Para descomponer una línea en palabras utilizaremos el método **Split** proporcionado por la clase **String** del espacio de nombres **System** de .NET (esta clase fue explicada en el capítulo 8).

El código completo de la aplicación que hemos denominado *Palabras* se muestra a continuación:

```
using System;
using System.IO;

////////////////////////////////////
// Utilizar un árbol de búsqueda para obtener la frecuencia con la
// que aparecen las palabras en un fichero de texto.
// Esta aplicación, además de las clases necesarias de la
// biblioteca de .NET, utiliza las clases: CArbolBinarioDeBusqueda
// derivada de CArbolBinB<T> y CDatos.
//
public class Palabras
{
    private static CArbolBinarioDeBusqueda arbolbb =
        new CArbolBinarioDeBusqueda();

    public static void SepararPalabras(string línea)
    {
        // Descomponer línea en palabras
        char[] seps = { ' ', ',', ';', ':', '\n', '\r', '\t', '\f' };
        string[] cadena = línea.Split(seps);
        CDatos obj;
        for (int i = 0; i < cadena.Length; ++i )
        {
            if (cadena[i].Length == 0) continue;
            if ((obj = (CDatos)arbolbb.Buscar(new CDatos(cadena[i]))) == null)
                arbolbb.Insertar(new CDatos(cadena[i], 1));
            else
                obj.Contador = obj.Contador + 1;
        }
    }

    public static void LeerFichero(string nombrefich)
    {
        StreamReader sr = null;
        try
        {
            // Asegurarse de que el fichero, existe y se puede leer
            if (!File.Exists(nombrefich) || Directory.Exists(nombrefich))
            {
```

```

        Console.WriteLine("No existe el fichero " + nombrefich);
        return;
    }

    FileAttributes atributos = File.GetAttributes(nombrefich);
    if ((atributos & FileAttributes.ReadOnly) != 0)
    {
        Console.WriteLine("El fichero " + nombrefich +
                           " no se puede leer");
        return;
    }

    // Abrir un flujo de entrada desde el fichero fuente
    sr = new StreamReader(nombrefich);

    // Buscar cadena en el fichero fuente
    string línea;

    while ((línea = sr.ReadLine()) != null)
    {
        // Si se alcanzó el final del fichero,
        // ReadLine devuelve null
        SepararPalabras(línea);
    }
}
catch(IOException e)
{
    Console.WriteLine("Error: " + e.Message);
}
finally
{
    // Cerrar el flujo
    try
    {
        if (sr != null) sr.Close();
    }
    catch(IOException e)
    {
        Console.WriteLine("Error: " + e.Message);
    }
}
}

public static void Main(string[] args)
{
    // Main debe recibir un parámetro: el nombre del fichero.
    // Por ejemplo: palabras palabras.txt
    if (args.Length < 1)
        Console.WriteLine("Sintaxis: palabras <fichero_de_texto>");
    else
        LeerFichero(args[0]);
}

```

```
        arbolbb.Inorden();
        Console.WriteLine();
        Console.WriteLine("Total palabras: " + arbolbb.totalPalabras);
        Console.WriteLine("Total palabras diferentes: " +
                           arbolbb.totalPalabrasDiferentes);
    }
}
```

EJERCICIOS PROPUESTOS

1. Se quiere escribir un programa para manipular ecuaciones algebraicas o polinómicas dependientes de las variables x e y . Por ejemplo:

$$2x^3y - xy^3 + 8.25 \text{ más } 5x^5y - 2x^3y + 7x^2 - 3 \text{ igual a } 5x^5y + 7x^2 - xy^3 + 5.25$$

Cada término del polinomio será representado por un objeto de una clase *CTermino* y cada polinomio por un objeto que sea una lista lineal simplemente enlazada ordenada de elementos *CTermino*. La lista lineal simplemente enlazada ordenada fue implementada en el primer ejercicio del apartado *Ejercicios resueltos* de este capítulo (clase *CListaLinealSEOrdenada*).

Los términos de un polinomio estarán ordenados ascendentemente primero por x y después por y . El algoritmo que emplearemos será el siguiente: a cada unidad del exponente de x le damos un peso k y a cada unidad del exponente de y un peso de 1 ; la suma de ambas cantidades nos da el valor utilizado para efectuar la ordenación requerida. El valor de k será la potencia de 10 que sea igual o mayor que el mayor de los exponentes de x e y del término a insertar.

La clase *CTermino* ya fue desarrollada en el apartado *Ejercicios propuestos* del capítulo 11. También se implementó una clase *CPolinomio*. Esta clase debe ser ahora reemplazada por una lista lineal simplemente enlazada ordenada.

2. En un fichero tenemos almacenados los nombres y las notas medias de los alumnos de un determinado curso. La estructura de cada uno de los registros del fichero se corresponde con los atributos de una clase como la siguiente:

```
public class CRegistro
{
    // Atributos
    private String nombre;
    private float nota;

    // Métodos
    // ...
}
```

Queremos leer los datos de este fichero para construir una estructura de datos en memoria que se ajuste a un *árbol binario de búsqueda* perfectamente *equilibrado*. Para ello, es aconsejable ordenar el fichero antes de crear el árbol. Esto facilita la creación del árbol binario con los dos requisitos impuestos: que sea de búsqueda y perfectamente equilibrado. En este ejercicio supondremos que partimos de un fichero ordenado. Más adelante, en el capítulo *Algoritmos* veremos cómo se ordena un fichero.

Cuando se muestre la información almacenada en el árbol, el listado de los nombres y sus correspondientes notas debe aparecer en orden ascendente. Por definición de árbol de búsqueda, para todo nodo, las claves menores que la del propio nodo forman el subárbol izquierdo y las mayores, el subárbol derecho. Según esto, la clave menor se encuentra en el nodo más a la izquierda y la clave mayor en el nodo más a la derecha del árbol. Por lo tanto, para visualizar los nombres en orden ascendente tendremos que recorrer el árbol en inorden. Entonces, si pensamos en el proceso inverso, esto es, si partimos de un fichero con las claves ordenadas y construimos un árbol perfectamente equilibrado, tenemos que utilizar la forma inorden para conseguir al mismo tiempo que el árbol sea de búsqueda. Es decir, el método que cree el árbol debe incluir los siguientes procesos en el orden mostrado:

```
crear el subárbol izquierdo
leer un registro del fichero y asignárselo al nodo actual
crear el subárbol derecho
```

Vemos que primero hay que construir el subárbol izquierdo, después la raíz y por último el subárbol derecho. Como las claves contenidas en el fichero están ordenadas, la clave menor se almacenará en el nodo más a la izquierda y la mayor en el nodo más a la derecha, dando así lugar a un árbol de búsqueda, además de perfectamente equilibrado.

3. El filtro *sort* lee líneas de texto del fichero estándar de entrada y las presenta en orden alfabético en el fichero estándar de salida. El ejemplo siguiente muestra la forma de utilizar *sort*:

```
sort[Entrar]
lo que puede hacerse
en cualquier momento
no se hará
en ningún momento.
(eof)
en cualquier momento
en ningún momento.
lo que puede hacerse
no se hará
```

Se desea escribir un programa de nombre *Ordenar*, que actúe como el filtro *sort*. Para ordenar las distintas líneas vamos a ir insertándolas en un árbol binario de búsqueda, de tal forma que al recorrerlo podamos presentar las líneas en orden alfabético. El programa se ejecutará utilizando la siguiente sintaxis:

```
ordenar fichero_de_texto [-r]
```

Si se especifica el atributo opcional *-r*, las líneas del fichero serán presentadas en orden alfabético descendente; si no se especifica, entonces se presentarán en orden alfabético ascendente.

4. Cuando diseñamos la clase *CListaLinealSE* construimos, para probar su funcionamiento, una clase *Test* y una clase *CDatos*. Añadir a la clase *CDatos* el método **ToString** y a la clase *CListaLinealSE* un indizador.

CAPÍTULO 17

© F.J.Ceballos/RA-MA

ALGORITMOS

En este capítulo vamos a exponer cómo resolver problemas muy comunes en programación. El primero que nos vamos a plantear es la *recursión*; se trata de un problema cuyo planteamiento forma parte de su solución. El segundo problema que vamos a abordar es la *ordenación* de objetos en general; la ordenación es tan común que no necesita explicación; algo tan cotidiano como una guía telefónica es un ejemplo de una lista clasificada. El localizar un determinado teléfono exige una *búsqueda* por algún método; el problema de búsqueda será el último que resolveremos.

RECURSIVIDAD

Se dice que un proceso es recursivo si forma parte de sí mismo, o sea que se define en función de sí mismo. Ejemplos típicos de recursión los podemos encontrar frecuentemente en problemas matemáticos, en estructuras de datos y en muchos otros problemas.

La recursión es un proceso extremadamente potente, pero consume muchos recursos, razón por la que la analizaremos detenidamente, para saber cuándo y cómo aplicarla. De este análisis deduciremos que aunque un problema por definición sea recursivo, no siempre será el método de solución más adecuado.

En las aplicaciones prácticas, antes de poner en marcha un proceso recursivo es necesario demostrar que el nivel máximo de recursión, esto es, el número de veces que se va a llamar a sí mismo, es no sólo finito, sino realmente pequeño. La razón es que se necesita cierta cantidad de memoria para almacenar el estado del proceso cada vez que se abandona temporalmente, debido a una llamada para ejecutar otro proceso que es él mismo. El estado del proceso de cálculo en curso hay

que almacenarlo para recuperarlo cuando se acabe la nueva ejecución del proceso y haya que reanudar la antigua.

En términos de un lenguaje de programación, un método es recursivo cuando se llama a sí mismo.

Un ejemplo es el método de Ackerman, A , el cual está definido para todos los valores enteros no negativos m y n de la forma siguiente:

$$\begin{aligned} A(0, n) &= n+1 \\ A(m, 0) &= A(m-1, 1) && (m > 0) \\ A(m, n) &= A(m-1, A(m, n-1)) && (m, n > 0) \end{aligned}$$

El pseudocódigo que especifica cómo solucionar este problema aplicando la recursión es el siguiente:

```
<método A(m,n)>
  IF (m es igual a 0) THEN
    devolver como resultado n+1
  ELSE IF (n es igual a 0) THEN
    devolver como resultado A(m-1,1)
  ELSE
    devolver como resultado A(m-1,A(m,n-1))
  ENDIF
END <método A(m,n)>
```

A continuación presentamos este método como parte de una clase *CRecursion*:

```
public class CRecursion
{
    // Método recursivo de Ackerman:
    //   A(0,n) = n+1
    //   A(m,0) = A(m-1,1)                (m > 0)
    //   A(m,n) = A(m-1,A(m,n-1))        (m,n > 0)
    public static int Ackerman(int m, int n)
    {
        if (m == 0)
            return n+1;
        else if (n == 0)
            return Ackerman(m-1, 1);
        else
            return Ackerman(m-1, Ackerman(m,n-1));
    }
}
```

Para que una solución recursiva sea correcta se han de cumplir algunas reglas:

- Siempre ha de existir una, o varias, condiciones que permitan devolver un resultado sin necesidad de una nueva llamada recursiva. Es lo que denominamos solución trivial al problema. En el caso del ejemplo expuesto, cuando m sea igual a cero, el resultado inmediato será $n+1$ y no hay llamada recursiva.
- Cuando los valores de los parámetros del método no cumplen la condición que da lugar a la solución trivial, el método se llamará a sí mismo (llamada recursiva porque la solución no es trivial).
- En todas las llamadas recursivas, el valor de los parámetros pasados se ha de modificar de forma que los nuevos cálculos a partir de ellos produzcan una solución que se vaya aproximando al valor de solución trivial. En el caso del ejemplo expuesto, en la llamada recursiva, en uno de los casos, se utiliza $m-1$ en lugar de m ($m-1$ está más próximo al valor trivial 0 que m).
- La solución retornada en un caso no trivial ha de ser correcta en la hipótesis de que la llamada recursiva devuelve una solución correcta. Este concepto es la base del principio de inducción y la parte más importante en el diseño y verificación de un algoritmo recursivo. Para el caso del ejemplo expuesto, en la hipótesis de que las llamadas *Ackerman*($m-1, 1$) y *Ackerman*($m-1, \text{Ackerman}(m, n-1)$) devuelven una solución correcta, el resultado final es correcto puesto que se devuelve la solución esperada.

Repase las reglas anteriores aplicándoles al problema de $n!$ que se expuso en el capítulo 9, le resultará mucho más sencillo entenderlas.

Para probar cómo funciona este algoritmo expuesto anteriormente, podemos escribir la aplicación siguiente:

```
public class Test
{
    public static void Main(string[] args)
    {
        int m = 0, n = 0, a = 0;
        Console.WriteLine("Cálculo de A(m,n)=A(m-1,A(m,n-1))\n");
        Console.Write("Valor de m: "); Leer<int>.Dato(ref m);
        Console.Write("Valor de n: "); Leer<int>.Dato(ref n);
        a = CRecursion.Ackerman(m,n);
        Console.WriteLine("\nA(" + m + ", " + n + ") = " + a);
    }
}
```

Esta función crece extremadamente rápida. Este crecimiento desmesurado puede dar lugar a una recursividad muy profunda y producir un desbordamiento de la pila de ejecución (excepción **StackOverflowException**). Esta excepción no

se puede detectar mediante un bloque **try-catch**. Por consiguiente, si se quiere detectar y evitar un desbordamiento de pila, habrá que escribir código para ello; por ejemplo, utilizando un contador para terminar el bucle recursivo.

Supongamos ahora que nos planteamos el problema de resolver el método de *Ackerman*, pero sin aplicar la recursión. Esto nos exigirá salvar las variables necesarias del proceso en curso, cada vez que el método se llame a sí mismo, con el fin de poder reanudarlo cuando finalice el nuevo proceso invocado.

La mejor forma de hacer esto es utilizar una pila, con el fin de almacenar los valores *m* y *n* cada vez que se invoque el método para una nueva ejecución y tomar estos valores de la cima de la pila, cuando esta nueva ejecución finalice, con el fin de reanudar la antigua (clase *CPila<T>*; capítulo 16).

El pseudocódigo para este método puede ser el siguiente:

```
<método A(m,n)>
  Utilizar una pila para almacenar los valores de m y n
  Iniciar la pila con los valores m,n
  DO
    Tomar los datos de la parte superior de la pila
    IF (m es igual a 0) THEN
      Amn = n+1
      IF (pila no vacía)
        sacar de la pila los valores: m, n
        meter en la pila los valores: m, Amn
      ELSE
        devolver como resultado Amn
      ENDIF
    ELSE IF (n es igual a 0) THEN
      meter en la pila los valores: m-1,1
    ELSE
      meter en la pila los valores: m-1, Amn
      meter en la pila los valores: m,n-1
    ENDIF
  WHILE (true)
END <método A(m,n)>
```

A continuación presentamos el código correspondiente a este método que hemos denominado *AckermanNR*. Dicho método se ha incluido en la interfaz de la clase *CRecursion* anterior y utiliza la clase *CPila*, implementada en el capítulo de *Estructuras dinámicas*, para crear una pila que almacene los valores *m* y *n* cada vez que es invocado para una nueva ejecución.

```
public static int AckermanNR(int m, int n)
{
    CPila<CDatos> pila = new CPila<CDatos>(); // pila de elementos (m,n)
```

```

CDatos dato;
int Ackerman_m_n = 0;
pila.MeterEnPila(new CDatos(m, n));
while (true)
{
    // Tomar los datos de la cima de la pila
    dato = pila.SacarDePila();
    m = dato.M;
    n = dato.N;
    if (m == 0) // Ackerman(0,n) = n+1
    {
        Ackerman_m_n = n + 1;
        // Sacar m y n de la pila
        dato = pila.SacarDePila();
        if (dato != null)
        {
            m = dato.M;
            n = dato.N;
            // Meter m y Ackerman_m_n en la pila
            pila.MeterEnPila(new CDatos(m, Ackerman_m_n));
        }
        else
            return Ackerman_m_n;
    }
    else if (n == 0) // Ackerman(m-1,1)
        // Meter m-1 y 1 en la pila
        pila.MeterEnPila(new CDatos(m - 1, 1));
    else // Ackerman(m-1,Ackerman(m,n-1))
    {
        // Meter m-1 y Ackerman_m_n en la pila
        pila.MeterEnPila(new CDatos(m - 1, Ackerman_m_n));
        // Meter m y n-1 en la pila
        pila.MeterEnPila(new CDatos(m, n - 1));
    }
}
}

```

Según se puede observar, los valores de m y n son encapsulados por objetos de la clase *CDatos* definida así:

```

public class CDatos
{
    // Propiedades
    public int M { get; set; }
    public int N { get; set; }

    // Métodos
    public CDatos(int x, int y) // constructor con parámetros
    {
        M = x;

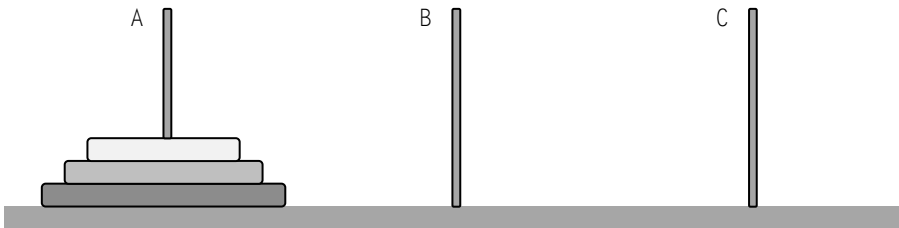
```

```

    N = y;
}
}

```

Un proceso en el que es realmente eficaz aplicar la recursión es el problema de las *torres de Hanoi*. Este problema consiste en tres barras verticales *A*, *B* y *C* y *n* discos, de diferentes tamaños, apilados inicialmente sobre la barra *A*, en orden de tamaño decreciente.



El objetivo es mover los discos desde la barra *A* a la *C*, conservando su orden, bajo las siguientes reglas:

1. Se moverá un solo disco cada vez.
2. Un disco no puede situarse sobre otro más pequeño.
3. Se utilizará la barra *B* como pila auxiliar.

Una posible solución es el algoritmo recursivo que se muestra a continuación:

1. Mover *n-1* discos de la barra *A* a la *B* (el disco *n* es el del fondo).
2. Mover el disco *n* de la barra *A* a la *C*, y
3. Mover los *n-1* discos de la barra *B* a la *C*.

Resumiendo estas condiciones en un cuadro obtenemos:

	<i>nº discos</i>	<i>origen</i>	<i>otra torre</i>	<i>destino</i>
<i>inicialmente</i>	<i>n</i>	<i>A</i>	<i>B</i>	<i>C</i>
<i>1</i>	<i>n-1</i>	<i>A</i>	<i>C</i>	<i>B</i>
<i>2</i>	<i>1</i>	<i>A</i>	<i>B</i>	<i>C</i>
<i>3</i>	<i>n-1</i>	<i>B</i>	<i>A</i>	<i>C</i>

El método a realizar será mover *n discos* de *origen* a *destino*:

```
Mover(n_discos, origen, otratorre, destino);
```

El pseudocódigo para este programa puede ser el siguiente:

```

<método mover(n_discos, A, B, C)>
  IF (n_discos es mayor que 0) THEN
    mover(n_discos-1, A, C, B)
    mover(disco_n, A, B, C)
    mover(n_discos-1, B, A, C)
  ENDIF
END <método mover>

```

A continuación presentamos el método correspondiente a este problema. El resultado será los movimientos realizados y el número total de movimientos.

```

public class CHanoi
{
    private static int movimientos = 0;

    public static int Mover(int n_discos, char a, char b, char c)
    {
        if (n_discos > 0)
        {
            Mover(n_discos-1, a, c, b);
            Console.WriteLine("mover disco de "+ a + " a " + c);
            movimientos++;
            Mover(n_discos-1, b, a, c);
        }
        return movimientos;
    }
}

```

Para probar cómo funciona este método escribimos la aplicación siguiente:

```

public class Test
{
    public static void Main(string[] args)
    {
        int n_discos = 0, movimientos = 0;
        Console.Write("Número de discos: ");
        Leer<int>.Dato(ref n_discos);
        movimientos = CHanoi.Mover(n_discos, 'A', 'B', 'C');
        Console.WriteLine("\nmovimientos efectuados: " + movimientos);
    }
}

```

Si ejecuta la aplicación anterior para $n_discos = 3$, el resultado será el siguiente:

```

Número de discos : 3
mover disco de A a C
mover disco de A a B
mover disco de C a B

```

```
mover disco de A a C  
mover disco de B a A  
mover disco de B a C  
mover disco de A a C
```

```
movimientos efectuados: 7
```

Como ejercicio se propone realizar el método *Mover* sin utilizar recursión.

ORDENACIÓN DE DATOS

Uno de los procedimientos más comunes y útiles en el procesamiento de datos es la ordenación de los mismos. Se considera ordenar al proceso de reorganizar un conjunto dado de objetos en una secuencia determinada. El objetivo de este proceso generalmente es facilitar la búsqueda de uno o más elementos pertenecientes a un conjunto. Son ejemplos de datos ordenados las listas de los alumnos matriculados en una cierta asignatura, las listas del censo, los índices alfabéticos de los libros, las guías telefónicas, etc. Esto quiere decir que muchos problemas están relacionados de alguna forma con el proceso de ordenación. Es por lo que la ordenación es un problema importante a considerar.

La ordenación, tanto numérica como alfanumérica, sigue las mismas reglas que empleamos nosotros en la vida normal. Esto es, un dato numérico es mayor que otro cuando su valor es más grande, y una cadena de caracteres es mayor que otra cuando está después por orden alfabético.

Podemos agrupar los métodos de ordenación en dos categorías: ordenación de matrices u ordenación interna (cuando los datos se guardan en memoria interna) y ordenación de ficheros u ordenación externa (cuando los datos se guardan en memoria externa; generalmente en discos).

En este apartado no se trata de analizar exhaustivamente todos los métodos de ordenación y ver sus prestaciones de eficiencia, rapidez, etc., sino que simplemente analizamos desde el punto de vista práctico los métodos más comunes para ordenación de matrices y de ficheros.

Método de la burbuja

Hay muchas formas de ordenar datos, pero una de las más conocidas es la ordenación por el método de la burbuja.

Veamos a continuación el algoritmo correspondiente a este método para ordenar una lista de menor a mayor, partiendo de que los datos a ordenar están almacenados en una matriz de n elementos:

1. Comparamos el primer elemento con el segundo, el segundo con el tercero, el tercero con el cuarto, etc. Cuando el resultado de una comparación sea “mayor que”, se intercambian los valores de los elementos comparados. Con esto conseguimos llevar el valor mayor a la posición n .
2. Repetimos el punto 1, ahora para los $n-1$ primeros elementos de la lista. Con esto conseguimos llevar el valor mayor de éstos a la posición $n-1$.
3. Repetimos el punto 1, ahora para los $n-2$ primeros elementos de la lista y así sucesivamente.
4. La ordenación estará realizada cuando al repetir el *iésimo* proceso de comparación no haya habido ningún intercambio o, en el peor de los casos, después de repetir el proceso de comparación descrito $n-1$ veces.

El pseudocódigo para este algoritmo puede ser el siguiente:

```
<método ordenar(matriz "a" de "n" elementos)>
["a" es un matriz cuyos elementos son  $a_0, a_1, \dots, a_{n-1}$ ]
  n = n-1
  DO WHILE ("a" no esté ordenado y n > 0 )
    i = 1
    DO WHILE ( i <= n )
      IF (  $a[i-1] > a[i]$  ) THEN
        permutar  $a[i-1]$  con  $a[i]$ 
      ENDIF
      i = i+1
    ENDDO
    n = n-1
  ENDDO
END <método ordenar>
```

La clase siguiente incluye el método *Ordenar* que utiliza este algoritmo para ordenar una matriz de tipo T .

```
////////////////////////////////////
// Ordenación por el método de la burbuja. El método "Ordenar"
// tiene dos parámetros. El primero es la matriz a ordenar y el
// segundo un delegado que ejecuta el método que define el tipo
// de comparación que se realizará sobre dos elementos.
//
public class CMatriz<T>
{
```

```
public delegate bool TipoOrdenacion(T x, T y);

public static void Ordenar(T[] m, TipoOrdenacion comparar)
{
    T aux;
    int i, número_de_elementos = m.Length;
    bool s = true;

    while (s && (--número_de_elementos > 0))
    {
        s = false; // no permutación
        for (i = 1; i <= número_de_elementos; i++)
            // ¿ el elemento (i-1) es mayor que el (i) ?
            if (comparar(m[i-1], m[i]))
            {
                // permutar los elementos (i-1) e (i)
                aux = m[i-1];
                m[i-1] = m[i];
                m[i] = aux;
                s = true; // permutación
            }
    }
}

////////////////////////////////////
```

Observe que **s** inicialmente vale **false** para cada iteración y toma el valor **true** cuando al menos se efectúa un cambio entre dos elementos. Si en una exploración a lo largo de la lista no se efectúa cambio alguno, **s** permanecerá valiendo **false**, lo que indica que la lista está ordenada, terminando así el proceso.

Cuando se analiza un método de ordenación, hay que determinar cuántas comparaciones e intercambios se realizan para el caso más favorable, para el caso medio y para el caso más desfavorable.

En el método de la burbuja se realizan $(n-1)(n/2)=(n^2-n)/2$ comparaciones en el caso más desfavorable, donde n es el número de elementos a ordenar. Para el caso más favorable (la lista está ordenada) el número de intercambios es 0. Para el caso medio es $3(n^2-n)/4$; hay tres intercambios por cada elemento desordenado. Y para el caso menos favorable, el número de intercambios es $3(n^2-n)/2$. El análisis matemático que conduce a estos valores queda fuera del propósito de este libro. El tiempo de ejecución es un múltiplo de n^2 y está directamente relacionado con el número de comparaciones y de intercambios.

La siguiente aplicación ordena una matriz **double** y otra de tipo **string** utilizando el método *Ordenar* de la clase *CMatriz*.


```

public class Test
{
    public static bool CompararDouble(double x, double y)
    {
        return x > y; // orden ascendente
    }

    public static bool CompararString(string x, string y)
    {
        return x.CompareTo(y) < 0; // orden descendente
    }

    public static void Main(string[] args)
    {
        // Burbuja:
        // Matriz numérica
        double[] m = { 3, 2, 1, 5, 4 };
        CMatriz<double>.Ordenar(m, CompararDouble);
        for (int i = 0; i < m.Length; i++)
            Console.Write(m[i] + " ");
        Console.WriteLine();

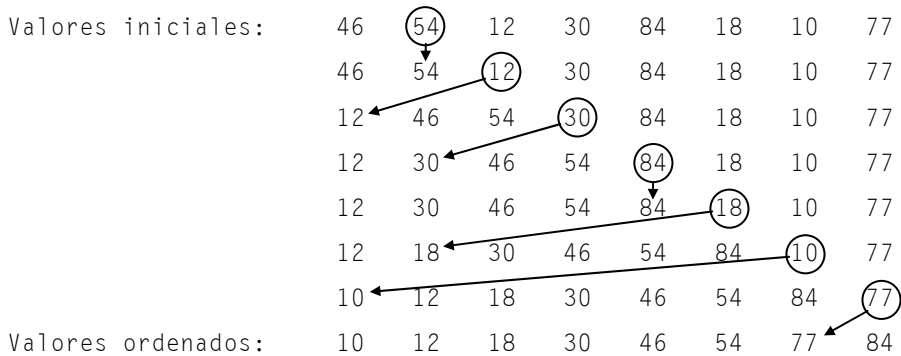
        // Matriz de cadenas de caracteres
        string[] s = { "ccc", "bbb", "aaa", "eee", "ddd" };
        CMatriz<string>.Ordenar(s, CompararString);
        for (int i = 0; i < s.Length; i++)
            Console.Write(s[i] + " ");
        Console.WriteLine();
    }
}

```

Obsérvese el segundo argumento pasado cuando se invoca al método *Ordenar*; es el nombre del procedimiento que se ejecutará durante el proceso de ordenación cuando sea necesario comparar dos elementos de la matriz para conocer cuál de ellos estará antes o después, en función del tipo de ordenación elegido.

Método de inserción

El algoritmo para este método de ordenación es el siguiente: inicialmente, se ordenan los dos primeros elementos de la matriz, luego se inserta el tercer elemento en la posición correcta con respecto a los dos primeros, a continuación se inserta el cuarto elemento en la posición correcta con respecto a los tres primeros elementos ya ordenados y así sucesivamente hasta llegar al último elemento de la matriz. Por ejemplo:



El pseudocódigo para este algoritmo puede ser el siguiente:

```

<método inserción(matriz "a" de "n" elementos)>
["a" es un matriz cuyos elementos son  $a_0, a_1, \dots, a_{n-1}$ ]
  i = 1
  DO WHILE ( i < n )
    x = a[i]
    insertar x en la posición correcta entre  $a_0$  y  $a_i$ 
  ENDDO
END <inserción>

```

La programación de este algoritmo, para el caso concreto de ordenar numéricamente una lista de valores, es la siguiente:

```

public static void Insercion(T[] m, TipoOrdenacion comparar)
{
    int i, k, n_elementos = m.Length;
    T x;

    // Desde el segundo elemento
    for (i = 1; i < n_elementos; i++)
    {
        x = m[i];
        k = i - 1;
        // Para k=-1, se ha alcanzado el extremo izquierdo.
        while (k >= 0 && comparar(m[k], x))
        {
            m[k + 1] = m[k]; // hacer hueco para insertar
            k--;
        }
        m[k + 1] = x; // insertar x en su lugar
    }
}

```

Análisis del método de inserción directa:

	<i>comparaciones</i>	<i>intercambios</i>
<i>caso más favorable</i>	$n-1$	$2(n-1)$
<i>caso medio</i>	$(n^2 + n - 2)/4$	$(n^2 + 9n - 10)/4$
<i>caso menos favorable</i>	$(n^2 + n)/2 - 1$	$(n^2 + 3n - 4)/2$

Para el método de inserción, el tiempo de ejecución es función de n^2 y está directamente relacionado con el número de comparaciones y de intercambios.

Método quicksort

El método de ordenación *quicksort* está generalmente considerado como el mejor algoritmo de ordenación disponible actualmente. El proceso seguido por este algoritmo es el siguiente:

1. Se selecciona un valor perteneciente al rango de valores de la matriz. Este valor se puede escoger aleatoriamente o haciendo la media de un pequeño conjunto de valores tomados de la matriz. El valor óptimo sería la mediana (el valor que es menor o igual que los valores correspondientes a la mitad de los elementos de la matriz y mayor o igual que los valores correspondientes a la otra mitad). No obstante, incluso en el peor de los casos (el valor escogido está en un extremo), *quicksort* funciona correctamente.
2. Se divide la matriz en dos partes: una con todos los elementos menores que el valor seleccionado y otra con todos los elementos mayores o iguales.
3. Se repiten los puntos 1 y 2 para cada una de las partes en la que se ha dividido la matriz, hasta que esté ordenada.

El proceso descrito es esencialmente recursivo. Según lo expuesto, elseudocódigo para este algoritmo puede ser el siguiente:

```

<método qs(matriz "a")>
  Se elige un valor x de la matriz
  DO WHILE ( "a" no esté dividido en dos partes )
    [dividir "a" en dos partes: a_inf y a_sup]
    a_inf con los elementos  $a_i < x$ 
    a_sup con los elementos  $a_i \geq x$ 
  ENDDO
  IF ( existe a_inf ) THEN
    qs( a_inf )
  ENDIF
  IF ( existe a_sup ) THEN
    qs( a_sup )
  ENDIF
END <qs>

```

A continuación se muestra una versión de este algoritmo, que selecciona el elemento medio de la matriz para proceder a dividirla en dos partes. Esto resulta fácil de implementar, aunque no siempre da lugar a una buena elección. A pesar de ello, funciona correctamente.

```
public static void Quicksort(T[] m, TipoOrdenacion comparar)
{
    Qs(m, 0, m.Length - 1, comparar);
}

// Método recursivo Qs
private static void Qs(T[] m, int inf, int sup,
                       TipoOrdenacion comparar)
{
    int izq, der;
    T mitad, x;

    izq = inf; der = sup;
    mitad = m[(izq + der) / 2];

    do
    {
        while (comparar(mitad, m[izq]) && izq < sup) izq++;
        while (comparar(m[der], mitad) && der > inf) der--;
        if (izq <= der)
        {
            x = m[izq]; m[izq] = m[der]; m[der] = x;
            izq++; der--;
        }
    }
    while (izq <= der);
    if (inf < der) Qs(m, inf, der, comparar);
    if (izq < sup) Qs(m, izq, sup, comparar);
}
```

Observamos que cuando el valor *mitad* se corresponde con uno de los valores de la lista, las condiciones $izq < sup$ y $der > inf$ de las sentencias:

```
while (comparar(mitad, m[izq]) && izq < sup) izq++;
while (comparar(m[der], mitad) && der > inf) der--;
```

no serían necesarias. En cambio, si el valor *mitad* es un valor no coincidente con un elemento de la lista, pero que está dentro del rango de valores al que pertenecen los elementos de la misma, esas condiciones son necesarias para evitar que se puedan sobrepasar los límites de los índices de la matriz.

Para experimentarlo, pruebe como ejemplo la lista de valores 1 1 3 1 1 y elija *mitad* = 2 fijo.

En el método *Quicksort*, en el caso más favorable, esto es, cada vez que se selecciona la mediana obteniéndose dos particiones iguales, se realizan $n \log n$ comparaciones y $n/6 \log n$ intercambios, donde n es el número de elementos a ordenar; en el caso medio, el rendimiento es inferior al caso óptimo en un factor de $2 \log 2$; y en el caso menos favorable, esto es, cada vez que se selecciona el valor mayor obteniéndose una partición de $n-1$ elementos y otra de un elemento, el rendimiento es del orden de $n \cdot n = n^2$. Con el fin de mejorar el caso menos favorable, se sugiere elegir, cada vez, un valor aleatoriamente o un valor que sea la mediana de un pequeño conjunto de valores tomados de la matriz.

El método Qs sin utilizar la recursión puede desarrollarse de la forma siguiente:

```
public static void QuicksortNR(T[] m, TipoOrdenacion comparar)
{
    QsNR(m, 0, m.Length - 1, comparar);
}

// Método quicksort no recursivo
private static void QsNR(T[] m, int inf, int sup,
                        TipoOrdenacion comparar)
{
    // Pila de elementos (inf, sup)
    CPila<CDatos> pila = new CPila<CDatos>();
    CDatos dato; // encapsula los atributos inf y sup

    int izq, der, p;
    T mitad, x;

    // Iniciar la pila con los valores: inf, sup
    pila.MeterEnPila(new CDatos(inf, sup));
    do
    {
        // Tomar los datos inf, sup de la parte superior de la pila
        dato = pila.SacarDePila();
        if (dato == null) break;
        inf = dato.Inf; sup = dato.Sup;
        do
        {
            // División de la matriz en dos partes
            izq = inf; der = sup;
            mitad = m[(izq + der) / 2];
            do
            {
                while (comparar(mitad, m[izq]) && izq < sup) izq++;
                while (comparar(m[der], mitad) && der > inf) der--;
                if (izq <= der)
                {
                    x = m[izq]; m[izq] = m[der]; m[der] = x;
                }
            }
        }
    }
}
```

```
        izq++; der--;  
    }  
}  
while (izq <= der);  
if (izq < sup)  
{  
    // Meter en la pila los valores: izq, sup  
    pila.MeterEnPila(new CDatos(izq, sup));  
}  
/* inf = inf; */ sup = der;  
}  
while (inf < der);  
}  
while (true);  
}
```

En esta solución observamos que después de cada paso se generan dos nuevas sublistas. Una de ellas la tratamos en la siguiente iteración y la otra la posponemos, guardando sus límites *inf* y *sup* en una pila. El método *QsNR* utiliza la clase *CPila*, implementada en el capítulo de *Estructuras dinámicas*, para crear una pila que almacene los valores *inf* y *sup* a los que nos hemos referido anteriormente. Estos límites son los atributos de objetos de una clase *CDatos* definida así:

```
public class CDatos  
{  
    // Propiedades  
    public int Inf { get; set; }  
    public int Sup { get; set; }  
  
    // Métodos  
    public CDatos(int i, int s) // constructor con parámetros  
    {  
        Inf = i;  
        Sup = s;  
    }  
}
```

Comparación de los métodos expuestos

Si medimos los tiempos consumidos por los métodos de ordenación estudiados anteriormente, observaremos que el método de la burbuja es el peor de los métodos; el método de inserción directa mejora considerablemente y el método *quick-sort* es el más rápido y mejor método de ordenación de matrices con diferencia.

BÚSQUEDA DE DATOS

El objetivo de ordenar un conjunto de objetos es, generalmente, facilitar la búsqueda de uno o más elementos pertenecientes a ese conjunto, aunque es posible realizar dicha búsqueda sin que el conjunto de objetos esté ordenado, pero esto trae como consecuencia un mayor tiempo de proceso.

Búsqueda secuencial

Este método de búsqueda, aunque válido, es el menos eficiente. Se basa en comparar el valor que se desea buscar con cada uno de los valores de la matriz. La matriz no tiene por qué estar ordenada.

El pseudocódigo para este método de búsqueda puede ser el siguiente:

```
<método búsqueda_S( matriz a, valor que queremos buscar)>
  i = 0
  DO WHILE ( no encontrado )
    IF ( valor = a[i] )
      encontrado
    ENDIF
    i = i+1
  ENDDO
END <búsqueda_S>
```

Como ejercicio, escribir el código correspondiente a un método que permita buscar un valor previamente leído, en una matriz.

Búsqueda binaria

Un método eficiente de búsqueda, que puede aplicarse a las matrices clasificadas, es la *búsqueda binaria*. Si partimos de que los elementos de la matriz están almacenados en orden ascendente, el proceso de búsqueda binaria puede describirse así: se selecciona el elemento del centro o aproximadamente del centro de la matriz. Si el valor a buscar no coincide con el elemento seleccionado y es mayor que él, se continúa la búsqueda en la segunda mitad de la matriz. Si, por el contrario, el valor a buscar es menor que el valor del elemento seleccionado, la búsqueda continúa en la primera mitad de la matriz. En ambos casos, se halla de nuevo el elemento central, correspondiente al nuevo intervalo de búsqueda, repitiéndose el ciclo. El proceso se repite hasta que se encuentra el valor a buscar, o bien hasta que el intervalo de búsqueda sea nulo, lo que querrá decir que el elemento buscado no figura en la matriz.

El pseudocódigo para este algoritmo puede ser el siguiente:

```
<método búsquedaBin( matriz a, valor que queremos buscar )>
DO WHILE ( no encontrado y exista un intervalo donde buscar )
    x = elemento mitad del intervalo de búsqueda
    IF ( valor > x ) THEN
        buscar "valor" en la segunda mitad del intervalo de búsqueda
    ELSE
        buscar "valor" en la primera mitad del intervalo de búsqueda
    ENDIF
ENDDO
IF ( se encontró valor ) THEN
    retornar su índice
ELSE
    retornar -1
ENDIF
END <búsquedaBin>
```

A continuación se muestra el código correspondiente a este método:

```
public static int BúsquedaBin(double[] m, double v)
{
    // El método BúsquedaBin devuelve como resultado la posición
    // del valor. Si el valor no se localiza devuelve -1.

    if (m.Length == 0) return -1;
    int mitad, inf = 0, sup = m.Length - 1;

    do
    {
        mitad = (inf + sup) / 2;
        if (v > m[mitad])
            inf = mitad + 1;
        else
            sup = mitad - 1;
    }
    while (m[mitad] != v && inf <= sup);

    if (m[mitad] == v)
        return mitad;
    else
        return -1;
}
```

Búsqueda de cadenas

Uno de los métodos más eficientes en la búsqueda de cadenas dentro de un texto es el algoritmo *Boyer y Moore*. La implementación básica de este método construye una tabla *delta* que se utilizará en la toma de decisiones durante la búsqueda de una subcadena. Dicha tabla contiene un número de entradas igual al número de

caracteres del código que se esté utilizando. Por ejemplo, si se está utilizando el código de caracteres ASCII la tabla será de 256 entradas. Cada entrada contiene el valor *delta* asociado con el carácter que representa. Por ejemplo, el valor *delta* asociado con *A* estará en la entrada 65 y el valor *delta* asociado con el *espacio en blanco*, en la entrada 32. El valor *delta* para un carácter es la posición de la ocurrencia más a la derecha de ese carácter respecto a la posición final en la cadena buscada. Las entradas correspondientes a los caracteres que no pertenecen a la cadena a buscar tienen un valor igual a la longitud de esta cadena.

Por lo tanto, para definir la tabla *delta* para una determinada subcadena a buscar, construimos una matriz con todos sus elementos iniciados a la longitud de dicha cadena, y luego, asignamos el valor *delta* para cada carácter de la subcadena, así:

```
for ( i = 0; i < longitud_cadena_patrón; i++ )
    delta[cadena_patrón[i]] = longitud_cadena_patrón - i - 1;
```

En el algoritmo de *Boyer y Moore* la comparación se realiza de derecha a izquierda, empezando desde el principio del texto. Es decir, se empieza comparando el último carácter de la cadena que se busca con el correspondiente carácter en el texto donde se busca; si los caracteres no coinciden, la cadena que se busca se desplaza hacia la derecha un número de caracteres igual al valor indicado por la entrada en la tabla *delta* correspondiente al carácter del *texto* que no coincide. Si el carácter no aparece en la cadena que se busca, su valor *delta* es la longitud de la cadena que se busca.

Veamos un ejemplo. Suponga que se desea buscar la cadena “cien” en el texto “Más vale un ya que cien después se hará”. La búsqueda comienza así:

```
Texto:           Más vale un ya que cien después se hará
Cadena a buscar: cien
```

El funcionamiento del algoritmo puede comprenderse mejor situando la cadena a buscar paralela al texto. La comparación es de derecha a izquierda; por lo tanto, se compara el último carácter en la cadena a buscar (*n*) con el carácter que está justamente encima en el texto (*espacio*). Como *n* es distinto de *espacio*, la cadena que se busca debe desplazarse a la derecha un número de caracteres igual al valor indicado por la entrada en la tabla *delta* que corresponde al carácter del *texto* que no coincide. Para la cadena “cien”,

```
delta['c'] = 3
delta['i'] = 2
delta['e'] = 1
delta['n'] = 0
```

El resto de las entradas valen 4 (longitud de la cadena). Según esto, la cadena que se busca se desplaza cuatro posiciones a la derecha (el espacio en blanco no aparece en la cadena que se busca).

Texto: Más vale un ya que cien después se hará
Cadena a buscar: cien

Ahora, *n* no coincide con *e*; luego la cadena se desplaza una posición a la derecha (*e* tiene un valor asociado de *uno*).

Texto: Más vale un ya que cien después se hará
Cadena a buscar: cien

n no coincide con *espacio*; se desplaza la cadena cuatro posiciones a la derecha.

Texto: Más vale un ya que cien después se hará
Cadena a buscar: cien

n no coincide con *y*; se desplaza la cadena cuatro posiciones a la derecha.

Texto: Más vale un ya que cien después se hará
Cadena a buscar: cien

n no coincide con *u*; se desplaza la cadena cuatro posiciones a la derecha.

Texto: Más vale un ya que cien después se hará
Cadena a buscar: cien

n no coincide con *i*; se desplaza la cadena dos posiciones a la derecha.

Texto: Más vale un ya que cien después se hará
Cadena a buscar: cien

Todos los caracteres de la cadena coinciden con los correspondientes caracteres en el texto. Para encontrar la cadena se han necesitado sólo $7 + 3$ comparaciones (7 hasta que se dio la coincidencia del carácter *n* de “cien” más 3 para verificar que coincidían los 3 caracteres restantes). El algoritmo directo habría realizado $20 + 3$ comparaciones, que en el peor de los casos, serían $i * longCadBuscar$, donde *i* es la posición más a la izquierda de la primera ocurrencia de la cadena a buscar en el texto (20 en el ejemplo anterior, suponiendo que la primera posición es la 1) y *longCadBuscar* es la longitud de la cadena a buscar (4 en el ejemplo anterior). En cambio, el algoritmo *Boyer y Moore* emplearía $k * (i + longCadBuscar)$ comparaciones, donde $k < 1$.

El algoritmo *Boyer y Moore* es más rápido porque tiene información sobre la cadena que se busca, en la tabla *delta*. El carácter que ha causado la no coincidencia en el texto indica cómo mover la cadena respecto del texto. Si el carácter no coincidente en el texto no existe en la cadena, ésta puede moverse sin problemas a la derecha un número de caracteres igual a su longitud, pues es un gasto de tiempo comparar la cadena con un carácter que ella no contiene. Cuando el carácter no coincidente en el texto está presente en la cadena, el valor *delta* para ese carácter alinea la ocurrencia más a la derecha de ese carácter en la cadena con el carácter en el texto.

A continuación se muestra el código correspondiente al algoritmo *Boyer y Moore*. El método *buscarCadena* es el que realiza el proceso descrito. Este método devuelve la posición de la cadena en el texto o -1 si la cadena no se encuentra (la primera posición es la 0).

```
public static int BuscarCadena(string stexto, string scadena)
{
    // Buscar una "cadena" en un "texto"
    char[] texto = stexto.ToCharArray();
    char[] cadena = scadena.ToCharArray();

    // Construir la tabla "delta"
    int[] delta = new int[256];
    int i, longCad = cadena.Length;

    // Iniciar la tabla "delta"
    for (i = 0; i < 256; i++)
        delta[i] = longCad;

    // Asignar valores a la tabla
    for (i = 0; i < longCad; ++i)
        delta[cadena[i]] = longCad - i - 1;

    // Algoritmo Boyer-Moore
    int j, longTex = texto.Length;
    i = longCad - 1; // i es el índice dentro del texto

    while (i < longTex)
    {
        j = longCad - 1; // índice dentro de la cadena a buscar
        // Mientras haya coincidencia de caracteres
        while (cadena[j] == texto[i])
        {
            if (j > 0)
            {
                // Siguiendo posición a la izquierda
                j--; i--;
            }
        }
    }
}
```

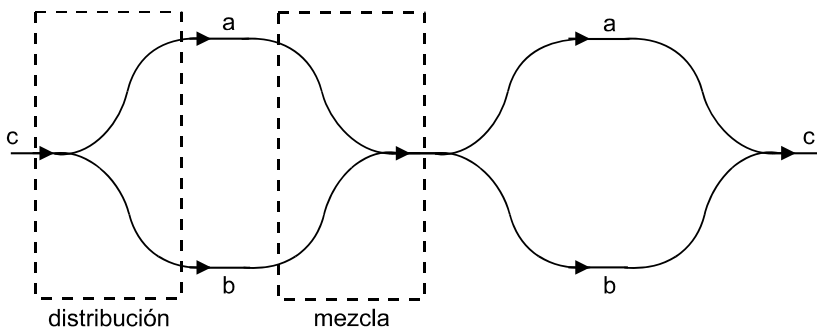
```
else
{
    // Se llegó al principio de la cadena, luego se encontró.
    return i;
}
}
// Los caracteres no coinciden. Mover i lo que indique el
// valor "delta" del carácter del texto que no coincide
i += delta[texto[i]];
}
return -1;
}
```

ORDENACIÓN DE FICHEROS EN DISCO

Para ordenar un fichero, dependiendo del tamaño del mismo, podremos proceder de alguna de las dos formas siguientes. Si el fichero es pequeño, tiene pocos registros, se puede copiar en memoria en una matriz y utilizando las técnicas vistas anteriormente, ordenamos dicha matriz y a continuación copiamos la matriz ordenada de nuevo en el fichero. Sin embargo, muchos ficheros son demasiado grandes y no cabrían en una matriz en memoria, por lo que para ordenarlos recurriremos a técnicas que actúen sobre el propio fichero.

Ordenación de ficheros. Acceso secuencial

El siguiente programa desarrolla un algoritmo de ordenación de un fichero utilizando el acceso secuencial, denominado *mezcla natural*. La secuencia inicial de los elementos viene dada en el fichero *c* y se utilizan dos ficheros auxiliares denominados *a* y *b*. Cada pasada consiste en una *fase de distribución* que reparte equitativamente los tramos ordenados del fichero *c* sobre los ficheros *a* y *b*, y una *fase que mezcla* los tramos de los ficheros *a* y *b* sobre el fichero *c*.



Este proceso se ilustra en el ejemplo siguiente. Partimos de un fichero *c*. Con el fin de ilustrar el método de *mezcla natural*, separaremos los tramos ordenados en los ficheros por un guión (-).

fichero c: 18 32 - 10 60 - 14 42 44 68 - 12 24 30 48

Fase de distribución:

fichero a: 18 32 - 14 42 44 68
fichero b: 10 60 - 12 24 30 48

Fase de mezcla:

fichero c: 10 18 32 60 - 12 14 24 30 42 44 48 68

Fase de distribución:

fichero a: 10 18 32 60
fichero b: 12 14 24 30 42 44 48 68

Fase de mezcla:

fichero c: 10 12 14 18 24 30 32 42 44 48 60 68

Para dejar ordenado el fichero del ejemplo hemos necesitado realizar dos pasadas. El proceso finaliza tan pronto como el número de tramos ordenados del fichero *c* sea 1. Una forma de reducir el número de pasadas es distribuir los tramos ordenados sobre más de dos ficheros.

Según lo expuesto, el algoritmo de ordenación *mezcla natural* podría ser así:

```
<método mezcla_natural()>
  n_tramos = 0;
  DO
    [Crear y abrir los ficheros temporales a y b]
    n_tramos = distribución();
    n_tramos = mezcla();
  WHILE (n_tramos != 1);
END <mezcla_natural()>
```

La estructura de la aplicación que permita ordenar un fichero utilizando el algoritmo descrito puede ser de la forma siguiente:

```
public class CMezclaNatural
{
    public static void MezclaNatural(string fichFuente)
    {
```

```
int nro_tramos = 0;
// a y b son dos ficheros temporales
do
{
    distribuir(fichFuente, a, b);
    nro_tramos = mezclar(a, b, fichFuente);
}
while (nro_tramos != 1);
}

public static int Distribuir(string fuente, string destinoA,
                             string destinoB)
{
    // Distribuir los tramos ordenados de fuente entre
    // destinoA y destinoB
}

public static int Mezclar(string fuenteA, string fuenteB,
                           string destino)
{
    // Fusionar ordenadamente los tramos de fuenteA y fuenteB
    // en destino
}

public static void Main(string[] args)
{
    string nombreFichero = args[0];
    mezclaNatural(nombreFichero);
}
}
```

La aplicación completa y comentada se muestra a continuación.

```
using System;
using System.IO;
////////////////////////////////////
// Ordenar un fichero utilizando el método de mezcla natural.
// Se trata de un fichero de texto que almacena una lista de
// nombres.
// El nombre del fichero se recibe a través de la línea de órdenes.
// La ordenación se realiza en orden alfabético ascendente.
// La aplicación está soportada por la clase CMezclaNatural.
// Métodos:
//     MezclaNatural
//     Distribuir
//     Mezclar
//     Main
//
public class CMezclaNatural
{
```

```

// Mezcla natural //////////////////////////////////////
public static void MezclaNatural(string fichFuente)
{
    // Definición de variables
    string a = "ftempa.tmp"; // fichero temporal
    string b = "ftempb.tmp"; // fichero temporal

    int nro_tramos = 0;
    do
    {
        nro_tramos = Distribuir(fichFuente, a, b);
        if (nro_tramos > 1)
            nro_tramos = Mezclar(a, b, fichFuente);
    }
    while (nro_tramos != 1);
    // Proceso finalizado. Borrar los ficheros temporales.
    File.Delete(a); File.Delete(b);
} // MezclaNatural

// Fase de distribución //////////////////////////////////
public static int Distribuir(string fuente, string destinoA,
                             string destinoB)
{
    // Abrir un flujo de entrada desde fuente que permita
    // leer la información línea a línea.
    StreamReader fc = new StreamReader(fuente);

    // Abrir un flujo de salida hacia destinoA
    StreamWriter fa = new StreamWriter(destinoA);

    // Abrir un flujo de salida hacia destinoB
    StreamWriter fb = new StreamWriter(destinoB);

    StreamWriter faux = fa; // faux será fa o fb
    string línea;           // última línea leída
    string línea_ant;       // línea anterior a la última leída
    int nro_tramos = 1;     // número total de tramos ordenados

    // Leer la primera línea (línea anterior)
    if ((línea_ant = fc.ReadLine()) != null)
    {
        // Escribe en fa la línea leída más el separador de línea
        fa.WriteLine(línea_ant);
    }
    else
    {
        faux = null; fc.Close(); fa.Close(); fb.Close();
        return 0;
    }

    // Leer la siguiente línea (línea actual)

```

```
while ((línea = fc.ReadLine()) != null)
{
    if (línea.CompareTo(línea_ant) < 0)
    {
        // Cambiar al otro fichero
        faux = (faux == fa) ? fb : fa;
        ++nro_tramos;
    }
    línea_ant = línea;
    // Escribe en faux la línea leída más el separador de línea
    faux.WriteLine(línea);
}
faux = null; fc.Close(); fa.Close(); fb.Close();
return nro_tramos;
} // Distribuir

// Fase de mezcla ////////////////////////////////////////////
public static int Mezclar(string fuenteA, string fuenteB,
                        string destino)
{
    // Abrir un flujo de entrada desde fuenteA que permita
    // leer la información línea a línea.
    StreamReader fa = new StreamReader(fuenteA);

    // Abrir un flujo de entrada desde fuenteB que permita
    // leer la información línea a línea.
    StreamReader fb = new StreamReader(fuenteB);

    // Abrir un flujo de salida hacia destino
    StreamWriter fc = new StreamWriter(destino);

    string líneaDeFa, líneaDeFb, líneaDeFa_ant, líneaDeFb_ant;
    int nro_tramos = 1;

    // Leemos las dos primeras líneas, una de fa y otra de fb
    líneaDeFa = fa.ReadLine();
    líneaDeFa_ant = líneaDeFa;
    líneaDeFb = fb.ReadLine();
    líneaDeFb_ant = líneaDeFb;

    // Vamos leyendo y comparando hasta que se acabe alguno de los
    // ficheros. La fusión se realiza entre pares de tramos
    // ordenados. Un tramo de fa y otro de fb darán lugar a un
    // tramo ordenado sobre fc.
    while (líneaDeFa != null && líneaDeFb != null)
    {
        if (líneaDeFa.CompareTo(líneaDeFb) < 0)           // if 1
        {
            if (líneaDeFa.CompareTo(líneaDeFa_ant) < 0) // if 2
            // Encontrado el final del tramo de fa
            {
```



```

        líneaDeFa_ant = líneaDeFa;
        // Copiamos el tramo ordenado de fb
        do
        {
            fc.WriteLine(líneaDeFb);
            líneaDeFb_ant = líneaDeFb;
        }
        while ((líneaDeFb = fb.ReadLine()) != null &&
            líneaDeFb.CompareTo(líneaDeFb_ant) > 0);
        ++nro_tramos;
        líneaDeFb_ant = líneaDeFb;
    }
    else // de if 2
    {
        // Copiamos la cadena leída de fa
        líneaDeFa_ant = líneaDeFa;
        fc.WriteLine(líneaDeFa);
        líneaDeFa = fa.ReadLine();
    }
}
else // de if 1
{
    if (líneaDeFb.CompareTo(líneaDeFb_ant) < 0) // if 3
    // Encontrado el final del tramo de fb
    {
        líneaDeFb_ant = líneaDeFb;
        // Copiamos el tramo ordenado de fa
        do
        {
            fc.WriteLine(líneaDeFa);
            líneaDeFa_ant = líneaDeFa;
        }
        while ((líneaDeFa = fa.ReadLine()) != null &&
            líneaDeFa.CompareTo(líneaDeFa_ant) > 0);
        ++nro_tramos;
        líneaDeFa_ant = líneaDeFa;
    }
    else // de if 3
    {
        // Copiamos la cadena leída de fb
        líneaDeFb_ant = líneaDeFb;
        fc.WriteLine(líneaDeFb);
        líneaDeFb = fb.ReadLine();
    }
}
} // de while

// En el caso de acabarse primero los datos de fb
if (líneaDeFb == null)
{
    fc.WriteLine(líneaDeFa);
}

```

```
        while ((líneaDeFa = fa.ReadLine()) != null)
        {
            fc.WriteLine(líneaDeFa);
        }
    }
    // En el caso de acabarse primero los datos de fa
    else if (líneaDeFa == null)
    {
        fc.WriteLine(líneaDeFb);
        while ((líneaDeFb = fb.ReadLine()) != null)
        {
            fc.WriteLine(líneaDeFb);
        }
    }
    fc.Close(); fa.Close(); fb.Close();
    return nro_tramos;
} // de Mezclar

public static void Main(string[] args)
{
    // Main debe recibir un parámetro: el fichero a ordenar.
    if (args.Length != 1)
        Console.WriteLine("Sintaxis: MezclaNatural " +
                           "<nombre_fichero>");
    else
    {
        string nombreFichero = args[0];
        try
        {
            // Asegurarse de que "nombreFichero" existe y se puede leer
            if (!File.Exists(nombreFichero) ||
                Directory.Exists(nombreFichero))
                throw new IOException("No existe el fichero " +
                                       nombreFichero);

            MezclaNatural(nombreFichero); // realizar la ordenación

            // Mostrar el contenido del fichero
            char resp;
            Console.Write("¿Desea ver el contenido del fichero? s/n: ");
            resp = (char)Console.Read();
            Console.ReadLine(); // limpiar
            if (resp == 's')
            {
                // Abrir un flujo de entrada desde nombreFichero
                // que permita leer la información línea a línea.
                StreamReader sr = new StreamReader(nombreFichero);

                // Leer el fichero y mostrarlo
                string línea;
                while ((línea = sr.ReadLine()) != null)
```

```
        Console.WriteLine(línea);
    }
    sr.Close();
}
catch (IOException e)
{
    Console.WriteLine("Error: " + e.Message);
}
}
```

////////////////////////////////////

Ordenación de ficheros. Acceso aleatorio

El acceso aleatorio a un fichero, a diferencia del secuencial, permite ordenar la información contenida en el mismo sin tener que copiarla sobre otro fichero, para lo cual aplicaremos un proceso análogo al aplicado a las matrices, lo que simplifica enormemente el proceso ordenación. Esto quiere decir que los métodos expuestos para ordenar matrices pueden ser aplicados también para ordenar ficheros utilizando el acceso aleatorio.

Como ejemplo, vamos a añadir a la clase *CListaTfños* de la aplicación realizada en el apartado *Un ejemplo de acceso aleatorio a un fichero* del capítulo 15 un método denominado *Quicksort* para ordenar el fichero “lista de teléfonos” encapsulado por la misma, en el que cada registro estaba formado por los campos: *nombre*, *dirección* y *teléfono*. La ordenación del fichero la realizaremos por el campo *nombre*, de tipo alfabético, empleando el método *Quicksort* explicado anteriormente en este mismo capítulo.

```

////////////////////////////////////////
// Definición de la clase CListaTfnos.
//
using System;
using System.IO;
public class CListaTfnos
{
    private FileStream fs;           // flujo subyacente
    private BinaryWriter bw;        // flujo hacia el fichero
    private BinaryReader br;        // flujo desde el fichero
    private int nregs;              // número de registros
    private int tamañoReg = 140;    // tamaño del registro en bytes
    private bool regsEliminados = false; // true si se
                                    // eliminaron registros

    public CListaTfnos(string fichero)
    {

```

```
// ...
}

public void CerrarFichero() { bw.Close(); br.Close(); fs.Close(); }

public int NúmeroDeRegs() { return nregs; } // número de registros

public bool EscribirReg( int i, CPersona obj )
{
    // Escribir un registro en la posición i del fichero
}

public void AñadirReg(CPersona obj)
{
    if (EscribirReg( nregs, obj )) nregs++;
}

public CPersona LeerReg( int i )
{
    // Leer el registro de la posición i del fichero
}

public bool EliminarReg(long tel)
{
    // Buscar el registro con el teléfono especificado y marcarlo
    // para posteriormente eliminarlo
}

public bool TieneRegsEliminados()
{
    return regsEliminados;
}

public int BuscarReg(string str, int pos)
{
    // Buscar un registro por una subcadena del nombre
    // a partir de un registro determinado
}

// Método Quicksort para ordenar el fichero //////////////////////////////////
public void Quicksort()
{
    Qs(0, nregs - 1);
}

private void Qs(int inf, int sup)
{
    int izq = inf, der = sup;
    CPersona mitad;
```

```

// Obtener del registro mitad, el campo por el que
// se va a ordenar el fichero.
mitad = LeerReg((izq + der)/2);
do
{
    while (LeerReg(izq) < (mitad) && izq < sup) izq++;
    while (mitad < LeerReg(der) && der > inf) der--;
    if (izq <= der)
    {
        PermutarRegistros(izq, der);
        izq++; der--;
    }
}
while (izq <= der);

if (inf < der) Qs(inf, der);
if (izq < sup) Qs(izq, sup);
}

// Permutar los registros de las posiciones i y j
private void PermutarRegistros(int i, int j)
{
    CPersona x, y;
    // Leer los registros de las posiciones i y j
    x = LeerReg(i);
    y = LeerReg(j);

    // Escribirlos en las posiciones j e i
    EscribirReg(j, x);
    EscribirReg(i, y);
}
}

```

El método *Quicksort* realiza la ordenación de los *nregs* registros del fichero vinculado con el flujo *fs*. Para ello invoca al método recursivo *Qs*.

El método *PermutarRegistros* es llamado por *Qs* (*Quicksort*) cuando hay que permutar dos registros del fichero para que queden correctamente ordenados.

Las dos líneas siguientes pertenecen al método *Qs*. Ambas utilizan el operador *<* para comparar dos registros: el devuelto por el método *LeerReg* y el referenciado por *mitad*. Esto implica añadir a la clase *CPersona* la sobrecarga de los operadores *<* y *>*.

```

while (LeerReg(izq) < (mitad) && izq < sup) izq++;
while (mitad < LeerReg(der) && der > inf) der--;

```

Vamos a editar la clase *CPersona* para añadir la sobrecarga de los operadores *<* y *>* para que permitan comparar dos objetos de esa clase por el campo *nombre*:

```
public class CPersona
{
    // Atributos
    private string nombre;
    private string dirección;
    private long teléfono;
    // ...

    public static bool operator <(CPersona p1, CPersona p2)
    {
        return p1.Nombre.CompareTo(p2.Nombre) < 0;
    }

    public static bool operator >(CPersona p1, CPersona p2)
    {
        return p1.Nombre.CompareTo(p2.Nombre) > 0;
    }
}
```

Como ejercicio, puede añadir al menú presentado por la aplicación *Test* a la que nos hemos referido anteriormente, que creaba una lista de teléfonos a partir de la clase *CListaTfnos*, una opción más, *ordenar*, que permita ordenar el fichero:

```
private static CListaTfnos listatfnos;
// ...

listatfnos = new CListaTfnos(fichero);
// ...

case 6: // ordenar
    listatfnos.Quicksort();
    break;
```

ALGORITMOS HASH

Los algoritmos *hash* son métodos de búsqueda, que proporcionan una longitud de búsqueda pequeña y una flexibilidad superior a la de otros métodos, como puede ser el método de *búsqueda binaria* que requiere que los elementos de la matriz estén ordenados.

Por *longitud de búsqueda* se entiende el número de accesos que es necesario efectuar sobre una matriz para encontrar el elemento deseado.

Este método de búsqueda permite, como operaciones básicas, además de la búsqueda de un elemento, insertar un nuevo elemento y eliminar un elemento existente.

Matrices hash

Una matriz producto de la aplicación de un algoritmo *hash* se denomina *matriz hash* y son las matrices que se utilizan con mayor frecuencia en los procesos donde se requiere un acceso rápido a los datos. Gráficamente estas matrices tienen la siguiente forma:

<i>Clave</i>	<i>Contenido</i>
5040	
3721	
...	
6375	

La matriz se organiza con elementos formados por dos miembros: *clave* y *contenido*. La *clave* constituye el medio de acceso a la matriz. Aplicando a la *clave* una función de acceso *fa*, previamente definida, obtenemos un número entero positivo *i* correspondiente a la posición del elemento en la matriz.

$$i = fa(clave)$$

Conociendo la posición, tenemos acceso al *contenido*. El miembro *contenido* puede albergar directamente la información, o bien una referencia a dicha información, cuando ésta sea muy extensa. El acceso, tal cual lo hemos definido, recibe el nombre de *acceso directo*.

Como ejemplo, suponer que la *clave* de acceso se corresponde con el número del documento nacional de identidad (*dni*) y que el contenido son los datos correspondientes a la persona que tiene ese *dni*. Una función de acceso, $i = fa(dni)$, que haga corresponder la posición del elemento en la matriz con el *dni* es inmediata: $i = dni$. Esta función así definida presenta un inconveniente y es que el número de valores posibles de *i* es demasiado grande para utilizar una matriz. Para solucionar este problema, siempre es posible, dada una matriz de longitud *L*, crear una función de acceso, *fa*, que genere un valor comprendido entre 0 y *L*, más comúnmente entre 1 y *L*. En este caso puede suceder que dos o más claves den lugar a un mismo valor de *i*:

$$i = fa(clave_1) = fa(clave_2)$$

El método *hash* está basado en esta técnica; el acceso a la matriz es directo a través del número *i* y cuando se produce una *colisión* (dos claves diferentes dan un mismo número *i*) este elemento se busca en una zona denominada *área de desbordamiento*.

Método hash abierto

Éste es uno de los métodos más utilizados. El algoritmo para acceder a un elemento de la matriz de longitud *L* es el siguiente:

1. Se calcula $i = fa(clave)$.
2. Si la posición *i* de la matriz está libre, se inserta la *clave* y el *contenido*. Si no está libre y la *clave* es la misma, error: “clave duplicada”. Si no está libre y la clave es diferente, incrementamos *i* en una unidad y repetimos el proceso descrito en este punto 2. Como ejemplo, véase la tabla de la figura siguiente.

En la figura, se observa que se quiere insertar la clave 6383. Supongamos que aplicando la función de acceso obtenemos un valor 3; esto es:

$$i = fa(6383) = 3$$

Como la posición 3 está ocupada y la clave es diferente, tenemos que incrementar *i* y volver de nuevo al punto 2 del algoritmo.

	Clave	Contenido	
	5040		0
	3721		1
			2
6883 →	4007		3
→	3900		4
→			5
			6
	6375		7

La *longitud media de búsqueda* en una *matriz hash abierta* viene dada por la expresión:

$$accesos = (2-k)/(2-2k)$$

siendo k igual al número de elementos existentes en la matriz dividido por L . Por ejemplo, si existen 60 elementos en una matriz de longitud $L=100$, el número medio de accesos para localizar un elemento será:

$$\text{accesos} = (2-60/100)/(2-2*60/100) = 1,75$$

En el método de *búsqueda binaria*, el número de accesos viene dado por el valor $\log_2 N$, siendo N el número de elementos de la matriz.

Para reducir al máximo el número de colisiones y, como consecuencia, obtener una longitud media de búsqueda baja, es importante elegir bien la función de acceso. Una *función de acceso* o *función hash* bastante utilizada y que proporciona una distribución de las claves uniforme y aleatoria es la *función mitad del cuadrado* que dice: “dada una clave C , se eleva al cuadrado (C^2) y se cogen n bits del medio, siendo $2^n \leq L$ ”. Por ejemplo, supongamos:

$L = 256$ lo que implica $n = 8$
 $C = 625$
 $C^2 = 390625$ ($0 \leq C^2 \leq 2^{32}-1$)
 $390625_{10} = 00000000000001011111010111100001_2$
 $n \text{ bits del medio: } 01011111_2 = 95_{10}$

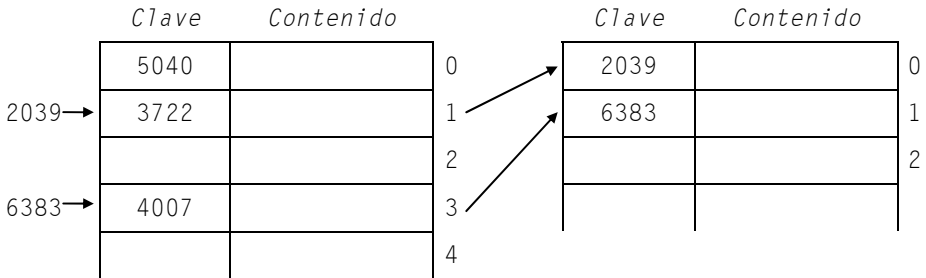
Otra función de acceso muy utilizada es la *función módulo* (resto de una división entera):

$$i = \text{módulo}(\text{clave}/L)$$

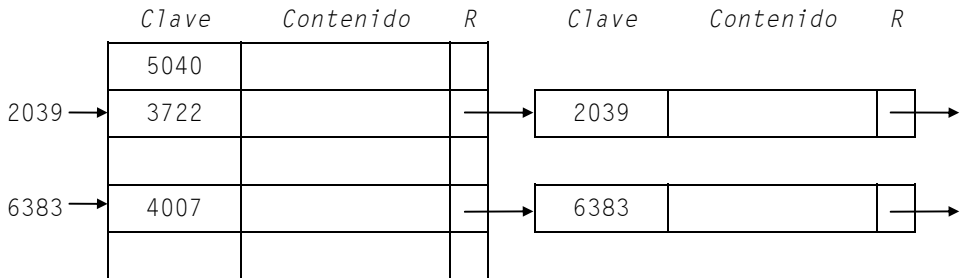
Cuando se utilice esta función es importante elegir un número primo para L , con la finalidad de que el número de colisiones sea pequeño. Esta función es llevada a cabo en C# por medio del operador %.

Método hash con desbordamiento

Una alternativa al método anterior es la de disponer de otra matriz separada, para insertar las claves que producen colisión, denominada *matriz de desbordamiento*, en la que se almacenan todas estas claves de forma consecutiva.



Otra forma alternativa más normal es organizar una lista encadenada por cada posición de la matriz donde se produzca una colisión.



Cada elemento de esta estructura incorpora un nuevo miembro *R*, el cual es una referencia a la lista encadenada de desbordamiento.

Eliminación de elementos

En el método *hash* la eliminación de un elemento no es tan simple como dejar vacío dicho elemento, ya que esto daría lugar a que los elementos insertados por colisión no puedan ser accedidos. Por ello, se suele utilizar un miembro complementario que sirva para poner una marca de que dicho elemento está eliminado. Esto permite acceder a otros elementos que dependen de él por colisiones, ya que la clave se conserva y también permite insertar un nuevo elemento en esa posición cuando se dé una nueva colisión.

Clase CHashAbierto

Como ejercicio escribimos a continuación una clase denominada *CHashAbierto* que proporciona los métodos necesarios para trabajar con matrices *hash* utilizando el método *hash abierto*, cuyo pseudocódigo se expone a continuación:

```

<método hash(matriz, elemento x)>
  [La matriz está iniciada a cero]
  i = clave módulo número_elementos
  DO WHILE (haya elementos libres)
    IF (elemento "i" está libre) THEN
      copiar elemento x en la posición i
    ELSE
      IF (clave duplicada) THEN
        error: clave duplicada
      ELSE
        [se ha producido una colisión]
        [avanzar al siguiente elemento]
        i = i+1
      IF (i = número_elementos) THEN
        i = 0
      ENDIF
    ENDIF
  ENDDO
END <hash>

```

La clase *CHashAbierto*<*T*> que vamos a implementar incluirá un atributo privado *matrizhash* para referenciar la *matriz hash*. Un objeto *CHashAbierto* encapsula una *matriz hash* de 101 elementos por omisión, que son referencias a objetos de tipo *T*, lo que permitirá almacenar elementos de cualquier clase. Así mismo, incluye los métodos indicados en la tabla siguiente:

Método	Significado
<i>CHashAbierto</i>	Es el constructor; está sobrecargado dos veces, una sin parámetros, que crea una matriz de 101 elementos, y otra con un parámetro, que especifica el número de elementos. Inicialmente todos los elementos almacenan el valor null .
<i>NúmeroDeElementos</i>	Método que devuelve el número de elementos de la matriz.
<i>NúmeroPrimo</i>	Método que devuelve un número primo de valor mayor o igual que el pasado como argumento. Siempre que se utilice la función de acceso <i>módulo</i> es importante elegir un número primo como longitud de la matriz, con la finalidad de que el número de colisiones sea pequeño.
<i>Fa</i>	Se trata de un método abstracto con la intención de redefinirlo en una subclase para permitir escribir la función de acceso que se desea aplicar.
<i>Existe</i>	Método abstracto que debe ser redefinido por el usuario en una subclase para permitir verificar si la clave del objeto a insertar en la tabla existe (compara dos objetos). Devuelve un valor de tipo bool indicando el resultado de la comparación (true para ==).

<i>HashIn</i>	Método <i>hash abierto</i> para añadir un elemento a la matriz. Si se añade, devuelve true , si no false .
<i>HashOut</i>	Método <i>hash abierto</i> para buscar un objeto con una clave determinada. Si se encuentra, devuelve una referencia de tipo object al mismo; en otro caso devuelve null .

A continuación se presenta el código correspondiente a la definición de la clase *CHashAbierto*:

```
using System;
////////////////////////////////////
// Clase abstracta CHashAbierto<T>: método hash abierto.
// Para utilizar los métodos proporcionados por esta clase,
// tendremos que crear una subclase de ella y redefinir los
// métodos: Fa (función de acceso) y Existe.
//
public abstract class CHashAbierto<T>
{
    // Atributos
    private T[] matrizhash;

    // Métodos
    public CHashAbierto()
    {
        matrizhash = new T[101];
    }

    public CHashAbierto(int númeroDeElementos)
    {
        if (númeroDeElementos < 1)
            númeroDeElementos = 101;
        else
            númeroDeElementos = NúmeroPrimo(númeroDeElementos);
        matrizhash = new T[númeroDeElementos];
    }

    public int NúmeroDeElementos() { return matrizhash.Length; }
    // Buscar un número primo a partir de un número dado //////////
    public int NúmeroPrimo(int n)
    {
        bool primo = false;
        int i, r = (int)Math.Sqrt((double)n);

        if (n % 2 == 0) n++;
        while (!primo)
        {
            primo = true;
            for (i = 3; i <= r; i += 2)
                if (n % i == 0) primo = false;
        }
    }
}
```

```

        if (!primo) n += 2; // siguiente impar
    }
    return n;
}

// Función de acceso //////////////////////////////////////
// Este método debe ser redefinido en una subclase para poder
// definir la función de acceso que el usuario desee aplicar.
public abstract int Fa(T obj);

// Método existe //////////////////////////////////////
// Este método debe ser redefinido en una subclase para que
// permita comparar las claves de dos objetos de los
// referenciados por la matriz hash.
public abstract bool Existe(T obj1, T obj2);

// Método hash abierto //////////////////////////////////////
public void HashIn(T x)
{
    int i;           // índice para acceder a un elemento
    int conta = 0; // contador
    bool insertado = false;

    i = Fa(x);       // función de acceso

    while (!insertado && conta < matrizhash.Length)
    {
        if (matrizhash[i] == null) // elemento libre
        {
            matrizhash[i] = x;
            insertado = true;
        }
        else // clave duplicada
            if (Existe(x, matrizhash[i]))
            {
                Console.WriteLine("error: clave duplicada");
                insertado = true;
            }
            else // colisión
            {
                // Siguiendo elemento libre
                i++; conta++;
                if (i == matrizhash.Length) i = 0;
            }
        }
        if (conta == matrizhash.Length)
            Console.WriteLine("error: matriz llena\n");
    }
}

public object HashOut(T x)
{

```

```
// x proporcionará el atributo utilizado para buscar. El resto
// de los atributos no interesan (son los que se desea conocer)
int i;          // índice para acceder a un elemento
int conta = 0; // contador
bool encontrado = false;

i = Fa(x);      // función de acceso
while (!encontrado && conta < matrizhash.Length)
{
    if (matrizhash[i] == null) return null;
    if (Existe(x, matrizhash[i]))
    {
        x = matrizhash[i];
        encontrado = true;
    }
    else // colisión
    {
        // Siguiente elemento libre
        i++; conta++;
        if (i == matrizhash.Length) i = 0;
    }
}
if (conta == matrizhash.Length) // no existe
    return null;
else
    return x;
}
}
////////////////////////////////////////////////////////////////
```

Un ejemplo de una matriz hash

Como ya hemos indicado, para utilizar esta clase tenemos que derivar otra de ella y redefinir los métodos *Fa* y *Existe* en función de la información encapsulada por los objetos de datos que deseemos manipular. Por ejemplo, supongamos que deseamos construir una *matriz hash* de objetos de la clase *CAumno*:

```
////////////////////////////////////////////////////////////////
// Definición de la clase CAumno
//
public class CAumno
{
    // Atributos
    private int matrícula;
    private string nombre;

    // Métodos
    public CAumno() {}
}
```

```

public CALumno(string nom, int mat)
{
    nombre = nom;
    matrícula = mat;
}

public int Matrícula
{
    get { return matrícula; }
    set { matrícula = value; }
}

public string Nombre
{
    get { return nombre; }
    set { nombre = value; }
}
}

```

Los objetos *CALumno* serán almacenados en la matriz utilizando como clave el número de matrícula. Según esto, derivamos la clase *HashAbierto* de la clase abstracta *CHashAbierto<CALumno>* y redefinimos los métodos *Fa* y *Existe* así:

```

using System;

////////////////////////////////////
// Clase derivada de la clase abstracta CHashAbierto<T>. Redefine
// los métodos: fa (función de acceso) y existe.
//
public class HashAbierto : CHashAbierto<CALumno>
{
    public HashAbierto(int nElementos) : base(nElementos) {}

    public override int Fa(CALumno obj)
    {
        return (int)((CALumno)obj).Matrícula % NúmeroDeElementos();
    }

    public override bool Existe(CALumno obj1, CALumno obj2)
    {
        if (((CALumno)obj1).Matrícula ==
            ((CALumno)obj2).Matrícula)
            return true;
        else
            return false;
    }
}
////////////////////////////////////

```

Observe que para definir la función de acceso *módulo (%)* necesitamos utilizar un valor numérico. Esto no quiere decir que la clave tenga que ser numérica, como sucede en nuestro ejemplo, sino que puede ser alfanumérica. Cuando se trabaje con claves alfanuméricas o alfabéticas, por ejemplo *nombre*, antes de aplicar la función de acceso es necesario convertir dicha clave en un valor numérico utilizando un algoritmo adecuado.

Finalmente, escribiremos una aplicación *Test* que permita crear un objeto *HashAbierto*, envoltorio de la *matriz hash*. Para probar su correcto funcionamiento escribiremos código que permita tanto añadir como buscar objetos en dicha matriz.

```
using System;
using MisClases.ES;
////////////////////////////////////
// Aplicación para trabajar con una matriz hash
//
public class Test
{
    public static void Main(string[] args)
    {
        // Definición de variables
        int n_elementos = 0; // número de elementos de la matriz hash
        string nombre;
        int matrícula = 0;
        CAlumno x;

        // Crear un objeto HashAbierto (encapsula la matriz hash)
        Console.WriteLine("número de elementos: ");
        Leer<int>.Dato(ref n_elementos);
        HashAbierto m = new HashAbierto(n_elementos);
        Console.WriteLine("Se construyó una matriz de " +
                           m.NúmeroDeElementos() + " elementos");

        // Introducir datos
        Console.WriteLine("Introducir datos. " +
                           "Para finalizar, matrícula = 0\n");
        Console.Write("matrícula: "); Leer<int>.Dato(ref matrícula);
        while (matrícula != 0)
        {
            Console.Write("nombre: "); nombre = Console.ReadLine();
            m.HashIn(new CAlumno(nombre, matrícula));
            Console.Write("matrícula: "); Leer<int>.Dato(ref matrícula);
        }

        // Buscar datos
        Console.WriteLine("Buscar datos. " +
                           "Para finalizar, matrícula = 0\n");
        Console.Write("matrícula: "); Leer<int>.Dato(ref matrícula);
```



```

while (matrícula != 0)
{
    x = (CAumno)m.HashOut(new CAumno("", matrícula));
    if (x != null)
        Console.WriteLine("nombre: " + x.Nombre);
    else
        Console.WriteLine("No existe");
    Console.Write("matrícula: "); Leer<int>.Dato(ref matrícula);
}
}
}

```

Como alternativa, la biblioteca de .NET proporciona en su espacio de nombres **System.Collections** la clase **Hashtable** que representa una colección de pares *clave y valor* organizados en función del código *hash* de la clave.

EJERCICIOS RESUELTOS

1. Comparar las dos siguientes versiones del método *búsqueda binaria* e indicar cuál de ellas es más eficaz.

```

public static int BúsquedaBin1(double[] m, double v)
{
    // El método BúsquedaBin1 devuelve como resultado la posición
    // del valor. Si el valor no se localiza devuelve -1.
    if (m.Length == 0) return -1;
    int mitad, inf = 0, sup = m.Length - 1;

    do
    {
        mitad = (inf + sup) / 2;
        if (v > m[mitad])
            inf = mitad + 1;
        else
            sup = mitad - 1;
    }
    while( m[mitad] != v && inf <= sup);

    if (m[mitad] == v)
        return mitad;
    else
        return -1;
}

```

```

public static int BúsquedaBin2(double[] m, double v)
{
    // El método BúsquedaBin2 devuelve como resultado la posición
    // del valor. Si el valor no se localiza devuelve -1.

```

```
if (m.Length == 0) return -1;
int mitad, inf = 0, sup = m.Length - 1;

do
{
    mitad = (inf + sup) / 2;
    if (v > m[mitad])
        inf = mitad + 1;
    else
        sup = mitad;
}
while ( inf < sup );

if (m[inf] == v)
    return inf;
else
    return -1;
}
```

En cada iteración, en ambos casos, se divide en partes iguales el intervalo entre los índices *inf* y *sup*. Por ello, el número necesario de comparaciones es como mucho $\log_2 n$, siendo n el número de elementos de la matriz. Hasta aquí el comportamiento de ambas versiones es el mismo, pero ¿qué pasa con la condición de la sentencia **while**? Se observa que en la primera versión dicha sentencia realiza dos comparaciones frente a una que realiza en la segunda versión, lo que se traducirá en un mayor tiempo de ejecución, resultando, por lo tanto, ser más eficiente la versión segunda.

La aplicación siguiente permite ver de una forma práctica que la versión segunda emplea menos tiempo de ejecución que la primera. Esta aplicación crea una matriz y, utilizando primero una versión y después la otra, realiza una búsqueda por cada uno de sus elementos y dos búsquedas más para dos valores no pertenecientes a la matriz, uno menor que el menor y otro mayor que el mayor. El tiempo de ejecución medido en milisegundos se obtiene por diferencia de los tiempos devueltos por la propiedad **DateTime.Now.Ticks** del espacio de nombres **System** al iniciar cada proceso de búsqueda y al finalizarlo.

```
class Test
{
    public static int BúsquedaBin1(double[] m, double v)
    {
        // Versión 1
    }

    public static int BúsquedaBin2(double[] m, double v)
    {
        // Versión 2
    }

    public static void Main(string[] args)
    {
        double[] a = new double[1000000];
        long ti = 0, tf = 0;
        int i = 0, n = a.Length;
        for (i = 0; i < n; i++)
            a[i] = i+1;

        // Versión 1
        ti = DateTime.Now.Ticks;
        i = BúsquedaBin1(a,0);
        for (i = 0; i < n; i++)
            i = BúsquedaBin1(a,i+1);
        i = BúsquedaBin1(a,n+1);
        tf = DateTime.Now.Ticks;
        Console.WriteLine((tf - ti)/10 + " milisegundos");

        // Versión 2
        ti = DateTime.Now.Ticks;
        i = BúsquedaBin2(a,0);
        for (i = 0; i < n; i++)
            i = BúsquedaBin2(a,i+1);
        i = BúsquedaBin2(a,n+1);
        tf = DateTime.Now.Ticks;
        Console.WriteLine((tf - ti)/10 + " milisegundos");
    }
}
```

EJERCICIOS PROPUESTOS

1. Un centro numérico es un número que separa una lista de números enteros (comenzando en 1) en dos grupos de números cuyas sumas son iguales. El primer centro numérico es el 6, el cual separa la lista (1 a 8) en los grupos: (1, 2, 3, 4, 5) y (7, 8) cuyas sumas son ambas iguales a 15. El segundo centro numérico es el 35, el cual separa la lista (1 a 49) en los grupos: (1 a 34) y (36 a 49) cuyas sumas son ambas iguales a 595. Escribir un programa que calcule los centros numéricos entre 1 y n .

El ejemplo (1 a 5) 6 (7 a 8), donde se observa que 6 es un centro numérico, sugiere ir probando si los valores 3, 4, 5, 6,..., cn ,..., $n-1$ son centros numéricos. En general cn es un centro numérico si la suma de todos los valores enteros desde 1 a $cn-1$ coincide con la suma desde $cn+1$ a lim_sup_grupo2 (límite superior del grupo segundo de números). Para que el programa sea eficiente, buscaremos el valor lim_sup_grupo2 entre los valores $cn+1$ y $n-1$ utilizando el método de *búsqueda binaria*. Recuerde que la suma de los valores enteros entre 1 y x viene dada por la expresión $(x * (x + 1))/2$.

CAPÍTULO 18

© F.J.Ceballos/RA-MA

HILOS

Uno de los pasos importantes que la informática dio en favor de los desarrolladores de software fue colocar un nivel de software por encima del hardware de un ordenador. Este nivel de software, conocido como sistema operativo, es en esencia una interfaz fácil de utilizar que nos permite controlar todas las partes del hardware, en la mayoría de los casos, sin un profundo conocimiento del mismo.

A su vez, los sistemas operativos también han experimentado un gran avance, pasando de los sistemas de un único procesador a los actuales sistemas operativos distribuidos o de red, o a los sistemas operativos con multiprocesadores. Esta evolución ha desembocado en un mejor aprovechamiento de todos los recursos disponibles, permitiéndonos ejecutar cada vez más tareas en menos tiempo.

El concepto central de cualquier sistema operativo es el de *proceso*. Cualquier ordenador hoy en día es capaz de hacer varias cosas simultáneamente; por ejemplo, puede estar imprimiendo un documento por la impresora y ejecutando un programa del usuario. Esto requiere que la UCP (unidad central de proceso) alterne de un programa a otro en muy cortos espacios de tiempo, lo que conocemos como tiempo compartido. De esta forma, todos los programas, incluyendo los que componen el sistema operativo, que tengan que ejecutarse simultáneamente (multiprogramación) se organizan en varios procesos secuenciales.

CONCEPTO DE PROCESO

Un proceso es un programa en ejecución. También, un mismo programa puede derivar en varios procesos; por ejemplo, podemos tener varios ejemplares en ejecución del programa *Microsoft Word*, y también un mismo programa puede que necesite utilizar diferentes procesos, por ejemplo, *Word* puede requerir ejecutar el editor de ecuaciones. Cada proceso consta de bloques de código y de datos carga-

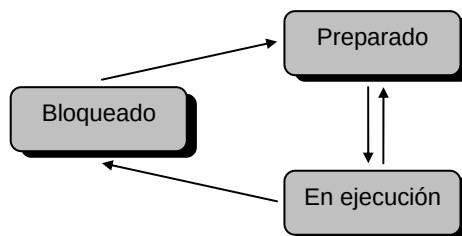
dos desde un fichero ejecutable o desde una biblioteca dinámica. También es propietario de otros recursos que se crean durante la vida de dicho proceso y se destruyen cuando finaliza. Por ejemplo, un proceso posee:

- su propio espacio de direcciones,
- sus variables,
- ficheros abiertos,
- referencias a procesos hijo,
- contador de programa, registros, pila,
- elementos de sincronismo, etc.

Lo anterior es equivalente a decir que cada proceso tiene su propia UCP virtual, lo que nos permite comprender mejor cómo un sistema puede ejecutar varios procesos simultáneamente, aunque la realidad es que los procesos, controlados por el sistema operativo, se van alternando en la utilización de la UCP.

Según lo expuesto, sería un error confundir un programa con un proceso. Para evitar este posible malentendido, considere el siguiente ejemplo: cuando instalamos un juego en nuestro ordenador lo hacemos siguiendo las instrucciones adjuntas. En este caso, las instrucciones serían el programa, la actividad que hay que desarrollar para realizar la instalación (leer las instrucciones, introducir el CD-ROM, etc.) el proceso y nosotros la UCP. Si mientras estamos desarrollando esta actividad, alguien solicita nuestra colaboración para otra cosa, registramos el punto en el que nos encontramos y acudimos a resolver lo propuesto. En este caso, la UCP alterna de un proceso a otro.

De lo anterior se deduce que un proceso puede estar en *ejecución* (está utilizando la UCP), *preparado* (está detenido temporalmente para que se ejecute otro proceso) o *bloqueado* (el proceso está esperando que ocurra algo para continuar; por ejemplo, datos de la entrada estándar). Entre estos tres estados son posibles, como muestra la figura siguiente, cuatro transiciones:



La parte del sistema operativo encargada de realizar la conmutación entre procesos, y de esta forma repartir la utilización de la UCP, es el *planificador*. Así, si un proceso en *ejecución* no puede continuar, pasa al estado de *bloqueado* o

también, si puede continuar y el planificador decide que ya ha sido ejecutado el tiempo suficiente, pasa al estado de *preparado*. Si el proceso está *bloqueado*, pasará a *preparado* cuando se dé el evento externo por el que se bloqueó, y si está *preparado*, pasa a *ejecución* cuando el planificador lo decida porque los demás procesos ya han tenido su parte de tiempo de UCP.

En la UCP puede haber varios programas con varios procesos ejecutándose concurrentemente (múltiples procesos ejecutándose simultáneamente), lo que supone aprovechar mejor no sólo el procesador, sino todos los recursos de la máquina en general, que serán compartidos por todos los procesos; por ejemplo, mientras un programa está llevando a cabo la impresión de un documento, otro puede estar haciendo ciertos cálculos y otro grabando ciertos registros en un fichero en disco. En este caso se utilizan distintos mecanismos para la sincronización y comunicación entre procesos. Tales conceptos son objeto de estudio en sistemas operativos.

Para clarificar lo expuesto, veamos con un ejemplo sencillo cómo un programa C# puede verse como un proceso cuando se ejecuta.

```
using System;

////////////////////////////////////
// Procesos.
//
public class Test
{
    // Un programa C# visto como un proceso

    public static int NRandom(int n)
    {
        Random rnd = new Random();
        int límiteSup = n, límiteInf = 1;
        return (int)((límiteSup - límiteInf + 1) * rnd.NextDouble() +
                    límiteInf); // valor entre 1 y n
    }

    public static void Esperar(int n)
    {
        long tmi = DateTime.Now.Ticks + n; // ticks
        while (DateTime.Now.Ticks < tmi) ; // esperar n ticks
    }

    public static void TomarMuestraTipoA(out int n)
    {
        n = NRandom(10);
        for (int i = 0; i < n; i++)
        {
            Console.WriteLine("Tomando muestra de tipo A");
        }
    }
}
```

```
        Esperar(500);
    }
}

public static void TomarMuestraTipoB(out int n)
{
    n = NRandom(20);
    for (int i = 0; i < n; i++)
    {
        Console.WriteLine("Tomando muestra de tipo B");
        Esperar(500);
    }
}

public static void Resultados(int n, int m)
{
    Console.WriteLine("Muestras de tipo A: " + n);
    Console.WriteLine("Muestras de tipo B: " + m);
    Console.WriteLine("Total: " + (n + m));
}

public static void Main(string[] args)
{
    int nMuestrasTipoA = 0;
    int nMuestrasTipoB = 0;

    TomarMuestraTipoA(out nMuestrasTipoA);
    TomarMuestraTipoB(out nMuestrasTipoB);
    Resultados(nMuestrasTipoA, nMuestrasTipoB);
}
}
```

Un *tick* o paso equivale a 100 nanosegundos.

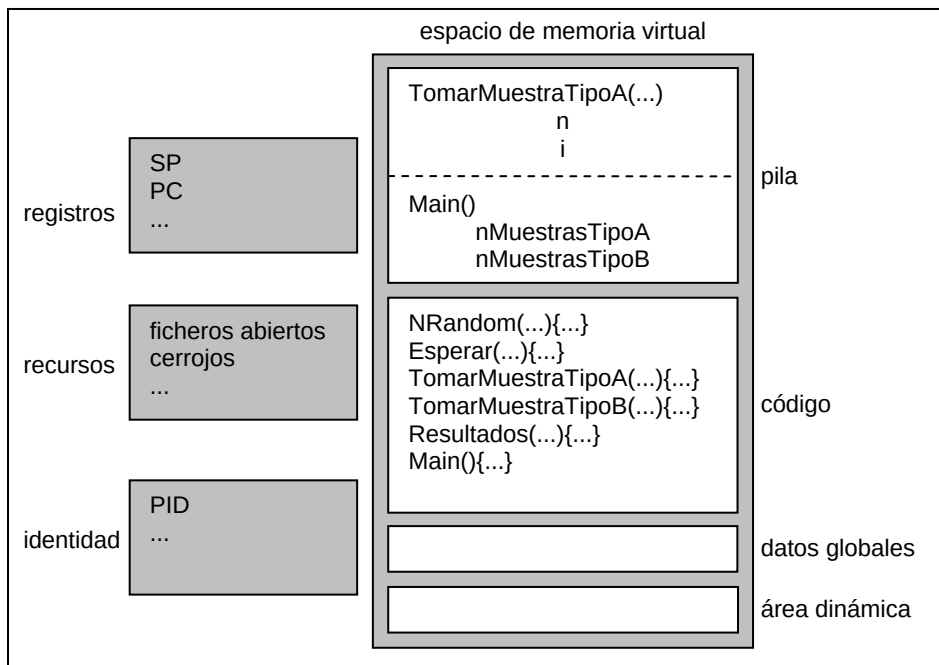
Cuando ejecute este programa observará un resultado como el siguiente:

```
Tomando muestra de tipo A
Tomando muestra de tipo A
Tomando muestra de tipo A
...
Tomando muestra de tipo B
Muestras de tipo A: 6
Muestras de tipo B: 12
Total: 18
```

La figura siguiente muestra el esquema de este programa en el espacio de memoria virtual de un proceso (cada proceso tiene su zona de memoria propia e independiente). Se puede observar que existen varias regiones de memoria (la figura muestra el instante en el que se ejecuta *TomarMuestraTipoA*):

- *Pila*: área de memoria donde se mantienen las variables locales y la información necesaria para continuar con el método llamante.
- *Código*: área de memoria donde se mantiene el código del programa.
- *Datos globales*: área de memoria donde se mantienen los datos globales.
- *Área dinámica*: área de memoria para asignación dinámica.

Para completar la relación de recursos del sistema necesarios para mantener el proceso a que da lugar la ejecución del programa anterior, hay que añadir los registros del procesador (el contador de programa, el puntero de la pila, etc.), las tablas que el sistema operativo mantiene para seguir la pista de los ficheros abiertos, de los elementos de sincronismo, etc.



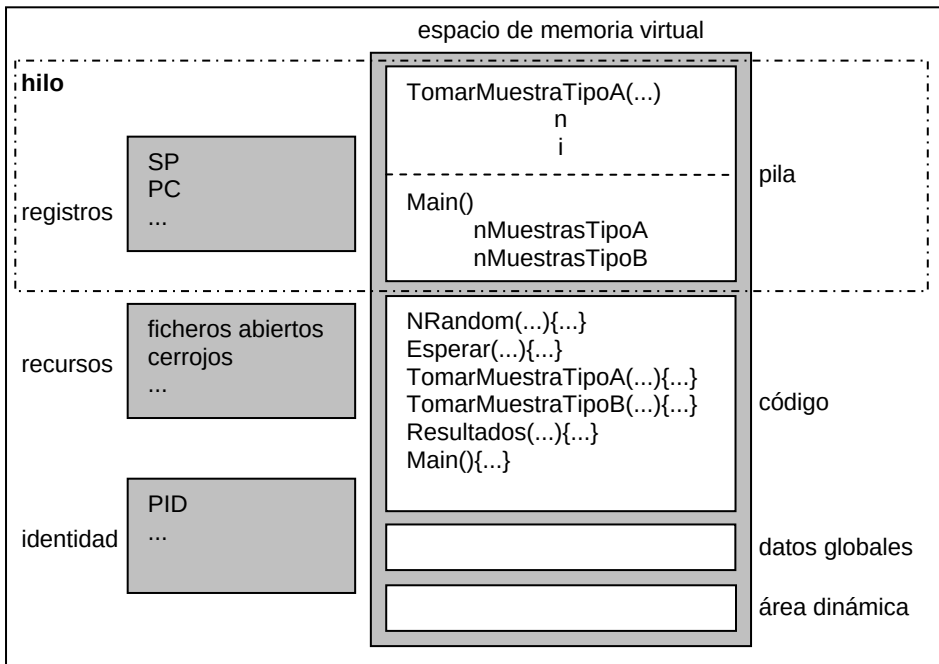
Estos recursos son privados para cada proceso; por ejemplo, un proceso no puede violar el espacio de direcciones de otro proceso, abrir y cerrar ficheros de otro, etc. Según esto, cada proceso es una entidad independiente, lo que significa que cuando coexistan varios procesos, se ejecutarán en espacios de memoria disjuntos y a la vez protegidos para que no puedan interferirse. No obstante, en muchas ocasiones es necesario que los procesos se comuniquen para poder intercambiar información, lo que resulta complicado al no compartir memoria. En

estos casos, los procesos podrán efectuar esa comunicación utilizando alguna de las estructuras que sí se comparten, por ejemplo el sistema de archivos.

Otra solución para salvar la dificultad que plantean los procesos para comunicarse entre sí es la utilización de hilos, porque la comunicación entre los mismos resulta muy sencilla frente a la comunicación entre procesos. El concepto de hilo lo estudiamos a continuación. Se trata de otro modelo de ejecución que también brinda la posibilidad de realizar programación concurrente.

HILOS

Un *hilo* (*thread*, llamado también proceso ligero) es la unidad de ejecución de un proceso y está asociado con una secuencia de instrucciones, un conjunto de registros y una pila. Normalmente, cuando se crea un proceso, el sistema operativo crea su primer hilo (hilo primario) el cual puede, a su vez, crear hilos adicionales. Desde este punto de vista, un proceso no se ejecuta, sino que es sólo el espacio de direcciones donde reside el código que es ejecutado mediante uno o más hilos.



Dicho de otra forma, un proceso puede ejecutar varias secuencias de instrucciones concurrentemente (simultáneamente), cada una de las cuales es un hilo que se ejecuta de manera independiente, aunque compartiendo el espacio de memoria

virtual del proceso y los recursos asignados por el sistema operativo al mismo. Sin embargo, cada hilo posee su propio contador de programa, además de otros registros, y su pila.

Según se ha expuesto en el apartado anterior, en un sistema operativo tradicional, cada proceso tiene un espacio de direcciones y un único *hilo* (este hilo lo denominaremos hilo primario para distinguirlo de los hilos por él creados que denominaremos hilos secundarios; no obstante, no hay clases de hilos, sólo hay hilos). Por ejemplo, considere un programa que incluya la siguiente secuencia de operaciones para actualizar el saldo de una cuenta bancaria cuando se efectúa un nuevo ingreso:

```
saldo = Cuenta.ObtenerSaldo();
saldo += ingreso;
Cuenta.EstablecerSaldo( saldo );
```

Cuando se ejecute el programa mencionado, se creará un proceso, en el que se ejecuta sólo un *hilo*, que ejecutará la secuencia de instrucciones programadas; este modelo de programación es en el que estamos acostumbrados a trabajar habitualmente. Pero, piense ahora en una entidad bancaria real; en ella, varios cajeros pueden actuar simultáneamente. Ejecutar el mismo programa por cada uno de los cajeros tiene un coste elevado (recuerde los recursos que necesita un proceso). En cambio, si el programa permitiera lanzar un hilo secundario por cada petición que hiciera un cajero para actualizar una cuenta, estaríamos en el caso de múltiples hilos ejecutándose concurrentemente (*multithreading*). Esta característica ya es una realidad en los sistemas operativos de hoy en día, proporcionada por bibliotecas nativas o de terceros. Precisamente, el objetivo de este capítulo es estudiar cómo crear hilos en nuestros programas.

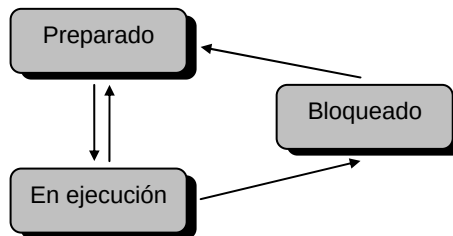
Como ya hemos indicado, cada hilo se ejecuta de forma estrictamente secuencial y tiene su propia pila, el estado de los registros de la UCP y su propio contador de programa. En cambio, comparten el mismo espacio de direcciones, lo que significa compartir también las mismas variables globales, el mismo conjunto de ficheros abiertos, procesos hijos (no hilos hijo), elementos de sincronización, etc. Entonces, ¿qué ventajas aporta un hilo respecto a un proceso? Atendiendo a los recursos propios de un hilo, su lanzamiento y su ejecución es mucho más económico que el lanzamiento y la ejecución de un proceso, y atendiendo a los recursos compartidos, la comunicación entre hilos es trivial. Por otra parte, muchos problemas pueden ser resueltos mejor con múltiples hilos; y si no, piense cómo escribiría un programa con un solo hilo de control para mostrar animación, sonido, visualizar documentos y traer ficheros de *Internet*, al mismo tiempo. No obstante, habrá situaciones en las que la mejor solución para ayudar en el trabajo sea crear un nuevo proceso (proceso hijo).

Los hilos comparten la UCP de la misma forma que lo hacen los procesos, pueden crear otros hilos y se pueden bloquear. Un ejemplo, imaginemos que alguien llega a un cajero para depositar dinero en una cuenta y casi al mismo tiempo, un segundo cliente inicia la misma operación sobre la misma cuenta en otro cajero. Para que los resultados sean correctos, el segundo cajero deberá quedar bloqueado hasta que el registro que está siendo actualizado por el primer cajero quede liberado; de no ser así, ambas operaciones podrían tomar como punto de partida el mismo saldo, el actual, lo que conduciría a un resultado erróneo (este ejemplo quedará más claro cuando estudiemos sincronización de hilos).

Resumiendo, sabemos que en la UCP puede haber varios programas con varios procesos ejecutándose concurrentemente, habilidad que se denomina multitarea, y a su vez, un proceso puede crear varios hilos y ejecutarlos de forma concurrente, lo que se traduce básicamente en una multitarea dentro de multitarea: el usuario sabe que puede ejecutar varias aplicaciones simultáneamente, y el programador sabe que cada aplicación puede ejecutar varios hilos a la vez.

Estados de un hilo

Igual que los procesos con un solo hilo de control, los *hilos* pueden encontrarse en uno de los siguientes estados:



- *Nuevo*. El hilo ha sido creado pero aún no ha sido activado. Cuando se active pasará al estado *preparado*.
- *Preparado*. El hilo está activo y está a la espera de que le sea asignada la UCP.
- *En ejecución*. El hilo está activo y le ha sido asignada la UCP (sólo los hilos activos, *preparados*, pueden ser ejecutados).
- *Bloqueado*. El hilo espera que otro elimine el bloqueo. Un hilo *bloqueado* puede estar:
 - ◊ *Dormido*. El hilo está bloqueado durante una cantidad de tiempo determinada (por ejemplo, tres segundos), después de la cual despertará y pasará al estado *preparado*.

- ◊ *Esperando*. El hilo está esperando a que ocurra alguna cosa: una condición, una operación de E/S o adquirir la propiedad de un objeto de sincronismo. Cuando ocurra, pasará al estado *preparado*.
- *Muerto*. El hilo ha finalizado (está muerto) pero todavía no ha sido recogido por su padre. Los hilos *muertos* no pueden alcanzar ningún otro estado.

Obsérvese en la figura anterior que no se muestran los estados *nuevo* y *muerto* ya que no son estados de transición durante la vida del hilo; esto es, no se puede transitar al estado *nuevo* ni desde el estado *muerto*.

La transición entre estados está controlada por el planificador: parte del núcleo del sistema operativo encargada de que todos los hilos que esperan ejecutarse tengan su oportunidad. Si un hilo *en ejecución* no puede continuar, pasará al estado *bloqueado*; o también, si puede continuar y el planificador decide que ya ha sido ejecutado el tiempo suficiente, pasará al estado *preparado*. Si el proceso está *bloqueado*, pasará a *preparado* cuando se dé el evento por el que espera; por ejemplo, puede estar esperando a que otro hilo elimine el bloqueo, o bien si está *dormido*, esperará a que pase el tiempo por el que fue enviado a este estado para ser activado; y si está *preparado*, pasará a *ejecución* cuando el planificador lo decida porque los demás hilos ya han tenido su parte de tiempo de UCP.

Cuándo se debe crear un hilo

Según lo expuesto anteriormente, cada vez que se crea un proceso, el sistema operativo crea un hilo primario. Para muchos procesos éste es el único hilo necesario. Sin embargo, un proceso puede crear otros hilos para ayudarse en su trabajo, utilizando la UCP al máximo posible. Por ejemplo, supongamos el diseño de una aplicación procesador de texto. ¿Sería acertado crear un hilo separado para manipular cualquier tarea de impresión? Esto permitiría al usuario continuar utilizando la aplicación mientras se está imprimiendo el documento. En cambio, ¿qué sucederá si los datos del documento cambian mientras se imprime? Éste es un problema que habría que resolver, quizás creando un fichero temporal que contenga los datos a imprimir.

Es evidente que los hilos son extraordinariamente útiles, pero también es evidente que si no se utilizan adecuadamente pueden introducir nuevos problemas mientras tratamos de resolver otros más antiguos. Por lo tanto, es un error pensar que la mejor forma de desarrollar una aplicación es dividirla en partes que se ejecuten cada una como un hilo.

PROGRAMAR CON HILOS

La mayoría del soporte que C# proporciona para trabajar con hilos reside en la clase **Thread** del espacio de nombres **System.Threading**. Además de la clase **Thread**, que permite definir un hilo, y del delegado **ThreadStart**, que se emplea para especificar el punto de entrada al hilo (para crear un hilo, este delegado se pasa como parámetro al constructor de la clase **Thread**), **System.Threading** proporciona clases para la sincronización de hilos y del acceso a datos, tales como **Mutex**, **Monitor**, **Interlocked** o **AutoResetEvent**, y también incluye una clase **ThreadPool** que permite utilizar un grupo de hilos suministrados por el sistema y una clase **Timer** que ejecuta métodos de devolución de llamada en hilos del grupo de hilos.

Clase Thread

Un proceso es un programa en ejecución. Al crear un proceso del sistema operativo, éste introduce un hilo (hilo principal) para ejecutar el código de dicho proceso. A partir de ese punto, pueden crearse y destruirse otros hilos en el dominio de la aplicación (hilos secundarios o trabajadores).

Un hilo, también llamado subproceso ligero, es un objeto de la clase **Thread**. El constructor de **Thread** acepta como único parámetro un delegado **ThreadStart** que contiene una referencia al método que se invocará mediante el objeto **Thread** cuando llame a su método **Start**. Si se llama a **Start** más de una vez, se lanzará una excepción **ThreadStateException**.

A lo largo de su existencia, un hilo siempre se encuentra en uno o más estados definidos por **ThreadState**. Se puede solicitar un nivel de prioridad de programación (tal y como lo define **ThreadPriority**) para un hilo, pero no se garantiza que el sistema operativo lo conceda.

Resumen de los métodos y propiedades de Thread

A continuación se resumen algunos de los métodos que se pueden utilizar para controlar hilos independientes:

- **Start**. Inicia la ejecución de un hilo.
- **Sleep**. Detiene un hilo durante un tiempo determinado.
- **Abort**. Detiene un hilo cuando alcanza un punto seguro.
- **Join**. Deja en espera un hilo hasta que finaliza otro hilo diferente. Si se utiliza con un valor de tiempo de espera, este método devolverá **true** cuando el hilo finaliza en el tiempo asignado.

En la versión 2.0 de .NET Framework, los métodos **Suspend** (interrumpe un hilo cuando alcanza un punto seguro) y **Resume** (reinicia un hilo suspendido) de la clase **Thread** fueron marcados como obsoletos.

Así mismo, los hilos contienen varias propiedades útiles, algunas de las cuales resumimos a continuación:

- **IsAlive**. Vale **true** si un hilo se encuentra activo.
- **IsBackground**. Permite especificar si un hilo es o no un hilo en segundo plano. Por omisión su valor es **false**.
- **Name**. Permite obtener o establecer el nombre de un hilo. Se utiliza principalmente con fines de depuración.
- **Priority**. Permite obtener o asignar a un hilo uno de los siguientes valores de prioridad definidos por la enumeración **ThreadPriority**: *Highest*, *AboveNormal*, *Normal*, *BelowNormal* y *Lowest*. Los sistemas operativos no están obligados a tener en cuenta la prioridad que se establezca para un hilo.
- **ThreadState**. Describe el estado o estados de un hilo.

Cuando se crea un hilo, éste pasa al estado **Unstarted**, que conserva hasta que cambia al estado iniciado mediante una llamada al método **Start**. En la tabla siguiente se muestran las acciones que pueden provocar un cambio de estado, junto con el nuevo estado correspondiente.

Acción	Nuevo estado resultante
Otro hilo llama a Thread.Start	No cambia
El hilo responde a Thread.Start y empieza a ejecutarse	Running
El hilo llama a Thread.Sleep	WaitSleepJoin
El hilo llama a Monitor.Wait en otro objeto	WaitSleepJoin
El hilo llama a Thread.Join en otro hilo	WaitSleepJoin
Otro hilo llama a Thread.Interrupt	Running
Otro hilo llama a Thread.Abort	AbortRequested
El hilo responde a un Thread.Abort	Aborted

Los estados indicados en la tabla anterior están definidos en la enumeración **ThreadState** del espacio de nombres **System.Threading**.

Así mismo, la propiedad **ThreadState** de un hilo proporciona el estado actual del mismo. Ahora bien, como el valor de **Running** es cero, para comprobar si un hilo está ejecutándose hay que utilizar una expresión como la siguiente:

```
if ((hilo.ThreadState &
    (ThreadState.Stopped | ThreadState.Unstarted)) == 0)
    // ...
```

CREACIÓN Y DESTRUCCIÓN DE HILOS

Para crear, iniciar y finalizar un hilo siga los pasos indicados a continuación:

1. Escriba el código que ejecutará el hilo en un método con el prototipo especificado por el delegado **ThreadStart**, sin parámetros, o **ParameterizedThreadStart**, con parámetros (los delegados fueron expuestos en el capítulo 10):

```
public delegate void ThreadStart();
public delegate void ParameterizedThreadStart(Object obj);
```

Código que ejecutará el hilo:

```
private void TareaSecundaria() // sin parámetros
{
    // Código que ejecuta el hilo
}
```

2. Defina un identificador de tipo **Thread** para el hilo:

```
Thread hiloSecundario; // identificador del hilo
```

3. Escriba el código para crear el hilo:

```
hiloSecundario = new Thread(TareaSecundaria);
```

4. Escriba el código para iniciar su ejecución:

```
hiloSecundario.Start();
```

5. Escriba el código que haga que el hilo primario espere a que finalice la ejecución del hilo secundario y libere los recursos, finalizando así el proceso:

```
hiloSecundario.Join();
```

Esta llamada, en muchos casos, no es necesaria porque una vez que finalicen todos los hilos secundarios, la máquina virtual de .NET finalizará el proceso bajo el que se ejecutan; esto es, finalizará el hilo principal.

Los puntos anteriores agrupados en una clase se verían de la forma siguiente:

```
using System;
using System.Threading;
////////////////////////////////////
// Hilos.
//
```



```

public class CHilo
{
    // Clase que define un hilo secundario
    private Thread hilo; // identificador del hilo

    public CHilo() : this(null) {}

    public CHilo(string nombreHilo)
    {
        // Crear el hilo identificado por hilo
        hilo = new Thread(ProchHilo);
        if (nombreHilo != null) hilo.Name = nombreHilo;
        // Iniciar el hilo
        hilo.Start();
    }

    public Thread HiloSubyacente
    {
        get { return hilo; } // referencia al objeto Thread
    }

    private void ProchHilo()
    {
        // Código que ejecuta el hilo
        Console.WriteLine("El hilo finalizó");
    }
}
////////////////////

```

La clase *CHilo* encapsula un objeto **Thread**, un hilo referenciado por *hilo*. El constructor de esta clase crea el hilo, opcionalmente permite asignarle un nombre, e inicia la ejecución del mismo. También define los métodos *HiloSubyacente* y *ProchHilo*. El primero devuelve una referencia al objeto **Thread** encapsulado, lo que nos permitirá acceder a los métodos del mismo, y el segundo es el proceso que ejecutará el hilo.

Ahora, cuando desde un programa creamos un objeto *CHilo*, el constructor de esta clase creará un hilo y lo ejecutará:

```

public class Test
{
    // Hilo primario
    public static void Main()
    {
        CHilo hiloSecundario = new CHilo();
        // Esperar a que el hilo secundario finalice
        hiloSecundario.HiloSubyacente.Join();
    }
}

```

Otra opción es modificar el constructor *CHilo* para que no inicie el hilo y dar libertad así al usuario de que lo inicie cuando quiera:

```
public CHilo(string nombreHilo)
{
    // Crear el hilo identificado por hilo
    hilo = new Thread(ProcHilo);
    if (nombreHilo != null) hilo.Name = nombreHilo;
}
```

Ahora la clase *Test* tendrá que iniciar explícitamente la ejecución del hilo. Obsérvese que en este otro ejemplo no invocamos a **Join**.

```
public class Test
{
    // Hilo primario
    public static void Main()
    {
        CHilo hiloSecundario = new CHilo();
        // Iniciar la ejecución del hilo
        hiloSecundario.HiloSubyacente.Start();
    }
}
```

Parámetros del método que ejecuta el hilo

El delegado **ParameterizedThreadStart** indica que el procedimiento que ejecuta el hilo puede también tener un parámetro de tipo **Object** para almacenar los datos que se le pasarán cuando se inicie el hilo. Por ejemplo:

```
private void ProcHilo(Object datos)
{
    // Código que ejecuta el hilo
    if (datos != null)
        Console.WriteLine(datos);
    Console.WriteLine(hilo.Name + " finalizó");
}
```

¿Quién pasa los datos al parámetro *datos* del procedimiento del hilo? Evidentemente el método que lo invoca y éste no es otro que **Start**. Para verlo prácticamente, vamos a modificar la clase *Test* así:

```
public class Test
{
    // Hilo primario
    public static void Main()
    {
        CHilo hiloSecundario = new CHilo("Mi hilo");
```

```

        string s = hiloSecundario.HiloSubyacente.Name +
            " inició la ejecución";
        // Iniciar la ejecución del hilo
        hiloSecundario.HiloSubyacente.Start(s);
    }
}

```

El resultado será ahora el siguiente:

Mi hilo inició su ejecución

Mi hilo finalizó

Un ejemplo

Para clarificar lo estudiado hasta ahora, veamos con un ejemplo sencillo cómo un programa C# puede verse como un proceso con varios hilos de ejecución. El programa, tomando como base la clase *CHilo* que acabamos de exponer, será la versión con hilos del programa *Test.cs* que realizamos como ejemplo al hablar de procesos.

```

using System;
using System.Threading;

////////////////////////////////////
// Hilos.
//
public class CHilo
{
    // Clase que define un hilo secundario

    private Thread hilo; // identificador del hilo
    private int nMuestras;

    public CHilo() : this(null) {}

    public CHilo(string nombreHilo)
    {
        nMuestras = 0;
        // Crear el hilo identificado por hilo
        hilo = new Thread(ProcHilo);
        if (nombreHilo != null) hilo.Name = nombreHilo;
    }

    public Thread HiloSubyacente
    {
        get { return hilo; } // referencia al objeto Thread
    }
}

```

```
private void ProcHilo(Object datos)
{
    nMuestras = NRandom((int)datos);
    for (int i = 0; i < nMuestras; i++)
    {
        Console.WriteLine(hilo.Name + " tomando muestra " + (i+1));
        Esperar(500); // ticks
    }
}

private int NRandom(int n)
{
    Random rnd = new Random();
    int límiteSup = n, límiteInf = 1;
    return (int)((límiteSup - límiteInf + 1) * rnd.NextDouble() +
        límiteInf); // valor entre 1 y n
}

private void Esperar(int n)
{
    long tmi = DateTime.Now.Ticks + n; // ticks
    while (DateTime.Now.Ticks < tmi) ; // esperar n ticks
}

public int MuestrasTomadas
{
    get { return nMuestras; }
}
}
////////////////////////////////////
```

Cuando construyamos un programa a partir de un objeto *CHilo* y lo ejecutemos, por ejemplo escribiendo la clase *Test* indicada a continuación,

```
using System;

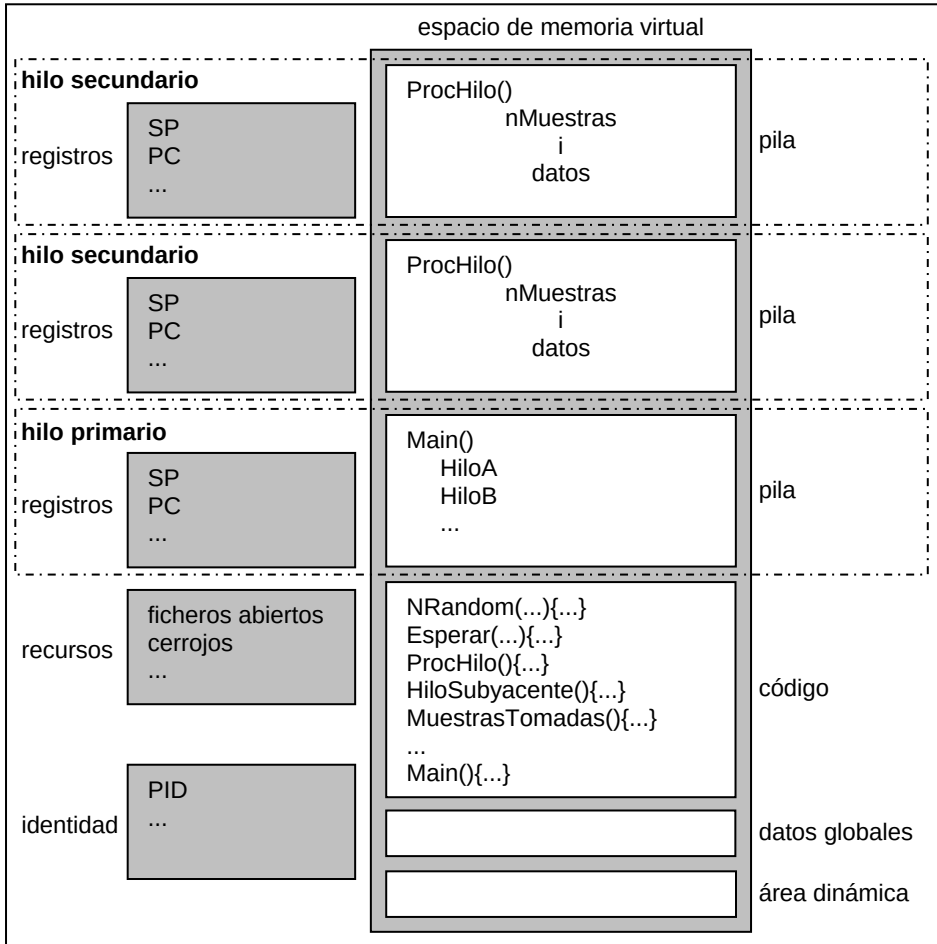
public class Test
{
    // Hilo primario
    public static void Main()
    {
        CHilo hiloA = new CHilo("Hilo A");
        CHilo hiloB = new CHilo("Hilo B");
        hiloA.HiloSubyacente.Start(10); // máximo 10 muestras
        hiloB.HiloSubyacente.Start(20); // máximo 20 muestras
        hiloA.HiloSubyacente.Join();
        hiloB.HiloSubyacente.Join();
        // Resultados
        Console.WriteLine(hiloA.HiloSubyacente.Name +
            " muestras tomadas: " + hiloA.MuestrasTomadas);
    }
}
```

```
        Console.WriteLine(hiloB.HiloSubyacente.Name +  
            " muestras tomadas: " + hiloB.MuestrasTomadas);  
        Console.WriteLine("Total: " + (hiloA.MuestrasTomadas +  
            hiloB.MuestrasTomadas));  
    }  
}
```

observará un resultado similar al siguiente:

```
Hilo A tomando muestra 1  
Hilo B tomando muestra 1  
Hilo A tomando muestra 2  
Hilo A tomando muestra 3  
Hilo B tomando muestra 2  
Hilo B tomando muestra 3  
Hilo B tomando muestra 4  
Hilo B tomando muestra 5  
Hilo B tomando muestra 6  
Hilo B tomando muestra 7  
Hilo B tomando muestra 8  
Hilo B tomando muestra 9  
Hilo B tomando muestra 10  
Hilo B tomando muestra 11  
Hilo B tomando muestra 12  
Hilo A tomando muestra 4  
Hilo A tomando muestra 5  
Hilo A tomando muestra 6  
Hilo A muestras tomadas: 6  
Hilo B muestras tomadas: 12  
Total: 18
```

La figura siguiente muestra el esquema de este programa en el espacio de memoria virtual de un proceso con un hilo de control y dos hilos secundarios.



Se puede observar que existen varias regiones de memoria:

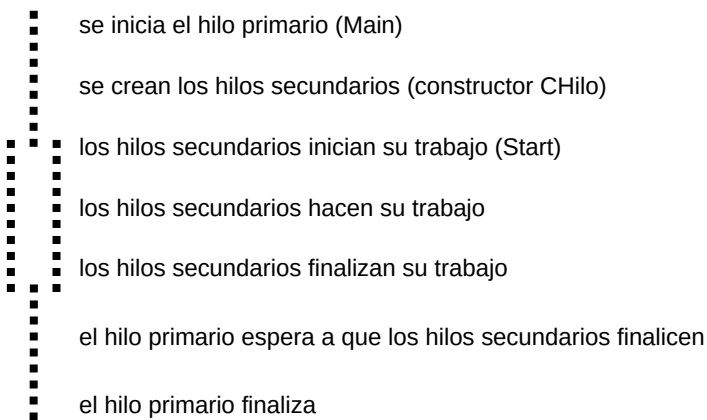
- *Pila*: área de memoria donde se mantienen las variables automáticas y la información necesaria para continuar con la ejecución del método que define el hilo. Hay una pila por hilo.
- *Código*: área de memoria donde se mantiene el código del programa.
- *Datos globales*: área de memoria donde se mantienen los datos globales.
- *Área dinámica*: área de memoria para asignación dinámica.

Para completar la relación de recursos del sistema necesarios para mantener el proceso a que da lugar la ejecución del programa anterior, hay que añadir los registros del procesador propios de cada hilo (el contador de programa, el puntero

de la pila, etc.), las tablas que el sistema operativo mantiene para seguir la pista de los ficheros abiertos, de los elementos de sincronismo, etc.

Esta otra figura que se muestra a continuación le ayudará a formarse una idea de cómo se ejecuta este programa desde un punto de vista teórico (ya que sólo tenemos un procesador físico).

Como se puede observar, la ejecución comienza en **Main** con un hilo primario, y después se crean dos hilos trabajadores identificados por *hiloA* e *hiloB*. Cada hilo secundario hace su trabajo (en el ejemplo, escriben un mensaje simulando que toman una muestra) mientras el hilo primario espera a que éstos finalicen. Finalmente, el hilo primario libera los recursos utilizados y el proceso termina.



Espera activa y pasiva

Cuando se diseña un hilo, lógicamente no sólo pensamos en el trabajo que tiene que desempeñar, sino en cómo su trabajo puede verse afectado por otros hilos. Por ejemplo, en el programa *Test.cs* expuesto anteriormente, la llamada por parte de los hilos al método *Esperar* provoca una espera activa, lo que significa que el hilo está consumiendo tiempo de UCP y, por lo tanto, entra en la planificación del sistema. Si se trata simplemente de esperar un tiempo, es mejor solución realizar la espera liberando la UCP para que ese tiempo pueda ser aprovechado por otros hilos.

En general un hilo realiza una espera pasiva poniéndose él mismo a dormir porque cuando está durmiendo, no entra en la planificación del sistema; esto es, el planificador no le asigna tiempo de UCP y, por consiguiente, detiene su ejecución. El método **Sleep** permite poner un hilo a dormir, el cual despertará cuando transcurra el tiempo especificado.

Como ejemplo, puede sustituir en el programa anterior la llamada al método *Esperar* de *CHilo* por una llamada al método **Sleep** de **Thread**

```
Thread.Sleep(50); // milisegundos
```

Finalizar un hilo

Un hilo termina de forma natural cuando su método asociado devuelve el control. Cuando esto sucede el hilo pasa al estado *muerto* (ha terminado) y no hay forma de salir de este estado. Esto es, una vez que el hilo está muerto, no puede ser arrancado otra vez; si deseamos ejecutar otra vez la tarea desempeñada por el hilo hay que construir un nuevo objeto hilo y enviarle el mensaje **Start**. Recuerde que si se llama a **Start** más de una vez, se lanzará una excepción **ThreadStateException**, por eso tenemos que construir el objeto hilo de nuevo.

Por ejemplo, supongamos una clase *Contador* que muestra un contador ascendente que será detenido cuando la cuenta alcance el valor del atributo *iteraciones*. La clase, además de este atributo y del método *ProcHilo* que muestra la cuenta, tiene dos constructores, uno con dos parámetros (nombre del hilo y número máximo de iteraciones) y otro sin parámetros (por omisión, sin nombre y 10 iteraciones), y un método *HiloCuenta* que devuelve una referencia al hilo.

```
using System;
using System.Threading;

////////////////////////////////////
// Clase que define un hilo.
//
public class Contador
{
    private Thread hilo;
    private int iteraciones;

    public Contador() : this(null, 10) { } // constructor

    public Contador(string nombre, int it) // constructor
    {
        iteraciones = it;
        hilo = new Thread(ProcHilo); // objeto hilo
        if (nombre != null) hilo.Name = nombre;
    }

    public Thread HiloSubyacente
    {
        get { return hilo; }
    }
}
```



```

public void ProcHilo()
{
    for (int i = 1; i <= iteraciones; i++)
    {
        Console.WriteLine("{0}, cuenta: {1}", hilo.Name, i);
        Thread.Sleep(250);
    }
    Console.WriteLine("Fin de la cuenta de {0}", hilo.Name);
}
}
//////////////////////////////////////////////////

```

La siguiente aplicación crea dos objetos *Contador*, cada uno de los cuales inicia un hilo:

```

using System;
//////////////////////////////////////////////////
// Hilos.
//
public class Test
{
    public static void Main(string[] args)
    {
        Contador contador1 = new Contador("Contador 1", 10);
        Contador contador2 = new Contador("Contador 2", 50);
        contador1.HiloSubyacente.Start();
        contador2.HiloSubyacente.Start();
    }
}
//////////////////////////////////////////////////

```

Cuando ejecutemos esta aplicación podremos observar cómo el “Contador 1” finaliza al alcanzar la cuenta de 10, y cómo el “Contador 2” continúa con su cuenta hasta llegar a 50. Una vez que concluyen todos los hilos en primer plano de un proceso, lo que hasta ahora hemos denominado hilos secundarios, la máquina virtual de .NET finalizará el proceso bajo el que se ejecutan; esto es, finalizará el hilo principal.

```

Contador 1, cuenta: 1
Contador 2, cuenta: 1
Contador 1, cuenta: 2
Contador 2, cuenta: 2
...
Contador 1, cuenta: 10
Contador 2, cuenta: 10
Fin de la cuenta de Contador 1
Contador 2, cuenta: 11
Contador 2, cuenta: 12
...

```

```
Contador 2, cuenta: 49
Contador 2, cuenta: 50
Fin de la cuenta de Contador 2
```

También podríamos añadir a la clase que define el hilo código que permitiera detener dicho hilo en un instante determinado. Veamos un ejemplo:

```
public class Contador
{
    private Thread hilo;
    private int iteraciones;
    private bool continuar;

    public Contador() : this(null, 10) { } // constructor

    public Contador(string nombre, int it) // constructor
    {
        continuar = true;
        iteraciones = it;
        hilo = new Thread(ProcHilo); // objeto hilo
        if (nombre != null) hilo.Name = nombre;
    }

    public Thread HiloSubyacente
    {
        get { return hilo; }
    }

    public void ProcHilo()
    {
        for (int i = 1; continuar && i <= iteraciones; i++)
        {
            Console.WriteLine("{0}, cuenta: {1}", hilo.Name, i);
            Thread.Sleep(250);
        }
        Console.WriteLine("Fin de la cuenta de {0}", hilo.Name);
    }

    public void Terminar()
    {
        continuar = false;
    }
}
```

La siguiente aplicación inicia un hilo de la clase *Contador*. El contador se detendrá cuando el usuario pulse la tecla *Entrar* porque, en este instante, el hilo invoca a su método *Terminar* que pone su atributo *cuenta* a **false** lo que hace que finalice la cuenta ejecutada por el método del hilo *ProcHilo*. Finalizado el hilo secundario, finaliza el hilo primario.

```

public class Test
{
    public static void Main(string[] args)
    {
        Console.WriteLine("Pulse [Entrar] para finalizar la cuenta");

        // Lanzar el hilo contador
        Contador contador = new Contador("Contador", 50);
        contador.HiloSubyacente.Start();
        // Permitir al hilo contador finalizar
        Console.ReadLine();
        contador.Terminar();
    }
}

```

Cuando se ejecuta el método **ReadLine**, el hilo principal pasa al estado bloqueado hasta que el usuario pulse la tecla *Entrar*.

Hilos en segundo plano

La propiedad **IsBackground** de un hilo permite obtener o establecer un valor booleano que indica si un hilo es o debería ser un hilo en segundo plano. Los hilos en segundo plano son como los hilos en primer plano que hemos visto hasta ahora, excepto que no impiden que finalice un proceso. Una vez que concluyen todos los hilos en primer plano de un proceso, la máquina virtual de .NET llama al método **Abort** de los hilos en segundo plano activos y finaliza dicho proceso. Para indicar que un hilo está en segundo plano, basta con asignar a su propiedad **IsBackground** el valor **true**. Por omisión esta propiedad vale **false**, indicando que el hilo está en primer plano.

Por ejemplo, vamos a añadir a la clase *Contador* el siguiente constructor con tres parámetros. En él se puede observar que el tercer parámetro de tipo **bool** permitirá asignar a la propiedad **IsBackground** del hilo el valor **true**, lo que le definirá como un hilo en segundo plano.

```

public Contador(string nombre, int it, bool segundoPlano)
    : this(nombre, it)
{
    hilo.IsBackground = segundoPlano;
}

```

Ahora, para ver los resultados, modifique el método **Main** de la clase *Test* como se indica a continuación:

```

public class Test
{

```

```

public static void Main(string[] args)
{
    Contador contador1 = new Contador("Contador 1", 10);
    Contador contador2 = new Contador("Contador 2", 50, true);
    contador1.HiloSubyacente.Start();
    contador2.HiloSubyacente.Start();
}
}

```

Cuando ahora ejecutemos esta aplicación podremos observar cómo el “Contador 1” finaliza al alcanzar la cuenta de 10 y, a diferencia de lo que vimos anteriormente, el “Contador 2” también es obligado a finalizar por tratarse de un hilo en segundo plano.

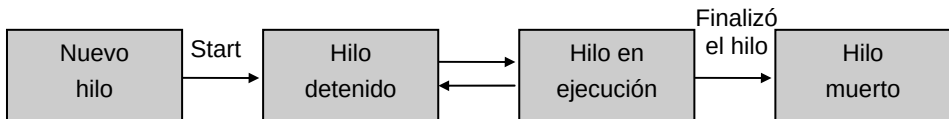
```

Contador 1, cuenta: 1
Contador 2, cuenta: 1
Contador 1, cuenta: 2
Contador 2, cuenta: 2
...
Contador 1, cuenta: 10
Contador 2, cuenta: 10
Fin de la cuenta de Contador 1
Contador 2, cuenta: 11

```

Ciclo de vida de un hilo

Ahora que ya hemos visto cómo realizar una determinada tarea utilizando un hilo, podemos deducir fácilmente que su ciclo de vida evoluciona según muestra la figura siguiente:



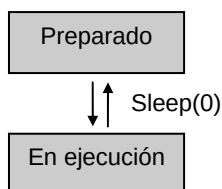
Un hilo, durante su ciclo de vida está transitando por los estados: *nuevo*, *preparado*, *en ejecución*, *bloqueado* y *muerto*, estudiados anteriormente. El estado *en ejecución* se corresponde en la figura con el bloque “hilo en ejecución”, el cual se alcanza desde el estado *preparado* al que pasa el hilo después de que haya sido creado y haber recibido el mensaje **Start**, y cuando el hilo está vivo y no está en ejecución es que está detenido (bloque “hilo detenido”).

Precisamente, el método **IsAlive** de **Thread** devuelve **true** si el hilo que recibe este mensaje ha sido arrancado (**Start**) y todavía no ha muerto.

Normalmente es el planificador el que controla cuándo un hilo debe estar en ejecución y cuándo pasa a estar detenido, pero en ocasiones tendremos que ser nosotros los que programemos las circunstancias bajo las cuales un hilo pueda pasar a ejecución, o bien deba pasar de ejecución a algunos de los estados *preparado* o *bloqueado* (bloqueado porque esté dormido, esté esperando a que otro hilo lo desbloquee, o esperando a que termine una operación de E/S, o bien esperando a apropiarse de un objeto de sincronismo).

Preparado

Al crear un hilo, éste se encuentra en el estado **Unstarted** y permanece en este estado hasta que el sistema operativo lo cambia al estado *preparado*, cosa que ocurre después de invocar a su método **Start**. Cuando se llama a **Start**, el sistema operativo sabe que el hilo está preparado para pasar al estado de ejecución (**Running**).



Un sistema operativo que admita multitareas prioritarias puede crear el efecto de ejecución simultánea de varios hilos desde varios procesos. Para ello, divide el tiempo de procesador disponible entre los hilos que lo necesitan, asignando un espacio de tiempo de procesador a cada hilo. Cuando transcurre este espacio de tiempo, el hilo que esté en ejecución actualmente se suspende para que otro hilo reanude su ejecución. Cuando el sistema cambia de un hilo a otro, guarda el contexto del hilo actual y vuelve a cargar el contexto guardado del siguiente hilo de la cola. Para suspender un hilo en ejecución de modo que permita que se ejecuten otros hilos en espera hay que invocar a *Thread.Sleep(0)*.

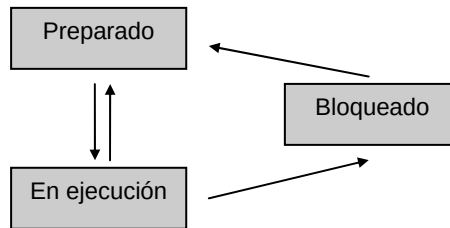
Bloqueado

Muchos métodos que ejecutan operaciones de entrada tienen que esperar por alguna circunstancia en el mundo exterior antes de que ellos puedan proseguir; este comportamiento se conoce como *bloqueo*. Por ejemplo, la sentencia siguiente lee un byte de la entrada estándar lanzando un hilo que ejecuta **Read**:

```
n = System.Console.Read();
```

Si en la entrada estándar hay un byte disponible la sentencia anterior se ejecuta satisfactoriamente y la ejecución del método que la contiene continúa. Sin em-

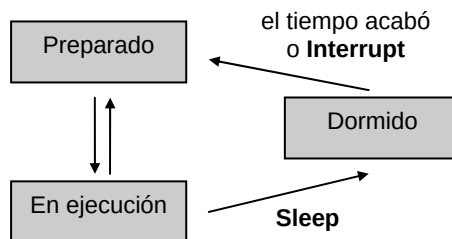
bargo, si no hay un byte disponible, **Read** tiene que esperar hasta que haya uno. Si el hilo que ejecuta **Read** se mantuviera en el estado de ejecución, la UCP quedaría ocupada y no se podría realizar nada más. En general, si un método necesita esperar una cantidad de tiempo indeterminada hasta que la ocurrencia que lo detiene tenga lugar, el hilo en ejecución debe salir de este estado. Todos los métodos C# que permiten leer datos se comportan de esta forma. Un hilo que gentilmente abandona el estado de ejecución hasta que se dé la ocurrencia que lo detiene se dice que está *bloqueado*.



C# implementa muchos de los bloqueos que ocurren durante una operación de E/S llamando a los métodos **Sleep** y **Wait** que veremos a continuación.

Dormido

Un hilo dormido pasa tiempo sin hacer nada, por lo tanto, no utiliza la UCP.



Una llamada al método **Sleep** solicita que el hilo actualmente en ejecución cese durante un tiempo especificado. Hay dos formas de llamar a este método:

```
public static void Thread.Sleep(int milisegundos);
public static void Thread.Sleep(TimeSpan t);
```

Especifique cero (0) para indicar que este hilo debe suspenderse de modo que permita que se ejecuten otros hilos en espera, o **Infinite** para bloquear el hilo indefinidamente.

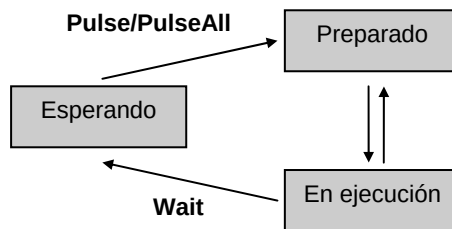
Se puede observar que el método **Sleep** es **static**. Dicho método opera sobre el hilo que actualmente se esté ejecutando.

La figura anterior indica que cuando un hilo despierta (el tiempo que tenía que dormir ha transcurrido) no continúa la ejecución, sino que se mueve al estado *preparado*. Pasará a ejecución cuando el planificador lo indique. Esto significa que una llamada a **Sleep** bloqueará un hilo por un tiempo superior al especificado.

La clase **Thread** proporciona también un método **Interrupt**. Cuando un hilo que está en el estado **WaitSleepJoin** (bloqueado como resultado de una llamada a **Wait**, **Sleep** o **Join**) recibe este mensaje, pasa automáticamente al estado *preparado*, y cuando pase a *ejecución*, ejecutará su manejador **ThreadInterruptedException**.

Esperando

Cuando un hilo llama a **Wait**, libera el bloqueo en el objeto de sincronización y entra en la cola de espera de ese objeto. El siguiente hilo de la cola de espera del objeto (si lo hubiera) adquiere el bloqueo y dispone de un uso exclusivo del mismo. Todos los hilos que llaman a **Wait** permanecen en la cola de espera hasta que reciben una señal de **Pulse** o **PulseAll**, enviada por el propietario del bloqueo. **Pulse** afecta al hilo situado al principio de la cola y **PulseAll** a todos los hilos de la cola. Cuando se recibe la señal, uno o más hilos abandonan la cola de espera y entran en la cola de hilos preparados pudiendo volver a adquirir el bloqueo.



SINCRONIZACIÓN DE HILOS

En los ejemplos que hemos visto hasta ahora cada hilo contenía todo lo que necesitaba para su ejecución: datos y métodos. Además, cada uno de ellos se podía ejecutar sin que interfiriera en la ejecución de cualquier otro hilo que se ejecutara concurrentemente con él. Estamos en el caso de *hilos independientes*.

Sin embargo, hay muchas situaciones en las que dos o más hilos ejecutándose concurrentemente deben acceder a los mismos recursos y/o datos. Como ejemplo, imagine la situación donde dos hilos acceden simultáneamente al mismo fichero de datos: un hilo para escribir datos en el fichero y otro para leerlos. Estamos en el caso de *hilos cooperantes*. Este tipo de situación puede crear resultados impre-

decibles, además de indeseables. En estos casos, simplemente se debe tomar el control de la situación y asegurar que cada hilo acceda a los recursos de una manera previsible, sincronizando las actividades que desarrollan cada uno de ellos, utilizando los mecanismos de sincronización más adecuados en cada caso.

En general, un hilo se sincroniza con otro hilo poniéndose él mismo a dormir. No obstante, antes de ponerse a dormir, debe poner en conocimiento del sistema qué evento debe ocurrir para reanudar su ejecución. De esta forma, cuando se produzca ese evento, el sistema despertará al hilo permitiéndole continuar la ejecución. Por ejemplo, si un hilo padre necesita esperar hasta que uno o más hilos hijo finalicen, se pone él mismo a dormir hasta que el hilo o hilos hijo pasen al estado *muerto*.

Cuando un hilo se pone a dormir (se bloquea), no entra en la planificación del sistema; esto es, el planificador no le asigna tiempo de UCP y, por consiguiente, detiene su ejecución.

Mecanismos de sincronización

Cuando se ejecuta un hilo hay que tener en cuenta la posible existencia de otros hilos que se ejecuten concurrentemente, especialmente si ese hilo comparte determinados recursos con estos otros. Por ello, y dado que los hilos se ejecutan en el mismo espacio de memoria dentro del proceso del que son “subprocesos”, hay que poner especial atención para evitar que dos hilos accedan a un recurso compartido al mismo tiempo. Este recurso compartido, usualmente, será un objeto y de no coordinar el acceso al mismo por los distintos hilos, el objeto puede terminar teniendo un estado no válido.

Además, la ejecución de un hilo puede depender del resultado de otros hilos; es el caso de hilos cooperantes, cooperación que se realizará a través de recursos compartidos.

La infraestructura de la máquina virtual de .NET proporciona diversas estrategias para sincronizar el acceso a objetos y miembros estáticos. Algunas de ellas se enumeran a continuación:

1. *Regiones de código sincronizado*. Se puede utilizar la clase **Monitor** o la instrucción **lock** para sincronizar sólo el bloque de código necesario.
2. *Sincronización manual*. Se pueden utilizar varios objetos de sincronización para crear mecanismos de sincronización propios.

3. **Métodos sincronizados.** Estos métodos son declarados en C# especificando el atributo **Synchronized**. Por ejemplo:

```
using System.Runtime.CompilerServices;
```

```
[MethodImplAttribute(MethodImplOptions.Synchronized)]
modificador tipo nombre_método([parámetros])
{
    // ...
}
```

4. Propiedad **Synchronized**. Algunas clases, como **Hashtable** y **Queue**, proporcionan una propiedad **Synchronized** que devuelve un contenedor seguro para hilos a un objeto de la clase.

Objetos de sincronización

La plataforma .NET proporciona varios objetos de sincronización que podemos utilizar para controlar las interacciones de los hilos y evitar las condiciones de carrera y otras anomalías que se puedan producir. Éstos pueden dividirse en tres categorías: *exclusión mutua*, *señalización* e *interbloqueo*. No obstante, algunos mecanismos de sincronización podrán ser utilizados en más de una categoría. También es importante recordar que la sincronización es cooperativa.

A continuación se resumen algunas de las clases de objetos de .NET Framework que se pueden utilizar para sincronizar la ejecución de varios hilos que comparten recursos comunes:

- **Monitor.** Un objeto de esta clase expone la capacidad de sincronizar el acceso a una región de código mediante la obtención y liberación de un bloqueo sobre un objeto concreto con los métodos **Monitor.Enter**, **Monitor.TryEnter** y **Monitor.Exit**. Una vez obtenido el bloqueo que habilita al hilo para ejecutar la región de código crítica, puede utilizar los métodos **Monitor.Wait** para liberar el bloqueo, si se mantiene, y espera su notificación, y **Monitor.Pulse** y **Monitor.PulseAll** para avisar al siguiente hilo en la cola de la cola de espera para continuar.
- **Mutex** (exclusión mutua). Son objetos de sincronización que sólo pueden ser propiedad de un solo hilo a la vez. Cuando un hilo necesita acceso a un recurso solicita la propiedad del objeto de exclusión mutua. Si está libre lo adquiere y accede al recurso y si está adquirido por otro hilo, espera a poder adquirirlo antes de utilizar el recurso. En este sentido, el método **WaitOne** hace que el hilo que lo invoque espere a poseer un objeto **Mutex** y el método **ReleaseMutex** permite liberarlo.

A diferencia de la clase **Monitor**, un **Mutex** puede ser local o global. Las exclusiones mutuas globales son visibles en todo el sistema operativo y se pueden utilizar para sincronizar hilos en varios procesos.

- **Semaphore**. Un semáforo a diferencia de un objeto de exclusión mutua puede controlar el acceso a varios recursos por distintos hilos simultáneamente. Para entrar en el semáforo, los hilos llaman al método **WaitOne** y para liberarlo llaman al método **Release**.
- **EventWaitHandle**. Esta clase permite a los hilos comunicarse con otros hilos por señalización. Un objeto **EventWaitHandle** es en realidad un controlador de espera de eventos, en estado señalizado o no señalizado. Pasará al estado señalizado para liberar uno o más hilos que están esperando por un recurso. Una vez esté en el estado señalizado, pasará a no señalizado de forma automática o manual.
- **AutoResetEvent**. Esta clase es una particularización de su clase base **EventWaitHandle**. Un objeto **AutoResetEvent** pasará automáticamente al estado no señalizado después de que un hilo que estaba bloqueado esperando por un recurso haya sido liberado para que accediera al mismo.
- **ManualResetEvent**. Esta clase es una particularización de su clase base **EventWaitHandle**. Un objeto **ManualResetEvent** permanecerá señalizado hasta que invoque a su método **Reset**. El método **Reset** cambia el estado de un objeto **ManualResetEvent** a no señalizado, ocasionando que los hilos que compitan por un recurso se bloqueen, y el método **Set** lo cambia a señalizado, permitiendo a uno o más hilos que están esperando por un recurso acceder al mismo.
- **Interlocked**. Ofrece operaciones atómicas para las variables compartidas por varios hilos.
- **ReaderWriterLock**. Define el bloqueo que implementa las semánticas de escritura única y de lectura múltiple.
- **Timer**. Ofrece un mecanismo para ejecutar tareas a intervalos específicos.
- **WaitHandle**. Esta clase es abstracta y es la clase base para los objetos de sincronización. Se derivan de esta clase **Mutex**, **EventWaitHandle** y **Semaphore**.

Secciones críticas

Una sección crítica es un segmento de código en la que se actualizan objetos de datos comunes a más de un hilo. Una forma sencilla de evitar los problemas que pudieran surgir por el acceso simultáneo de dos o más hilos a ese segmento de código es utilizando el mecanismo de exclusión mutua. Este mecanismo garantiza que la sección crítica será ejecutada sólo por un hilo cada vez y se puede implementar utilizando cerrojos (**lock**) o monitores (**Monitor**).

La forma más simple de crear un cerrojo es utilizando la sentencia **lock** (**SyncLock** en Visual Basic). Esta sentencia bloquea el acceso a la sección crítica de código. Un hilo que acceda a la sección crítica cuando el cerrojo está echado por-que la está ejecutando otro hilo se bloquea hasta que el cerrojo sea liberado.

```
private Object obj1 = new Object();
// ...
```

```
lock (obj1)
{
    // Sección de código crítica
}
```

La sentencia **lock** establece un bloqueo de exclusión mutua sobre el objeto a sincronizar. Este objeto (en el ejemplo *obj1*) será bloqueado por el hilo que desea ejecutar la sección crítica y el bloqueo será liberado cuando finalice la ejecución de la misma, momento en el que el objeto podrá ser bloqueado por otro hilo que esté esperando para ejecutar esa sección crítica. El objeto se utiliza para definir el ámbito del bloqueo (en el ejemplo anterior el bloqueo se limita al ámbito de *obj1*), por eso se recomienda definirlo **private** con el fin de evitar que cualquier otro código de la aplicación que tuviera acceso al mismo, en el caso de ser público, comparta el mismo bloqueo, lo que podría crear situaciones de interbloqueo, en las que dos o más hilos esperan a que se libere el mismo objeto. La sentencia **lock** ha sido implementada utilizando los métodos **Enter** y **Exit** de un objeto **Monitor**, y utilizando la sentencia **try...catch...finally** para asegurar que el bloqueo es liberado. A continuación se muestra un ejemplo de cómo se hace esto:

```
private Object obj1 = new Object();
// ...
```

```
try
{
    do
    {
        Monitor.Enter(obj1);

        // Sección de código crítica
    }
}
```

```
        Monitor.Exit(obj1);
    }
    while (condición);
}
catch (Exception ex)
{
    Debug.WriteLine("Error inesperado en el hilo " + idHilo);
    Debug.WriteLine(ex.Message);
}
finally
{
    Monitor.Exit(obj1);
}
```

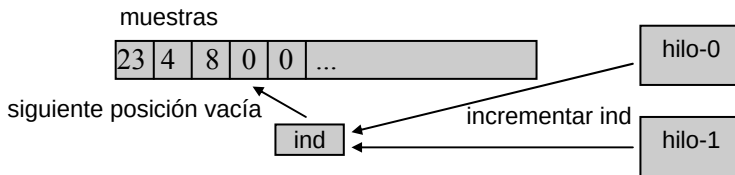
Evidentemente, la clase **Monitor** del espacio de nombres **System.Threading** proporciona además otra funcionalidad que puede utilizarse junto con la sentencia **lock**, o con los métodos sincronizados como veremos más adelante, como son los métodos:

- **Enter** y **TryEnter**. Marcan el principio de una sección crítica. Ningún otro hilo puede entrar en la sección crítica, hasta que sea liberado el recurso bloqueado. **TryEnter** permite a un hilo esperar bloqueado por un recurso durante un tiempo especificado; devuelve un valor **bool** indicando si el intento de adquirir el recurso tuvo o no éxito, valor que puede ser utilizado para evitar interbloqueos.
- **Wait**. Este método es llamado por un hilo en una sección crítica para liberar el bloqueo que tiene sobre un recurso y permitir a otros hilos obtener acceso al mismo. Después, el hilo se queda esperando hasta que el recurso esté disponible otra vez.
- **Pulse** y **PulseAll**. Envía una señal a uno o varios hilos en espera para notificarles que el estado de un recurso bloqueado ha cambiado y el propietario del bloqueo está en condiciones de liberarlo, con lo cual pueden competir para adquirirlo.
- **Exit**. Libera el bloqueo sobre un recurso. Esta acción también marca el final de una sección crítica protegida por el recurso bloqueado.

Ejemplo que implementa una sección crítica

Supongamos una aplicación en la que dos hilos de un proceso acceden a una única matriz de datos con la intención de registrar los resultados obtenidos durante un experimento. El programa podría simularse así:

- Creamos un objeto *CBufer* que envuelva una matriz unidimensional con el propósito de almacenar los datos adquiridos a través de una tarjeta que actúa como interfaz entre nuestra aplicación y el medio utilizado para realizar el experimento. En nuestro ejemplo, simularemos cada uno de los datos adquiridos con un valor obtenido a partir de un proceso sencillo.



- Creamos uno o más hilos para que adquieran los datos y los vayan almacenando en la matriz hasta llenarla, instante en el que su ejecución finalizará.

Para empezar, implementemos una clase *CBufer* para manipular una matriz unidimensional de tipo **double** con *n* elementos. Dicha clase incluirá los atributos:

- *muestra*: matriz de tipo **double**.
- *ind*: índice del siguiente elemento vacío.

y los métodos y propiedades:

- *CBufer*: es el constructor de la clase. Crea la matriz con *n* elementos, valor que se pasa como argumento, o con 10 si el valor pasado no es válido.
- *Indice*: propiedad para acceder al índice *ind* del siguiente elemento vacío.
- *Tamaño*: método que devuelve el número de elementos de la matriz.
- Un indizador para acceder a los valores encapsulados por el objeto *CBufer*.

```
public class CBufer
{
    // Atributos
    private double[] muestra;
    private int ind = 0;
    // Métodos
    public CBufer(int n)
    {
        if (n < 1) n = 10;
        muestra = new double[n];
    }

    public int Indice
    {
        get { return ind; }
        set { ind = value; }
    }
}
```

```
public int Tamaño()
{
    return muestra.Length;
}

public double this[int i]
{
    get { return muestra[i]; }
    set { muestra[i] = value; }
}
}
```

Uno o más hilos serán los encargados de adquirir los datos. Quiere esto decir que cuando se lancen estos hilos, el constructor de la clase que define cada uno de ellos debe recibir como argumento el objeto *CBufer* donde serán almacenados los datos que serán adquiridos invocando el método *AdquirirDato* de la clase del hilo. *AdquirirDato* obtiene el siguiente valor a almacenar en la matriz; recibe como argumento el nombre del hilo en ejecución y devuelve el índice del siguiente elemento vacío. Según lo expuesto, la clase que define cada uno de los hilos puede ser así:

```
public class CHilo
{
    private Thread hiloTrabajador;
    private CBufer m;    // objeto para almacenar los datos

    public CHilo(string nombre, CBufer muestras)
    {
        m = muestras;
        hiloTrabajador = new Thread(ProcHilo); // objeto hilo
        hiloTrabajador.Name = nombre;
        hiloTrabajador.Start(); // iniciar el hilo
    }

    public void ProcHilo()
    {
        int i = 0;
        do
        {
            i = AdquirirDato(hiloTrabajador.Name); // adquirir datos
        }
        while (i < m.Tamaño());
    }

    // Generador de números aleatorios
    private Random rnd = new Random();

    public int AdquirirDato(string hilo)
    {

```

```

        if (m.Indice >= m.Tamaño()) return m.Tamaño();
        double x = rnd.NextDouble();
        Console.WriteLine(hilo + " tomó la muestra " + m.Indice);
        m[m.Indice] = x;
        m.Indice++;
        return m.Indice;
    }
}

```

Para lanzar los hilos que adquirirán los datos, implementaremos una aplicación como la siguiente:

```

public class Test
{
    public static void Main(string[] args)
    {
        CBufuer muestras = new CBufuer(100);
        CHilo adquirirDatos_0 = new CHilo("Hilo-0", muestras);
    }
}

```

En la aplicación anterior observamos que **Main** crea el objeto *muestras* donde el hilo *adquirirDatos_0* almacenará los datos. Si ejecuta esta aplicación el resultado será el siguiente:

```

Hilo-0 tomó la muestra 0
Hilo-0 tomó la muestra 1
Hilo-0 tomó la muestra 2
Hilo-0 tomó la muestra 3
Hilo-0 tomó la muestra 4
Hilo-0 tomó la muestra 5
Hilo-0 tomó la muestra 6
Hilo-0 tomó la muestra 7
...
Hilo-0 tomó la muestra 99

```

Observando los resultados vemos que todo se ha desarrollado normalmente. Modifiquemos la aplicación *Test* para que ahora utilice dos hilos en lugar de uno para adquirir los datos:

```

public class Test
{
    public static void Main(string[] args)
    {
        CBufuer muestras = new CBufuer(100);
        CHilo adquirirDatos_0 = new CHilo("Hilo-0", muestras);
        CHilo adquirirDatos_1 = new CHilo("Hilo-1", muestras);
    }
}

```

Ahora el método **Main** de la aplicación *Test* lanza dos hilos: *adquirirDatos_0* y *adquirirDatos_1*. Cuando se lanza un hilo, el retorno al proceso padre es inmediato. Por eso podemos suponer que la ejecución del hilo *adquirirDatos_1* se inicia paralelamente a la de *adquirirDatos_0*. Si ahora ejecutamos la aplicación, el resultado obtenido será similar al siguiente:

```
Hilo-0 tomó la muestra 0
Hilo-0 tomó la muestra 1
...
Hilo-0 tomó la muestra 28
Hilo-1 tomó la muestra 28
...
Hilo-1 tomó la muestra 60
Hilo-0 tomó la muestra 61
Hilo-1 tomó la muestra 62
...
Hilo-1 tomó la muestra 98
Hilo-1 tomó la muestra 99
```

Excepción no controlada: System.IndexOutOfRangeException: Índice fuera de los límites de la matriz...

Analicemos los resultados. Cuando se ejecutó sólo un hilo, la matriz se llenó totalmente sin problemas; esto es, no faltaron muestras, tampoco se perdieron por realizar almacenamientos consecutivos en el mismo elemento y no hubo accesos a elementos fuera de los límites establecidos (el número de muestra coincide con el índice del elemento de la matriz donde está almacenada). En cambio, al ejecutarse los dos hilos concurrentemente, sí se han dado esos problemas (observe que la muestra 28 se ha tomado dos veces y que en total se han tomado 101 muestras, cuando el número de elementos de la matriz es 100).

En sistemas que soporten la planificación por *cuantos*, un hilo en ejecución puede ser interrumpido después de cualquier línea del método siguiente; por ejemplo, supongamos según el código mostrado a continuación que uno de los hilos se interrumpe después de la línea 6; no se incrementó el índice. Si esto ocurre, cuando se ejecute el otro hilo, almacenará la muestra adquirida en el último elemento utilizado; y si suponemos que este hilo es interrumpido después de la línea 7, se incrementa el índice, cuando se ejecute de nuevo el hilo que se interrumpió en la línea 6, volverá a incrementar el índice dejando un elemento vacío.

```
1. public int AdquirirDato(string hilo)
2. {
3.     if (m.Indice >= m.Tamaño()) return m.Tamaño();
4.     double x = rnd.NextDouble();
5.     Console.WriteLine(hilo + " tomó la muestra " + m.Indice);
6.     m[m.Indice] = x;
7.     m.Indice++;
```



```

8.    return m.Indice;
9. }

```

Lógicamente los problemas expuestos aparecen porque dos hilos están accediendo a un mismo objeto de datos sin ningún sincronismo, problema que se conoce con el nombre de *condiciones de carrera*. Y la sección de código en la que se actualizan objetos de datos comunes a más de un hilo recibe el nombre de *sección crítica*. Por lo tanto, la forma de evitar los problemas planteados es que cuando un hilo esté ejecutando código de su sección crítica ningún otro hilo pueda hacerlo. El mecanismo que asegura esto recibe el nombre de *exclusión mutua*.

Exclusión mutua

En C#, el acceso a un “objeto” puede ser bloqueado (en este caso se dice que se ha echado el cerrojo -- *lock*) para facilitar la implementación del mecanismo “exclusión mutua”. Los bloqueos de objetos permiten restringir el acceso a un bloque de código, normalmente denominado sección crítica. Mientras que un hilo posea el bloqueo de un objeto, ningún otro hilo podrá realizar ese bloqueo.

```

        Echar el cerrojo
        Sección crítica
        Quitar el cerrojo

```

Con un objeto **Monitor**, igual que con **lock**, podemos hacer estas dos cosas: *echar el cerrojo* y *quitar el cerrojo*. Un hilo cuando bloquea un objeto (denominado objeto de sincronismo) por alguno de los mecanismos citados es para echar el cerrojo a una sección crítica, de forma que ningún otro hilo pueda acceder a ella, y cuando lo libera, quita el cerrojo.

Y ¿cómo se implementa en C# una *sección crítica*? La forma más sencilla de crear una *sección crítica* es agrupando el código definido como crítico en una sentencia **lock** (cerrojo).

En el ejemplo anterior, la sección de código crítica es el cuerpo del método *AdquirirDato* (los hilos comparten *m.Indice*). Esto quiere decir que un hilo no debe acceder a este código cuando otro hilo lo esté ejecutando, para lo cual, utilizaremos una sentencia **lock** así:

```

public class CHilo
{
    // ...
    public int AdquirirDato(string hilo)
    {
        lock (m) // m es el objeto a sincronizar
        {

```

```

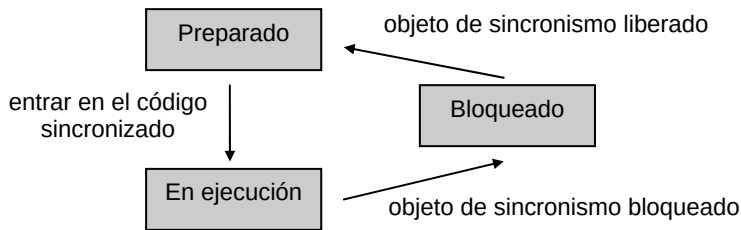
        if (m.Indice >= m.Tamaño()) return m.Tamaño();
        double x = rnd.NextDouble();
        Console.WriteLine(hilo + " tomó la muestra " + m.Indice);
        m[m.Indice] = x;
        m.Indice++;
        return m.Indice;
    }
}
}

```

Si ahora ejecuta de nuevo la aplicación *Test* comprobará que todo funciona como esperábamos.

Observe que el objeto a sincronizar es el objeto referenciado por *m* ya que tiene un atributo *ind* que es compartido por todos los hilos secundarios.

Un hilo que quiera ejecutar una sección de código sincronizada debe primero intentar bloquear el objeto cuyos atributos son compartidos por los hilos en esa sección. Si el objeto no está bloqueado por otro hilo, entonces lo bloqueará y ejecutará el código sincronizado y cuando finalice liberará el bloqueo. En cambio, si el objeto está bloqueado por otro hilo, entonces el hilo que lo intentó se bloqueará y sólo retornará al estado *preparado* cuando el objeto esté disponible.



Las secciones de código sincronizadas, llamadas secciones críticas, denotan que el acceso a ellas es crítico para el éxito de la ejecución de los hilos del programa. Por ello, en ocasiones, nos referimos a las secciones críticas como *operaciones atómicas*, significando que ellas representan para cualquier hilo una operación que debe ejecutarse de una sola vez.

Lo que no se debe hacer es lo que se muestra a continuación, ya que si el bucle **while** pertenece a la sección crítica, el planificador no podrá bloquear el hilo hasta que no termine de ejecutarse y, por lo tanto, no podrá asignar tiempo de UCP al otro hilo. Cuando el bucle **while** finalice, la matriz ya estará llena, lo que supone que el bucle **while** nunca se ejecutará para el otro hilo.

```

public void ProcHilo()
{
    int i = 0;

```

```

lock (m) // m es el objeto a sincronizar
{
    do
    {
        i = AdquirirDato(hiloTrabajador.Name); // adquirir datos
    }
    while (i < m.Tamaño());
}

// Generador de números aleatorios
private Random rnd = new Random();

public int AdquirirDato(string hilo)
{
    if (m.Indice >= m.Tamaño()) return m.Tamaño();
    double x = rnd.NextDouble();
    Console.WriteLine(hilo + " tomó la muestra " + m.Indice);
    m[m.Indice] = x;
    m.Indice++;
    return m.Indice;
}

```

Monitor reentrante

Supongamos que en alguna ocasión necesitamos que un bloque de código sincronizado tenga que llamar a otro bloque de código también sincronizado para el mismo objeto de sincronismo. Por ejemplo, para facilitar la comprensión de lo que se trata de explicar, vamos a suponer que el descriptor de acceso **set** del indicador de *CBufer* también está sincronizado:

```

public class CBufer
{
    // ...
    public double this[int i]
    {
        get { return muestra[i]; }
        set
        {
            lock (this)
            {
                muestra[i] = value;
            }
        }
    }
}

```

Tenga presente que la sentencia **lock** está implementada utilizando un objeto **Monitor**. Estos objetos exponen la capacidad de sincronizar el acceso a un bloque

de código mediante la obtención y liberación de un bloqueo en un objeto concreto con los métodos **Monitor.Enter** y **Monitor.Exit**:

```
Monitor.Enter(objeto-de sincronismo);  
    // Sección de código crítica  
Monitor.Exit(objeto-de sincronismo);
```

Ahora, el método *AdquirirDato* de *CHilo*, cuando ejecute $m[m.Indice] = x$, invocará al indizador de *CBufer* para ejecutar el descriptor de acceso **set**. Cuando un hilo trata de ejecutar el método *AdquirirDato* primero obtiene el bloqueo del objeto *CBufer* referenciado por *m*. A continuación ejecuta este bloque de código sincronizado, el cual llama al descriptor **set** del indizador de *CBufer*. Como éste está también sincronizado con el mismo objeto de sincronismo, el hilo intenta obtener otra vez el bloqueo del objeto *CBufer*. Parece lógico que el hilo debe bloquearse así mismo puesto que trata de obtener el bloqueo que él mismo debe liberar, cosa que no puede hacer hasta que no finalice la ejecución de *AdquirirDato*. En cambio no sucede así. ¿Por qué? Porque la máquina virtual de .NET permite a un hilo volver a obtener el bloqueo de un objeto del que ya lo tiene, porque los monitores C# son reentrantes.

Resumiendo, es válido que un mismo hilo invoque a **Enter** más de una vez sin bloquearse; sin embargo, es necesario realizar el mismo número de llamadas a **Exit** antes de que se desbloqueen otros hilos en espera por el objeto bloqueado.

Utilizar Wait y Pulse/PulseAll

Hemos visto que las secciones críticas son muy fáciles de utilizar, pero sólo se pueden emplear para sincronizar hilos involucrados en una única tarea; en el ejemplo anterior la tarea era única: almacenar datos en una matriz. Los métodos **Wait** y **Pulse/PulseAll** proporcionan una alternativa más para compartir un objeto, pero con la diferencia de que permiten sincronizar hilos involucrados en tareas distintas, una dependiente de la otra. Piense, por ejemplo, en un sistema que cada vez que genera un mensaje lo encapsula en un objeto *CMensaje* con el fin de manipularlo. En este caso, las tareas involucradas sobre el objeto *CMensaje* son: una, almacenar el mensaje generado y otra, obtener el mensaje almacenado para mostrarlo. Claramente se ve que una tarea depende de la otra; evidentemente, un mensaje no puede ser mostrado si antes no se ha producido.

Supongamos la clase *CMensaje* según se muestra a continuación:

```
using System.Threading;  
using System.Runtime.CompilerServices; // para Synchronized  
/////////////////////////////////////  
// Sincronización de hilos: Wait y Pulse/PulseAll.  
//
```

```

public class CMensaje
{
    private string textoMensaje;
    private int númeroMensaje;

    [MethodImplAttribute(MethodImplOptions.Synchronized)]
    public void Almacenar(int nmsj)
    {
        númeroMensaje = nmsj;
        // Suponer operaciones para buscar el mensaje en una tabla
        // de mensajes; resultado:
        textoMensaje = "mensaje";
    }

    [MethodImplAttribute(MethodImplOptions.Synchronized)]
    public string Obtener()
    {
        // Componer el mensaje bajo un determinado formato
        string mensaje;
        mensaje = textoMensaje + " #" + númeroMensaje;
        return mensaje;
    }
}

```

Se ha considerado que los métodos *Almacenar* y *Obtener* son *operaciones atómicas*, significando que para cualquier hilo deben ejecutarse de una sola vez; dicho de otra forma, se han definido como secciones críticas.

En el ejemplo expuesto, estamos pensando en un sistema que tendrá un productor de mensajes (mensajes de aviso, de error, etc.) para generar y almacenar los mensajes producidos (sólo se recuerda el último mensaje) y un consumidor de mensajes que mostrará al usuario el texto de cada mensaje que se produzca. Tanto el productor como el consumidor serán hilos que se supone están alerta para desempeñar su función cuando sea requerida.

```

public class Productor : CHilo
{
    private CMensaje mensaje;    // último mensaje producido
    // Cuando se produce un mensaje, el productor
    // almacena el texto en su miembro "mensaje"
}

public class Consumidor : CHilo
{
    private CMensaje mensaje;    // mensaje a mostrar
    // Cuando se ha producido un mensaje, el consumidor
    // lo obtiene de su miembro "mensaje" y lo muestra
}

```

Ambas clases se derivan de la clase abstracta *CHilo* que ha sido escrita así:

```
public abstract class CHilo
{
    // Clase que define un hilo secundario
    private Thread hilo; // identificador del hilo

    public CHilo() : this(null) {}

    public CHilo(string nombreHilo)
    {
        // Crear el hilo identificado por hilo
        hilo = new Thread(ProcHilo);
        if (nombreHilo != null) hilo.Name = nombreHilo;
    }

    public Thread HiloSubyacente
    {
        get { return hilo; } // referencia al objeto Thread
    }

    public abstract void ProcHilo();
}
```

Completemos el código del productor. La clase *Productor* tiene un constructor que inicia el atributo **Name** del hilo con el nombre dado al mismo y el atributo *mensaje* con el objeto *CMensaje* pasado como argumento; este mismo objeto será el que utilice el consumidor para mostrar el último mensaje producido. Como se puede observar, ésta es una forma sencilla de hacer que dos o más hilos compartan datos. Así mismo, sobrescribe el método *ProcHilo* para almacenar el mensaje que se produzca en el objeto *CMensaje*; este método simula que cada *msecs* milisegundos, valor generado aleatoriamente para cada mensaje, se produce el mensaje de número *númeroMsj*, valor generado también aleatoriamente. El hilo permanece dormido y despierta cada vez que se produce un mensaje. Según lo expuesto, una aproximación a la implementación de esta clase puede ser la siguiente:

```
public class Productor : CHilo
{
    private CMensaje mensaje; // último mensaje producido

    public Productor(string nombreHilo, CMensaje c)
        : base(nombreHilo)
    {
        mensaje = c;
    }

    // Generador de números aleatorios
```

```

private Random rnd = new Random();

public override void ProcHilo()
{
    int númeroMsj; // número de mensaje

    while (true)
    {
        númeroMsj = (int)(rnd.NextDouble() * 100);
        mensaje.Almacenar(númeroMsj); // almacena el mensaje
        Console.WriteLine(HiloSubyacente.Name +
                           "    almacena: mensaje #" + númeroMsj);
        int msecs = (int)(rnd.NextDouble() * 100);
        // Poner a dormir el hilo hasta que se produzca el
        // siguiente mensaje.
        Thread.Sleep(msecs);
    }
}
}

```

Completemos a continuación el código del consumidor. La clase *Consumidor* tiene un constructor que inicia el atributo **Name** del hilo con el nombre dado al mismo y el atributo *mensaje* con el objeto *CMensaje* que comparte con el productor. Así mismo, sobrescribe el método *ProcHilo* para obtener el mensaje almacenado en el objeto *mensaje* y mostrarlo. Según esto, la implementación de esta clase puede ser la siguiente:

```

public class Consumidor : CHilo
{
    private CMensaje mensaje;      // mensaje a mostrar

    public Consumidor(string nombreHilo, CMensaje c)
        : base(nombreHilo)
    {
        mensaje = c;
    }

    public override void ProcHilo()
    {
        string msj;
        while (true)
        {
            msj = mensaje.Obtener(); // obtiene el último mensaje
            Console.WriteLine(HiloSubyacente.Name + " obtuvo:  " + msj);
        }
    }
}

```

Una aplicación que lance los hilos productor y consumidor y muestre los resultados que producen puede ser la siguiente:

```
public class Test
{
    public static void Main(string[] args)
    {
        CMensaje mensaje = new CMensaje();
        Productor productor1 = new Productor("Productor-1", mensaje);
        Consumidor consumidor1 = new Consumidor("Consumidor-1", mensaje);

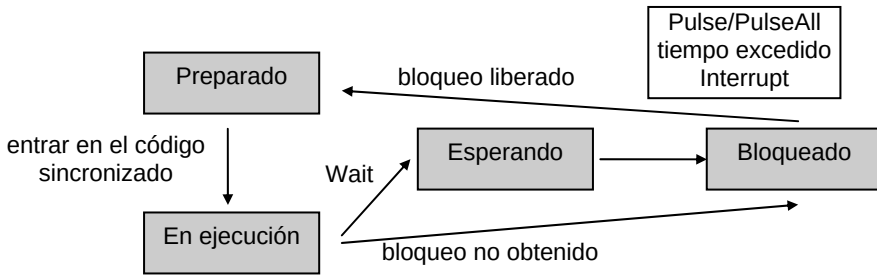
        productor1.HiloSubyacente.Start();
        consumidor1.HiloSubyacente.Start();
    }
}
```

Cuando ejecute la aplicación anterior, tenga presente que los hilos productor y consumidor trabajarán indefinidamente. Por lo tanto, para detener la ejecución tendrá que pulsar las teclas *Ctrl+C*. En lugar de esto, podríamos haber utilizado la técnica mostrada en el apartado *Finalizar un hilo*. No lo hemos hecho para no complicar el código y centrarnos en el tema de sincronización. Una vez que haya ejecutado la aplicación, observará resultados análogos a los siguientes:

```
Productor-1  almacena: mensaje #15
Consumidor-1 obtuvo:  mensaje #15
Consumidor-1 obtuvo:  mensaje #15
Consumidor-1 obtuvo:  mensaje #15
Consumidor-1 obtuvo:  mensaje #15
...
Productor-1  almacena: mensaje #19
Consumidor-1 obtuvo:  mensaje #19
Consumidor-1 obtuvo:  mensaje #19
Consumidor-1 obtuvo:  mensaje #19
Consumidor-1 obtuvo:  mensaje #19
Consumidor-1 obtuvo:  mensaje #19
...
```

Un análisis sencillo nos conduce a la conclusión de que independientemente de que los métodos se hayan definido como secciones críticas, no existe una sincronización entre el productor y el consumidor. El consumidor muestra el último mensaje producido cada vez que el planificador le asigna tiempo de UCP, en lugar de hacerlo única y exclusivamente cada vez que se produzca un mensaje.

Para conseguir la sincronización deseada, utilizaremos los métodos **Wait** y **PulseAll** de un monitor asociado con el objeto *CMensaje*. La siguiente figura muestra las transiciones de estados cuando intervienen estos métodos:



Partimos de que los métodos *Almacenar* y *Obtener* son métodos sincronizados; es lo que indica su atributo **Synchronized**. Esto significa que esos métodos sólo los puede ejecutar un subproceso cada vez. ¿Qué tipo de bloqueo se efectúa? Los métodos estáticos bloquean el tipo, mientras que los no estáticos, caso del ejemplo, bloquean el objeto para el que se ejecutan, esto es, **this** es el objeto sincronizado.

Cuando un hilo llama a **Wait**, libera el bloqueo que tiene sobre el objeto y entra en la cola de espera por ese objeto. El siguiente hilo de la cola de espera del objeto (si lo hubiera) adquiere el bloqueo y dispone de un uso exclusivo del mismo. Todos los hilos que llaman a **Wait** permanecen en la cola de espera hasta que reciben una señal de **Pulse** o **PulseAll**, enviada por el propietario del bloqueo. **Pulse** afecta al hilo situado al principio de la cola y **PulseAll** a todos los hilos de la cola. Cuando se recibe la señal, uno o más hilos abandonan la cola de espera y entran en la cola de hilos preparados pudiendo volver a adquirir el bloqueo.

Evidentemente, el método **Pulse** es más rápido que **PulseAll**, pero su forma de proceder nos puede conducir a situaciones no deseadas cuando hay varios hilos esperando en el mismo objeto. En este caso, rara vez se utiliza, y por seguridad se sugiere utilizar **PulseAll**.

Una de las diferencias entre **Sleep** y **Wait** es que el primero, cuando es llamado, no cede el control del monitor, mientras que el segundo sí.

Lo anteriormente expuesto conduce a la conclusión de que los métodos **Wait**, **Pulse** y **PulseAll** deben ser invocados desde código sincronizado.

Aplicando la teoría expuesta vamos a continuación a sincronizar los hilos productor y consumidor del ejemplo que estamos desarrollando. Para ello, hay que tener presente que no se puede mostrar un mensaje que aún no se ha generado (por supuesto, los mensajes se muestran una sola vez) y no se puede almacenar un mensaje, si aún no se ha mostrado el último generado.

Entonces, añadiremos a la clase *CMensaje* un atributo *disponible* de tipo **bool**, que valga **false** cuando no haya ningún mensaje que mostrar, y **true** en caso contrario.

```
public class CMensaje
{
    private string textoMensaje;
    private int númeroMensaje;
    private bool disponible = false;
    // ...
}
```

Ahora, cuando el hilo productor obtenga el bloqueo del objeto *CMensaje* y ejecute el método sincronizado *Almacenar*, lo primero que hará será interrogar el atributo *disponible*. Si su valor es **true**, el hilo se pondrá a esperar hasta que se muestre el último mensaje producido liberando el bloqueo que tiene sobre el objeto, y si vale **false** (no hay un mensaje disponible), almacena el nuevo mensaje, cambia el atributo *disponible* a **true**, e invoca a **PulseAll** para despertar a todos los hilos que estén esperando por el objeto de sincronización.

```
[MethodImplAttribute(MethodImplOptions.Synchronized)]
public void Almacenar(int nmsj)
{
    while (disponible == true)
    {
        // El último mensaje aún no ha sido mostrado.
        // El hilo se pone a esperar liberando el bloqueo sobre this.
        Monitor.Wait(this);
    }
    númeroMensaje = nmsj;
    // Suponer operaciones para buscar el mensaje en una tabla
    // de mensajes; resultado:
    textoMensaje = "mensaje";
    disponible = true;
    Monitor.PulseAll(this);
}
```

Así mismo, cuando el hilo consumidor obtenga el bloqueo del objeto *CMensaje* y ejecute el método sincronizado *Obtener*, lo primero que hará será interrogar el atributo *disponible*. Si su valor es **false**, el hilo esperará hasta que haya un mensaje, liberando el bloqueo que tiene sobre el objeto, y si su valor es **true**, cambia el atributo *disponible* a **false**, invoca a **PulseAll** para despertar a todos los hilos que estén esperando por el objeto de sincronización y retorna el mensaje.

```
[MethodImplAttribute(MethodImplOptions.Synchronized)]
public string Obtener()
{

```

```

while (disponible == false)
{
    // No hay mensaje.
    // El hilo se pone a esperar liberando el bloqueo sobre this.
    Monitor.Wait(this);
}
disponible = false;
Monitor.PulseAll(this);
// Componer el mensaje bajo un determinado formato
string mensaje;
mensaje = textoMensaje + " #" + númeroMensaje;
return mensaje;
}

```

Una vez modificados los métodos *almacenar* y *obtener* de la clase *CMensaje*, ejecute de nuevo la aplicación *Test*. Observará que ahora los resultados sí son los esperados:

```

Productor-1  almacena: mensaje #74
Consumidor-1 obtuvo:  mensaje #74
Productor-1  almacena: mensaje #17
Consumidor-1 obtuvo:  mensaje #17
Productor-1  almacena: mensaje #85
Consumidor-1 obtuvo:  mensaje #85
Productor-1  almacena: mensaje #3
Consumidor-1 obtuvo:  mensaje #3
Productor-1  almacena: mensaje #91
Consumidor-1 obtuvo:  mensaje #91
...

```

¿Por qué los métodos *Almacenar* y *Obtener* utilizan un bucle?

Antes de proceder a la explicación, añada más consumidores y observe los resultados. Por ejemplo:

```

public class Test
{
    public static void Main(string[] args)
    {
        CMensaje mensaje = new CMensaje();
        Productor productor1 = new Productor("Productor-1", mensaje);
        Consumidor consumidor1 = new Consumidor("Consumidor-1", mensaje);
        Consumidor consumidor2 = new Consumidor("Consumidor-2", mensaje);

        productor1.HiloSubyacente.Start();
        consumidor1.HiloSubyacente.Start();
        consumidor2.HiloSubyacente.Start();
    }
}

```

A continuación edite los métodos *Almacenar* y *Obtener*, y cambie las sentencias **while** por **if**:

```
1. if (disponible == false) // antes: while (disponible == false)
2. {
3.     Monitor.Wait(this);
4. }
5. disponible = false;
6. Monitor.PulseAll(this);
7. string mensaje;
8. mensaje = textoMensaje + " #" + númeroMensaje;
9. return mensaje;
```

Compile y ejecute de nuevo la aplicación. Compare los resultados con los obtenidos anteriormente. ¿Qué ha ocurrido?

```
Productor-1 almacena: mensaje #14
Consumidor-1 obtuvo:  mensaje #14
Consumidor-2 obtuvo:  mensaje #14
```

El peligro de utilizar una sentencia **if** en lugar de **while** es que algunas veces el hilo que obtiene el bloqueo del objeto, *Consumidor-2*, ejecuta la línea 1 y suponiendo que la condición es cierta se pone a esperar liberando el bloqueo que tiene sobre el objeto. Más tarde, otro hilo adquiere el monitor, *Consumidor-1*, suponiendo que la condición es falsa ejecuta la línea 6 y despierta a los hilos que esperan por obtener el bloqueo del objeto de sincronización. Los hilos en el estado *preparado* compiten por la UCP. Supongamos que el planificador se la adjudica al hilo original, *Consumidor-2*; éste continuará donde lo dejó, a partir de la línea 3, independientemente de que el objeto de sincronización esté bloqueado o liberado. El resultado es que retorna el mismo mensaje que *Consumidor-1*. Utilizando una sentencia **while** en lugar de **if**, cuando *Consumidor-2* continúe donde lo dejó, a partir de la línea 3, volverá a ejecutar la línea 1 y se pondrá de nuevo a esperar por ser la condición cierta.

Objeto Mutex

Cuando dos o más hilos tienen que obtener acceso a un recurso compartido al mismo tiempo, el sistema necesita un mecanismo de sincronización para garantizar que sólo uno de los hilos utilice el recurso en ese momento. Un objeto **Mutex** (**Mutual exclusion** - exclusión mutua) da acceso exclusivo al recurso compartido a un solo hilo. Cuando un hilo adquiere un **Mutex**, el siguiente hilo que intenta adquirir dicho objeto se suspende hasta que el primer hilo lo libera.

Para que un hilo pueda obtener la propiedad de un objeto **Mutex** tiene que llamar al método **WaitOne** de éste y para liberarlo al método **ReleaseMutex**.

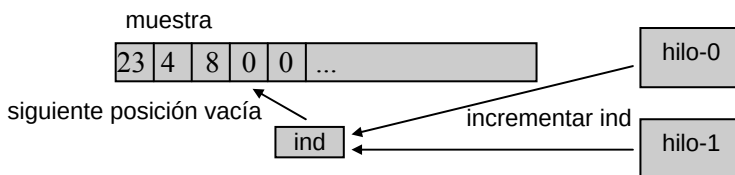
La clase **Mutex** exige la identidad del hilo, por lo que sólo podrá liberar el **Mutex** el hilo que lo adquirió. Por el contrario, la clase **Semaphore**, como estudiaremos a continuación, no exige la identidad del hilo. Si un hilo libera un objeto **Mutex** que no posee, se lanza una excepción **ApplicationException**.

Si un hilo finaliza sin haber liberado el **Mutex** que posee, se dice que la exclusión mutua está abandonada. En este caso, el siguiente hilo en espera que obtenga la propiedad del **Mutex** hará que se lance una excepción **AbandonedMutexException**.

Como la clase **Mutex** se deriva de **WaitHandle**, también puede llamar a sus métodos estáticos **WaitAll** o **WaitAny** para solicitar la propiedad del **Mutex** en combinación con otros controladores de espera.

Si un hilo posee un objeto **Mutex**, dicho hilo puede solicitar su propiedad en llamadas repetidas sin bloquearse; no obstante, deberá liberar el **Mutex** las veces que sean necesarias para deshacerse de su propiedad.

En base a lo expuesto, vamos a modificar la aplicación que realizamos anteriormente para almacenar los datos que supuestamente adquiriríamos a través de una tarjeta que actuaba como interfaz entre nuestra aplicación y el medio utilizado para realizar un experimento. Gráficamente exponíamos la estructura del problema como se indica a continuación, e implementábamos la clase **CBufer** para almacenar los datos, **CHilo** para proporcionar la funcionalidad para adquirir los datos y **Test** para poner a prueba las clases anteriores:



La modificación es sencilla. Basta definir un **Mutex** en la clase **CHilo** para definir la sección crítica en el método *AdquirirDatos*:

```

public class CHilo
{
    private Thread hiloTrabajador;
    private CBufet m; // objeto para almacenar los datos
    private static Mutex obMutex = new Mutex();

    public CHilo(string nombre, CBufet muestras) // constructor
    {
        m = muestras;
        hiloTrabajador = new Thread(ProcHilo); // objeto hilo
    }
}
  
```

```
        hiloTrabajador.Name = nombre;
        hiloTrabajador.Start(); // iniciar el hilo
    }

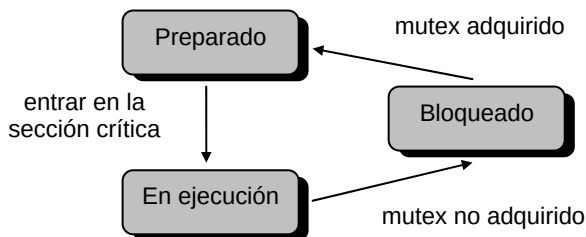
    public void Prochilo()
    {
        int i = 0;
        do
        {
            i = AdquirirDato(hiloTrabajador.Name); // adquirir datos
        }
        while (i < m.Tamaño());
    }

    // Generador de números aleatorios
    private Random rnd = new Random();

    public int AdquirirDato(string hilo)
    {
        obMutex.WaitOne(); // el hilo adquiere la propiedad del Mutex
        if (m.Indice >= m.Tamaño()) return m.Tamaño();
        double x = rnd.NextDouble();
        Console.WriteLine(hilo + " tomó la muestra " + m.Indice);
        m[m.Indice] = x;
        m.Indice++;
        obMutex.ReleaseMutex(); // el hilo libera la propiedad del Mutex
        return m.Indice;
    }
}
```

Si ahora ejecuta de nuevo la aplicación *Test*, comprobará que todo funciona como esperábamos.

Un hilo que quiera ejecutar el código de la sección crítica debe primero intentar adquirir el control del *obMutex*. Si el *obMutex* está disponible, esto es, si no está adquirido por otro hilo, entonces lo adquirirá y ejecutará el código de la sección crítica, y cuando finalice lo liberará. En cambio, si *obMutex* está adquirido por otro hilo, entonces el hilo que lo intentó se bloqueará y sólo retornará al estado *preparado* cuando el *obMutex* esté disponible.



Objeto Semaphore

Un *semáforo*, a diferencia de un objeto *exclusión mutua*, puede controlar varios recursos accedidos por distintos hilos simultáneamente. Por ejemplo, si un ordenador tiene dos puertos serie, en un instante determinado podrían ser utilizados como máximo por dos hilos, acción que puede ser controlada por un solo semáforo en lugar de por dos objetos **Mutex**. Para ello, el semáforo necesita ser iniciado con el número de recursos inicialmente disponibles; en el ejemplo, por dos. Después, cada vez que se asigne uno de estos recursos, se decrementa dicho contador en una unidad. Este contador recibe el nombre de *contador de referencias* y por definición nunca puede ser negativo. Según lo expuesto, un objeto *exclusión mutua* es como un semáforo binario; es decir, un semáforo que sólo tiene los valores 0 y 1 por lo que únicamente puede ser utilizado por un hilo a la vez. En el caso de los semáforos, el sistema no sigue la pista del hilo que adquiere el semáforo, especialmente porque éste no es exclusivo, por lo que puede ser liberado por cualquier otro hilo.

La cuenta de un semáforo se decrementa cada vez que un hilo entra en él y se incrementa cuando un hilo lo libera. Cuando la cuenta es cero, las solicitudes posteriores se bloquean hasta que otros hilos liberan el semáforo. Cuando todos los hilos han liberado el semáforo, la cuenta está en el valor máximo especificado al crear el semáforo. Para decrementar el contador del semáforo, los hilos llaman al método **WaitOne**, que se hereda de la clase **WaitHandle**, y para incrementarlo llaman al método **Release**.

Conceptualmente, un semáforo es un contador entero no negativo. Cuando un semáforo tiene su contador de referencias a valor cero, indica que está *ocupado*, y si lo tiene a algún valor positivo, indica que está *libre*. El constructor tiene la sintaxis siguiente; el primer argumento es el valor inicial del contador de referencias y el segundo, el número máximo de entradas del semáforo:

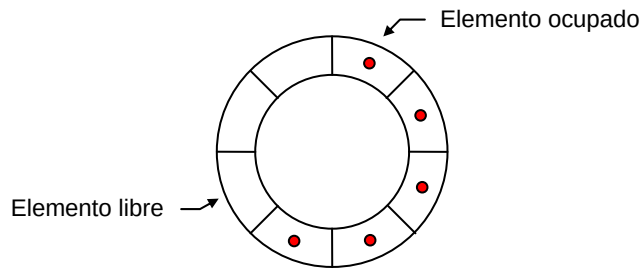
```
public Semaphore(cuenta inicial, número máximo de entradas)
```

Es responsabilidad del programador garantizar que un hilo no libere el semáforo más veces de las necesarias. Por ejemplo, supongamos un semáforo que tiene un recuento máximo de dos y en el que entran un hilo A y un hilo B. Si un error de programación en el hilo B hace que éste llame dos veces a **Release**, ambas llamadas tendrán éxito; el recuento del semáforo está completo. Entonces, cuando finalmente el hilo A llame al método **Release**, se lanzará una excepción **SemaphoreFullException**.

Un escenario habitual de uso de los semáforos implica la existencia de un hilo productor y un hilo consumidor, donde uno de los hilos siempre incrementa el contador del semáforo y el otro siempre lo disminuye.

Problema del productor-consumidor

El productor-consumidor es un problema clásico de hilos cooperantes. En esencia, se trata de dos hilos que cooperan entre sí para solucionar un problema. Uno de ellos recibe el calificativo de “productor” porque su labor es generar datos y almacenarlos, por ejemplo, en una matriz circular, y el otro recibe el calificativo de “consumidor” porque su labor es extraer de la lista circular los datos generados por el productor. Es obvio que muchas aplicaciones de la vida ordinaria reproducen este problema. Un ejemplo es el administrador de impresión en un servidor de red; los productores son los usuarios de la red y el consumidor la impresora o impresoras.



La figura anterior muestra un esquema de cómo podemos imaginar el problema del *productor-consumidor*. Un área de almacenamiento, por ejemplo una matriz circular y dos hilos. El productor deposita datos en esa área y el consumidor los recoge. Se trata, por lo tanto, de una zona de datos compartida por ambos hilos, con alguna particularidad:

- El productor no puede depositar datos en la matriz si está llena, sólo puede hacerlo si hay elementos vacíos. Esto exige un elemento de sincronización, por ejemplo, un semáforo contador que nos indique si hay elementos vacíos, *semVacíos*, con un valor inicial igual al *tamaño de la matriz* (inicialmente todos los elementos están vacíos; semáforo libre).
- El consumidor no puede recoger datos de la matriz si está vacía, sólo si hay elementos llenos. Esto exige un elemento de sincronización, por ejemplo, un semáforo contador que nos indique si hay elementos llenos, *semLlenos* igual a cero (inicialmente hay cero elementos llenos; semáforo ocupado).
- Partiendo de que la matriz inicialmente está vacía y que el productor está ejecutando la operación de depositar un dato, el consumidor no puede recogerlo hasta que el productor haya finalizado su operación, y viceversa, como todos los datos están consecutivos en la matriz, el productor debe esperar a que el consumidor termine de recoger un dato para él depositar uno. Esto es, sólo un hilo puede acceder a la matriz a la vez. El hilo tiene que ejecutar una sección crítica, lo que requiere utilizar un **Mutex** (*obMutex*).

A continuación se expone el programa completo ampliamente comentado, el cual consta de tres hilos: un hilo principal y dos hilos secundarios. El hilo principal (**Main**) lanza los hilos *productor* y *consumidor*, y se queda esperando hasta que éstos terminen. El proceso se inicia al pulsar la tecla *Entrar* y finaliza de igual forma.

```
using System;
// Sincronización de hilos. Productor - consumidor con semáforos
//
public class Test
{
    public static void Main(string[] args)
    {
        CMatriz matriz = new CMatriz(10);
        Productor productor1 = new Productor("Productor-1", matriz);
        Consumidor consumidor1 = new Consumidor("Consumidor-1", matriz);
        Consumidor consumidor2 = new Consumidor("Consumidor-2", matriz);

        Console.WriteLine("Pulse [Entrar] para continuar y");
        Console.WriteLine("vuelva a pulsar [Entrar] para finalizar.");

        Console.ReadLine(); // ejecución detenida hasta pulsar [Entrar]
        // Iniciar la ejecución de los hilos
        productor1.HiloSubyacente.Start();
        consumidor1.HiloSubyacente.Start();
        consumidor2.HiloSubyacente.Start();
        Console.ReadLine(); // ejecución detenida hasta pulsar [Entrar]

        // Permitir a los hilos finalizar
        productor1.Terminar();
        consumidor1.Terminar();
        consumidor2.Terminar();
    }
}
```

La variable *matriz* es un objeto de la clase *CMatriz*. Esta clase estará formada por los atributos:

<i>m</i>	Matriz de <i>n</i> enteros positivos.
<i>indProd</i>	Índice del elemento donde el productor debe insertar el siguiente elemento. Su valor será: 0, 1, 2,..., <i>n</i> -1, 0, 1, 2,...
<i>indCons</i>	Índice del elemento donde el consumidor debe obtener el siguiente elemento. Su valor será: 0, 1, 2,..., <i>n</i> -1, 0, 1, 2,...
<i>obMutex</i>	Objeto Mutex para definir una sección crítica.
<i>semVacios</i>	Número de elementos vacíos en un instante determinado.
<i>semLlenos</i>	Número de elementos llenos en un instante determinado.

y por los métodos:

<i>Almacenar</i>	Almacena un dato en el siguiente elemento vacío.
<i>Obtener</i>	Obtiene el siguiente dato aún no extraído.

El código correspondiente a esta clase se muestra a continuación:

```
using System;
using System.Threading;

////////////////////////////////////
// Sincronización de hilos: Semaphore.
//
public class CMatriz
{
    private int[] m;          // matriz de datos
    private int indProd = 0;  // índice productor
    private int indCons = 0;  // índice consumidor

    private static Mutex obMutex = new Mutex(); // exclusión mutua
    private Semaphore semVacios; // contador de elementos vacíos
    private Semaphore semLlenos; // contador de elementos llenos

    public CMatriz(int n)
    {
        if (n < 1) n = 10;
        m = new int[n];
        // Semaphore(cuenta inicial, número máximo de entradas)
        semVacios = new Semaphore(m.Length, m.Length);
        semLlenos = new Semaphore(0, m.Length);
    }

    public void Almacenar(int num)
    {
        // Esperar si no hay elementos vacíos, si procede
        semVacios.WaitOne(); // decrementar el contador de vacíos
        obMutex.WaitOne();  // entrada en la sección crítica
        m[indProd] = num;
        indProd = (indProd + 1) % m.Length;
        Thread.Sleep(50);   // simula que trabaja (no libera el bloqueo)
        obMutex.ReleaseMutex(); // salida de la sección crítica
        semLlenos.Release(); // incrementar el contador de llenos
    }

    public int Obtener()
    {
        // Esperar a que haya elementos llenos, si procede
        semLlenos.WaitOne(); // decrementar el contador de llenos
        obMutex.WaitOne();  // entrada en la sección crítica
        int num = m[indCons];
```

```

        indCons = (indCons + 1) % m.Length;
        Thread.Sleep(50); // simula que trabaja (no libera el bloqueo)
        obMutex.ReleaseMutex(); // salida de la sección crítica
        semVacios.Release(); // incrementar el contador de vacíos
        return num;
    }
}
////////////////////////////////////

```

En el problema del productor y del consumidor los recursos que estos hilos deben adquirir para poder ejecutarse son los elementos vacíos y los elementos llenos de la matriz, respectivamente. Cada uno de estos tipos de recursos los representaremos por sendas variables que actuarán como semáforos: *semLlenos* y *semVacios*. Un valor cero equivale a semáforo en *rojo* y un valor distinto de cero a semáforo en *verde*.

El objeto *semLlenos* es un semáforo inicialmente en *rojo* para el consumidor (porque no hay ningún elemento lleno, esto es, no se puede *Obtener*) que representa los elementos actualmente llenos de la matriz y *semVacios* es un semáforo inicialmente en *verde* para el productor (porque todos los elementos están vacíos, esto es, se puede *Almacenar*) que representa los elementos actualmente vacíos de la matriz. Por lo tanto, el contador de *semLlenos* debe valer inicialmente cero y el de *semVacios* debe valer *m.Length*.

Cuando un hilo necesita un recurso de un tipo particular, decrementa el contador del semáforo correspondiente, y cuando lo libera lo incrementa. Por ejemplo, cuando el productor quiere *Almacenar* un dato necesita el recurso “elementos vacíos”, de tal forma que cada vez que lo adquiere lo decrementa; cuando llegue a cero implica semáforo en rojo indicando que el recurso “elementos vacíos” está ocupado. Lógicamente decrementar *semVacios* implica incrementar *semLlenos*.

```

semVacios.WaitOne(); // decrementar el contador de vacíos
obMutex.WaitOne(); // entrada en la sección crítica
m[indProd] = num;
indProd = (indProd + 1) % m.Length;
Thread.Sleep(50); // simula que trabaja (no libera el bloqueo)
obMutex.ReleaseMutex(); // salida de la sección crítica
semLlenos.Release(); // incrementar el contador de llenos

```

El código anterior, que pertenece al hilo productor, decrementa el contador del semáforo *semVacios*, inserta un dato en el siguiente elemento vacío de la matriz y, lógicamente, incrementa el contador del semáforo *semLlenos*. Si el contador de *semVacios* fuera cero, el hilo productor pasaría al estado bloqueado hasta que el hilo consumidor extraiga uno o más datos y, por consiguiente, incremente *semVacios*. Un razonamiento análogo haríamos para el consumidor.

Según lo expuesto, el hilo productor básicamente se limitará a llamar al método *Almacenar* de la clase *CMatriz*. Esto es:

```
using System;
using System.Threading;

////////////////////////////////////
// Sincronización de hilos. Hilo productor.
//
public class Productor : CHilo
{
    private CMatriz matriz;
    private bool continuar = true;

    public Productor(string nombreHilo, CMatriz m)
        : base(nombreHilo)
    {
        matriz = m;
    }

    // Generador de números aleatorios
    private Random rnd = new Random();

    public override void ProcHilo()
    {
        int número; // número producido

        while (continuar)
        {
            número = (int)(rnd.NextDouble() * 100);
            matriz.Almacenar(número); // almacena el número
            Console.WriteLine(HiloSubyacente.Name +
                             " almacena: número " + número);
        }
    }

    public void Terminar()
    {
        continuar = false;
    }
}
////////////////////////////////////
```

Análogamente, el hilo consumidor básicamente se limitará a llamar al método *Obtener* de la clase *CMatriz*. Esto es:

```
using System;
using System.Threading;

////////////////////////////////////
```

```
// Sincronización de hilos. Hilo consumidor.
//
public class Consumidor : CHilo
{
    private CMatriz matriz;
    private bool continuar = true;

    public Consumidor(string nombreHilo, CMatriz m)
        : base(nombreHilo)
    {
        matriz = m;
    }

    public override void Prochilo()
    {
        int número;

        while (continuar)
        {
            número = matriz.Obtener();
            Console.WriteLine(HiloSubyacente.Name +
                             " obtuvo:  número " + número);
        }
    }

    public void Terminar()
    {
        continuar = false;
    }
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

La clase *CHilo* aporta la funcionalidad común para crear un hilo. Es la clase base para las clases *Productor* y *Consumidor* y ya fue expuesta anteriormente:

```
using System.Threading;

// Clase que define un hilo secundario
public abstract class CHilo
{
    private Thread hilo; // identificador del hilo

    public CHilo() : this(null) {}

    public CHilo(string nombreHilo)
    {
        // Crear el hilo identificado por hilo
        hilo = new Thread(Prochilo);
        if (nombreHilo != null) hilo.Name = nombreHilo;
    }
}
```

```
public Thread HiloSubyacente
{
    get { return hilo; } // referencia al objeto Thread
}

public abstract void ProcHilo();
}
```

Finalmente, ejecute la aplicación y observe los resultados. Obsérvese también la necesidad de la sección crítica ya que todos los hilos comparten la misma matriz de datos.

Controladores de espera

Los controladores de espera son objetos que indican a un hilo el estado de otro hilo mediante señales. Son utilizados por los hilos para notificar a otros hilos que necesitan tener acceso exclusivo a un recurso. De este modo, los demás hilos deberán esperar a utilizar dicho recurso, bloqueándose, hasta que el controlador de espera no se encuentre en uso.

Los controladores de espera tienen dos estados: señalizado y no señalizado. El controlador de espera se encontrará en el estado señalizado cuando no lo utilice ningún hilo. Si está siendo utilizado por algún hilo, su estado será no señalizado. Se podrían comparar con los tornos de acceso a un recinto: “espere a la señal verde para pasar” (en este símil, las formas de provocar esa señal pueden ser varias, lectura de una banda magnética, de un código de barras, etc.).

La clase **WaitHandle** representa los objetos de sincronización de la máquina virtual .NET que permiten operaciones de espera. Es una clase abstracta y entre sus clases derivadas están las siguientes:

- **Mutex** y **Semaphore**, que ya fueron estudiadas.
- La clase **EventWaitHandle** y sus clases derivadas, **AutoResetEvent** y **ManualResetEvent**. La clase **EventWaitHandle** define controladores de espera de eventos.

Un hilo puede solicitar la propiedad de un controlador de espera llamando a uno de los métodos **WaitOne**, **WaitAny** o **WaitAll**. Estos métodos definidos en la clase **WaitHandle** son llamadas para determinar si un hilo puede continuar ejecutándose o, por el contrario, queda bloqueado. A continuación se resume la acción que ejecutan cuando un hilo los invoca:

- **WaitOne.** Bloquea el hilo actual hasta que el estado del objeto **WaitHandle** implicado en la llamada pase a señalizado (**Set**) o hasta que pase un tiempo. Opcionalmente puede especificar dos parámetros: un entero para medir el intervalo de tiempo (-1 indica un tiempo infinito) y un **bool** (este parámetro no tiene ningún efecto en casi todos los casos). Devuelve **true** cuando el objeto **WaitHandle** actual pase a señalizado; en otro caso devuelve **false**.
- **WaitAny.** Acepta como argumento una matriz de controladores de espera y hace que el hilo que llama espere hasta que el estado de uno de los controladores de espera especificados llame a **Set**. Devuelve el índice de la matriz del objeto **WaitHandle** señalizado; en otro caso devuelve **WaitTimeout**.
- **WaitAll.** Acepta como argumento una matriz de controladores de espera y hace que el hilo que llama espere hasta que el estado de todos los controladores de espera especificados llamen a **Set**. Devuelve **true** cuando todos los objetos **WaitHandle** pasen a señalizado; en otro caso devuelve **false**.

La clase **EventWaitHandle** derivada de **WaitHandle** hereda los métodos de ésta y define otros como **Set** y **Reset**:

- **Set.** Define como señalizado el estado de un controlador de espera de eventos específico y reanuda la ejecución de los hilos en espera.
- **Reset.** Define como no señalizado el estado de un controlador de espera de eventos específico.

Un objeto **EventWaitHandle** es un controlador de espera de eventos, en estado señalizado o no señalizado (se utiliza el término “evento” porque la acción de señalar indica a los hilos en espera que se ha producido un evento). Pasará al estado señalizado para liberar uno o más hilos que están esperando por un recurso. Una vez esté en el estado señalizado, pasará a no señalizado de forma automática cuando se trate de un objeto **AutoResetEvent** o manual cuando se trate de un objeto **ManualResetEvent**.

Supongamos un hilo que invoca a **WaitOne** por medio de un objeto **AutoResetEvent** y que este objeto se encuentra en el estado no señalado. El hilo se bloquea en espera de que el hilo que controla el recurso en ese momento indique que dicho recurso está disponible mediante una llamada a **Set**. Una llamada a **Set** indica a **AutoResetEvent** que libere un hilo en espera. **AutoResetEvent** permanece señalado hasta que se libera un único hilo en espera y, a continuación, vuelve automáticamente al estado de no señalado. Si no hay ningún hilo en espera, el estado permanece señalado indefinidamente.

El estado inicial de un objeto **AutoResetEvent** se puede controlar pasando un valor **bool** al constructor: **true** objeto señalado y **false** no señalado.

El siguiente ejemplo aclara lo expuesto. En él, el hilo principal crea *numHilos* hilos secundarios. Estos hilos quedan bloqueados porque realizan una llamada a **WaitOne** a través de un objeto **AutoResetEvent** no señalado, *objWH*. Finalmente, el hilo principal permite liberar los hilos secundarios, uno a uno, enviando señales **Set**. En este ejemplo también se puede ver el uso de la clase **Interlocked**.

```
using System;
using System.Threading;

public class Test
{
    // Utilización de los controladores de espera de eventos:
    // EventWaitHandle y AutoResetEvent.
    //
    private static EventWaitHandle objWH;

    // Contador para asegurarnos que todos los hilos son arrancados
    // y bloqueados antes de ser liberados. Utilizamos un long para
    // mostrar el uso de los métodos de Interlocked para 64 bits.
    private static long ContadorDeHilos = 0;

    public static void Main()
    {
        const int numHilos = 3; // número de hilos

        // Crear un objeto AutoResetEvent inicialmente no señalado.
        objWH = new AutoResetEvent(false);

        // Crear e iniciar numHilos hilos numerados. Utilizamos el
        // delegado ParameterizedThreadStart para pasar el número
        // de hilo como argumento.
        for (int i = 1; i <= numHilos; i++)
        {
            Thread hilo = new Thread(
                new ParameterizedThreadStart(ProcHilo));
            hilo.Start(i);
        }

        // Esperar hasta que todos los hilos se hayan iniciado y
        // bloqueado. Cuando varios hilos usan un valor de 64 bits en
        // un sistema de 32 bits, hay que acceder al valor a través de
        // Interlocked para garantizar la seguridad en el proceso.
        while (Interlocked.Read(ref ContadorDeHilos) < numHilos)
        {
            Thread.Sleep(400);
        }
    }
}
```



```

// Liberar un hilo cada vez que el usuario pulsa [Entrar],
// hasta que todos los hilos hayan sido liberados.
while (Interlocked.Read(ref ContadorDeHilos) > 0)
{
    Console.WriteLine("Pulse [Entrar] para liberar un hilo bloqueado.");
    Console.ReadLine();

    // La señal Set sobre un objeto AutoResetEvent, libera un
    // solo hilo, permitiéndole, a continuación, decrementar
    // el contador de hilos bloqueados: ContadorDeHilos.
    objWH.Set();
}
}

public static void Prochilo(object datos)
{
    Console.WriteLine("Hilo {0} bloqueado.", datos);
    // Incrementar el contador de hilos bloqueados.
    Interlocked.Increment(ref ContadorDeHilos);

    // Esperar una señal (Set).
    objWH.WaitOne();
    Console.WriteLine("Hilo {0} liberado.", datos);
    // Decrementar el contador de hilos bloqueados.
    Interlocked.Decrement(ref ContadorDeHilos);
}
}

```

Cuando ejecute este programa observará la siguiente solución:

```

Hilo 1 bloqueado.
Hilo 2 bloqueado.
Hilo 3 bloqueado.
Pulse [Entrar] para liberar un hilo bloqueado.

Hilo 1 liberado.
Pulse [Entrar] para liberar un hilo bloqueado.

Hilo 2 liberado.
Pulse [Entrar] para liberar un hilo bloqueado.

Hilo 3 liberado.

```

Un objeto **ManualResetEvent** es igual que un objeto **AutoResetEvent** con la diferencia de que los cambios de señalizado a no señalizado y viceversa hay que realizarlos manualmente invocando a los métodos **Reset** y **Set**. El método **Reset** cambia el estado de un objeto **ManualResetEvent** a no señalizado y el método **Set** lo cambia a señalizado.

Como ejemplo podemos modificar el código anterior sustituyendo el objeto **AutoResetEvent** por un objeto **ManualResetEvent**. Obsérvese que ahora el bucle **while** no es necesario, ya que el controlador de espera queda en el estado señalado después de cambiar de no señalado a señalado la primera vez, por lo que todos los hilos bloqueados irán liberándose uno a continuación de otro.

```
objWH = new ManualResetEvent(false);

for (int i = 1; i <= numHilos; i++)
{
    Thread hilo = new Thread(
        new ParameterizedThreadStart(ProcHilo)
    );
    hilo.Start(i);
}

while (Interlocked.Read(ref ContadorDeHilos) < numHilos)
{
    Thread.Sleep(400);
}

Console.WriteLine("Pulse [Entrar] para liberar los hilos bloqueados.");
Console.ReadLine();
objWH.Set();
```

Cuando ejecute este programa observará la siguiente solución:

```
Hilo 1 bloqueado.
Hilo 2 bloqueado.
Hilo 3 bloqueado.
Pulse [Entrar] para liberar un hilo bloqueado.

Hilo 1 liberado.
Hilo 2 liberado.
Hilo 3 liberado.
```

Los controladores de espera de eventos son útiles en muchos de los escenarios de sincronización donde habitualmente utilizamos objetos **Monitor**, siendo, a menudo, más fáciles de utilizar que los métodos **Wait** y **Pulse** de la clase **Monitor**, proporcionando un mayor control sobre la señalización.

Detener un hilo de forma controlada

Cuando se cierra una aplicación y hay hilos secundarios que aún no han finalizado, es probable que surjan problemas si éstos no se detienen de forma controlada. A continuación se explica cómo realizar este proceso correctamente. Los pasos a seguir son los siguientes:

1. Cuando se vaya a cerrar la aplicación, el hilo principal informará a los hilos secundarios que deben detenerse. Para informar de esta acción, se puede utilizar un controlador de espera de eventos.
2. Cuando el controlador de espera informe a los hilos secundarios de que deben detenerse, el hilo principal esperará a que éstos le informen de que han parado.
3. Cada hilo secundario verificará en cada iteración si le ha sido solicitado que se detenga, en cuyo caso se realizarán las operaciones de terminación y de limpieza necesarias e informará al hilo primario de que ha parado utilizando para ello otro controlador de espera de eventos.

Como ejemplo, vamos a retomar la aplicación basada en la clase *Contador* que realizamos en el apartado *Finalizar un hilo* anteriormente en este mismo capítulo, para modificarla según lo explicado. La versión realizada entonces finalizaba el hilo al pulsar la tecla *Entrar*, pero sin ningún control. Supongamos ahora que el proceso del hilo requiere finalizar con una cuenta par cuando sea detenido. La solución sería la siguiente:

```
public class Contador
{
    private Thread hilo;
    private int iteraciones;

    private ManualResetEvent PararHilo;
    private ManualResetEvent HiloParado;

    public Contador() : this(null, 10) { } // constructor

    public Contador(string nombre, int it) // constructor
    {
        iteraciones = it;
        hilo = new Thread(ProcHilo); // objeto hilo
        if (nombre != null) hilo.Name = nombre;
        PararHilo = new ManualResetEvent(false);
        HiloParado = new ManualResetEvent(false);
    }

    public Thread HiloSubyacente
    {
        get { return hilo; }
    }

    public void ProcHilo()
    {
        for (int i = 1; i <= iteraciones; i++)
        {
```

```
        Console.WriteLine("{0}, cuenta: {1}", hilo.Name, i);
        Thread.Sleep(250);
        if (PararHilo.WaitOne(0, false))
        {
            // Tareas de finalización. Por ejemplo, finalizar sólo
            // cuando i es par
            if (i % 2 != 0) continue;
            HiloParado.Set(); // señal de que el hilo ha parado
            break;
        }
    }
    Console.WriteLine("Fin de la cuenta de {0}", hilo.Name);
}
```

```
public void Terminar()
{
    // Método invocado desde el hilo principal para
    // detener la ejecución del hilo secundario
    if (hilo == null || !hilo.IsAlive)
        return;

    // Cambiar el estado de este controlador de espera a señalizado
    // para informar al hilo secundario de que debe parar
    PararHilo.Set();

    // Esperar a que el hilo secundario informe de que ha parado
    while (hilo.IsAlive)
    {
        // Se espera por HiloParado un tiempo apropiado. Si durante
        // ese intervalo de tiempo HiloParado pasa a señalizado,
        // se ejecuta break sin esperar a que finalice el tiempo.
        if (HiloParado.WaitOne(100, false))
            break;
    }
    hilo = null;
}
}
```

Si la finalización del hilo principal dependiera de más de un controlador de espera de eventos, utilizaríamos **WaitAny** o **WaitAll**, dependiendo de nuestras exigencias. Por ejemplo, la llamada a **WaitOne** de *Terminar* podría escribirse también así:

```
if (WaitHandle.WaitAll(
    (new ManualResetEvent[] { HiloParado }), 100, false))
```

Interbloqueo

Cuando un hilo se queda complementemente bloqueado y no puede progresar porque no puede acceder a los recursos que necesita, ocurre lo que se denomina inanición (*starvation*); si esto ocurre entre dos o más hilos porque esperan por una condición recíproca que nunca puede ser satisfecha, estamos en un caso de *interbloqueo* (*deadlock*; algunos autores prefieren denominarlo *abrazo mortal*). Por ejemplo, dos hilos necesitan imprimir un documento almacenado en el disco, para lo que necesitan los recursos disco e impresora. Puesto que los hilos se están ejecutando paralelamente, suponga que uno ya ha adquirido el disco y el otro la impresora. Esto significa que ambos hilos quedarán bloqueados, cada uno de ellos esperando por el recurso que tiene el otro.

Para la mayoría de los programadores C#, la mejor forma de evitar el interbloqueo es prevenirlo, mejor que probar y detectarlo. En cualquier caso, cualquiera de las técnicas existentes para manejar los interbloqueos se sale fuera del objetivo de este capítulo.

GRUPO DE HILOS

Muchas aplicaciones crean hilos que pasan mucho tiempo en estado de inactividad, esperando a que se produzca un evento. Otros hilos pueden entrar en estado de inactividad y activarse periódicamente para comprobar si se ha producido algún cambio o para actualizar la información de estado. En casos como los enunciados, la agrupación de hilos permite utilizarlos de forma más eficaz, ya que suministra a la aplicación un grupo de hilos de trabajo administrados por el sistema. Un hilo supervisa el estado de varias operaciones de espera que se encuentran en la cola del grupo de hilos. Cuando una operación de espera finaliza, un hilo del grupo ejecuta la función de devolución de llamada correspondiente.

C# proporciona soporte para trabajar con grupos de hilos a través de la clase **ThreadPool** del espacio de nombres **System.Threading**.

PLANIFICACIÓN DE HILOS

Muchos ordenadores tienen sólo una UCP, así que los hilos que requieran ejecutarse deben compartirla. La ejecución de múltiples hilos sobre una única UCP, en cierto orden, es llamada *planificación*. Las bibliotecas de hilos, nativas o externas, normalmente soportan varios algoritmos de planificación. Por ejemplo, muchos sistemas soportan un algoritmo de planificación determinista (en cualquier momento se puede saber qué hilo se está ejecutando o cuánto tiempo continuará eje-

cutándose) muy simple, conocido como *fixed priority scheduling* (planificación por prioridad: el hilo que se elige para su ejecución es el de prioridad más alta).

Lo anteriormente expuesto significa que cada hilo tiene asignado una prioridad definida por un valor numérico entre un mínimo y un máximo, de forma que cuando varios hilos estén *preparados*, será elegido para su ejecución el de mayor prioridad. Solamente cuando la ejecución de ese hilo se detenga por cualquier causa, podrá ejecutarse un hilo de menor prioridad; y cuando un hilo con prioridad más alta que el que actualmente se está ejecutando se mueva al estado *preparado*, pasará automáticamente a ejecutarse.

Según lo expuesto, el sistema en cuestión no reemplazará el hilo actual en ejecución por otro hilo de la misma prioridad. En otras palabras, no aplicará una planificación por *cuantos* (*cuanto* o rodaja de tiempo (*time-slice*): tiempo máximo que un hilo puede retener la UCP; esta planificación da lugar a un sistema no determinista).

Después de lo dicho, sería bueno al programar que nuestros hilos cedieran voluntariamente el control algunas veces. Un hilo dado puede renunciar a su derecho de ejecutarse para ceder el control a otro de la misma o de mayor prioridad llamando al método *Thread.Sleep(0)*. Un intento de ceder la UCP a hilos de menor prioridad se ignorará.

La política de planificación por prioridades expuesta puede verse en algún momento alterada por el planificador. Por ejemplo, el planificador de hilos puede elegir para su ejecución a un hilo de menor prioridad para evitar que quede completamente bloqueado porque no pueda progresar por falta de los recursos necesarios para ello (puede morir por falta de recursos: inanición (*starvation*)). Por esta razón, la exactitud de los algoritmos programados no debe basarse en la prioridad de los hilos.

¿Qué ocurre con los hilos que tengan igual prioridad? Cuando todos los hilos que compiten por la UCP tienen la misma prioridad, el planificador elige para su ejecución al siguiente según el orden resultante de aplicar el algoritmo *round-robin* (no *preemptive*). En este caso, la cola de hilos listos para ejecutarse se trata como una cola circular *FIFO*. La UCP será cedida a otro hilo bien porque:

- un hilo de prioridad más alta ha alcanzado el estado de *preparado*;
- cede la UCP, o finaliza;
- se supera el *cuanto* (*quantum*): tiempo máximo que un hilo puede retener la UCP. Esta tercera condición sólo es aplicable en sistemas que soporten la planificación por *cuantos*.

Resumiendo, cuando se ejecuta un proceso que tiene varios hilos *preparados*, el sistema asigna la UCP en función de la prioridad que tenga asignada el hilo activo: de mayor prioridad a menor prioridad. Por otra parte, cuando el sistema asigna la UCP a un hilo, trata de igual forma a todos los hilos de la misma prioridad. Esto es, asigna un *cuanto* al primer hilo *preparado* de prioridad P, cuando éste finaliza su intervalo de tiempo asigna otro *cuanto* al siguiente hilo *preparado* de prioridad P y así sucesivamente. Cuando todos los hilos de prioridad P han tenido su intervalo de tiempo, se empieza otra vez por el primero.

Según lo expuesto, ¿cómo permitir la ejecución de hilos con prioridad inferior? La respuesta está en saber que muchos hilos del sistema son detenidos de vez en cuando, por motivos diferentes. Así, cuando todos los hilos de prioridad P estén detenidos, el sistema asigna *cuantos* a los hilos *preparados* de prioridad P-1. Un razonamiento análogo nos conduce a pensar que los hilos de prioridad P-2 sólo pueden ejecutarse cuando los hilos de prioridades P y P-1 estén detenidos. Parece entonces que los procesos de prioridad 1 nunca se ejecutarán, o que se ejecutarán de tarde en tarde. Pero la verdad es que no es así. La mayoría de los hilos consumen su tiempo durmiendo, lo que permite la ejecución de los hilos de prioridades bajas con una frecuencia, probablemente, un poco inferior.

Asignar prioridades a los hilos

La propiedad **Priority** de la clase **Thread** obtiene o establece un valor que indica la prioridad de programación de un hilo. El valor predeterminado es **ThreadPriority.Normal**. No obstante, es posible aumentar o disminuir esta prioridad. Para modificar la prioridad de un hilo utilice la sintaxis siguiente:

```
hilo.Priority = ThreadPriority.nueva_prioridad;
```

donde *nueva_prioridad* es un valor **Highest**, **AboveNormal**, **Normal**, **BelowNormal** o **Lowest** de la enumeración **ThreadPriority**.

Los sistemas operativos no están obligados a tener en cuenta la prioridad de un subproceso.

Como ejemplo, vamos a retomar la aplicación basada en la clase *Contador* que realizamos en el apartado *Detener un hilo de forma controlada* anteriormente en este mismo capítulo, para modificarla en el sentido de poder asignar prioridades a los hilos que la aplicación lance. Por ejemplo, supongamos que apoyándonos en la clase mencionada, ejecutamos el siguiente código:

```
using System;
using System.Threading;
```

```
////////////////////////////////////  
// Hilos.  
//  
public class Test  
{  
    public static void Main(string[] args)  
    {  
        Console.WriteLine("Pulse [Entrar] para finalizar la cuenta");  
        // Crear y lanzar los hilos contadores  
        Contador contador1 = new Contador("Contador-1");  
        Contador contador2 = new Contador("Contador-2");  
        Contador contador3 = new Contador("Contador-3",  
                                           ThreadPriority.BelowNormal);  
        EsperaAnimada punto = new EsperaAnimada(); // se auto-ejecuta  
        contador1.HiloSubyacente.Start();  
        contador2.HiloSubyacente.Start();  
        contador3.HiloSubyacente.Start();  
        // Permitir a los hilos contadores finalizar  
        Console.ReadLine();  
        contador1.Terminar();  
        contador2.Terminar();  
        contador3.Terminar();  
    }  
}
```

```
////////////////////////////////////
```

La solución que esperamos ver cuando se ejecute este código es similar a la siguiente:

Pulse [Entrar] para finalizar la cuenta

.

Contador-1 con prioridad Normal contó = 3.979.820

.

Contador-2 con prioridad Normal contó = 3.971.754

Contador-3 con prioridad BelowNormal contó = 22.480

Observamos que los hilos *Contador-1* y *Contador-2* con prioridad normal han realizado más o menos la misma cuenta y que *Contador-3* con prioridad por debajo de lo normal ha realizado una cuenta inferior. La prioridad es un valor definido en la enumeración **ThreadPriority**. Este dato será pasado al constructor *Contador* cuando creemos un nuevo objeto de esta clase. Si no se especifica se supondrá prioridad normal. Según lo expuesto, la clase *Contador* quedaría modificada de la siguiente forma:

```
using System;  
using System.Threading;
```



```

////////////////////////////////////
// Clase que define un hilo.
//
public class Contador
{
    private Thread hilo;

    private ManualResetEvent PararHilo;
    private ManualResetEvent HiloParado;

    public Contador() : this("Hilo ") { }
    public Contador(string nombre)
        : this(nombre, ThreadPriority.Normal) { }

    public Contador(string nombre, ThreadPriority p) // constructor
    {
        hilo = new Thread(ProcHilo); // objeto hilo
        hilo.Priority = p; // asignar la prioridad al hilo
        if (nombre != null) hilo.Name = nombre;

        PararHilo = new ManualResetEvent(false);
        HiloParado = new ManualResetEvent(false);
    }

    public Thread HiloSubyacente
    {
        get { return hilo; }
    }

    public void ProcHilo()
    {
        long i = 0;
        while (true)
        {
            i++;
            if (PararHilo.WaitOne(0, true))
            {
                // Tareas de finalización. Por ejemplo, finalizar sólo
                // cuando i es par
                if (i % 2 != 0) continue;
                Console.WriteLine("\n{0} con prioridad {1,-11} " +
                    "contó = {2,13}", Thread.CurrentThread.Name,
                    Thread.CurrentThread.Priority.ToString(),
                    i.ToString("N0"));

                HiloParado.Set(); // señal de que el hilo ha parado
                break;
            }
        }
    }
}

```

```
public void Terminar()
{
    // Detener la ejecución del hilo secundario
    if (hilo == null || !hilo.IsAlive)
        return;

    // Informar al hilo secundario de que debe parar
    PararHilo.Set();

    // Esperar a que el hilo secundario informe de que ha parado
    while (hilo.IsAlive)
    {
        if (HiloParado.WaitOne(100, true))
            break;
    }
    hilo = null;
}
}
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
```

Volviendo a la solución anterior, la línea de puntos que allí se muestra corresponde a una espera animada producida por un hilo en segundo plano de la clase *EsperaAnimada*:

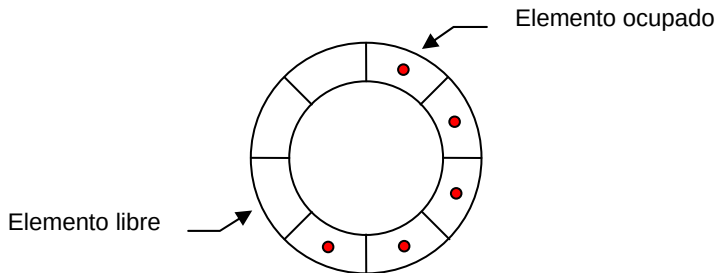
```
class EsperaAnimada
{
    // Hilo en segundo plano.
    // Muestra puntos para realizar una espera animada.
    private Thread hilo;

    public EsperaAnimada() // constructor
    {
        hilo = new Thread(ProcHilo); // objeto hilo
        hilo.IsBackground = true;
        hilo.Start();
    }

    public void ProcHilo()
    {
        while (true)
        {
            Console.Write(". ");
            Thread.Sleep(300);
        }
    }
}
```

EJERCICIOS RESUELTOS

1. Realizar una aplicación que utilice dos hilos, un productor y un consumidor, trabajando sobre una única matriz de enteros positivos. Esto es, un hilo productor generará enteros que almacenará en una matriz circular y un hilo consumidor obtendrá de esa matriz los enteros generados por el productor. Este problema fue resuelto anteriormente utilizando semáforos. Ahora se pide volverlo a escribir utilizando los métodos **Wait** y **PulseAll** y los contadores que sean necesarios para construir los semáforos.



La figura anterior muestra un esquema del problema del *productor* y el *consumidor*. Para una correcta sincronización entre los hilos, el productor deberá bloquear la matriz sólo mientras se esté insertando un dato y el consumidor lo hará sólo mientras se esté extrayendo. Cuando la matriz esté vacía, el hilo consumidor se pondrá a esperar hasta que haya datos. Así mismo, cuando la matriz esté llena, el hilo productor se pondrá a esperar hasta que haya elementos libres.

La matriz que almacenará los datos será un objeto de la clase *CMatriz*. Esta clase estará formada por los atributos:

<i>m</i>	Matriz de <i>n</i> enteros positivos.
<i>indProd</i>	Índice del elemento donde el productor debe insertar el siguiente elemento. Su valor será: 0, 1, 2,..., <i>n</i> -1, 0, 1, 2,...
<i>indCons</i>	Índice del elemento donde el consumidor debe obtener el siguiente elemento. Su valor será: 0, 1, 2,..., <i>n</i> -1, 0, 1, 2,...
<i>elementosVacíos</i>	Número de elementos vacíos en un instante determinado.
<i>elementosLlenos</i>	Número de elementos llenos en un instante determinado.

y por los métodos:

<i>almacenar</i>	Almacena un dato en el siguiente elemento vacío.
<i>obtener</i>	Obtiene el siguiente dato aún no extraído.

El código correspondiente a esta clase se muestra a continuación:

```
using System;
using System.Threading;
using System.Runtime.CompilerServices; // para Synchronized

////////////////////////////////////
// Sincronización de hilos: Wait y PulseAll.
//
public class CMatriz
{
    private int[] m;
    private int indProd = 0; // índice productor
    private int indCons = 0; // índice consumidor
    private int elementosVacíos, elementosLlenos;

    public CMatriz(int n)
    {
        if (n < 1) n = 10;
        m = new int[n];
        elementosVacíos = m.Length;
        elementosLlenos = 0;
    }

    [MethodImplAttribute(MethodImplOptions.Synchronized)]
    public void Almacenar(int num)
    {
        // Esperar a que haya elementos vacíos
        while (elementosVacíos == 0)
        {
            // El hilo se pone a esperar liberando el bloqueo sobre this.
            Monitor.Wait(this);
        }
        elementosVacíos--;
        elementosLlenos++;
        m[indProd] = num;
        indProd = (indProd + 1) % m.Length;
        // Despertar hilos;
        Monitor.PulseAll(this);
    }

    [MethodImplAttribute(MethodImplOptions.Synchronized)]
    public int Obtener()
    {
        // Esperar a que haya elementos llenos
        while (elementosLlenos == 0)
        {
            // El hilo se pone a esperar liberando el bloqueo sobre this.
            Monitor.Wait(this);
        }
        elementosVacíos++;
        elementosLlenos--;
        int num = m[indCons];
```

```

        indCons = (indCons + 1) % m.Length;
        // Despertar hilos;
        Monitor.PulseAll(this);
        return num;
    }
}
/////////////////////////////////////////////////////////////////

```

En el problema del productor y del consumidor los recursos que estos hilos deben adquirir para poder ejecutarse son los elementos vacíos y los elementos llenos de la matriz, respectivamente. Cada uno de estos tipos de recursos los representaremos por sendas variables que actuarán como semáforos: *elementosLlenos* y *elementosVacíos*. Un valor cero equivale a semáforo en rojo y un valor distinto de cero a semáforo en verde.

elementosLlenos es un semáforo inicialmente en rojo para el consumidor (porque no hay ningún elemento lleno, esto es, no se puede *obtener*) que representa los elementos actualmente llenos de la matriz y *elementosVacíos* es un semáforo inicialmente en verde para el productor (porque todos los elementos están vacíos, esto es, se puede *almacenar*) que representa los elementos actualmente vacíos de la matriz. Por lo tanto, el contador de *elementosLlenos* debe valer inicialmente cero y el de *elementosVacíos* debe valer *m.Length*.

Cuando un hilo necesita un recurso de un tipo particular, decrementa el contador del semáforo correspondiente, y cuando lo libera lo incrementa. Por ejemplo, cuando el productor quiere *almacenar* un dato necesita el recurso “elementos vacíos”, de tal forma que cada vez que lo adquiere lo decrementa; cuando llegue a cero implica semáforo en rojo indicando que el recurso “elementos vacíos” está ocupado. Lógicamente decrementar *elementosVacíos* implica incrementar *elementosLlenos*.

```

// Esperar a que haya elementos vacíos
while (elementosVacíos == 0)
{
    // El hilo se pone a esperar liberando el bloqueo sobre this.
    Monitor.Wait(this);
}
elementosVacíos--;
elementosLlenos++;
m[indProd] = num;
indProd = (indProd + 1) % m.Length;
// Despertar hilos;
Monitor.PulseAll(this);

```

El código anterior, que pertenece al hilo productor, decrementa el contador del semáforo *elementosVacíos*, inserta un dato en el siguiente elemento vacío de

la matriz y, lógicamente, incrementa el contador del semáforo *elementosLlenos*. Si el contador de *elementosVacíos* fuera cero, el hilo productor pasaría al estado bloqueado hasta que el hilo consumidor extraiga uno o más datos y, por consiguiente, incremente *elementosVacíos*. Un razonamiento análogo haríamos para el consumidor.

Las clases *CHilo*, *Productor*, *Consumidor* y *Test* son las mismas que implementamos anteriormente.

2. Se desea realizar una aplicación que simule una carrera de caballos en un hipódromo. En el hipódromo existe un tablón que muestra en todo momento un marcador que indica la posición de cada caballo durante la carrera. La carrera se dará por finalizada cuando llegue el primer caballo.

Analizando el enunciado, distinguimos: el hipódromo, el tablón, el marcador que aparece sobre el tablón y los caballos.

- El hipódromo será un objeto de la clase *CHipodromo*, que aportará el método **Main**. Esto es, *CHipodromo* será el hilo principal, ya que sobre él se realiza la carrera, lo que implica poner en marcha el marcador del tablón donde vayan a ser expuestos los resultados, y dar salida a los caballos participantes.
- El tablón incluirá un marcador que refleje la posición de cada caballo durante la carrera y el número de participantes. Se corresponderá con un objeto de la clase *CTablon*.
- El marcador será un hilo de ejecución de la clase *CMarcador*. Su misión es mostrar las posiciones de los participantes sobre el tablón.
- Los caballos se corresponderán con hilos de ejecución de la clase *CCaballo*.

Según lo expuesto, el método **Main** de la clase *CHipodromo* definirá el tablón y los participantes, creará e iniciará el marcador, y creará los caballos e iniciará la carrera:

```
public class CHipodromo
{
    public static void Main(string[] args)
    {
        string nomCaballo = null;
        const int num_participantes = 5;

        CTablon Tablon = new CTablon(num_participantes);
        CCaballo[] participante = new CCaballo[num_participantes];
        CMarcador Marcador = new CMarcador(Tablon);

        // Iniciar el hilo marcador
        Marcador.HiloSubyacente.Start();
    }
}
```

```

    for (int i = 0; i < num_participantes; ++i)
    {
        // Datos de los participantes
        nomCaballo = "Caballo " + i;
        participante[i] = new CCaballo(i, nomCaballo, Tablon);
        // Iniciar los hilos de los caballos participantes
        participante[i].HiloSubyacente.Start();
    }
}
}

```

El tablón es una clase con dos atributos: una matriz dinámica que almacenará las posiciones de los participantes durante la carrera y el número de participantes. Así mismo, proporciona métodos para actualizar la posición de cada participante a medida de que éste va avanzando, para indicar el final de la carrera, y para obtener la posición actual de un participante, o bien el número de participantes.

```

public class CTablon
{
    private int[] m_Posicion; // posición de los caballos durante
                             // la carrera
    private int m_nParticipantes; // número de participantes

    public CTablon(int participantes)
    {
        // Crear la matriz de posiciones de los participantes.
        // Los elementos son automáticamente iniciados a cero.
        m_nParticipantes = participantes;
        m_Posicion = new int[m_nParticipantes];
    }

    public bool FinCarrera()
    {
        for(int i = 0; i < m_nParticipantes; ++i)
        {
            if (m_Posicion[i] == 75) // distancia recorrer
                return true;         // final de la carrera
        }
        return false;                // continuar
    }

    public void IncrementarPosicion(int dorsal)
    {
        m_Posicion[dorsal]++; // participante avanza
    }

    public int Posicion(int dorsal)
    {
        return (m_Posicion[dorsal]); // posición actual
    }
}

```

```
public int NumParticipantes()
{
    return m_nParticipantes; // número de participantes
}
}
```

El marcador es un objeto hilo de la clase *CMarcador* derivada de *CHilo*. Su misión es mostrar en cada instante (en el ejemplo, cada 500 milisegundos) la distancia recorrida por los caballos. El resultado se presenta de la forma siguiente:

POSICIONES DE CARRERA

```
-----
.....*
```

Para ello, el método *ProcHilo* (método que contiene el código que ejecutará el hilo) obtendrá del tablón la posición de cada participante y la mostrará.

```
public class CMarcador : CHilo
{
    private CTablon m_Tablon; // acceso al tablón

    public CMarcador(CTablon Tablon)
    {
        m_Tablon = Tablon;
    }

    public override void ProcHilo()
    {
        int nParticipantes;
        nParticipantes = m_Tablon.NumParticipantes();
        while(!m_Tablon.FinCarrera())
        {
            Console.WriteLine("POSICIONES DE CARRERA");
            Console.WriteLine("-----");
            for (int i = 0; i < nParticipantes; ++i)
            {
                for (int p = 0; p < m_Tablon.Posicion(i)-1; ++p)
                    Console.Write("."); // distancia recorrida
                Console.WriteLine("*"); // * = caballo
            }
            Thread.Sleep(500);
        }
    }
}
```


Cada caballo participante de la carrera es un objeto hilo de la clase *CCaballo* derivada de *CHilo* (clase que ya fue expuesta anteriormente en este mismo capítulo). El código que ejecuta este hilo (método *ProcHilo*) simplemente avanza la posición del participante. El resto de los métodos, se explican por sí mismos.

```
public class CCaballo : CHilo
{
    private CTablon Tablon; // acceso al tablón
    private int dorsal;      // número de caballo
    private string nombre;   // nombre del caballo

    public CCaballo()
    {
        nombre = "Caballo desconocido";
        dorsal = 0;
    }

    public CCaballo(int dor, string nom, CTablon t)
    {
        dorsal = dor;
        nombre = nom;
        Tablon = t;
    }

    public override void ProcHilo()
    {
        // El argumento de Random es para asegurar que el valor
        // inicial de la secuencia de números pseudoaleatorios es
        // diferente para cada caballo y para cada ejecución.
        Random rnd = new Random(
            unchecked((int)DateTime.Now.Ticks)*(dorsal+1));
        int limiteInf = 1, limiteSup = 1000; // milisegundos
        while(!Tablon.FinCarrera())
        {
            Thread.Sleep((int)(
                (limiteSup-limiteInf+1) * rnd.NextDouble() + limiteInf));
            // Avanzar
            Tablon.IncrementarPosicion(dorsal);
        }
    }
}
```

EJERCICIOS PROPUESTOS

1. Escribir una aplicación que lance tres hilos que ordenen otras tantas matrices, todas de la misma dimensión, utilizando: el primero el método de ordenación de la burbuja, el segundo el de inserción y el tercero el método *quicksort*. Visualizar

como resultado el nombre de los métodos de ordenación colocados de más rápido a menos rápido.

2. Se desea realizar una aplicación que simule el funcionamiento de un aeropuerto con p pistas. En el aeropuerto existe una torre de control encargada de determinar qué aviones pueden aterrizar o despegar. Cuando un avión está en una pista ningún otro puede hacer uso de la misma. Cada avión realizará lo siguiente:
 - Cuando esté volando y quiera aterrizar, llamará a la torre de control para preguntar si puede hacerlo. Si la torre de control le da permiso, le asignará una pista libre. Si no hay pistas libres, le indicará que debe esperar y seguir volando hasta que se le asigne una pista.
 - Cuando se le asigne una pista, el avión aterrizará, dejará a los pasajeros que lleva, recogerá a los nuevos, despegará y volverá a volar.

Implemente una aplicación que lance $a > p$ hilos de ejecución que simulen otros tantos aviones, cuyo comportamiento será el descrito. La aplicación mostrará por pantalla la secuencia de acciones que se vayan sucediendo por cada uno de los aviones.

INTERFACES GRÁFICAS Y APLICACIONES PARA INTERNET

Ahora que ya tiene una visión global de las características del lenguaje C# y conoce los mecanismos de la programación orientada a objetos, la estructura de un programa, las clases de la biblioteca de .NET más utilizadas, las sentencias de control, el trabajo con matrices, cómo escribir sus propias clases, subclases, espacios de nombres e interfaces, cómo dar tratamiento a las excepciones, cómo trabajar con ficheros, en general con flujos, cómo crear estructuras dinámicas de datos, cómo lanzar hilos de ejecución, en una palabra, ahora que ya ha aprendido a programar en C#, ¿qué puede hacer a partir de aquí? Pues el siguiente objetivo importante es crear aplicaciones con interfaz gráfica y aplicaciones para Internet.

Esto es, nos referimos a programas que interactúan con los usuarios a través de ventanas y con otros programas a través de Internet; de todo esto haremos una breve introducción a continuación. Esto requiere aprender a utilizar las clases diseñadas para desarrollar entornos de ventanas y para interactuar con Internet. Pero hay muchas cosas más para crear aplicaciones profesionales para la Web; entre ellas destacan ADO.NET (acceso a bases de datos desde C#) y ASP.NET (clientes Web, servicios Web y páginas Web).

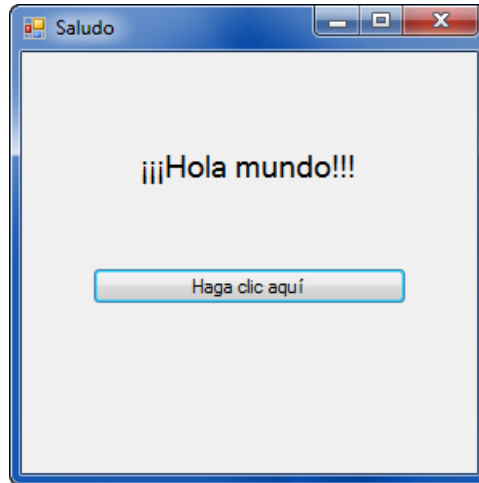
Como comprenderá, todo lo que hay a partir de aquí da para escribir más de un libro; el autor ha escrito uno titulado *Enciclopedia de Microsoft Visual C#* cuyo contenido puede ver en http://www.fjceballos.es/publicaciones_cs.html. Con el fin de que se haga una idea de lo que estamos hablando, este capítulo simplemente trata de poner en su conocimiento algunos de los nuevos temas que puede abordar si está interesado en desarrollar con C#. Por eso, a continuación haremos una breve introducción a las interfaces gráficas y a las aplicaciones para Internet, ya que el objetivo de este capítulo no es otro que transmitirle que con C# puede hacer,

además de aplicaciones para trabajar en una máquina a nivel local, múltiples programas para trabajar en Internet con objetivos muy diferentes y, por supuesto, con interfaces gráficas en ambos casos.

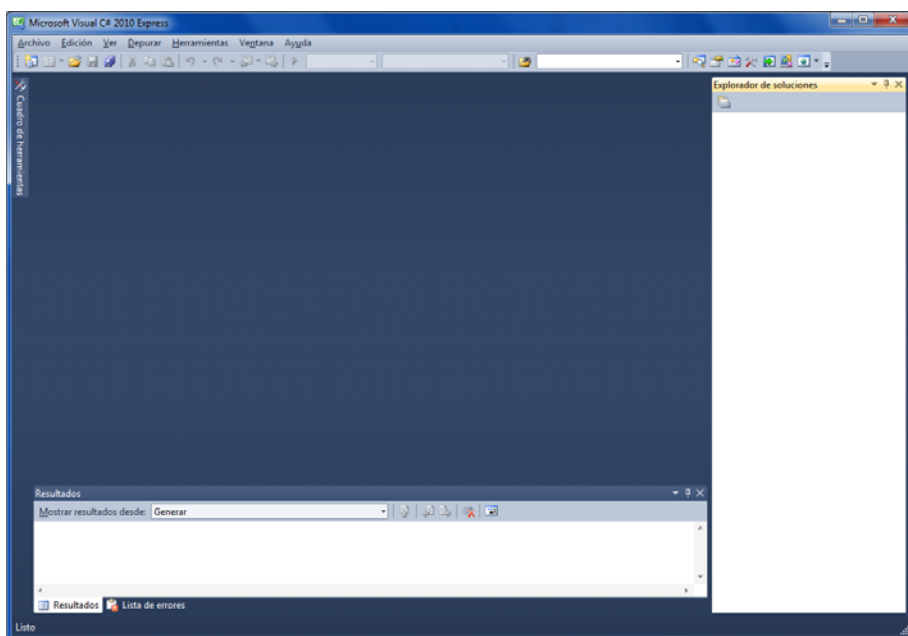
INTERFACES GRÁFICAS

Visual C# Express (o Visual Studio) permite diseñar la interfaz gráfica de una aplicación de manera visual, sin más que arrastrar con el ratón los controles que necesitemos sobre la ventana destino de los mismos. Una rejilla mostrada sobre la ventana nos ayudará a colocar estos controles y a darles el tamaño adecuado, y una página de propiedades nos facilitará la modificación de los valores de las propiedades de cada uno de los controles. Todo lo expuesto, lo realizaremos sin tener que escribir ni una sola línea de código. Después, un editor de código inteligente nos ayudará a escribir el código necesario y detectará los errores sintácticos que introduzcamos, y un depurador nos ayudará a poner a punto nuestra aplicación cuando lo necesitemos.

Como ejemplo, vamos a realizar una aplicación Windows denominada *Saludos*, que presente una interfaz al usuario como la de la figura siguiente:



Para empezar, arranque Visual C# Express. Se visualizará una ventana como la siguiente:

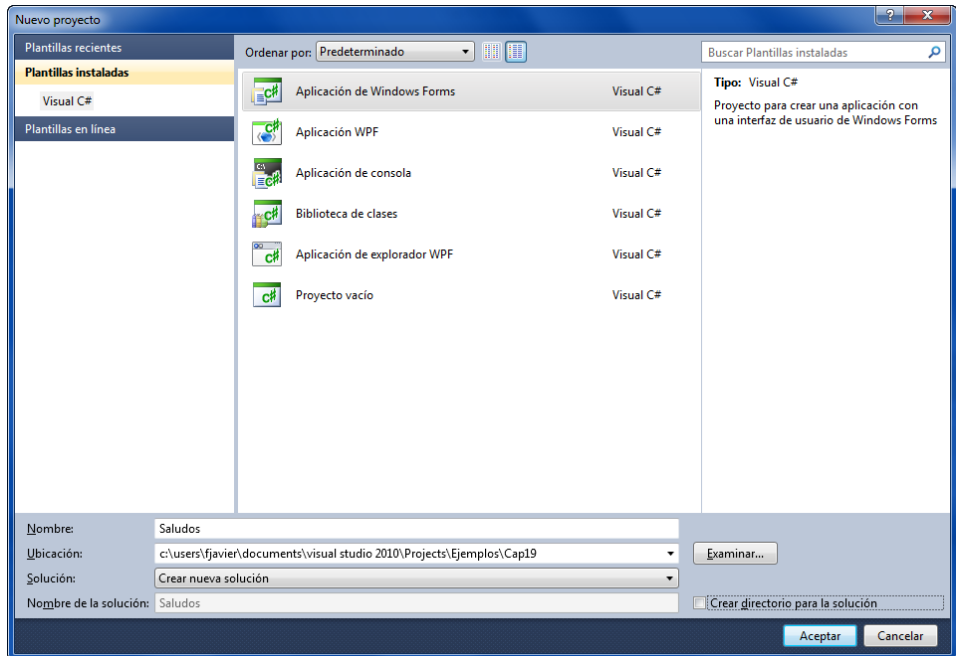


¿Cuáles son los siguientes pasos para desarrollar una aplicación Windows? En general, para construir una aplicación de este tipo con Visual C# Express, siga los pasos indicados a continuación:

1. Cree un nuevo proyecto (una nueva aplicación), entendiendo por proyecto, un conjunto de ficheros, normalmente distribuidos en carpetas, y recursos que pueden ser compilados como una sola unidad. Visual C# Express mostrará una página de diseño con un formulario vacío por omisión.
2. Dibuje los controles sobre el formulario. Los controles serán tomados de una caja de herramientas.
3. Defina las propiedades del formulario y de los controles.
4. Escriba el código para controlar los eventos, que consideremos, de cada uno de los objetos.
5. Guarde, compile y ejecute la aplicación.
6. Opcionalmente, utilice un depurador para poner a punto la aplicación.

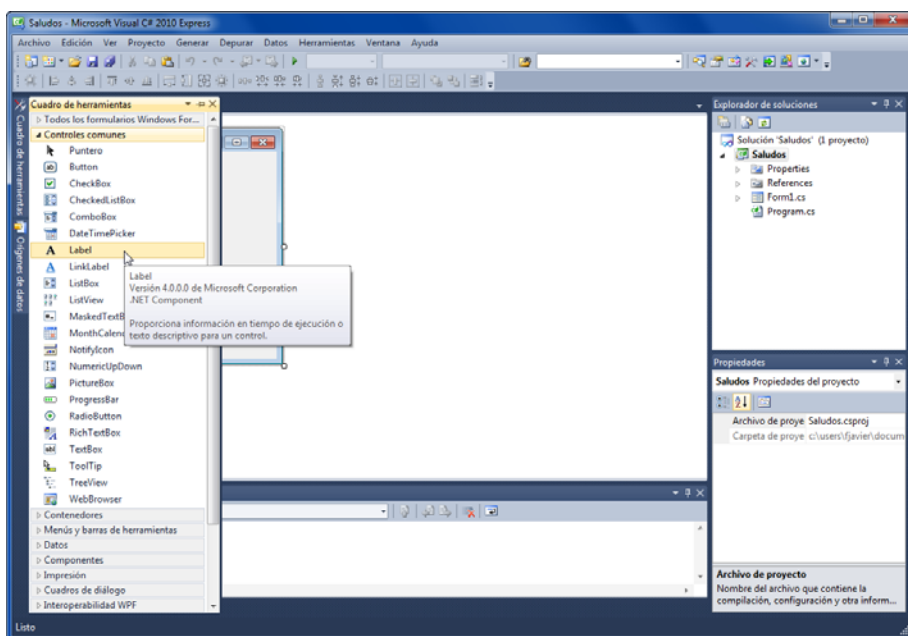
Crear un nuevo proyecto

Para crear un nuevo proyecto, diríjase a la barra de menús y ejecute *Archivo > Nuevo Proyecto*. En el diálogo que se visualiza, seleccione el tipo de proyecto *Visual C#*, la plantilla *Aplicación para Windows Forms*, el nombre *Saludos*, la ubicación y haga clic en el botón *Aceptar*:



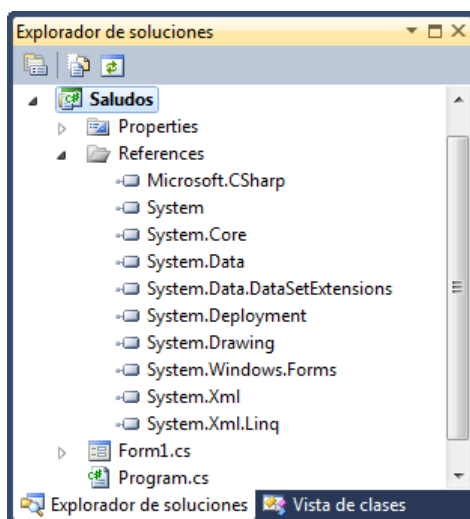
Obsérvese que se ha elegido la carpeta donde se almacenará el proyecto. Esta tarea puede posponerse sin que afecte al desarrollo de la aplicación. Ahora bien, si desea que esta tarea se realice automáticamente en el momento de crear el proyecto, caso del autor, ejecute *Herramientas > Opciones*, seleccione la opción *Proyectos y Soluciones* y marque la casilla *Guardar los proyectos nuevos al crearlos*.

Después de crear una nueva aplicación Windows, el entorno de desarrollo Visual C# Express mostrará un formulario, *Form1*, en el diseñador. También pondrá a nuestra disposición una caja de herramientas con una gran cantidad de controles listos para ser incluidos en un formulario.



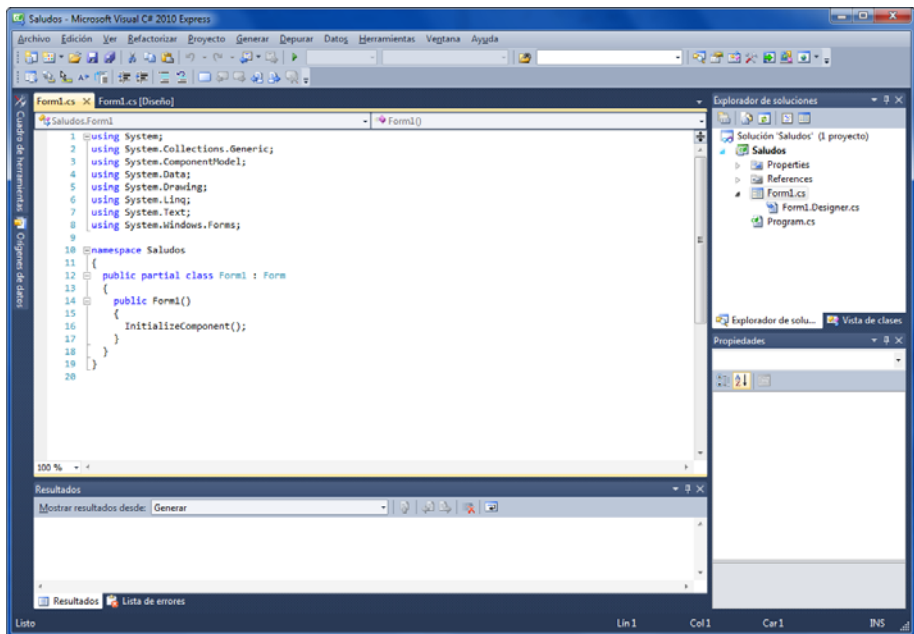
Otra característica interesante de este entorno de desarrollo es la ayuda dinámica que facilita. Se trata de un sistema de ayuda inteligente sensible al contexto; esto es, automáticamente, mientras escribe, se mostrará la ayuda relacionada con el contexto de trabajo.

En la esquina superior derecha también se localiza otra ventana con una o más páginas: explorador de soluciones, vista de clases, etc.; en la figura siguiente vemos el *Explorador de soluciones*:



El explorador de soluciones muestra el nombre de la solución (una solución engloba uno o más proyectos), el nombre del proyecto (un proyecto administra los ficheros que componen la aplicación) y el de todos los formularios, referencias, recursos y demás elementos de la aplicación; en nuestro caso, observamos un formulario, denominado *Form1*, descrito por los ficheros de código *Form1.cs* y *Form1.Designer.cs*; el primero es el utilizado por el programador para escribir el código y el segundo, el utilizado por el diseñador de formularios. También se observa un nodo *References* que agrupa las referencias a las bibliotecas de clases de objetos que utilizará la aplicación en curso; podemos añadir nuevas referencias a otras bibliotecas haciendo clic con el botón derecho del ratón sobre ese nodo.

Así mismo, en su parte superior, muestra una barra de botones que permiten ver el *código*, el *diseñador* de formularios, la ventana de *propiedades*, etc. Por ejemplo, si estamos viendo el diseñador de formularios y hacemos clic en el botón *ver código*, la página de diseño será sustituida por el editor de código, como se puede observar en la figura siguiente:

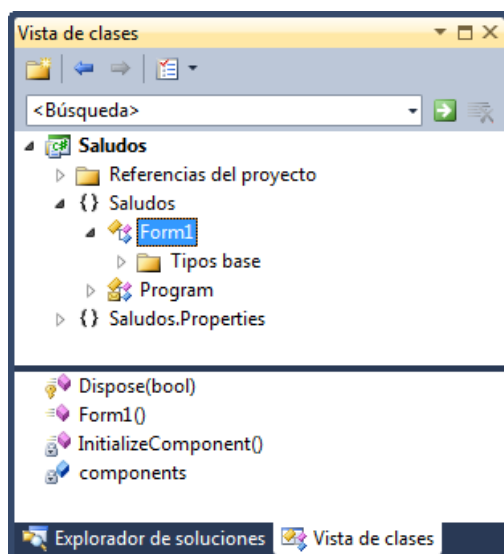


Una característica digna de resaltar del editor de Visual C# Express es la incorporación de bloques de código contraíbles. En la figura superior podemos ver varios de estos bloques; si hacemos clic en el nodo *-*, contraeremos el bloque y el nodo se convertirá en otro *+* que permitirá expandir de nuevo el bloque.

Otra característica del editor es la finalización y el formato de código automáticos. Por ejemplo, al escribir un método, el editor mostrará automáticamente la

ayuda en línea de la palabra clave (**public**, **void**, **int**, etc.) que intenta escribir; si escribimos una sentencia **if** exactamente igual. Puede personalizar las características del editor ejecutando *Herramientas > Opciones > Editor de texto*.

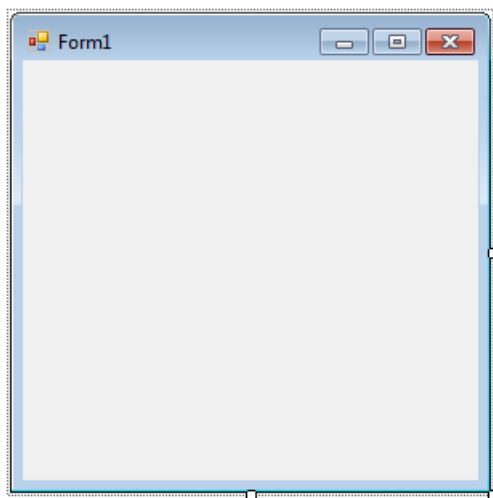
Si cuando se está visualizando el explorador de soluciones desea mostrar la vista de clases de su aplicación, sólo tiene que hacer clic en la pestaña *Vista de clases* (si no está presente esta pestaña, haga clic en *Ver > Vista de clases*). Esta ventana en su parte superior muestra las clases que componen la aplicación y en su parte inferior los métodos pertenecientes a la clase seleccionada.



Expandiendo el nodo del proyecto, vemos, en primer lugar, el espacio de nombres al que pertenece: *Saludos* (un espacio de nombres define un ámbito). Si ahora expandimos este otro nodo, veremos que incluye una clase, la que define el objeto *Form1*, y si expandimos a su vez este nodo, podremos observar su clase base. En la figura podemos ver seleccionada la clase *Form1*, que define un constructor, *Form1*, los métodos *Dispose* e *InitializeComponent* y el atributo *components*.

El formulario

El formulario es el plano de fondo para los controles. Después de crear un nuevo proyecto, la página de diseño muestra uno como el de la figura siguiente. Lo que ve en la figura es el aspecto gráfico de un objeto de la clase *Form1*. Para modificar su tamaño ponga el cursor del ratón sobre alguno de los lados del cuadrado que lo rodea y arrastre en el sentido deseado.



Si ahora ejecutamos este programa, para lo cual podemos pulsar las teclas *Ctrl+F5*, o bien elegir la orden *Iniciar sin depurar* del menú *Depurar* (si esta orden no está presente puede agregarla ejecutando *Herramientas > Personalizar*), aparecerá sobre la pantalla la ventana, con el tamaño asignado, y podremos actuar sobre cualquiera de sus controles, o bien sobre las órdenes del menú de control, para minimizarla, maximizarla, moverla, ajustar su tamaño, etc. Ésta es la parte que el diseñador de Visual C# realiza por nosotros y para nosotros; pruébelo. Finalmente, para cerrar la ejecución de la aplicación disponemos de varias posibilidades:

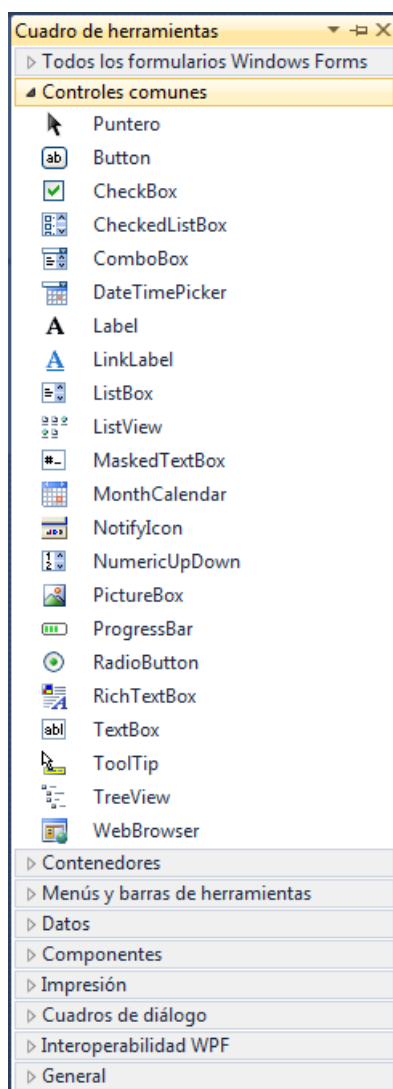
1. Hacer clic en el botón  que cierra la ventana.
2. Hacer un doble clic en el icono situado a la izquierda en la barra de título de la ventana.
3. Activar el menú de control de la ventana *Form1* y ejecutar *Cerrar*.
4. Pulsar las teclas *Alt + F4*.

Dibujar los controles

En Visual C# disponemos fundamentalmente de dos tipos de objetos: *ventanas* y *controles*. Las ventanas son los objetos sobre los que se dibujan los controles como cajas de texto, botones o etiquetas, dando lugar a la interfaz gráfica que el usuario tiene que utilizar para comunicarse con la aplicación y que genéricamente denominamos formulario.

Para añadir un control a un formulario, utilizaremos la caja de herramientas que se muestra en la figura siguiente. Cada herramienta de la caja crea un único control. El significado de los controles más comunes se expone a continuación.

Puntero. El puntero no es un control. Se utiliza para seleccionar, mover y ajustar el tamaño de los objetos.



Label. Una *etiqueta* permite mostrar un texto de una o más líneas que no pueda ser modificado por el usuario. Son útiles para dar instrucciones al usuario.

LinkLabel. Se trata de una etiqueta de Windows que puede mostrar hipervínculos.

Button. Un *botón de pulsación*, normalmente tendrá asociada una orden con él. Esta orden se ejecutará cuando el usuario haga clic sobre el botón.

TextBox. Una *caja de texto* es un área dentro del formulario en la que el usuario puede escribir o visualizar texto.

MaskedTextBox. Una *caja de texto* mejorada que soporta una sintaxis declarativa para aceptar o rechazar la entrada del usuario.

MenuStrip. Permite añadir una barra de menús a una ventana. Reemplaza y mejora al control *MainMenu* de versiones anteriores.

CheckBox. Una *casilla de verificación* se utiliza para seleccionar una opción. Utilizando estos controles se pueden elegir varias opciones de un grupo.

RadioButton. El control *botón de opción* se utiliza para seleccionar una opción entre varias. Utilizando estos controles se puede elegir una opción de un grupo de ellas.

GroupBox. Un *marco* se utiliza para realzar el aspecto del formulario. También los utilizamos para formar grupos de botones de opción, o bien para agrupar controles relacionados entre sí.

PictureBox. Una *caja de imagen* se utiliza normalmente para mostrar gráficos de un fichero de mapa de bits, meta-archivo, icono, JPEG, GIF o PNG.

Panel. Control que actúa como contenedor de otros controles.

FlowLayoutPanel. Representa un panel que coloca dinámicamente su contenido vertical u horizontalmente.

TableLayoutPanel. Representa un panel que coloca dinámicamente su contenido en una rejilla de filas y columnas.

DataGridView. Proporciona una tabla para visualizar los datos de una forma personalizada.

ListBox. El control *lista fija* (lista desplegada) contiene una lista de elementos de la que el usuario puede seleccionar uno o varios elementos.

CheckedListBox. Se trata de un control *lista fija* en el que se muestra una casilla de verificación a la izquierda de cada elemento.

ComboBox. El control *lista desplegable* combina una caja de texto y una lista desplegable. Permite al usuario escribir lo que desea seleccionar o elegir un elemento de la lista.

ListView. El control *vista de lista*, muestra una colección de elementos que se pueden visualizar mediante una de cuatro vistas distintas.

TreeView. Muestra una colección jerárquica de elementos con etiquetas. Se trata de una estructura en árbol en la que cada nodo del mismo es un objeto de la clase **TreeNode**.

TabControl. Es un control que agrupa un conjunto relacionado de páginas de fichas.

DateTimePicker. Control que permite seleccionar la fecha y hora.

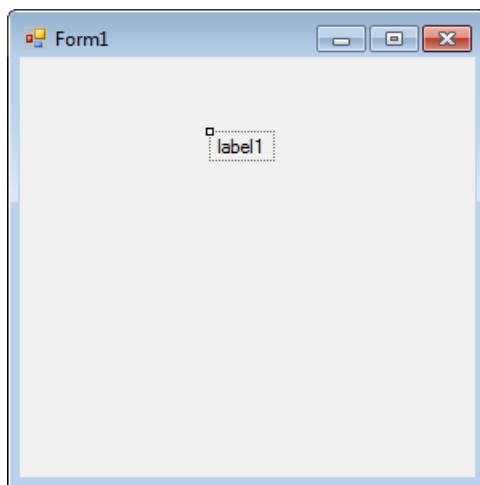
MonthCalendar. Control de calendario mensual.

HScrollBar y *VScrollBar*. La barra de desplazamiento horizontal y la barra de desplazamiento vertical permiten seleccionar un valor dentro de un rango de valores. Estos controles son utilizados independientemente de otros objetos, y no son lo mismo que las barras de desplazamiento de una ventana.

Timer. El temporizador permite activar procesos a intervalos regulares de tiempo.

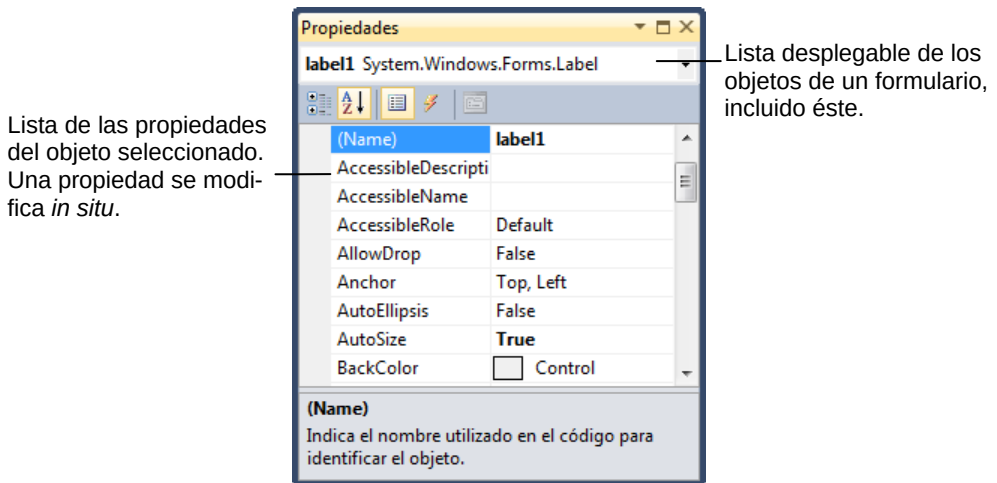
Otros controles de interés son la barra de progreso (*ProgressBar*), la caja de texto enriquecido (*RichTextBox*), las descripciones breves (*ToolTip*), la barra de estado (*StatusStrip*), las cajas de diálogo estándar (*OpenFileDialog*, *FontDialog*, *PrintDialog*,...), etc.

Siguiendo con nuestra aplicación, seleccionamos de la caja de herramientas que acabamos de describir los controles que vamos a utilizar. En primer lugar vamos a añadir al formulario una etiqueta. Para ello, hacemos clic sobre la herramienta etiqueta y, sin soltar el botón del ratón, la arrastramos sobre el formulario. Cuando soltemos el botón del ratón aparecerá una etiqueta de un tamaño predeterminado, según se muestra en la figura siguiente:

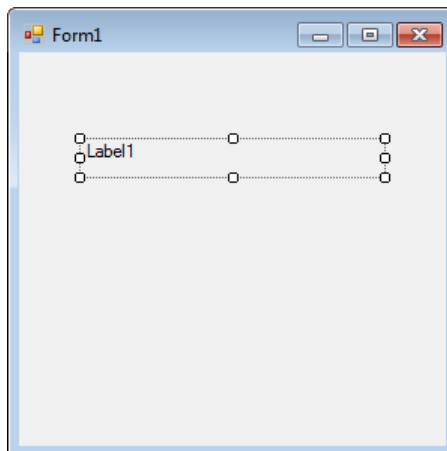


Observe en la página de propiedades del entorno de desarrollo, mostrada en la figura siguiente, las propiedades **Name**, nombre, y **AutoSize**, autoajustable. La primera tiene asignado el valor *label1* que es el nombre por defecto dado al control etiqueta, y la segunda tiene asignado por defecto el valor **True**, lo que hace que el tamaño de la etiqueta se ajuste automáticamente a su contenido. Si quiere ajustar su tamaño manualmente, debe asignar a esta propiedad el valor **False**.

El nombre de un control se utiliza para referirnos a dicho control en el código de la aplicación.



Para practicar un poco más, ponga la propiedad **AutoSize** a valor **False**. Después ajuste su tamaño y céntrela horizontalmente. Para realizar esta última operación, seleccione el control y ejecute la orden *Formato > Centrar en el formulario > Horizontalmente*.



Ahora se observa sobre la etiqueta un rectángulo con unos cuadrados distribuidos a lo largo de su perímetro, que reciben el nombre de *modificadores de tamaño*, indicando que se puede modificar el tamaño del control que estamos dibujando. Para modificar el tamaño de un control, primero selecciónelo haciendo clic sobre él, después apunte con el ratón a alguno de los lados del rectángulo que lo envuelve, observe que aparece una doble flecha (si el lado tiene un modificador de tamaño), y, entonces, con el botón izquierdo del ratón pulsado, arrastre en el sentido que desee ajustar el tamaño.

También puede mover el control a un lugar deseado dentro del formulario. Para mover un control, primero selecciónelo haciendo clic sobre él y después apunte con el ratón a alguna zona perteneciente al mismo y, con el botón izquierdo del ratón pulsado, arrastre hasta situarlo en el lugar deseado.

Borrar un control

Para borrar un control, primero se selecciona haciendo clic sobre él y, a continuación, se pulsa la tecla *Supr* (*Del*). Para borrar dos o más controles, primero se seleccionan haciendo clic sobre cada uno de ellos, al mismo tiempo que se mantiene pulsada la tecla *Ctrl*, y después se pulsa *Supr*.

Se pueden seleccionar también dos o más controles contiguos, pulsando el botón izquierdo del ratón y arrastrando el ratón hasta rodearlos.

Propiedades de los objetos

Cada clase de objeto tiene predefinido un conjunto de propiedades, como nombre, tamaño, color, etc. Las propiedades de un objeto representan todos los atributos que por definición están asociados con ese objeto.

Algunas propiedades las tienen varios objetos y otras son únicas para un objeto determinado. Por ejemplo, la propiedad **TabIndex** (orden *Tab*) la tienen muchos objetos, pero la propiedad **Interval** sólo la tiene el temporizador. Cuando se selecciona más de un objeto, la página de propiedades visualiza las propiedades comunes a esos objetos.

Cada propiedad de un objeto tiene un valor por defecto que puede ser modificado *in situ* si se desea. Por ejemplo, la propiedad **Name** del formulario del ejemplo que nos ocupa, tiene el valor *Form1*.

Para cambiar el valor de una propiedad de un objeto, siga los pasos indicados a continuación:

1. Seleccione el objeto. Para ello, haga clic sobre el objeto o pulse sucesivamente la tecla *Tab* hasta que esté seleccionado (el control seleccionado aparecerá rodeado por un rectángulo con los modificadores de tamaño).
2. Seleccione en la lista de propiedades la propiedad que desea cambiar.
3. Modifique el valor que actualmente tiene la propiedad seleccionada. El valor actual de la propiedad en cuestión aparece escrito a continuación del nombre de la misma. Para cambiar este valor, sobrescriba el valor actual o, si es posible, seleccione uno de la lista que se despliega haciendo clic sobre la flecha (▾) que aparece a la derecha del valor actual. Para algunas propiedades, esta flecha es sustituida por tres puntos (...). En este caso se visualizará una caja de diálogo.

Se puede también modificar una propiedad durante la ejecución de la aplicación. Esto implica añadir el código necesario en el método que deba realizar la modificación.

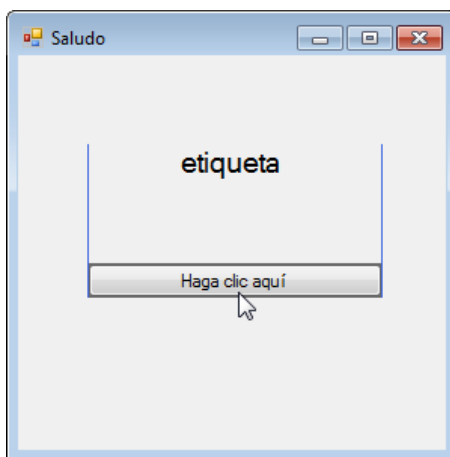
Para verificar el valor de una misma propiedad en varios objetos, seleccione ésta en la página de propiedades para uno de ellos y, a continuación, pase de un objeto al siguiente haciendo clic con el ratón sobre cada uno de ellos, o simplemente pulsando la tecla *Tab*.

Siguiendo con nuestro ejemplo, vamos a cambiar el título *Form1* del formulario por el título *Saludo*. Para ello, seleccione el formulario y a continuación la propiedad **Text** en la página de propiedades. Después, sobrescriba el texto “Form1” con el texto “Saludo”.

Veamos ahora las propiedades de la etiqueta. Seleccione la etiqueta y observe la lista de propiedades. Algunas de estas propiedades son **BackColor** (color del fondo de la etiqueta), **Name** (identificador de la etiqueta para referirnos a ella en el código) y **Text** (contenido de la etiqueta). Siguiendo los pasos descritos anteriormente, cambie el valor actual de la propiedad **Name** al valor *etSaludo*, el valor *Label1* de la propiedad **Text** a “etiqueta” y alinee este texto para que se muestre centrado tanto horizontal como verticalmente; esto requiere asignar a la propiedad **TextAlign** el valor *MiddleCenter*. A continuación, vamos a modificar el tipo de la letra de la etiqueta. Para ello, seleccione la propiedad **Font** en la página de propiedades, pulse el botón situado a la derecha del valor actual de la propiedad y elija como tamaño, por ejemplo, 14; las otras características las dejamos como están.

El paso siguiente será añadir un botón. Para ello, hacemos clic sobre la herramienta *Button* de la caja de herramientas y arrastramos el botón sobre el formulario. Movemos el botón y ajustamos su tamaño para conseguir el diseño

que observamos en la figura siguiente. Ahora modificamos sus propiedades y asignamos a **Text** (título) el valor *Haga clic aquí*, y a **Name**, el valor *btSaludo*.



También observamos que al colocar el control aparecen unas líneas indicando la alineación de éste con respecto a otros controles. Es una ayuda para alinear los controles que coloquemos dentro del formulario. Puede elegir entre los modos *SnapLines* (líneas de ayuda), es el modo que estamos utilizando, o *SnapToGrid* (rejilla de ayuda; se visualizan los puntos que dibujan la rejilla). Para elegir el modo de ayuda, ejecute *Herramientas > Opciones*, seleccione la opción *Diseñador de formularios Windows* y asigne a la propiedad **LayoutMode** el modo deseado. Para que las opciones elegidas tengan efecto, tiene que cerrar el diseñador y volverlo a abrir.

Bloquear la posición de todos los controles

Una vez que haya ajustado el tamaño de los objetos y haya situado los controles en su posición definitiva, puede seleccionar el formulario y bloquear sus controles para que no puedan ser movidos accidentalmente. Para ello, ejecute la orden *Bloquear controles* del menú *Formato*. Para desbloquearlos, proceda de la misma forma.

Icono de la aplicación

Todos los formularios visualizan un icono en la esquina superior izquierda que generalmente ilustra la finalidad de la aplicación y que también aparece cuando se minimiza el formulario. Por omisión, Visual C# utiliza un icono genérico.

Para utilizar su propio icono (de 16×16 o de 32×32 píxeles), sólo tiene que asignarlo a la propiedad **Icon** del formulario; esto es, seleccione el formulario, vaya a la página de propiedades, elija la propiedad **Icon**, pulse el botón que se muestra a la derecha y asigne el fichero *.ico* que contiene el icono.

Escribir los controladores de eventos

Sabemos que el nombre de un objeto, propiedad **Name**, nos permite referirnos a él dentro del código de la aplicación; por ejemplo, en las líneas de código siguiente, la primera asigna el valor “¡¡¡Hola mundo!!!” a la propiedad **Text** del objeto *etSaludo* y la siguiente obtiene el valor de la caja de texto y lo almacena en la variable *sTexto*:

```
etSaludo.Text = "¡¡¡Hola mundo!!!";  
string sTexto = etSaludo.Text;
```

En C# la forma general de referirse a una propiedad de un determinado objeto, según hemos aprendido al estudiar programación orientada a objetos, es:

Objeto.Propiedad

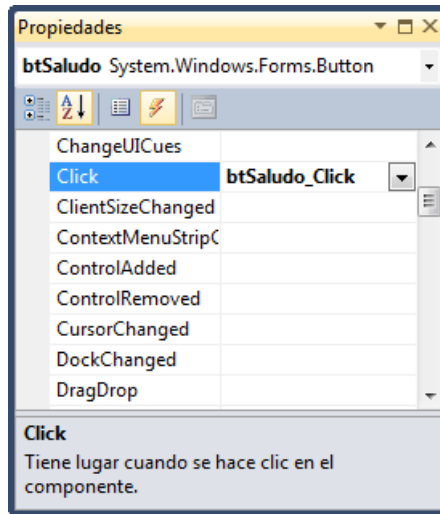
donde *Objeto* es el nombre del formulario o control y *Propiedad* es el nombre de la propiedad del objeto cuyo valor queremos asignar u obtener.

Una vez que hemos creado la interfaz o medio de comunicación entre la aplicación y el usuario, tenemos que escribir los métodos para controlar, de cada uno de los objetos, aquellos eventos que necesitemos manipular.

Hemos dicho que una aplicación en Windows es conducida por *eventos y orientada a objetos*. Esto es, cuando sobre un objeto ocurre un suceso (por ejemplo, el usuario hizo clic sobre un botón) se produce un evento (por ejemplo, el evento **Click**); si nosotros deseamos que nuestra aplicación responda a ese evento, tendremos que escribir un método que incluya el código que debe ejecutarse. El método pertenecerá a la interfaz del objeto o del objeto padre. Por ejemplo, el método que responda al evento **Click** de un botón pertenecerá a la interfaz de su ventana padre, esto es, a su contenedor.

¿Dónde podemos ver la lista de los eventos a los que puede responder un objeto de nuestra aplicación? Pues en la de propiedades.

Por ejemplo, seleccione el botón *btSaludo* en la ventana de diseño, vaya a la ventana de propiedades y muestre la lista de eventos para el control seleccionado, haciendo clic en el botón *Eventos*. Haga doble clic en el evento **Click**, o bien escriba manualmente el nombre del controlador y pulse *Entrar*.



El resultado es que se añade a la clase *Form1* un manejador para este evento (fichero *Form1.Designer.cs*) y el método *btSaludo_Click* que responderá al mismo (fichero *Form1.cs*):

```
this.btSaludo.Click += new System.EventHandler(this.btSaludo_Click);

private void btSaludo_Click(object sender, EventArgs e)
{
    // Escriba aquí el código que tiene que ejecutarse para responder
    // al evento Click que se genera al pulsar el botón
}
```

La interpretación del código anterior es: el método *btSaludo_Click* manipula (EventHandler) el evento **Click** de *btSaludo* (*btSaludo.Click*).

El primer parámetro del método hace referencia al objeto que generó el evento y el segundo contiene información que depende del evento.

Una vez añadido el controlador para el evento **Click** del botón *btSaludo*, ¿cómo lo completamos? Lo que deseábamos era que la etiqueta mostrara el mensaje “¡¡¡Hola mundo!!!” cuando el usuario hiciera clic en el botón. Según esto, complete este controlador así:

```
private void btSaludo_Click(object sender, EventArgs e)
{
    etSaludo.Text = "¡¡¡Hola mundo!!!";
}
```

Para añadir el controlador anterior, también podríamos habernos dirigido a la página de diseño y haber hecho doble clic sobre el botón de pulsación.

Un detalle de estilo a la hora de escribir el código. Observe que *Visual C# Express* para no anteponer a los nombres de las clases y a otros elementos del lenguaje el nombre del espacio de nombres al que pertenece (por ejemplo, **System.Object** en lugar de escribir solamente el nombre de la clase **Object**), añade al principio del código fuente las sentencias **using**, que se muestran a continuación, que los especifican.

```
using System;  
using System.Windows.Forms;
```

Análogamente a como las carpetas o directorios ayudan a organizar los ficheros en un disco duro, los espacios de nombres ayudan a organizar las clases en grupos para facilitar el acceso a las mismas y proporcionan una forma de crear tipos globales únicos, evitando conflictos en el caso de clases de igual nombre pero de distintos fabricantes, ya que se diferenciarán en su espacio de nombres.

Además del evento **Click**, hay otros eventos asociados con un botón de pulsación, según se puede observar en la figura anterior.

Guardar la aplicación

Una vez finalizada la aplicación, se debe guardar en el disco para que pueda tener continuidad; por ejemplo, por si más tarde se quiere modificar. Esta operación puede ser que se realice automáticamente cuando se compila o se ejecuta la aplicación, y si no, puede requerir guardar la aplicación en cualquier instante ejecutando la orden *Guardar todo* del menú *Archivo*.

Si desplegamos el menú *Archivo*, nos encontraremos, además de con la orden *Guardar todo*, con dos órdenes más: *Guardar nombre-fichero* y *Guardar nombre-fichero como...* La orden *Guardar nombre-fichero* guarda en el disco el fichero actualmente seleccionado y la orden *Guardar nombre-fichero como...* realiza la misma operación, y además nos permite cambiar el nombre, lo cual es útil cuando el fichero ya existe.

No es conveniente que utilice los nombres que Visual C# asigna por defecto, porque pueden ser fácilmente sobrescritos al guardar aplicaciones posteriores.

Verificar la aplicación

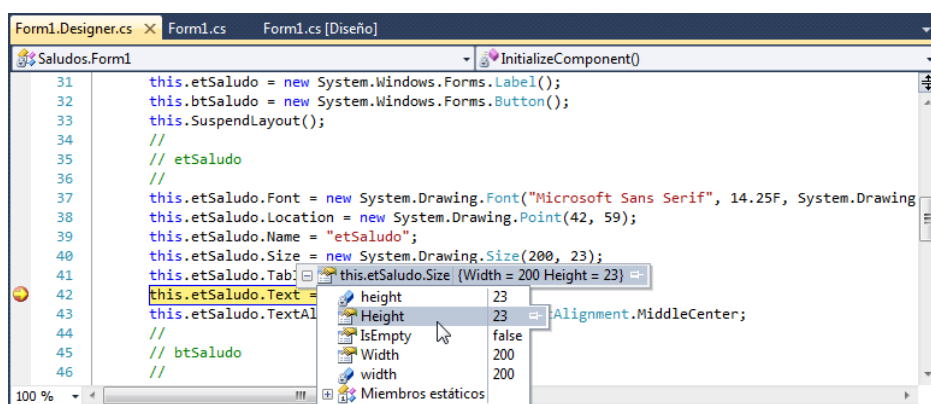
Para ver cómo se ejecuta la aplicación y los resultados que produce, hay que seleccionar la orden *Iniciar sin depurar* del menú *Depurar* o pulsar *Ctrl+F5*.

Si durante la ejecución encuentra problemas o la solución no es satisfactoria y no es capaz de solucionarlos por sus propios medios, puede utilizar, fundamentalmente, las órdenes *Paso a paso por instrucciones* (*F11*), *Paso a paso por procedimientos* (*F10*), *Alternar puntos de interrupción* (*F9*), todas ellas del menú *Depurar*, para hacer un seguimiento paso a paso de la aplicación, y las órdenes del menú *Depurar > Ventanas*, para observar los valores que van tomando las variables y expresiones de la aplicación.

La orden *Paso a paso por instrucciones* permite ejecutar cada método de la aplicación paso a paso. Esta modalidad se activa y se continúa pulsando *F11*. Si no quiere que los métodos invocados a su vez por el método en ejecución se ejecuten línea a línea, sino de una sola vez, utilice la tecla *F10* (*Paso a paso por procedimientos*). Para detener la depuración pulse las teclas *Mayús+F5*.

La orden *Alternar puntos de interrupción* (*F9*) permite colocar una pausa en cualquier línea. Esto permite ejecutar la aplicación hasta la pausa en un solo paso (*F5*), y ver en la ventana *Variables locales* los valores que tienen las variables en ese instante. Para poner o quitar una pausa, se coloca el cursor donde se desea que tenga lugar dicha pausa y se pulsa *F9*, o bien se hace clic con el ratón sobre la barra situada a la izquierda del código.

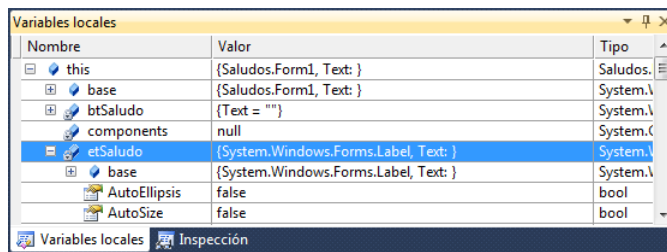
Alternativamente al menú de depuración, puede utilizar la barra de herramientas de depuración. La figura siguiente muestra esta barra dentro de la ventana de código en un proceso de depuración. La línea de código sombreada es la siguiente sentencia a ejecutar.



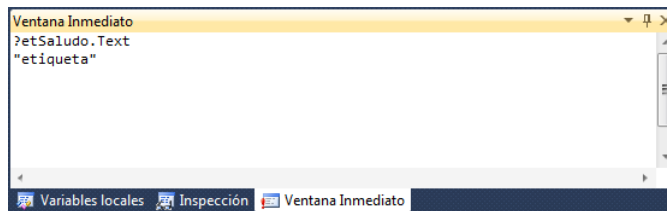
También puede utilizar el ratón para arrastrar el puntero de ejecución (observe la flecha en el margen izquierdo de la ventana anterior) a otro lugar dentro del mismo método con la intención de alterar el flujo normal de ejecución.

Durante el proceso de depuración, puede situar el puntero del ratón sobre el elemento del código cuyo valor quiere inspeccionar; le aparecerá una etiqueta con el valor, como puede observar en la ventana de código anterior.

También, puede ver en la ventana *Variables locales* los valores de las variables y expresiones que desee. Además, en la ventana *Inspección* puede escribir la expresión que desea ver.



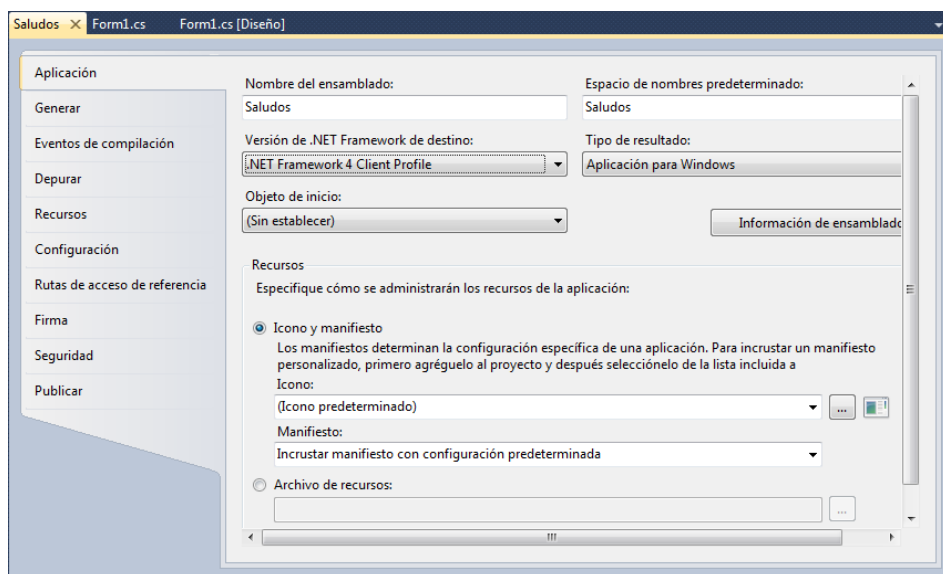
Así mismo, según se observa en la figura siguiente, puede ejecutar en la *ventana de comandos* cualquier sentencia de una forma inmediata. Para mostrar u ocultar esta ventana ejecute la orden *Otras ventanas > Ventana de comandos* del menú *Ver*. El resultado del ejemplo mostrado es el contenido de la propiedad **Text** de la etiqueta *etSaludo* (observe el uso del símbolo ?).



Una vez iniciada la ejecución de la aplicación, si se pulsa la tecla *F5*, la ejecución continúa desde la última sentencia ejecutada en un método hasta finalizar ese método o hasta otro punto de parada.

Propiedades del proyecto

Para establecer las propiedades del proyecto actual hay que ejecutar la orden *Proyecto > Propiedades de nombre-proyecto...* Se le mostrará una ventana con varios paneles. Seleccione el deseado y modifique las propiedades que considere oportunas.

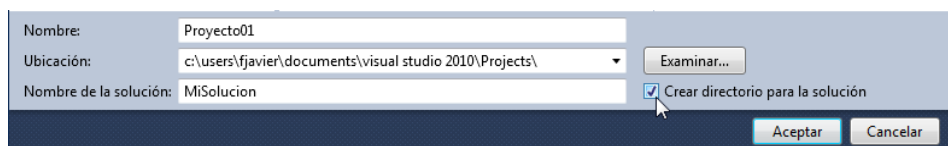


Crear soluciones de varios proyectos

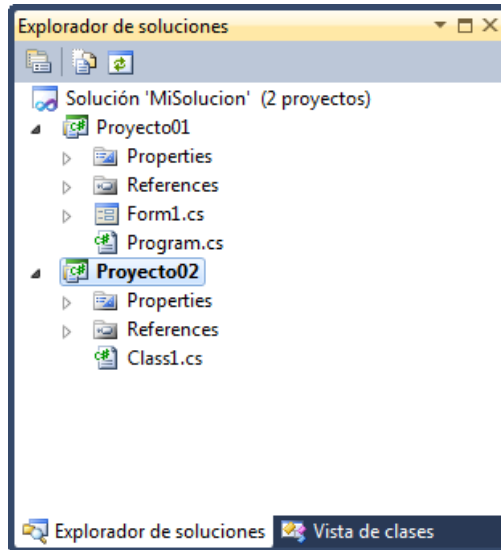
Una solución agrupa uno o más proyectos. Por omisión, cuando se crea un nuevo proyecto, en la misma carpeta física se crea la solución (fichero con extensión *.sln*) a la que pertenece, con el mismo nombre que el proyecto. Esta solución permite que los ficheros que forman parte del proyecto se almacenen bajo una estructura de directorios que facilite su posterior localización así como las tareas de compartir la solución con otros desarrolladores de un supuesto equipo.

¿Qué tenemos que hacer si necesitamos agrupar varios proyectos bajo una misma solución? Crear una solución vacía, o con un proyecto inicial, y añadir nuevos proyectos a la solución o añadir nuevos proyectos a la solución existente. Asegúrese de que se va a mostrar siempre el nombre de la solución en el explorador de soluciones. Para ello, ejecute *Herramientas > Opciones > Proyectos y Soluciones > Mostrar solución siempre*.

Para crear una nueva solución con un proyecto inicial, ejecute la orden *Archivos > Nuevo > Proyecto*. En la ventana de diálogo que se visualiza, escriba el nombre del proyecto, el nombre de la solución, seleccione la casilla *Crear directorio para la solución* y elija su ubicación:



El resultado será una solución denominada *MiSolucion* con un proyecto denominado *Proyecto01*. Para añadir un nuevo proyecto a esta solución, haga clic con el botón secundario del ratón sobre el nombre de la solución y ejecute *Agregar > Nuevo proyecto*.



La ejecución de una solución se inicia a partir del proyecto activo. Esto quiere decir que de entre todos los proyectos que componen una solución, sólo uno puede estar activo. Para establecer cuál será el proyecto activo de entre todos los que componen la solución, haga clic con el botón secundario del ratón sobre el nombre de ese proyecto y ejecute *Establecer como proyecto activo*.

La generación de varios proyectos bajo una misma solución es útil cuando uno o más proyectos proporcionan utilidades (por ejemplo, una biblioteca de clases) a otro u otros proyectos, porque esta forma de trabajar facilita el desarrollo y puesta a punto de todos ellos a la vez.

ADO.NET

Muchas de las aplicaciones, distribuidas o no, trabajan sobre bases de datos. Por esta razón, Microsoft decidió crear una tecnología de acceso a datos potente y fácil de utilizar: ADO.NET.

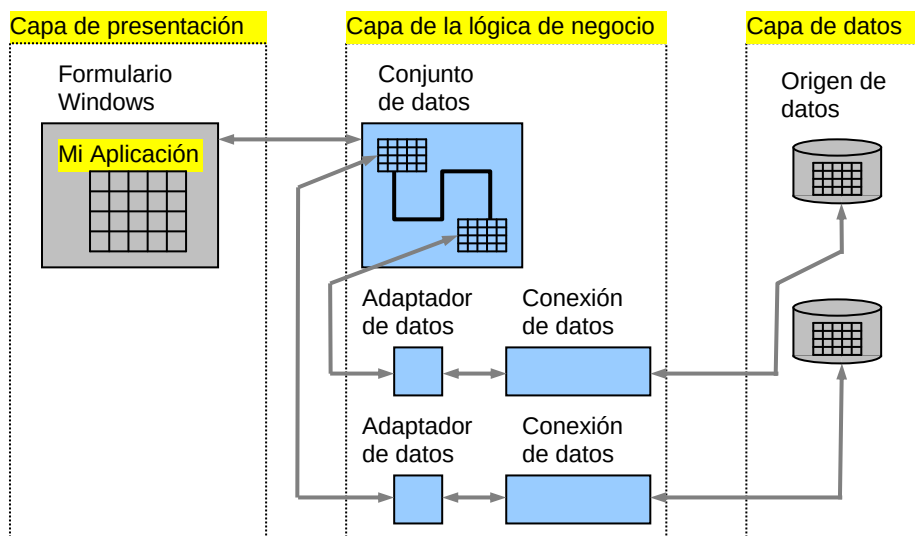
- ADO.NET *no depende de conexiones* continuamente activas, esto es, se diseñó en torno a una arquitectura donde las aplicaciones se conectan a la base de datos sólo durante el tiempo necesario para extraer o actualizar los datos. De esta forma, la base de datos no contiene conexiones que la mayor parte del

tiempo permanecen inactivas, lo que se traduce en dar servicio a muchos más usuarios y facilita la escalabilidad.

- Las interacciones con la base de datos se realizan mediante *órdenes para acceso a los datos*, que son objetos que encapsulan las sentencias SQL o los procedimientos almacenados que definen la operación a realizar sobre el origen de datos.
- Los datos requeridos normalmente se almacenan en memoria caché en *conjuntos de datos*, lo que permite trabajar sin conexión sobre una copia temporal de los datos obtenidos. Los conjuntos de datos son independientes de los orígenes de datos. Cuando sea necesario, se puede restablecer la conexión con la base de datos y actualizarla desde el conjunto de datos.
- En ADO.NET, el formato de transferencia de datos es XML. La representación XML de los datos no utiliza información binaria (muchos servidores de seguridad bloquean la información binaria), sino que se basa en texto, lo que permite enviarla mediante cualquier protocolo, como por ejemplo HTTP.

Componentes de ADO.NET

ADO.NET es un conjunto de clases, pertenecientes al espacio de nombres **System.Data**, para acceso a los datos de un origen de datos. Dicho de otra forma, ADO.NET proporciona un conjunto de componentes para crear aplicaciones distribuidas de uso compartido de datos.

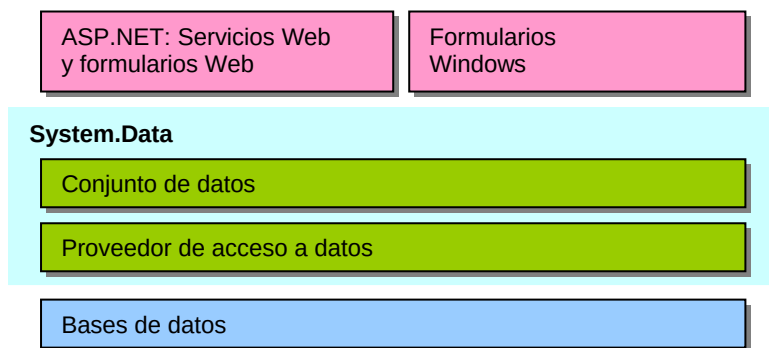


Dichos componentes están diseñados para separar el acceso a los datos de la manipulación de los mismos y son: **DataSet** y el *proveedor de datos* de .NET Framework, que es un conjunto de componentes entre los que se incluyen los objetos conexión (**Connection**), de órdenes (**Command**), lector de datos (**DataReader**) y adaptador de datos (**DataAdapter**).

Para una mejor comprensión, la figura anterior muestra cómo trabajan conjuntamente los objetos mencionados entre sí, para que una aplicación pueda interactuar con un origen de datos.

Para resumir, se puede decir que el trabajo de conexión con la base de datos, o la ejecución de una sentencia SQL determinada, la realiza el proveedor de acceso a datos.

En cambio, recuperar esos datos (para tratarlos, manipularlos o volcarlos a un determinado control o dispositivo) es una acción ejecutada por una capa superior. El resultado es un conjunto de datos agrupados en tablas.



ACCEDER A LA WEB

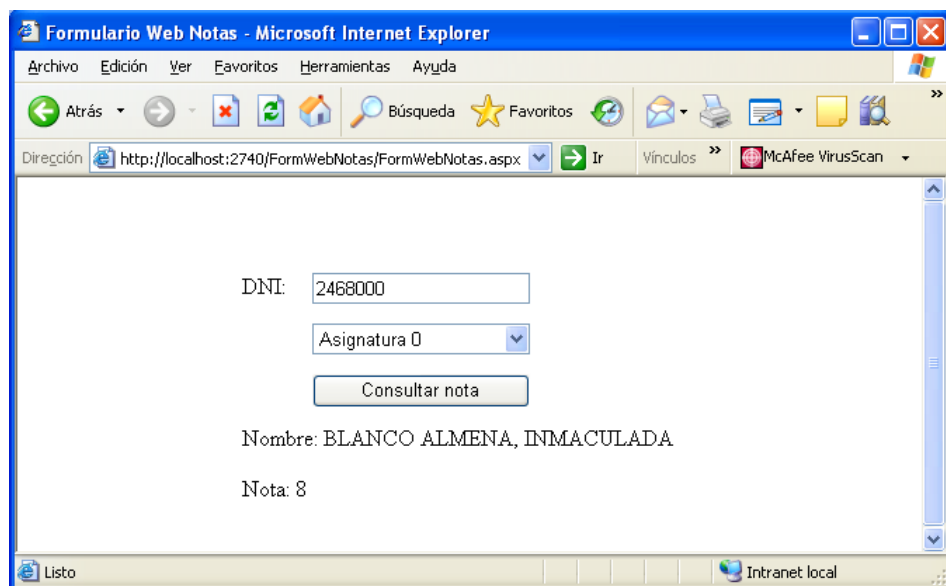
ASP.NET es un entorno de programación que se ejecuta en un servidor Web para producir y administrar de forma dinámica formularios Web. *Visual Web Developer Express* proporciona un diseñador de formularios Web, un editor, controles y una herramienta de depuración, para facilitar la generación de aplicaciones, a las que, una vez colocadas en un servidor, se podrá acceder desde exploradores y otros dispositivos cliente como teléfonos móviles o asistentes digitales personales (PDA). Si no dispone de *Visual Studio*, puede descargar *Visual Web Developer Express* de Internet, desde la misma página de la que descargó *Visual C# Express*. Cuando lo tenga instalado, habilite la configuración para expertos: *Herramientas > Configuración*.

Como ejemplo, vamos a crear un formulario Web para que un alumno pueda consultar a través de Internet las notas de las asignaturas que ha cursado. Para desarrollar esta aplicación Web, podemos seguir los pasos indicados a continuación:

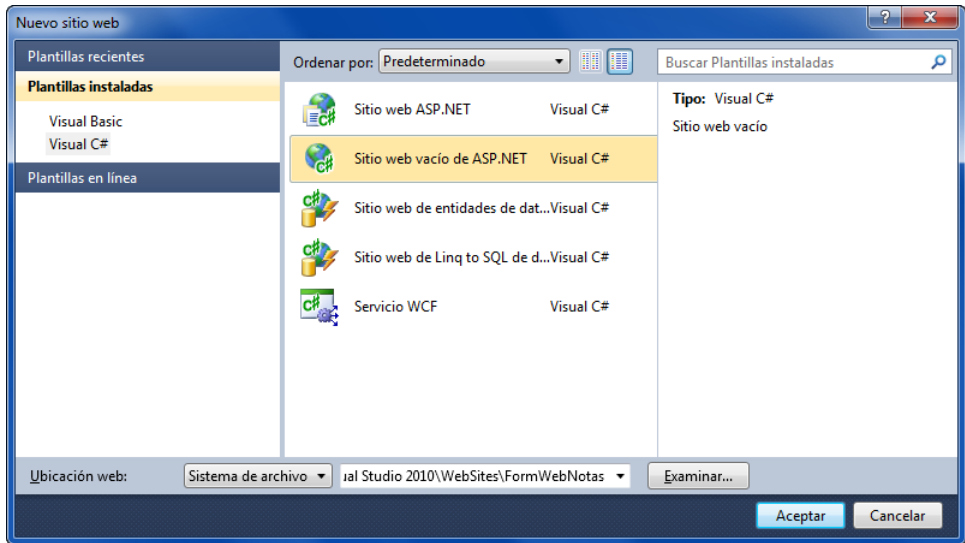
- Crear un nuevo formulario Web.
- Agregar controles y texto a la página.
- Crear controladores de eventos para los controles.
- Generar y ejecutar el formulario Web.

Crear un nuevo formulario Web

Un formulario Web está formado por dos componentes: los elementos visuales y el código. Estos componentes pueden estar almacenados en el mismo fichero, o bien en ficheros diferentes, que es lo más habitual. En este último caso, los elementos visuales (HTML, controles de servidor y texto estático) se crean desde un fichero *.aspx* y el código que se ejecutará se almacena en otro fichero; en nuestro caso se trata de código C#.

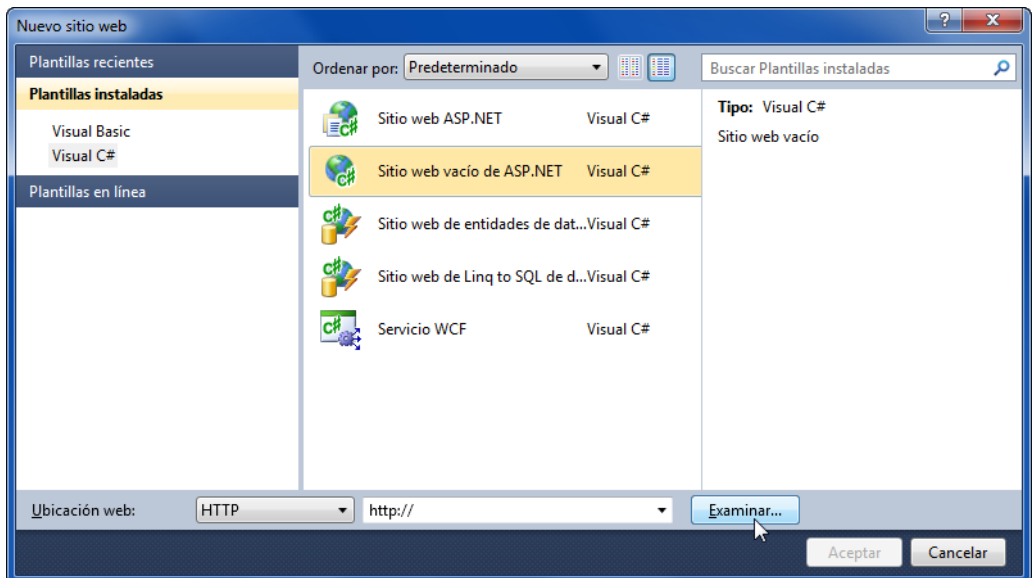


Si tiene instalado *Visual Web Developer Express*, el primer paso es crear el proyecto y el formulario. Para ello, en el menú *Archivo* elija *Nuevo Sitio Web*. En el cuadro de diálogo “Nuevo sitio web” que se visualiza, seleccione como lenguaje Visual C# y como plantilla *Sitio web vacío de ASP.NET* según puede observar en la figura siguiente:

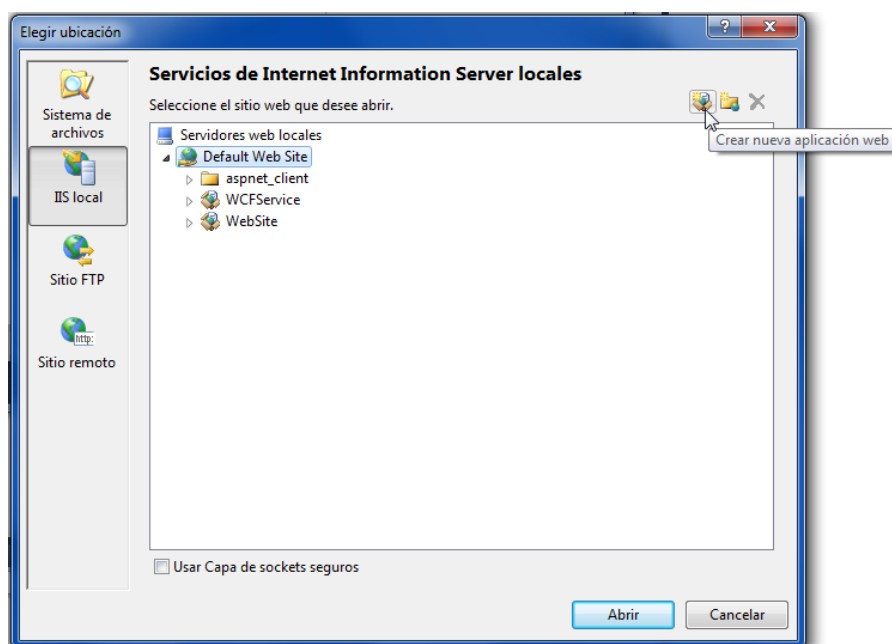


En la lista *Ubicación web* seleccione “Sistema de archivos” y haga clic en el botón *Examinar* para crear o seleccionar la carpeta donde desea ubicar la aplicación; en nuestro caso en *FormWebNotas*. Finalmente, haga clic en *Aceptar*.

Si en su equipo tiene instalado IIS (se trata del servidor de aplicaciones *Internet Information Server*), puede ubicar la aplicación en este servidor según puede observar en las figuras siguientes:

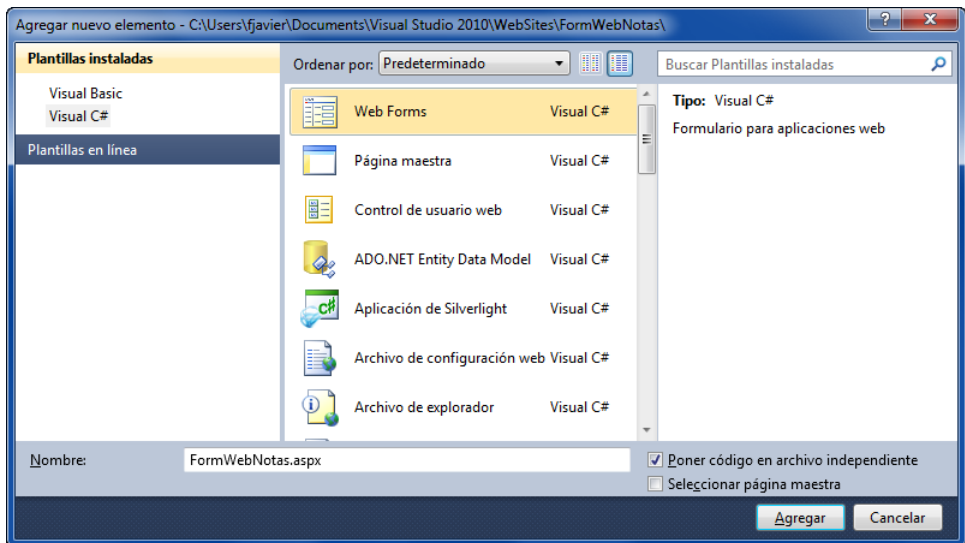


En la lista *Ubicación web* seleccione “HTTP” y haga clic en *Examinar*. Se visualiza la ventana “Elegir ubicación” que se muestra a continuación:

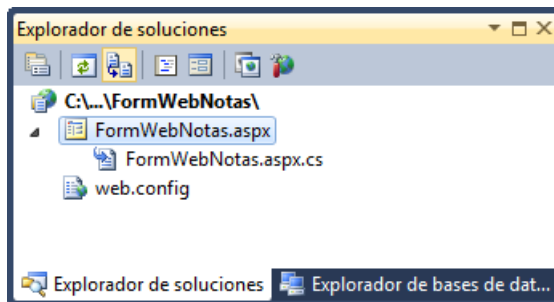


Haga clic en el botón “IIS local”, después haga clic en el botón “Crear nueva aplicación Web”, ponga nombre al directorio virtual (en nuestro caso *FormWeb-Notas*) y haga clic en el botón *Abrir* para volver al diálogo “Nuevo sitio Web”. Observaríamos que el nombre del servidor sería *http://localhost*.

Continuando con nuestra aplicación, ubicada en nuestro sistema de ficheros, vamos a añadir a la misma un formulario Web, para construir la página Web presentada al principio de este apartado. Para ello, vaya al explorador de soluciones y, utilizando el botón derecho del ratón, haga clic sobre el nombre de la aplicación y ejecute *Agregar nuevo elemento*. En el diálogo que se visualiza, seleccione el elemento *Web Forms*, asigne a la página Web el nombre *FormWebNotas.aspx*, deje marcada la casilla *Poner código en archivo independiente* y haga clic en el botón *Aceptar*.



Deténgase un momento y observe las carpetas y ficheros que se han creado. Puede comprobar en el *Explorador de soluciones* que en la carpeta *FormWebNotas* se han creado los ficheros *FormWebNotas.aspx* y *FormWebNotas.aspx.cs*, además del fichero de configuración *web.config* que se generó cuando se creó la aplicación. El fichero fuente *FormWebNotas.aspx.cs* contiene el código C# que da soporte a la página XHTML *FormWebNotas.aspx*.



Observe también la primera línea de la página Web *FormWebNotas.aspx*; vincula la página Web con el fichero de código C# *FormWebNotas.aspx.cs*.

```
<%@ Page Language="C#" AutoEventWireup="true"
    CodeFile="FormWebNotas.aspx.cs" Inherits="FormWebNotas" %>
```

Diríjase ahora al diseñador de formularios. Está mostrando el código HTML de la página Web. En su parte inferior muestra tres botones, *Diseño*, *Dividir y Código*, que muestran diferentes vistas del fichero *FormWebNotas.aspx*; la primera corresponde al diseñador gráfico que muestra los elementos tal cual serán vi-

sualizados, la segunda muestra simultáneamente el diseñador gráfico y la ventana de código, y la tercera muestra la ventana de código XHTML que da lugar a los elementos de la página. Se puede trabajar en cualquiera de las dos vistas. Cada una de ellas se actualiza con los cambios efectuados en la otra.



Agregar controles y texto a la página

El siguiente paso es añadir a la vista *Diseño* los elementos que mostrará el formulario, y establecer sus propiedades en la ventana *Propiedades*.

Para agregar un control al panel de *Diseño*, diríjase al *Cuadro de herramientas*, selecciónelo del panel *Estándar* (controles de servidor Web), y arrástrelo sobre la vista *Diseño*.

Para colocar los controles en la página Web disponemos de dos modos: modo *cuadrícula* (*grid layout*) y modo *flujo* (*flow layout*). De manera predeterminada un control tiene asignado el modo *flujo*. En este modo los elementos fluyen de izquierda a derecha en la línea, y de arriba abajo en la página. En cambio, en el modo *cuadrícula* puede arrastrar controles a la página y colocarlos utilizando sus coordenadas absolutas *x* e *y*. Para cambiar a modo cuadrícula (posición absoluta o relativa) seleccione el control en el panel de *Diseño* y elija posición *Absoluta* entre las opciones que se visualizan al ejecutar la orden *Establecer posición* del menú *Formato* del entorno de desarrollo.

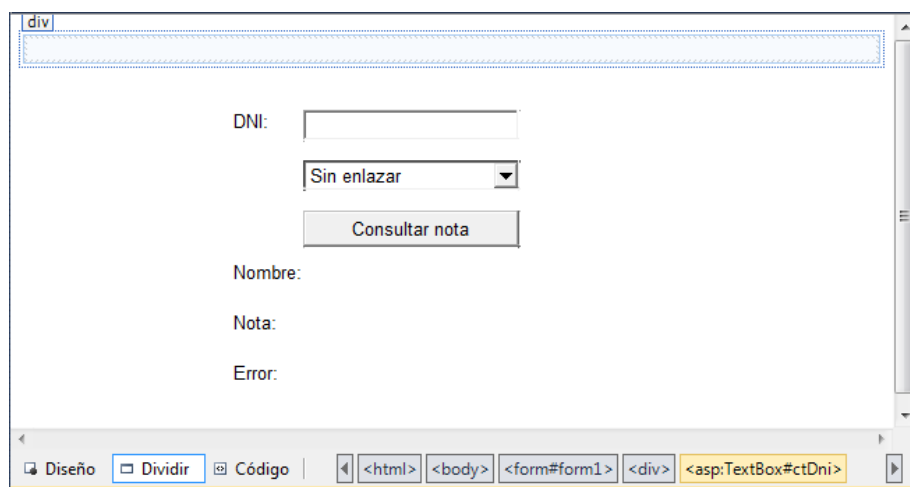
Los modos *cuadrícula* y *flujo* tienen ventajas e inconvenientes. Cualquier explorador Web puede mostrar documentos HTML que utilicen el modo flujo; además, cuando se ajuste el tamaño de la página, los controles se volverán a colocar. El modo cuadrícula proporciona un mayor control sobre el diseño de la página, pero no es tan flexible; esto es, si el tamaño de la página se hace más pequeño o la resolución del equipo es menor que la del equipo de desarrollo, puede que sólo se muestre una parte de la página. No obstante, el autor utilizará este modo.

Una vez colocado un control sobre el formulario, para establecer sus propiedades, selecciónelo y, a continuación, escriba los valores adecuados en la ventana de *Propiedades*. También puede mejorar la vista del conjunto de controles del formulario utilizando las órdenes del menú *Formato*, o la barra de herramientas *Diseño*.

Según lo explicado, y observando la figura siguiente correspondiente al formulario Web que deseamos diseñar, añada al panel de *Diseño* los controles indicados en la tabla siguiente con las propiedades especificadas:

Objeto	Propiedad	Valor
Etiqueta	Text	DNI:
Caja de texto	ID	ctDni
Lista desplegable	ID	lsdAsignatura
Botón de pulsación	ID Text	btConsultarNota Consultar nota
Etiqueta	ID Text	etNombre Nombre:
Etiqueta	ID Text	etNota Nota:
Etiqueta	ID Text	etError Error:

Finalmente, haga clic sobre el formulario y cambie su propiedad **Title** al valor *Formulario Web Notas*.

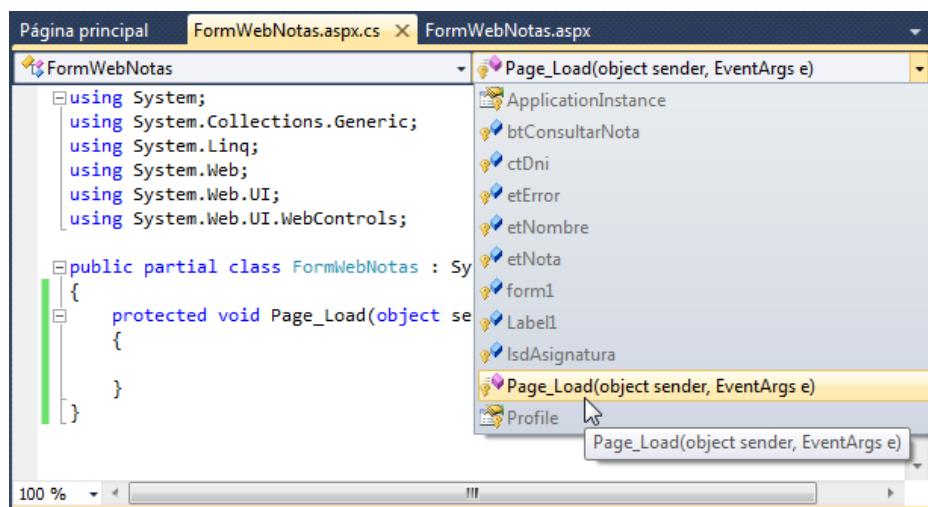


Ciclo de vida de una página

Cuando un usuario solicita una página *.aspx* (un formulario Web), el servidor carga esa página y una vez completada la solicitud la descarga. El explorador presenta la página al usuario y éste interactúa con ella, causando que se envíe de nuevo al servidor por cada acción que necesite procesamiento, como hacer un clic en un botón; procesada la acción, la página será devuelta al explorador. Entre las *acciones de ida y vuelta* ASP.NET guarda la página y las propiedades de los controles, esto es, el estado de vista (propiedad **ViewState**) de los controles.

El proceso de una página Web (un formulario Web) ocurre en varias fases, que podemos resumir en: iniciación, carga del estado de vista, validación y control de eventos. Durante estas fases se producen eventos, como **Init**, **Load** o **Unload**. El evento **Init** es el primero en el ciclo de vida de una página y se produce cuando la página es iniciada; para responder a este evento hay que sobrescribir el método **OnInit** heredado de la clase base (en nuestro caso la clase *FormWebNotas* se deriva de **Page**). Una vez iniciada la página, se produce el evento **Load** y cuando la página se descarga se produce el evento **Unload**. Para responder a estos eventos podemos proceder de la forma tradicional, o bien sobrescribiendo los métodos **OnLoad** y **OnUnload**, respectivamente, heredados de la clase base.

Según lo explicado, diríjase al explorador de soluciones y haga clic en el botón *Ver código* del explorador de soluciones. Esto hará que se visualice la página de código que muestra la figura siguiente. Observe que esta página expone en su parte superior dos listas desplegables: la de la izquierda contiene la lista de clases de la página Web y la de la derecha, la lista de controles, propiedades y métodos del objeto seleccionado en la lista de la izquierda:



Por ejemplo, en la figura anterior se ve que se ha seleccionado el controlador *Page_Load* para controlar el evento **Load** del formulario. El resultado es que se añade a la clase *FormWebNotas* derivada de **Page** el método *Page_Load* para responder al evento **Load** de la página Web:

```
protected void Page_Load(object sender, EventArgs e)
{
    // Introduzca aquí el código para iniciar la página
}
```

Crear controladores de eventos para los controles

Los controles de servidor de los formularios Web, análogamente a como vimos cuando trabajamos con formularios Windows, pueden producir eventos, muchos de los cuales se generan por las acciones que el usuario realiza sobre el formulario Web mostrado por el explorador. Por ejemplo, un botón de pulsación (control de servidor Web de la clase **Button**) producirá un evento **Click** en cuanto un usuario haga clic sobre él, y el código para responder al evento producido se ejecutará en el servidor.

Pensemos entonces en cómo se sucederán los hechos cuando un alumno solicite que se ejecute nuestra aplicación Web desde un explorador:

1. Se visualiza el formulario Web. La lista desplegable muestra la lista de asignaturas que el alumno puede consultar, la cual se carga desde un fichero *asignatura.txt* que almacenaremos en la carpeta *App_Data*. Guarde también en esta carpeta los ficheros *NotasAsig?.txt* (? = 0, 1, 2, etc.).

Para añadir la carpeta *App_Data* al proyecto, diríjase al explorador de soluciones, haga clic con el botón secundario del ratón y ejecute *Agregar carpeta ASP.NET > App_Data*. Después copie los ficheros indicados en esta carpeta.

2. El alumno escribe su DNI y selecciona de la lista la asignatura de la que quiere consultar la nota.
3. Después, hace clic en el botón *Consultar nota*. La página se envía al servidor. ASP.NET analiza el evento que se ha producido y si existe un controlador para ese evento, se ejecuta inmediatamente. Cuando la ejecución del código finalice, la página se vuelve a enviar al explorador con los cambios realizados por el código: la etiqueta *etNombre* mostrará el nombre del alumno y la etiqueta *etNota* la nota obtenida en la asignatura seleccionada, o bien la etiqueta *etError* visualizará un mensaje de error si fue esto lo que ocurrió.

Una vez descrito cómo tienen que suceder las acciones, podemos pasar a escribir el código que permita procesar tales acciones. Esto requiere conocer cómo es el ciclo de vida de una página Web, cosa que ya hemos explicado en el apartado anterior. Según se expuso allí, cuando se carga la página Web se produce el evento **Load**; la respuesta a este evento será la ejecución del método *Page_Load*. Por lo tanto, éste es el lugar idóneo para iniciar la lista *lsdAsignatura* con las asignaturas, y la etiqueta de error *etError*:

```
protected void Page_Load(object sender, EventArgs e)
{
    etError.Text = "";
    if (lsdAsignatura.Items.Count == 0)
        CargarListaDesplegable();
}
```

A continuación, añadimos a la clase *FormWebNotas* el método privado *CargarListaDesplegable*. Este método lee los nombres de las asignaturas del fichero *asignaturas.txt* y los añade a la lista desplegable. Si durante este proceso ocurre algún error, se muestra a través de la etiqueta *etError*.

```
private void CargarListaDesplegable()
{
    System.IO.StreamReader sr = null;
    string str;
    try
    {
        sr = new System.IO.StreamReader("C:/ ...su ruta.../" +
                                         "FormWebNotas/App_Data/asignaturas.txt");
        str = sr.ReadLine();
        while (str != null)
        {
            lsdAsignatura.Items.Add(str);
            str = sr.ReadLine();
        }
    }
    catch (System.IO.IOException exc)
    {
        etError.Text = "Error: " + exc.Message;
    }
    finally
    {
        if (sr != null) sr.Close();
    }
}
```

Para escribir un controlador para el evento **Click** del botón *btConsultarNota*, diríjase al panel *Diseño* y haga doble clic sobre el botón. Puede hacerlo también a

través del panel de eventos en la ventana de propiedades del botón. Después, complete dicho controlador como se indica a continuación.

```
protected void btConsultarNota_Click(object sender, EventArgs e)
{
    BuscarNota(lsdAsignatura.SelectedIndex);
}
```

El método *BuscarNota* recibe como argumento el índice, 0, 1, 2, etc., de la asignatura seleccionada y abre el fichero *NotasAsig?.txt* (? = 0, 1, 2, etc.) que almacena el acta de notas de esa asignatura. Cada línea de este fichero de texto tiene tres datos, DNI, nombre y nota, separados por un tabulador horizontal (carácter de código 9). Una vez abierto el fichero, lee una línea, la divide en sus campos y verifica si contiene el dato *ctDni.Text*; si no lo contiene, repite el proceso con la siguiente línea y así sucesivamente; si lo contiene, muestra en la etiqueta *etNombre* el nombre y en la etiqueta *etNota*, la nota. Si durante este proceso ocurre algún error, se muestra a través de la etiqueta *etError*.

```
private void BuscarNota(int nAsig)
{
    System.IO.StreamReader sr = null;
    string str;
    char tab = '\t';
    string[] sDatos = null;
    bool bEncontrado = false;
    try
    {
        sr = new System.IO.StreamReader("C:/...su ruta.../" +
            "FormWebNotas/App_Data/NotasAsig" + nAsig + ".txt");
        str = sr.ReadLine();
        while (str != null && !bEncontrado)
        {
            sDatos = str.Split(tab);
            if (ctDni.Text == sDatos[0])
                bEncontrado = true;
            if (!bEncontrado)
                str = sr.ReadLine();
        }
        if (bEncontrado)
        {
            etNombre.Text = "Nombre: " + sDatos[1];
            etNota.Text = "Nota: " + sDatos[2];
        }
        else
        {
            etNombre.Text = "";
            etNota.Text = "";
            etError.Text = "No figura en el acta";
        }
    }
    catch { }
}
```

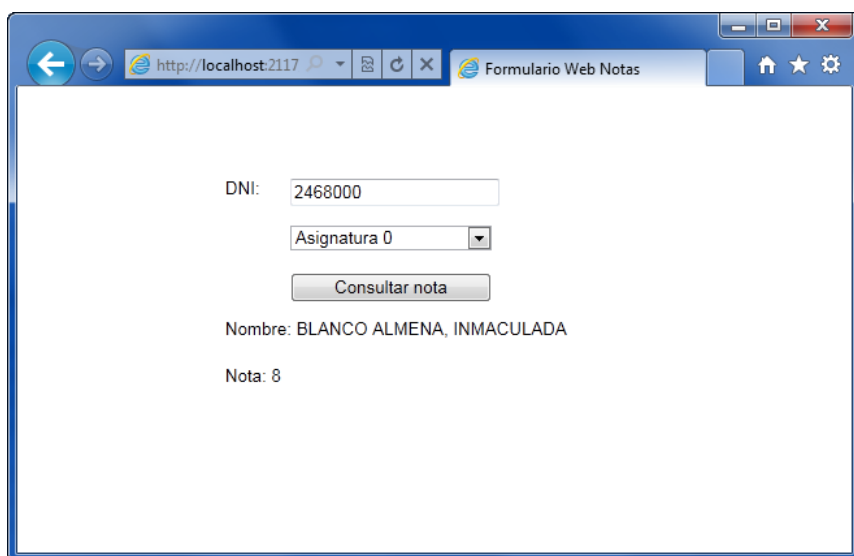
```
    }  
  }  
  catch (System.IO.IOException exc)  
  {  
    etError.Text = "Error: " + exc.Message;  
  }  
  finally  
  {  
    if (sr != null) sr.Close();  
  }  
}
```

Generar la aplicación Web y ejecutarla

Antes de ejecutar la aplicación Web ASP.NET, debe compilarse el fichero de clase de código subyacente (.aspx.cs) de la página Web junto con los demás ficheros de clase incluidos en el proyecto. A continuación, se podrá ver la página en cualquier explorador.

Para compilar el proyecto, seleccione la orden *Generar sitio Web* del menú *Generar* y para ejecutarlo, elija la orden *Iniciar sin depurar* del menú *Depurar*. También puede ejecutar las dos operaciones anteriores pulsando las teclas *Ctrl+F5*.

Cuando el servidor recibe una solicitud para la página, se crea un objeto de la clase que la define y se ejecuta. En nuestro caso, el resultado será similar al siguiente:



The screenshot shows a web browser window with the address bar displaying 'http://localhost:2117'. The page title is 'Formulario Web Notas'. The form contains the following elements:

- A label 'DNI:' followed by a text input field containing '2468000'.
- A label 'Asignatura 0' followed by a dropdown menu.
- A button labeled 'Consultar nota'.
- A label 'Nombre: BLANCO ALMENA, INMACULADA'.
- A label 'Nota: 8'.

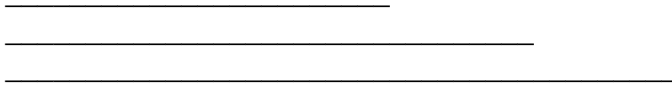
Observe el URL `http://localhost:2117/FormWebNotas/FormWebNotas.aspx`; *localhost* es el nombre del servidor Web donde están almacenados los ficheros que componen la aplicación Web a la que se accede a través de la página *FormWebNotas.aspx*. En este caso, el servidor viene incluido con Visual Web Developer Express para facilitar la puesta a punto de las aplicaciones.

Para que esta aplicación pueda ser igualmente accedida desde cualquier punto de Internet, hay que instalarla en un servidor que proporcione los servicios de IIS. Lógicamente, en el URL utilizado ahora, habrá que sustituir *localhost:2117* por el nombre del nuevo servidor. Por ejemplo:

`http://atc.aut.uah.es/FormWebNotas/FormWebNotas.aspx`

P A R T E

4



Apéndices

- Entorno de desarrollo
- .NET para Linux
- CD

ENTORNO DE DESARROLLO

Cuando se utiliza un entorno de desarrollo integrado (EDI), lo primero que hay que hacer una vez instalado es asegurarse de que las rutas donde se localizan las herramientas, las bibliotecas, la documentación y los ficheros fuente hayan sido establecidas; algunos EDI sólo requieren la ruta donde se instaló el compilador. Este proceso normalmente se ejecuta automáticamente durante el proceso de instalación de dicho entorno. Si no es así, el entorno proporcionará algún menú con las órdenes apropiadas para realizar dicho proceso. Por ejemplo, en los EDI que se presentan a continuación las rutas a las que nos referimos quedan establecidas durante la instalación de los mismos.

VISUAL STUDIO

Visual Studio proporciona una variedad de herramientas tanto para desarrolladores individuales como para equipos de desarrollo.

<http://www.microsoft.com/express/>

Ediciones Visual Studio Express

Es la nueva línea de productos que expanden Visual Studio ofreciendo herramientas ligeras y sencillas de aprender y de usar para aficionados, entusiastas y aprendices que quieren crear sitios Web y aplicaciones para Windows. Su descarga es gratuita.

- Visual Web Developer Express Edition.
- Visual Basic Express Edition.
- Visual C# Express Edition.

- Visual C++ Express Edition.
- SQL Server Express Edition.
- y Visual Studio Express for Windows Phone.

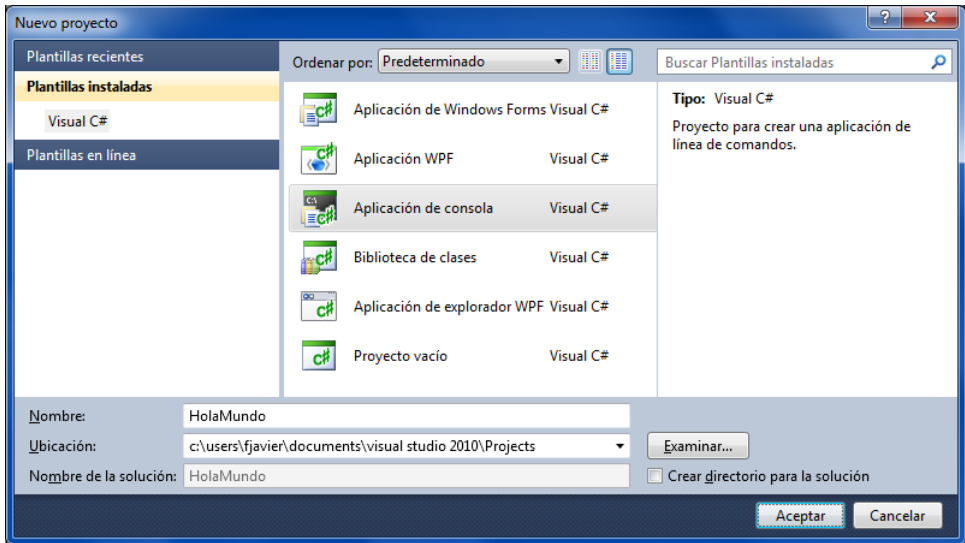
Ediciones de Visual Studio para profesionales

Visual Studio en sus versiones profesional, premium y ultimate, ofrece una flexible herramienta de desarrollo para desarrolladores de aplicaciones de línea de negocio o programadores ocasionales que estén construyendo aplicaciones móviles, basadas en Windows o para la Web, soluciones sobre Microsoft Office System utilizando Excel, Word e InfoPath, y con herramientas de software de ciclo de vida productivas, integradas y extensibles que permiten a las empresas reducir la complejidad en la creación de soluciones orientadas a servicios. La utilización de estas versiones requiere comprar una licencia.

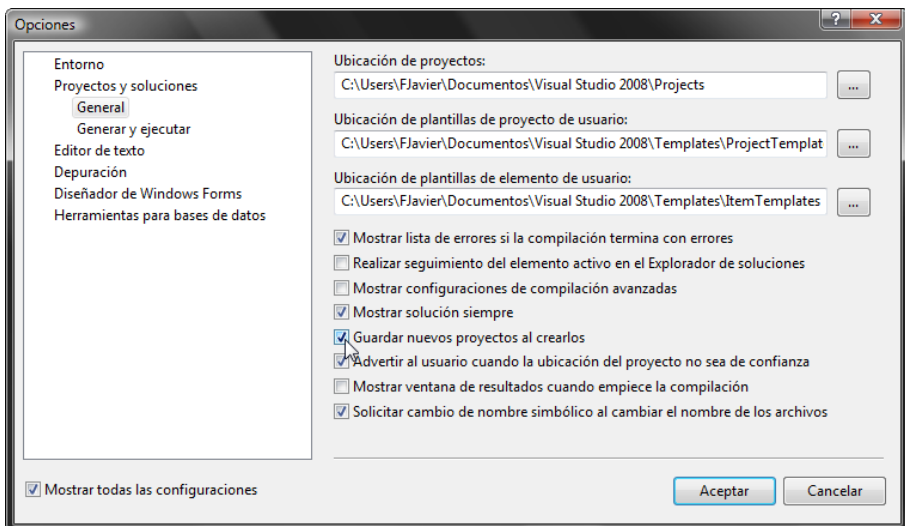
APLICACIÓN DE CONSOLA

Para editar y ejecutar el programa *HolaMundo* que desarrollamos en el capítulo 1 utilizando cualquiera de las ediciones de Visual Studio, los pasos a seguir son los siguientes:

1. Partiendo de la página de inicio de *MS Visual C# Express*, hacemos clic en *Nuevo proyecto...* para crear un proyecto nuevo, o bien ejecutamos la orden *Archivo > Nuevo proyecto*. Esta acción hará que se visualice una ventana que mostrará los tipos de plantillas que puede utilizar; la elección de una u otra dependerá del tipo de aplicación que deseemos construir. La figura siguiente muestra esta ventana:

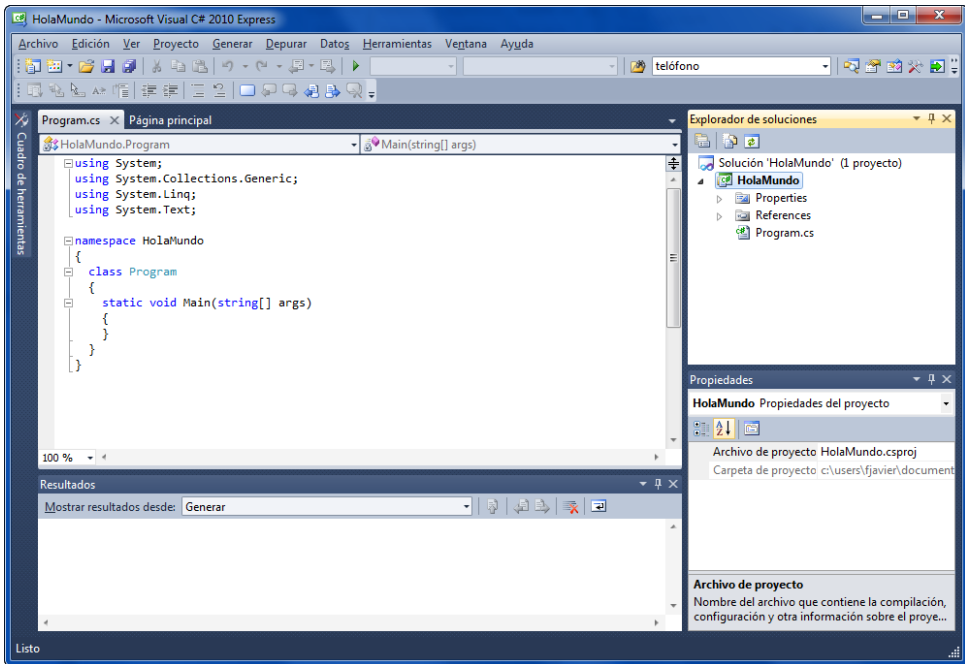


Para que la ventana anterior muestre la lista *Ubicación* y la caja *Nombre de la solución* tiene que habilitar, si no lo está, la opción “Guardar nuevos proyectos al crearlos”. Para ello, ejecute la orden *Herramientas > Opciones > Proyectos y soluciones* y seleccione esta opción en la ventana que se visualiza:



Para nuestro ejemplo, elegimos la plantilla “Aplicación de consola”. Después, especificamos el nombre del proyecto y su ubicación; observe que el proyecto será creado en una carpeta con el mismo nombre. A continuación pulsamos el botón *Aceptar* y obtendremos el esqueleto del proyecto de acuerdo con la plantilla seleccionada. Para cambiar el nombre asignado al fichero *.cs*, hace-

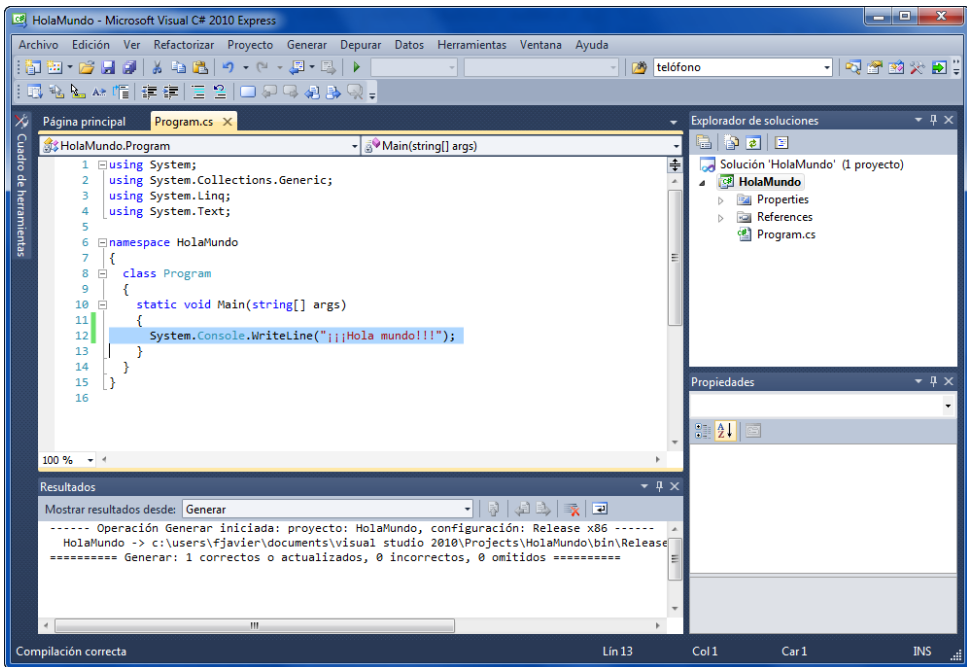
mos clic sobre *Program.cs* y utilizando el botón derecho del ratón, seleccionamos del menú contextual que se visualiza la orden *Cambiar nombre*; también podemos hacer este cambio a través de la ventana de propiedades.



Si necesitáramos añadir un fichero nuevo a este proyecto, haríamos clic con el botón derecho del ratón sobre el nombre del proyecto y seleccionaríamos *Agregar > Nuevo elemento*. Después, elegiríamos el tipo de elemento que deseamos añadir (por ejemplo *Clase*) y, finalmente, haríamos clic en el botón *Agregar*. Lo normal es que una aplicación esté compuesta por varias clases, por lo que la operación de añadir nuevos elementos será muy habitual.

Para que el EDI muestre el menú *Generar*, además de otras opciones en los menús ya visualizados, puede establecer la configuración para expertos: *Herramientas > Configuración > Configuración para expertos*.

2. A continuación editamos el código que compone el programa. Después, antes de compilar la aplicación, podemos verificar que se trata de una aplicación de consola: *Proyecto > Propiedades de HolaMundo > Aplicación > Tipo de resultado > Aplicación de consola*.

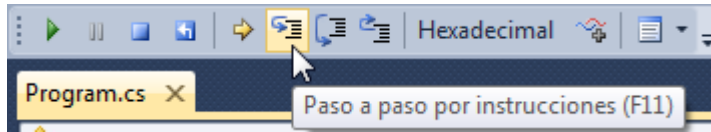


3. Para compilar el programa, ejecutamos la orden *Generar HolaMundo* del menú *Generar*. Finalmente, para ejecutar el programa seleccionamos la orden *Iniciar sin depurar* del menú *Depurar*, o bien pulsamos las teclas *Ctrl+F5*. También puede ejecutar el programa seleccionando la orden *Iniciar depuración* del menú *Depurar*, o bien pulsando la tecla *F5*. Los resultados de la compilación se muestran en la ventana *Resultados*. La acción de ejecutar el programa acarrea también su compilación si fuera necesario.

DEPURAR UNA APLICACIÓN

¿Por qué se depura una aplicación? Porque los resultados que estamos obteniendo con la misma no son correctos y no sabemos por qué. El proceso de depuración consiste en ejecutar la aplicación paso a paso, indistintamente por sentencias o por métodos, con el fin de observar el flujo seguido durante su ejecución, así como los resultados intermedios que se van sucediendo, con la finalidad de detectar las anomalías que producen un resultado final erróneo.

Por ejemplo, para depurar una aplicación utilizando el depurador del entorno de desarrollo de *Visual C# Express*, ejecute la orden *Depurar > Paso por instrucciones* y utilice las órdenes del menú *Depurar* o los botones correspondientes de la barra de herramientas (para saber el significado de cada botón, ponga el puntero del ratón sobre cada uno de ellos).



De forma resumida, las órdenes disponibles para depurar una aplicación son las siguientes:

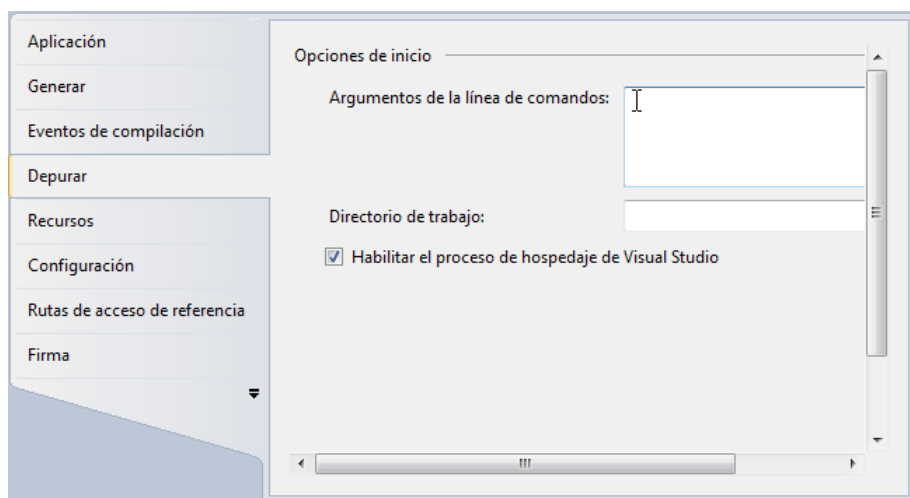
- *Iniciar depuración* o *F5*. Inicia la ejecución de la aplicación en modo depuración hasta encontrar un punto de parada o hasta el final si no hay puntos de parada.
- *Alternar puntos de interrupción* o *F9*. Pone o quita un punto de parada en la línea sobre la que está el punto de inserción.
- *Detener depuración* o *Mayús+F5*. Detiene el proceso de depuración.
- *Paso a paso por instrucciones* o *F11*. Ejecuta la aplicación paso a paso. Si la línea a ejecutar coincide con una llamada a un método definido por el usuario, dicho método también se ejecutará paso a paso.
- *Paso a paso por procedimientos* o *F10*. Ejecuta la aplicación paso a paso. Si la línea a ejecutar coincide con una llamada a un método definido por el usuario, dicho método no se ejecutará paso a paso, sino de una sola vez.
- *Paso a paso para salir* o *Mayús+F11*. Cuando un método definido por el usuario ha sido invocado para ejecutarse paso a paso, utilizando esta orden se puede finalizar su ejecución en un solo paso.
- *Ejecutar hasta el cursor* o *Ctrl+F10*. Ejecuta el código que hay entre la última línea ejecutada y la línea donde se encuentra el punto de inserción.

Para ver el valor de una variable, coloque el punto de inserción sobre ella.

Para ejecutar la aplicación en un solo paso, seleccione la orden *Iniciar sin depurar* (*Ctrl+F5*) del menú *Depurar*.

ARGUMENTOS EN LA LÍNEA DE ÓRDENES

Si ejecuta una aplicación desde el EDI que requiere argumentos en la línea de órdenes, ¿dónde se introducen estos argumentos? Haga clic sobre el nombre del proyecto con el botón derecho del ratón y seleccione la orden *Propiedades*. Después, en la ventana que se visualiza, seleccione el panel *Depurar* y escriba los argumentos según puede observar en la figura:



.NET PARA LINUX

Los desarrolladores sobre GNU/Linux pueden ahora también desarrollar aplicaciones multiplataforma basadas en .NET gracias al proyecto Mono.

¿Qué es Mono? Es un proyecto para construir una plataforma de desarrollo libre, basada en Linux, compatible con Microsoft .NET. Durante su desarrollo, ha contado con el apoyo de algunos pesos pesados de la industria informática, sobre todo del mundo Linux. Es el caso de Ximian, empresa dirigida por el mejicano Miguel de Icaza, que se mostró interesada en colaborar con el desarrollo de este entorno de programación desde el principio. Más tarde, la compañía norteamericana Novell adquirió esta empresa y se constituyó en uno de sus principales patrocinadores.

Mono, en su versión 2.8 o superior, incluye un compilador para C# y bibliotecas de ejecución (*runtimes*) para Java y C#, así como otras herramientas de ayuda al desarrollo de aplicaciones multiplataforma (*cross-platform*; para varias computadoras/sistemas operativos).

Mono actualmente soporta: .NET 4.0, excepto WPF, EntityFramework y WF, y WCF limitado, y entre sus características destacan:

- Entorno de desarrollo multiplataforma. Se ejecuta en Linux, OS X, BSD, y Microsoft Windows, incluyendo x86, x86-64, ARM, s390, PowerPC, etc.
- Multilenguaje. Se puede desarrollar en C# 4.0 (incluyendo LINQ y programación dinámica), VB 8, Java, Python, Ruby, Eiffel, F#, Oxygene, etc.
- La API es compatible con Microsoft ASP.NET, ADO.NET, Silverlight y aplicaciones Windows Forms.

Para más detalles sobre el futuro de este proyecto visite la página Web <http://www.mono-project.com>. En esta página encontrará también un enlace, *Downloads*, desde el cual podrá descargarse el software correspondiente a Mono para las distintas plataformas anteriormente comentadas, así como instrucciones para su utilización.

INSTALACIÓN DE MONO

Si instaló la distribución *openSUSE* ya tiene instalado Mono. En otro caso, descargue Mono para la distribución que usted tiene instalada de la dirección de Internet <http://www.go-mono.com/mono-downloads/> y siga las instrucciones que allí se le indican.

Es conveniente que a través de la utilidad *YaST* (grupo *Software*) añada el repositorio que permite actualizar *mono* para *openSUSE* y que está indicado en la página de descargas de este proyecto:

<http://ftp.novell.com/pub/mono/download-stable/>

EJECUTAR UN PROGRAMA C# EN LÍNEA DE ÓRDENES

Escriba el programa *HolaMundo.cs* utilizando un editor de texto sin formato:

```
class HolaMundo
{
    public static void Main(string[] args)
    {
        System.Console.WriteLine("Hola mundo!!!");
    }
}
```

Ahora compile el programa escribiendo la orden:

```
mcs HolaMundo.cs
```

El resultado del proceso de compilación es el fichero *HolaMundo.exe*. Para ejecutarlo, escriba la orden:

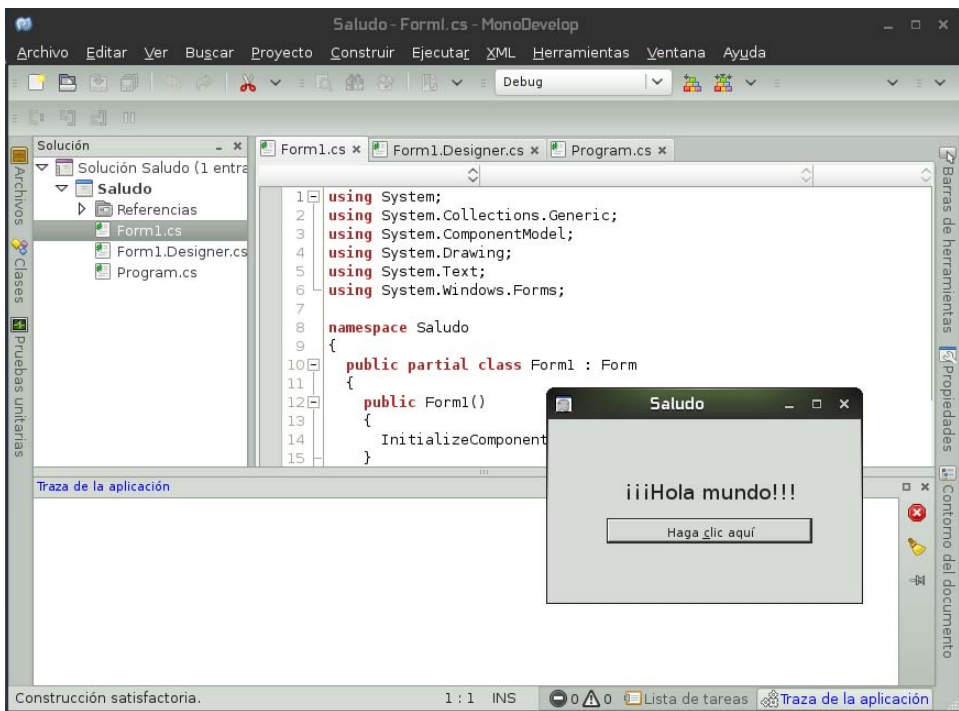
```
mono HolaMundo.exe
```

INSTALAR MONODEVELOP

MonoDevelop es un entorno de desarrollo integrado (EDI) basado en el EDI *SharpDevelop* escrito para Windows.

Para instalarlo, suponiendo que instaló la distribución *openSUSE*, inicie *YaST: Sistema* > *Instalar software*; se le requerirá privilegios de administrador (*root*). Una vez haya iniciado *YaST*, en el diálogo que se visualiza busque los paquetes relacionados con *MonoDevelop* (*Search: monodevelop*) e instálelos.

Para arrancarlo, muestre el navegador de aplicaciones y seleccione *Mono Develop Environment* del grupo *Nuevas aplicaciones/Desarrollo*:



APÉNDICE C
© F.J.Ceballos/RA-MA

CD

La forma de descargar el CD se indica en el prólogo.

ÍNDICE

- :
- ;, 387
- ?
- ?, 67
- @
- @, 66, 69, 492
- A
- Abort, 688, 701
- abstract, 435
- acceso aleatorio, 513
- acceso secuencial, 491
- Ackerman, 634
- Acos, 133
- Add, 186, 252
- ADO.NET, 778
- agrupación interna, 246
- aleatorios, números, 255
- algoritmo Boyer y Moore, 650
- algoritmos hash, 664
- ámbito de una variable, 70
- anidar if, 144
- anidar while, do, o for, 156
- añadir un elemento a una matriz, 328
- aplicación, 280
- aplicación, crear, 759
- aplicación, depurar, 775
- Append, 204
- árbol, 590
 - binario, 591
 - de búsqueda, 595
 - perfectamente equilibrado, 607
 - recorrer, 593
- archivo, 490
- argumento de un tipo primitivo, 229
- argumento por referencia, pasar, 230
- argumento por valor, pasar, 230
- argumentos, 95
 - con nombre, 240
 - en la línea de órdenes, 234, 800
 - pasar, 230
- Array, 179, 182, 249
- ArrayList, 184
- ArrayList, 253
- as, 106
- asignación, 22
 - compuesta, 358
 - de objetos, 290
- Asin, 133
- ASP.NET, 7, 780
- Atan, 133
- Atan2, 133
- atómicas, operaciones, 716
- atrapar la excepción, 471
- atributo de la clase, 99
- atributo static, 307
- atributo, iniciar, 267
- atributos, 37, 265, 267
 - con el mismo nombre, 393
 - de un fichero, 538

AutoResetEvent, 708, 737
ayuda dinámica, 761

B

BackColor, 130
barra de desplazamiento, 767
barra de menús, 766
base, 52, 391, 394, 396, 397
BaseStream, 500
Begin, 514
biblioteca de clases, 14
bibliotecas dinámicas, 90
BinaryFormatter, 535
BinaryReader, 506
BinarySearch, 251
BinaryWriter, 504
bit, 4
bloque, 19, 93
bloque de finalización, 474, 495
bloquear los controles, 771
bool, 20
Boolean, 119
borrar
 los elementos de una lista, 556
 nodo, 601
 un control, 769
 un elemento de una lista, 555
botón de opción, 766
botón de pulsación, 765
boxing, 122
boxing/unboxing, 185
Boyer y Moore, 650
break, 151, 165
BufferHeight, 131
BufferWidth, 131
burbuja, 640
buscar nodo, 599
buscar un elemento en una lista, 554
buscar un elemento en una matriz, 329
búsqueda binaria, 251, 649
búsqueda de cadenas, 650
búsqueda secuencial, 649
byte, 4, 21
Byte, 119

C

C Sharp, 3
C#, 7

cadenas de caracteres, 195
 concatenar, 67
 leer y escribir, 196
caja de herramientas, 764
caja de imagen, 766
caja de texto, 766
calendario, 767
campo, 490
Capacity, 204, 328
capturar cualquier excepción, 473
carácter nulo, 64
caracteres C#, 61
caracteres \r\n, 127
caracteres, manipular, 190
casilla de verificación, 766
cast, 72
catch, 117, 471
Ceiling, 133
ChangeExtension, 502
char, 21
Char, 119, 185
checked, 237
ciclo de vida de una página Web, 787
clase, 34, 265
 abstracta, 111, 435
 anidada, 313
 anónima, 315
 Array, 182, 249
 base, 48, 52, 381
 base directa, 409
 base indirecta, 409
 BinaryFormatter, 535
 BinaryReader, 506
 BinaryWriter, 504
 CArbolBinB, 604
 CArbolBinE, 608
 Console, 129
 Convert, 118
 crear, 36
 CVector, 452
 derivada, 48, 381, 386
 Directory, 502
 EndOfStreamException, 472
 EventWaitHandle, 737
 Exception, 116, 467, 476
 File, 502
 FileStream, 492
 genérica, 560
 genérica Leer, 485
 genérica, definición, 452

- ul style="list-style-type: none;">
- clase
 - genérica, restricciones, 486
 - IOException, 468
 - Leer, 122, 483
 - LinkedList, 568
 - lista lineal simplemente enlazada, 557
 - Math, 132
 - Object, 48, 244, 305
 - Path, 502
 - PrintDocument, 529
 - Random, 134
 - Stream, 492
 - StreamReader, 500
 - StreamWriter, 497
 - String, 37, 197
 - StringBuilder, 203
 - SystemException, 468
 - TextReader, 500
 - TextWriter, 497
 - Thread, 688
 - ValueType, 336, 362
 - WaitHandle, 736
- clases derivadas, constructor copia, 400
- clases derivadas, conversiones, 401
- clases genéricas, 451
 - herencia, 457
- clases para tipos de datos, 119
- clases y métodos finales, 436
- class, 36
- Clear, 130, 186, 250
- CListaCircularDE, 583
- CListaCircularSE, 573
- CListaLinealSE, 561
- Clonar, 422, 448
- Clone, 182, 187, 228
- Close, 495
- CLR, 7
- cola, 578
- colección de elementos, 766
- colección jerárquica, 767
- colecciones, 184, 252, 451
 - crear, 252
 - genéricas, 254
- colocar el control, 771
- Combine, 502
- comentario, 24
- Compare, 199
- CompareOrdinal, 200
- CompareTo, 199, 462, 568
- compilación, 13
- compilador, 6
- complejos, 371
- comportamiento, 38, 56
- Concat, 199
- concatenar, 67
- condiciones de carrera, 715
- Console, 11, 129
- ConsoleColor, 130
- ConsoleKey, 132
- ConsoleKeyInfo, 132
- const, 69, 70, 282, 307
- constante simbólica, 69
- constructor, 45, 96, 285
 - copia, 291, 401
 - copia virtual, 422
 - de la clase base, invocar, 397
 - de una clase derivada, 52, 397
 - estático, 308
 - llamar, 289
 - por omisión, 45, 285
 - sobrecargar, 288
 - virtual, 422
- consulta dinámica, 435
- contador, 191
- Contains, 187
- contenedor, 766
- continue, 165
- control el acceso, 387
 - a una clase, 281
- control, borrar, 769
- control, mover, 769
- controladores de espera, 736
- conversión, 72
 - de un tipo Object en un tipo valor, 122
 - de un tipo valor en un tipo referencia, 122
 - explícita, 72, 253
 - explícita en clases derivadas, 414
 - implícita, 72
 - implícita en clases derivadas, 412
- conversiones, 106
 - entre tipos de datos, 73
 - explícitas, 368
 - implícitas, 367
 - personalizadas, 365
- Convert, 118
- copia de objetos, 400
- copia profunda, 187
- copia superficial, 187
- copiar una matriz numérica, 227
- Copy, 228, 502

- CopyTo, 198
- Cos, 134
- Cosh, 134
- Count, 186, 252
- CR, 127
- creación de un programa, 9
- crear un objeto, 95
- crear una aplicación, 759
- crear una clase, 36
- crear una nueva excepción, 475
- CreateDirectory, 502
- CRLF, 127
- cuanto, 744
- Current, 514
- CursorLeft, 130
- CursorSize, 130
- CursorTop, 130
- CursorVisible, 130
- CVector, 452

D

- decimal, 22
- Decimal, 119
- declaración, 92
 - de una variable, 19
 - parcial, 311
- default, 454, 569
- definición de un método, 94
- definir una clase derivada, 390
- delegado, 317
- delegados, 321
 - genéricos, 459
- delegate, 320
- Delete, 502
- depuración, 15
- depurar, 799
 - una aplicación, 775
- desbordamiento, 237
- destructor, 293
 - de una clase derivada, 402
- Dictionary, 255
- DictionaryEntry, 192
- dimensión de una matriz, 182
- dimensiones de la consola, 131
- Directory, 502
- directriz using, 91, 335
- DiscardBufferedData, 501
- Dispose, 295, 296, 445
- dispositivos estándar, 529

- dll, 90
- do ... while, 158
- double, 21
- Double, 119

E

- E, 133
- ejecución, 13
- ejecutar, 764
- eliminar un elemento de una matriz, 328
- else, 28
- else if, 146
- empaquetado/desempaquetado, 185
- encapsulación, 99
- End, 514
- EndOfStreamException, 472, 511
- EndsWith, 201
- ensamblador, 4
- Enter, 710
- entorno de desarrollo integrado, 9, 15, 795
- enum, 64
- enumeración, 570
- Epsilon, 119
- Equals, 182, 244, 305, 361, 568
- espacio de nombres, 87, 333, 774
 - alias, 265, 281, 334, 335
 - declarar, 334
 - global, 335
 - System.IO, 491
- espera activa, 697
- espera pasiva, 697
- estados de un hilo, 686
- estados de un proceso, 680
- estructura else if, 146
- estructura interna, 267
- estructura Racional, 355
- estructuras, 63, 336
 - abstractas de datos, 548
 - dinámicas, 547
- etiqueta, 765
- eventos, 772
 - lista, 772
- EventWaitHandle, 708, 737
- excepciones, 116, 465
 - atrapar, 471
 - capturar cualquier, 473
 - crear, 475
 - lanzar, 469
 - manejar, 469

excepciones
 relanzar, 473
 utilizar, 481
 Exception, 116, 467, 476
 exclusión mutua, 715
 Exists, 502
 Exit, 341, 710
 Exp, 133
 explicit, 367
 expresión, 26
 condicional, 28
 expresiones booleanas, 73
 expresiones lambda, 321

F

false, 28
 fecha actual del sistema, 310
 fecha y hora, 278, 767
 fichero, 490
 fichero temporal, 529
 File, 502
 File.Create, 536
 File.OpenRead, 536
 FileStream, 492
 filtros, 499
 fin de fichero, 126, 511
 finalizar la ejecución, 764
 Finalize, 246, 293
 finally, 474, 495
 Find, 568
 float, 21
 Floor, 133
 flow layout, 785
 flujo, 110, 491
 de bytes, 492
 de caracteres, 497
 de datos de tipos primitivos, 504
 Flush, 500
 for, 161
 foreach, 165, 187, 217, 250, 570
 ForegroundColor, 130
 FormatException, 121
 formato, 16
 especificaciones, 115
 formulario Web, crear, 781

G

garbage collector, 297

GC, 297
 Generic, 254
 get, 39, 57, 269
 GetAttributes, 502
 GetDirectoryName, 502
 GetEnumerator, 570
 GetExtension, 502
 GetFileName, 502
 GetFileNameWithoutExtension, 502
 GetFiles, 502
 GetHashCode, 361
 GetLength, 226
 GetParent, 502
 GetPathRoot, 502
 global, 335
 goto, 166
 grid layout, 785
 grupos de hilos, 743
 guardar un objeto, 534
 guardar una aplicación, 774

H

Hanoi, 638
 HasExtension, 502
 hash, 664
 HashMap, métodos, 194
 Hashtable, 192, 254
 HasMorePages, 531
 herencia, 48, 381, 382
 múltiple, 387, 444
 simple, 48, 387
 herramientas, caja, 764
 hilo, 684
 ¿está vivo?, 702
 bloqueado, 686, 703
 ciclo de vida, 702
 dormido, 686, 704
 ejecución, 686
 en ejecución, suspender, 703
 esperando, 687, 705, 723
 muerto, 687
 preparado, 686, 703
 terminar, 698
 hilos
 ceder UCP, 744
 cooperantes, 705
 crear, 690
 detener de forma controlada, 740
 en primer plano, 701

hilos
 en segundo plano, 701
 estados, 686, 689
 grupos, 743
 independientes, 705
 parámetros, 692
 sincronizar, 706
HTTP, 783

I

IComparable, 462, 568
Icon, 772
icono de la aplicación, 771
identificadores, 68
IDictionary, interfaz, 192
IDisposable, 295, 445
IEEERemainder, 133
IEnumerable, 570
IEnumerator, 570
IEquatable, 568
if, 27, 141
if anidados, 144
IIS, 792
implementación de una clase, 274
implicit, 367
impresión, 530
In, 111
inanciación, 743, 744
IndexOf, 187
IndexOfAny, 202
IndexOutOfRangeException, 182
indizadores, 369
inorden, 593
inserción, 643
Insert, 186, 202, 204
insertar nodo, 600
insertar un elemento en una lista, 552
instancia, 35
int, 21
Int16, 119
Int32, 119
Int64, 119
interbloqueo, 743
interfaces, 437, 438
 múltiples, 445
interfaz, 36, 268, 284
 definición, 437
 gráfica, 758
 IComparable, 462, 568

 IDisposable, 295, 445
 IEquatable, 568
 para qué sirve, 445
 pública, 99
 tipo de datos, 442
 utilizar, 439
 vs. clase abstracta, 442
Interlocked, 708
Intern, 249
internal, 89, 387
intérprete, 6
invocar a un método redefinido, 413
IOException, 468
is, 414
IsAlive, 689, 702
IsBackground, 689, 701
ISerializable, 534
IsInfinity, 120
IsInterned, 248
IsNaN, 120, 174, 218
IsPathRoot, 502
Item, 252
iterador, 187, 570

J

jerarquía de clases, 381, 387, 403
JIT, 7
Join, 688

K

Key, 132
KeyAvailable, 131
KeyChar, 132
KeyValuePair, 255

L

L, 65
lanzar una excepción, 469
LargestWindowHeight, 131
LargestWindowWidth, 131
LastIndexOf, 187
LastIndexOfAny, 202
LayoutMode, 771
leer, 120
Leer, clase, 483
Length, 39, 182, 200, 204, 514
lenguaje máquina, 5

lenguajes de alto nivel, 5
 LF, 127
 library, 90
 línea, 112
 líneas de ayuda, 771
 LinkedList, 255, 568
 LinkedListNode, 255
 Linux, 803
 List, 255
 lista, 766
 circular, 571
 circular doblemente enlazada, 583
 de los eventos, 772
 desplegable, 766
 doblemente enlazada, 582
 lineal simplemente enlazada, 548
 lineal, recorrer, 556
 listas lineales, 548
 literal, 64
 de cadena de caracteres, 66
 de cadena plano (@), 66
 de un solo carácter, 66
 entero, 64
 nulo, 67
 real, 65
 llamar a un constructor, 289
 lock, 709, 715
 Log, 133
 Log10, 133
 long, 21
 longitud de una matriz, 182

M

Main, 11, 43, 95, 101
 argumentos, 234
 ManualResetEvent, 708, 739
 mapa ordenado, 193
 máquina virtual, 6
 marco, 766
 Math, 132
 matrices, 178
 de objetos, 324
 dentadas, 207
 métodos, 182
 matriz
 acceder a un elemento, 181
 asociativa, 190, 193
 buscar un valor, 251
 como argumento, 226

 como valor retornado, 227
 crear, 180
 de cadenas de caracteres, 213
 de matrices, 207, 228
 de objetos, 214
 declarar, 179
 dentada, 208
 es asociativa, 191
 es un objeto, 180
 invertir sus elementos, 251
 multidimensional, 206
 numérica multidimensional, 206
 numérica unidimensional, 179
 ordenar, 250
 poner a cero todos sus elementos, 250
 Max, 133
 máximo común divisor, 357
 MaxValue, 119
 memoria para objetos String, 247
 memoria, asignar y liberar, 96
 mensajes, 35, 97
 menú genérico, construir, 259
 Message, 471
 método, 35, 39, 94
 abreviado, 278
 abstracto, 435
 anónimo, 320
 consulta dinámica, 435
 de inserción, 643
 de la burbuja, 640
 de la clase, 99
 de quicksort, 645
 recursivo, 236
 Seek, 514
 Sleep, 697
 sobrecargado, 47, 277
 static, 310
 virtual, 415
 virtual, llamada, 421
 Write, 57
 métodos, 30, 38, 265, 268
 de una clase derivada, 393
 en línea, 434
 extensores, 315
 genéricos, 458
 ocultos, 391
 parametrizados, 458
 por omisión, 391
 sincronizados, 723
 sobrecargados, 239

- mezcla natural, 654
- miembro privado, 37
- miembro público, 39
- miembros del objeto, 99
- miembros heredados, 52, 388
- miembros que son referencias, 297
- Min, 133
- MinValue, 119
- modificador, 94
- modificadores de acceso, 271
- modificadores de tamaño, 769
- Modifiers, 132
- Monitor, 707, 710, 717
- monitor reentrante, 718
- Mono, 803
- Move, 502
- mover el control, 769
- MSIL, 7
- MulticastDelegate, 324
- multidifusión, 324
- Mutex, 707, 726

N

- naked, 487
- Name, 689
- NaN, 119, 218
- NegativeInfinity, 119
- new, 95, 101, 392, 396, 416
- Next, 134
- NextDouble, 134
- nivel de protección predeterminado, 99
- nodo de un árbol, 592
- null, 64, 67, 126, 215, 550, 569
- número primo, 670
- números al azar, 134
- números aleatorios, 255

O

- object, 182
- Object, 48, 244, 305
- object como tipo retornado, 569
- objeto, 34
 - aplicación, 83
 - String, crear, 246
- objetos de sincronización, 707
- ocultación de datos, 271
- operador
 - , 364

- ::, 335
- [], 186, 202, 205, 369
- +, 354, 359
- ==, 361
- == en String, 245
- as, 106
- condicional, 78
- de indexación, 369
- is, 414
- new, 95, 101
- sobrecargado, 349
- ternario, 78
- operadores, 73
 - a nivel de bits, 74
 - aritméticos, 26
 - de asignación, 75
 - de comparación, 28
 - de conversión, 366
 - de relación, 28
 - lógicos, 73
 - prioridad, 78
 - unitarios, 74
- operator, 349, 366
- ordenación, 640
 - fichero, 654
 - fichero utilizando acceso aleatorio, 661
- out, 233
- OutOfMemoryException, 96
- override, 416

P

- Page, 788
- palabras clave, 68
- panel, 766
- paquetes, 87
- ParameterizedThreadStart, 690
- parámetros, 95
 - con valor por omisión, 241
 - de salida, 233
 - de tipo, 455
 - número variable de, 242, 279
 - pasados por referencia, 105, 225
 - pasados por valor, 105, 225
- Parse, 119, 120
- partial, 312
- pasar argumentos, 105
 - por referencia, 230
 - por valor, 230
- pasar un tipo referencia por referencia, 231

path, 12
 Path, 502
 Peek, 501
 PI, 133
 pila, 577
 planificación, 743
 planificador, 680, 687
 polimorfismo, 424
 POO, 33
 posición del cursor, 129
 Position, 513
 PositivelyInfinity, 119
 postorden, 593
 Pow, 133
 Predicate, 461
 preorden, 593
 Print, 530
 PrintDocument, 529
 PrintPage, 530
 prioridad de los hilos, 745
 prioridad de los operadores, 78
 Priority, 689, 745
 private, 37, 273, 387
 proceso, 679
 ligero, 684
 productor-consumidor, 730, 749
 programa, 4, 680
 programación genérica, 451
 programación orientada a objetos, 33
 propiedad auto-implementada, 270
 propiedad Icon, 772
 propiedad In, 111
 propiedad Length, 39, 514
 propiedad Position, 513
 propiedades, 38, 39, 265, 769
 protección de una clase, 89
 protección, niveles de, 90
 protected, 273, 387
 public, 39, 89, 90, 272, 387
 Pulse/PulseAll, 710, 718, 723
 puntero, 765

Q

Queue, 253, 255
 quicksort, 645

R

raíz cuadrada, 27

raíz de un árbol, 592
 random, 255
 Random, 134
 con argumento, 755
 Rank, 182
 Read, 112, 128, 501
 ReaderWriterLock, 708
 ReadKey, 131
 ReadLine, 112, 126, 215, 501
 readonly, 282
 ReadString, 521
 recolector de basura, 96, 105, 297
 recorrer un árbol, 593
 recursión, 633
 recursividad, 236
 redefinir miembros de la clase base, 395
 redefinir un método, 50
 virtual, 416
 ref, 230
 ReferenceEquals, 198, 245
 referencia, 63, 101, 102
 nula, 64
 referencias, 198, 762
 a clases derivadas, 411
 a datos, 101
 y objetos String, 246
 registro, 490
 rejilla de ayuda, 771
 Remove, 186, 205, 252
 RemoveAt, 186
 Replace, 205
 representación interna, 284
 reservar de memoria, 327
 Reset, 737
 restricción de tipo naked, 487
 restricciones where, 486
 Resume, 689
 return, 94
 Reverse, 251
 round-robin, 744
 Running, 689

S

salida estándar, 25
 sbyte, 20
 SByte, 119
 SDK, 9
 sealed, 436
 sección crítica, 709, 710, 715

- secuencia de escape, 62
 - Seek, 514
 - SeekOrigin, 514
 - seleccionar un objeto, 770
 - semáforos, 729
 - Semaphore, 708
 - sentencia
 - break, 165
 - compuesta, 93
 - continue, 165
 - de asignación, 22
 - do ... while, 158
 - for, 161
 - foreach, 217
 - goto, 166
 - if, 27, 141
 - return, 94
 - simple, 93
 - switch, 149
 - while, 153
 - seriación, 533
 - Serializable, 534
 - Serialization, 535
 - serializar/deserializar un objeto, 535
 - set, 39, 57
 - Set, 737
 - SetAttributes, 502
 - SetCursorPosition, 130
 - SetIn, 111
 - SetOut, 111
 - short, 21
 - Sin, 133
 - sincronización, 706
 - de hilos, 706
 - secciones críticas, 710
 - semáforos, 729
 - Sing, 134
 - Single, 119
 - Sinh, 134
 - sitio Web, 781
 - Sleep, 688, 697, 723
 - Sleep(0), 703
 - sobrecarga
 - de los operadores ++ y --, 363
 - de los operadores de asignación, 358
 - de los operadores unarios/binarios, 364
 - de métodos, 277
 - de métodos, 239
 - de operadores binarios, 355
 - de un operador unario, 363
 - de un operador, propiedades, 352
 - de un operador, restricciones, 355
 - del operador +, 359
 - del operador ==, 361
 - sólo lectura, 282
 - Sort, 250
 - SortedDictionary, 255
 - SortedList, 193, 254, 255
 - Split, 202
 - Sqrt, 27, 133
 - Stack, 253, 255
 - StackOverflowException, 635
 - Start, 688
 - StartsWith, 201
 - starvation, 743, 744
 - static, 70, 99, 307
 - constructor, 308
 - stream, 491
 - Stream, 492
 - StreamReader, 500
 - StreamWriter, 497
 - string, 22, 195
 - String, 22, 37, 197
 - comparación, 245
 - constructor, 197
 - StringBuilder, 203
 - struct, 336
 - subclase, 48, 381
 - subproceso, 684
 - Substring, 201
 - superclase, 48, 381
 - Suspend, 689
 - switch, 149
 - Synchronized, 723
 - System, 11, 109
 - System.Array, 179
 - System.Boolean, 63
 - System.Collections.Generic, 255
 - System.Double, 63
 - System.Int32, 63
 - System.IO, 491
 - System.String, 22, 64
 - System.Threading, 688
 - SystemException, 468
- T**
- tabla, 766
 - tamaño de los controles, 769
 - Tan, 134

Tanh, 134
 tecla pulsada, 131
 temporizador, 767
 TextReader, 500
 TextWriter, 497
 this, 54, 282, 396
 thread, 684
 Thread, 688
 Thread.Sleep(0), 744
 ThreadPool, 743
 ThreadPriority, 688, 745
 ThreadStart, 688, 690
 ThreadState, 689
 throw, 469
 tick, 682
 tiempo de ejecución, 676
 Timer, 708
 time-slice, 744
 tipo
 abstracto de datos, 352
 bool, 20
 byte, 21
 char, 21
 de un objeto, 414
 decimal, 22
 double, 21
 float, 21
 int, 21
 long, 21
 polimórfico, 424
 referencia, 63
 sbyte, 20
 short, 21
 string, 22
 uint, 21
 ulong, 21
 ushort, 21
 valor, 63
 tipos de datos, 20
 tipos enumerados, 64
 tipos genéricos, 255
 tipos parametrizados, 451
 tipos referencia y valor, 569
 Title, 130
 ToDouble, 119
 ToInt32, 118
 ToLower, 200
 torres de Hanoi, 638
 ToSingle, 119
 ToString, 119, 120, 198, 205, 245

ToUpper, 201
 Trim, 201
 true, 28
 try, 117, 471
 try ... catch, 167
 TryEnter, 710

U

U, 65
 uint, 21
 UInt16, 119
 UInt32, 119
 UInt64, 119
 UL, 65
 ulong, 21
 unboxing, 122
 unchecked, 237
 UNICODE, 21
 ushort, 21
 using, 91
 sentencia, 295
 UTF-8, 497

V

valor, 63
 valor predeterminado, 454
 value, 57
 ValueType, 362
 variable, 19, 70
 iniciar, 71
 local, 20, 71
 miembro de una clase, 71
 virtual, 415
 Visual Studio, 758
 void, 39, 94

W

Wait, 710, 718, 723
 WaitAll, 737
 WaitAny, 737
 WaitHandle, 708, 736
 WaitOne, 737
 WaitSleepJoin, 689
 where, 486
 while, 153
 while, do, o for anidados, 156
 WindowHeight, 131

WindowWidth, 131

Write, 57, 113, 494, 500, 521

WriteLine, 11, 25, 113, 500

Y

yield break, 571

yield return, 571

Del mismo autor

- Curso de programación con **PASCAL** ISBN: 978-84-86381-36-3
224 págs.
 - Curso de programación **GW BASIC/BASICA** ISBN: 978-84-86381-87-5
320 págs.
 - Manual para **TURBO BASIC** ISBN: 978-84-86381-43-1
Guía del programador 444 págs.
 - Manual para **Quick C 2** ISBN: 978-84-86381-65-3
Guía del programador 540 págs.
 - Manual para **Quick BASIC 4.5** ISBN: 978-84-86381-74-5
Guía del programador 496 págs.
 - Curso de programación **Microsoft COBOL** ISBN: 978-84-7897-001-8
480 págs.
 - Enciclopedia del lenguaje **C** ISBN: 978-84-7897-053-7
888 págs.
 - Curso de programación **QBASIC y MS-DOS 5** ISBN: 978-84-7897-059-9
384 págs.
 - Curso de programación **RM/COBOL-85** ISBN: 978-84-7897-070-4
396 págs.
 - El abecé de **MS-DOS 6** ISBN: 978-84-7897-114-5
224 págs.
 - Microsoft **Visual C ++** (ver. 1.5x de 16 bits) ISBN: 978-84-7897-180-0
Aplicaciones para Windows 846 págs. + 2 disquetes
 - Microsoft **Visual C ++** ISBN: 978-84-7897-561-7
Aplicaciones para Win32 (2ª edición) 792 págs. + disquete
 - Microsoft **Visual C ++** ISBN: 978-84-7897-344-6
Programación avanzada en Win32 888 págs. + CD-ROM
 - **Visual Basic 6** ISBN: 978-84-7897-357-6
Curso de programación (2ª edición) 528 págs. + disquete
 - Enciclopedia de Microsoft **Visual Basic 6** ISBN: 978-84-7897-386-6
1072 págs. + CD-ROM
 - El lenguaje de programación **Java** ISBN: 978-84-7897-485-6
320 págs. + CD-ROM
 - El lenguaje de programación **C#** ISBN: 978-84-7897-500-6
320 págs. + CD-ROM
-

Del mismo autor

- El lenguaje de programación **Visual Basic.NET** ISBN: 978-84-7897-525-9
464 págs. + CD-ROM
 - **Microsoft C#** ISBN: 978-84-7897-737-6
Curso de programación 866 págs. + CD-ROM
 - **Java 2** ISBN: 978-84-7897-745-1
Lenguaje y aplicaciones 392 págs. + CD-ROM
 - Programación orientada a objetos con **C ++** (4ª edición) ISBN: 978-84-7897-761-1
648 págs. + CD-ROM
 - **C/C++** ISBN: 978-84-7897-762-8
Curso de programación (3ª edición) 708 págs. + CD-ROM
 - **Microsoft Visual Basic .NET** ISBN: 978-84-7897-812-0
Curso de programación 832 págs. + CD-ROM
 - **Microsoft C#** ISBN: 978-84-7897-813-7
Lenguaje y aplicaciones (2ª edición) 520 págs. + CD-ROM
 - **Java 2.** Interfaces gráficas y aplicaciones para Internet (3ª edición) ISBN: 978-84-7897-859-5
718 págs. + CD-ROM
 - **Aplicaciones .Net multiplataforma** (Proyecto Mono) ISBN: 978-84-7897-880-9
212 págs. + CD-ROM
 - Enciclopedia del lenguaje **C ++** (2ª edición) ISBN: 978-84-7897-915-8
902 págs. + CD-ROM
 - Enciclopedia de Microsoft **Visual C#** (3ª edición) ISBN: 978-84-7897-986-8
1110 págs. + CD-ROM
 - Enciclopedia de Microsoft **Visual Basic** (2ª edición) ISBN: 978-84-7897-987-5
1090 págs. + CD-ROM
 - **Microsoft Visual Basic .NET** ISBN: 978-84-9964-020-4
Lenguaje y aplicaciones (3ª edición) 520 págs. + CD-ROM
 - **Java 2.** ISBN: 978-84-9964-032-7
Curso de programación (4ª edición) 820 págs. + CD-ROM
-

INSTALACIÓN

Para instalar el kit de desarrollo de C# descargue la plataforma de desarrollo de la dirección:

<http://www.microsoft.com/downloads/en/default.aspx>.

PLATAFORMA WINDOWS

Instalación de .NET Framework SDK

Hay que instalar .NET Framework Redistributable Package antes de instalar .NET Framework SDK. El primer paquete incluye todo lo necesario para ejecutar aplicaciones desarrolladas con .NET Framework. El segundo paquete incorpora todo lo necesario para escribir, construir, verificar y desplegar aplicaciones desarrolladas con .NET Framework.

Para realizar la instalación, siga las instrucciones mostradas por el asistente de instalación. Esta instalación se realiza en la carpeta *Microsoft.NET* de Windows y en la carpeta *Microsoft.NET* de archivos de programas.

Instalación de Microsoft Visual Studio

La instalación de Visual Studio (el paquete completo o las versiones *Express* que necesite) no requiere la instalación previa del SDK porque está incluido en éste. Descargue de Internet los paquetes Visual C# Express y Visual Web Developer Express (ambos incluyen SQL Server Express y la ayuda en línea), o bien, si tiene acceso, Visual Studio, e instáuelos.

Ejemplos del libro

La forma de descargar el CD se indica en el prólogo.

PLATAFORMA LINUX

Véase el apéndice B.

LICENCIA

Todo el contenido del CD, excepto los ejemplos del libro, es propiedad de las firmas que los representan (*Microsoft*, etc.). La inclusión en este libro se debe a su gentileza y es totalmente gratuita y con la finalidad de apoyar el aprendizaje del software correspondiente. Para obtener más información y actualizaciones visite las direcciones indicadas en dicho software:

<http://www.microsoft.com/es/es/default.aspx>

Al realizar el proceso de instalación, haga el favor de consultar el acuerdo de licencia para cada uno de los productos.

WEB DEL AUTOR: <http://www.fjceballos.es>

En esta Web podrá echar una ojeada a mis publicaciones más recientes y acceder a la descarga del software necesario para el estudio de esta obra así como a otros recursos.

Microsoft C#TM

Curso de Programación

2ª EDICIÓN

C#, pronunciado *C Sharp*, es actualmente uno de los lenguajes de programación más populares en informática y comunicaciones. El objetivo de Microsoft, que tiene todos los derechos sobre la plataforma de desarrollo *.NET Framework* en la que está incluido el lenguaje C#, es permitir a los programadores abordar el desarrollo de aplicaciones complejas con facilidad y rapidez. Es un poco como tomar todas las cosas buenas de Visual Basic y añadirlas a C++, aunque recortando algunas de las tradiciones más ocultas y difíciles de conocer de C y C++.

C# es un lenguaje de programación orientado a objetos cuyas características lo sitúan, junto con Java, en el producto ideal para desarrollar programas para la Web. Pero con C#, no sólo se pueden escribir programas para la Web; también permite desarrollar aplicaciones orientadas a objetos de uso general.

Por lo tanto, C# le permitirá crear programas para su uso personal, para su grupo de trabajo, para una empresa, aplicaciones distribuidas a través de Internet, aplicaciones de bases de datos, páginas Web y otras muchas cosas.

Microsoft C# - Curso de programación es un libro:

- Compatible con C# 4.0.
- Con ejemplos claros y sencillos, fáciles de entender, que ilustran los fundamentos de la programación C#.
- Que le permitirá aprender programación orientada a objetos.
- Que le enseñará a trabajar con estructuras dinámicas de datos, con ficheros, con excepciones y con hilos.
- Con el que aprenderá a desarrollar aplicaciones.
- Y que le introducirá en el diseño de interfaces gráficas y en el desarrollo de aplicaciones para Internet.
- Todo lo expuesto tiene continuación en mi otro libro **Enciclopedia de Microsoft Visual C#**.

WWW



Podrá descargarse de **www.ra-ma.es**, en la página Web correspondiente al libro, un CD-ROM con los ejemplos realizados, con los apéndices, así como el software necesario para que el lector pueda reproducirlos durante el estudio.



ra-ma.es



Ra-Ma[®]